

Git & GitHub Mastery

From Zero → Internals → Production → Senior Engineer

Volume 2 of 4: Integration, undo, remotes, recovery and advanced Git

Baseline: Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Every Git transcript in this book is real output from the lab scripts under `labs/`, reproducible with `labs/verify-all.sh`. GitHub-side behavior is described from GitHub's documentation and is marked as such.

Contents of this volume

- Chapter 8: Merge
- Chapter 9: Rebase
- Chapter 10: Cherry-pick
- Chapter 11: Reset, Revert, Restore
- Chapter 12: Remote Operations
- Chapter 13: Recovery
- Chapter 14A: History investigation
- Chapter 14B: Configuration, Aliases, Tags and Signing
- Chapter 14C: Stash Internals, Rerere, Attributes, Hooks
- Chapter 14D: The Frontier

Chapter 8: Merge

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch08/`.

8.1 Why this matters

Three questions a CTO can ask after an ordinary release week:

1. "Both pull requests were green. `main` is red. The two branches did not touch a single file in common. How?"
2. "Ravi's retry setting disappeared from the config in a merge commit. `git log -p` shows no diff for that commit. Who removed the line, and how do we prove it?"
3. "We reverted the bad merge on Friday. On Monday we merged the repaired branch, the merge was clean, and half of the feature is missing. Where did it go?"

The answers are in sections 8.15, 8.16 and 8.18. None needs a bug in Git. Each follows from what a merge is.

The stakes are measurable. In a 2024 poll of working developers, 61% of roughly 1,480 respondents had seen a production bug caused by a bad conflict resolution at least once, and 48% of 1,511 did not know that "ours" and "theirs" swap between a merge and a rebase ([poll results](#); a self-selected sample, so read it as direction, not as a population estimate).

Hold on to one idea through the chapter. A merge is computed from exactly three snapshots: your commit, their commit, and one common ancestor called the merge base. Git's FAQ states it plainly: "Git does not consider the history or the individual commits that have happened on those branches at all" ([gitfaq](#)). Git compares lines of text in those three snapshots. It does not know what the lines mean. Every surprise in this chapter is one of those two facts showing through.

The sample project is `evalkit`, a small evaluation harness for LLM outputs: `config/eval.yaml` holds the run configuration, `prompts/judge.txt` the prompt for the judge model, and `evalkit/metrics.py` the scoring functions. You work on `main`. Asha and Ravi work on branches.

8.2 Divergence and the merge base

In one sentence. The merge base of two commits is their best common ancestor: the most recent commit that both histories contain, and the third input of every merge.

Analogy. Two editors photocopy a manuscript on Monday and mark up their copies separately. On Friday you can combine their work only if you still have the Monday copy: without it you cannot tell a sentence that one editor added from a sentence that the other deleted. The merge base is the Monday copy. The analogy breaks in two places. Git finds the Monday copy by walking parent links, not by date. And a tangled history can have two equally good Monday copies (section 8.5).

Precisely. A common ancestor of commits A and B is a commit reachable from both by following parents (Chapter 7 introduced ancestry and divergence). The manual defines the rest: "One common ancestor is better than another common ancestor if the latter is an ancestor of the former. A common ancestor that does not have any better common ancestor is a best common ancestor, i.e. a merge base" ([git-merge-base](#)). Two branches have diverged when each contains commits that the other lacks. Four commands answer every question about this:

Question	Command
Which commit is the merge base?	<code>git merge-base A B</code> (add <code>--all</code> to print every merge base)
Is A an ancestor of B?	<code>git merge-base --is-ancestor A B</code> (exit status 0 means yes, 1 means no)
How many commits does each side have that the other lacks?	<code>git rev-list --left-right --count A...B</code>
Which commits are they?	<code>git log --oneline --left-right A...B</code>

Inside `.git`. Nothing is stored. The merge base is computed from the `parent` lines of commit objects every time a command needs it. No file says where a branch "came from".

See it. `main` has one commit of its own, `feature/rationale` has two:

```
$ git log --oneline --graph --all
* c089834 Use temperature 0 for reproducible evals
| * 58a5e60 Ask the judge to quote evidence
| * 5aec6e0 Ask the judge for a rationale
|/
* 6ae3c51 Add eval config, judge prompt and metrics
$ git merge-base main feature/rationale
6ae3c51e176b786603aea088486677fe30f409bd
$ git rev-list --left-right --count main...feature/rationale
1      2
$ git log --oneline --left-right main...feature/rationale
```

```
< c089834 Use temperature 0 for reproducible evals
> 58a5e60 Ask the judge to quote evidence
> 5aec6e0 Ask the judge for a rationale
```

The two counts read "one commit only on the left side, two only on the right". In the `--left-right` log, `<` marks commits reachable only from `main` and `>` commits reachable only from the branch.

Three dots mean something different to `git diff`, and the difference matters for reviews:

```
# Three dots in git diff: from the merge base to the right-hand side.
$ git diff --stat main...feature/rationale
prompts/judge.txt | 2 ++
1 file changed, 2 insertions(+)
# Two dots (or a space): the two tips compared directly.
$ git diff --stat main..feature/rationale
config/eval.yaml | 2 +-
prompts/judge.txt | 2 ++
2 files changed, 3 insertions(+), 1 deletion(-)
```

With three dots, `git diff` compares the merge base with the branch tip: what the branch did, and nothing that `main` did meanwhile. With two dots it compares the two tips, so the temperature change that `main` made appears as if the branch had undone it.

Picture.

```
      5aec6e0---58a5e60  feature/rationale
      /
6ae3c51---c089834      main  (HEAD -> main)
      ^
      merge base of main and feature/rationale
```

In production. The three-dot diff is what a pull request page shows, which is why a pull request does not list changes that landed on the base branch after you forked (Chapter 17). The age of the merge base also predicts merge pain. A branch that forked three weeks ago is combined against a three-week-old snapshot, and everything both sides did since then is in play. For a branch that must live long, merging `main` into it at intervals moves the merge base forward and keeps each merge small.

8.3 Fast-forward: a merge that creates nothing

In one sentence. When your current commit is an ancestor of the commit you merge, there is nothing to combine, so Git moves your branch ref to that commit and creates no new object.

Analogy. A bookmark in a manuscript that somebody else kept writing. You move the bookmark to the last page. The analogy breaks on one point: nothing in the book records that the bookmark moved, or that the new pages were ever a separate branch. Only your local reflog knows.

Precisely. The condition is `git merge-base --is-ancestor HEAD <other>`. Put differently, the merge base is your own tip. Fast-forwarding is the default (`--ff`). The opposite case has its own message: if `<other>` is already an ancestor of `HEAD`, the merge prints "Already up to date." and moves nothing.

Inside .git. The file `refs/heads/main` is rewritten with the new commit ID. `ORIG_HEAD` receives the old one. The reflogs of `HEAD` and of `main` each gain one line. The index and the working tree are updated as by a branch switch. The object database does not change: count the commits before and after. One detail found while testing: every `git merge` call rewrites `ORIG_HEAD` with the tip it started from, even a call that prints "Already up to date." or refuses. As an undo pointer it is valid only until the next attempt; the reflog is the durable record.

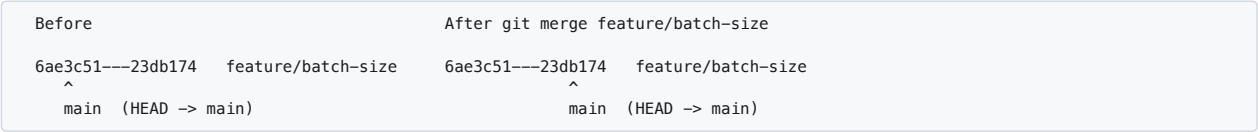
See it.

```
$ git log --oneline --graph --all
* 23db174 Raise batch size to 32
* 6ae3c51 Add eval config, judge prompt and metrics
$ git merge-base --is-ancestor main feature/batch-size
[exit status: 0]
$ git rev-list --count --all
2
```

```
$ git merge feature/batch-size
Updating 6ae3c51..23db174
Fast-forward
 config/eval.yaml | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline --graph --all
* 23db174 Raise batch size to 32
* 6ae3c51 Add eval config, judge prompt and metrics
$ git rev-list --count --all
2
$ git reflog -2
23db174 HEAD@{0}: merge feature/batch-size: Fast-forward
6ae3c51 HEAD@{1}: checkout: moving from feature/batch-size to main
$ cat .git/ORIG_HEAD
6ae3c51e176b786603aea088486677fe30f409bd
```

"Updating 6ae3c51..23db174" names the old and the new position of `main`. There are still two commits. The relog entry says `merge feature/batch-size: Fast-forward`, and `ORIG_HEAD` holds 6ae3c51, the commit to return to if this was a mistake.

Picture.



Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
git merge <other>, fast-forward 🟡	files updated to the tree of <other>	rewritten to match <other>	still ref: refs/heads/main	moves to the tip of <other>	ORIG_HEAD set to the old tip; reflogs of HEAD and the branch gain an entry; no new objects	unchanged	unchanged
git merge <other>, "Already up to date." 🟢	unchanged	unchanged	unchanged	unchanged	ORIG_HEAD rewritten with the current tip; nothing else	unchanged	unchanged

In production. A fast-forward is what `git pull` does on a branch where you have no commits of your own (Chapter 12). The risk is the missing trace. If somebody fast-forwards an unfinished branch into `main`, the history looks as if the work-in-progress commits had been made on `main` directly, and no merge commit exists to revert. Lab 6.1 reproduces this and recovers with `ORIG_HEAD`. Section 8.12 shows how to forbid it.

8.4 The true merge: three inputs, one rule table

In one sentence. When both sides have commits that the other lacks, Git combines the two tips against their merge base, path by path and then line by line, and records the result as a commit with two parents.

Analogy. The editor-in-chief has three copies on the desk: Monday's, yours and theirs. If only one of you changed a page, take that version. If both made the same change, take it once. If both changed it differently, open the page and compare paragraphs. The analogy breaks where it matters most: the editor-in-chief reads for meaning, Git compares text.

Precisely. Call the three trees base, ours (the commit HEAD points to) and theirs (the commit named on the command line). For every path Git compares three blob IDs. X, Y and Z stand for different contents:

Base	Ours	Theirs	Result	Path in the transcript below
X	X	X	X: nobody changed it	evalkit/metrics.py
X	Y	X	Y: only ours changed it	README.md
X	X	Y	Y: only theirs changed it	requirements.txt
X	Y	Y	Y: both made the same change	prompts/judge.txt
X	Y	Z	merge the content of the three files (section 8.7)	config/eval.yaml
absent	absent	Y	Y: added by theirs	evalkit/rouge.py
X	X	absent	absent: deleted by theirs, untouched by ours	scripts/legacy_eval.sh
X	Y	absent	conflict: modify/delete (section 8.11)	
absent	Y	Z	conflict: add/add (section 8.11)	

The table is not an interpretation. The manual of `git read-tree`, Git's original three-way merge plumbing, states the first rows as "stage 2 and 3 are the same; take one or the other", "stage 1 and stage 2 are the same and stage 3 is different; take stage 3", and "stage 1 and stage 3 are the same and stage 2 is different take stage 2" ([git-read-tree](#)). Stages 1, 2 and 3 are base, ours and theirs (section 8.8).

Two consequences follow. First, most paths are settled without reading a file: equal content has an equal blob ID (Chapter 2), so comparing three IDs is enough. Second, the rows mention three snapshots and nothing else. The commits between the base and each tip do not take part. A change that both sides made is taken once and silently. A change that one side made and later undid counts as "that side did nothing", which section 8.15 turns into a production incident.

Inside .git. A clean true merge writes a blob for every file merged by content, the trees above those blobs, and one commit object with two parent lines. Then it does what any commit does: the current branch ref moves to the new commit and the reflogs gain a line. `ORIG_HEAD` holds the previous tip.

See it. After the fast-forward of section 8.3, `main` and a new branch each gain one commit:

```
$ git log --oneline --graph --all
* 017bef5 Use temperature 0 for reproducible evals
| * 67feed7 Raise timeout to 60 seconds
|/
* 23db174 Raise batch size to 32
```



```
* 6ae3c51 Add eval config, judge prompt and metrics
$ git merge-base --is-ancestor main feature/timeout
[exit status: 1]
$ git merge-base main feature/timeout
23db1749f261b3f9ca7627a25923389c32a4a64d
$ git rev-list --left-right --count main...feature/timeout
1      1
```

Neither tip is an ancestor of the other, so a fast-forward is impossible. The merge base is `23db174`.

```
$ git merge feature/timeout
Auto-merging config/eval.yaml
Merge made by the 'ort' strategy.
  config/eval.yaml | 2 +-
  1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline --graph --all
* 6b10077 Merge branch 'feature/timeout'
|\
| * 67feed7 Raise timeout to 60 seconds
* | 017bef5 Use temperature 0 for reproducible evals
|/
* 23db174 Raise batch size to 32
* 6ae3c51 Add eval config, judge prompt and metrics
$ git rev-list --count --all
5
```

"Auto-merging config/eval.yaml" reports a content merge: both sides changed that file, differently. "Merge made by the 'ort' strategy." names the algorithm (section 8.6). The repository went from four commits to five. The new one has two parents:

```
$ git cat-file -p HEAD
tree 25ed240b50b6ea31af8a6023d66eac8ce10d5364
parent 017bef5a99c9b0472c2c1fd4181b9d01c243e76c
parent 67feed7ad67adc99e081b8c29515db39c4726dbb
author Lab User <you@example.com> 1788756960 +0530
committer Lab User <you@example.com> 1788756960 +0530

Merge branch 'feature/timeout'
$ git reflog -1
6b10077 HEAD@{0}: merge feature/timeout: Merge made by the 'ort' strategy.
$ cat config/eval.yaml
model: judge-large-v2
temperature: 0.0
max_tokens: 512
batch_size: 32
timeout_s: 60
retries: 2
seed: 7
```

The first `parent` is where you were (`017bef5`), the second is what you merged (`67feed7`). The merged file carries `temperature: 0.0` from your side and `timeout_s: 60` from theirs. Merging either branch again finds nothing to do:

```
$ git merge feature/timeout
Already up to date.
$ git merge feature/batch-size
Already up to date.
```

Now the rule table on a larger example. Compare the abbreviated blob IDs of the three listings path by path before you look at the result:

```
$ git merge-base main feature/rouge
e650b75915a1645cdb222c495fa22e6273eab187
# base (the merge base, tagged "base" by the demo):
$ git ls-tree -r --abbrev=7 base
100644 blob af7f9b5    README.md
100644 blob c5b3327    config/eval.yaml
100644 blob 9bbcd7a    evalkit/metrics.py
100644 blob cfe0ef0    prompts/judge.txt
100644 blob 3aecde9    requirements.txt
100644 blob 4a41a25    scripts/legacy_eval.sh
# ours (main):
$ git ls-tree -r --abbrev=7 main
100644 blob aa477a1    README.md
100644 blob 0e8b97c    config/eval.yaml
100644 blob 9bbcd7a    evalkit/metrics.py
100644 blob 38cb437    prompts/judge.txt
100644 blob 3aecde9    requirements.txt
100644 blob 4a41a25    scripts/legacy_eval.sh
# theirs (feature/rouge):
$ git ls-tree -r --abbrev=7 feature/rouge
100644 blob af7f9b5    README.md
100644 blob caf8b0d    config/eval.yaml
100644 blob 9bbcd7a    evalkit/metrics.py
100644 blob fe0b96e    evalkit/rouge.py
100644 blob 38cb437    prompts/judge.txt
100644 blob 4a7b8a2    requirements.txt
```

```
$ git merge feature/rouge
Auto-merging config/eval.yaml
Merge made by the 'ort' strategy.
 config/eval.yaml      | 2 +-
 evalkit/rouge.py      | 2 ++
 requirements.txt      | 1 +
 scripts/legacy_eval.sh | 2 --
4 files changed, 4 insertions(+), 3 deletions(-)
create mode 100644 evalkit/rouge.py
delete mode 100644 scripts/legacy_eval.sh
```

```
# result (the tree of the merge commit):
$ git ls-tree -r --abbrev=7 HEAD
100644 blob aa477a1    README.md
100644 blob cef358f    config/eval.yaml
100644 blob 9bbcd7a    evalkit/metrics.py
100644 blob fe0b96e    evalkit/rouge.py
100644 blob 38cb437    prompts/judge.txt
100644 blob 4a7b8a2    requirements.txt
```

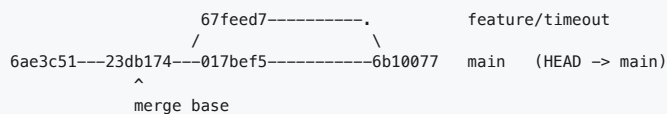
README.md is af7f9b5 in base and theirs and aa477a1 in ours: the result is aa477a1. requirements.txt changed only in theirs: the result is 4a7b8a2. prompts/judge.txt is 38cb437 on both sides because Asha and you added the same line independently: taken once, no message. scripts/legacy_eval.sh is unchanged in ours and missing in theirs: deleted. evalkit/rouge.py exists only in theirs: added. Only config/eval.yaml has three different IDs, and its result, cef358f, is a new blob:

```
# What ours changed since the base (HEAD^1 is the first parent of the merge):
$ git diff -U0 base HEAD^1 -- config/eval.yaml
diff --git a/config/eval.yaml b/config/eval.yaml
index c5b3327..0e8b97c 100644
--- a/config/eval.yaml
+++ b/config/eval.yaml
@@ -2,2 @@ model: judge-large-v2
-temperature: 0.2
+temperature: 0.0

# What theirs changed since the base (HEAD^2 is the second parent):
$ git diff -U0 base HEAD^2 -- config/eval.yaml
diff --git a/config/eval.yaml b/config/eval.yaml
index c5b3327..caf8b0d 100644
--- a/config/eval.yaml
+++ b/config/eval.yaml
@@ -5,5 @@ batch_size: 16
-timeout_s: 30
+timeout_s: 60

# The merged file carries both changes:
$ git diff -U0 base HEAD -- config/eval.yaml
diff --git a/config/eval.yaml b/config/eval.yaml
index c5b3327..cef358f 100644
--- a/config/eval.yaml
+++ b/config/eval.yaml
@@ -2,2 @@ model: judge-large-v2
-temperature: 0.2
+temperature: 0.0
@@ -5,5 @@ batch_size: 16
-timeout_s: 30
+timeout_s: 60
```

Picture.



Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
git merge <other>, true merge without conflict 🟡	files updated to the merged tree	rewritten to match the merged tree	still ref: refs/heads/main	moves to the new merge commit	new blobs, trees and one commit; ORIG_HEAD set to the old tip; reflogs gain an entry	unchanged	unchanged

In production. The tree of a merge commit is a snapshot that neither author wrote and that no test has run on. Both parents can be green while the merge is red (section 8.15), so CI has to run on the merge result.

GitHub, not Git. For a pull_request event GitHub Actions checks out a merge of the pull request's head into the current base by default, so the tests run against the merged result and not against the head branch alone (events that trigger workflows). That merge is computed on GitHub's servers (section 8.17). Chapter 20A covers what this means for GITHUB_SHA.

8.5 Criss-cross histories: more than one merge base

In one sentence. When two branches have each merged the other, they can have two best common ancestors, and Git then merges those ancestors with each other first and uses the result as the base.

Analogy. Two Monday copies, each containing a decision that one editor made and the other rejected. Before the editor-in-chief can compare Friday's copies, the two Monday copies have to be reconciled into one. The analogy breaks when they cannot be: Git then keeps the disagreement inside the base, as conflict markers.

Precisely. `git merge-base --all` prints every merge base. Without `--all` you get one, and "it is unspecified which best one is output" ([git-merge-base](#)). The default strategy handles the case as follows: "When there is more than one common ancestor that can be used for 3-way merge, it creates a merged tree of the common ancestors and uses that as the reference tree for the 3-way merge" ([git-merge](#)). That merged tree is called the virtual merge base.

Inside .git. The virtual merge base is computed during the merge and never becomes a commit. Its only visible trace is stage 1 of a conflicted path.

See it. `main` and `release/1.0` disagreed about the judge temperature. Asha merged `main` into the release branch and kept 0.7. At the same time you merged the release branch into `main` and kept 0.0:

```
$ git log --oneline --graph --all
* bd573ed Merge release/1.0 into main, keep temperature 0.0
|\
| | * 4f30da1 Merge main into release/1.0, keep temperature 0.7
| | /|
| | /
| | /|
| | /|
* | 5d28836 Use temperature 0 for reproducible evals
| * 2725147 Raise temperature for the 1.0 judge
|/
* 36749ac Add eval config, judge prompt and metrics
$ git merge-base main release/1.0
272514788670fe9d71d00daeca408d3494e15041
$ git merge-base --all main release/1.0
272514788670fe9d71d00daeca408d3494e15041
5d288362f8c011ad12da5920c8cc763443e61c5d
```

Two merge bases: 2725147 (her commit) and 5d28836 (yours). What would happen if Git picked one? `git merge-tree` (section 8.17) can be told which base to use:

```
# Merge with only the first candidate as the base:
$ tree=$(git merge-tree --write-tree --merge-base=release/1.0~1 main release/1.0)
$ git show $tree:config/eval.yaml | grep temperature
temperature: 0.0

# Merge with only the second candidate as the base:
$ tree=$(git merge-tree --write-tree --merge-base=main~1 main release/1.0)
$ git show $tree:config/eval.yaml | grep temperature
temperature: 0.7
```

Two silent, opposite answers. Against her commit as base, the release tip "did nothing" to the temperature and your side wins. Against your commit as base, her side wins. Neither would show a conflict, and nobody would have decided either. The real merge does better:

```
$ git merge release/1.0
Auto-merging config/eval.yaml
CONFLICT (content): Merge conflict in config/eval.yaml
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git ls-files -u
100644 eeb99652ab2e2dc7f9c07b8a0db026159a245750 1      config/eval.yaml
100644 0e8b97cd35b023b2949d7efeed9b9dbd491d065ae8 2      config/eval.yaml
100644 3e21100524f1361d04738d41f1012b9221082155 3      config/eval.yaml
$ git show :1:config/eval.yaml
model: judge-large-v2
<<<<<<<<< Temporary merge branch 1
temperature: 0.0
=====
temperature: 0.7
>>>>>>>> Temporary merge branch 2
max_tokens: 512
batch_size: 16
timeout_s: 30
retries: 2
seed: 7
```

Stage 1, the base version, contains conflict markers. That file is the virtual merge base: the result of merging the two base commits with each other, labelled "Temporary merge branch 1" and "2". Because both tips differ from it, the outer merge conflicts and the decision comes back to you. The `diff3` conflict style (section 8.9) shows the nesting in one view. The inner markers are nine characters long, the outer ones seven:

```
$ git merge --abort
$ git -c merge.conflictStyle=diff3 merge release/1.0
Auto-merging config/eval.yaml
CONFLICT (content): Merge conflict in config/eval.yaml
Automatic merge failed: fix conflicts and then commit the result.
```

```

[exit status: 1]
$ cat config/eval.yaml
model: judge-large-v2
<<<<<< HEAD
temperature: 0.0
|||||| merged common ancestors
<<<<<<< Temporary merge branch 1
temperature: 0.0
||||||| 36749ac
temperature: 0.2
=====
temperature: 0.7
>>>>>>> Temporary merge branch 2
=====
temperature: 0.7
>>>>>> release/1.0
max_tokens: 512
batch_size: 16
timeout_s: 30
retries: 2
seed: 7

```

Picture.

```

36749ac---5d28836---bd573ed   main   (HEAD -> main)
      \               /
       \             /X\
        \           /   \
         2725147---4f30da1   release/1.0

```

merge bases of main and release/1.0: 5d28836 and 2725147

Observed behavior : The base version of a conflicted file (git show :1:path, or the middle section in diff3 style) contains conflict markers labelled "Temporary merge branch 1" and "Temporary merge branch 2".

Git state : git merge-base --all prints two commits; neither is an ancestor of the other.

Mechanism : The ort strategy first merges the two merge bases with each other. That inner merge conflicts, and its result, markers included, becomes the base.

Root cause : A criss-cross. Each branch merged the other, and the two merges resolved the same conflict in opposite ways.

Why Git does this : Either base alone would silently reapply one side's old decision, as the two-answer transcript shows. A base that matches neither tip returns the question to a human.

Correct fix : Resolve it as an ordinary conflict. Read the inner block as the two earlier decisions and the outer sides as the two current values.

Prevention : Between long-lived branches, merge in one direction only. Keep other branches short enough that they never merge each other.

In production. The usual producers are a release branch and `main` that merge each other "to stay in sync", and two developers who each pull the other's feature branch. The reported symptom is "the same conflict keeps coming back". Run `git merge-base --all` before theorizing.

8.6 Strategies and strategy options

In one sentence. A merge strategy is the algorithm that turns the inputs into a result tree, selected with `-s` ; a strategy option, passed with `-X`, adjusts how one strategy behaves.

Precisely. These are the strategies that the Git 2.55 manual lists, in its own words where quoted:

Strategy	Heads	What the manual says	Where you meet it
<code>ort</code>	2	"the default merge strategy when pulling or merging one branch"; 3-way; builds a virtual base when there are several merge bases; detects renames; defaults to the histogram diff algorithm	Every ordinary merge; also under rebase, cherry-pick and revert (Chapters 9 to 11)
<code>recursive</code>	2	"now a synonym for <code>ort</code> "	Old scripts and old tutorials
<code>resolve</code>	2	3-way; "does not handle renames"	Legacy; section 8.20 shows why to leave it alone
<code>octopus</code>	more than 2	"refuses to do a complex merge that needs manual resolution"; the default for several heads	Section 8.14
<code>ours</code>	any	the result tree "is always that of the current branch head, effectively ignoring all changes from all other branches"	Recording that an obsolete branch is superseded
<code>subtree</code>	2	"a modified <code>ort</code> strategy" that shifts one tree to match a subdirectory of the other	Embedding another project in a subdirectory

The options that `ort` accepts with `-X` are `ours` and `theirs` (which side wins a conflicting hunk), the whitespace options `ignore-space-change`, `ignore-all-space`, `ignore-space-at-eol` and `ignore-cr-at-eol`, the rename options `find-renames[=<n>]` and `no-renames` (section 8.11), and `diff-algorithm=<name>`, `renormalize` and `subtree[=<path>]`.

`-X ours` **is not** `-s ours`. You changed the temperature; Asha changed the temperature and the seed, and added a prompt line.

```
# ours changed temperature; theirs changed temperature AND seed, and added a prompt line.
$ git diff --stat main...feature/creative-judge
config/eval.yaml | 4 ++--
prompts/judge.txt | 1 +
2 files changed, 3 insertions(+), 2 deletions(-)
$ grep -n -e temperature -e seed config/eval.yaml
2:temperature: 0.0
7:seed: 7
```

```
$ git merge -X ours feature/creative-judge
Auto-merging config/eval.yaml
Merge made by the 'ort' strategy.
config/eval.yaml | 2 +-
prompts/judge.txt | 1 +
2 files changed, 2 insertions(+), 1 deletion(-)
$ grep -n -e temperature -e seed config/eval.yaml
2:temperature: 0.0
7:seed: 1234
$ tail -1 prompts/judge.txt
Explain your verdict in one sentence.
```

`-X ours` is a real three-way merge in which only the conflicting hunk is decided in your favour (temperature stays 0.0). Her seed and her prompt line arrive. `-X theirs` is the mirror image:

```
$ git merge -X theirs feature/creative-judge
Auto-merging config/eval.yaml
Merge made by the 'ort' strategy.
config/eval.yaml | 4 ++--
prompts/judge.txt | 1 +
2 files changed, 3 insertions(+), 2 deletions(-)
$ grep -n -e temperature -e seed config/eval.yaml
2:temperature: 0.7
7:seed: 1234
```

`-s ours` does not look at her tree at all:

```
$ git merge -s ours feature/creative-judge
Merge made by the 'ours' strategy.
$ grep -n -e temperature -e seed config/eval.yaml
2:temperature: 0.0
7:seed: 7
$ tail -1 prompts/judge.txt
Answer with PASS or FAIL.
# The tree of the merge commit is the tree of its first parent:
$ git rev-parse HEAD^{tree} HEAD^1^{tree}
3dfab55ba26e160c0bf7d8f7533173a3d264618f
3dfab55ba26e160c0bf7d8f7533173a3d264618f
# Yet the history says the branch is merged:
$ git branch --merged
feature/creative-judge
* main
```

No seed change, no prompt line, and the tree of the merge commit is identical to the tree of its first parent. Yet `git branch --merged` lists her branch, because her commits are now ancestors of `main`. The merge recorded ancestry and discarded content.

	<code>-X ours</code>	<code>-s ours</code>
What it is	an option of the <code>ort</code> strategy	a different strategy
Their non-conflicting changes	merged	discarded
Conflicting hunks	your side, without a conflict being reported	not applicable
Tree of the merge commit	a real merge result	identical to the first parent's tree
Legitimate use	rare: mechanical conflicts where one side is known to be right	declare a branch obsolete so that it never merges again

There is no `-s theirs`, and `recursive` still works as a name:

```
$ git merge -s theirs feature/creative-judge
Could not find merge strategy 'theirs'.
Available strategies are: octopus ours recursive resolve subtree.
[exit status: 1]
```

```
$ git merge -s recursive -X ours feature/creative-judge
Auto-merging config/eval.yaml
Merge made by the 'recursive' strategy.
config/eval.yaml | 2 +-
prompts/judge.txt | 1 +
2 files changed, 2 insertions(+), 1 deletion(-)
```

The list of "available strategies" lacks `ort` because it enumerates helper programs named `git-merge-<strategy>` on disk, and `ort` is built into `git merge` itself (section 8.20).

Whitespace options. A formatter re-indent a YAML list on one branch while you rename a metric on `main`:

```
$ git merge chore/yaml-format
Auto-merging config/eval.yaml
CONFLICT (content): Merge conflict in config/eval.yaml
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ cat config/eval.yaml
model: judge-large-v2
metrics:
<<<<<<< HEAD
- exact_match
- token_f1
=====
- exact_match
- f1
>>>>>>> chore/yaml-format
seed: 7
$ git merge --abort
```

```
$ git merge -X ignore-space-change chore/yaml-format
Auto-merging config/eval.yaml
Merge made by the 'ort' strategy.
$ cat config/eval.yaml
model: judge-large-v2
metrics:
- exact_match
- token_f1
seed: 7
```

The conflict is gone, and so is the reformatting. The merged file has your two-space indentation, and the merge printed no diffstat because its tree equals your tree.

```
Observed behavior : After git merge -X ignore-space-change, the formatting branch counts as
                    merged, but the file is not reformatted.
Git state          : The merge commit's tree is identical to its first parent's tree.
Mechanism          : With a whitespace option, "if their version only introduces whitespace
                    changes to a line, our version is used" (git-merge manual).
Root cause         : The option does not merge whitespace changes. It discards them
                    wherever they meet a line that our side kept or changed.
Why Git does this  : A line cannot carry their indentation and our text at the same time
                    without Git inventing a third version.
Correct fix        : Run the formatter again on the merge result and commit.
Prevention         : Land formatting-only changes when no other branch is open on those files,
                    or have every branch run the same formatter before merging.
```

In production. `-X ours` and `-X theirs` turn "Git stopped and asked" into "Git chose and said nothing". They are defensible where one side is correct by construction, as with a regenerated lock file. They are indefensible as a way to make a red merge green. The audit command of section 8.16 still shows the choice, because it repeats the merge without your options.

8.7 Exactly why conflicts occur

In one sentence. A content conflict occurs when both sides changed the same lines of a file, or lines with no unchanged line between them, and did not make the identical change.

Analogy. Two people mark up the same printed page. Marks in different paragraphs can both be typed in. Marks in the same sentence need a conversation. The analogy breaks at the boundary: Git's unit is the line and its test is adjacency, not whether the two edits have anything to do with each other.

Precisely. For a path in the "X, Y, Z" row of the rule table, Git merges the three file contents. It finds what ours changed relative to the base and what theirs changed relative to the base, each as a set of hunks: runs of base lines that were replaced, removed or inserted into. Hunks that do not touch are both applied. Hunks that cover the same base lines with the same replacement are applied once. Everything else is a conflict. The manual's wording is "When both sides made changes to the same area, however, Git cannot randomly pick one side over the other" ([git-merge](#)). The unit is the line. A one-word change in a long line conflicts with any other change to that line.

See it. `git merge-file` runs this file-level merge on three ordinary files, with no repository involved. Its exit status is the number of conflicts. Start with changes in different regions:

```
$ cat base.yaml
model: judge-large-v2
temperature: 0.2
max_tokens: 512
batch_size: 16
timeout_s: 30
$ sed 's/temperature: 0.2/temperature: 0.0/' base.yaml > ours.yaml
$ sed 's/timeout_s: 30/timeout_s: 60/' base.yaml > theirs.yaml
$ git merge-file -p ours.yaml base.yaml theirs.yaml
model: judge-large-v2
temperature: 0.0
max_tokens: 512
```

```
batch_size: 16
timeout_s: 60
[exit status: 0]
```

Same line, different values: a conflict. Same line, same value: none.

```
$ sed 's/temperature: 0.2/temperature: 0.7/' base.yaml > theirs.yaml
$ git merge-file -p ours.yaml base.yaml theirs.yaml
model: judge-large-v2
<<<<<< ours.yaml
temperature: 0.0
=====
temperature: 0.7
>>>>>> theirs.yaml
max_tokens: 512
batch_size: 16
timeout_s: 30
[exit status: 1]
```

```
$ sed 's/temperature: 0.2/temperature: 0.0/' base.yaml > theirs.yaml
$ git merge-file -p ours.yaml base.yaml theirs.yaml
model: judge-large-v2
temperature: 0.0
max_tokens: 512
batch_size: 16
timeout_s: 30
[exit status: 0]
```

The case that surprises people is the next one. Ours changes line 2, theirs changes line 3:

```
# ours changes line 2, theirs changes line 3: no unchanged line between them.
$ sed 's/max_tokens: 512/max_tokens: 1024/' base.yaml > theirs.yaml
$ git merge-file -p ours.yaml base.yaml theirs.yaml
model: judge-large-v2
<<<<<< ours.yaml
temperature: 0.0
max_tokens: 512
=====
temperature: 0.2
max_tokens: 1024
>>>>>> theirs.yaml
batch_size: 16
timeout_s: 30
[exit status: 1]
```

```
# ours changes line 2, theirs changes line 4: line 3 is unchanged on both sides.
$ sed 's/batch_size: 16/batch_size: 32/' base.yaml > theirs.yaml
$ git merge-file -p ours.yaml base.yaml theirs.yaml
model: judge-large-v2
temperature: 0.0
max_tokens: 512
batch_size: 32
timeout_s: 30
[exit status: 0]
```

Observed behavior : A conflict, although the two branches changed different lines.
Git state : Ours changed line 2 of the file, theirs changed line 3.
Mechanism : The merge works on hunks. Two hunks with no unchanged line between them form one region, and that region was changed by both sides.
Root cause : Adjacency. One unchanged line between the edits (the last transcript) is enough to keep them apart.
Why Git does this : It needs a boundary between "independent" and "the same area", and an unchanged line is the only evidence of independence that text offers.
Correct fix : Take the changed line from each side: temperature 0.0 and max_tokens 1024.
Prevention : None needed. Recognize the pattern and resolve it in seconds. Do not reach for -X ours, which would drop their line.

Picture.

base lines	1	2	3	4	5	
ours changes		#				
theirs changes					#	far apart: both applied
theirs changes			#			touches ours: one region, conflict
theirs changes				#		one unchanged line between: both applied

What Git never does is conflict on meaning. Two changes that are textually far apart and logically incompatible merge without a word (section 8.15).

In production. Some files conflict far more often than others. A Jupyter notebook stores outputs and an execution counter next to the code, so two people who only ran the same notebook have changed the same lines, and a conflicted merge can leave a file that is no longer valid JSON ([nbdime](#); Chapter 28). Append-only files are the other regular source: changelogs, migration lists, dependency lists. Every branch inserts after the same last line, which is the same region.

8.8 Anatomy of a conflict: working tree, index, `.git`

In one sentence. A conflicted merge is a merge that stopped halfway and wrote its unfinished state into three places: marker blocks in working tree files, up to three index entries per conflicted path, and a handful of files in `.git`.

Analogy. A customs desk. Everything that passed inspection is already through. Each contested parcel sits on the counter with three documents: the original declaration, your version and their version. The analogy breaks on time: nothing here expires until you conclude the merge or abort it.

Precisely. The manual lists what happens when a merge cannot complete ([git-merge](#)):

1. "The `HEAD` pointer stays the same."
2. `MERGE_HEAD` is set to the other branch's tip.
3. Paths that merged cleanly are updated in the index and in the working tree.
4. For conflicting paths the index records up to three versions: "stage 1 stores the version from the common ancestor, stage 2 from `HEAD`, and stage 3 from `MERGE_HEAD`". The working tree files contain the merge result with conflict markers.
5. `AUTO_MERGE` points to a tree that matches what was written to the working tree.
6. Nothing else changes.

Ours and theirs, exactly. In a merge, ours is stage 2: the commit you were on when you typed `git merge`. Theirs is stage 3: the commit you named. It has nothing to do with who wrote what. If Asha checks out your branch and merges `main`, your commits are "ours" to her. During a rebase the roles are swapped relative to intuition (Chapter 9).

See it. You set the temperature to 0.0 on `main`. Asha set it to 0.7 on her branch, changed the seed in the same commit, and added a prompt line in another:

```
$ git log --oneline --graph --all
* 5397d5f Use temperature 0 for reproducible evals
| * 45a7a67 Ask the judge for a rationale
| * 640bfe1 Raise temperature and reseed for judge diversity
|/
* 6ae3c51 Add eval config, judge prompt and metrics
$ git merge feature/creative-judge
Auto-merging config/eval.yaml
CONFLICT (content): Merge conflict in config/eval.yaml
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
```

The exit status is 1. `git status` separates what is done from what is open:

```
$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Changes to be committed:
  modified:   prompts/judge.txt

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   config/eval.yaml
```

`prompts/judge.txt` merged cleanly and is already staged. Only `config/eval.yaml` is unmerged. Its working tree copy:

```
$ cat config/eval.yaml
model: judge-large-v2
<<<<<< HEAD
temperature: 0.0
=====
temperature: 0.7
>>>>>> feature/creative-judge
max_tokens: 512
batch_size: 16
timeout_s: 30
retries: 2
seed: 1234
```

Between `<<<<<< HEAD` and `=====` is our version of the region, between `=====` and `>>>>>> feature/creative-judge` is theirs. The last line, `seed: 1234`, is her seed change, merged outside the markers. A conflicted file is mostly finished. Only the marked regions are open.

The index holds the three whole files:

```
$ git ls-files -s
100644 c5b33275d88c041381cff59f23d3cd88b9f22c98 1      config/eval.yaml
100644 0e8b97cd35b023b2949d7efeed9dbd491d065ae8 2      config/eval.yaml
100644 db5c556ef5edb6802e492cd59b5a0b9721a882da 3      config/eval.yaml
100644 9bbcd7ae8147833209c2dbce940d2fd9d681c8ac 0      evalkit/metrics.py
100644 38cb43793f7d697e3d7c88c83da85d1fcd783d68 0      prompts/judge.txt
```



```
$ git ls-files -u
100644 c5b33275d88c041381cff59f23d3cd88b9f22c98 1      config/eval.yaml
100644 0e8b97cd35b023b2949d7efeed9dbd491d065ae8 2      config/eval.yaml
100644 db5c556ef5edb6802e492cd59b5a0b9721a882da 3      config/eval.yaml
```

The third column is the stage. Merged paths are at stage 0. The conflicted path has no stage 0 entry and three others. The syntax `:<stage>:` `<path>` names those blobs, so any command that reads objects can read them:

```
$ git show :1:config/eval.yaml
model: judge-large-v2
temperature: 0.2
max_tokens: 512
batch_size: 16
timeout_s: 30
retries: 2
seed: 7
# What our side did to the base, and what their side did:
$ git diff -U0 :1:config/eval.yaml :2:config/eval.yaml
diff --git a/config/eval.yaml b/config/eval.yaml
index c5b3327..0e8b97c 100644
--- a/config/eval.yaml
+++ b/config/eval.yaml
@@ -2,2 @@ model: judge-large-v2
-temperature: 0.2
+temperature: 0.0
$ git diff -U0 :1:config/eval.yaml :3:config/eval.yaml
diff --git a/config/eval.yaml b/config/eval.yaml
index c5b3327..db5c556 100644
--- a/config/eval.yaml
+++ b/config/eval.yaml
@@ -2,2 @@ model: judge-large-v2
-temperature: 0.2
+temperature: 0.7
@@ -7,7 @@ retries: 2
-seed: 7
+seed: 1234
```

Those two diffs, base to ours and base to theirs, are the most useful commands in conflict resolution: they show what each side intended. Here they reveal something the marker block did not: their side changed two lines, ours one.

Inside `.git`.

```
$ ls .git | grep -E 'MERGE|ORIG_HEAD'
AUTO_MERGE
MERGE_HEAD
MERGE_MODE
MERGE_MSG
ORIG_HEAD
$ cat .git/MERGE_HEAD
45a7a672d9b049a35269a5e7d09e7920fa1d4c87
$ git rev-parse feature/creative-judge
45a7a672d9b049a35269a5e7d09e7920fa1d4c87
$ git rev-parse HEAD ORIG_HEAD
5397d5f953dda5e3acd0d0970078420baf6e0488
5397d5f953dda5e3acd0d0970078420baf6e0488
$ cat .git/MERGE_MSG
Merge branch 'feature/creative-judge'

# Conflicts:
#      config/eval.yaml
$ git cat-file -t AUTO_MERGE
tree
```

`MERGE_HEAD` is the commit being merged. It is what will make the next commit a merge commit. `HEAD` has not moved, and `ORIG_HEAD` equals it. `MERGE_MSG` is the prepared message, with the conflicted paths as a comment. `MERGE_MODE` is empty unless you passed `--no-ff`. `AUTO_MERGE` is a tree, the snapshot of what Git wrote to the working tree, markers included. During a conflict `git diff` prints a combined diff, one column per side:

```
$ git diff
diff --cc config/eval.yaml
index 0e8b97c,db5c556..0000000
--- a/config/eval.yaml
+++ b/config/eval.yaml
@@@ -1,5 -1,5 +1,9 @@@
 model: judge-large-v2
++<<<<<< HEAD
+temperature: 0.0
++=====
+ temperature: 0.7
++>>>>>> feature/creative-judge
+ max_tokens: 512
+ batch_size: 16
+ timeout_s: 30
```

The first column compares the working tree file with ours (stage 2), the second with theirs (stage 3). A + in both columns marks a line that neither side has, which here means the marker lines. `git diff --ours`, `--theirs` and `--base` compare the working tree file with one stage in the ordinary two-way format.

While the merge is open, Git refuses the two things that would lose track of it:

```
$ git commit -m "Merge feature/creative-judge"
error: Committing is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
U      config/eval.yaml
[exit status: 128]
$ git switch feature/creative-judge
fatal: cannot switch branch while merging
Consider "git merge --quit" or "git worktree add".
[exit status: 128]
```

Picture.

Working tree	Index (stage, path, blob)	HEAD -> main -> 5397d5f
+-----+	+-----+	+-----+
config/eval.yaml	1 config/eval.yaml c5b3327	config/eval.yaml
merged, except one	2 config/eval.yaml 0e8b97c	0e8b97c
marker block (line 2)	3 config/eval.yaml db5c556	
prompts/judge.txt	0 prompts/judge.txt 38cb437	prompts/judge.txt
merged	0 evalkit/metrics.py 9bbcd7a	cfe0ef0
+-----+	+-----+	+-----+
.git/MERGE_HEAD = 45a7a67 (their tip) .git/ORIG_HEAD = 5397d5f .git/MERGE_MSG .git/AUTO_MERGE (a tree)		

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
git merge <other> that stops on a conflict ●	clean paths updated; conflicted files rewritten with marker blocks	clean paths staged at stage 0; conflicted paths at stages 1, 2, 3	unchanged	unchanged	MERGE_HEAD, MERGE_MSG, MERGE_MODE, AUTO_MERGE created; ORIG_HEAD set; new blobs and trees; no commit	unchanged	unchanged

In production. `git mergetool` hands the three stage versions to an external tool chosen by `merge.tool`, and IDE merge editors work from the same stages. When a tool and the command line seem to disagree, `git ls-files -u` is the ground truth.

8.9 Conflict styles: `merge`, `diff3`, `zdiff3`

In one sentence. The conflict style decides how much of the three versions is written between the markers: `merge` shows ours and theirs, `diff3` adds the base, and `zdiff3` adds the base while moving lines that both sides share at the edges out of the block.

Precisely. `merge.conflictStyle` selects the style, and `merge` is the default. `git checkout --conflict=<style> <path>` and `git restore --conflict=<style> <path>` rewrite one conflicted file in another style from the index stages, without repeating the merge.

See it. Both branches added the same guard against missing predictions and then changed the comparison differently. In the default style:

```
# The function as both branches found it (the merge base):
$ git show main~1:evalkit/metrics.py | head -2
def exact_match(pred, gold):
    return float(pred == gold)
$ git merge feature/case-insensitive
Auto-merging evalkit/metrics.py
CONFLICT (content): Merge conflict in evalkit/metrics.py
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ head -9 evalkit/metrics.py
def exact_match(pred, gold):
    if pred is None:
        return 0.0
<<<<<< HEAD
    return float(pred.strip() == gold.strip())
=====
    return float(pred.lower() == gold.lower())
>>>>>> feature/case-insensitive
```

Two candidate lines and no hint of what the line was before. Did one side add `.strip()` and the other `.lower()`, or did someone remove something? In `diff3` style the section after `|||||` is the base:

```
$ git checkout --conflict=diff3 evalkit/metrics.py
Recreated 1 merge conflict
$ head -13 evalkit/metrics.py
def exact_match(pred, gold):
<<<<<< ours
    if pred is None:
```

```

        return 0.0
    return float(pred.strip() == gold.strip())
|||||| base
    return float(pred == gold)
=====
    if pred is None:
        return 0.0
    return float(pred.lower() == gold.lower())
>>>>>> theirs

```

Now the base is visible, and it settles the question: the original compared `pred == gold`, so each side added one normalization and the right answer applies both. The price is a larger block. The guard lines that both sides added identically are inside the conflict, twice. `zdiff3` keeps the base and removes that noise:

```

$ git restore --conflict=zdiff3 evalkit/metrics.py
$ head -11 evalkit/metrics.py
def exact_match(pred, gold):
    if pred is None:
        return 0.0
<<<<<< ours
    return float(pred.strip() == gold.strip())
|||||| base
    return float(pred == gold)
=====
    return float(pred.lower() == gold.lower())
>>>>>> theirs

```

The labels changed to `ours`, `base` and `theirs` because these two files were rebuilt from index stages, and stages carry no branch names. A fresh merge with the setting in place labels the base with its abbreviated commit ID:

```

$ git merge --abort
$ git config set merge.conflictStyle zdiff3
$ git config get --show-origin merge.conflictStyle
file:.git/config      zdiff3
$ git merge feature/case-insensitive
Auto-merging evalkit/metrics.py
CONFLICT (content): Merge conflict in evalkit/metrics.py
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ head -11 evalkit/metrics.py
def exact_match(pred, gold):
    if pred is None:
        return 0.0
<<<<<< HEAD
    return float(pred.strip() == gold.strip())
|||||| 6ae3c51
    return float(pred == gold)
=====
    return float(pred.lower() == gold.lower())
>>>>>> feature/case-insensitive

```

Picture.

<<<<<< HEAD	our version starts	(stage 2)
6ae3c51	base version starts	(stage 1; the label is the merge base)
=====	their version starts	(stage 3)
>>>>>> feature/case-insensitive	end of the block	

How to read a three-part block. Compare each side with the base, not the sides with each other. A side that equals the base did nothing in this region: take the other side. An empty base means both sides added text at the same place: usually keep both. Two sides that each differ from the base are two intents, and you need to serve both.

In production. Set `git config set --global merge.conflictStyle zdiff3` once. The setting changes only how conflicts are written to your working tree and nothing that is committed, so it is a personal choice, not a team decision. The one thing to check is tooling that parses conflict markers, which must accept the `||||||` section. Lab 6.7 is a conflict that cannot be resolved correctly without the base.

8.10 The resolution workflow

Step 1: see what is open. `git status`, or `git diff --name-only --diff-filter=U` for only the unmerged paths.

Step 2: learn both intents. Resolving a conflict is an investigation, not an editing chore. The stage diffs of section 8.8 show what changed. `git log --merge` shows why: it lists the commits from either side that touch a conflicted path.

```

$ git log --oneline --left-right main...feature/creative-judge
< 5397d5f Use temperature 0 for reproducible evals
> 45a7a67 Ask the judge for a rationale
> 640bfe1 Raise temperature and reseed for judge diversity
$ git log --oneline --left-right --merge

```

```
< 5397d5f Use temperature 0 for reproducible evals
> 640bfe1 Raise temperature and reseed for judge diversity
```

Of Asha's two commits, only the one that touched `config/eval.yaml` is listed. Add `-p` to see the patches. If the intent is still unclear, ask the author.

Step 3: write the result. Edit the file until it says what the merged project should say, and delete the marker lines. The answer may be one side, both sides, or new text. Here the release requirement is reproducible scores, so the temperature stays at 0.0:

```
# In your editor: delete the three marker lines and the 0.7 line. Keep temperature 0.0.
$ cat config/eval.yaml
model: judge-large-v2
temperature: 0.0
max_tokens: 512
batch_size: 16
timeout_s: 30
retries: 2
seed: 1234
$ git diff AUTO_MERGE
diff --git a/config/eval.yaml b/config/eval.yaml
index 46bae64..1642b99 100644
--- a/config/eval.yaml
+++ b/config/eval.yaml
@@ -1,9 +1,5 @@
 model: judge-large-v2
-<<<<<<< HEAD
 temperature: 0.0
-=====
-temperature: 0.7
->>>>>> feature/creative-judge
 max_tokens: 512
 batch_size: 16
 timeout_s: 30
```

`git diff AUTO_MERGE` compares your file with what Git left behind, so it shows exactly what your resolution did: removed three marker lines and their candidate.

Step 4: mark it resolved. `git add` on an unmerged path replaces stages 1 to 3 with one stage 0 entry:

```
$ git add config/eval.yaml
$ git ls-files -s
100644 1642b99ceb64bd0a58b86f8d20ab8b50fd1c9cee 0      config/eval.yaml
100644 9bbcd7ae8147833209c2dbce940d2fd9d681c8ac 0      evalkit/metrics.py
100644 38cb43793f7d697e3d7c88c83da85d1fcd783d68 0      prompts/judge.txt
$ git status
On branch main
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
  modified:   config/eval.yaml
  modified:   prompts/judge.txt
```

`git add` does not check your work. It stages whatever is in the file, markers included (section 8.19). Run the tests now, and run `git diff --cached --check`, which reports leftover markers.

Step 5: conclude. `git merge --continue` checks that a merge is in progress and then runs `git commit`. Plain `git commit` does the same job. Use the message to record the decision:

```
$ git merge --continue
[main a94e9f6] Merge branch 'feature/creative-judge'
$ git log --oneline --graph
* a94e9f6 Merge branch 'feature/creative-judge'
|\
| * 45a7a67 Ask the judge for a rationale
| * 640bfe1 Raise temperature and reseed for judge diversity
* | 5397d5f Use temperature 0 for reproducible evals
|/
* 6ae3c51 Add eval config, judge prompt and metrics
$ ls .git | grep -E 'MERGE|ORIG_HEAD'
ORIG_HEAD
```

```
$ git show --stat --format=medium HEAD
commit a94e9f6aa320603f006ce6c0414285c7a24e1fb7
Merge: 5397d5f 45a7a67
Author: Lab User <you@example.com>
Date:   Mon Sep 7 10:33:00 2026 +0530

    Merge branch 'feature/creative-judge'

    Keep temperature 0.0: reproducible scores are a release requirement.
    The new seed and the rationale line in the judge prompt are taken as they are.

config/eval.yaml | 2 +--
```

```
prompts/judge.txt | 1 +
2 files changed, 2 insertions(+), 1 deletion(-)
```

The `MERGE_*` files are gone. The stat is relative to the first parent: it shows what the merge brought to `main`.

Whole-file shortcuts. `git restore --ours <path>` and `git restore --theirs <path>` write stage 2 or stage 3 to the working tree. They take the whole file, not the conflicted hunks:

```
$ git status --short
UU config/eval.yaml
M prompts/judge.txt
$ grep -n -e '<<<' -e temperature -e '>>>' -e seed config/eval.yaml
2:<<<<<< HEAD
3:temperature: 0.0
5:temperature: 0.7
6:>>>>>> feature/creative-judge
11:seed: 1234
$ git restore --ours config/eval.yaml
$ grep -n -e temperature -e seed config/eval.yaml
2:temperature: 0.0
7:seed: 7
$ git status --short
UU config/eval.yaml
M prompts/judge.txt
```

After `--ours` the seed is 7 again. Asha's non-conflicting change in this file is gone from the working tree, and if you stage this, from the merge. The path is still `UU` because the index was not touched.

```
$ git restore --theirs config/eval.yaml
$ grep -n -e temperature -e seed config/eval.yaml
2:temperature: 0.7
7:seed: 1234
$ git status --short
UU config/eval.yaml
M prompts/judge.txt
```

`git restore --merge <path>` rebuilds the marker version from the stages. It is the undo for both shortcuts and for a botched edit:

```
$ git restore --merge config/eval.yaml
$ grep -n -e '<<<' -e temperature -e '>>>' -e seed config/eval.yaml
2:<<<<<< ours
3:temperature: 0.0
5:temperature: 0.7
6:>>>>>> theirs
11:seed: 1234
```

Older scripts use `git checkout` for the same three operations:

```
$ git checkout --ours config/eval.yaml
Updated 1 path from the index
$ git checkout --theirs config/eval.yaml
Updated 1 path from the index
$ git checkout --merge config/eval.yaml
Recreated 1 merge conflict
$ git status --short
UU config/eval.yaml
M prompts/judge.txt
```

`--merge` also works after `git add`. The index keeps a record of the stages that the resolution replaced:

```
# Take their whole file, stage it, then change your mind.
$ git restore --theirs config/eval.yaml
$ git add config/eval.yaml
$ git status --short
M config/eval.yaml
M prompts/judge.txt
$ git ls-files -s config/eval.yaml
100644 db5c556ef5edb6802e492cd59b5a0b9721a882da 0      config/eval.yaml
$ git restore --merge config/eval.yaml
$ git status --short
UU config/eval.yaml
M prompts/judge.txt
$ git ls-files -s config/eval.yaml
100644 c5b33275d88c041381cff59f23d3cd88b9f22c98 1      config/eval.yaml
100644 0e8b97cd35b023b2949d7efeed9dbd491d065ae8 2      config/eval.yaml
100644 db5c556ef5edb6802e492cd59b5a0b9721a882da 3      config/eval.yaml
```

The three exits. `--continue` needs every path resolved:

```
$ git merge feature/creative-judge
Auto-merging config/eval.yaml
CONFLICT (content): Merge conflict in config/eval.yaml
```

```
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git merge --continue
error: Committing is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
U      config/eval.yaml
[exit status: 128]
```

`--abort` returns to the state before the merge. The manual defines it as `git reset --merge` while `MERGE_HEAD` exists, and the relog shows the reset:

```
$ git merge --abort
$ git status --short --branch
## main
$ ls .git | grep -E 'MERGE|ORIG_HEAD'
ORIG_HEAD
$ git relog -2
5397d5f HEAD@{0}: reset: moving to HEAD
5397d5f HEAD@{1}: commit: Use temperature 0 for reproducible evals
```

Outside a merge, both commands say so:

```
$ git merge --abort
fatal: There is no merge to abort (MERGE_HEAD missing).
[exit status: 128]
$ git merge --continue
fatal: There is no merge in progress (MERGE_HEAD missing).
[exit status: 128]
```

`--quit` is the odd one. It deletes the merge state and leaves the index and the working tree as they are:

```
$ git merge feature/creative-judge
Auto-merging config/eval.yaml
CONFLICT (content): Merge conflict in config/eval.yaml
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git merge --quit
[exit status: 0]
$ ls .git | grep -E 'MERGE|ORIG_HEAD'
ORIG_HEAD
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   prompts/judge.txt

Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
    both modified:   config/eval.yaml
```

```
# The merge is forgotten, the half-merged files are not. A commit made now has ONE parent:
$ git add config/eval.yaml
$ git commit -q -m "Resolve temperature conflict"
$ git log --oneline --graph --all
* cdacf6e Resolve temperature conflict
* 5397d5f Use temperature 0 for reproducible evals
| * 45a7a67 Ask the judge for a rationale
| * 640bfe1 Raise temperature and reseed for judge diversity
|/
* 6ae3c51 Add eval config, judge prompt and metrics
$ git branch --no-merged
feature/creative-judge
```

Without `MERGE_HEAD` the commit has one parent. The content of Asha's branch is in `main`, but her commits are not ancestors of it, so the branch still counts as unmerged. To get the pre-merge state back after `--quit`, use `git reset --merge` 🟡:

```
# The clean way back after --quit: reset the index and the files the merge touched.
$ git merge --quit
$ git merge --abort
fatal: There is no merge to abort (MERGE_HEAD missing).
[exit status: 128]
$ git reset --merge
$ git status --short --branch
## main
```

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git restore --ours <path></code>	file overwritten with the whole stage 2 or	unchanged (path stays unmerged)	unchanged	unchanged	unchanged	unchanged	unchanged

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
or <code>--theirs <path></code> 	stage 3 version; your edits to it are lost						
<code>git restore --merge <path></code> 	file overwritten with a fresh marker version; your edits to it are lost	stages 1 to 3 restored if the path was already staged	unchanged	unchanged	unchanged	unchanged	unchanged
<code>git add <path></code> on an unmerged path 	unchanged	stages 1 to 3 replaced by one stage 0 entry with the file's content	unchanged	unchanged	new blob	unchanged	unchanged
<code>git merge --continue</code> 	unchanged	unchanged	still ref: refs/heads/main	moves to the new merge commit	merge commit and trees written; MERGE_* and AUTO_MERGE removed; reflogs gain commit (merge)	unchanged	unchanged
<code>git merge --abort</code> 	merge results removed; files as before the merge, with the exceptions of section 8.19	reset to HEAD	unchanged	unchanged	MERGE_* and AUTO_MERGE removed; reflog gains reset: moving to HEAD	unchanged	unchanged
<code>git merge --quit</code> 	unchanged	unchanged, still unmerged	unchanged	unchanged	MERGE_* and AUTO_MERGE removed	unchanged	unchanged

In production. The merge commit message is the only place where a resolution can explain itself. One sentence there answers the question that somebody will ask in six months.

8.11 Conflict types beyond content

The word in parentheses after `CONFLICT` names the type. Only some types put markers in a file. The table collects what the transcripts of this section show:

Type	<code>git status --short</code>	Stages present	Working tree after the merge stops	Resolve with
content	<code>UU</code>	1, 2, 3	file with marker blocks	edit, <code>git add</code>
add/add	<code>AA</code>	2, 3	file with one block holding both whole files	edit or choose, <code>git add</code>
modify/delete	<code>UD</code> (deleted by them) or <code>DU</code> (deleted by us)	1 and the modifying side	the modified version, no markers	<code>git add</code> to keep, <code>git rm</code> to delete
rename/rename	<code>DD</code> old name, <code>AU</code> our name, <code>UA</code> their name	one stage per path	both new files	<code>git add</code> one name, <code>git rm</code> the other two
rename/delete	<code>UD</code> on the new name	1, 2	the renamed file	<code>git add</code> or <code>git rm</code>
binary (reported as content)	<code>UU</code>	1, 2, 3	our version, no markers	pick a stage or regenerate, <code>git add</code>
file/directory	<code>AU</code> on a renamed path	2	the file moved aside, the directory in place	rename, <code>git add</code> , <code>git rm</code>
file location	<code>UA</code> on the suggested path	3	the file at the suggested path	<code>git add</code> to accept, or move it

add/add. Both branches create the same path with different content. There is no stage 1, because there was no common version:

```
$ git merge feature/disk-cache
Auto-merging evalkit/cache.py
CONFLICT (add/add): Merge conflict in evalkit/cache.py
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git status --short
AA evalkit/cache.py
$ git ls-files -u
100644 7f040298ad3ac9af50f69563114d5f3f5fff3576 2      evalkit/cache.py
100644 8d7d969205f3f1edf8a7f316f04e2481e6df6a31 3      evalkit/cache.py
$ git show :1:evalkit/cache.py
fatal: path 'evalkit/cache.py' is in the index, but not at stage 1
hint: Did you mean ':2:evalkit/cache.py'?
[exit status: 128]
```

```
$ cat evalkit/cache.py
<<<<<< HEAD
_CACHE = {}
```

```
def load(key):
    return _CACHE.get(key)
=====
import json

def load(path):
    with open(path) as f:
        return json.load(f)
>>>>>> feature/disk-cache
```

Two modules with the same name are usually two answers to one need. The resolution is a design conversation.

modify/delete. One side edited a file, the other deleted it. Git keeps the edited version in the working tree and waits:

```
$ git merge cleanup/remove-legacy
CONFLICT (modify/delete): scripts/legacy_eval.sh deleted in cleanup/remove-legacy and modified in HEAD. Version HEAD of
scripts/legacy_eval.sh left in tree.
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add/rm <file>..." as appropriate to mark resolution)
    deleted by them: scripts/legacy_eval.sh

no changes added to commit (use "git add" and/or "git commit -a")
```

```
$ git status --short
UD scripts/legacy_eval.sh
$ git ls-files -u
100644 4a41a25aa3c40cf94e023b2f4e69974e8d1b0e5f 1      scripts/legacy_eval.sh
100644 26610f62ce9e5b2744199a6dd9e966eb004ca890 2      scripts/legacy_eval.sh
$ ls scripts
legacy_eval.sh
```

Stage 3 does not exist: "their version" is no file. The whole-file shortcuts behave inconsistently here:

```
# Stage 3 does not exist for this path. The two commands react differently:
$ git checkout --theirs scripts/legacy_eval.sh
error: path 'scripts/legacy_eval.sh' does not have their version
[exit status: 1]
$ git restore --theirs scripts/legacy_eval.sh
[exit status: 0]
$ test -f scripts/legacy_eval.sh
[exit status: 1]
$ git status --short
UD scripts/legacy_eval.sh
$ git restore --ours scripts/legacy_eval.sh
$ test -f scripts/legacy_eval.sh
[exit status: 0]
$ git restore --merge scripts/legacy_eval.sh
error: path 'scripts/legacy_eval.sh' does not have all necessary versions
[exit status: 1]
```

`git checkout --theirs` refuses. `git restore --theirs` succeeds silently and removes the file from the working tree: restoring "their version" of a path that they deleted means deleting it. Neither marks the path as resolved. From the branch that did the deleting, the same conflict is `DU`:

```
$ git merge main
CONFLICT (modify/delete): scripts/legacy_eval.sh deleted in HEAD and modified in main. Version main of scripts/legacy_eval.sh
left in tree.
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git status --short
DU scripts/legacy_eval.sh
$ git ls-files -u
100644 4a41a25aa3c40cf94e023b2f4e69974e8d1b0e5f 1      scripts/legacy_eval.sh
100644 26610f62ce9e5b2744199a6dd9e966eb004ca890 3      scripts/legacy_eval.sh
# Decision on this branch: keep the edited file.
$ git add scripts/legacy_eval.sh
$ git status --short
A  scripts/legacy_eval.sh
```

The resolution is `git add` (the file lives) or `git rm` (the deletion stands):

```
# Back on main, same conflict as in the first snippet. Decision: the deletion wins.
$ git rm scripts/legacy_eval.sh
rm 'scripts/legacy_eval.sh'
$ git status --short
D  scripts/legacy_eval.sh
```



```
$ git merge --continue
[main 7e2950a] Merge branch 'cleanup/remove-legacy'
$ git ls-files
config/eval.yaml
evalkit/metrics.py
prompts/judge.txt
```

The question is not "which file wins" but "why was it deleted, and does the edit need a new home?" Lab 6.4 is such a case.

Renames. Git does not record renames (Chapter 4). The merge detects them by comparing content, with the same 50% similarity threshold as `git diff`. When detection succeeds, a rename on one side and an edit on the other is no conflict at all:

```
$ git log --oneline --graph --all
* e5b08ff Strip whitespace before comparing
| * 727adba Rename the metrics module to scoring
|/
* 6ae3c51 Add eval config, judge prompt and metrics
$ git merge refactor/scoring-module
Merge made by the 'ort' strategy.
  evalkit/{metrics.py => scoring.py} | 0
  1 file changed, 0 insertions(+), 0 deletions(-)
  rename evalkit/{metrics.py => scoring.py} (100%)
$ git ls-files evalkit
evalkit/scoring.py
$ head -2 evalkit/scoring.py
def exact_match(pred, gold):
    return float(pred.strip() == gold.strip())
```

Your edit to `metrics.py` arrived in `scoring.py`. Two different renames of the same file do conflict, and three paths need an answer:

```
$ git merge refactor/scorers
CONFLICT (rename/rename): evalkit/metrics.py renamed to evalkit/scoring.py in HEAD and to evalkit/scorers.py in
refactor/scorers.
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git status --short
DD evalkit/metrics.py
UA evalkit/scorers.py
AU evalkit/scoring.py
$ git ls-files -u
100644 9bbcd7ae8147833209c2dbce940d2fd9d681c8ac 1      evalkit/metrics.py
100644 9bbcd7ae8147833209c2dbce940d2fd9d681c8ac 3      evalkit/scorers.py
100644 9bbcd7ae8147833209c2dbce940d2fd9d681c8ac 2      evalkit/scoring.py
$ ls evalkit
scorers.py
scoring.py
```

```
# Decision: the module is called scoring. All three paths need an answer.
$ git add evalkit/scoring.py
$ git rm evalkit/scorers.py
rm 'evalkit/scorers.py'
$ git rm evalkit/metrics.py
rm 'evalkit/metrics.py'
$ git status --short
$ git merge --continue
[main 02943f6] Merge branch 'refactor/scorers'
$ git ls-files evalkit
evalkit/scoring.py
```

A rename against a deletion of the same file is reported under the new name:

```
$ git merge cleanup/drop-metrics
CONFLICT (rename/delete): evalkit/metrics.py renamed to evalkit/scoring.py in HEAD, but deleted in cleanup/drop-metrics.
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git status --short
UD evalkit/scoring.py
$ git ls-files -u
100644 9bbcd7ae8147833209c2dbce940d2fd9d681c8ac 1      evalkit/scoring.py
100644 9bbcd7ae8147833209c2dbce940d2fd9d681c8ac 2      evalkit/scoring.py
```

The dangerous case is a rename that Git does not recognize. Ravi moved `metrics.py` to `scoring.py` and rewrote most of it in the same commit. Only 39% of the content survived:

```
$ git diff --summary main...refactor/score-objects
delete mode 100644 evalkit/metrics.py
create mode 100644 evalkit/scoring.py
$ git diff --summary --find-renames=30% main...refactor/score-objects
rename evalkit/{metrics.py => scoring.py} (39%)
$ git merge refactor/score-objects
CONFLICT (modify/delete): evalkit/metrics.py deleted in refactor/score-objects and modified in HEAD. Version HEAD of
evalkit/metrics.py left in tree.
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
```

```
$ git status --short
UD evalkit/metrics.py
A evalkit/scoring.py
```

Observed behavior : CONFLICT (modify/delete) on evalkit/metrics.py, although nobody decided to delete the module. The new evalkit/scoring.py is staged without your edit.

Git state : Theirs has no metrics.py and a scoring.py that shares 39% of its content.

Mechanism : Rename detection pairs a deleted path with an added path only when their similarity reaches the threshold, 50% by default. Below that, Git sees a deletion and an addition.

Root cause : A move and a rewrite in one commit. The snapshot holds no rename, only two paths with different content.

Why Git does this : Trees store names and blob IDs. A rename is an inference from similarity, and every inference needs a cutoff.

Correct fix : Abort and merge again with `-X find-renames=30%`, or port your edit into the new file by hand and `git rm` the old path.

Prevention : Land the move as its own change and merge it into every open branch before the rewrite starts. Two commits on one branch are not enough for the merge, which compares only the tips, though they keep `git log --follow` working.

With a lower threshold the same merge becomes a content conflict inside the new file, which is the conflict you can reason about:

```
$ git merge --abort
$ git merge -X find-renames=30% refactor/score-objects
Auto-merging evalkit/scoring.py
CONFLICT (content): Merge conflict in evalkit/scoring.py
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git status --short
D evalkit/metrics.py
UU evalkit/scoring.py
$ grep -n -B1 -A5 '<<<<<<' evalkit/scoring.py
11-def exact_match(pred, gold):
12:<<<<<< HEAD:evalkit/metrics.py
13-     return float(pred.strip()) == gold.strip()
14-=====
15-     return Score("exact_match", float(pred == gold))
16->>>>>> refactor/score-objects:evalkit/scoring.py
17-
```

The labels now include paths, `HEAD:evalkit/metrics.py` against `refactor/score-objects:evalkit/scoring.py`, because the two sides know the file under different names.

Binary files. Git cannot merge them line by line. It reports a content conflict, records all three stages, leaves your version in the working tree and writes no markers:

```
$ git merge feature/new-palette
warning: Cannot merge binary files: reports/confusion.png (HEAD vs. feature/new-palette)
Auto-merging reports/confusion.png
CONFLICT (content): Merge conflict in reports/confusion.png
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git status --short
UU reports/confusion.png
$ git ls-files -u
100644 ba14cea3b6767bd483472d2b9ba04134aef2afae 1      reports/confusion.png
100644 32a12ecade27b0c495b09a683cd0c12510e81468 2      reports/confusion.png
100644 c4a999a147a8710d34a4971df06840aaca348ba5 3      reports/confusion.png
$ git diff
diff --cc reports/confusion.png
index 32a12ec,c4a999a..0000000
Binary files differ
```

```
# The working tree file is our version (stage 2), byte for byte:
$ git hash-object reports/confusion.png
32a12ecade27b0c495b09a683cd0c12510e81468
```

```
# Decision: regenerate later; for now take their file.
$ git restore --theirs reports/confusion.png
$ git hash-object reports/confusion.png
c4a999a147a8710d34a4971df06840aaca348ba5
$ git status --short
UU reports/confusion.png
$ git add reports/confusion.png
$ git status --short
M reports/confusion.png
$ git merge --continue
[main 4d7fbd3] Merge branch 'feature/new-palette'
```

For a generated artifact the honest resolution is to regenerate it from the merged sources. Text files that must never be merged line by line, such as lock files, can be given the same treatment with the `merge` attribute (`-merge` or `merge=binary` in `.gitattributes`; Chapter 14C).

File against directory. One branch adds a file named `docs`, the other a directory named `docs`. Both cannot exist, so Git moves the file aside under a name that says where it came from:

```
$ git merge feature/docs-directory
CONFLICT (file/directory): directory in the way of docs from HEAD; moving it to docs~HEAD instead.
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git status --short
D docs
A docs/calibration.md
AU docs~HEAD
$ git ls-files -s docs docs~HEAD
100644 37cc29f51e7d512294b921cadd0f36e408ede85a 0 docs/calibration.md
100644 7ef0c38ff113fe6b29f348c4cce89653ba08aac7 2 docs~HEAD
```

Directory renames. You moved the package under `src/`. Meanwhile Asha added a new file in the old directory. Git notices that the directory was renamed and asks:

```
$ git diff --name-status feature/cache...main
R100 evalkit/__init__.py src/evalkit/__init__.py
R100 evalkit/metrics.py src/evalkit/metrics.py
R100 evalkit/report.py src/evalkit/report.py
$ git diff --name-status main...feature/cache
A evalkit/cache.py
$ git merge feature/cache
CONFLICT (file location): evalkit/cache.py added in feature/cache inside a directory that was renamed in HEAD, suggesting it should perhaps be moved to src/evalkit/cache.py.
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git status --short
UA src/evalkit/cache.py
$ git ls-files -u
100644 7f040298ad3ac9af50f69563114d5f3f5fff3576 3 src/evalkit/cache.py
$ ls src/evalkit
__init__.py
cache.py
metrics.py
report.py
```

There are no markers and only one stage. The file is already at the suggested path, so accepting is one command:

```
# Git has already written the file where it suggests. Accepting the suggestion is one command:
$ git add src/evalkit/cache.py
$ git status --short
A src/evalkit/cache.py
$ git merge --continue
[main d1b5f4e] Merge branch 'feature/cache'
$ git ls-files src
src/evalkit/__init__.py
src/evalkit/cache.py
src/evalkit/metrics.py
src/evalkit/report.py
```

`merge.directoryRenames` controls this. Its default is `conflict`. The other two values act without asking:

```
# The same merge with merge.directoryRenames=true does not stop:
$ git -c merge.directoryRenames=true merge feature/cache
Path updated: evalkit/cache.py added in feature/cache inside a directory that was renamed in HEAD; moving it to
src/evalkit/cache.py.
Merge made by the 'ort' strategy.
 src/evalkit/cache.py | 5 ++++
 1 file changed, 5 insertions(+)
 create mode 100644 src/evalkit/cache.py
# With merge.directoryRenames=false the new file stays in the old directory, without a word:
$ git -c merge.directoryRenames=false merge feature/cache
Merge made by the 'ort' strategy.
 evalkit/cache.py | 5 ++++
 1 file changed, 5 insertions(+)
 create mode 100644 evalkit/cache.py
$ git ls-files evalkit src
evalkit/cache.py
src/evalkit/__init__.py
src/evalkit/metrics.py
src/evalkit/report.py
```

With `false` the merge is clean and wrong: `evalkit/cache.py` sits in a directory that no longer contains the package. The default costs one `git add`.

In production. Refactoring branches are where these types appear together. Announce large moves, land them quickly, and merge them into open feature branches the same day.

8.12 Controlling the result: `--ff-only`, `--no-ff`, `--no-commit`, `--squash`

In one sentence. Four options decide whether a merge creates a commit, what kind, and when; `merge.ff` can make `--no-ff` or `--ff-only` the default.

Option	When a fast-forward is possible	When the branches have diverged
<code>none (--ff)</code>	fast-forward	merge commit
<code>--no-ff</code>	merge commit	merge commit
<code>--ff-only</code>	fast-forward	refuses; nothing moves; exit status 128
<code>--no-commit</code>	fast-forward (the option has no effect)	stops before committing, as if a conflict had been resolved
<code>--squash</code>	stages the result; no commit; no <code>MERGE_HEAD</code>	the same

`--no-ff` forces a merge commit where a fast-forward would do. The branch stays visible in the graph and can be reverted as one unit:

```
# main is an ancestor of feature/rationale, so a plain merge would fast-forward.
$ git merge --no-ff feature/rationale
Merge made by the 'ort' strategy.
 prompts/judge.txt | 2 ++
 1 file changed, 2 insertions(+)
$ git log --oneline --graph
*   ec7d2b5 Merge branch 'feature/rationale'
| \
| * 58a5e60 Ask the judge to quote evidence
| * 5aec6e0 Ask the judge for a rationale
| /
* 6ae3c51 Add eval config, judge prompt and metrics
```

`--no-commit` is meant to let you inspect a merge before it is recorded. Alone, it does not stop a fast-forward. The manual says so directly: "there is no way to stop those merges with `--no-commit`".

```
$ git merge --no-commit feature/rationale
Updating 6ae3c51..58a5e60
Fast-forward
 prompts/judge.txt | 2 ++
 1 file changed, 2 insertions(+)
$ git log --oneline --graph
* 58a5e60 Ask the judge to quote evidence
* 5aec6e0 Ask the judge for a rationale
* 6ae3c51 Add eval config, judge prompt and metrics
```

Combined with `--no-ff` it stops reliably. The state is that of a merge whose conflicts are all resolved: `MERGE_HEAD` exists, `HEAD` has not moved, and `MERGE_MODE` remembers `no-ff`.

```
$ git merge --no-ff --no-commit feature/rationale
Automatic merge went well; stopped before committing as requested
$ git status
On branch main
All conflicts fixed but you are still merging.
 (use "git commit" to conclude merge)

Changes to be committed:
  modified:   prompts/judge.txt

$ ls .git | grep -E 'MERGE|ORIG_HEAD'
AUTO_MERGE
MERGE_HEAD
MERGE_MODE
MERGE_MSG
ORIG_HEAD
$ cat .git/MERGE_MODE; echo
no-ff
$ git log --oneline -1
6ae3c51 Add eval config, judge prompt and metrics
$ git merge --abort
```

`--ff-only` is the safe way to bring a branch up to date. Either it fast-forwards or it moves nothing:

```
$ git log --oneline --graph --all
* d80b525 Use temperature 0 for reproducible evals
| * 58a5e60 Ask the judge to quote evidence
| * 5aec6e0 Ask the judge for a rationale
| /
* 6ae3c51 Add eval config, judge prompt and metrics
$ git merge --ff-only feature/rationale
hint: Diverging branches can't be fast-forwarded, you need to either:
hint:
hint:   git merge --no-ff
hint:
hint: or:
hint:
hint:   git rebase
hint:
hint: Disable this message with "git config set advice.diverging false"
```

```
fatal: Not possible to fast-forward, aborting.
[exit status: 128]
$ git status --short --branch
## main
```

`--squash` computes the same result as a merge and then deliberately forgets that it was one:

```
$ git merge --squash feature/rationale
Automatic merge went well; stopped before committing as requested
Squash commit -- not updating HEAD
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   prompts/judge.txt

$ ls .git | grep -E 'MERGE|ORIG_HEAD|SQUASH'
AUTO_MERGE
ORIG_HEAD
SQUASH_MSG
$ head -3 .git/SQUASH_MSG
Squashed commit of the following:

commit 58a5e6004856ba05b88a2920c3684d15859a867b
```

The index and the working tree hold the merge result. HEAD has not moved. There is no `MERGE_HEAD`, so `git merge --abort` has nothing to abort (the way back is `git reset --merge`). `SQUASH_MSG` is a draft message. The commit you make next is an ordinary one:

```
$ git commit -q -m "Ask the judge for a rationale and for evidence"
$ git log --oneline --graph --all
* 4b0b1d4 Ask the judge for a rationale and for evidence
* d80b525 Use temperature 0 for reproducible evals
| * 58a5e60 Ask the judge to quote evidence
| * 5a6c6e0 Ask the judge for a rationale
|/
* 6ae3c51 Add eval config, judge prompt and metrics
$ git show -s --format="%h parents: %p" HEAD
4b0b1d4 parents: d80b525
$ git branch --no-merged
feature/rationale
$ git branch -d feature/rationale
error: the branch 'feature/rationale' is not fully merged
hint: If you are sure you want to delete it, run 'git branch -D feature/rationale'
hint: Disable this message with "git config set advice.forceDeleteBranch false"
[exit status: 1]
```

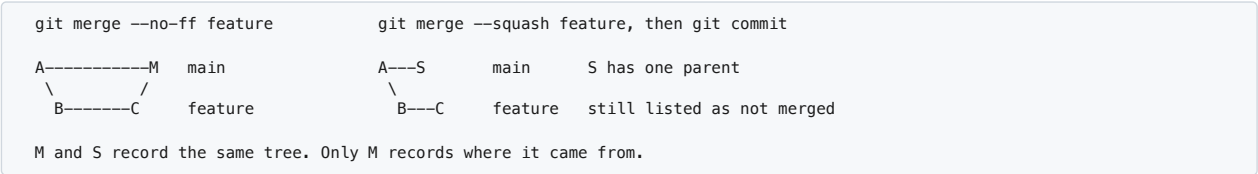
One parent. The content of the branch is in `main`; its commits are not, so `git branch -d` refuses to delete it here, where the branch has no upstream (with an upstream that still contains the branch, `-d` compares with that and deletes with a warning: Chapter 7, section 7.5). A squash merge transfers content without ancestry, the exact opposite of `-s ours`. That matters when the branch lives on:

```
# The squash commit recorded no second parent, so the merge base has not moved:
$ git log --oneline -1 $(git merge-base main feature/rationale)
6ae3c51 Add eval config, judge prompt and metrics
$ git merge --squash feature/rationale
Auto-merging prompts/judge.txt
CONFLICT (content): Merge conflict in prompts/judge.txt
Squash commit -- not updating HEAD
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ cat prompts/judge.txt
You are a strict grader.
Answer with PASS or FAIL.
<<<<<< HEAD
Explain your verdict in one sentence.
=====
Explain your verdict in two sentences.
>>>>>> feature/rationale
Quote the evidence you used.
```

Observed behavior	: Squashing the same branch into main a second time conflicts on a line that the first squash had already delivered.
Git state	: <code>git merge-base main feature/rationale</code> is still 6ae3c51, the original fork.
Mechanism	: The first squash commit has one parent, so no commit of the branch became an ancestor of main. The second merge compares both tips with the old base again: main "changed" the line (through the squash) and so did the branch.
Root cause	: Content was merged without ancestry, so the merge base never moved.
Why Git does this	: The merge base is computed from parent links, and a squash writes none.
Correct fix	: Resolve the conflict in favour of the newer text. Then stop reusing the branch.
Prevention	: One squash per branch: delete the branch afterwards and start the next one from main. For branches that must live on, use real merge commits.

Git's FAQ makes the same recommendation: "if you want to merge two long-lived branches repeatedly, it's best to always use a regular merge commit" ([gitfaq](#)).

Picture.



merge.ff sets the default: false behaves like --no-ff, only like --ff-only.

```
# docs/readme is one commit ahead of main. merge.ff=false behaves like --no-ff:
$ git -c merge.ff=false merge docs/readme
Merge made by the 'ort' strategy.
 README.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
# merge.ff=only behaves like --ff-only:
$ git -c merge.ff=only -c advice.diverging=false merge feature/rationale
fatal: Not possible to fast-forward, aborting.
[exit status: 128]
```

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
git merge --no-ff --no-commit <other> 🟡	files updated to the merge result	merge result staged	unchanged	unchanged	MERGE_HEAD , MERGE_MSG , MERGE_MODE (no-ff), AUTO_MERGE created; ORIG_HEAD set	unchanged	unchanged
git merge --squash <other> 🟡	files updated to the merge result (with markers on conflict)	merge result staged	unchanged	unchanged	SQUASH_MSG created; ORIG_HEAD set; no MERGE_HEAD	unchanged	unchanged
git merge --ff-only <other> on diverged branches 🟢	unchanged	unchanged	unchanged	unchanged	ORIG_HEAD rewritten with the current tip; nothing else	unchanged	unchanged

GitHub, not Git. The merge methods of a pull request map onto these options, but they run on GitHub's servers and are not identical to the local commands. The default method is documented as a merge "using the --no-ff option". For the squash method, GitHub's documentation carries the warning of the root-cause box above: if you keep working on the head branch, "later pull requests can include commits that were already squashed into the base branch" ([pull request merges](#)). Chapter 17 covers the merge methods.

In production. Habits that work: --ff-only to update your local main, so that a surprise divergence stops you; --no-ff to integrate a feature branch as one revertable unit; --squash when the branch history is noise, then delete the branch; --no-ff --no-commit to test the merged tree before it becomes a commit.

8.13 Anatomy of a merge commit and first-parent history

In one sentence. A merge commit is an ordinary commit object with more than one parent line, and the order of those lines records which branch received the merge.

Analogy. A river and a tributary. Downstream of the confluence there is one river, and the map still shows which channel was the main stream. The analogy breaks because in Git the "main stream" is not a property of a branch name: it is whichever commit was checked out when the merge was made.

Precisely. A merge commit stores a complete tree, like every commit. It stores no diff, no conflict record and no list of merged commits. Parent 1 is the commit HEAD pointed to when the merge was created. Parent 2 is the commit that was merged. M^1 and M^2 name them. M^1..M^2 is the set of commits that the merge brought in, and M^~ is shorthand for the same range plus M itself.

Inside .git. One commit object. No flag and no separate storage distinguishes a merge from any other commit.

See it.

```
$ git log --oneline --graph
* 630eac8 Raise batch size to 32
* 821b8ff Merge branch 'feature/rationale'
| \
| * 58a5e60 Ask the judge to quote evidence
| * 5aec6e0 Ask the judge for a rationale
* | c089834 Use temperature 0 for reproducible evals
|/
* 6ae3c51 Add eval config, judge prompt and metrics
$ git cat-file -p 821b8ff
tree 3e976d90175716430f0495d15e71b1a0069e93e1
parent c089834537b955876054c69c0f4bfbe6230b0cb2
```

```
parent 58a5e6004856ba05b88a2920c3684d15859a867b
author Lab User <you@example.com> 1788756120 +0530
committer Lab User <you@example.com> 1788756120 +0530

Merge branch 'feature/rationale'
```

```
$ git rev-parse --short 821b8ff^1
c089834
$ git rev-parse --short 821b8ff^2
58a5e60
# The commits that came in through the merge: reachable from parent 2, not from parent 1.
$ git log --oneline 821b8ff^1..821b8ff^2
58a5e60 Ask the judge to quote evidence
5aec6e0 Ask the judge for a rationale
# The same range plus the merge itself, with the ^- shorthand:
$ git log --oneline 821b8ff^-
821b8ff Merge branch 'feature/rationale'
58a5e60 Ask the judge to quote evidence
5aec6e0 Ask the judge for a rationale
```

`git log --first-parent` follows only parent 1 at every merge. On a main branch that receives work through merges, this is the list of integrations, one line each:

```
$ git log --oneline --first-parent
630eac8 Raise batch size to 32
821b8ff Merge branch 'feature/rationale'
c089834 Use temperature 0 for reproducible evals
6ae3c51 Add eval config, judge prompt and metrics
$ git log --oneline --merges
821b8ff Merge branch 'feature/rationale'
$ git log --oneline --no-merges
630eac8 Raise batch size to 32
c089834 Use temperature 0 for reproducible evals
58a5e60 Ask the judge to quote evidence
5aec6e0 Ask the judge for a rationale
6ae3c51 Add eval config, judge prompt and metrics
```

A merge commit has one diff per parent, and they answer different questions:

```
# What main gained from the merge:
$ git diff --stat 821b8ff^1 821b8ff
prompts/judge.txt | 2 ++
1 file changed, 2 insertions(+)
# What the feature branch would have gained from it:
$ git diff --stat 821b8ff^2 821b8ff
config/eval.yaml | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```

Picture.

```
      5aec6e0---58a5e60      feature/rationale
      /               \
6ae3c51---c089834-----821b8ff---630eac8  main  (HEAD -> main)

821b8ff^1 = c089834   where main was when the merge was made
821b8ff^2 = 58a5e60   what was merged
git log --first-parent main: 630eac8, 821b8ff, c089834, 6ae3c51
```

The first-parent flip. "First parent" means "what was checked out", not "main". Watch what happens when someone merges `main` into a feature branch and then fast-forwards `main` to it:

```
$ git log --oneline --first-parent -3 main
cfc99c8 Retry the judge three times
630eac8 Raise batch size to 32
821b8ff Merge branch 'feature/rationale'
$ git switch -q feature/httpx
$ git merge main
Merge made by the 'ort' strategy.
config/eval.yaml | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git switch -q main
$ git merge feature/httpx
Updating cfc99c8..3f3fd07
Fast-forward
 requirements.txt | 1 +
1 file changed, 1 insertion(+)
create mode 100644 requirements.txt
$ git log --oneline --graph -4
* 3f3fd07 Merge branch 'main' into feature/httpx
| \
| * cfc99c8 Retry the judge three times
* | 042925e Add httpx for the judge client
|/
* 630eac8 Raise batch size to 32
$ git log --oneline --first-parent -3 main
```

```
3f3fd07 Merge branch 'main' into feature/httpx
042925e Add httpx for the judge client
630eac8 Raise batch size to 32
```

Before, the first-parent history of `main` contained "Retry the judge three times". Afterwards that commit is on the second-parent side and Ravi's branch commit sits on the mainline. Nothing is lost, but every tool that reads the mainline, `git log --first-parent`, `git bisect start --first-parent`, a changelog generator, now walks through the feature branch. The prevention is the pair of rules that most teams enforce anyway: integrate into `main` with `--no-ff` merges (GitHub's default merge method does this), and never fast-forward `main` to a branch that has merged `main`.

In production. On a team that merges pull requests, `git log --first-parent --oneline main` is the release history: one line per integrated unit, in the order it landed. When a deployment breaks, each line is a candidate for `git revert -m 1` (section 8.18).

8.14 Octopus merges

In one sentence. Naming several branches in one `git merge` creates a single commit with three or more parents, made by a separate strategy that refuses any conflict.

See it.

```
$ git merge topic/prompt topic/deps topic/docs
Trying simple merge with topic/prompt
Trying simple merge with topic/deps
Trying simple merge with topic/docs
Merge made by the 'octopus' strategy.
 README.md      | 2 ++
 prompts/judge.txt | 1 +
 requirements.txt | 1 +
 3 files changed, 4 insertions(+)
```

```
$ git log --oneline --graph
*---. 6cd1b13 Merge branches 'topic/prompt', 'topic/deps' and 'topic/docs'
| \ \ \
| | | * 436e24c Document how to run
| | * | 6e9e321 Add httpx for the judge client
| | | /
| * / f18e761 Ask the judge for a rationale
| | /
| / 5ab7b4d Use temperature 0 for reproducible evals
| /
* adad948 Add README and requirements
* 6ae3c51 Add eval config, judge prompt and metrics
$ git cat-file -p HEAD
tree 25553fcd3840e13707bf49acc52fbfc81ef0fba
parent 5ab7b4d031080365343d89dcd529d514e9f9b01c
parent f18e7614e71850ee7b09c35e79b2e1a668d5f925
parent 6e9e3211e05899f0aa9598b4d578584f9189d724
parent 436e24c23d49f55d3231c8e61c555e6945411e92
author Lab User <you@example.com> 1788756420 +0530
committer Lab User <you@example.com> 1788756420 +0530

Merge branches 'topic/prompt', 'topic/deps' and 'topic/docs'
$ git reflog -1
6cd1b13 HEAD@{0}: merge topic/prompt topic/deps topic/docs: Merge made by the 'octopus' strategy.
```

Four parents: your tip first, then the three branches in the order you named them. Now with a branch that touches the same line as `main`:

```
# topic/temperature changes the same line as main.
$ git merge topic/prompt topic/temperature topic/docs
Trying simple merge with topic/prompt
Trying simple merge with topic/temperature
Simple merge did not work, trying automatic merge.
Auto-merging config/eval.yaml
ERROR: content conflict in config/eval.yaml
fatal: merge program failed
Automated merge did not work.
Should not be doing an octopus.
Merge with strategy octopus failed.
[exit status: 2]
$ git status --short --branch
## main
$ ls .git | grep -E 'MERGE|ORIG_HEAD'
ORIG_HEAD
```

Exit status 2, no `MERGE_HEAD`, a clean status. An octopus merge that hits a conflict is rolled back, not paused. Merge the conflicting branch separately, resolve it, and bundle the rest.

In production. The octopus fits one job: bundling independent, conflict-free topic branches, such as documentation and dependency bumps, into an integration branch. It has costs: the audit command of section 8.16 skips it, and `git revert -m` has to choose among more than two parents. Most application teams never need it.

8.15 Clean for Git, wrong for humans

In one sentence. A merge without conflicts means that no two text changes overlapped; it says nothing about whether the result builds, passes its tests or makes sense.

Analogy. A spell-checker passes two paragraphs that contradict each other, because every word is spelled correctly. The analogy breaks only in degree: Git checks even less, one property, the overlap of changed lines.

Semantic conflicts. Ravi renames `accuracy()` to `mean_score()` and updates the only caller he can see. Asha, from the same starting commit, adds a new module that calls `accuracy()`. A shell script stands in for the test suite: it fails when a module imports a name that `metrics.py` does not define.

```
$ git log --oneline --graph --all
* 5008877 Add a leaderboard ranked by accuracy
| * d1c3cff Rename accuracy() to mean_score()
|/
* 68c652c Add the report module and the import check
* 6ae3c51 Add eval config, judge prompt and metrics
$ git merge -q refactor/mean-score
$ git merge feature/leaderboard
Merge made by the 'ort' strategy.
evalkit/leaderboard.py | 5 +++++
1 file changed, 5 insertions(+)
create mode 100644 evalkit/leaderboard.py
```

```
$ sh ci/check_imports.sh
FAIL evalkit/leaderboard.py imports accuracy, which evalkit/metrics.py does not define
[exit status: 1]
```

```
# The same check on each parent of the merge commit:
$ git switch -q --detach main^1 && sh ci/check_imports.sh
OK every import from evalkit.metrics resolves
$ git switch -q --detach main^2 && sh ci/check_imports.sh
OK every import from evalkit.metrics resolves
$ git switch -q main
```

Both parents pass. The merge fails. The reason is visible when you list what each side changed:

```
# Paths each side changed since the merge base:
$ git diff --name-only main^2...main^1
evalkit/metrics.py
evalkit/report.py
$ git diff --name-only main^1...main^2
evalkit/leaderboard.py
```

Observed behavior	: Two green branches, a merge without conflict, a red merge commit.
Git state	: The sides changed disjoint paths: <code>metrics.py</code> and <code>report.py</code> on one, <code>leaderboard.py</code> on the other.
Mechanism	: By the rule table, a path changed on one side only is taken as it is. No file-level merge ran. There was nothing for Git to compare.
Root cause	: The dependency between the two changes, a renamed function and a new caller of the old name, lives in the Python import system, not in the text of any one file.
Why Git does this	: Git merges snapshots of text. It has no parser, no type checker and no test runner, for any language.
Correct fix	: Update the caller, in a commit on top of the merge (below) or inside the merge commit (see "Evil merges").
Prevention	: Test the merge result before it reaches main: CI on the pull request's merge commit, plus a rule that the branch is up to date with main, or a merge queue (Chapter 18).

```
# The repair is a normal, reviewable commit on top of the merge.
$ git diff --stat
evalkit/leaderboard.py | 4 ++--
1 file changed, 2 insertions(+), 2 deletions(-)
$ git commit -q -a -m "Use mean_score() in the leaderboard"
$ sh ci/check_imports.sh
OK every import from evalkit.metrics resolves
```

The quiet variant is worse than the loud one. If one branch renames a configuration key and another adds code that reads the old key with a default, nothing fails. An evaluation run uses the default, the scores shift, and the merge that caused it shows no conflict and no red check. Lab 6.5 is a third form: two migrations with the same number.

Changed on both sides, reverted on one. A fix is applied to `main` and to a branch. `main` then reverts it, for a good reason. The branch is merged:

```
$ git log --oneline --graph --all
* a2a0277 Revert "Allow long answers: max_tokens 2048"
* 523ed40 Allow long answers: max_tokens 2048
| * 4fac9ee Ask for full sentences
```

```
| * c8e320d Allow long answers: max_tokens 2048
|/
* 6ae3c51 Add eval config, judge prompt and metrics
$ grep max_tokens config/eval.yaml
max_tokens: 512
```

```
$ git merge feature/long-answers
Merge made by the 'ort' strategy.
  config/eval.yaml | 2 +-
  prompts/judge.txt | 1 +
  2 files changed, 2 insertions(+), 1 deletion(-)
$ grep max_tokens config/eval.yaml
max_tokens: 2048
```

The reverted change is back, without a conflict and without a message. The three inputs explain it:

```
# base, ours, theirs, result:
$ git show $(git merge-base HEAD^1 HEAD^2):config/eval.yaml | grep max_tokens
max_tokens: 512
$ git show HEAD^1:config/eval.yaml | grep max_tokens
max_tokens: 512
$ git show HEAD^2:config/eval.yaml | grep max_tokens
max_tokens: 2048
$ git show HEAD:config/eval.yaml | grep max_tokens
max_tokens: 2048
```

Base 512, ours 512, theirs 2048: the row "only theirs changed it". The apply-then-revert pair on `main` cancels out, and the merge never sees it. The manual describes this behavior and admits that "some people find this behavior confusing". If the revert must win, say so in the tree that the merge will see: revert the change on the branch too before merging.

Evil merges. The glossary definition: "An evil merge is a merge that introduces changes that do not appear in any parent." It happens through a resolution that also changes something outside the conflict, an edit between `git merge --no-commit` and the commit, or `git commit --amend` on a merge. The name is harsher than the practice. Fixing a semantic conflict inside the merge commit is such a change, and it keeps every mainline commit green, which `git bisect` needs. The cost is visibility: `git log -p` shows no diff for a merge. The manual asks you to "refrain from abusing this option to sneak substantial changes into a merge commit". A defensible team rule: anything beyond resolving the conflict goes into a normal commit, or is named in the merge message.

8.16 Auditing a merge

In one sentence. A merge commit stores a tree and no diff, so "what did this merge do" has to be computed, and Git offers three computations that answer three different questions.

View	Command	Shows	Blind to
default log	<code>git log -p</code>	no diff at all for merge commits	everything a merge did
first-parent diff	<code>git show --first-parent M, git diff M^1 M</code>	everything the merge brought to the mainline	which part is branch content and which is hand-made
combined diff	<code>git show M</code> (the <code>--cc</code> format)	lines that differ from every parent	resolutions that took one side unchanged
remerge diff	<code>git show --remerge-diff M, git log --merges --remerge-diff</code>	the difference between a mechanical re-merge and the recorded result	octopus merges, which it skips

See it. A history with three merges: one conflict resolved by keeping our temperature, one clean merge that also set `retries` to 0, a line that neither branch touched, and one honest clean merge.

```
$ git log --oneline --first-parent -5
9a6385a Merge branch 'topic/max-tokens'
fc83756 Merge branch 'topic/docs'
8353ff1 Add requirements
70bae5f Merge branch 'feature/creative-judge'
5397d5f Use temperature 0 for reproducible evals
# By default git log shows no diff for a merge commit, with -p or with --stat:
$ git log -1 -p --format='%h %s' fc83756
fc83756 Merge branch 'topic/docs'
$ git log -1 --stat --format='%h %s' fc83756
fc83756 Merge branch 'topic/docs'
```

```
# git show uses the dense combined format (--cc) for merges.
$ git show --format='%h %s' 70bae5f
70bae5f Merge branch 'feature/creative-judge'

$ git show --format='%h %s' fc83756
fc83756 Merge branch 'topic/docs'

diff --cc config/eval.yaml
index 1642b99,1642b99..29fd375
--- a/config/eval.yaml
```

```

+++ b/config/eval.yaml
@@ -3,5 -3,5 +3,5 @@ temperature: 0.
  max_tokens: 512
  batch_size: 16
  timeout_s: 30
--retries: 2
++retries: 0
seed: 1234

```

`git show` prints nothing for the first merge, the one with the hand-resolved conflict. The dense combined format omits hunks where the result equals one of the parents, and that resolution took our side exactly. For the second merge it shows the smuggled line, with `--` and `++` because the result differs from both parents. The remerge diff shows both:

```

$ git log --merges --remerge-diff --format='%h %s'
== 9a6385a Merge branch 'topic/max-tokens'
== fc83756 Merge branch 'topic/docs'

diff --git a/config/eval.yaml b/config/eval.yaml
index 1642b99..29fd375 100644
--- a/config/eval.yaml
+++ b/config/eval.yaml
@@ -3,5 +3,5 @@ temperature: 0.0
  max_tokens: 512
  batch_size: 16
  timeout_s: 30
-retries: 2
+retries: 0
seed: 1234
== 70bae5f Merge branch 'feature/creative-judge'

diff --git a/config/eval.yaml b/config/eval.yaml
remerge CONFLICT (content): Merge conflict in config/eval.yaml
index 2c6727a..1642b99 100644
--- a/config/eval.yaml
+++ b/config/eval.yaml
@@ -1,9 +1,5 @@
model: judge-large-v2
-<<<<<<< 5397d5f (Use temperature 0 for reproducible evals)
temperature: 0.0
=====
-temperature: 0.7
->>>>>> 45a7a67 (Ask the judge for a rationale)
max_tokens: 512
batch_size: 16
timeout_s: 30

```

Read it as "what a human did after Git stopped". Git repeats each merge into a temporary tree, conflict markers included, and diffs that tree against the recorded one. No output means that the merge is exactly what Git computes on its own (9a6385a). Removed marker lines with one candidate kept are a resolution (70bae5f). A change with no markers nearby came from neither branch (fc83756). The first-parent view answers the other question, "what did `main` receive":

```

# Everything a merge brought to main, as one ordinary diff against its first parent:
$ git show --first-parent --format='%h %s' fc83756
fc83756 Merge branch 'topic/docs'

diff --git a/README.md b/README.md
new file mode 100644
index 0000000..e8fd9b1
--- /dev/null
+++ b/README.md
@@ -0,0 +1 @@
+# evalkit
diff --git a/config/eval.yaml b/config/eval.yaml
index 1642b99..29fd375 100644
--- a/config/eval.yaml
+++ b/config/eval.yaml
@@ -3,5 +3,5 @@ temperature: 0.0
  max_tokens: 512
  batch_size: 16
  timeout_s: 30
-retries: 2
+retries: 0
seed: 1234

```

The remerge diff repeats the merge with default settings, so a merge made with `-X ours` or `-s ours` shows up as a diff, however clean it looked when it was made. Its limits: Git 2.36 or later, two-parent merges only, and an output format that the manual declares "subject to change", so read it and do not parse it.

In production. This answers the second question of section 8.1. `git log --merges --remerge-diff --first-parent <last-release>..main` lists every hand-made change in every merge since the last release. Lab 6.6 uses it to find a merge where a whole-file `--theirs` deleted a teammate's retry setting.

8.17 `git merge-tree`: a merge without a working tree

In one sentence. `git merge-tree --write-tree` performs a real merge with the same strategy as `git merge`, using only the object database: no index, no working tree and no ref update.

Precisely. The command prints the ID of the resulting tree. Its exit status is 0 for a clean merge and 1 for a conflict. On a conflict it also prints the stage entries that `git merge` would have put into the index, and the messages. It creates objects and changes nothing else, so it works in a bare repository, where `git merge` cannot run:

```
$ git clone -q --bare evalkit server.git
$ cd server.git
$ git rev-parse --is-bare-repository
true
$ git log --oneline --graph --all
* 7c974a2 Add a README
| * 5397d5f Use temperature 0 for reproducible evals
|/
| * 45a7a67 Ask the judge for a rationale
| * 640bfe1 Raise temperature and reseed for judge diversity
|/
* 6ae3c51 Add eval config, judge prompt and metrics
$ git merge docs/readme
fatal: this operation must be run in a work tree
[exit status: 128]

$ git merge-tree --write-tree main docs/readme
0c05f61e275e97d119913a164e7312d48cd50d66
[exit status: 0]
$ tree=$(git merge-tree --write-tree main docs/readme)
$ git ls-tree -r --abbrev=7 $tree
100644 blob af7f9b5 README.md
100644 blob 0e8b97c config/eval.yaml
100644 blob 9bbcd7a evalkit/metrics.py
100644 blob cfe0ef0 prompts/judge.txt
```

A tree is not a commit. Two more plumbing commands turn it into a merge and publish it:

```
$ commit=$(git commit-tree $tree -p main -p docs/readme -m "Merge branch 'docs/readme'")
$ git update-ref refs/heads/main $commit
$ git log --oneline --graph -4 main
* 40fb5aa Merge branch 'docs/readme'
|\
| * 7c974a2 Add a README
| * | 5397d5f Use temperature 0 for reproducible evals
|/
* 6ae3c51 Add eval config, judge prompt and metrics
```

A conflicting pair of branches:

```
$ git merge-tree --write-tree main feature/creative-judge
9615415b8675a4fb67a61eb8a37e816decd78066
100644 c5b33275d88c041381cff59f23d3cd88b9f22c98 1 config/eval.yaml
100644 0e8b97cd35b023b2949d7efeed9dbd491d065ae8 2 config/eval.yaml
100644 db5c556ef5edb6802e492cd59b5a0b9721a882da 3 config/eval.yaml

Auto-merging config/eval.yaml
CONFLICT (content): Merge conflict in config/eval.yaml
[exit status: 1]
```

The tree is still written. It contains the file with markers, labelled with the names from the command line:

```
$ tree=$(git merge-tree --write-tree main feature/creative-judge | head -1)
$ git show $tree:config/eval.yaml | head -6
model: judge-large-v2
<<<<<< main
temperature: 0.0
=====
temperature: 0.7
>>>>>> feature/creative-judge
```

```
# A yes/no answer for a "can this be merged?" indicator:
$ git merge-tree --quiet main feature/creative-judge
[exit status: 1]
$ git merge-tree --write-tree --name-only --no-messages main feature/creative-judge
9615415b8675a4fb67a61eb8a37e816decd78066
config/eval.yaml
[exit status: 1]
```

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
git merge-tree --write-tree A B 	unchanged	unchanged	unchanged	unchanged	new tree and blob objects only (<code>--quiet</code> avoids writing most of them)	unchanged	unchanged

GitHub, not Git. A server has no working tree to merge in. GitHub's engineering blog lists "It can't check out the repository" among its requirements for a merge implementation, and its changelog states: "When GitHub creates merge commits, like to test whether a pull request can be merged cleanly or to actually merge a pull request, it now uses the `merge-ort` strategy" ([blog, 2023](#); [changelog, 2022](#)). Neither page says which command GitHub runs; `git merge-tree` is how stock Git exposes the same strategy without a working tree. This is described from GitHub's publications, not run here. One documented consequence: "Workflows will not run on `pull_request` activity if the pull request has a merge conflict" ([events that trigger workflows](#)).

In production. `git merge-tree --write-tree --name-only main feature/x` is a dry run that cannot disturb your working tree. Use it before a risky merge, or in a script that reports which branches conflict with `main`. Decide by the exit status, as the manual warns, not by whether the list of conflicted files is empty.

8.18 Pointers: rerere, reverting a merge, `git add --resolved`

rerere. "Reuse recorded resolution" remembers how you resolved a conflict and replays the resolution when the identical conflict appears again. It is off until you set `rerere.enabled`. It matters for long-lived branches, criss-cross histories and aborted merges. Chapter 14C covers it.

Reverting a merge. `git revert` refuses a merge commit until you say which parent is the mainline:

```
$ git log --oneline --graph
* 821b8ff Merge branch 'feature/rationale'
| \
| * 58a5e60 Ask the judge to quote evidence
| * 5aec6e0 Ask the judge for a rationale
* | c089834 Use temperature 0 for reproducible evals
|/
* 6ae3c51 Add eval config, judge prompt and metrics
$ git revert --no-edit HEAD
error: commit 821b8ff2e4e35c648578cd9e11514117ae0fd1b9 is a merge but no -m option was given.
fatal: revert failed
[exit status: 128]
$ git revert --no-edit -m 1 HEAD
[main 504ae98] Revert "Merge branch 'feature/rationale'"
Date: Mon Sep 7 10:15:00 2026 +0530
1 file changed, 2 deletions(-)
$ cat prompts/judge.txt
You are a strict grader.
Answer with PASS or FAIL.
```

`-m 1` creates a commit that undoes the difference between the merge and its first parent. It undoes content. It cannot undo ancestry: the merge commit is still in the history and the branch still counts as merged. Asha adds one more commit and the branch is merged again:

```
$ git log --oneline main..feature/rationale
b752dc4 Give the judge 60 seconds for longer answers
$ git merge feature/rationale
Auto-merging config/eval.yaml
Merge made by the 'ort' strategy.
 config/eval.yaml | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
# The timeout arrived. The two prompt lines of the first merge did not come back:
$ grep timeout_s config/eval.yaml
timeout_s: 60
$ cat prompts/judge.txt
You are a strict grader.
Answer with PASS or FAIL.
$ git branch --merged
 feature/rationale
* main
```

Only the new commit arrived. The two prompt lines from the first merge are missing, because their commits are ancestors of `main` already and the merge base is past them. The manual states the rule: "Reverting a merge commit declares that you will never want the tree changes brought in by the merge. As a result, later merges will only bring in tree changes introduced by commits that are not ancestors of the previously reverted merge" ([git-revert](#)). That is the third question of section 8.1. The repair, reverting the revert before merging again, is in Chapter 11.

`git add --resolved`. Added in Git 2.56 (not run here). According to the 2.56 documentation it updates the index "for unmerged paths matching `<paths>` where no conflict markers remain in the working tree", and "any path with leftover conflict markers causes the command to refuse to stage any files" ([git-add at 2.56](#)). The release notes add that it leaves "unrelated local changes unstaged" ([release notes](#)). It closes both traps of the next section.

8.19 What can go wrong

Before the merge starts. `git merge` protects uncommitted work by refusing to start. The manual gives two rules: it stops when local changes "overlap with files that `git pull`/`git merge` may need to update", and it aborts "if there are any changes registered in the index relative to the

HEAD commit". Under the second rule a staged change blocks the merge even in a file that the merge does not touch, and the message does not say why:

```
# A staged change in a file the merge does not even touch:
$ echo "# scoring helpers" >> evalkit/metrics.py
$ git add evalkit/metrics.py
$ git merge feature/creative-judge
error: Your local changes to the following files would be overwritten by merge:
    evalkit/metrics.py
Merge with strategy ort failed.
[exit status: 2]
```

An untracked file that the merge would create is protected in the same way:

```
# The branch adds NOTES.md. You have an untracked file with the same name:
$ echo "my scratch notes" > NOTES.md
$ git merge feature/creative-judge
error: The following untracked working tree files would be overwritten by merge:
    NOTES.md
Please move or remove them before you merge.
Aborting
Merge with strategy ort failed.
[exit status: 2]
```

Uncommitted work in paths that the merge does not touch is allowed, and it survives both the merge and an abort. The transcript uses an untracked file; an unstaged edit to a tracked file behaved the same in the sandbox:

```
# An uncommitted edit in a file the merge does not touch survives merge and abort.
$ echo "# evalkit" > NOTES.md
$ git merge feature/creative-judge
Auto-merging config/eval.yaml
CONFLICT (content): Merge conflict in config/eval.yaml
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git status --short
UU config/eval.yaml
M  prompts/judge.txt
?? NOTES.md
$ git merge --abort
$ git status --short
?? NOTES.md
```

`--autostash`. An unstaged edit in a file that the merge does touch stops the merge, unless you let Git stash it (`merge.autoStash` makes that the default):

```
# An uncommitted edit in a file the merge DOES touch stops the merge before it starts...
$ echo "Be concise." >> prompts/judge.txt
$ git merge feature/creative-judge
error: Your local changes to the following files would be overwritten by merge:
    prompts/judge.txt
Please commit your changes or stash them before you merge.
Aborting
Merge with strategy ort failed.
[exit status: 2]
# ...unless Git stashes it for you.
$ git merge --autostash feature/creative-judge
Created autostash: 02ed132
Auto-merging config/eval.yaml
CONFLICT (content): Merge conflict in config/eval.yaml
Automatic merge failed; fix conflicts and then commit the result.
When finished, apply stashed changes with `git stash pop`
[exit status: 1]
$ ls .git | grep -E 'MERGE|ORIG_HEAD'
AUTO_MERGE
MERGE_AUTOSTASH
MERGE_HEAD
MERGE_MODE
MERGE_MSG
ORIG_HEAD
$ git merge --abort
Applied autostash.
$ git status --short
M  prompts/judge.txt
```

The hint printed after a conflict deserves a closer look:

```
# An uncommitted edit in a file that the merge also changes (on another line):
$ sed -e 's/strict grader/strict but fair grader/' prompts/judge.txt > j && mv j prompts/judge.txt
$ git status --short
M  prompts/judge.txt
$ git merge --autostash feature/creative-judge
Created autostash: 1b5e0ea
Auto-merging config/eval.yaml
CONFLICT (content): Merge conflict in config/eval.yaml
```

```
Automatic merge failed; fix conflicts and then commit the result.
When finished, apply stashed changes with `git stash pop`
[exit status: 1]
$ git status --short
UU config/eval.yaml
M  prompts/judge.txt
# The hint says "git stash pop". The stash list is empty:
$ git stash list
$ git stash pop
No stash entries found.
[exit status: 1]
$ git log -1 --format="%h %s" MERGE_AUTOSTASH
1b5e0ea 0n main: autostash
```

```
# Resolve the conflict as in section 8.10, then conclude the merge:
$ git add config/eval.yaml
$ git merge --continue
[main 29f236b] Merge branch 'feature/creative-judge'
Applied autostash.
$ git status --short
M  prompts/judge.txt
$ cat prompts/judge.txt
You are a strict but fair grader.
Answer with PASS or FAIL.
Explain your verdict in one sentence.
$ git stash list
```

Root cause. The autostash is a stash-shaped commit that `.git/MERGE_AUTOSTASH` points to, not an entry in the stash list, so `git stash pop` finds nothing. `git merge --continue` and `git merge --abort` apply it for you. It reaches the stash list after `git merge --quit` or `git reset --merge` (per the manual), or when applying it conflicts (seen in the sandbox). Only then is `git stash pop` the right command.

During the merge. `git add` records whatever is in the file. A second merge and a partial commit are refused while one is open:

```
$ git merge feature/creative-judge
Auto-merging config/eval.yaml
CONFLICT (content): Merge conflict in config/eval.yaml
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
# Nothing was edited. git add accepts the file anyway:
$ git add config/eval.yaml
$ git status --short
M  config/eval.yaml
M  prompts/judge.txt
$ git diff --cached --check
config/eval.yaml:2: leftover conflict marker
config/eval.yaml:4: leftover conflict marker
config/eval.yaml:6: leftover conflict marker
[exit status: 2]
```

```
$ git merge feature/creative-judge
fatal: You have not concluded your merge (MERGE_HEAD exists).
Please, commit your changes before you merge.
[exit status: 128]
$ git commit -m "Resolve config" config/eval.yaml
fatal: cannot do a partial commit during a merge.
[exit status: 128]
```

The abort that loses work. `git merge --abort` is `git reset --merge`. It resets every staged path, and it cannot tell a merge result from an edit of yours that predates the merge and that you staged meanwhile, for example with the `git add -A` reflex. The manual warns that abort "will in some cases be unable to reconstruct the original (pre-merge) changes". Lab 6.7 reproduces the loss and recovers the edit from an object that few people know exists: before a true merge starts on a tree with local changes, Git 2.55 runs `git stash create` as a safety copy ([builtin/merge.c](#)). No ref points to that commit, so it lasts only until unreachable objects are pruned (Chapter 13). `git fsck` lists it as a dangling commit.

Symptom	Diagnosis	Fix	Prevention
error: Your local changes ... would be overwritten by merge	<code>git status</code> : an unstaged edit in a file the merge updates, or any staged change	commit or stash, then merge; or <code>git merge --autostash</code>	merge from a clean tree
Conflict markers in a commit; a syntax error at <<<<<<<	<code>git grep -n -E</code> '^(<<<<<<< ===== >>>>>>>)'	remove them in a new commit	<code>git diff --cached --check</code> before concluding; the same check in CI
A teammate's change vanished in a merge	<code>git show --remerge-diff <merge>; git log --full-history -- <path></code>	restore the lines in a new commit that says why	no whole-file <code>--ours</code> or <code>--theirs</code> unless the whole file is the decision
Merge was clean, build is red	run the check on <code>M^1</code> and on <code>M^2</code> : both pass	fix forward, or <code>git revert -m 1 M</code>	CI on the merge result (section 8.15)

Symptom	Diagnosis	Fix	Prevention
The same conflict returns at every merge	<code>git merge-base --all</code> prints two commits, or the branch was squash-merged before	resolve once more; enable <code>rerere</code>	one-directional merges; delete squashed branches
<code>fatal: There is no merge to abort (MERGE_HEAD missing).</code> with unmerged paths in <code>git status</code>	a <code>--squash</code> merge, or <code>--quit</code> was used	<code>git reset --merge</code>	check the state first: <code>ls .git grep MERGE</code>
An uncommitted edit is gone after <code>git merge --abort</code>	it was staged during the merge	<code>git fsck</code> , then restore from the "WIP on" commit (Lab 6.7)	commit or stash first; stage paths by name
Branch counts as merged, its changes are missing	a reverted merge, <code>-s ours</code> , or a whitespace option: <code>git show --remerge-diff <merge></code>	revert the revert (Chapter 11), or reapply	read the merge before trusting <code>git branch --merged</code>

8.20 When not to use it, and dangerous edge cases

When not to merge. Do not merge to silence a conflict: `-X ours`, `-s ours` and whole-file checkouts make the conflict disappear by deleting one side's answer. Do not merge `main` into a short-lived feature branch by habit: each merge adds a commit that explains nothing and sets up the first-parent flip of section 8.13; for a private branch, a rebase is often cleaner (Chapter 9). Do not squash-merge a branch that will keep living. Do not start a merge on top of uncommitted work that you cannot afford to lose.

Unrelated histories. Two commits with no common ancestor have no merge base, and Git refuses:

```
$ git merge-base main imported/prompt-library
[exit status: 1]
$ git merge imported/prompt-library
fatal: refusing to merge unrelated histories
[exit status: 128]
```

The usual cause is a mistake, such as a second `git init` whose history is then pulled into an existing repository. `--allow-unrelated-histories` exists for the rare intended case, joining two projects.

Annotated tags. Merging a tag can create a merge commit where you expected a fast-forward. Here it does not:

```
# v0.1 is an annotated tag, one commit ahead of main, stored as refs/tags/v0.1:
$ git cat-file -t v0.1
tag
$ git merge v0.1
Updating 5397d5f..2f528c3
Fast-forward
 README.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

The "MERGING TAG" section of the 2.55 manual says that Git "always creates a merge commit" for an annotated tag. The description of `--ff` on the same page is the accurate one: the merge commit is forced only for a tag "that is not stored in its natural place in the `refs/tags/` hierarchy". In the sandbox, the same tag object stored under another name produced a commit titled "Merge tag" that carried the tag message. `git merge --ff-only <tag>` never creates one.

Legacy strategies. `resolve` and `octopus` are shell scripts from 2005 that still ship with Git. On a criss-cross built like the one in section 8.5, `-s resolve` does this:

```
$ git merge-base --all main release/1.0
f66460bde7a1c01671e6268b8f4209a580951592
5397d5f953dda5e3acd0d0970078420baf6e0488
$ git merge -s resolve release/1.0
Trying simple merge.
Simple merge failed, trying Automatic merge.
Added config/eval.yaml in both, but differently.
fatal: unable to read blob object e69de29bb2d1d6434b8b29ae775ad8c2e48c5391
error: Could not stat : No such file or directory
ERROR: content conflict in config/eval.yaml
fatal: merge program failed
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git status --short
AA config/eval.yaml
$ git ls-files -u
100644 0e8b97cd35b023b2949d7efeed9dbd491d065ae8 2      config/eval.yaml
100644 3e21100524f1361d04738d41f1012b9221082155 3      config/eval.yaml
# The path is unmerged, and the file contains no conflict marker:
$ grep -n -e "<<<<<<<" -e temperature -e ">>>>>>>" config/eval.yaml
2:temperature: 0.0
$ git merge --abort
```

An internal error, and a state that is worse than a conflict: the path is unmerged, there is no stage 1, and the working tree file contains our version with no markers. The script behind it is in Git's exec path:


```
# The strategies that exist as separate programs (ort is built into git merge itself):
$ ls "$(git --exec-path)" | grep -E "^git-merge-(octopus|ours|recursive|resolve|subtree)$"
git-merge-octopus
git-merge-ours
git-merge-recursive
git-merge-resolve
git-merge-subtree
$ head -6 "$(git --exec-path)/git-merge-resolve"
#!/bin/sh
#
# Copyright (c) 2005 Linus Torvalds
# Copyright (c) 2005 Junio C Hamano
#
# Resolve two trees, using enhanced multi-base read-tree.
```

There is no reason to pass `-s resolve` or `-s recursive` today.

Ignored files. A merge silently overwrites an ignored file when the other branch tracks a file at that path. That is the default, `--overwrite-ignore`. Chapter 4, section 4.16, shows the same loss with `git switch`; `git merge --no-overwrite-ignore` aborts instead.

Submodules. A submodule path merges by fast-forwarding to the descendant commit when one side's commit descends from the other's. Otherwise it is a conflict (Chapter 23).

8.21 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git merge-base</code> , <code>git merge-file -p</code> , <code>git ls-files -u</code> , <code>git show :<n>:<path></code> , <code>git log --merge</code> , <code>git log --remerge-diff</code>	● SAFE	nothing	not needed	not needed
<code>git merge-tree --write-tree A B</code>	● SAFE	adds unreferenced objects	it is the preview	not needed
<code>git merge <other></code> (fast-forward or true merge)	● CAUTION	moves the current branch; rewrites index and working tree	<code>git merge-tree --write-tree --name-only HEAD <other></code> ; <code>git diff HEAD...<other></code>	before pushing: <code>git reset --merge ORIG_HEAD</code> (check it first, or take the commit from the reflog); after pushing: <code>git revert -m 1</code> (Chapter 11)
<code>git merge -X ours</code> , <code>-X theirs</code> , <code>-X ignore-space-change</code>	● CAUTION	as above, and silently drops one side of every conflicting hunk	merge without the option first	as above; audit with <code>--remerge-diff</code>
<code>git merge -s ours <other></code>	● CAUTION	records <code><other></code> as merged and takes none of its content	<code>git diff HEAD...<other></code> shows what is discarded	as above
<code>git merge --squash <other></code>	● CAUTION	index and working tree; no commit	as for <code>git merge</code>	<code>git reset --merge</code>
<code>git add <path></code> on an unmerged path	● SAFE	collapses stages 1 to 3 into stage 0	<code>git diff</code> , <code>git diff AUTO_MERGE</code>	<code>git restore --merge <path></code>
<code>git merge --continue</code> , <code>git commit</code> during a merge	● CAUTION	creates the merge commit and moves the branch	<code>git diff --cached --check</code> ; run the tests	<code>git reset --merge ORIG_HEAD</code> before pushing
<code>git merge --quit</code>	● CAUTION	deletes the merge state only	<code>git status</code>	<code>git reset --merge</code> ; no command resumes the merge
<code>git restore --ours --theirs --merge <path></code> and the <code>git checkout</code> forms	● DANGEROUS	overwrites the working tree file	<code>git diff --ours</code> , <code>git diff --theirs</code>	see below
<code>git merge --abort</code> , <code>git reset --merge</code>	● DANGEROUS	resets index and working tree to HEAD; deletes the merge state	<code>git status</code> , <code>git diff --cached --stat</code>	see below

For the ● commands:

- `git restore --ours|--theirs|--merge <path>`. Changes the working tree file only. Destroys your hand edits to that file, which exist nowhere else until you `git add` them. Preview with `git diff --ours` or `--theirs`. Recovery: `git restore --merge <path>` brings back the starting point, not your edits. Appropriate when the whole file from one side is the decision, or to restart a botched resolution.
- `git merge --abort` **and** `git reset --merge`. Change the index, the working tree and the merge state. Destroy every resolution made so far and any uncommitted edit that was staged during the merge. Preview with `git status`. Recovery: staged content survives as unreferenced objects that `git fsck` lists (Lab 6.7, Chapter 13); unstaged resolution edits are gone. Appropriate when the merge was a mistake or needs a different approach.

8.22 Version notes

Version note. Older behavior: `recursive` was the default strategy for two heads "from Git v0.99.9k until v2.33.0". Current behavior: `ort` is the default and `recursive` "is now a synonym for `ort`". Since: default in Git 2.34 ([release notes](#)), synonym in 2.50 ([merge strategies](#)). Recommended: configure nothing, and read "recursive" in older material as "the default strategy".

Version note. Older behavior: conflict styles `merge` and `diff3`. Current behavior: the default is still `merge`; `zdiff3` is available. Since: Git 2.35 ([release notes](#)). Recommended: `merge.conflictStyle=zdiff3`.

Version note. Older behavior: a resolution could be reviewed only through combined diffs. Current behavior: `--remerge-diff` for `git log` and `git show`. Since: Git 2.36 ([release notes](#)). Recommended: use it for every merge that had a conflict.

Version note. Older behavior: `git merge-tree <base> <branch1> <branch2>`, a trivial-merge mode that the manual now calls deprecated. Current behavior: `git merge-tree --write-tree` runs a full merge. Since: Git 2.38 ([release notes](#)); the mode that only checks for a clean merge arrived in 2.50 ([release notes](#)). Recommended: the `--write-tree` form.

Version note. Older behavior: `git add` stages a conflicted file with its markers. Current behavior on 2.55: the same. Since: Git 2.56 adds `git add --resolved` (not run here). The 2.56 release notes also list a fix for `git merge-base` without `--all`, which "could return incorrect results on repositories with v1 commit graphs and clock skew"; whether that affects a given 2.55 repository was not tested. Recommended: upgrade when convenient.

Outdated advice. Pro Git's "Advanced Merging" chapter teaches index stages and `--conflict=diff3` well, but it describes the `recursive` strategy and mentions neither `ort` nor `zdiff3`. Older answers that recommend `git checkout --theirs` to "accept incoming changes" recommend the whole-file shortcut of section 8.10.

Unverified. The release in which `git merge` began to write a stash-shaped commit before every true merge on a dirty tree was not determined; the behavior was observed on 2.55.0 and read in its source. Which command GitHub runs to compute merges is not stated in the pages cited in section 8.17.

8.23 Practice

- Labs 6.1 to 6.7 in the Module 6 lab manual: seven scenarios, from predicting a fast-forward to recovering an edit after an abort. Answers are in solutions.
- Replay any transcript with `labs/run ch08/<demo>`, for example `labs/run ch08/conflict-anatomy`. The sandbox stays in place afterwards.
- Three drills in those sandboxes:
 1. In `ch08/why-conflicts`, edit `base.yaml` so that the adjacent-line case merges cleanly, and prove it with `git merge-file -p`.
 2. In `ch08/merge-base/crisscross`, predict the output of `git merge-base --all main release/1.0` after `main` merges `release/1.0` once more. Then do it.
 3. In `ch08/merge-tree/server.git`, write a loop over all branches that prints "clean" or "conflict" against `main`, using only exit statuses.

8.24 Interview questions

1. Define "merge base". How can two commits have more than one, and what does the default strategy do then?
2. `git merge feature` printed "Fast-forward". What changed in `.git`, and what did not?
3. State the three-way rule for a single path as a table of base, ours and theirs. Which rows never read file content?
4. Two branches changed different lines of one file and the merge still conflicted. Give the exact condition under which that happens.
5. During a conflict, what are index stages 1, 2 and 3? How do you read "their" version of a file without touching the working tree?
6. A conflict was resolved with `git checkout --theirs path`, and a teammate's unrelated fix in that file disappeared. Explain the mechanism and prove it from history.
7. Compare `git merge -X ours`, `git merge -s ours` and `git merge --squash` in terms of content and ancestry.
8. Both branches were green, the merge had no conflict, and `main` is red. Why can Git not detect this, and which process controls can?
9. What do `git show <merge>`, `git show --first-parent <merge>` and `git show --remerge-diff <merge>` each display? Which one audits a conflict resolution?
10. A change was made on two branches and reverted on one. After the merge it is back. Explain with the merge base.
11. You reverted a merge and later merged the repaired branch; part of the feature is missing. What happened, and what is the correct sequence?

12. How can a server with no working tree decide whether a pull request merges cleanly? What does stock Git offer for that?

8.25 Sources

Primary sources

- [git-merge](#), [git-merge-base](#), [git-merge-tree](#), [git-merge-file](#), [git-read-tree](#), [gitrevisions](#), [git-status](#), [git-restore](#), [git-checkout](#), [git-log](#), [git-diff](#), [git-revert](#), [git-rerere](#), [gitattributes](#), [gitglossary](#), [gitfaq](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that every quotation and transcript was checked against.
- Git source and documentation at fixed tags: [merge strategies](#), [merge configuration](#), [git-add at 2.56](#), [builtin/merge.c at 2.55](#).
- Release notes: [2.34](#), [2.35](#), [2.36](#), [2.38](#), [2.50](#), [2.56](#).
- GitHub: [Pull request merges](#), [Events that trigger workflows](#), [changelog of 12 September 2022](#), [Scaling merge-ort across GitHub](#) (27 July 2023).

Secondary sources

- Pro Git, [Basic Branching and Merging](#) and [Advanced Merging](#). Caveat: see the "Outdated advice" note in section 8.22.
- Julia Evans, [Git poll results](#) (2024). Self-selected samples.
- [nbdime](#) documentation, for notebook diffs and merges.
- The Phase 0 report of this course, sections 1, 4, 12 and 13.

Videos (optional; the assessments in the Phase 0 report rest on captions and chapter lists, not on full viewing)

- David Mahler, ["Branching and Merging"](#) (29 min, 2017). Fast-forward against three-way merges drawn on the commit graph. Conceptually excellent; uses `master` and the commands that predate `git restore`.
- ThePrimeagen for Boot.dev, ["Git and GitHub - Full Course"](#) (2024): merge from 1:03:27, merge conflicts from 2:41:25. Current commands; a digressive commentary style.
- Elijah Newren, the author of `ort`, at [Git Merge 2022](#) and [2025](#): the merge backend and `--remerge-diff`. Contributor level; watch after this chapter.

Further reading

- Chapter 9 for the other way to integrate, Chapter 11 for undoing merges, Chapter 14C for rerere and merge drivers, Chapter 17 for merge methods on GitHub.

Chapter 9: Rebase

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch09/`.

9.1 Why this matters

Four questions a CTO can ask after a week in which someone rebased:

1. "Asha pushed a commit to the feature branch on Tuesday. On Wednesday it was gone from GitHub, and after her next pull it was gone from her laptop as well. Nobody deleted anything. Where is it?"
2. "The pull request had three commits before the review and three commits after it, with the same titles. The reviewer approved the first version. What exactly is different in the second one?"
3. "Why does the history of the ingest branch contain every commit twice?"
4. "A developer says the rebase finished without an error and one of his commits is not in the branch any more. Is that possible?"

All four have precise answers, and all four answers start from one fact: **a rebase does not move or change commits. It writes new commits and then moves a ref.** The old commits are still in the object database. Other clones still have them. The server still has them until someone pushes over them. Every surprise in this chapter is a consequence of two histories existing side by side for a while: the one you rewrote and the one everybody else still holds.

The answers are in sections 9.17, 9.14, 9.15 and 9.11, in that order. The fourth one is "yes".

A note on reading the transcripts. An interactive rebase opens an editor. In this book a small script plays the person at the keyboard: it prints the list as Git opened it, applies the edit, and prints the list as saved. Those two blocks stand for what you would see and do in your own editor.

9.2 What a rebase is

In one sentence. 🟡 `git rebase <upstream>` takes the commits that your branch has and `<upstream>` does not have, creates a copy of each one on top of `<upstream>`, in order, and then points your branch at the last copy.

Analogy. You wrote three amendments to version 4 of a contract. Meanwhile the other party issued version 6. Rebasing is redoing your three amendments, one after the other, against version 6, so that the file reads as if you had started from version 6. The analogy breaks in two places. First, the three amendments you wrote against version 4 are not shredded: they stay in the drawer (the object database), reachable through the reflog. Second, nobody retypes anything: each amendment is carried over by a three-way merge that Git computes, and that merge can stop and ask you to decide.

Precisely. The manual describes `git rebase <upstream>` as four steps ([git-rebase](#)):

1. Make the list of commits to replay: everything reachable from the current branch and not from `<upstream>`, the same set that `git log <upstream>..HEAD` prints, minus commits whose change is already in `<upstream>` (section 9.12) and minus merge commits (section 9.10).
2. Detach HEAD at `<upstream>`.
3. Replay the commits one by one, oldest first. Each replay is the cherry-pick operation of Chapter 10: a three-way merge whose base is the parent of the original commit, whose "ours" side is HEAD, and whose "theirs" side is the original commit. The result is committed with the original author, author date and message, and with you, now, as committer.
4. Move the branch ref to the last new commit and attach HEAD to the branch again.

Two words in that description deserve attention. "Reachable": Git has no notion of a parent branch (Chapter 7), so the set of commits is computed from the graph, not from where you believe the branch started. "Copy": a commit object is never modified (Chapter 3); the copies are new objects with new commit IDs (section 9.3).

Inside .git. New commit objects, and new tree objects wherever the combined content differs from anything stored before; blobs are reused when file content is unchanged. One branch ref is rewritten, once, at the end. HEAD is detached while the rebase runs. `ORIG_HEAD` records the old tip. The reflogs of HEAD and of the branch record every step. Section 9.4 walks through all of it.

See it. The project in this chapter is `ragkit`, a small retrieval-augmented answering service. `feat/rerank` was cut from the first commit of `main`; since then `main` gained two commits:

```
$ git log --oneline --graph --decorate --all
* 82f1fbb (HEAD -> main) Upgrade model to small-v2
* 589d18b Add README
| * bd62876 (feat/rerank) Enable reranking in config
| * af65a92 Call reranker from retriever
| * 5ee19f0 Add reranker skeleton
|/
* 8afc6bd Add retriever and model config
```

```
$ git switch feat/rerank
Switched to branch 'feat/rerank'
$ git rebase main
Rebasing (1/3)
Rebasing (2/3)
Rebasing (3/3)
Successfully rebased and updated refs/heads/feat/rerank.
```

Rebasing (1/3) is a progress line; in a terminal the three lines overwrite each other. The last line names the one ref that moved.

```
$ git log --oneline --graph --decorate --all
* 976a161 (HEAD -> feat/rerank) Enable reranking in config
* ade2990 Call reranker from retriever
* 742ab58 Add reranker skeleton
* 82f1fbb (main) Upgrade model to small-v2
* 589d18b Add README
* 8afc6bd Add retriever and model config
$ git log --oneline main..ORIG_HEAD
bd62876 Enable reranking in config
af65a92 Call reranker from retriever
5ee19f0 Add reranker skeleton
```

The branch now holds 742ab58, ade2990 and 976a161, on top of 82f1fbb. The second command shows that the three original commits still exist: ORIG_HEAD points at the old tip bd62876.

Now do the same rebase by hand with the operations that the manual names: detach, cherry-pick the range, move the branch.

```
$ git switch --detach main
HEAD is now at 82f1fbb Upgrade model to small-v2
$ git cherry-pick main..feat/rerank
[detached HEAD 0dea106] Add reranker skeleton
Date: Mon Sep 7 10:03:00 2026 +0530
1 file changed, 2 insertions(+)
create mode 100644 app/rerank.py
[detached HEAD 20d0897] Call reranker from retriever
Date: Mon Sep 7 10:04:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
Auto-merging config/model.yaml
[detached HEAD 4caa98e] Enable reranking in config
Date: Mon Sep 7 10:05:00 2026 +0530
1 file changed, 1 insertion(+)
```

```
$ git switch -C feat/rerank
Switched to and reset branch 'feat/rerank'
$ git log --oneline --graph --decorate --all
* 4caa98e (HEAD -> feat/rerank) Enable reranking in config
* 20d0897 Call reranker from retriever
* 0dea106 Add reranker skeleton
* 82f1fbb (main) Upgrade model to small-v2
* 589d18b Add README
* 8afc6bd Add retriever and model config
```

`git switch -C <branch>` (`git checkout -B` in the manual and in older scripts) creates the branch at HEAD or, if it exists, resets it to HEAD. That is the whole trick of step 4. Until that command ran, `feat/rerank` still pointed at the old commits and the three new commits belonged to no branch.

Picture.

```
Before
      5ee19f0---af65a92---bd62876      feat/rerank
      /
8afc6bd---589d18b---82f1fbb          main

After "git rebase main", run on feat/rerank

      5ee19f0---af65a92---bd62876      no branch: ORIG_HEAD, feat/rerank@{1}
      /
8afc6bd---589d18b---82f1fbb          main
                        \
                        742ab58---ade2990---976a161  feat/rerank  (HEAD -> feat/rerank)
```

In production. The everyday uses are bringing a feature branch up to date before review, so that reviewer and CI see your change on top of today's `main`, and tidying your own commits before anyone else reads them (section 9.6). Both are safe on a branch that only you use. Both become a team incident on a branch that others have based work on (section 9.15), and the difference between the two cases is the most important judgment in this chapter.

9.3 Why every rebased commit has a new ID

In one sentence. A commit ID is a hash of the commit object, the commit object names its parent and its tree, and a replayed commit has a different parent and usually a different tree, so it cannot have the same ID.

Precisely. Look at the first commit of the branch before and after the rebase of section 9.2:

```
# The first commit of the branch, before the rebase (reachable through ORIG_HEAD) ...
$ git cat-file -p ORIG_HEAD~2
tree bff5bbd64dc61559cfed48b794c4ac66aa8fd087
parent 8afc6bd28c2572af09f1b6c37d733535230e526f
author Lab User <you@example.com> 1788755580 +0530
committer Lab User <you@example.com> 1788755580 +0530

Add reranker skeleton
# ... and its replacement after the rebase.
$ git cat-file -p HEAD~2
tree 8cc3ecec7607fe1db26fe237762047f282c08338
parent 82f1fbb78c09021e2e673bf06e3c77fc58f14a50
author Lab User <you@example.com> 1788755580 +0530
committer Lab User <you@example.com> 1788756000 +0530

Add reranker skeleton
```

Compare the two objects field by field.

Field	Before	After	Why
tree	bff5bbd...	8cc3ecec...	The new snapshot also contains the README and the model upgrade from <code>main</code>
parent	8afc6bd...	82f1fbb...	That is the point of the operation
author	Lab User, 1788755580	identical	Rebase keeps who wrote the change and when
committer	Lab User, 1788755580	Lab User, 1788756000	The committer is whoever created this object, at the time it was created
message	identical	identical	Unless you reword it

Three of the five inputs to the hash changed. A copy would get a new ID even with the same parent and tree, because the committer date moves; that is why Git does not copy commits that would come out identical. It fast-forwards over them unless you pass `--no-ff`. And because each commit names its parent by ID, a new ID for one commit forces a new ID for every commit after it. IDs change from the first rewritten commit onward, never before it.

What did not change is the change itself. A patch ID is a hash of a commit's diff with line numbers ignored ([git-patch-id](#)):

```
$ git show ORIG_HEAD~2 | git patch-id --stable
cf1c4fb899863a713cf682e6a8101ba01f834cb4 5ee19f03aa988c15717abc4b6c14104fc8d23a97
$ git show HEAD~2 | git patch-id --stable
cf1c4fb899863a713cf682e6a8101ba01f834cb4 742ab5805e0a3077be749e44fb92159e284ee0ad
```

Same patch ID in the first column, different commit IDs in the second. Git uses this equivalence to recognise "the same change under another ID" in sections 9.12 and 9.15 and in Chapter 10.

The hand-made rebase of section 9.2 makes the same point from the other side:

```
# Keep the hand-made result under a tag, put the branch back, and let "git rebase" do the job.
$ git tag by-hand
$ git reset --hard "feat/rerank@{1}"
HEAD is now at bd62876 Enable reranking in config
$ git rebase main
Rebasing (1/3)
Rebasing (2/3)
Rebasing (3/3)
Successfully rebased and updated refs/heads/feat/rerank.
$ git log --format="%h tree %t %s" main..by-hand
4caa98e tree 273096f Enable reranking in config
20d0897 tree 9b070ab Call reranker from retriever
0dea106 tree 8cc3ece Add reranker skeleton
$ git log --format="%h tree %t %s" main..feat/rerank
a4cc3fb tree 273096f Enable reranking in config
a9cc881 tree 9b070ab Call reranker from retriever
2efc812 tree 8cc3ece Add reranker skeleton
```

Commit for commit, the trees are identical: `git rebase` and three hand-typed cherry-picks built the same snapshots. The commit IDs differ only because the real rebase ran a few lab minutes later, so its committer dates differ. Now look at the IDs that `git rebase main feat/rerank` prints in section 9.5: `0dea106`, `20d0897`, `4caa98e`, the IDs of the hand-made commits. That demo runs in another sandbox, with another command, at the same lab minute, and so produces byte-identical commit objects. A commit ID does not record which command made the commit. It is a function of content: tree, parents, author, committer, dates, message.

In production. Anything that stored an old commit ID now refers to a commit that is no longer on your branch: a review comment anchored to a commit, a CI status, a deployment record, the commit ID that a training job wrote into its run metadata. Before you rebase, ask what has already recorded these IDs. Tags do not follow a rebase (section 9.9 shows one that stayed behind). And signatures are part of the commit object (Chapter 14B): a replayed commit is a new object, signed only if you sign it now, with `-S` or `commit.gpgSign`.

GitHub, not Git. The "Rebase and merge" button performs this operation on GitHub's servers. The commits that land on the base branch always have new commit IDs and an updated committer, and they are unsigned, because GitHub cannot sign on your behalf ([merge methods](#), [signature verification](#)). Described from the documentation; Chapter 17 covers the three merge buttons.

9.4 What rebase does internally

In one sentence. A rebase is a small interpreter: it writes a list of instructions into a state directory inside `.git`, executes them one at a time on a detached HEAD, and touches the branch ref only when the list is empty.

See it. The way to see the machinery is to make it stop in the middle. In this repository the second commit of `feat/rerank` changes a line that `main` has also changed:

```
$ git log --oneline --graph --decorate --all
* 439e4c6 (main) Raise TOP_K to 8 after recall regression
| * ad106e3 (HEAD -> feat/rerank) Enable reranking in config
| * 569e6c9 Fetch 20 candidates for the reranker
| * 5ee19f0 Add reranker skeleton
|/
* 8afc6bd Add retriever and model config
```

```
$ git rebase main
Rebasing (1/3)
Rebasing (2/3)
Auto-merging app/retriever.py
CONFLICT (content): Merge conflict in app/retriever.py
error: could not apply 569e6c9... Fetch 20 candidates for the reranker
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
hint: Disable this message with "git config set advice.mergeConflict false"
Could not apply 569e6c9... # Fetch 20 candidates for the reranker
[exit status: 1]
```

The first commit was replayed. The second one stopped with a conflict, and the command exited with status 1. Section 9.11 deals with the conflict. Here, look at the state Git is in.

```
$ cat .git/HEAD
1c9f69a02aeeb08430a5015ba7af214c9b96d0a0
$ git branch
* (no branch, rebasing feat/rerank)
  feat/rerank
  main
$ git log --oneline --decorate -3
1c9f69a (HEAD) Add reranker skeleton
439e4c6 (main) Raise TOP_K to 8 after recall regression
8afc6bd Add retriever and model config
```

`.git/HEAD` holds a raw commit ID, not `ref: refs/heads/...`: HEAD is detached (Chapter 7). It sits on `1c9f69a`, the copy of "Add reranker skeleton", which sits on the tip of `main`. `git branch` describes the situation as `(no branch, rebasing feat/rerank)`.

```
$ ls .git/rebase-merge
author-script
done
drop_redundant_commits
end
git-rebase-todo
git-rebase-todo.backup
head-name
interactive
message
msgnum
no-reschedule-failed-exec
onto
orig-head
patch
rewritten-list
stopped-sha
```

```
$ cat .git/rebase-merge/head-name
refs/heads/feat/rerank
$ cat .git/rebase-merge/orig-head
ad106e3ecc6f4d2e14376112d9580c6892a12093
$ cat .git/rebase-merge/onto
439e4c6ba104e7816d32d16fd3bd9548e536b33f
$ cat .git/rebase-merge/done
pick 5ee19f03aa988c15717abc4b6c14104fc8d23a97 # Add reranker skeleton
pick 569e6c983316561062d42644d649129e50eada5d # Fetch 20 candidates for the reranker
$ cat .git/rebase-merge/git-rebase-todo
```

```

pick ad106e3ecc6f4d2e14376112d9580c6892a12093 # Enable reranking in config
$ cat .git/rebase-merge/msgnum .git/rebase-merge/end
2
3
$ cat .git/rebase-merge/author-script
GIT_AUTHOR_NAME='Lab User'
GIT_AUTHOR_EMAIL='you@example.com'
GIT_AUTHOR_DATE='@1788755640 +0530'

```

What the files mean:

head-name	the ref that will be moved at the end: refs/heads/feat/rerank
orig-head	where that ref pointed when the rebase started
onto	the commit that the copies are built on
git-rebase-todo	the instructions not yet executed, one per line: Git calls this the todo list
done	the instructions already executed
msgnum, end	the position: instruction 2 of 3
author-script	author name, email and date of the commit being replayed, reused for its copy
message	the commit message that will be reused when you continue
stopped-sha	the commit at which the rebase stopped
rewritten-list	"old ID, new ID" for each commit copied so far; input for the post-rewrite hook
interactive	marker of the merge backend (see the note on "git status" below)

In running text this book writes "to-do list" for the instruction list; Git spells it as one word, as the file name shows.

```

$ git rev-parse --short ORIG_HEAD
ad106e3
$ git rev-parse --short REBASE_HEAD
569e6c9
$ git rev-parse --short feat/rerank
ad106e3
$ git rev-parse --short HEAD
1c9f69a

```

Four names, three commits. `ORIG_HEAD` is the tip of the branch before the rebase began. `REBASE_HEAD` is the commit that is being replayed right now; it exists only while a rebase is stopped. `HEAD` is the last copy made. And `feat/rerank` still equals `ORIG_HEAD`: **the branch has not moved**.

Version note. Older behavior: the glossary called every such file, `ORIG_HEAD` and `REBASE_HEAD` included, a pseudoref, and most tutorials still do. Current behavior: "pseudoref" means only `FETCH_HEAD` and `MERGE_HEAD`; the others are ordinary refs that live at the root of the ref namespace. Since: Git 2.46 ([glossary](#)). Recommended: say "root ref", and read them with `git rev-parse`, not with `cat`.

```

$ git status
interactive rebase in progress; onto 439e4c6
Last commands done (2 commands done):
  pick 5ee19f0 # Add reranker skeleton
  pick 569e6c9 # Fetch 20 candidates for the reranker
Next command to do (1 remaining command):
  pick ad106e3 # Enable reranking in config
  (use "git rebase --edit-todo" to view and edit)
You are currently rebasing branch 'feat/rerank' on '439e4c6'.
  (fix conflicts and then run "git rebase --continue")
  (use "git rebase --skip" to skip this patch)
  (use "git rebase --abort" to check out the original branch)

Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
    both modified:   app/retriever.py

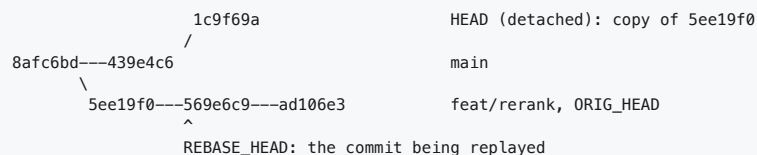
no changes added to commit (use "git add" and/or "git commit -a")

```

`git status` reads the state directory back to you: what was done, what remains, and the three ways out.

Root cause. "interactive rebase in progress" appears although no `-i` was typed. The default backend of `git rebase` is the machinery that once served only interactive rebases; it always writes the marker file `interactive`, and `git status` reports what it finds.

Picture. The repository while the rebase is stopped:



Now leave the way you came:


```
$ git rebase --abort
$ git status
On branch feat/rerank
nothing to commit, working tree clean
$ git reflog -3
ad106e3 HEAD@{0}: rebase (abort): returning to refs/heads/feat/rerank
1c9f69a HEAD@{1}: rebase (pick): Add reranker skeleton
439e4c6 HEAD@{2}: rebase (start): checkout main
```

`--abort` only had to attach HEAD to the branch again and restore the index and working tree from it. The copy `1c9f69a` is now reachable from nothing except the reflog.

When a rebase runs to the end, the two reflogs record it differently. This is the trail of the rebase in section 9.2:

```
$ git reflog -6
976a161 HEAD@{0}: rebase (finish): returning to refs/heads/feat/rerank
976a161 HEAD@{1}: rebase (pick): Enable reranking in config
ade2990 HEAD@{2}: rebase (pick): Call reranker from retriever
742ab58 HEAD@{3}: rebase (pick): Add reranker skeleton
82f1fbb HEAD@{4}: rebase (start): checkout main
bd62876 HEAD@{5}: checkout: moving from main to feat/rerank
$ git reflog show feat/rerank
976a161 feat/rerank@{0}: rebase (finish): refs/heads/feat/rerank onto 82f1fbb78c09021e2e673bf06e3c77fc58f14a50
bd62876 feat/rerank@{1}: commit: Enable reranking in config
af65a92 feat/rerank@{2}: commit: Call reranker from retriever
5ee19f0 feat/rerank@{3}: commit: Add reranker skeleton
8afc6bd feat/rerank@{4}: branch: Created from HEAD
```

The HEAD reflog has one line per step: `rebase (start)` is the detach, each `rebase (pick)` is one copy, `rebase (finish)` is the return to the branch. The branch reflog has one line for the whole rebase, and the line below it is the old tip. `feat/rerank@{1}` therefore means "this branch before its last rebase", and that is the most reliable undo handle you have (section 9.16).

State table. One row per phase.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git rebase <upstream></code> , runs to the end	Becomes the snapshot of the last copy	Same snapshot	Detached during the run, attached to the branch again at the end	Moved once, at the end, to the last copy	<code>ORIG_HEAD</code> set to the old tip; new commit and tree objects; one reflog entry per step for HEAD, one for the branch; state directory created and removed	unchanged	unchanged
The same, stopped at a conflict	Conflict markers in the conflicted files; everything else at the state of HEAD plus the cleanly merged changes	Stages 1, 2 and 3 for conflicted paths	Detached at the last copy	unchanged: still the old tip	State directory <code>.git/rebase-merge/</code> ; <code>REBASE_HEAD</code> ; <code>ORIG_HEAD</code>	unchanged	unchanged
<code>git rebase --continue</code>	unchanged by the commit itself, then updated by the following steps	Committed as the copy, then updated	Advances to the new copy and onward	Moved if the list becomes empty	Next instructions executed	unchanged	unchanged
<code>git rebase --skip</code>	Reset to HEAD: the conflicted merge is discarded	Reset to HEAD	Stays, then advances with the following steps	Moved if the list becomes empty	The stopped commit gets no copy	unchanged	unchanged
<code>git rebase --abort</code>	Reset to the old tip: resolution work in progress is lost	Reset to the old tip	Attached to the branch again	unchanged	State directory and <code>REBASE_HEAD</code> removed; <code>ORIG_HEAD</code> stays	unchanged	unchanged
<code>git rebase --quit</code>	unchanged	unchanged	Stays detached where it is	unchanged	State directory removed; an autostash moves to the stash list	unchanged	unchanged

The other backend. Until Git 2.26 the default implementation turned each commit into a patch and applied the patches, the way `git format-patch` and `git am` do. It is still there:

```
$ git rebase --apply main
First, rewinding head to replay your work on top of it...
Applying: Add reranker skeleton
Applying: Fetch 20 candidates for the reranker
Using index info to reconstruct a base tree...
M    app/retriever.py
Falling back to patching base and 3-way merge...
Auto-merging app/retriever.py
CONFLICT (content): Merge conflict in app/retriever.py
error: Failed to merge in the changes.
hint: Use 'git am --show-current-patch=diff' to see the failed patch
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
```

```

hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
hint: Disable this message with "git config set advice.mergeConflict false"
Patch failed at 0002 Fetch 20 candidates for the reranker
[exit status: 1]

```

```

$ ls .git | grep rebase
rebase-apply
$ ls .git/rebase-apply | head -5
0001
0002
0003
abort-safety
apply-opt
$ git status | head -2
rebase in progress; onto 439e4c6
You are currently rebasing branch 'feat/rerank' on '439e4c6'.
$ head -5 app/retriever.py
<<<<<< HEAD
TOP_K = 8
=====
TOP_K = 20
>>>>>> Fetch 20 candidates for the reranker

```

Different words ("rewinding head", "Patch failed at 0002"), a different state directory (`.git/rebase-apply`, with one numbered patch file per commit), "rebase in progress" without the word interactive, and a conflict label that carries only the subject of your commit, not its ID. You will meet these in old answers and old scripts. The manual lists what the apply backend does worse: it works from patch context and can apply a hunk in the wrong place without reporting a conflict, it cannot detect directory renames, and it drops empty commits ([git-rebase](#), ["Behavioral differences"](#)). Leave `rebase.backend` alone.

In production. "Is a rebase in progress here?" The first line of `git status` answers it, and so does `ls .git | grep rebase`. If the answer is yes, nothing is lost: the branch ref has not moved, and `git rebase --abort` returns to it. Do not delete the state directory by hand; that leaves HEAD detached on a half-built history, which is what `--quit` does on purpose.

9.5 Choosing what moves and where it lands

In one sentence. Every form of the command answers three questions: which branch is rewritten, which commits are replayed, and on which commit the copies are built.

Precisely.

```
git rebase [--onto <newbase>] [<upstream> [<branch>]]
```

- `<branch>` is the branch that gets rewritten. Default: the current branch. If you name it, Git switches to it as part of the command.
- `<upstream>` is only a boundary. The commits replayed are `<upstream>..<branch>`: reachable from the branch, not reachable from `<upstream>`.
- `<newbase>` is where the copies are built. Default: `<upstream>`.

Form	Commits replayed	Built on
<code>git rebase main</code>	<code>main..HEAD</code>	<code>main</code>
<code>git rebase main feat/x</code>	<code>main..feat/x</code> , after switching to <code>feat/x</code>	<code>main</code>
<code>git rebase</code>	from the configured upstream branch, using the fork point (section 9.15)	that upstream
<code>git rebase --onto N U</code>	<code>U..HEAD</code>	<code>N</code>
<code>git rebase --keep-base main</code>	<code>main..HEAD</code>	the merge base of <code>main</code> and <code>HEAD</code>
<code>git rebase --root</code>	every commit reachable from <code>HEAD</code>	nothing: the root commit is recreated
<code>git rebase -i HEAD~3</code>	the last three commits	<code>HEAD~3</code> , where they already are

The two-argument form, started from `main`:

```

$ git status --short --branch
## main
$ git rebase main feat/rerank
Rebasing (1/3)
Rebasing (2/3)
Rebasing (3/3)
Successfully rebased and updated refs/heads/feat/rerank.
$ git status --short --branch
## feat/rerank
$ git reflog -5
4caa98e HEAD@{0}: rebase (finish): returning to refs/heads/feat/rerank
4caa98e HEAD@{1}: rebase (pick): Enable reranking in config
20d0897 HEAD@{2}: rebase (pick): Call reranker from retriever
0dea106 HEAD@{3}: rebase (pick): Add reranker skeleton
82f1fbb HEAD@{4}: rebase (start): checkout main

```

You end up on `feat/rerank`. Git does not check out the old tip of `feat/rerank` on the way: it detaches at `main`, makes the copies, and attaches HEAD to the branch at the end, which is what the five reflog lines show. Without any argument, the branch needs a configured upstream:

```
$ git rebase
There is no tracking information for the current branch.
Please specify which branch you want to rebase against.
See git-rebase(1) for details.

    git rebase '<branch>'

If you wish to set tracking information for this branch you can do so with:

    git branch --set-upstream-to=<remote>/<branch> feat/rerank

[exit status: 1]
```

--onto: three worked cases. With `--onto`, the boundary and the destination are different commits.

```
git rebase --onto N U B      replay U..B on top of N

Before                      After

  o---o---N                  o---o---N---x'---y'---z'   B
   \                          \
  o---U---x---y---z   B      o---U
```

Case 1: a branch stacked on a branch that was squash-merged. `feat/ingest-cleaner` was built on `feat/ingest-loader`. The loader work has since landed on `main` as one squashed commit, with a small change made during review:

```
$ git log --oneline --graph --decorate --all
* 2b2840e (main) Add document loader (#41)
* 9ba5170 Add README
| * 05dc4ce (HEAD -> feat/ingest-cleaner) Collapse repeated whitespace
| * 3c47896 Add text cleaner
| * 7fea52c (feat/ingest-loader) Close files after loading
| * 531d3da Add document loader
|/
* 8afc6bd Add retriever and model config
```

```
# What a plain "git rebase main" would replay:
$ git log --oneline main..feat/ingest-cleaner
05dc4ce Collapse repeated whitespace
3c47896 Add text cleaner
7fea52c Close files after loading
531d3da Add document loader
# What belongs to this branch alone:
$ git log --oneline feat/ingest-loader..feat/ingest-cleaner
05dc4ce Collapse repeated whitespace
3c47896 Add text cleaner
```

```
$ git rebase main
Rebasing (1/4)
Auto-merging ingest/loader.py
CONFLICT (add/add): Merge conflict in ingest/loader.py
error: could not apply 531d3da... Add document loader
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
hint: Disable this message with "git config set advice.mergeConflict false"
Could not apply 531d3da... # Add document loader
[exit status: 1]
$ git rebase --abort
```

Observed behavior	: "git rebase main" stops at once with a conflict in <code>ingest/loader.py</code> , a file your branch never touched.
Git state	: <code>main..feat/ingest-cleaner</code> contains four commits: your two, and the two loader commits below them.
Mechanism	: Rebase replays everything not reachable from <code>main</code> . The loader commits are not reachable from <code>main</code> : <code>main</code> has their content in another commit (the squash), not the commits themselves.
Root cause	: Git has no record that your branch "belongs on top of" another branch. It knows reachability only.
Why Git does this	: A branch is a ref to a commit. "My commits" is something only you can define, by naming a boundary.
Correct fix	: <code>git rebase --onto main feat/ingest-loader feat/ingest-cleaner</code>
Prevention	: For a stacked branch, name the boundary whenever the branch below it lands in another form.

```
$ git rebase --onto main feat/ingest-loader feat/ingest-cleaner
Rebasing (1/2)
Rebasing (2/2)
```

```
Successfully rebased and updated refs/heads/feat/ingest-cleaner.
$ git log --oneline --graph --decorate --all
* f35e612 (HEAD -> feat/ingest-cleaner) Collapse repeated whitespace
* 085d42e Add text cleaner
* 2b2840e (main) Add document loader (#41)
* 9ba5170 Add README
| * 7fea52c (feat/ingest-loader) Close files after loading
| * 531d3da Add document loader
|/
* 8afc6bd Add retriever and model config
```

GitHub, not Git. "Squash and merge" puts one new commit on the base branch; the commits of the head branch do not become ancestors of it. GitHub's documentation warns that continuing to work on a head branch after it was squash-merged makes later pull requests list the squashed commits again and repeat conflicts ([about merge methods](#)). Case 1 is the Git-side remedy.

Case 2: sideways, onto an older line. `fix/timeout` was cut from `main`, which already carries work for 2.0. The fix has to ship from `release/1.4`:

```
$ git log --oneline --graph --decorate --all
* 91476e2 (HEAD -> fix/timeout) Return no documents when search times out
* 1c42ebc Add a timeout to search calls
* c49bad6 (main) Switch default model to large-v3 for 2.0
* 16dfe8e Start 2.0: add async search client
* 8afc6bd (release/1.4) Add retriever and model config
```

```
$ git rebase --onto release/1.4 main fix/timeout
Rebasing (1/2)
Rebasing (2/2)
Successfully rebased and updated refs/heads/fix/timeout.
$ git log --oneline --graph --decorate --all
* 594044f (HEAD -> fix/timeout) Return no documents when search times out
* 58c909f Add a timeout to search calls
| * c49bad6 (main) Switch default model to large-v3 for 2.0
| * 16dfe8e Start 2.0: add async search client
|/
* 8afc6bd (release/1.4) Add retriever and model config
```

```
$ git diff --stat release/1.4 fix/timeout
app/retriever.py | 6 ++++-
1 file changed, 5 insertions(+), 1 deletion(-)
$ ls app
retriever.py
```

`main` served only as the boundary: "my commits are the ones after `main`". The check confirms that one file differs from the release and that the 2.0 client did not come along.

Case 3: cutting commits out of the middle. Two experiment commits sit in the middle of `feat/rerank`:

```
$ git log --oneline --decorate
6ab8f55 (HEAD -> feat/rerank) Enable reranking in config
45f4f23 Call reranker from retriever
e30ba74 Experiment: cache cross-encoder scores
195f8d0 Experiment: cross-encoder scoring
5ee19f0 Add reranker skeleton
8afc6bd (main) Add retriever and model config
```

```
# Keep everything up to feat/rerank~4, drop ~3 and ~2, replay ~1 and the tip.
$ git rebase --onto feat/rerank~4 feat/rerank~2 feat/rerank
Rebasing (1/2)
Rebasing (2/2)
Successfully rebased and updated refs/heads/feat/rerank.
$ git log --oneline --decorate
31e8a4d (HEAD -> feat/rerank) Enable reranking in config
a15178d Call reranker from retriever
5ee19f0 Add reranker skeleton
8afc6bd (main) Add retriever and model config
```

```
$ git diff --stat ORIG_HEAD HEAD
app/cross_encoder.py | 7 -----
1 file changed, 7 deletions(-)
```

The copies are built on `feat/rerank~4`, and the boundary `feat/rerank~2` excludes everything up to and including the second experiment. The diff against `ORIG_HEAD` shows exactly one file gone. The same result comes from `drop` in an interactive rebase (section 9.6); this form is the one you can script.

`--root`. Without a boundary, the root commit itself is in the range, so even the first commit can be reworded:

```
$ git rebase -i --root
--- todo list as Git opened it (comment lines removed) ---
```

```

pick 8afc6bd # Add retriever and model config
pick 589d18b # Add README
pick 82f1fbb # Upgrade model to small-v2
--- todo list as saved ---
reword 8afc6bd # Add retriever and model config
pick 589d18b # Add README
pick 82f1fbb # Upgrade model to small-v2
Rebasing (1/3)
[detached HEAD 580b0da] Add retriever and default model configuration
Date: Mon Sep 7 10:02:00 2026 +0530
2 files changed, 7 insertions(+)
create mode 100644 app/retriever.py
create mode 100644 config/model.yaml
Rebasing (2/3)
Rebasing (3/3)
Successfully rebased and updated refs/heads/main.

```

```

$ git log --oneline --decorate
6cdce43 (HEAD -> main) Upgrade model to small-v2
ffe456d Add README
580b0da Add retriever and default model configuration
$ git log --oneline ORIG_HEAD
82f1fbb Upgrade model to small-v2
589d18b Add README
8afc6bd Add retriever and model config
$ git merge-base HEAD ORIG_HEAD
[exit status: 1]

```

Every commit has a new ID, and `git merge-base` exits with status 1: the new history and the old one have no commit in common. On a repository that others have cloned, that is a rewrite of the whole project, with every consequence of section 9.15 multiplied by the number of clones.

`--keep-base`. Tidy a branch without catching up. The copies are built on the merge base the branch already has:

```

$ git log --oneline --graph --decorate --all
* 9ba5170 (main) Add README
| * 028ba01 (HEAD -> feat/metrics) fixup! Add exact_match metric
| * a25e84c Add token-level F1 metric
| * 375df0b Test exact_match
| * 143e55a Add exact_match metric
|/
* 8afc6bd Add retriever and model config
$ git merge-base main feat/metrics
8afc6bd28c2572af09f1b6c37d733535230e526f

```

```

$ git rebase --autosquash --keep-base main
Rebasing (2/4)
Rebasing (3/4)
Rebasing (4/4)
Successfully rebased and updated refs/heads/feat/metrics.

```

```

$ git log --oneline --graph --decorate --all
* e4d27a0 (HEAD -> feat/metrics) Add token-level F1 metric
* 2b30819 Test exact_match
* fa4c85b Add exact_match metric
| * 9ba5170 (main) Add README
|/
* 8afc6bd Add retriever and model config
$ git merge-base main feat/metrics
8afc6bd28c2572af09f1b6c37d733535230e526f

```

The `fixup!` commit was folded into its target (section 9.7), and the branch still forks from `8afc6bd` although `main` has moved on. The progress counter starts at 2 because instruction 1, a pick whose parent is already the base, needed no new commit: Git fast-forwards over instructions that would recreate what exists.

In production. Reviewers of a force-pushed branch want to know what changed in your commits. If one push both tidies and catches up, that answer is mixed with a week of upstream changes. Tidy with `--keep-base` and push; catch up with a plain rebase and push again. Each push then has one explanation.

9.6 Interactive rebase

In one sentence. `git rebase -i <base>` shows you the to-do list before executing it, and the list is a program: one instruction per line, executed from top to bottom, that you may edit, reorder, extend or shorten.

Analogy. An edit decision list in film editing: a list of takes with an instruction for each (keep, cut, join with the previous one, retitle). The footage is not altered; a new reel is assembled from it. The analogy breaks where commits depend on each other: move a take and the film still plays, move a commit in front of the commit it builds on and the replay stops with a conflict.

Precisely. The instructions, as listed by Git itself in the transcript below:

Instruction	Short	Effect
<code>pick <commit></code>	<code>p</code>	Replay the commit as it is
<code>reword <commit></code>	<code>r</code>	Replay it, then open the editor on its message
<code>edit <commit></code>	<code>e</code>	Replay it, then stop so that you can amend or split it
<code>squash <commit></code>	<code>s</code>	Meld it into the commit on the line above; the editor opens on both messages
<code>fixup <commit></code>	<code>f</code>	Meld it into the commit above and keep that commit's message. <code>fixup -C</code> keeps this commit's message instead; <code>-c</code> also opens the editor
<code>drop <commit></code>	<code>d</code>	Leave the commit out
<code>exec <command></code>	<code>x</code>	Run a shell command; a non-zero exit status stops the rebase
<code>break</code>	<code>b</code>	Stop here
<code>label, reset, merge</code>	<code>l, t, m</code>	Rebuild a branch structure (section 9.10)
<code>update-ref <ref></code>	<code>u</code>	Move another branch to this point when the rebase finishes (section 9.9)
a line moved		The commit is replayed at its new position
a line deleted		The same as <code>drop</code> , without a word

`<base>` is the last commit you want to keep as it is: `git rebase -i main` offers every commit of the branch, `git rebase -i HEAD~3` the last three.

Inside .git. Nothing new. It is the machinery of section 9.4, with one difference: the instruction file is handed to your editor before execution starts. The editor is `sequence.editor` or `GIT_SEQUENCE_EDITOR` if set, otherwise your normal commit message editor.

See it. A branch as it looks after a day of real work:

```
$ git log --oneline --decorate
8313de0 (HEAD -> feat/metrics) fix test
031999d Add token-level F1 metirc
25dc48e debug print
cdb5f2f add test
a1056e1 wip
d33e9b2 Add exact_match metric
061c92d (main) Add lint script
8afc6bd Add retriever and model config
```

First, the list exactly as Git writes it. Using `cat` as the editor prints the file and saves it unchanged, so this rebase does nothing:

```
# Using "cat" as the editor prints the todo list exactly as Git wrote it and saves it unchanged.
$ GIT_SEQUENCE_EDITOR=cat git rebase -i main
pick d33e9b2 # Add exact_match metric
pick a1056e1 # wip
pick cdb5f2f # add test
pick 25dc48e # debug print
pick 031999d # Add token-level F1 metirc
pick 8313de0 # fix test

# Rebase 061c92d..8313de0 onto 061c92d (6 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                               commit's log message, unless -C is used, in which case
#                               keep only this commit's message; -c is same as -C but
#                               opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
#     create a merge commit using the original merge commit's
#     message (or the oneline, if no original merge commit was
#     specified); use -c <commit> to reword the commit message
# u, update-ref <ref> = track a placeholder for the <ref> to be updated
#                       to this position in the new commits. The <ref> is
#                       updated at the end of the rebase
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#
Successfully rebased and updated refs/heads/feat/metrics.
```

The list runs oldest first, the opposite of `git log`. The comment block is the complete reference. Now six rebases, each with one edit, so that each instruction can be seen alone. In practice you make all edits in one pass (Lab 9.1).

`reword`, on the commit with the typo in its subject:

```
$ git rebase -i main
--- todo list as Git opened it (comment lines removed) ---
pick d33e9b2 # Add exact_match metric
pick a1056e1 # wip
pick cdb5f2f # add test
pick 25dc48e # debug print
pick 031999d # Add token-level F1 metric
pick 8313de0 # fix test
--- todo list as saved ---
pick d33e9b2 # Add exact_match metric
pick a1056e1 # wip
pick cdb5f2f # add test
pick 25dc48e # debug print
reword 031999d # Add token-level F1 metric
pick 8313de0 # fix test
Rebasing (5/6)
[detached HEAD a2fd360] Add token-level F1 metric
Date: Mon Sep 7 10:08:00 2026 +0530
1 file changed, 4 insertions(+)
create mode 100644 eval/f1.py
Rebasing (6/6)
Successfully rebased and updated refs/heads/feat/metrics.
$ git log --oneline
fcea3ea fix test
a2fd360 Add token-level F1 metric
25dc48e debug print
cdb5f2f add test
a1056e1 wip
d33e9b2 Add exact_match metric
061c92d Add lint script
8afc6bd Add retriever and model config
```

Progress starts at (5/6). Lines 1 to 4 were unchanged, so Git fast-forwarded over them, and the first four commits kept their IDs. Only the reworded commit and the one after it are new objects (a2fd360, fcea3ea).

`fixup`, to fold "wip" into the commit above it:

```
$ git rebase -i main
--- todo list as Git opened it (comment lines removed) ---
pick d33e9b2 # Add exact_match metric
pick a1056e1 # wip
pick cdb5f2f # add test
pick 25dc48e # debug print
pick a2fd360 # Add token-level F1 metric
pick fcea3ea # fix test
--- todo list as saved ---
pick d33e9b2 # Add exact_match metric
fixup a1056e1 # wip
pick cdb5f2f # add test
pick 25dc48e # debug print
pick a2fd360 # Add token-level F1 metric
pick fcea3ea # fix test
Rebasing (2/6)
Rebasing (3/6)
Rebasing (4/6)
Rebasing (5/6)
Rebasing (6/6)
Successfully rebased and updated refs/heads/feat/metrics.
$ git log --oneline
92f52c5 fix test
3838789 Add token-level F1 metric
0d58a02 debug print
babel2d add test
753f401 Add exact_match metric
061c92d Add lint script
8afc6bd Add retriever and model config
```

Six commits became five, and all five have new IDs: the first because "wip" was melded into it, the others because their parent changed.

Reordering, to bring "fix test" directly behind "add test":

```
$ git rebase -i main
--- todo list as Git opened it (comment lines removed) ---
pick 753f401 # Add exact_match metric
pick babel2d # add test
pick 0d58a02 # debug print
pick 3838789 # Add token-level F1 metric
pick 92f52c5 # fix test
--- todo list as saved ---
pick 753f401 # Add exact_match metric
pick babel2d # add test
pick 92f52c5 # fix test
```

```

pick 0d58a02 # debug print
pick 3838789 # Add token-level F1 metric
Rebasing (3/5)
Rebasing (4/5)
Rebasing (5/5)
Successfully rebased and updated refs/heads/feat/metrics.
$ git log --oneline
8f81a9f Add token-level F1 metric
9013484 debug print
1537b45 fix test
babe12d add test
753f401 Add exact_match metric
061c92d Add lint script
8afc6bd Add retriever and model config

```

squash, now that the two test commits are neighbours. The editor opens on the combined message, and the scripted editor types a better one:

```

$ git rebase -i main
--- todo list as Git opened it (comment lines removed) ---
pick 753f401 # Add exact_match metric
pick babe12d # add test
pick 1537b45 # fix test
pick 9013484 # debug print
pick 8f81a9f # Add token-level F1 metric
--- todo list as saved ---
pick 753f401 # Add exact_match metric
pick babe12d # add test
squash 1537b45 # fix test
pick 9013484 # debug print
pick 8f81a9f # Add token-level F1 metric
Rebasing (3/5)
[detached HEAD 62a8273] Test exact_match, including surrounding whitespace
Date: Mon Sep 7 10:06:00 2026 +0530
1 file changed, 5 insertions(+)
create mode 100644 tests/test_metrics.py
Rebasing (4/5)
Rebasing (5/5)
Successfully rebased and updated refs/heads/feat/metrics.
$ git log --oneline
45ad5b3 Add token-level F1 metric
fe1bccf debug print
62a8273 Test exact_match, including surrounding whitespace
753f401 Add exact_match metric
061c92d Add lint script
8afc6bd Add retriever and model config

```

What does the editor show at that moment? On a cleaner branch, with `cat` standing in for the message editor:

```

$ git rebase -i main
--- todo list as Git opened it (comment lines removed) ---
pick 143e55a # Add exact_match metric
pick 375df0b # Test exact_match
pick a25e84c # Add token-level F1 metric
--- todo list as saved ---
pick 143e55a # Add exact_match metric
squash 375df0b # Test exact_match
pick a25e84c # Add token-level F1 metric
Rebasing (2/3)
# This is a combination of 2 commits.
# This is the 1st commit message:

Add exact_match metric

# This is the commit message #2:

Test exact_match

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Mon Sep 7 10:03:00 2026 +0530
#
# interactive rebase in progress; onto 8afc6bd
# Last commands done (2 commands done):
#   pick 143e55a # Add exact_match metric
#   squash 375df0b # Test exact_match
# Next command to do (1 remaining command):
#   pick a25e84c # Add token-level F1 metric
# You are currently rebasing branch 'feat/metrics' on '8afc6bd'.
#
# Changes to be committed:
#   new file:   eval/metrics.py
#   new file:   tests/test_metrics.py
#
[detached HEAD 5a2e35c] Add exact_match metric
Date: Mon Sep 7 10:03:00 2026 +0530
2 files changed, 6 insertions(+)
create mode 100644 eval/metrics.py
create mode 100644 tests/test_metrics.py

```



```
Rebasing (3/3)
Successfully rebased and updated refs/heads/feat/metrics.
```

Both messages, each under a comment line. Lines that start with `#` are removed when you save, so saving this buffer unchanged gives a message made of both texts. `fixup` skips this step and keeps the first message.

`drop`, for the debugging commit:

```
$ git rebase -i main
--- todo list as Git opened it (comment lines removed) ---
pick 753f401 # Add exact_match metric
pick 62a8273 # Test exact_match, including surrounding whitespace
pick fe1bccf # debug print
pick 45ad5b3 # Add token-level F1 metric
--- todo list as saved ---
pick 753f401 # Add exact_match metric
pick 62a8273 # Test exact_match, including surrounding whitespace
drop fe1bccf # debug print
pick 45ad5b3 # Add token-level F1 metric
Rebasing (4/4)
Successfully rebased and updated refs/heads/feat/metrics.
$ git log --oneline --decorate
d1d1482 (HEAD -> feat/metrics) Add token-level F1 metric
62a8273 Test exact_match, including surrounding whitespace
753f401 Add exact_match metric
061c92d (main) Add lint script
8afc6bd Add retriever and model config
```

```
$ git show --stat --format="%h %s" HEAD~2 HEAD~1 HEAD
753f401 Add exact_match metric

eval/metrics.py | 2 ++
1 file changed, 2 insertions(+)
62a8273 Test exact_match, including surrounding whitespace

tests/test_metrics.py | 5 ++++
1 file changed, 5 insertions(+)
d1d1482 Add token-level F1 metric

eval/f1.py | 4 +++
1 file changed, 4 insertions(+)
```

`edit`: **stop inside the history**. One commit of this branch mixes a new metric with a configuration change:

```
$ git log --oneline --decorate
4394181 (HEAD -> feat/metrics) Test both metrics
0527727 Add F1 metric and raise max_tokens
c1adfe1 Add exact_match metric
8afc6bd (main) Add retriever and model config
$ git show --stat --format="%h %s" HEAD~1
0527727 Add F1 metric and raise max_tokens

config/model.yaml | 2 +-
eval/f1.py          | 4 +++
2 files changed, 5 insertions(+), 1 deletion(-)
```

```
$ git rebase -i main
--- todo list as Git opened it (comment lines removed) ---
pick c1adfe1 # Add exact_match metric
pick 0527727 # Add F1 metric and raise max_tokens
pick 4394181 # Test both metrics
--- todo list as saved ---
pick c1adfe1 # Add exact_match metric
edit 0527727 # Add F1 metric and raise max_tokens
pick 4394181 # Test both metrics
Rebasing (2/3)
Stopped at 0527727... # Add F1 metric and raise max_tokens
You can amend the commit now, with

    git commit --amend

Once you are satisfied with your changes, run

    git rebase --continue
```

The rebase has replayed that commit and stopped. You are on a detached HEAD with the commit as the tip. To split it, take the commit back while keeping its changes in the working tree, and commit them in two portions:

```
$ git reset HEAD^
Unstaged changes after reset:
M   config/model.yaml
$ git status --short
M   config/model.yaml
?? eval/f1.py
$ git add eval/f1.py
```

```
$ git commit -m "Add token-level F1 metric"
[detached HEAD 47852a1] Add token-level F1 metric
1 file changed, 4 insertions(+)
 create mode 100644 eval/f1.py
$ git commit -a -m "Raise max_tokens to 1024 for long answers"
[detached HEAD 6501387] Raise max_tokens to 1024 for long answers
1 file changed, 1 insertion(+), 1 deletion(-)
```

```
$ git rebase --continue
Rebasing (3/3)
Successfully rebased and updated refs/heads/feat/metrics.
$ git log --oneline --decorate
d0c6470 (HEAD -> feat/metrics) Test both metrics
6501387 Raise max_tokens to 1024 for long answers
47852a1 Add token-level F1 metric
cladfe1 Add exact_match metric
8afc6bd (main) Add retriever and model config
```

`git reset HEAD^` is a mixed reset (Chapter 11): `HEAD` and the index go back one commit, the files stay. (`HEAD~1` names the same commit and is the safer spelling in shells that treat `^` as a pattern character.) It has a side effect that matters after the rebase:

```
# ORIG_HEAD no longer names the tip from before the rebase: "git reset" overwrote it.
$ git rev-parse --short ORIG_HEAD
0527727
$ git diff --stat ORIG_HEAD HEAD
 tests/test_metrics.py | 8 ++++++
1 file changed, 8 insertions(+)
# The relog of the branch still has the old tip. The two trees are identical, so this prints nothing:
$ git diff --stat feat/metrics@{1} feat/metrics
$ git relog show feat/metrics -2
d0c6470 feat/metrics@{0}: rebase (finish): refs/heads/feat/metrics onto 8afc6bd28c2572af09f1b6c37d733535230e526f
4394181 feat/metrics@{1}: commit: Test both metrics
```

`git reset` writes `ORIG_HEAD` too, so `ORIG_HEAD` no longer names the tip from before the rebase. The relog of the branch does.

`exec`: **test every commit**. A rewritten history is a series of snapshots that nobody has run. `--exec` adds an `exec` line after every commit:

```
$ git rebase --exec "sh scripts/check.sh" main
Rebasing (2/6)
Executing: sh scripts/check.sh
check passed
Rebasing (3/6)
Rebasing (4/6)
Executing: sh scripts/check.sh
eval/f1.py:3: print("DEBUG", p, g)
check failed: remove the debug print
warning: execution failed: sh scripts/check.sh
You can fix the problem, and then run
```

```
git rebase --continue
```

```
[exit status: 1]
```

```
$ git log --oneline --decorate -2
67c1620 (HEAD) Add token-level F1 metric
625fdb8 Add exact_match metric
$ cat .git/rebase-merge/git-rebase-todo
pick f73498cc94810e2909007a8eed1a685eedc687d # Test both metrics
exec sh scripts/check.sh
```

The check passed after the first commit and failed after the second. The rebase stopped there, with the failing commit as `HEAD` and the rest of the list waiting. Repair the commit and continue:

```
# In an editor: delete the DEBUG line from eval/f1.py. Then:
$ git diff --stat
eval/f1.py | 1 -
1 file changed, 1 deletion(-)
$ git commit -a --amend --no-edit
[detached HEAD 674e131] Add token-level F1 metric
Date: Mon Sep 7 10:05:00 2026 +0530
1 file changed, 4 insertions(+)
 create mode 100644 eval/f1.py
$ git rebase --continue
Rebasing (5/6)
Rebasing (6/6)
Executing: sh scripts/check.sh
check passed
Successfully rebased and updated refs/heads/feat/metrics.
```

`exec` is an ordinary line. With `-i --exec` you see the generated lines and may remove some. Here only the last one is kept:

```

$ git rebase -i --exec "sh scripts/check.sh" main
--- todo list as Git opened it (comment lines removed) ---
pick 625fdb8 # Add exact_match metric
exec sh scripts/check.sh
pick 67c1620 # Add token-level F1 metric
exec sh scripts/check.sh
pick f73498c # Test both metrics
exec sh scripts/check.sh
--- todo list as saved ---
pick 625fdb8 # Add exact_match metric
pick 67c1620 # Add token-level F1 metric
pick f73498c # Test both metrics
exec sh scripts/check.sh
Rebasing (4/4)
Executing: sh scripts/check.sh
eval/f1.py:3: print("DEBUG", p, g)
check failed: remove the debug print
warning: execution failed: sh scripts/check.sh
You can fix the problem, and then run

git rebase --continue

```

The check now fails at the tip. You learn that the branch is broken, not which commit broke it. That is the argument for one `exec` per commit.

`break` and changing the plan. `break` stops the rebase without touching a commit, so that you can look around:

```

$ git rebase -i main
--- todo list as Git opened it (comment lines removed) ---
pick d33e9b2 # Add exact_match metric
pick a1056e1 # wip
pick cdb5f2f # add test
pick 25dc48e # debug print
pick 031999d # Add token-level F1 metirc
pick 8313de0 # fix test
--- todo list as saved ---
pick d33e9b2 # Add exact_match metric
pick a1056e1 # wip
pick cdb5f2f # add test
break
pick 25dc48e # debug print
pick 031999d # Add token-level F1 metirc
pick 8313de0 # fix test
Rebasing (4/7)
Stopped at cdb5f2f (add test)

```

While any rebase is stopped, you can reopen the remaining instructions in the editor:

```
git rebase --edit-todo
```

```

$ git rebase --edit-todo
--- todo list as Git opened it (comment lines removed) ---
pick 25dc48e # debug print
pick 031999d # Add token-level F1 metirc
pick 8313de0 # fix test
--- todo list as saved ---
drop 25dc48e # debug print
pick 031999d # Add token-level F1 metirc
pick 8313de0 # fix test

```

```

$ git rebase --continue
Rebasing (5/7)
Rebasing (6/7)
Rebasing (7/7)
Successfully rebased and updated refs/heads/feat/metrics.
$ git log --oneline --decorate
2d1d481 (HEAD -> feat/metrics) fix test
c7c12a0 Add token-level F1 metirc
cdb5f2f add test
a1056e1 wip
d33e9b2 Add exact_match metric
061c92d (main) Add lint script
8afc6bd Add retriever and model config
$ sh scripts/check.sh
check passed

```

Only the instructions not yet executed are offered. What is done is done; to change that, abort and start again.

In production. The purpose of a tidy history is not beauty. `git bisect` (Chapter 14A) can find a regression only if every commit builds, and a `revert` removes one decision only if a commit contains one decision. A branch cleaned up with `git rebase -i --exec "<your test command>" main` has been proved commit by commit. The one line to remember from the comment block is the one in capitals: a deleted line is a lost commit, and Git does not ask. `rebase.missingCommitsCheck` turns that silence into a warning or an error (Lab 9.1).

9.7 Fixup commits and `--autosquash`

In one sentence. A fixup commit is a correction recorded now, as a commit of its own, whose subject line names the earlier commit it belongs to; `--autosquash` reads those subjects and arranges the to-do list for you.

Precisely.  `git commit` creates them with three kinds of subject (`git-commit`):

Created with	Subject of the new commit	Becomes in the list	Effect on the target
<code>git commit --fixup=<commit></code>	<code>fixup! <target subject></code>	<code>fixup</code>	Content corrected, message kept
<code>git commit --squash=<commit></code>	<code>squash! <target subject></code>	<code>squash</code>	Content corrected, editor opens on both messages
<code>git commit --fixup=amend:<commit></code>	<code>amend! <target subject></code>	<code>fixup -C</code>	Content corrected, message replaced
<code>git commit --fixup=reword:<commit></code>	<code>amend! <target subject></code>	<code>fixup -C</code>	Message replaced, content untouched

See it. Three review comments on a three-commit branch, each answered with a commit addressed to the commit it concerns:

```
# Review comment 1: exact_match must ignore surrounding whitespace. That belongs in the first commit.
$ git diff
diff --git a/eval/metrics.py b/eval/metrics.py
index a82fda1..652e0e2 100644
--- a/eval/metrics.py
+++ b/eval/metrics.py
@@ -1,2 +1,2 @@
 def exact_match(pred, gold):
-    return pred == gold
+    return pred.strip() == gold.strip()
$ git commit -a --fixup=HEAD~2
[feat/metrics 2c37c8c] fixup! Add exact_match metric
1 file changed, 1 insertion(+), 1 deletion(-)
```

```
# Review comment 2: cover disjoint answers in the F1 test. That belongs in the last commit, with a note.
$ git commit -a --squash=HEAD~1 -m "Also covers the zero-overlap case."
[feat/metrics 02094a9] squash! Add token-level F1 metric
1 file changed, 3 insertions(+)
```

```
# Review comment 3: the message of "Test exact_match" should say what is tested. Only the message changes.
$ git commit --fixup=reword:HEAD~3
[feat/metrics eaf99b7] amend! Test exact_match
```

```
$ git log --oneline --decorate
eaf99b7 (HEAD -> feat/metrics) amend! Test exact_match
02094a9 squash! Add token-level F1 metric
2c37c8c fixup! Add exact_match metric
a25e84c Add token-level F1 metric
375df0b Test exact_match
143e55a Add exact_match metric
8afc6bd (main) Add retriever and model config
```

```
$ git rebase -i --autosquash main
--- todo list as Git opened it (comment lines removed) ---
pick 143e55a # Add exact_match metric
fixup 2c37c8c # fixup! Add exact_match metric
pick 375df0b # Test exact_match
fixup -C eaf99b7 # amend! Test exact_match # empty
pick a25e84c # Add token-level F1 metric
squash 02094a9 # squash! Add token-level F1 metric
--- todo list as saved ---
pick 143e55a # Add exact_match metric
fixup 2c37c8c # fixup! Add exact_match metric
pick 375df0b # Test exact_match
fixup -C eaf99b7 # amend! Test exact_match # empty
pick a25e84c # Add token-level F1 metric
squash 02094a9 # squash! Add token-level F1 metric
Rebasing (2/6)
Rebasing (3/6)
Rebasing (4/6)
Rebasing (5/6)
Rebasing (6/6)
[detached HEAD 7f88f67] Add token-level F1 metric
Date: Mon Sep 7 10:05:00 2026 +0530
2 files changed, 11 insertions(+)
create mode 100644 eval/f1.py
Successfully rebased and updated refs/heads/feat/metrics.
```

The list arrives already arranged: every correction sits directly below its target with the right instruction, and `# empty` marks the commit that carries only a message. The editor opens once, for the `squash`.

```
$ git log --oneline --decorate
7f88f67 (HEAD -> feat/metrics) Add token-level F1 metric
8c07d2a Test exact_match on identical strings
6b055fc Add exact_match metric
```

```

8afc6bd (main) Add retriever and model config
$ git log -1 --format=%B
Add token-level F1 metric

Tested on identical and on disjoint answers.

$ git diff --stat ORIG_HEAD HEAD

```

Three commits again, with the corrected messages. The last command prints nothing: the tree at the tip is identical before and after. Autosquash changes how the history is cut into commits, not what the branch contains. Without `-i` the list is not shown at all:

```

# One more fix, this time without opening the todo list at all.
$ git commit -a --fixup=HEAD
[feat/metrics 06c9467] fixup! Add token-level F1 metric
1 file changed, 1 insertion(+), 1 deletion(-)
$ git rebase --autosquash main
Rebasing (4/4)
Successfully rebased and updated refs/heads/feat/metrics.
$ git log --oneline --decorate
dc93e0c (HEAD -> feat/metrics) Add token-level F1 metric
8c07d2a Test exact_match on identical strings
6b055fc Add exact_match metric
8afc6bd (main) Add retriever and model config

```

One trap in the configuration. `rebase.autoSquash` exists so that you need not type the option:

```

$ git config set rebase.autoSquash true
$ git log --oneline --decorate
028ba01 (HEAD -> feat/metrics) fixup! Add exact_match metric
a25e84c Add token-level F1 metric
375df0b Test exact_match
143e55a Add exact_match metric
8afc6bd (main) Add retriever and model config
$ git rebase main
Current branch feat/metrics is up to date.
$ git log --oneline --decorate -1
028ba01 (HEAD -> feat/metrics) fixup! Add exact_match metric

```

Root cause. The setting applies to interactive rebases only; the manual says "by default for interactive mode". A plain `git rebase main` on a branch that is already on top of `main` has nothing to do, reports "up to date", and leaves the `fixup!` commit in place. Type `--autosquash`, or use `-i`.

In production. During a review, answer each comment with a `fixup` commit and push normally: the reviewer sees every response as a small diff, and nothing is force-pushed meanwhile. Before the merge, one `git rebase -i --autosquash` and one force push fold everything. A CI job that fails while a subject starts with `fixup!`, `squash!` or `amend!` keeps unfolded corrections out of `main`; the check in Lab 9.3 is one line of `git log --grep`.

9.8 `--autostash`

In one sentence. A rebase refuses to start while tracked files have uncommitted changes; `--autostash` puts those changes into a stash commit, rebases, and applies the stash to the result.

See it.

```

# An uncommitted experiment in the working tree:
$ git status --short
M app/rerank.py
$ git rebase main
error: cannot rebase: You have unstaged changes.
error: Please commit or stash them.
[exit status: 1]

```

```

$ git rebase --autostash main
Created autostash: 855c10e
Rebasing (1/3)
Rebasing (2/3)
Rebasing (3/3)
Applied autostash.
Successfully rebased and updated refs/heads/feat/rerank.
$ git status --short
M app/rerank.py
$ git stash list

```

The edit is back in the working tree and the stash list is empty: an autostash is a stash commit (Chapter 14C) that is never put on the list. So where is your change while a rebase is stopped at a conflict?

```
$ git status --short
UU app/retriever.py
$ git stash list
$ cat .git/rebase-merge/autostash
a73072956efd7f13df50384c97d52782d1dc32af
$ git stash show -p $(cat .git/rebase-merge/autostash)
diff --git a/config/model.yaml b/config/model.yaml
index 2d94ebc..d0061c7 100644
--- a/config/model.yaml
+++ b/config/model.yaml
@@ -2,3 +2,4 @@ model: small-v1
  temperature: 0.2
  max_tokens: 512
  rerank: true
+top_p: 0.9
```

In no file of the working tree, and not on the stash list. Its ID is in the state directory. The two ways out treat it differently:

```
$ git rebase --abort
Applied autostash.
$ git status --short
M config/model.yaml
```

```
# The same stop again. This time the rebase is left with --quit.
$ git rebase --quit
Autostash exists; creating a new stash entry.
Your changes are safe in the stash.
You can run "git stash pop" or "git stash drop" at any time.
$ git stash list
stash@{0}: autostash
$ git status --short
UU app/retriever.py
```

The last step can itself conflict, when the rebase brought in a change to the lines you had edited:

```
# main gained a commit that rewrites the README sentence. You have an uncommitted edit to the same sentence.
$ git status --short
M README.md
$ git rebase --autostash main
Created autostash: 946f166
Rebasing (1/3)
Rebasing (2/3)
Rebasing (3/3)
Your local changes are stashed, however applying them
resulted in conflicts. You can either resolve the conflicts
and then discard the stash with "git stash drop", or, if you
do not want to resolve them now, run "git reset --hard" and
apply the local changes later by running "git stash pop".
Successfully rebased and updated refs/heads/feat/rerank.
```

```
$ git status --short
UU README.md
$ git stash list
stash@{0}: autostash
$ cat README.md
# ragkit

<<<<<< Updated upstream
Retrieval-augmented answering service with reranking.
=====
Retrieval-augmented answering service (internal).
>>>>>> Stashed changes
$ git log --oneline --decorate -2
20835dd (HEAD -> feat/rerank) Enable reranking in config
c994617 Call reranker from retriever
```

Read the last line of the rebase output first: the rebase succeeded and the branch has moved. What conflicted is the stash application afterwards. Git kept the stash on the list, and the markers are labelled `Updated upstream` (the rebased branch) and `Stashed changes` (your edit). Resolve the file, then `git stash drop`.

In production. `rebase.autoStash=true` makes this the default, and `git pull --rebase --autostash` does the same for pulls. The cost is that uncommitted work travels through a mechanism you do not see. If the change matters, commit it as work in progress before you rebase; a commit is in the reflog, an edit in the working tree is not.

9.9 Stacked branches and `--update-refs`

In one sentence. `--update-refs` makes a rebase move every local branch that points at one of the replayed commits, not only the branch you are on.

Precisely. For each such branch, Git adds a line `update-ref refs/heads/<name>` to the list, directly after the commit the branch points at, and sets the branch when the list is finished.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git rebase --update-refs <upstream></code>	As a plain rebase	As a plain rebase	As a plain rebase	As a plain rebase	Every local branch that pointed into the range is set to the corresponding copy, with the reflog message "rewritten during rebase". Not moved: branches checked out in another worktree, tags, remote-tracking branches	unchanged	unchanged

See it. Three branches, each built on the one below: a stack, the shape behind "stacked pull requests".

```
$ git log --oneline --graph --decorate --all
* 9cfa98f (main) Add README
| * e86ad55 (HEAD -> feat/ingest-chunker) Make chunk size configurable
| * f3f4739 Add chunker
| * b10b821 (feat/ingest-cleaner) Add text cleaner
| * 7fea52c (feat/ingest-loader) Close files after loading
| * 531d3da Add document loader
|/
* 8afc6bd Add retriever and model config
```

A plain rebase of the top branch:

```
$ git rebase main
Rebasing (1/5)
Rebasing (2/5)
Rebasing (3/5)
Rebasing (4/5)
Rebasing (5/5)
Successfully rebased and updated refs/heads/feat/ingest-chunker.
$ git log --oneline --graph --decorate --all
* a30ab1a (HEAD -> feat/ingest-chunker) Make chunk size configurable
* 59e5878 Add chunker
* 477c93c Add text cleaner
* 8b13775 Close files after loading
* 8e64e59 Add document loader
* 9cfa98f (main) Add README
| * b10b821 (feat/ingest-cleaner) Add text cleaner
| * 7fea52c (feat/ingest-loader) Close files after loading
| * 531d3da Add document loader
|/
* 8afc6bd Add retriever and model config
```

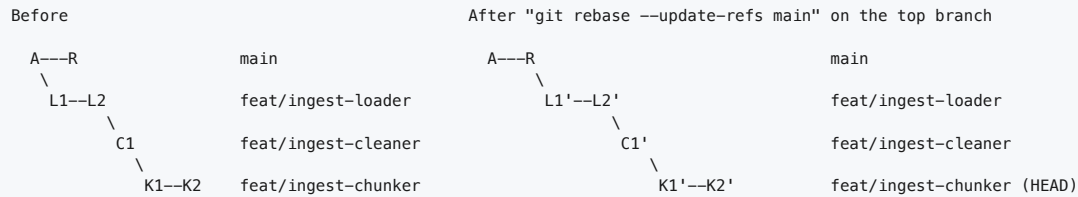
Only `feat/ingest-chunker` moved. The two lower branches still point at the old commits, and the top branch now contains private copies of their work. With the option, after going back to the start:

```
$ git rebase -i --update-refs main
--- todo list as Git opened it (comment lines removed) ---
pick 531d3da # Add document loader
pick 7fea52c # Close files after loading
update-ref refs/heads/feat/ingest-loader
pick b10b821 # Add text cleaner
update-ref refs/heads/feat/ingest-cleaner
pick f3f4739 # Add chunker
pick e86ad55 # Make chunk size configurable
--- todo list as saved ---
pick 531d3da # Add document loader
pick 7fea52c # Close files after loading
update-ref refs/heads/feat/ingest-loader
pick b10b821 # Add text cleaner
update-ref refs/heads/feat/ingest-cleaner
pick f3f4739 # Add chunker
pick e86ad55 # Make chunk size configurable
Rebasing (1/7)
Rebasing (2/7)
Rebasing (3/7)
Rebasing (4/7)
Rebasing (5/7)
Rebasing (6/7)
Rebasing (7/7)
Successfully rebased and updated refs/heads/feat/ingest-chunker.
Updated the following refs with --update-refs:
    refs/heads/feat/ingest-cleaner
    refs/heads/feat/ingest-loader
```

```
$ git log --oneline --graph --decorate --all
* 4b52940 (HEAD -> feat/ingest-chunker) Make chunk size configurable
* c439cc8 Add chunker
* 06c29f9 (feat/ingest-cleaner) Add text cleaner
* 1599a67 (feat/ingest-loader) Close files after loading
* eed2aa8 Add document loader
* 9cfa98f (main) Add README
```

```
* 8afc6bd Add retriever and model config
$ git rebase --onto feat/ingest-loader -2
1599a67 feat/ingest-loader@{0}: rewritten during rebase
7fea52c feat/ingest-loader@{1}: commit: Close files after loading
```

Picture.



The option moves every branch in the range, including one you did not think of as part of the stack:

```
# A "backup" branch that points into the range is a branch like any other:
$ git branch backup/chunker
$ git tag before-rebase
$ git rebase --update-refs main
Rebasing (1/8)
Rebasing (2/8)
Rebasing (3/8)
Rebasing (4/8)
Rebasing (5/8)
Rebasing (6/8)
Rebasing (7/8)
Rebasing (8/8)
Successfully rebased and updated refs/heads/feat/ingest-chunker.
Updated the following refs with --update-refs:
  refs/heads/backup/chunker
  refs/heads/feat/ingest-cleaner
  refs/heads/feat/ingest-loader
$ git log --oneline --decorate -1 backup/chunker
2925218 (HEAD -> feat/ingest-chunker, backup/chunker) Make chunk size configurable
$ git log --oneline --decorate -1 before-rebase
4b52940 (tag: before-rebase) Make chunk size configurable
```

Root cause. A "backup" made with `git branch` is a branch that points into the rebased range, so `--update-refs` moved it along with the others, and it no longer backs anything up. The tag stayed. Before a risky rebase, mark the old tip with a tag, or write down the ID, or rely on the reflog.

The undo is also per branch. 🛑 `git reset --hard ORIG_HEAD` (section 9.16) restores the branch you are on; each of the others has to be put back from its own reflog (Lab 9.7). Publishing is per branch as well, in one command, where every ref gets its own lease (section 9.17):

```
$ git push --force-with-lease --force-if-includes origin feat/ingest-loader feat/ingest-cleaner feat/ingest-chunker
To ../server.git
+ 31e0e7e...82bd9b8 feat/ingest-chunker -> feat/ingest-chunker (forced update)
+ b10b821...3616f79 feat/ingest-cleaner -> feat/ingest-cleaner (forced update)
+ 7fea52c...129c95a feat/ingest-loader -> feat/ingest-loader (forced update)
```

In production. A large change reviewed as a chain of small pull requests lives or dies by this option: every review fix in the lowest branch has to ripple upward. `git config set rebase.updateRefs true` makes it the default.

9.10 Branches that contain merges: `--rebase-merges`

In one sentence. By default a rebase leaves merge commits out and lays all other commits out in one line; `--rebase-merges` rebuilds the branch structure, merge commits included.

See it. `feat/ingest` contains a merge of a side branch:

```
$ git log --oneline --graph --decorate --all
* 9cfa98f (main) Add README
| * 75c9283 (HEAD -> feat/ingest) Wire loader, cleaner and chunker together
| * 7537190 Merge branch 'feat/ingest-cleaner' into feat/ingest
| |
| | * dc5df93 (feat/ingest-cleaner) Add text cleaner
| | * 9a23ecd Add chunker
| |
| |
| * 1279adf Add document loader
|/
* 8afc6bd Add retriever and model config
```

```
$ git rebase main
Rebasing (1/4)
```



```

Rebasing (2/4)
Rebasing (3/4)
Rebasing (4/4)
Successfully rebased and updated refs/heads/feat/ingest.
$ git log --oneline --graph --decorate feat/ingest
* c623381 (HEAD -> feat/ingest) Wire loader, cleaner and chunker together
* f4b3cef Add text cleaner
* 043ec63 Add chunker
* a10544a Add document loader
* 9cfa98f (main) Add README
* 8afc6bd Add retriever and model config

```

Four commits in a line; the merge commit is gone. Whatever existed only in the merge commit, such as a conflict resolution or a fix made while merging, is not replayed, so the conflict it resolved comes back at one of the single commits. With `-r`, after going back:

```

$ git rebase -i --rebase-merges main
--- todo list as Git opened it (comment lines removed) ---
label onto
reset onto
pick 1279adf # Add document loader
label branch-point
pick dc5df93 # Add text cleaner
label feat-ingest-cleaner
reset branch-point # Add document loader
pick 9a23ecd # Add chunker
merge -C 7537190 feat-ingest-cleaner # Merge branch 'feat/ingest-cleaner' into feat/ingest
pick 75c9283 # Wire loader, cleaner and chunker together
--- todo list as saved ---
label onto
reset onto
pick 1279adf # Add document loader
label branch-point
pick dc5df93 # Add text cleaner
label feat-ingest-cleaner
reset branch-point # Add document loader
pick 9a23ecd # Add chunker
merge -C 7537190 feat-ingest-cleaner # Merge branch 'feat/ingest-cleaner' into feat/ingest
pick 75c9283 # Wire loader, cleaner and chunker together
Rebasing (1/10)
Rebasing (2/10)
Rebasing (3/10)
Rebasing (4/10)
Rebasing (5/10)
Rebasing (6/10)
Rebasing (7/10)
Rebasing (8/10)
Rebasing (9/10)
Rebasing (10/10)
Successfully rebased and updated refs/heads/feat/ingest.

```

The list has become a small graph program. `label onto` names the new base. `reset onto` moves HEAD there. After the first pick, `label branch-point` remembers where the side branch forks. The side commit is picked and labelled. `reset branch-point` goes back, the mainline commit is picked, and `merge -C 7537190 feat-ingest-cleaner` makes a new merge with the message of the old one. Labels are temporary refs under `refs/rewritten/`, deleted when the rebase ends.

```

$ git log --oneline --graph --decorate --all
* a52bef6 (HEAD -> feat/ingest) Wire loader, cleaner and chunker together
* 5fff212 Merge branch 'feat/ingest-cleaner' into feat/ingest
|\
| * 3452235 Add text cleaner
* | 7187f81 Add chunker
|/
* 8cf8070 Add document loader
* 9cfa98f (main) Add README
| * dc5df93 (feat/ingest-cleaner) Add text cleaner
| * 1279adf Add document loader
|/
* 8afc6bd Add retriever and model config

```

Two details. The branch `feat/ingest-cleaner` still points at the old commit `dc5df93`: `-r` rebuilds commits, it does not move other branches; add `--update-refs` for that. And the new merge is a fresh merge: the manual states that conflict resolutions and manual amendments in the original merge commits have to be redone by hand. Recorded resolutions (`rerere`, Chapter 14C) reduce that work.

```

$ git rebase --preserve-merges main
fatal: --preserve-merges was replaced by --rebase-merges
Note: Your 'pull.rebase' configuration may also be set to 'preserve',
which is no longer supported; use 'merges' instead
[exit status: 128]

```

Outdated advice. `git rebase -p` or `--preserve-merges`, and `pull.rebase=preserve`, appear in older tutorials. The option was deprecated in Git 2.22 and removed in 2.34; Git 2.55 answers with the fatal message above. Use `--rebase-merges` and `pull.rebase=merges` ([git-rebase](#)).

In production. When a branch contains merges, first ask whether it should be rebased at all. A long-lived integration branch that others pull from should not. On your own topic branch into which you merged `main` twice, the plain, flattening rebase is usually what you want.

9.11 Conflicts during a rebase: who is "ours"

In one sentence. During a rebase, "ours" is the new base plus the copies made so far, and "theirs" is your own commit, the one being replayed: the opposite of what the words suggest.

Analogy. A rebase checks out the other side's latest commit and then merges your commits into it, one at a time. In each of those merges the house belongs to the other side and your commit is the visitor. The analogy is almost literal; it breaks only if you think of "ours" as a person. It is a position: whatever HEAD is.

Precisely. Each replay is a three-way merge into HEAD (Chapter 8, section 8.8, explains the stages). Stage 2, "ours", is HEAD: the upstream commit plus what has been replayed. Stage 3, "theirs", is `REBASE_HEAD`, your commit. Stage 1 is the parent of your commit. The local manual says it in one line: "the sides are swapped".

See it. The stopped rebase of section 9.4 again:

```
# The rebase of section 9.4 has stopped at the same commit again.
$ cat app/retriever.py
<<<<<< HEAD
TOP_K = 8
=====
TOP_K = 20
>>>>>> 569e6c9 (Fetch 20 candidates for the reranker)

def retrieve(query):
    return rerank(search(query, TOP_K))

$ git ls-files -u
100644 35e331a734226399bf019e7226f303fcb01eeabb 1      app/retriever.py
100644 ef8c46811d15d96c680eb865313544b9f5680026 2      app/retriever.py
100644 e67ee8b37c4aa0f84cbd0a310f0a7cc7f7c02f00 3      app/retriever.py
$ git show :1:app/retriever.py | head -1
TOP_K = 5
$ git show :2:app/retriever.py | head -1
TOP_K = 8
$ git show :3:app/retriever.py | head -1
TOP_K = 20

# Stage 2, "ours", is HEAD: the upstream commits plus what has been replayed so far.
$ git log --oneline -2 HEAD
1c9f69a Add reranker skeleton
439e4c6 Raise TOP_K to 8 after recall regression
# Stage 3, "theirs", is the commit being replayed: your own work.
$ git log --oneline -1 REBASE_HEAD
569e6c9 Fetch 20 candidates for the reranker
```

The upper half of the markers, labelled `HEAD`, is the value from `main`. The lower half, labelled with the ID and subject of your commit, is your value. Now integrate the same two branches with a merge, from the same branch:

```
# The same two branches integrated with a merge instead (run from feat/rerank):
$ git merge main
Auto-merging app/retriever.py
CONFLICT (content): Merge conflict in app/retriever.py
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ cat app/retriever.py
<<<<<< HEAD
TOP_K = 20
=====
TOP_K = 8
>>>>>> main

def retrieve(query):
    return rerank(search(query, TOP_K))
$ git show :2:app/retriever.py | head -1
TOP_K = 20
$ git show :3:app/retriever.py | head -1
TOP_K = 8
$ git merge --abort
```

Same two values, opposite positions.

	git merge main, run on your branch	git rebase main, run on your branch
<<<<<< HEAD, stage 2, --ours, -X ours	your branch	main plus your commits replayed so far
>>>>>>, stage 3, --theirs, -X theirs	main	your commit being replayed
Stage 1, the base	the merge base of the two branches	the parent of your commit

	git merge main, run on your branch	git rebase main, run on your branch
How often it can stop	once	once per replayed commit

To resolve, put the content you want into the file, stage it, and continue. Taking your own version whole is `--theirs` here (`git checkout --theirs -- <path>` in older scripts):

```
# Back in the stopped rebase. Take the version of the commit being replayed (stage 3):
$ git restore --theirs app/retriever.py
$ cat app/retriever.py
TOP_K = 20

def retrieve(query):
    return rerank(search(query, TOP_K))
$ git add app/retriever.py
$ git status --short
M app/retriever.py
```

```
$ git rebase --continue
[detached HEAD 6dc07b7] Fetch 20 candidates for the reranker
1 file changed, 2 insertions(+), 2 deletions(-)
Rebasing (3/3)
Successfully rebased and updated refs/heads/feat/rerank.
$ git log --oneline --graph --decorate --all
* a4749c1 (HEAD -> feat/rerank) Enable reranking in config
* 6dc07b7 Fetch 20 candidates for the reranker
* b7e0843 Add reranker skeleton
* 439e4c6 (main) Raise TOP_K to 8 after recall regression
* 8afc6bd Add retriever and model config
$ git reflog -5
a4749c1 HEAD@{0}: rebase (finish): returning to refs/heads/feat/rerank
a4749c1 HEAD@{1}: rebase (pick): Enable reranking in config
6dc07b7 HEAD@{2}: rebase (continue): Fetch 20 candidates for the reranker
b7e0843 HEAD@{3}: rebase (pick): Add reranker skeleton
439e4c6 HEAD@{4}: rebase (start): checkout main
```

`--continue` commits the index as the copy of the stopped commit, opening your editor on its message first (the scripted editor accepts it), and runs the remaining instructions. The first copy is `b7e0843` here and was `1c9f69a` above: the demo restarted the rebase after the merge experiment, a few lab minutes later, and a later committer date gives another ID. If you know in advance which side should win every conflicting hunk, say so and the rebase does not stop:

```
# The same rebase, telling the merge machinery in advance to prefer "theirs" in conflicting hunks:
$ git rebase -X theirs main
Rebasing (1/3)
Rebasing (2/3)
Rebasing (3/3)
Successfully rebased and updated refs/heads/feat/rerank.
$ head -1 app/retriever.py
TOP_K = 20
```

`-X theirs` favours your commits. `-X ours` would favour `main` and discard your side of every conflicting hunk without a message.

The ways out of a stop.

Command	Effect
 <code>git rebase --continue</code>	Commit the staged resolution as the copy and go on
 <code>git rebase --skip</code>	Make no copy of this commit and go on
 <code>git rebase --abort</code>	Put branch, index and working tree back as they were before the rebase
 <code>git rebase --quit</code>	Forget the rebase and leave HEAD, index and working tree as they are now
 <code>git rebase --show-current-patch</code>	Show the commit being replayed; the same as <code>git show REBASE_HEAD</code>

`--abort` was shown in section 9.4. `--skip` gives a branch without the commit, and with the value from `main`:

```
$ git rebase --skip
Rebasing (3/3)
Successfully rebased and updated refs/heads/feat/rerank.
$ git log --oneline --decorate main..HEAD
c4f824e (HEAD -> feat/rerank) Enable reranking in config
0c83908 Add reranker skeleton
$ cat app/retriever.py
TOP_K = 8

def retrieve(query):
    return search(query, TOP_K)
```

`--quit` removes the state directory and nothing else:

```
$ git rebase --quit
$ git status
```

```
HEAD detached from refs/heads/feat/rerank
Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
    both modified:   app/retriever.py

no changes added to commit (use "git add" and/or "git commit -a")
$ git log --oneline --graph --decorate --all
* c92ee53 (HEAD) Add reranker skeleton
* 439e4c6 (main) Raise TOP_K to 8 after recall regression
| * ad106e3 (feat/rerank) Enable reranking in config
| * 569e6c9 Fetch 20 candidates for the reranker
| * 5ee19f0 Add reranker skeleton
|/
* 8afc6bd Add retriever and model config
```

The branch is untouched, HEAD is detached on the half-built history, and the index still holds the conflict. Use it when you want to keep that state and take over by hand. And if you try to move on before the conflict is resolved, or start a second rebase:

```
$ git rebase --continue
app/retriever.py: needs merge
You must edit all merge conflicts and then
mark them as resolved using git add
[exit status: 1]
```

```
$ git rebase main
fatal: It seems that there is already a rebase-merge directory, and
I wonder if you are in the middle of another rebase. If that is the
case, please try
    git rebase (--continue | --abort | --skip)
If that is not the case, please
    rm -fr ".git/rebase-merge"
and run me again. I am stopping in case you still have something
valuable there.

[exit status: 128]
```

Take the first suggestion of that message, never the `rm -fr`, unless `git status` has told you there is nothing to keep.

Two traps. The first one is the swap itself:

```
# The rebase is stopped at "Fetch 20 candidates for the reranker". You want your version, so you type:
$ git restore --ours app/retriever.py
$ git add app/retriever.py
$ git status --short
$ git rebase --continue
Rebasing (3/3)
Successfully rebased and updated refs/heads/feat/rerank.
```

```
$ git log --oneline --decorate main..HEAD
7c7ac13 (HEAD -> feat/rerank) Enable reranking in config
0c83908 Add reranker skeleton
$ git range-diff main ORIG_HEAD HEAD
1: 5ee19f0 = 1: 0c83908 Add reranker skeleton
2: 569e6c9 < -: ----- Fetch 20 candidates for the reranker
3: ad106e3 = 2: 7c7ac13 Enable reranking in config
```

Observed behavior	: The rebase finishes normally, and "Fetch 20 candidates for the reranker" is not in the branch.
Git state	: At the stop, stage 2 ("ours") was main's version of the file. After restore and add, the index equals HEAD: "git status --short" printed nothing.
Mechanism	: --continue found nothing to commit for this instruction. A commit that has become empty is dropped (the default, --empty=drop), and the next instruction runs. No message.
Root cause	: "ours" was read as "my change". In a rebase it is the side you are rebasing onto.
Why Git does this	: A replay with an empty result normally means the change is already upstream, which is the common and harmless case (section 9.12).
Correct fix	: git reset --hard ORIG_HEAD, rebase again, take --theirs or edit by hand.
Prevention	: Read "git status" before --continue. Run "git range-diff" after every rebase that stopped.

The second trap is a habit from ordinary work:

```
# Stopped at the same commit. This time the resolution is right, but the next command is not:
$ git restore --theirs app/retriever.py
$ git add app/retriever.py
$ git commit --amend --no-edit
[detached HEAD 1e6f47e] Add reranker skeleton
Date: Mon Sep 7 10:03:00 2026 +0530
2 files changed, 4 insertions(+), 2 deletions(-)
create mode 100644 app/rerank.py
$ git rebase --continue
```

```
Rebasing (3/3)
Successfully rebased and updated refs/heads/feat/rerank.
```

```
$ git log --oneline --decorate main..HEAD
036744c (HEAD -> feat/rerank) Enable reranking in config
1e6f47e Add reranker skeleton
$ git show --stat --format="%h %s" HEAD~1
1e6f47e Add reranker skeleton

app/rerank.py      | 2 ++
app/retriever.py   | 4 ++--
2 files changed, 4 insertions(+), 2 deletions(-)
```

At a conflict stop, HEAD is the previous copy. `git commit --amend` therefore folded the resolution into "Add reranker skeleton", which now changes two files, and the stopped commit, left with nothing, was dropped. After a conflict the sequence is `git add`, then `git rebase --continue`. `--amend` belongs to an `edit` stop, where Git itself suggests it.

In production. In Julia Evans's 2024 polls, 48% of 1,511 respondents did not know that the two sides swap between merge and rebase, and 61% of about 1,480 had seen a production bug caused by a bad conflict resolution; both samples are self-selected ([poll results](#)). Assume the confusion exists on your team. A branch of twelve commits that all touch the lines `main` changed can stop twelve times. Reduce the number of replays first (squash with `--keep-base`), or record resolutions with `rerere` (Chapter 14C), or integrate with one merge.

9.12 Commits that are already upstream

In one sentence. A rebase leaves out commits whose change the new base already contains, and it notices in two different ways: before replaying, by patch ID, and after replaying, by an empty result.

See it. First case: `main` received the middle commit of your branch through a cherry-pick.

```
$ git log --oneline --graph --decorate --all
* b0f4913 (main) Retry flaky downloads three times
* b0d0bef Add README
| * cb69662 (HEAD -> feat/ingest) Add chunker
| * 3667d58 Retry flaky downloads three times
| * 2c4fb0f Add document loader
|/
* 3b687c6 Add ingest settings
* 8afc6bd Add retriever and model config
```

```
$ git cherry -v main feat/ingest
+ 2c4fb0f9d67529c6c5e0593806e7b7e3ae166e1b Add document loader
- 3667d58e78b9db6022247e4772f7514a2eb4cd8f Retry flaky downloads three times
+ cb696620666fc31778e69cac508a0e92db27af29 Add chunker
$ git log --oneline --left-right --cherry-mark main...feat/ingest
= b0f4913 Retry flaky downloads three times
< b0d0bef Add README
> cb69662 Add chunker
= 3667d58 Retry flaky downloads three times
> 2c4fb0f Add document loader
```

`git cherry` marks with `-` each of your commits that has an equivalent upstream and with `+` each that has none. `git log --cherry-mark` puts `=` on both members of the pair. Both compare patch IDs (section 9.3), and the rebase runs the same comparison before it writes the list:

```
$ git rebase main
warning: skipped previously applied commit 3667d58
hint: use --reapply-cherry-picks to include skipped commits
hint: Disable this message with "git config set advice.skippedCherryPicks false"
Rebasing (1/2)
Rebasing (2/2)
Successfully rebased and updated refs/heads/feat/ingest.
$ git log --oneline --graph --decorate --all
* a329fe1 (HEAD -> feat/ingest) Add chunker
* b0b024d Add document loader
* b0f4913 (main) Retry flaky downloads three times
* b0d0bef Add README
* 3b687c6 Add ingest settings
* 8afc6bd Add retriever and model config
```

Second case: your first two commits reached `main` as one squashed commit. No single commit upstream has the diff of either of yours, so the comparison finds nothing:

```
$ git log --oneline --graph --decorate --all
* dd84d96 (main) Add document loader with download retries (#52)
* b0d0bef Add README
| * cb69662 (HEAD -> feat/ingest) Add chunker
| * 3667d58 Retry flaky downloads three times
| * 2c4fb0f Add document loader
|/
* 3b687c6 Add ingest settings
* 8afc6bd Add retriever and model config
```

```
$ git cherry -v main feat/ingest
+ 2c4fb0f9d67529c6c5e0593806e7b7e3ae166e1b Add document loader
+ 3667d58e78b9db6022247e4772f7514a2eb4cd8f Retry flaky downloads three times
+ cb696620666fc31778e69cac508a0e92db27af29 Add chunker
```

```
$ git rebase main
Rebasing (1/3)
dropping 2c4fb0f9d67529c6c5e0593806e7b7e3ae166e1b Add document loader -- patch contents already upstream
Rebasing (2/3)
dropping 3667d58e78b9db6022247e4772f7514a2eb4cd8f Retry flaky downloads three times -- patch contents already upstream
Rebasing (3/3)
Successfully rebased and updated refs/heads/feat/ingest.
$ git log --oneline --graph --decorate --all
* a499222 (HEAD -> feat/ingest) Add chunker
* dd84d96 (main) Add document loader with download retries (#52)
* b0d0bef Add README
* 3b687c6 Add ingest settings
* 8afc6bd Add retriever and model config
```

Both commits were replayed, both changed nothing, because their content is already there, and both were dropped as empty. That is the second mechanism, governed by `--empty: drop` is the default, `keep` records an empty commit, `stop` pauses so that you decide. With `-i` the default is `stop`. `--reapply-cherry-picks` switches the first mechanism off; the commit is then replayed and caught by the second:

```
$ git rebase --reapply-cherry-picks main
Rebasing (1/3)
Rebasing (2/3)
dropping 3667d58e78b9db6022247e4772f7514a2eb4cd8f Retry flaky downloads three times -- patch contents already upstream
Rebasing (3/3)
Successfully rebased and updated refs/heads/feat/ingest.
$ git log --oneline --decorate main..HEAD
fb8c073 (HEAD -> feat/ingest) Add chunker
b58e077 Add document loader
```

Where it stops working. "Already upstream" means the same diff, or a replay with an empty result. If the upstream version differs from yours, for instance because a reviewer's change went into the squash, neither mechanism applies and the replay conflicts. That was case 1 of section 9.5, and the answer there was to name the boundary with `--onto`.

In production. After a pull request is merged with "Squash and merge", a plain `git rebase main` on the follow-up branch does the right thing exactly when the squash equals the sum of your commits. Check with `git cherry -v main` first: for lines marked `+` that you believe are merged, expect an empty replay or a conflict.

9.13 `git pull --rebase`

In one sentence. 🟡 `git pull --rebase` is `git fetch` followed by a rebase of your unpushed commits onto the fetched upstream branch.

See it. You committed on `main`; Asha pushed to `main` in the meantime:

```
$ git fetch
From ../server
 8afc6bd..eb7188f main -> origin/main
$ git status --short --branch
## main...origin/main [ahead 1, behind 1]
$ git log --oneline --graph --decorate --all
* 139c7e6 (HEAD -> main) Raise max_tokens to 1024 for long answers
| * eb7188f (origin/main, origin/HEAD) Add README
|/
* 8afc6bd Add retriever and model config
```

A plain `git pull` refuses here until you choose how to reconcile (Chapter 12, section 12.6). Choosing rebase:

```
$ git pull --rebase
Rebasing (1/1)
Successfully rebased and updated refs/heads/main.
$ git log --oneline --graph --decorate --all
* ad181ff (HEAD -> main) Raise max_tokens to 1024 for long answers
* eb7188f (origin/main, origin/HEAD) Add README
* 8afc6bd Add retriever and model config
$ git reflog -4
ad181ff HEAD@{0}: pull --rebase (finish): returning to refs/heads/main
ad181ff HEAD@{1}: pull --rebase (pick): Raise max_tokens to 1024 for long answers
eb7188f HEAD@{2}: pull --rebase (start): checkout eb7188fc630b934cc36f4a355cc5e9c51f2540aa
139c7e6 HEAD@{3}: commit: Raise max_tokens to 1024 for long answers
```

```
$ git push
To ../server.git
 eb7188f..ad181ff main -> main
```

Your commit `139c7e6` was replayed as `ad181ff` on top of Asha's, the reflog labels the steps `pull --rebase`, and the push is a fast-forward for the server.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git pull --rebase</code>	Upstream tip plus your replayed commits	Same	Detached during the rebase, then back on the branch	Upstream tip plus copies of your unpushed commits	Remote-tracking branches and <code>FETCH_HEAD</code> updated by the fetch; <code>ORIG_HEAD</code> ; reflogs	Read, not changed	unchanged

This is history rewriting, and it is harmless here for one reason: the only commits rewritten are ones that exist nowhere but in your clone. `pull.rebase=true` makes it the default (section 9.19). When the upstream branch itself was rewritten, `git pull --rebase` consults the reflog of the remote-tracking branch to avoid replaying commits that upstream discarded. Sections 9.15 and 9.17 show that logic doing good and doing harm.

9.14 Reviewing a rebase with `git range-diff`

In one sentence.  `git range-diff` compares two versions of a series of commits: it pairs each old commit with its new counterpart and shows how the two patches differ.

Precisely. The most convenient spelling is `git range-diff <base> <old tip> <new tip>` (`git-range-diff`). Commits are paired by similarity of their patches, not by position or ID. Between the two columns stands `=` (same patch), `!` (patch changed), `<` (only in the old series) or `>` (only in the new one). For a `!` pair Git prints a diff of the two diffs: the outer marker says whether a line belongs to the old (`-`) or the new (`+`) patch, the inner marker is the patch's own.

See it. A rebase with one conflict, resolved with a value that neither side had:

```
$ git log --oneline --decorate main..feat/rerank
1388956 (HEAD -> feat/rerank) Enable reranking in config
bad8965 Fetch 20 candidates and filter weak matches
5ee19f0 Add reranker skeleton
# git rebase main stopped with a conflict in app/retriever.py: main says TOP_K = 8, your commit says 20.
# You settle on 12 in the editor, then:
$ git add app/retriever.py
$ git rebase --continue
[detached HEAD 54541c3] Fetch 20 candidates and filter weak matches
2 files changed, 6 insertions(+), 3 deletions(-)
Rebasing (3/3)
Successfully rebased and updated refs/heads/feat/rerank.
```

```
$ git range-diff main ORIG_HEAD HEAD
1: 5ee19f0 = 1: 1c9f69a Add reranker skeleton
2: bad8965 ! 2: 54541c3 Fetch 20 candidates and filter weak matches
@@ app/rerank.py

    ## app/retriever.py ##
    @@
    --TOP_K = 5
    +-TOP_K = 20
    +-TOP_K = 8
    ++TOP_K = 12

    def retrieve(query):
        - return search(query, TOP_K)
3: 1388956 = 3: 2468748 Enable reranking in config
```

Commits 1 and 3 are unchanged as patches. Commit 2 is not. Read the four marked lines in pairs: the old patch turned `TOP_K = 5` into `20`; the new patch turns `8` into `12`. Same title, different change. This is the answer to the second question of section 9.1, and it is invisible in a list of commit titles.

```
$ git range-diff main feat/rerank@{1} feat/rerank | head -3
1: 5ee19f0 = 1: 1c9f69a Add reranker skeleton
2: bad8965 ! 2: 54541c3 Fetch 20 candidates and filter weak matches
@@ app/rerank.py
$ git range-diff ORIG_HEAD...HEAD | head -4
-: ----- > 1: 439e4c6 Raise TOP_K to 8 after recall regression
1: 5ee19f0 = 2: 1c9f69a Add reranker skeleton
2: bad8965 ! 3: 54541c3 Fetch 20 candidates and filter weak matches
@@ app/rerank.py
```

The reflog form needs no `ORIG_HEAD`. The three-dot form has no base, so the commit that came from `main` is reported as new. Why not compare the two tips directly?

```
$ git diff ORIG_HEAD HEAD
diff --git a/app/retriever.py b/app/retriever.py
index e67ee8b..4321be2 100644
--- a/app/retriever.py
+++ b/app/retriever.py
@@ -1,4 +1,4 @@
-TOP_K = 20
+TOP_K = 12
```

```
def retrieve(query):
    return rerank(search(query, TOP_K))
```

That is a comparison of two snapshots. Here it happens to show the resolution; on a real branch it also contains everything that arrived from upstream. Range-diff compares the commits themselves.

The reviewer does not need your `ORIG_HEAD`. After a fetch, the previous tip is in the reflog of the reviewer's own remote-tracking branch:

```
# In the reviewer clone. The author has rebased feat/ingest onto main and force-pushed it.
$ git fetch
From ../server
 8afc6bd..589d18b  main      -> origin/main
+ dc5df93...321b338  feat/ingest -> origin/feat/ingest (forced update)
# The remote-tracking branch before and after that fetch:
$ git rev-parse "origin/feat/ingest@{1}" origin/feat/ingest
dc5df9356273e7c0c2807ce808c378782ea2525a
321b33811c66cc02a14c52dbe5b64ecdbd9c506d
```

```
$ git range-diff origin/main "origin/feat/ingest@{1}" origin/feat/ingest
1: 1279adf = 1: 5859371 Add document loader
2: dc5df93 = 2: 321b338 Add text cleaner
```

Two `=` lines: nothing changed except the base, and the earlier review still stands.

A limit to know. The pairing is a heuristic. Run the tool on the clean rebase of section 9.2, in which nothing was edited:

```
# The clean rebase of section 9.2. Nothing was edited, and yet:
$ git range-diff main ORIG_HEAD HEAD
1: 5ee19f0 = 1: 742ab58 Add reranker skeleton
2: af65a92 = 2: ade2990 Call reranker from retriever
3: bd62876 < -: ----- Enable reranking in config
-: ----- > 3: 976a161 Enable reranking in config
```

The third commit is reported as removed and added. Its patch is one added line with three lines of context, and one of those context lines changed underneath it: `main` had moved the model from `small-v1` to `small-v2`. For so small a patch that is too large a difference, and range-diff declines to pair the two. Raise the tolerance and the pair appears, with the context line as its only difference:

```
$ git range-diff --creation-factor=90 main ORIG_HEAD HEAD
1: 5ee19f0 = 1: 742ab58 Add reranker skeleton
2: af65a92 = 2: ade2990 Call reranker from retriever
3: bd62876 ! 3: 976a161 Enable reranking in config
@@ Commit message

    ## config/model.yaml ##
@@
- model: small-v1
+ model: small-v2
  temperature: 0.2
  max_tokens: 512
+rerank: true
```

`<` and `>` under one subject are a prompt to look closer. A lost commit has `<` and no partner.

In production. Run it after every rebase that stopped, before you push, and paste the output into the pull request when you force-push a branch under review. The format is for people; the manual warns that it may change, so do not parse it.

9.15 Rebasing a branch that other people use

In one sentence. If you rebase a branch that someone else has built on and publish the result, their clone holds the old commits, the server holds the new ones, and Git sees two separate lines of history that happen to make the same changes.

Analogy. You reprint chapters 1 and 2 of a shared manuscript with corrections and replace the copies in the office. Your co-author is at home with chapter 3 stapled to the old chapters 1 and 2. If she staples the new print on as well, the manuscript contains chapters 1 and 2 twice. The analogy breaks at one point: a person would recognise the duplicate text, and Git, which identifies commits by ID, does not, unless you ask it to compare patches.

See it. A bare repository plays the server; `you/` and `asha/` are two clones. You pushed two commits to `feat/ingest`. Asha fetched them and committed "Add chunker" on top, not yet pushed. Now you rebase onto the new `main` and publish:

```
$ git log --oneline --graph --decorate --all
* 589d18b (origin/main, main) Add README
| * dc5df93 (HEAD -> feat/ingest, origin/feat/ingest) Add text cleaner
| * 1279adf Add document loader
|/
* 8afc6bd Add retriever and model config
$ git rebase main
Rebasing (1/2)
```



```
Rebasing (2/2)
Successfully rebased and updated refs/heads/feat/ingest.
```

```
$ git push
To ../server.git
! [rejected]        feat/ingest -> feat/ingest (non-fast-forward)
error: failed to push some refs to '../server.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
[exit status: 1]
$ git push --force-with-lease --force-if-includes
To ../server.git
+ dc5df93...8aca274 feat/ingest -> feat/ingest (forced update)
```

The plain push is rejected because the server's tip is not an ancestor of yours. The hint recommends `git pull`. After a rebase of your own that advice is wrong: a pull would bring the old commits back, merged next to your copies or, with `--rebase`, in place of them. The second command replaces the server's branch; `+` and the three dots mean a forced, non-fast-forward update. In Asha's clone:

```
$ cd ../asha
$ git log --oneline --graph --decorate --all
* c60bc13 (HEAD -> feat/ingest) Add chunker
* dc5df93 (origin/feat/ingest) Add text cleaner
* 1279adf Add document loader
* 8afc6bd (origin/main, origin/HEAD, main) Add retriever and model config
```

```
$ git fetch
From ../server
+ dc5df93...8aca274 feat/ingest -> origin/feat/ingest (forced update)
 8afc6bd..589d18b main -> origin/main
$ git status
On branch feat/ingest
Your branch and 'origin/feat/ingest' have diverged,
and have 3 and 3 different commits each, respectively.
(use "git pull" if you want to integrate the remote branch with yours)

nothing to commit, working tree clean
```

"3 and 3 different commits": her side has the two old commits and her own, the other side has the README commit and the two copies. She does what the status line suggests, with a merge:

```
$ git pull --no-rebase
Merge made by the 'ort' strategy.
 README.md | 3 +++
 1 file changed, 3 insertions(+)
 create mode 100644 README.md
$ git log --oneline --graph --decorate
* f0c9542 (HEAD -> feat/ingest) Merge branch 'feat/ingest' of ../server into feat/ingest
|\
| * 8aca274 (origin/feat/ingest) Add text cleaner
| * 0fbfd81 Add document loader
| * 589d18b (origin/main, origin/HEAD) Add README
* | c60bc13 Add chunker
* | dc5df93 Add text cleaner
* | 1279adf Add document loader
|/
* 8afc6bd (main) Add retriever and model config
```

```
$ git log --oneline --left-right --cherry-mark ORIG_HEAD...origin/feat/ingest
= 8aca274 Add text cleaner
= 0fbfd81 Add document loader
> 589d18b Add README
< c60bc13 Add chunker
= dc5df93 Add text cleaner
= 1279adf Add document loader
$ git diff --stat ORIG_HEAD HEAD
 README.md | 3 +++
 1 file changed, 3 insertions(+)
```

"Add document loader" and "Add text cleaner" are now in the history twice. `--cherry-mark` shows the pairs with `=`. The diff shows that the merge changed one file: the content is not duplicated, the history is.

Root cause. Git identifies a commit by its ID. Your copies have new IDs, so for Git they are different commits from the originals, and a merge keeps the commits of both sides. This is the third question of section 9.1.

Picture.

```
L = Add document loader   C = Add text cleaner   K = Add chunker   R = Add README   ' = copy made by your rebase
```



The repair, on her side. Go back to before the merge, then replay only her own commit onto the new upstream:

```
$ git reset --hard ORIG_HEAD
HEAD is now at c60bc13 Add chunker
$ git merge-base --fork-point origin/feat/ingest feat/ingest
dc5df9356273e7c0c2807ce808c378782ea2525a
$ git pull --rebase
Rebasing (1/1)
Successfully rebased and updated refs/heads/feat/ingest.
$ git log --oneline --graph --decorate
* fbce73d (HEAD -> feat/ingest) Add chunker
* 8aca274 (origin/feat/ingest) Add text cleaner
* 0fbfd81 Add document loader
* 589d18b (origin/main, origin/HEAD) Add README
* 8afc6bd (main) Add retriever and model config
```

`git merge-base --fork-point` looks into the relog of `origin/feat/ingest`, finds that her branch left that remote-tracking branch when its tip was `dc5df93`, and so identifies "Add chunker" as the only commit that is hers. `git pull --rebase` uses the same logic. The explicit form, `git rebase --onto origin/feat/ingest dc5df93`, is in Lab 9.6.

Why it is more than noise. Every later reader of the history meets each change twice. Worse, if Asha pushes her merge, the commits you removed are back on the server. When the purpose of the rewrite was to remove something, a credential or a large file (Chapter 21B), one stale clone that merges and pushes undoes the cleanup.

The rule, and its working form. Pro Git states it as: "Do not rebase commits that exist outside your repository and that people may have based work on" ([Rebasing](#)). The working form is a question to ask before you rebase anything you have pushed: who else has commits on top of this? Nobody, as on the branch of your own pull request: rebase, and publish as section 9.17 describes. Somebody: agree on it first and send them the repair command, or do not rebase; merge instead.

GitHub, not Git. A new ruleset has "Block force pushes" enabled by default, and classic branch protection rules disable force pushes by default ([available rules](#), [about protected branches](#)). Use that for `main` and for release branches, and leave topic branches open to their owners. Described from the documentation; Chapter 18 has the details.

9.16 Recovering from a bad rebase

In one sentence. A rebase destroys nothing: the old tip is named by `ORIG_HEAD` until another command overwrites that ref, and by the relog of the branch for weeks.

Precisely. Three handles, from the most short-lived to the most durable:

Handle	Usable	Caveat
<code>git rebase --abort</code>	While the rebase is in progress	Discards resolution work done so far
<code>ORIG_HEAD</code>	Directly after the rebase	Rewritten by the next <code>git reset</code> , <code>git merge</code> , <code>git rebase</code> or <code>git am (gitrevisions)</code>
<code><branch>@{n}</code> , the relog of the branch	Until the entry expires	One entry per rebase, so count in the listing; a deleted branch has no relog

See it. An interactive rebase in which the wrong two lines were deleted:

```
$ git log --oneline --decorate
6ab8f55 (HEAD -> feat/rerank) Enable reranking in config
45f4f23 Call reranker from retriever
e30ba74 Experiment: cache cross-encoder scores
195f8d0 Experiment: cross-encoder scoring
5ee19f0 Add reranker skeleton
8afc6bd (main) Add retriever and model config
# The plan was to drop the two experiment commits. The wrong lines were deleted from the todo list.
$ git rebase -i main
--- todo list as Git opened it (comment lines removed) ---
pick 5ee19f0 # Add reranker skeleton
pick 195f8d0 # Experiment: cross-encoder scoring
pick e30ba74 # Experiment: cache cross-encoder scores
pick 45f4f23 # Call reranker from retriever
pick 6ab8f55 # Enable reranking in config
--- todo list as saved ---
pick 5ee19f0 # Add reranker skeleton
pick 195f8d0 # Experiment: cross-encoder scoring
```

```
pick e30ba74 # Experiment: cache cross-encoder scores
Successfully rebased and updated refs/heads/feat/rerank.
$ git log --oneline --decorate
e30ba74 (HEAD -> feat/rerank) Experiment: cache cross-encoder scores
195f8d0 Experiment: cross-encoder scoring
5ee19f0 Add reranker skeleton
8afc6bd (main) Add retriever and model config
```

Two commits are gone and Git said nothing. Noticed at once, the undo is one command:

```
$ git rev-parse --short ORIG_HEAD
6ab8f55
$ git reset --hard ORIG_HEAD
HEAD is now at 6ab8f55 Enable reranking in config
$ git log --oneline --decorate
6ab8f55 (HEAD -> feat/rerank) Enable reranking in config
45f4f23 Call reranker from retriever
e30ba74 Experiment: cache cross-encoder scores
195f8d0 Experiment: cross-encoder scoring
5ee19f0 Add reranker skeleton
8afc6bd (main) Add retriever and model config
```

🔴 `git reset --hard <commit>` moves the current branch to that commit and overwrites the index and the working tree with it. It destroys uncommitted changes to tracked files. Preview with `git status` (it should be clean, as it is after a finished rebase) and `git diff ORIG_HEAD HEAD`. What it moves away from stays recoverable: the rebased tip is now `<branch>@{1}`. It is the appropriate tool here; Chapter 11, section 11.5, covers it in full.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git reset --hard ORIG_HEAD</code>	Overwritten with the old tip	Same	Stays on the branch	Moved back to the old tip	<code>ORIG_HEAD</code> rewritten to the tip before this reset; reflog entries	unchanged	unchanged

Now the same mistake, not noticed, followed by a reset for another reason:

```
# The same bad rebase again. This time you do not notice, and remove one more commit with a reset:
$ git reset --hard HEAD~1
HEAD is now at 195f8d0 Experiment: cross-encoder scoring
$ git log --oneline --decorate
195f8d0 (HEAD -> feat/rerank) Experiment: cross-encoder scoring
5ee19f0 Add reranker skeleton
8afc6bd (main) Add retriever and model config
$ git rev-parse --short ORIG_HEAD
e30ba74
```

`ORIG_HEAD` is `e30ba74`: the tip before the reset, which is the damaged branch. The reflog of the branch has the whole story:

```
$ git reflog show feat/rerank
195f8d0 feat/rerank@{0}: reset: moving to HEAD~1
e30ba74 feat/rerank@{1}: rebase (finish): refs/heads/feat/rerank onto 8afc6bd28c2572af09f1b6c37d733535230e526f
6ab8f55 feat/rerank@{2}: reset: moving to ORIG_HEAD
e30ba74 feat/rerank@{3}: rebase (finish): refs/heads/feat/rerank onto 8afc6bd28c2572af09f1b6c37d733535230e526f
6ab8f55 feat/rerank@{4}: commit: Enable reranking in config
45f4f23 feat/rerank@{5}: commit: Call reranker from retriever
e30ba74 feat/rerank@{6}: commit: Experiment: cache cross-encoder scores
195f8d0 feat/rerank@{7}: commit: Experiment: cross-encoder scoring
5ee19f0 feat/rerank@{8}: commit: Add reranker skeleton
8afc6bd feat/rerank@{9}: branch: Created from HEAD
```

Read it from the top: a reset, the second bad rebase, the reset to `ORIG_HEAD`, the first bad rebase, and below that the five original commits. The last good state is `feat/rerank@{2}`.

```
$ git branch rescue/rerank feat/rerank@{2}
$ git log --oneline rescue/rerank
6ab8f55 Enable reranking in config
45f4f23 Call reranker from retriever
e30ba74 Experiment: cache cross-encoder scores
195f8d0 Experiment: cross-encoder scoring
5ee19f0 Add reranker skeleton
8afc6bd Add retriever and model config
$ git reset --hard rescue/rerank
HEAD is now at 6ab8f55 Enable reranking in config
$ git log --oneline --decorate
6ab8f55 (HEAD -> feat/rerank, rescue/rerank) Enable reranking in config
45f4f23 Call reranker from retriever
e30ba74 Experiment: cache cross-encoder scores
195f8d0 Experiment: cross-encoder scoring
5ee19f0 Add reranker skeleton
8afc6bd (main) Add retriever and model config
```

The rescue branch comes first for a reason. It is a ref, so the commits stay reachable whatever you type next, and you can inspect them before moving the real branch.

In production. By default a relog entry whose commit is no longer reachable from the branch tip, which is what a rebase leaves behind, expires after 30 days, and the unreachable objects are pruned after a further two weeks ([gc configuration](#)); Chapter 13 has the details. Recovery is local: if you had already force-pushed the damaged branch, restore it here first and publish it again as described next.

9.17 Publishing a rebased branch

In one sentence. After a rebase the server's branch is not an ancestor of yours, so a normal push is refused; you have to overwrite the server's ref, and the only choice is which conditions you attach to the overwrite.

Analogy. A compare-and-swap. A plain force is an unconditional write. A lease is "set the ref to X only if it still is Y". The analogy breaks at Y: you do not type the expected value. Git reads it from your remote-tracking branch, and anything that fetches can change that behind your back.

Precisely. Every forced push is : it replaces history on a remote. The forms differ in what must be true first ([git-push](#); Chapter 12, section 12.8, treats them in depth):

Command	The server's ref is overwritten if
<code>git push --force</code>	always
<code>git push --force-with-lease</code>	it equals your <code>origin/<branch></code>
<code>git push --force-with-lease --force-if-includes</code>	as above, and the tip of <code>origin/<branch></code> is reachable from a relog entry of your local branch: your branch has contained it at some point
<code>git push --force-with-lease=<branch>:<expected ID></code>	it equals the ID you wrote

See it. The shared branch of section 9.15, but this time Asha pushed "Add chunker" after your last fetch:

```
# Asha pushed "Add chunker" to feat/ingest after your last fetch. You do not know that yet.
$ git rebase main
Rebasing (1/2)
Rebasing (2/2)
Successfully rebased and updated refs/heads/feat/ingest.
$ git log --oneline --graph --decorate --all
* 321b338 (HEAD -> feat/ingest) Add text cleaner
* 5859371 Add document loader
* 589d18b (origin/main, main) Add README
| * dc5df93 (origin/feat/ingest) Add text cleaner
| * 1279adf Add document loader
|/
* 8afc6bd Add retriever and model config
```

```
$ git push
To ../server.git
! [rejected]        feat/ingest -> feat/ingest (fetch first)
error: failed to push some refs to '../server.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
[exit status: 1]
```

```
$ git push --force-with-lease
To ../server.git
! [rejected]        feat/ingest -> feat/ingest (stale info)
error: failed to push some refs to '../server.git'
[exit status: 1]
```

stale info: the server has a commit that your `origin/feat/ingest` does not know. The lease did its job. Now the failure mode that the manual itself warns about:

```
# Something fetches for you: an editor, a status prompt, or you out of habit.
$ git fetch
From ../server
dc5df93..c60bc13 feat/ingest -> origin/feat/ingest
# The lease now compares against the freshly fetched value. A dry run shows it would overwrite:
$ git push --force-with-lease --dry-run
To ../server.git
+ c60bc13...321b338 feat/ingest -> feat/ingest (forced update)
```

The fetch moved `origin/feat/ingest` to Asha's commit. The server and your remote-tracking branch agree again, the lease is satisfied, and the dry run shows that the push would replace `c60bc13`, a commit you have never looked at. The second check catches it:

```
$ git push --force-with-lease --force-if-includes
To ../server.git
! [rejected]        feat/ingest -> feat/ingest (remote ref updated since checkout)
error: failed to push some refs to '../server.git'
hint: Updates were rejected because the tip of the remote-tracking branch has
```

```
hint: been updated since the last checkout. If you want to integrate the
hint: remote changes, use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
[exit status: 1]
```

c60bc13 is in no reflog entry of your branch, so your rewrite cannot have been based on it. The fix is to rebase what is there:

```
# Rebase what is really on the server, then publish with both checks.
$ git reset --hard origin/feat/ingest
HEAD is now at c60bc13 Add chunker
$ git rebase main
Rebasing (1/3)
Rebasing (2/3)
Rebasing (3/3)
Successfully rebased and updated refs/heads/feat/ingest.
$ git push --force-with-lease --force-if-includes
To ../server.git
+ c60bc13...e3f3ba1 feat/ingest -> feat/ingest (forced update)
```

The explicit form does not depend on the remote-tracking branch at all. You state the commit you rebased from:

```
# Asha pushed in the meantime, and something fetched for you. The explicit lease still refuses:
$ git fetch
From ../server
dc5df93..c60bc13 feat/ingest -> origin/feat/ingest
$ git push --force-with-lease=feat/ingest:dc5df9356273e7c0c2807ce808c378782ea2525a
To ../server.git
! [rejected]          feat/ingest -> feat/ingest (stale info)
error: failed to push some refs to '../server.git'
[exit status: 1]
```

Why plain --force is the worst of them. The same situation. The server's branch first, then the careless push:

```
$ git -C ../server.git log --oneline feat/ingest
c60bc13 Add chunker
dc5df93 Add text cleaner
1279adf Add document loader
8afc6bd Add retriever and model config
```

```
$ git rebase main
Rebasing (1/2)
Rebasing (2/2)
Successfully rebased and updated refs/heads/feat/ingest.
$ git push --force
To ../server.git
+ c60bc13...8aca274 feat/ingest -> feat/ingest (forced update)
```

```
$ git -C ../server.git log --oneline feat/ingest
8aca274 Add text cleaner
0fbfd81 Add document loader
589d18b Add README
8afc6bd Add retriever and model config
$ git -C ../server.git reflog show feat/ingest
```

Asha's commit is gone from the server's branch, and the last command prints nothing: a bare repository keeps no reflog by default, so the server cannot tell you what the branch used to be. Then Asha pulls:

```
$ cd ../asha
$ git log --oneline --decorate -1
c60bc13 (HEAD -> feat/ingest, origin/feat/ingest) Add chunker
$ git pull --rebase
From ../server
+ c60bc13...8aca274 feat/ingest -> origin/feat/ingest (forced update)
8afc6bd..589d18b main -> origin/main
Successfully rebased and updated refs/heads/feat/ingest.
$ git log --oneline --graph --decorate
* 8aca274 (HEAD -> feat/ingest, origin/feat/ingest) Add text cleaner
* 0fbfd81 Add document loader
* 589d18b (origin/main, origin/HEAD) Add README
* 8afc6bd (main) Add retriever and model config
```

```
$ git reflog show origin/feat/ingest
8aca274 refs/remotes/origin/feat/ingest@{0}: pull --rebase: forced-update
c60bc13 refs/remotes/origin/feat/ingest@{1}: update by push
$ git merge-base --fork-point origin/feat/ingest feat/ingest@{1}
c60bc13f871297055fb1d2e19bd6a39f7a4f1f62
```

Observed behavior	: "git pull --rebase" reports success, and "Add chunker", a commit Asha had pushed, is no longer in her branch. (The first question of section 9.1.)
Git state	: The reflog of her origin/feat/ingest holds c60bc13 ("update by push") below the forced update.
Mechanism	: pull --rebase replays the commits after the fork point. The fork point is

	the newest commit of her branch that was once the tip of origin/feat/ingest: c60bc13, her own commit. Nothing comes after it, so nothing is replayed, and her branch is set to the new upstream.
Root cause	: By pushing, she made the commit part of upstream. Upstream no longer has it, and Git reads that as "upstream discarded this commit". Your force push is what discarded it.
Why Git does this	: It is the logic that repaired her history in section 9.15: do not replay what upstream dropped.
Correct fix	: The commit is in the reflog of her branch: git cherry-pick feat/ingest@{1}, then push.
Prevention	: No plain --force on a branch that anyone else pushes to. The lease with --force-if-includes refused in exactly this situation.

```
$ git reflog show feat/ingest -2
8aca274 feat/ingest@{0}: pull --rebase (finish): refs/heads/feat/ingest onto 8aca274dbad90b9d7b5ad3a578f59a5283073474
c60bc13 feat/ingest@{1}: commit: Add chunker
$ git cherry-pick feat/ingest@{1}
[feat/ingest 82659ee] Add chunker
Date: Mon Sep 7 10:05:00 2026 +0530
1 file changed, 2 insertions(+)
create mode 100644 ingest/chunker.py
$ git push
To ../server.git
 8aca274..82659ee feat/ingest -> feat/ingest
$ git log --oneline --graph --decorate
* 82659ee (HEAD -> feat/ingest, origin/feat/ingest) Add chunker
* 8aca274 Add text cleaner
* 0fbfd81 Add document loader
* 589d18b (origin/main, origin/HEAD) Add README
* 8afc6bd (main) Add retriever and model config
```

The account that this book owes for a  command: a forced push changes the server's ref, for every ref the push covers. It can destroy commits that exist only on the server and in other people's clones. Preview with `git fetch` followed by `git log --oneline HEAD..origin/<branch>`, which lists what you are about to remove, or with `--dry-run`. Recovery depends on a clone that still has the commits; GitHub's own instruments are summarised in Chapter 13. Appropriate use of plain `--force` and of the bare lease: none. The two guarded forms cost the same keystrokes, and with them everything removed from the server is something your clone has, so your own reflog can restore it.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
Forced push, accepted	unchanged	unchanged	unchanged	unchanged	origin/<branch> set to the pushed commit	Branch ref replaced; the old commits are no longer on the branch	Pull requests for the branch follow it; where the rules dismiss stale approvals, an approval is dismissed when the diff changes (described from the documentation)
Forced push, rejected	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged

In production. `git config set --global push.useForceIfIncludes true` adds the second check to every lease, and an alias (Chapter 14B) shortens the rest. Editors and prompts that fetch in the background are the usual way the bare lease is defeated; the manual's own example is a scheduled `git fetch`.

9.18 Rebase or merge

Both integrate the same work. Do it twice, once each way, and compare:

```
$ git switch -c demo/merged main
Switched to a new branch 'demo/merged'
$ git merge feat/rerank
Auto-merging config/model.yaml
Merge made by the 'ort' strategy.
 app/rerank.py      | 2 ++
 app/retriever.py   | 2 +-
 config/model.yaml  | 1 +
 3 files changed, 4 insertions(+), 1 deletion(-)
create mode 100644 app/rerank.py
$ git log --oneline --graph --decorate demo/merged
* 9d34396 (HEAD -> demo/merged) Merge branch 'feat/rerank' into demo/merged
|\
| * bd62876 (feat/rerank) Enable reranking in config
| * af65a92 Call reranker from retriever
| * 5ee19f0 Add reranker skeleton
* | 82f1fbb (main) Upgrade model to small-v2
* | 589d18b Add README
|/
* 8afc6bd Add retriever and model config
```

```
$ git rev-parse "demo/merged^{tree}" "demo/rebased^{tree}"
273096fac387738589eb6596acff61b6fb3cf487
```

```
273096fac387738589eb6596acff61b6fb3cf487
$ git diff --stat demo/merged demo/rebased
```

`demo/rebased` is the same branch rebased onto `main`. The two results have the same tree, to the byte. What differs is everything else:

```
# Commit, tree and subject of everything that is new relative to main, for each result:
$ git log --format="%h tree %t %s" main..demo/merged
9d34396 tree 273096f Merge branch 'feat/rerank' into demo/merged
bd62876 tree 96ba5b8 Enable reranking in config
af65a92 tree ece63c5 Call reranker from retriever
5ee19f0 tree bff5bbd Add reranker skeleton
$ git log --format="%h tree %t %s" main..demo/rebased
668e89e tree 273096f Enable reranking in config
60fc46f tree 9b070ab Call reranker from retriever
bbbc3e4 tree 8cc3ece Add reranker skeleton
```

```
$ git log --oneline --first-parent main..demo/merged
9d34396 Merge branch 'feat/rerank' into demo/merged
$ git log --oneline --first-parent main..demo/rebased
668e89e Enable reranking in config
60fc46f Call reranker from retriever
bbbc3e4 Add reranker skeleton
```

The merge added one new snapshot, the merge commit, and kept your three commits with the trees you built and tested. The rebase produced three new commits; two of their trees (`8cc3ece`, `9b070ab`) are combinations of your early commits with `main` that nobody has ever run. Read along first parents, the merged history says "one feature landed" and the rebased one lists three changes.

Question	Merge	Rebase
Commit IDs of the branch	Kept	New
Shape of history	Fork and join are recorded	One line
New snapshots that nobody ran	One: the merge result	One per replayed commit
Conflicts	Resolved once, recorded in the merge commit	Resolved per commit, up to once for each
Record of what was integrated, and when	The merge commit; <code>--first-parent</code> reads as a list of integrations	None
Undoing the whole integration	One revert with <code>-m 1</code> (Chapter 11)	One revert per commit, or of the range
On a branch that others use	Safe: it only adds commits	Needs a forced push and everybody's cooperation
Reading and bisecting	More paths to walk	Simpler, provided every commit builds

Neither column wins. Three questions decide a given case. Is the branch shared? Then merge. What does the target branch require? A "Require linear history" rule on GitHub blocks merge commits, which leaves squash or rebase ([available rules](#)). Must every intermediate commit work? Then a rebase needs `--exec`, or a squash. Many teams combine the two: rebase in private, and let the pull request's merge method (Chapter 17) decide what lands on `main`.

9.19 Configuration

Setting	Default	Effect
<code>rebase.autoSquash</code>	false	<code>--autosquash</code> for interactive rebases (section 9.7)
<code>rebase.autoStash</code>	false	<code>--autostash</code> for every rebase
<code>rebase.updateRefs</code>	false	<code>--update-refs</code> for every rebase
<code>rebase.missingCommitsCheck</code>	ignore	warn or error when a line was deleted from the list
<code>pull.rebase</code>	unset	git pull rebases; true, merges or interactive
<code>push.useForceIfIncludes</code>	false	Adds <code>--force-if-includes</code> to every lease
<code>merge.conflictStyle</code>	merge	<code>zdiff3</code> also shows the base in rebase conflicts (Chapter 8)
<code>rerere.enabled</code>	off unless <code>.git/rr-cache</code> exists	Records conflict resolutions and replays them (Chapter 14C)

```
git config set --global rebase.autoSquash true
git config set --global rebase.autoStash true
git config set --global rebase.updateRefs true
```

The `set` and `get` subcommands need Git 2.46 or later; on older versions write `git config --global rebase.autoSquash true`. Inside `labs/shell`, `--global` writes to the isolated lab configuration. The manual lists further keys: `rebase.backend`, `rebase.forkPoint`, `rebase.rebaseMerges`, `rebase.abbreviateCommands`, `rebase.instructionFormat`, `rebase.rescheduleFailedExec` and `sequence.editor`.

9.20 git replay and git history

Two experimental commands do parts of this chapter's work without its machinery. Chapter 14D covers them.

`git replay` replays a range onto a new base without touching the working tree or the index, so it also works in a bare repository:

```
$ git status --short --branch
## main
$ git replay --onto=main main..feat/rerank
[exit status: 0]
$ git log --oneline --graph --decorate --all
* 4caa98e (feat/rerank) Enable reranking in config
* 20d0897 Call reranker from retriever
* 0dea106 Add reranker skeleton
* 82f1fbb (HEAD -> main) Upgrade model to small-v2
* 589d18b Add README
* 8afc6bd Add retriever and model config
$ git relog show feat/rerank -2
4caa98e feat/rerank@{0}: replay --onto 82f1fbb78c09021e2e673bf06e3c77fc58f14a50
bd62876 feat/rerank@{1}: commit: Enable reranking in config
```

`feat/rerank` was rebased while `main` stayed checked out, and no state directory was involved. The copies are `0dea106`, `20d0897` and `4caa98e`: the third command in this chapter to produce those byte-identical commits (section 9.3). The price of having no working tree is that it cannot stop for a conflict:

```
$ git replay --onto=main main..feat/rerank
[exit status: 1]
$ git log --oneline --decorate -1 feat/rerank
ad106e3 (feat/rerank) Enable reranking in config
$ git status --short --branch
## main
```

Exit status 1, no output, and the branch is where it was. The command was introduced in Git 2.44 as a server-side tool and updates refs itself since 2.53; its `--linearize` option was added in Git 2.56 (not run here) ([git-replay](#)).

`git history`, new in Git 2.54, offers single-purpose rewrites: `git history reword <commit>`, `git history split <commit>` and, since 2.55, `git history fixup <commit>`; `drop` was added in Git 2.56 (not run here). It moves every descendant branch by default, runs no hooks, does not work on histories that contain merges, and refuses operations that would conflict ([git-history](#)). Both manual pages carry the word EXPERIMENTAL in their first line.

9.21 What can go wrong

Symptom	Diagnosis	Fix	Prevention
A commit is missing after a rebase that stopped at a conflict	<code>git range-diff <base> <branch>@{1}</code> shows it with <code><</code> and no partner	<code>git reset --hard <branch>@{1}</code> , rebase again, resolve with the stage you mean	Know the swap (9.11); range-diff before every push
A commit is missing after an interactive rebase without conflicts	Same command; a line was deleted from the list	Same reset, then redo	<code>rebase.missingCommitsCheck=error</code> ; write <code>drop</code>
The resolution of a conflict ended up in the wrong commit	<code>git show --stat</code> of the previous commit lists the conflicted file	Reset and redo	After a conflict: <code>git add</code> , <code>git rebase --continue</code> , never <code>--amend</code>
A conflict in a file your branch never changed	<code>git log --oneline <upstream>..HEAD</code> lists commits that are not yours	<code>git rebase --abort</code> , then <code>--onto</code> with the right boundary	Name the boundary for stacked branches (9.5)
The same conflict at one commit after another	Several commits touch the lines upstream changed	Abort; squash first, enable <code>rerere</code> , or merge	Short-lived branches
Lower branches of a stack still point at old commits	<code>git log --graph --all</code> shows copies above the originals	Put the top branch back from its relog, redo with <code>--update-refs</code>	<code>rebase.updateRefs=true</code>
<code>ORIG_HEAD</code> is not the tip from before the rebase	A <code>git reset</code> ran since, possibly inside an <code>edit</code> stop	Use the branch relog	Prefer <code><branch>@{1}</code>
"already a rebase-merge directory"	A rebase is in progress; <code>git status</code> says where	<code>--continue</code> or <code>--abort</code> it	Read the prompt or <code>git status</code> before starting
HEAD is detached after leaving a rebase	<code>--quit</code> was used, or the state directory was deleted	<code>git reset --hard</code> if the index is unmerged, then <code>git switch <branch></code>	Leave with <code>--abort</code>
Tests fail on a middle commit after the rebase	Replayed commits are snapshots nobody ran	<code>git rebase --exec "<tests>"</code> , amend at each stop	Always <code>--exec</code> in a cleanup rebase
The lease is rejected	Someone pushed; <code>git fetch</code> , then <code>git log HEAD..origin/<branch></code>	Rebase what is on the server (9.17)	Ask before rebasing a branch others push to

Symptom	Diagnosis	Fix	Prevention
A teammate has every commit twice	<code>git log --oneline --left-right --cherry-mark <old>...<new></code> shows = pairs	The repair of section 9.15	Announce rebases; send the command
A teammate's pushed commit disappeared	The server ref was replaced; the commit is in their branch reflog	Cherry-pick it from the reflog, push	No plain <code>--force</code> ; block force pushes on shared branches

9.22 When not to use it, and dangerous edge cases

Do not rebase:

- **A branch that others have based work on.** `main`, release branches, a feature branch that two people push to. The cost lands on them (section 9.15).
- **Commits whose IDs are already recorded where they must stay valid.** A released tag, a deployment record, a model card, a submodule pointer in another repository (Chapter 23).
- **A long branch that conflicts with upstream at many commits.** One merge is one resolution.

Edge cases that have hurt people:

- `-X ours` **during a rebase discards your side** of every conflicting hunk, because of the swap. `-s ours` is worse: it empties every one of your commits, which the manual says "makes little sense".
- `--root` **on a published repository**, and any rebase that crosses a tag others use: the tag keeps pointing at the old commit, and the new history no longer contains it.
- **An instruction list is a script.** `exec` lines run shell commands in your working tree with your privileges. Read a list you did not write.
- **Hooks are not a control here.** `pre-rebase` can refuse a rebase and `--no-verify` skips it; the manual describes which other hooks run during a rebase as an accident of implementation. Enforce nothing through them.
- **Worktrees.** `--update-refs` does not move a branch that is checked out in another worktree (Chapter 25), so a stack can end up half moved.
- **Deleting the branch to start over** deletes its reflog, which is the most durable undo handle of section 9.16.

9.23 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git range-diff</code> , <code>git cherry</code> , <code>git log --cherry-mark</code> , <code>git rebase --show-current-patch</code>	● SAFE	Nothing	not needed	not needed
<code>git commit --fixup=<commit></code> , <code>-s squash=<commit></code>	● SAFE	Adds a commit	<code>git diff --cached</code>	Fold it in, or drop it
<code>git rebase <upstream></code> , <code>--onto</code> , <code>-i</code> , <code>--autosquash</code> , <code>--keep-base</code> , <code>--rebase-merges</code>	● CAUTION	New commits; the branch ref, at the end	<code>git log --oneline <upstream>..HEAD</code> lists what will be replayed	<code>git reset --hard <branch>@{1}</code>
<code>git rebase --update-refs</code>	● CAUTION	Also every local branch inside the range	With <code>-i</code> , the <code>update-ref</code> lines	Each branch from its own reflog
<code>git rebase --root</code>	● CAUTION	Every commit of the branch	<code>git log --oneline</code>	Reflog; for a published repository there is no clean recovery
<code>git rebase --continue</code>	● CAUTION	Commits the index as the copy, runs on	<code>git status</code> , <code>git diff --cached</code>	Reflog, after the rebase
<code>git rebase --skip</code>	● CAUTION	Omits the stopped commit	<code>git rebase --show-current-patch</code>	The original is in the old history; redo
<code>git rebase --abort</code>	● CAUTION	Index and working tree back to the old tip	<code>git status</code>	None for resolution work in progress
<code>git rebase --quit</code>	● CAUTION	Removes the state directory; HEAD stays detached	<code>git status</code>	<code>git switch <branch></code>
Reopening the list while stopped (9.6)	● SAFE	The remaining instructions	It shows them	Abort the rebase
<code>git pull --rebase</code>	● CAUTION	Fetches; rewrites your unpushed commits	<code>git fetch</code> , then <code>git log --oneline @{u}..HEAD</code>	<code>git reset --hard <branch>@{1}</code>
<code>git reset --hard ORIG_HEAD</code>	● DANGEROUS	Branch, index and working tree	<code>git status</code> , <code>git diff ORIG_HEAD HEAD</code>	Reflog for commits; none for uncommitted changes
<code>git push --force-with-lease</code> <code>--force-if-includes</code> , or the lease with an explicit ID	● DANGEROUS	The server's ref, if it still is what you rebased from	Add <code>--dry-run</code>	The old tip is in your own reflogs
<code>git push --force-with-lease</code> alone	● DANGEROUS	The server's ref; the check passes wrongly after a	<code>--dry-run</code> ; <code>git log HEAD..origin/<branch></code>	Only from a clone that has the commits

Command	Label	What it changes	Preview	Recovery
		background fetch		
<code>git push --force</code>	 DANGEROUS	The server's ref, unconditionally	Same	Same

9.24 Version notes

Version note. Older behavior: `git rebase` turned commits into patches and applied them (the apply backend). Current behavior: the merge backend replays with the merge machinery; `--apply` selects the old one. Since: Git 2.26 ([release notes](#)). Recommended: the default.

Version note. Older behavior: `--autosquash` worked only together with `-i`. Current behavior: it also works without `-i`; `rebase.autoSquash` still applies to interactive rebases only. Since: Git 2.44 ([release notes](#)). Recommended: type the option in scripts.

Version note. Older behavior: a rebase moved one branch, and stacks were repaired by hand with `--onto`. Current behavior: `--update-refs` moves every local branch in the range. Since: Git 2.38 ([release notes](#)). Recommended: `rebase.updateRefs=true` if you work with stacks.

Version note. Older behavior: `git push --force` was the only way to publish a rewrite. Current behavior: `--force-with-lease` and `--force-if-includes`. Since: Git 1.8.5 and Git 2.30 ([git-push](#)). Recommended: both together, or the explicit lease; the manual still calls the lease forms without an explicit value experimental.

Feature	Since	Note
<code>--rebase-merges</code>	Git 2.18	<code>--preserve-merges</code> deprecated in 2.22, removed in 2.34
<code>git range-diff</code>	Git 2.19	<code>--diff-merges</code> since 2.48
<code>git pull</code> stops on diverged branches	Git 2.33.1 and 2.34	Chapter 12
<code>git config set, get, list</code>	Git 2.46	The older option forms still work
<code>git rebase --trailer</code>	Git 2.54	Appends a trailer to every rebased commit
<code>git replay</code>	Git 2.44; updates refs itself since 2.53	<code>--linearize</code> was added in Git 2.56 (not run here)
<code>git history</code>	Git 2.54; <code>fixup</code> since 2.55	<code>drop</code> was added in Git 2.56 (not run here)

Sources for the table: the release-note links in section 1 of the Phase 0 report.

Older installations matter for scripts: Ubuntu 24.04 LTS ships Git 2.43.0 ([Launchpad](#)), where `git config set` does not exist and `--autosquash` needs `-i`.

Unverified. The releases that introduced `rebase.rebaseMerges`, `rebase.missingCommitsCheck`, `--keep-base`, the `--empty` option of `rebase`, and the `amend!` commits with `fixup -C` were not found in the sources available for this chapter. All of them were run on Git 2.55.0 and are shown above.

9.25 Practice

- Labs 9.1 to 9.7 in the Module 9 lab manual: clean up a messy branch; transplant with `--onto`; autosquash fixups; a conflict and who is "ours"; break a branch and recover it; a shared branch and a teammate's duplicated history; `--update-refs` on a stack.
- Replay any transcript with `labs/run ch09/<demo>`, for example `labs/run ch09/force-push`. The sandbox stays in place, so you can continue by hand.
- Three drills in sandboxes left by replays:
 - In `ch09/range-diff-context`, find the smallest `--creation-factor` at which the third commit is paired.
 - In `ch09/onto-cut`, run `git reset --hard ORIG_HEAD`, remove the two commits with `drop` instead, and compare the trees with `git diff 31e8a4d HEAD`.
 - In `ch09/force-damage`, in Asha's clone, find the ID of her original "Add chunker" commit in two ways that do not use `feat/ingest@{1}`.

9.26 Interview questions

1. A rebase "moves" a branch onto `main`. Describe what happens to objects, refs and HEAD, step by step, and name what has not changed when the command returns.
2. Why does every rebased commit have a new ID even when its diff is identical? Which fields of the commit object changed, and which did not?
3. A rebase is stopped at a conflict. Where do the branch ref, HEAD, `ORIG_HEAD` and `REBASE_HEAD` point? What does that imply for `--abort`?
4. During a rebase, what do `--ours` and `--theirs` refer to, and why? Give one concrete way this loses work without any error message.
5. `git rebase main` on a feature branch conflicts in a file the branch never touched. Give two different root causes and the command that fixes each.
6. You force-pushed a rebased branch. A teammate now has every commit twice. Explain how that happened, how they repair it, and what you would have done differently.
7. Compare `--force`, `--force-with-lease`, and `--force-with-lease --force-if-includes`. Describe a sequence of events in which the second overwrites a colleague's commit and the third refuses.
8. A colleague's pushed commit vanished from their own branch after `git pull --rebase`. Nobody ran a destructive command on their machine. Explain the mechanism and recover the commit.
9. The reviewer approved three commits; after your rebase there are still three with the same titles. How do you show what changed, and how does the reviewer do it without access to your clone?
10. `ORIG_HEAD` does not point where you expected after a rebase. Why, and what do you use instead?
11. When would you choose a merge over a rebase for the same integration, and when the reverse? Argue both sides for a team that bisects often.
12. What do `--update-refs` and `--rebase-merges` each add to a plain rebase, and what does each still leave for you to do?

9.27 Sources

Primary sources

- [git-rebase](#), including its sections "Behavioral differences", "Recovering from upstream rebase" and "Rebasing merges"; [git-range-diff](#); [git-push](#); [git-pull](#); [git-commit](#) for `--fixup` and `--squash`; [git-cherry](#); [git-patch-id](#); [gitrevisions](#) for `ORIG_HEAD` and `REBASE_HEAD`; [gitglossary](#); [githooks](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that the transcripts were checked against.
- [git-replay](#) and [git-history](#); the pages on [git-scm.com](#) describe 2.56.
- Release notes: [2.26.0](#), [2.34.0](#), [2.38.0](#), [2.44.0](#), [2.46.0](#), [2.54.0](#).
- GitHub Docs: [about merge methods](#), [pull request merges](#), [available rules for rulesets](#).

Secondary sources

- Pro Git, [Rebasing](#) and [Rewriting History](#). Caveats: `master` throughout, and `git checkout` in the first; neither mentions `--update-refs`, `--force-with-lease` or `git range-diff`; the second still demonstrates `git filter-branch`, with a warning.
- Julia Evans, [git rebase: what can go wrong?](#), [Confusing git terminology](#) and [Some Git poll results](#). The polls are self-selected samples.
- [git rebase in depth](#), a sandbox walkthrough. Its author and year could not be confirmed from the page, and it does not cover `exec`.
- The Phase 0 report of this course, sections 1, 4, 12 and 13.

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- Philomatics, ["git rebase - Why, When & How to fix conflicts"](#), 10 minutes, June 2024. Current; short and opinionated.
- Tobias Günther for freeCodeCamp, ["Advanced Git Tutorial"](#), 34 minutes, November 2021. Interactive rebase and cherry-pick; mixes `master` and `main` and uses some `checkout`.
- Scott Chacon, ["So You Think You Know Git Part 2"](#), 23 minutes, March 2024. Fixup commits and `--update-refs` for stacked branches.
- Colt Steele, ["Git Rebase Vs. Merge"](#), 20 minutes, March 2021. Uses `git switch`; `master` naming.

Further reading

- Chapter 10 for the single replay that a rebase repeats; Chapter 12 for push and pull; Chapter 13 for reflogs and retention; Chapter 14C for `rerere`.

Chapter 10: Cherry-pick

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch10/`.

10.1 Why this matters

Three questions a CTO can ask about a maintenance release:

1. "The fix for the empty-prompt crash was merged to `main` last week. Customers on 1.4 still crash. Is the fix in the 1.4 branch or not, and how do you prove it either way?"
2. "The backport applied without a single conflict, and the 1.4 build broke anyway. How is that possible?"
3. "When we merged the release branch back into `main`, the log showed the same commit title twice. Did something go wrong?"

The answers come from one fact. **A cherry-pick does not move or reuse a commit. It recomputes the change that a commit made and records that change as a new commit, with a new ID, on the branch you are on.** Git stores no link between the original and the copy. You can ask Git whether two commits make the same change, and you can write the link into the message yourself, but ancestry, the thing `git branch --contains` and `git merge` rely on, knows nothing about it.

The project in this chapter is `gateway`, a small service that forwards prompts to a model API. `main` is the development line; `release/1.4` is the maintenance line that customers run.

10.2 What cherry-pick does

In one sentence. 🟡 `git cherry-pick <commit>` takes the difference between a commit and its parent, merges that difference into your current branch with a three-way merge, and commits the result.

Analogy. A colleague edited another copy of a document with change tracking on. You do not paste their paragraph over yours; you ask what they changed, compared with what they started from, and make that change in your copy, which has drifted in the meantime. The analogy breaks at judgment. A person notices when the change refers to a definition that exists only in the other copy. Git does not: the change applies, and your document is now wrong (Lab 10.1).

Precisely. A cherry-pick is a three-way merge (Chapter 8, section 8.4) with an unusual choice of base. Call the picked commit C and its parent P:

- base: P, the parent of the picked commit
- ours: HEAD, the branch you are on
- theirs: C, the picked commit

The merge computes "what changed from P to C" and "what changed from P to HEAD" and combines the two. Whatever else exists on C's branch plays no part. This is more than applying a patch. A patch carries a few lines of context and fails when they do not match; the merge works with three complete versions of each file.

Inside `.git`. One new commit object, with new tree and blob objects where content is new. The current branch ref advances by one commit, and the HEAD reflog gets an entry that starts with `cherry-pick: . ORIG_HEAD` is not written. While a pick is stopped, the root ref `CHERRY_PICK_HEAD` names the commit being picked (section 10.6).

See it. Asha fixed a crash on `main`. The release branch has one commit of its own:

```
$ git log --graph --decorate --all --format="%h %an: %s%d"
* 3056255 Lab User: Allow three retries on the 1.4 line (HEAD -> release/1.4)
| * 0cca736 Lab User: Document the streaming client (main)
| * f98ffd3 Asha Rao: Reject empty prompts before calling the model
| * ca6dd48 Lab User: Add streaming client
| * 6db3c0c Lab User: Start 1.5 development
|/
* 93787ec Lab User: Add model client
```

```
$ git show --format="%h %s" main~1
f98ffd3 Reject empty prompts before calling the model

diff --git a/src/client.py b/src/client.py
index bf888b0..d2b1af9 100644
--- a/src/client.py
+++ b/src/client.py
@@ -2,4 +2,6 @@ TIMEOUT_S = 30
    MAX_RETRIES = 2

    def call_model(prompt):
+        if not prompt.strip():
```

```
+         raise ValueError("empty prompt")
        return post("/v1/generate", prompt, timeout=TIMEOUT_S)
```

The hunk's context includes the line `MAX_RETRIES = 2`. On `release/1.4` that line says 3. As a patch, the change is refused:

```
# On release/1.4 the line MAX_RETRIES = 2 reads MAX_RETRIES = 3. Try the change as a plain patch:
$ git format-patch -1 --stdout main~1 | git apply --check
error: patch failed: src/client.py:2
error: src/client.py: patch does not apply
[exit status: 1]
```

Now predict the cherry-pick. `git merge-tree` (Chapter 8, section 8.17) performs a merge without touching the working tree and prints the ID of the resulting tree; give it the parent of the picked commit as the base:

```
# A three-way merge with the parent of the picked commit as the base:
$ git merge-tree --write-tree --merge-base=main~2 HEAD main~1
1a4fb1358dcd46f1552e678a514c54c49e146e42
```

```
$ git cherry-pick main~1
Auto-merging src/client.py
[release/1.4 3c81466] Reject empty prompts before calling the model
Author: Asha Rao <asha@example.com>
Date: Mon Sep 7 10:05:00 2026 +0530
1 file changed, 2 insertions(+)
$ git rev-parse HEAD^{tree}
1a4fb1358dcd46f1552e678a514c54c49e146e42
$ cat src/client.py
TIMEOUT_S = 30
MAX_RETRIES = 3

def call_model(prompt):
    if not prompt.strip():
        raise ValueError("empty prompt")
    return post("/v1/generate", prompt, timeout=TIMEOUT_S)
```

The cherry-pick succeeded where the patch failed, and the tree of the new commit is `1a4fb13...`, the tree that the three-way merge predicted. The result keeps the release's `MAX_RETRIES = 3` and gains the two new lines: each side's change relative to the base, combined.

Picture.

```

      6db3c0c-----ca6dd48----f98ffd3----0cca736      main
      /               P             C
93787ec \
        3056255-----3c81466                      release/1.4 (HEAD)
        |               |
        ours            new commit: tree = merge(base P; ours 3056255; theirs C), one parent
```

State table.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in <code>.git</code>	Remote	GitHub
<code>git cherry-pick <commit></code> , clean	Gains the change	Same	Stays on the branch	Advances to the new commit	New objects; one HEAD reflog entry and one branch reflog entry	unchanged	unchanged
The same, stopped at a conflict	Conflict markers in conflicted files	Stages 1, 2, 3 for those paths	unchanged	unchanged	<code>CHERRY_PICK_HEAD</code> , <code>MERGE_MSG</code> ; for a sequence also <code>.git/sequencer/</code>	unchanged	unchanged

What it demands of your working tree. The manual says the working tree must be clean. Git 2.55 is more precise than that:

```
# An uncommitted edit in VERSION. The picked commit changes only src/client.py.
$ git status --short
M VERSION
$ git cherry-pick main~1
Auto-merging src/client.py
[release/1.4 62a5848] Reject empty prompts before calling the model
Author: Asha Rao <asha@example.com>
Date: Mon Sep 7 10:05:00 2026 +0530
1 file changed, 2 insertions(+)
$ git status --short
M VERSION
```

```
# Now the uncommitted edit is in the file that the picked commit changes.
$ git status --short
M src/client.py
$ git cherry-pick main~1
error: Your local changes to the following files would be overwritten by merge:
      src/client.py
Please commit your changes or stash them before you merge.
Aborting
fatal: cherry-pick failed
```

```
[exit status: 128]
$ git status --short
M src/client.py
```

```
# A staged change, in a file the picked commit does not touch.
$ git add VERSION
$ git cherry-pick main~1
error: your local changes would be overwritten by cherry-pick.
hint: commit your changes or stash them to proceed.
fatal: cherry-pick failed
[exit status: 128]
$ git status --short
M VERSION
```

An unstaged edit in a file the pick does not write is left alone. An unstaged edit in a file it must write, or any staged change, stops the command before it does anything, and nothing is lost in either refusal.

In production. The standard case is a fix that has to exist on more than one line of development: on `main` and on the release branch that customers run (section 10.9). The other everyday case is a commit made on the wrong branch: pick it onto the right one, then remove it from the wrong one (Chapter 11).

10.3 The new commit and its new ID

In one sentence. The commit that a cherry-pick creates shares the author, the author date and the message with the original, and nothing else.

```
# The original commit on main ...
$ git cat-file -p main~1
tree 1f6a01cb332c8e3fe47c6a154f79b3cbb046ee11
parent ca6dd4819cdc3b57d478ca0a5ebc7b9c3c8c3e6c
author Asha Rao <asha@example.com> 1788755700 +0530
committer Asha Rao <asha@example.com> 1788755700 +0530
```

```
Reject empty prompts before calling the model
# ... and the commit that the cherry-pick created.
$ git cat-file -p HEAD
tree 1a4fb1358dcd46f1552e678a514c54c49e146e42
parent 3056255676479f09f344885216ba2f1c8ac61aa1
author Asha Rao <asha@example.com> 1788755700 +0530
committer Lab User <you@example.com> 1788756120 +0530
```

Reject empty prompts before calling the model

Field	Original f98ffd3	Copy 3c81466
tree	1f6a01c...	1a4fb13...: the release's files plus the change
parent	ca6dd48... on main	3056255... on release/1.4
author	Asha Rao, 1788755700	identical
committer	Asha Rao, 1788755700	Lab User, 1788756120: whoever ran the cherry-pick, when it ran
message	identical	identical

A different tree, a different parent and a different committer line give a different hash (Chapter 6). Read the two objects once more and look for a field that points from the copy to the original. There is none.

```
$ git log --graph --decorate --all --format="%h %an: %s%d"
* 3c81466 Asha Rao: Reject empty prompts before calling the model (HEAD -> release/1.4)
* 3056255 Lab User: Allow three retries on the 1.4 line
| * 0cca736 Lab User: Document the streaming client (main)
| * f98ffd3 Asha Rao: Reject empty prompts before calling the model
| * ca6dd48 Lab User: Add streaming client
| * 6db3c0c Lab User: Start 1.5 development
|/
* 93787ec Lab User: Add model client
$ git reflog -1
3c81466 HEAD@{0}: cherry-pick: Reject empty prompts before calling the model
```

Two commits with the same author and subject, on two branches, unrelated in the graph. The mental model "a commit is a diff" is the right one for predicting what a cherry-pick does; the model "a commit is a snapshot with parents" (Chapter 2) is the right one for understanding what it leaves behind.

10.4 Options: `-x`, `-e`, `-n`, and `-m` for merges

`-x` **records where the change came from.** It appends the line (cherry picked from commit <full ID>) to the message. That line is the only link between copy and original that travels with the history; section 10.9 builds the backport workflow on it. The manual's advice is to use it between publicly visible branches and not for commits from a private branch, whose IDs mean nothing to anyone else.

`-e` opens the editor on the message. With `-x`, the line is already in the buffer:

```
$ git cherry-pick -e -x main~1
Auto-merging src/client.py
--- message as Git opened it (comment lines removed) ---
Reject empty prompts before calling the model

(cherry picked from commit f98ffd3d15c63376ff2fe882f990e11ff6722de6)
--- message as saved ---
Reject empty prompts before calling the model

Backported to 1.4 because customers on 1.4.0 hit the error path.

(cherry picked from commit f98ffd3d15c63376ff2fe882f990e11ff6722de6)
[release/1.4 26424e6] Reject empty prompts before calling the model
Author: Asha Rao <asha@example.com>
Date: Mon Sep 7 10:05:00 2026 +0530
1 file changed, 2 insertions(+)
$ git log -1 --format=%B
Reject empty prompts before calling the model

Backported to 1.4 because customers on 1.4.0 hit the error path.

(cherry picked from commit f98ffd3d15c63376ff2fe882f990e11ff6722de6)
```

The scripted editor added one paragraph and kept the rest, as you would. Delete the last line in the editor and the link is gone.

 `-n` applies the change without committing.

```
$ git cherry-pick -n main~1
Auto-merging src/client.py
$ git status --short
M src/client.py
$ git rev-parse --verify --quiet CHERRY_PICK_HEAD
[exit status: 1]
```

```
$ git commit -m "Reject empty prompts (1.4 variant)"
[release/1.4 0a6f5fa] Reject empty prompts (1.4 variant)
1 file changed, 2 insertions(+)
$ git log -1 --format="author %an, committer %cn"
author Lab User, committer Lab User
$ git log -1 --format="author %an, committer %cn" main~1
author Asha Rao, committer Asha Rao
```

The change is staged, and `CHERRY_PICK_HEAD` does not exist: as far as Git's state is concerned, no cherry-pick is in progress. The original message is still on offer, because Git leaves it in `.git/MERGE_MSG` and a `git commit` without `-m` proposes it. The authorship is not: the commit above has you as its author, not Asha. Use `-n` to combine several picks into one commit or to adapt a change before committing it, and restore the credit yourself: `git commit -C <original>` reuses the original's message and authorship.

`-m <parent-number>` is required for a merge commit. "The difference between a commit and its parent" is ambiguous when there are two parents:

```
$ git log --oneline --graph --decorate --all
* 598b598 (main) Merge branch 'feat/limits'
| \
| * 308a7ab (feat/limits) Allow short bursts above the rate limit
| * ad79f94 Add a per-minute rate limit
* | ad39bb6 Add README
|/
* 6db3c0c Start 1.5 development
* 93787ec (HEAD -> release/1.4) Add model client
```

```
$ git cherry-pick main
error: commit 598b5987e817298dd14cfa3d7bb7d31c2ff315a4 is a merge but no -m option was given.
fatal: cherry-pick failed
[exit status: 128]
```

```
$ git cherry-pick -m 1 -x main
[release/1.4 ba59152] Merge branch 'feat/limits'
Date: Mon Sep 7 10:07:00 2026 +0530
1 file changed, 2 insertions(+)
create mode 100644 src/limits.py
$ git show --stat --format="%h %s%n%b" HEAD
ba59152 Merge branch 'feat/limits'
(cherry picked from commit 598b5987e817298dd14cfa3d7bb7d31c2ff315a4)

src/limits.py | 2 ++
1 file changed, 2 insertions(+)
$ git log --oneline --graph --decorate release/1.4
* ba59152 (HEAD -> release/1.4) Merge branch 'feat/limits'
* 93787ec Add model client
```

`-m 1` says: take parent 1 as the base. The change relative to the first parent is everything the merged branch brought in, so the two commits of `feat/limits` arrive as one. Look at the last graph: the new commit is titled "Merge branch 'feat/limits'" and has a single parent. It is not a merge. For Git the commits of `feat/limits` are still unmerged here: `git branch --no-merged` lists the branch, and a real merge later would add those commits to the history next to this copy.

Other options in brief: `-s` adds a Signed-off-by trailer, `-S` signs the new commit, `-X <option>` passes an option to the merge machinery, and `--ff` fast-forwards instead of copying when HEAD already is the parent of the picked commit.

10.5 Several commits and ranges

In one sentence. Given several commits, cherry-pick copies them one after the other; given a range, it copies the commits of the range from oldest to newest.

Precisely. Commits that you list are applied in the order you list them. A range `A..B` means "reachable from B and not from A" ([gitrevisions](#)), so A itself is not included; write `A^..B` to include it. The range notations are the subject of Lab 10.4 and Chapter 14A.

See it. The release branch is one commit behind four commits on `main`. Take the last three:

```
$ git log --oneline --graph --decorate --all
* c3a7a9f (main) Apply the timeout to streaming calls
* ebcee16 Retry failed generate calls
* 08f4771 Add streaming client
* 1a6b393 Move to the v2 generate endpoint
* 6db3c0c Start 1.5 development
* 93787ec (HEAD -> release/1.4) Add model client
```

```
$ git cherry-pick main~3..main
[release/1.4 e0c8403] Add streaming client
Date: Mon Sep 7 10:05:00 2026 +0530
1 file changed, 2 insertions(+)
create mode 100644 src/stream.py
Auto-merging src/client.py
CONFLICT (content): Merge conflict in src/client.py
error: could not apply ebcee16... Retry failed generate calls
hint: After resolving the conflicts, mark them with
hint: "git add/rm <paths>spec", then run
hint: "git cherry-pick --continue".
hint: You can instead skip this commit with "git cherry-pick --skip".
hint: To abort and get back to the state before "git cherry-pick",
hint: run "git cherry-pick --abort".
hint: Disable this message with "git config set advice.mergeConflict false"
[exit status: 1]
```

`main~3..main` is three commits; `main~3`, "Move to the v2 generate endpoint", is the excluded boundary. The first of the three was copied. The second conflicts, because it edits a line that the excluded commit had changed, and the sequence stopped there with exit status 1.

10.6 CHERRY_PICK_HEAD, the sequencer, and the four ways out

In one sentence. A stopped cherry-pick is described by one root ref, `CHERRY_PICK_HEAD`, and, when more than one commit was requested, by a directory `.git/sequencer/` that holds the rest of the plan.

See it. The state after the stop of section 10.5:

```
$ git log --oneline --decorate -2
e0c8403 (HEAD -> release/1.4) Add streaming client
93787ec Add model client
$ git rev-parse --short CHERRY_PICK_HEAD
ebcee16
$ ls .git/sequencer
abort-safety
head
todo
$ cat .git/sequencer/todo
pick ebcee16 Retry failed generate calls
pick c3a7a9f Apply the timeout to streaming calls
$ cat .git/sequencer/head
93787ec2cb8ce6539803c9712b5335894e6c0f8a
```

`CHERRY_PICK_HEAD` is the commit that could not be applied. The instruction file in `.git/sequencer/`, the third name in the listing, holds it and everything after it; `sequencer/head` is where HEAD was before the first pick. Note what is different from a rebase (Chapter 9, section 9.4): HEAD is not detached. The branch has already moved to `e0c8403`, the copy of the first commit. A cherry-pick of a single commit creates no `sequencer` directory at all; `CHERRY_PICK_HEAD` and the prepared message are its whole state.

```
$ git status
On branch release/1.4
You are currently cherry-picking commit ebcee16.
(fix conflicts and run "git cherry-pick --continue")
```



```
(use "git cherry-pick --skip" to skip this patch)
(use "git cherry-pick --abort" to cancel the cherry-pick operation)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   src/client.py

no changes added to commit (use "git add" and/or "git commit -a")
```

Command	Effect
<code>git cherry-pick --continue</code>	Commit the staged resolution as the copy, then go on with the list
 <code>git cherry-pick --skip</code>	Make no copy of this commit, then go on
 <code>git cherry-pick --abort</code>	Return the branch, the index and the working tree to the state before the first pick
 <code>git cherry-pick --quit</code>	Forget the sequence; leave everything as it is now

`--abort` goes back to `sequencer/head`, so the pick that had succeeded is undone as well:

```
$ git cherry-pick --abort
$ git log --oneline --decorate -2
93787ec (HEAD -> release/1.4) Add model client
$ git status --short --branch
## release/1.4
```

`--skip` drops the conflicting commit and applies the rest:

```
$ git cherry-pick --skip
[release/1.4 9ad6c66] Apply the timeout to streaming calls
Date: Mon Sep 7 10:07:00 2026 +0530
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline --decorate -3
9ad6c66 (HEAD -> release/1.4) Apply the timeout to streaming calls
b97b1c7 Add streaming client
93787ec Add model client
```

`--quit` only deletes the bookkeeping. The completed copy stays on the branch, and the conflict stays in the index and the working tree:

```
$ git cherry-pick --quit
$ git status --short --branch
## release/1.4
UU src/client.py
$ git log --oneline --decorate -2
99912f6 (HEAD -> release/1.4) Add streaming client
93787ec Add model client
$ git cherry-pick --continue
error: no cherry-pick or revert in progress
fatal: cherry-pick failed
[exit status: 128]
```

And `--continue`, after a real resolution:

```
# Resolve in an editor: keep the v1 endpoint and add the retries argument. Then:
$ git add src/client.py
$ git cherry-pick --continue
[release/1.4 27a23eb] Retry failed generate calls
Author: Asha Rao <asha@example.com>
Date: Mon Sep 7 10:06:00 2026 +0530
 1 file changed, 1 insertion(+), 1 deletion(-)
[release/1.4 dd535d1] Apply the timeout to streaming calls
Date: Mon Sep 7 10:07:00 2026 +0530
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline --decorate -4
dd535d1 (HEAD -> release/1.4) Apply the timeout to streaming calls
27a23eb Retry failed generate calls
19e9ae9 Add streaming client
93787ec Add model client
```

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in <code>.git</code>	Remote	GitHub
<code>git cherry-pick --abort</code>	Back to the state before the sequence; resolution work is lost	Same	Stays on the branch	Moved back to <code>sequencer/head</code>	<code>CHERRY_PICK_HEAD</code> and <code>.git/sequencer/</code> removed	unchanged	unchanged
<code>git cherry-pick --quit</code>	unchanged	unchanged	unchanged	unchanged: completed picks stay	<code>CHERRY_PICK_HEAD</code> and <code>.git/sequencer/</code> removed	unchanged	unchanged

In production. A sequence that stops in a CI job or a release script leaves a branch that is partly advanced. A script that cherry-picks must check the exit status and decide: `--abort` to return to a known state, never a blind retry. `git status` names the commit it is stuck on.

10.7 Conflicts

In one sentence. In a cherry-pick conflict, "ours" is your branch, "theirs" is the picked commit, and the base is the picked commit's parent: a version of the file that your branch may never have contained.

See it. One commit this time, with the conflict style set to `diff3` so that the markers show the base as well (Chapter 8, section 8.9):

```
$ git config get merge.conflictStyle
diff3
$ git cherry-pick -x main~1
Auto-merging src/client.py
CONFLICT (content): Merge conflict in src/client.py
error: could not apply ebcee16... Retry failed generate calls
hint: After resolving the conflicts, mark them with
hint: "git add/rm <paths>", then run
hint: "git cherry-pick --continue".
hint: You can instead skip this commit with "git cherry-pick --skip".
hint: To abort and get back to the state before "git cherry-pick",
hint: run "git cherry-pick --abort".
hint: Disable this message with "git config set advice.mergeConflict false"
[exit status: 1]
```

```
$ cat src/client.py
TIMEOUT_S = 30
MAX_RETRIES = 2

def call_model(prompt):
<<<<<< HEAD
    return post("/v1/generate", prompt, timeout=TIMEOUT_S)
||||| parent of ebcee16 (Retry failed generate calls)
    return post("/v2/generate", prompt, timeout=TIMEOUT_S)
=====
    return post("/v2/generate", prompt, timeout=TIMEOUT_S, retries=MAX_RETRIES)
>>>>>> ebcee16 (Retry failed generate calls)
```

```
$ git ls-files -u
100644 c1997a3b3072639101b24a264539d1ea68338ebd 1      src/client.py
100644 bf888b0fe23f1bba1fac2339cf6673748d3e3c5a 2      src/client.py
100644 e4a1d5abcb7ec04dfc8d525dc56542b6520af8d0 3      src/client.py
$ git show :1:src/client.py | tail -1
    return post("/v2/generate", prompt, timeout=TIMEOUT_S)
$ git show :2:src/client.py | tail -1
    return post("/v1/generate", prompt, timeout=TIMEOUT_S)
$ git show :3:src/client.py | tail -1
    return post("/v2/generate", prompt, timeout=TIMEOUT_S, retries=MAX_RETRIES)
```

Three versions of one line. Yours calls `/v1/generate`. The picked commit calls `/v2/generate` with a `retries` argument. The base, labelled parent of `ebcee16`, calls `/v2/generate` without it. That base is not the merge base of the two branches:

```
# The merge base of the two branches has the v1 line ...
$ git show $(git merge-base HEAD main~1):src/client.py | tail -1
    return post("/v1/generate", prompt, timeout=TIMEOUT_S)
# ... but stage 1 is the parent of the picked commit, which already has v2.
$ git show main~2:src/client.py | tail -1
    return post("/v2/generate", prompt, timeout=TIMEOUT_S)
```

With the base in view, the conflict reads as an instruction. The change to carry over is the difference between base and theirs: add `retries=MAX_RETRIES`. Apply that to your line, which still says `/v1/`:

```
# Resolve in an editor: the release keeps the v1 endpoint and gains the retries argument.
$ git add src/client.py
$ git cherry-pick --continue
[release/1.4 1805e42] Retry failed generate calls
Author: Asha Rao <asha@example.com>
Date: Mon Sep 7 10:06:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
```

```
$ git log -1 --format=fuller
commit 1805e42b7d53fbd067b2e155af6150dcea1e88a6
Author: Asha Rao <asha@example.com>
AuthorDate: Mon Sep 7 10:06:00 2026 +0530
Commit: Lab User <you@example.com>
CommitDate: Mon Sep 7 10:18:00 2026 +0530

    Retry failed generate calls

    (cherry picked from commit ebcee168893f1604f1fcaa96b382049f9d46fd23)
$ git show --format= HEAD
diff --git a/src/client.py b/src/client.py
index bf888b0..32a510e 100644
--- a/src/client.py
+++ b/src/client.py
@@ -2,4 +2,4 @@ TIMEOUT_S = 30
MAX_RETRIES = 2

def call_model(prompt):
```

```
- return post("/v1/generate", prompt, timeout=TIMEOUT_S)
+ return post("/v1/generate", prompt, timeout=TIMEOUT_S, retries=MAX_RETRIES)
```

The copy changes one thing, the argument, as the original did. Taking "theirs" whole would also have switched the release to the v2 endpoint: an unrelated change from `main`, smuggled in through a conflict resolution (Lab 10.2). The sides are not swapped here as they are in a rebase: `HEAD` is your branch.

How to finish. Git has prepared the commit message before you resolve anything:

```
# git cherry-pick -x main~1 has stopped with a conflict. Git has already prepared the message:
$ cat .git/MERGE_MSG
Retry failed generate calls

(cherry picked from commit ebcee168893f1604f1fcaa96b382049f9d46fd23)

# Conflicts:
#      src/client.py
$ git rev-parse --short CHERRY_PICK_HEAD
ebcee16
```

`git cherry-pick --continue` commits with that message and removes the comment lines:

```
# The conflict is resolved and staged. First way: --continue.
$ git cherry-pick --continue
[release/1.4 3926e9d] Retry failed generate calls
Author: Asha Rao <asha@example.com>
Date: Mon Sep 7 10:06:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
$ git log -1 --format="author %an, committer %cn%n%n%B"
author Asha Rao, committer Lab User

Retry failed generate calls

(cherry picked from commit ebcee168893f1604f1fcaa96b382049f9d46fd23)
```

A plain `git commit` also completes the pick, and the author is preserved either way because `CHERRY_PICK_HEAD` still exists. But the message suffers:

```
# The same stop, resolved the same way. Second way: git commit --no-edit.
$ git commit --no-edit
[release/1.4 1682e80] Retry failed generate calls
Author: Asha Rao <asha@example.com>
Date: Mon Sep 7 10:06:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
$ git log -1 --format="author %an, committer %cn%n%n%B"
author Asha Rao, committer Lab User

Retry failed generate calls

(cherry picked from commit ebcee168893f1604f1fcaa96b382049f9d46fd23)

# Conflicts:
#      src/client.py
```

```
# Third way: git commit with a message of your own.
$ git commit -m "Retry failed generate calls (1.4)"
[release/1.4 b5f3d00] Retry failed generate calls (1.4)
Author: Asha Rao <asha@example.com>
Date: Mon Sep 7 10:06:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
$ git log -1 --format="author %an, committer %cn%n%n%B"
author Asha Rao, committer Lab User

Retry failed generate calls (1.4)

$ git rev-parse --verify --quiet CHERRY_PICK_HEAD
[exit status: 1]
```

`--no-edit` skips the editor and with it the cleanup that strips comment lines, so `# Conflicts:` becomes part of the message. `-m` replaces the prepared message, and the line that `-x` wrote is gone. Finish a cherry-pick with `--continue`.

One disagreement between documentation and behavior belongs here. The local manual says of `-x` that the line is added "only for cherry picks without conflicts". The transcripts above show Git 2.55.0 writing it into the prepared message of a conflicted pick, and `--continue` keeping it.

10.8 Changes that are already there: `--empty`

In one sentence. When the picked change is already present on your branch, the merge result equals `HEAD`, there is nothing to commit, and `--empty` decides what happens next.

```
# The fix is already on release/1.4. Someone picks it a second time.
$ git cherry-pick -x main~1
Auto-merging src/client.py
The previous cherry-pick is now empty, possibly due to conflict resolution.
If you wish to commit it anyway, use:

    git commit --allow-empty

Otherwise, please use 'git cherry-pick --skip'
On branch release/1.4
You are currently cherry-picking commit f98ffd3.
  (all conflicts fixed: run "git cherry-pick --continue")
  (use "git cherry-pick --skip" to skip this patch)
  (use "git cherry-pick --abort" to cancel the cherry-pick operation)

nothing to commit, working tree clean
[exit status: 1]
```

The default is to stop and ask. `git cherry-pick --skip` moves on; the suggested `git commit --allow-empty` would record a commit without changes. The decision can be made in advance:

```
$ git cherry-pick --empty=drop main~1
Auto-merging src/client.py
dropping f98ffd3d15c63376ff2fe882f990e11ff6722de6 Reject empty prompts before calling the model -- patch contents already
upstream
$ git log --oneline --decorate -2
4f88542 (HEAD -> release/1.4) Reject empty prompts before calling the model
3056255 Allow three retries on the 1.4 line
```

```
$ git cherry-pick --empty=keep main~1
Auto-merging src/client.py
[release/1.4 a11cd35] Reject empty prompts before calling the model
Author: Asha Rao <asha@example.com>
Date: Mon Sep 7 10:05:00 2026 +0530
$ git show --stat --format="%h %s" HEAD
a11cd35 Reject empty prompts before calling the model
```

Option	A pick that turns out empty is
<code>--empty=stop</code> (default)	paused, for you to decide
<code>--empty=drop</code>	left out, with a one-line notice
<code>--empty=keep</code>	committed as an empty commit
<code>--keep-redundant-commits</code>	the older name of <code>--empty=keep</code> ; deprecated

The old name still works:

```
$ git cherry-pick --keep-redundant-commits main~1
Auto-merging src/client.py
[release/1.4 6c698c5] Reject empty prompts before calling the model
Author: Asha Rao <asha@example.com>
Date: Mon Sep 7 10:05:00 2026 +0530
$ git log --oneline --decorate -3
6c698c5 (HEAD -> release/1.4) Reject empty prompts before calling the model
4f88542 Reject empty prompts before calling the model
3056255 Allow three retries on the 1.4 line
```

`--allow-empty` is a different option: it concerns commits that were empty to begin with.

Version note. Older behavior: the only control was `--keep-redundant-commits`. Current behavior: `--empty=(drop|keep|stop)`, with the old option kept as a deprecated synonym. Since: Git 2.45 ([release notes](#)). Recommended: `--empty=drop` in scripts that backport lists of commits, so that a commit which is already present does not halt an unattended job.

10.9 The backport workflow

In one sentence. A backport is a fix that lands on the development line first and is then copied to a maintenance line with `git cherry-pick -x`, so that the copy says where it came from.

See it.

```
$ git log --oneline -3 main
0cca736 Document the streaming client
f98ffd3 Reject empty prompts before calling the model
ca6dd48 Add streaming client
$ git cherry-pick -x f98ffd3
Auto-merging src/client.py
[release/1.4 c308105] Reject empty prompts before calling the model
Author: Asha Rao <asha@example.com>
```

```
Date: Mon Sep 7 10:05:00 2026 +0530
1 file changed, 2 insertions(+)
$ git log -1 --format=fuller
commit c3081051fe3d7125255eac2894828854bc2682ad
Author: Asha Rao <asha@example.com>
AuthorDate: Mon Sep 7 10:05:00 2026 +0530
Commit: Lab User <you@example.com>
CommitDate: Mon Sep 7 10:09:00 2026 +0530
```

Reject empty prompts before calling the model

(cherry picked from commit f98ffd3d15c63376ff2fe882f990e11ff6722de6)

`--format=fuller` shows both identities. Asha wrote the change at 10:05; you committed this copy at 10:09. The last line of the message is the link to the original.

Now the first question of section 10.1: is the fix in the 1.4 branch? Ask by commit ID and the answer is misleading:

```
# Which branches contain the fix? Asking by commit ID finds only the original.
$ git branch --contains f98ffd3
main
```

`--contains` follows ancestry, and the original commit is not an ancestor of the release branch; its copy is. Ask for the recorded line instead:

```
$ git log --all --oneline --grep='cherry picked from commit f98ffd3'
c308105 Reject empty prompts before calling the model
$ git branch --contains $(git log --all --format=%h --grep='cherry picked from commit f98ffd3')
* release/1.4
```

That is a proof: a commit on `release/1.4` states that it is a copy of `f98ffd3`. A third way is to ask whether an equivalent change exists, by patch ID (section 10.10):

```
$ git cherry -v release/1.4 main
+ 6db3c0c6eace6d80b965fbca258fd96d13ad0e76 Start 1.5 development
+ ca6dd4819cdc3b57d478ca0a5ebc7b9c3c8c3e6c Add streaming client
+ f98ffd3d15c63376ff2fe882f990e11ff6722de6 Reject empty prompts before calling the model
+ 0cca736b9a1a5fe9a95303513732a6c623631f05 Document the streaming client
```

+ in front of the fix means "no equivalent on the release", and here that is wrong. The release differs from `main` in a line inside the hunk's context (`MAX_RETRIES = 3`), so the diff of the copy is not textually the diff of the original, and their patch IDs differ. A context line was enough. Patch comparison finds exact copies only; the `-x` line survives adaptation.

The procedure.

1. Merge the fix into `main` through the normal review.
2. `git switch release/1.4`, then `git cherry-pick -x <commit>`.
3. Build and test on the release branch. "Applied cleanly" says that the text merged, not that the code works there (Lab 10.1).
4. Publish the release branch the way your team publishes it: a push, or a pull request whose base is the release branch.

Which direction? Two conventions are in use, and a senior engineer should be able to name both. The Git project commits a fix on the oldest supported branch that needs it and merges upward, so that newer branches contain older ones and cherry-picks are the exception ([gitworkflows](#)). Trunk-based practice fixes on the main line first and cherry-picks to the release branch, on the argument that a fix made on the release branch can be forgotten on `main` and come back as a regression ([branch for release](#), [Microsoft Release Flow](#)). This chapter demonstrates the second; the mechanics are the same in both.

In production. For an ML service the maintenance line is often the version that a customer's evaluation was signed off against. A backport changes that line. Record it where the sign-off lives: the `-x` line gives the auditor the original commit, and the original commit leads to the review and the tests.

10.10 Duplicate commits and how to find them

In one sentence. After a cherry-pick the same change exists under two commit IDs, and `git cherry` and `git log --cherry-mark` find such pairs by comparing patch IDs.

Precisely. A patch ID is a hash of a commit's diff with line numbers ignored ([git-patch-id](#)). `git cherry <upstream> <head>` lists the commits of `<head>` that are not in `<upstream>` and marks each with `-` if `<upstream>` contains a commit with the same patch ID, otherwise with `+`. `git log --left-right --cherry-mark A...B` walks both sides of the symmetric difference and marks such pairs with `=`.

See it. A release branch whose own commit lies outside the context of the fix, so that the copy is exact:

```
$ git log --oneline --graph --decorate --all
* c84bed2 (HEAD -> release/1.4) Reject empty prompts before calling the model
* f395bc9 Raise the timeout to 60 seconds on the 1.4 line
| * 0cca736 (main) Document the streaming client
| * f98ffd3 Reject empty prompts before calling the model
```

```
| * ca6dd48 Add streaming client
| * 6db3c0c Start 1.5 development
|/
* 93787ec Add model client
```

```
$ git cherry -v release/1.4 main
+ 6db3c0c6eace6d80b965fbca258fd96d13ad0e76 Start 1.5 development
+ ca6dd4819cdc3b57d478ca0a5ebc7b9c3c8c3e6c Add streaming client
- f98ffd3d15c63376ff2fe882f990e11ff6722de6 Reject empty prompts before calling the model
+ 0cca736b9a1a5fe9a95303513732a6c623631f05 Document the streaming client
$ git cherry -v main release/1.4
+ f395bc9ef18134d47129560a98d5bc6c5d2b32ef Raise the timeout to 60 seconds on the 1.4 line
- c84bed2a3af7e44ee969c4b890987cfa6e76769e Reject empty prompts before calling the model
```

```
$ git log --oneline --left-right --cherry-mark release/1.4...main
< c84bed2 Reject empty prompts before calling the model
< f395bc9 Raise the timeout to 60 seconds on the 1.4 line
> 0cca736 Document the streaming client
= f98ffd3 Reject empty prompts before calling the model
> ca6dd48 Add streaming client
> 6db3c0c Start 1.5 development
```

```
$ git log --oneline --left-right --cherry-pick release/1.4...main
< f395bc9 Raise the timeout to 60 seconds on the 1.4 line
> 0cca736 Document the streaming client
> ca6dd48 Add streaming client
> 6db3c0c Start 1.5 development
```

< is a commit only on the left side (the release), > only on the right (main), = a commit whose change also exists on the other side. --cherry-pick leaves the pairs out, which gives the real difference between the two lines. For "what is on main that the release still lacks", add --right-only. The evidence underneath:

```
$ git show main~1 | git patch-id --stable
25663a50190a2730649d2f06cd81f82ff10a1575 f98ffd3d15c63376ff2fe882f990e11ff6722de6
$ git show release/1.4 | git patch-id --stable
25663a50190a2730649d2f06cd81f82ff10a1575 c84bed2a3af7e44ee969c4b890987cfa6e76769e
```

What happens when the two lines meet again. Merge the release branch back into main:

```
$ git switch main
Switched to branch 'main'
$ git merge -m "Merge branch release/1.4 into main" release/1.4
Auto-merging src/client.py
Merge made by the 'ort' strategy.
 src/client.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline --graph --decorate -7
* cf36074 (HEAD -> main) Merge branch release/1.4 into main
|\
| * c84bed2 (release/1.4) Reject empty prompts before calling the model
| * f395bc9 Raise the timeout to 60 seconds on the 1.4 line
* | 0cca736 Document the streaming client
* | f98ffd3 Reject empty prompts before calling the model
* | ca6dd48 Add streaming client
* | 6db3c0c Start 1.5 development
|/
```

```
$ git log --oneline --grep="Reject empty prompts"
c84bed2 Reject empty prompts before calling the model
f98ffd3 Reject empty prompts before calling the model
```

The merge is clean: both sides made the identical change, and identical changes do not conflict (Chapter 8, section 8.4). The history now holds the change twice, once per ID. That answers the third question of section 10.1: nothing went wrong. It becomes a problem when a copy was adapted. Then the two sides changed the same lines differently, and the merge either conflicts or, with unlucky context, combines them in a way nobody reviewed.

In production. Anything that counts commits double-counts duplicates: a changelog generated from `git log`, a "commits since last release" metric. Generate such lists with `--cherry-pick`, and remember its limit from section 10.9: an adapted backport is invisible to patch comparison. Lab 10.3 works through that case.

10.11 Revert, the inverse operation

In one sentence. 🟡 `git revert <commit>` is the same three-way merge with base and "theirs" exchanged: the base is the commit itself and "theirs" is its parent, so the change is applied backwards.

	base	ours	theirs	Result
<code>git cherry-pick C</code>	parent of C	HEAD	C	HEAD plus the change of C

	base	ours	theirs	Result
git revert C	C	HEAD	parent of C	HEAD minus the change of C

See it. Predict the tree with `git merge-tree`, then revert:

```
$ git log --oneline --decorate -3
4f88542 (HEAD -> release/1.4) Reject empty prompts before calling the model
3056255 Allow three retries on the 1.4 line
93787ec Add model client
$ git rev-parse HEAD~1^{tree}
c0427e5100e84ffed7a94307938f8c547a0919d6
```

```
# Base: the commit to undo (HEAD). Ours: HEAD. Theirs: the parent of the commit to undo.
$ git merge-tree --write-tree --merge-base=HEAD HEAD HEAD~1
c0427e5100e84ffed7a94307938f8c547a0919d6
```

```
$ git revert --no-edit HEAD
[release/1.4 57c5a52] Revert "Reject empty prompts before calling the model"
Date: Mon Sep 7 10:12:00 2026 +0530
1 file changed, 2 deletions(-)
$ git rev-parse HEAD^{tree}
c0427e5100e84ffed7a94307938f8c547a0919d6
$ git log -1 --format=%B
Revert "Reject empty prompts before calling the model"

This reverts commit 4f885421c38859bdc722fa1961433160eca5ab6a.
```

The predicted tree, the tree of the revert commit and the tree of the commit before the backport are one and the same, `c0427e5...`. A revert is a new commit like any other. It uses the same sequencer, with `REVERT_HEAD` in place of `CHERRY_PICK_HEAD` and the same four ways out, and it needs `-m` for a merge commit for the same reason. Chapter 11, sections 11.8 and 11.9, covers it in full, including what reverting a merge does to later merges.

10.12 What can go wrong

Symptom	Diagnosis	Fix	Prevention
"is a merge but no -m option was given"	The picked commit has two parents	<code>-m 1</code> if you want what the merge brought in; consider merging instead	Pick the commits of the branch, not the merge
"The previous cherry-pick is now empty"	The change is already on this branch	<code>git cherry-pick --skip</code>	<code>--empty=drop</code> in scripts; check with <code>git cherry</code> first
"your local changes would be overwritten"	A staged change, or an edit in a file the pick writes	Commit or stash, then pick	Start from a clean <code>git status</code>
A conflict in lines the fix did not seem to touch	Stage 1 is the parent of the picked commit, not your branch's past: <code>git show :1: <path></code>	Apply the difference between stage 1 and stage 3 to your version	<code>merge.conflictStyle=zdiff3</code>
The backport applied cleanly and the build fails	The commit uses something added by an earlier commit you did not pick: <code>git log -S'<name>' main</code> finds it	Remove the pick; pick the prerequisite as well, or adapt the fix	Test on the target branch before pushing
The backport contains changes the original did not	A conflict was resolved by taking "theirs" whole: compare <code>git show</code> of both commits	Redo the pick, resolve by hand	Read the three stages (10.7)
The message contains # Conflicts:	Finished with <code>git commit --no-edit</code>	<code>git commit --amend</code>	Finish with <code>git cherry-pick --continue</code>
The (cherry picked from ...) line is missing	<code>-x</code> was not used, the line was deleted in the editor, or the pick was finished with <code>git commit -m</code>	<code>git commit --amend</code> and add the line	<code>-x</code> always for backports
The author is you, not the original author	<code>-n</code> followed by a plain commit	<code>git commit --amend -C <original></code> restores author and message	Avoid <code>-n</code> for single backports
"The fix is not in the release", says --contains	Ancestry does not know about copies	Search for the <code>-x</code> line; <code>git cherry -v</code>	Teach the team section 10.9
<code>git cherry</code> shows + for a commit that was backported	The copy was adapted, or its context differs	Search for the <code>-x</code> line	Do not rely on patch IDs alone
A branch is half advanced after an interrupted sequence	<code>git status; git reflog -5</code>	<code>git cherry-pick --abort</code> , or <code>git reset --hard <ID before the sequence></code>	Check exit status in scripts

10.13 When not to use it, and dangerous edge cases

Do not cherry-pick:

- **When you want everything a branch has.** Merge it. The local workflow manual puts the difference in one sentence: "merging works at the branch level, while cherry-picking works at the commit level", and a merge commit is a promise that everything from its parents is included ([gitworkflows](#)). A cherry-pick promises nothing about the commits around it.
- **To move your own unpublished commits to another base.** That is a rebase with `--onto` (Chapter 9, section 9.5), which copies the commits and moves the branch in one operation.
- **As the routine way to keep two long-lived branches in step.** Duplicates accumulate, every later merge between the two has to reconcile them, and "what does A have that B lacks" stops being answerable from the graph.
- **A commit that depends on commits you are not taking.** It will apply, and it will not work.
- **A long range across lines that have diverged.** Every commit in it can conflict, each against a base your branch never had.

Edge cases:

- `-m` **on a merge** produces a single-parent commit that looks like a merge in the log (section 10.4).
- `-n` discards authorship and the in-progress state: `git status` no longer says that a cherry-pick is under way.
- `-x` **pointing at a private commit.** If the original lives only in your clone, or its branch is later rebased, the recorded ID refers to a commit nobody can find.
- **Signatures.** The copy is a new object. It is not signed by the original author; it is signed by you if you sign, and otherwise not at all (Chapter 14B).
- **Review.** A backport shows the original author's name on code that was merged, and possibly edited, by someone else on another branch. Review it as new code.

10.14 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git cherry</code> , <code>git log --cherry-mark</code> , <code>git patch-id</code> , <code>git merge-tree --write-tree</code>	● SAFE	Nothing in refs, index or working tree (<code>merge-tree</code> writes objects)	not needed	not needed
<code>git cherry-pick <commit></code> , with or without <code>-x</code> , <code>-e</code> , <code>-m</code>	● CAUTION	Adds commits to the current branch, which moves it, and updates index and working tree, as <code>git merge</code> does; refuses to overwrite local changes	<code>git show <commit></code> ; <code>git merge-tree --write-tree --merge-base=<commit>^ HEAD <commit></code>	<code>git reset --hard <ID before></code> from the reflog; <code>git revert</code> if already pushed
<code>git cherry-pick -n <commit></code>	● CAUTION	Index and working tree; no commit, no in-progress state	<code>git show <commit></code>	<code>git restore --staged --worktree <paths></code>
<code>git cherry-pick --continue</code>	● CAUTION	Commits the staged resolution, which moves the branch, and goes on	<code>git diff --cached</code>	As for a pick
<code>git cherry-pick --skip</code>	● CAUTION	Discards the stopped pick, goes on	<code>git status</code>	Pick that commit again
<code>git cherry-pick --abort</code>	● CAUTION	Branch, index and working tree back to the state before the sequence	<code>git status</code>	Completed copies are in the reflog; resolution work is gone
<code>git cherry-pick --quit</code>	● CAUTION	Removes the sequencer state only	<code>git status</code>	Nothing to recover
<code>git revert <commit></code>	● CAUTION	Adds a commit that undoes the change: the branch moves, index and working tree are updated; nothing is removed	<code>git show <commit></code>	Revert the revert

`git reset --hard`, named twice above as a recovery, is ● : it overwrites uncommitted changes in tracked files (Chapter 11).

10.15 Version notes

Version note. Older behavior: tutorials call `CHERRY_PICK_HEAD` and `REVERT_HEAD` pseudorefs. Current behavior: the glossary reserves "pseudoref" for `FETCH_HEAD` and `MERGE_HEAD`; the others are root refs. Since: Git 2.46 ([glossary](#)). Recommended: read them with `git rev-parse`.

- `--empty=(drop|keep|stop)` exists since Git 2.45 and soft-deprecates `--keep-redundant-commits` (section 10.8).
- That `git cherry-pick --no-commit` does not set `CHERRY_PICK_HEAD` is the behavior of Git 2.55, shown in section 10.4; the release notes of Git 2.56 record that the documentation was clarified on this point ([release notes](#)).
- `-r` is accepted and does nothing. The manual explains that `-x` used to be the default and `-r` switched it off.
- The experimental `git replay --advance=<branch>` cherry-picks a range onto a branch without a working tree (Chapter 14D).

Unverified. The release that introduced `--merge-base` for `git merge-tree`, and the release in which `-x` began to be recorded for conflicted picks (the manual still says it is not), were not found in the sources available for this chapter. Both behaviors were run on Git 2.55.0 and are shown above.

10.16 Practice

- Labs 10.1 to 10.3 in the Module 10 lab manual: backport a fix with `-x`; a cherry-pick conflict; detect duplicates with `git cherry`. Lab 10.4, on range notation, is in its own file.
- Replay any transcript with `labs/run ch10/<demo>`, for example `labs/run ch10/pick-conflict`.
- Two drills: in the sandbox of `ch10/pick-three-way`, predict the tree of `git cherry-pick main` with `git merge-tree` before you run it; in `ch10/duplicates`, where the replay has already merged the release into `main`, list what `main` had before that merge and the release lacked, with one `git log` command (the old tip is `main~1`).

10.17 Interview questions

1. "Cherry-pick applies a patch." What is imprecise about that sentence, and when does the difference show?
2. Name the base, "ours" and "theirs" of a cherry-pick. How does a revert differ?
3. Which fields does the copy share with the original commit, and which not? What follows for `git branch --contains`?
4. A fix was backported with `-x`. Give three ways to establish that the release branch contains it, and say which of them can give a wrong answer and why.
5. A backport applied without conflict and broke the release build. Explain the mechanism and how you would have caught it.
6. In a cherry-pick conflict, stage 1 shows a line that never existed on your branch. Where does it come from, and how do you use it to resolve correctly?
7. Compare the state after a stopped cherry-pick of a range with the state after a stopped rebase. Where is HEAD, and what has happened to the branch?
8. What does `git cherry-pick -m 1 <merge>` create, and why is it not equivalent to merging the branch?
9. Two long-lived branches exchange fixes by cherry-pick in both directions. What problems accumulate, and what would you change?
10. When do you fix on the oldest branch and merge upward, and when on `main` with a backport? Argue both.

10.18 Sources

Primary sources

- [git-cherry-pick](#), [git-cherry](#), [git-patch-id](#), [git-revert](#), [gitrevisions](#), [gitworkflows](#), [git-commit](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that the transcripts were checked against; the `--cherry-mark`, `--cherry-pick` and `--left-right` options are described in `git help -m log`.
- Release notes: [2.45.0](#), [2.46.0](#), [2.56.0](#).

Secondary sources

- Julia Evans, [How git cherry-pick and revert use 3-way merge](#), the clearest short account of the base used by each.
- Derrick Stolee, [Commits are snapshots, not diffs](#) (GitHub Blog, 2020), on how cherry-pick and rebase replay changes although commits store snapshots.
- [Branch for release](#) on trunkbaseddevelopment.com and Microsoft's [Release Flow](#), for the "fix on the main line, then cherry-pick" convention.
- The Phase 0 report of this course, sections 1, 4, 12 and 13.

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- Tobias Günther for freeCodeCamp, "[Advanced Git Tutorial](#)", 34 minutes, November 2021. Includes cherry-picking; mixes `master` and `main`.
- ProCodrr, [episode 9 on cherry-pick](#), 8 minutes, March 2023, in Hindi. Mostly current; `master` naming.

Further reading

- Chapter 9, where the same replay runs once per commit; Chapter 11 for revert; Chapter 14A for range notation and `git log` as a query language.

Chapter 11: Reset, Revert, Restore

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch11/`.

11.1 Why this matters

Three questions a CTO can ask in the hour after something went wrong:

1. "A bad configuration change is on `main` and three people have already pulled it. What exactly do we run, and what must nobody run?"
2. "An engineer ran `git reset --hard` and says two days of work are gone. Which part can you bring back, which part can you not, and why?"
3. "We reverted the merge of the reranker branch last week. Today the fixed branch was merged again and half of the feature is missing in production. How is that possible?"

All three are undo questions, and undo is where Git costs its users the most time. On 1 October 2026 the highest-voted question under Stack Overflow's `git` tag was "How do I undo the most recent local commits in Git?" (score 27,242), and six more undo questions were in the top 25 ([Stack Overflow](#)). A study of 80,370 Stack Overflow questions found that the five commands whose questions draw the most views are `git revert`, `git reflog`, `git stash`, `git clean` and `git reset` ([preprint](#)).

The cause is that Git has no single undo. It has five commands (`restore`, `reset`, `revert`, `clean`, `stash`), and each is defined by the places it writes to. The short answers to the three questions:

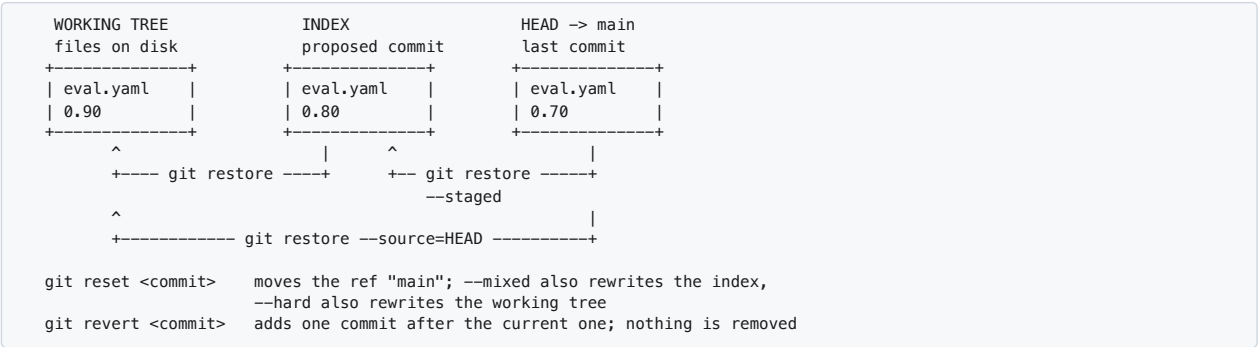
1. Add a new commit with `git revert`, and never rewrite a branch that other people already have (sections 11.2 and 11.8).
2. The committed part comes back through the reflog, the staged part survives as a blob without a name, and the part that was never staged is gone, because Git never stored it (section 11.5).
3. Reverting a merge undoes its content and leaves its place in the commit graph, so a later merge of the same branch skips everything the first merge brought (section 11.9).

11.2 The map: four places and one deciding question

The four places. Every undo command writes to one or more of these, and to nothing else:

Place	What it holds
Working tree (Chapter 4)	The files on disk, including edits that exist nowhere else
Index (Chapter 5)	The proposed next commit: one blob ID per tracked path
Current branch ref (Chapter 7)	One commit ID under <code>refs/heads/</code> ; HEAD normally points at this ref
History (Chapters 6 and 12)	The commits reachable from that ref. After a push, other repositories hold them too

Picture. One file with a different version in each place, and the direction in which each command copies content:



Git's manual draws the same lines: `git restore` "does not update your branch", `git reset` is about "updating your branch, moving the tip", and `git revert` is about "making a new commit that reverts the changes made by other commits" ([git\(1\)](#), section "Reset, restore and revert").

The deciding question: is this history private or shared? A commit is *shared* when a ref that other people can fetch reaches it. Everything else is *private*: it exists only in your repository.

- Private history may be **rewritten**. `git reset`, `git commit --amend` and `git rebase` replace commits with new ones or drop them from the branch. Nobody else can notice, because nobody else has the old commits.
- Shared history is corrected by **adding** commits. `git revert` records the inverse change on top, and everyone's next pull brings it in like any other commit.

The reason is mechanical, not etiquette. A branch in a teammate's clone is a ref that holds a commit ID. If you move your branch backwards, their ref still names the commits you removed. The server rejects your plain push as a non-fast-forward (Lab 8.2). If you force the push, the next teammate who pushes from a clone that still has those commits puts them straight back, or merges them with your replacements (Chapter 9, Rebase, and Chapter 12, Remote Operations).

Before any undo that touches commits, check which side of the line you are on:

```
git fetch                # refresh what you know about the server
git status --sb          # "ahead 2": two commits the upstream lacks
git log --oneline @{u}..  # the commits that are still private
git branch -r --contains <commit> # remote-tracking branches that contain <commit>
```

The last two commands read remote-tracking branches, which record the state of the last fetch. Without the `git fetch` in front, they answer for an older moment.

GitHub, not Git. A new ruleset on GitHub has "Block force pushes" enabled by default, and classic branch protection rules disable force pushes by default ([available rules](#)). On a protected `main`, rewriting is normally not even available, and a revert commit is the only route.

11.3 `git restore`: copy a stored version over files or index entries

In one sentence. `git restore` copies a stored version of the named paths into the working tree, into the index, or into both, and moves no ref.

Analogy. A photocopier with two output trays: you pick the original and the tray, and whatever lay in the tray is replaced. The analogy breaks at the replaced sheet. You do not get it back.

Precisely. Chapter 4 (The Working Tree) introduced the command. Two choices define every form of it (`git-restore`): the destination (the working tree by default, the index with `--staged`, both with `--staged --worktree`) and the source (`--source=<tree>`; without it, the index when only the working tree is written, and HEAD as soon as `--staged` is given). Every form that writes the working tree is . `--staged` alone is .

Inside `.git`. A restore rewrites files, or blob IDs in index entries. No object is created, no ref moves, and no reflog entry is written: there is no record to go back to.

See it. One file, `eval.yaml`, with a different threshold in each place: 0.70 in HEAD, 0.80 staged, 0.90 on disk. Every command below starts from a fresh copy of this state.

```
$ git log --oneline
9d940ca Raise threshold to 0.70
d44696e Raise threshold to 0.60
4d17008 Add eval config
$ git show HEAD:eval.yaml
threshold: 0.70
$ git show :eval.yaml
threshold: 0.80
$ cat eval.yaml
threshold: 0.90
$ git status -s
MM eval.yaml
```

Plain `git restore` copies the index over the file. The edit to 0.90 is destroyed and the staged 0.80 stays:

```
$ git restore eval.yaml
$ git show :eval.yaml
threshold: 0.80
$ cat eval.yaml
threshold: 0.80
$ git status -s
M  eval.yaml
```

`--staged` copies HEAD over the index entry and leaves the file alone. This is "unstage":

```
$ git restore --staged eval.yaml
$ git show :eval.yaml
threshold: 0.70
$ cat eval.yaml
threshold: 0.90
$ git status -s
M  eval.yaml
```

`git reset -- <path>` is the older spelling of the same operation. The result is identical, and reset adds a status line:

```
$ git reset -- eval.yaml
Unstaged changes after reset:
M   eval.yaml
$ git show :eval.yaml
threshold: 0.70
```

```
$ cat eval.yaml
threshold: 0.90
$ git status -s
M eval.yaml
```

In both transcripts the staged version, 0.80, is no longer in the index, and it is not on disk either, because the file had moved on to 0.90. A version that existed only in the index is now a blob that nothing names (section 11.5 shows how to find one).

With `--source` the content comes from a commit. Without `--staged`, only the file is written, so the index still holds 0.80 and the status stays MM:

```
$ git restore --source=HEAD~2 eval.yaml
$ git show :eval.yaml
threshold: 0.80
$ cat eval.yaml
threshold: 0.50
$ git status -s
MM eval.yaml
```

Add `--staged --worktree` and both places receive the old version. The last command proves that HEAD did not move: this is an old version of one file placed on top of the current commit, ready to be committed as a new change.

```
$ git restore --source=HEAD~2 --staged --worktree eval.yaml
$ git show :eval.yaml
threshold: 0.50
$ cat eval.yaml
threshold: 0.50
$ git status -s
M eval.yaml
$ git log --oneline -1
9d940ca Raise threshold to 0.70
```

The remaining form, `git restore --staged --worktree <path>`, sets both places to HEAD's version. Older scripts write `git checkout --<path>` for `git restore <path>`, `git reset HEAD <path>` for `git restore --staged <path>`, and `git checkout <commit> -- <path>` for the last transcript, index included (labs/run ch11/restore-variants shows this one).

Restoring part of a file. `git restore -p` walks through the differences hunk by hunk and asks about each one. Here `score.py` has two unstaged changes, a debug line and a real fix. Answering `y` to the first hunk and `n` to the second discards only the debug line. The demo pipes the two answers in, so each prompt is followed directly by the next output:

```
$ printf 'y\nn\n' | git restore -p score.py
diff --git a/score.py b/score.py
index 9f65fd2..564c708 100644
--- a/score.py
+++ b/score.py
@@ -2,6 +2,7 @@ import json

def load(path):
+ print("DEBUG loading", path)
    with open(path) as f:
        return [json.loads(line) for line in f]

(1/2) Discard this hunk from worktree [y,n,q,a,d,k,K,j,J,g,/,e,p,P,?]? @@ -16,7 +17,7 @@ def exact_match(pred, gold):

def accuracy(rows):
    hits = sum(exact_match(r["pred"], r["gold"]) for r in rows)
- return hits / len(rows)
+ return hits / max(len(rows), 1)

if __name__ == "__main__":
(2/2) Discard this hunk from worktree [y,n,q,a,d,K,J,g,/,e,p,P,?]?
```

```
$ git diff
diff --git a/score.py b/score.py
index 9f65fd2..7c6b98d 100644
--- a/score.py
+++ b/score.py
@@ -16,7 +16,7 @@ def exact_match(pred, gold):

def accuracy(rows):
    hits = sum(exact_match(r["pred"], r["gold"]) for r in rows)
- return hits / len(rows)
+ return hits / max(len(rows), 1)

if __name__ == "__main__":
```

`-p` also combines with `--staged` and with `--source`.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git restore --staged <path></code> , <code>git reset --<path></code>	unchanged	entries for <path> copied from HEAD	unchanged	unchanged	unchanged	unchanged	unchanged
<code>git restore --staged --worktree <path></code>	<path> overwritten from HEAD	copied from HEAD	unchanged	unchanged	unchanged	unchanged	unchanged

The rows for `git restore <path>` and the two `--source` forms are in the state table of Chapter 4, section 4.7.

The same section answers the five questions for the  forms. In short: they destroy unstaged edits in the named paths without asking, the preview is `git diff -- <path>` (or `git diff <commit> -- <path>`), and nothing brings back content that was never staged.

In production. An evaluation regressed after someone tuned `prompts/system.txt` three commits ago, and those commits also contain good changes. `git restore --source=<good commit> prompts/system.txt` followed by an ordinary commit puts one file back and leaves history intact. It is safe on shared branches, because it only adds a commit (Lab 8.7, card 4).

11.4 `git reset`: move the branch, then choose how far the change spreads

In one sentence.  `git reset <commit>` makes the current branch point at <commit>, and its mode decides whether the index (`--mixed`, the default) and the working tree (`--hard`) are rewritten to match.

Analogy. A bookmark in a lab notebook. `--soft` moves the bookmark back. `--mixed` also clears the page you had drafted for the next entry. `--hard` also clears the desk. The analogy breaks in a useful way: no page is torn out. The entries after the bookmark stay in the notebook, and Git discards unmarked pages only after weeks (section 11.5).

Precisely. With a commit argument and no path, reset does up to three things ([git-reset](#)):

1. It writes the current commit ID to `ORIG_HEAD` and makes the branch that HEAD points at name <commit>. `--soft` does only this.
2. It rewrites the index to match <commit>. `--mixed` does this as well.
3. It rewrites the tracked files in the working tree to match <commit>. Only `--hard` does this.

<commit> defaults to HEAD. So `git reset` alone unstages everything, and `git reset --hard` alone discards every uncommitted change to tracked files. On a detached HEAD there is no branch to move, and HEAD itself is rewritten.

See it. The starting state is the one from section 11.3: 0.70 in HEAD (commit `9d940ca`), 0.80 in the index, 0.90 on disk. Each mode runs in its own copy.

```
$ git reset --soft HEAD~1
$ git log --oneline
d44696e Raise threshold to 0.60
4d17008 Add eval config
$ git show HEAD:eval.yaml
threshold: 0.60
$ git show :eval.yaml
threshold: 0.80
$ cat eval.yaml
threshold: 0.90
$ git status -s
MM eval.yaml
```

The log lost a commit and HEAD now holds 0.60. The index and the file are untouched. The status is still `MM`, but it is now measured against the older commit: `git commit` at this point would record 0.80 directly on top of `d44696e`.

```
$ git reset --mixed HEAD~1
Unstaged changes after reset:
M    eval.yaml
$ git log --oneline
d44696e Raise threshold to 0.60
4d17008 Add eval config
$ git show HEAD:eval.yaml
threshold: 0.60
$ git show :eval.yaml
threshold: 0.60
$ cat eval.yaml
threshold: 0.90
$ git status -s
M eval.yaml
```

The index now equals the new HEAD, and Git reports what is left as unstaged. The file still says 0.90.

```
$ git reset --hard HEAD~1
HEAD is now at d44696e Raise threshold to 0.60
$ git log --oneline
d44696e Raise threshold to 0.60
4d17008 Add eval config
```

```
$ git show HEAD:eval.yaml
threshold: 0.60
$ git show :eval.yaml
threshold: 0.60
$ cat eval.yaml
threshold: 0.60
$ git status -s
```

All three places hold 0.60. The staged 0.80 and the unstaged 0.90 are gone from every place you can name.

eval.yaml in	HEAD	Index	Working tree	git status -s
before	0.70	0.80	0.90	MM
after git reset --soft HEAD~1	0.60	0.80	0.90	MM
after git reset --mixed HEAD~1	0.60	0.60	0.90	M
after git reset --hard HEAD~1	0.60	0.60	0.60	clean

Inside .git.

```
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
9d940caa6545e66b9555bf56a26ea9242b19e402
$ git reset --soft HEAD~1
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
d44696eb333776b2089f9627a30f31733c0df8be
$ cat .git/ORIG_HEAD
9d940caa6545e66b9555bf56a26ea9242b19e402
$ git reflog -2
d44696e HEAD@{0}: reset: moving to HEAD~1
9d940ca HEAD@{1}: commit: Raise threshold to 0.70
$ git reflog show -2 main
d44696e main@{0}: reset: moving to HEAD~1
9d940ca main@{1}: commit: Raise threshold to 0.70
```

The file `HEAD` still says `ref: refs/heads/main`. What changed is the content of `refs/heads/main`. `ORIG_HEAD` holds the previous ID, and both `reflogs` gained the line `reset: moving to HEAD~1`. No object was deleted: commit `9d940ca` is still in the object database.

Picture.

```
before                                after git reset HEAD~1 (any mode)

4d17008---d44696e---9d940ca          4d17008---d44696e      main (HEAD -> main)
    |                               /
    |                               \
main (HEAD -> main)                9d940ca   no branch: ORIG_HEAD, HEAD@{1}, main@{1}
```

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git reset --soft <commit></code>	unchanged	unchanged	file unchanged; resolves to <commit>	set to <commit>	ORIG_HEAD written; one reflow line each for HEAD and the branch	unchanged	unchanged
<code>git reset [--mixed] <commit></code>	unchanged	rewritten to match <commit>	as above	set to <commit>	as above	unchanged	unchanged
<code>git reset --hard <commit></code>	tracked files rewritten to match <commit>; uncommitted changes destroyed	rewritten to match <commit>	as above	set to <commit>	as above	unchanged	unchanged

ORIG_HEAD. ORIG_HEAD holds one commit ID. The manual says it is created by commands that move HEAD "in a drastic way" (`git am`, `git merge`, `git rebase`, `git reset`), to record where HEAD was before ([gitrevisions](#)). Resetting to it undoes the reset:

```
$ git reset --soft ORIG_HEAD
$ git log --oneline -1
9d940ca Raise threshold to 0.70
$ git status -s
MM eval.yaml
$ git rev-parse --short ORIG_HEAD
d44696e
```

The undo was itself a reset, so it overwrote `ORIG_HEAD`, which now names `d44696e`. `ORIG_HEAD` is one slot, not a history, and more commands write it than the manual lists. Here a merge is followed by a stash:

```
$ git merge feature/reranker
Merge made by the 'ort' strategy.
```

```

rerank.py | 2 ++
1 file changed, 2 insertions(+)
create mode 100644 rerank.py
$ git log --oneline -1 ORIG_HEAD
5522f41 Add serving config
$ git stash push -m "wip: new recall numbers"
Saved working directory and index state On main: wip: new recall numbers
$ git log --oneline -1 ORIG_HEAD
26ec8ae Merge branch 'feature/reranker'

```

After the merge, `ORIG_HEAD` named `5522f41`, the commit to go back to. `git stash push` ends with an internal reset to `HEAD`, every reset writes `ORIG_HEAD`, and so it now names the merge commit itself: `git reset --hard ORIG_HEAD` would leave the merge in place. The rule is to use `ORIG_HEAD` only as the very next command. After anything else, read the reflog, which has the whole sequence:

```

$ git reflog -3
26ec8ae HEAD@{0}: reset: moving to HEAD
26ec8ae HEAD@{1}: merge feature/reranker: Merge made by the 'ort' strategy.
5522f41 HEAD@{2}: commit: Add serving config
$ git reset --hard HEAD@{2}
HEAD is now at 5522f41 Add serving config
$ git log --oneline
5522f41 Add serving config
9c3779d Add BM25 ranker and baseline metrics
$ git stash pop -q
$ git status -s
M metrics.txt

```

In production. Three commits named "wip", "wip 2" and "fix typo" before a review: `git reset --soft @{u}` moves the branch back to what is pushed and leaves the combined change staged for one clean commit (Lab 8.7, card 1). A commit that mixes a refactoring with a behavior change: `git reset HEAD~1` leaves both in the working tree, unstaged, ready to be split with `git add -p` (Chapter 5).

11.5 Why `git reset --hard` is

`--soft` and `--mixed` never write a file. `--hard` overwrites files, and what that costs depends on one question for each piece of work: **did Git ever store it as an object?** The demo puts four kinds of work into one repository:

```

$ git log --oneline
e574174 Blend dense scores into ranker
67b1722 Record baseline metrics
9a98a06 Add BM25 ranker
$ printf 'alpha: [0.5, 0.7, 0.9]\n' > sweep.yaml
$ git add sweep.yaml
$ printf 'recall@10: 0.78\n' > metrics.txt
$ printf 'try alpha=0.9 next\n' > notes.txt
$ git status -s
M metrics.txt
A sweep.yaml
?? notes.txt

```

There is the committed change `e574174`; `sweep.yaml`, staged and never committed; the edit to `metrics.txt`, never staged; and `notes.txt`, untracked.

```

$ git reset --hard HEAD~1
HEAD is now at 67b1722 Record baseline metrics
$ git log --oneline
67b1722 Record baseline metrics
9a98a06 Add BM25 ranker
$ git status -s
?? notes.txt
$ ls
metrics.txt
notes.txt
rank.py
$ cat metrics.txt
recall@10: 0.71

```

The branch is back at `67b1722`, `metrics.txt` says 0.71 again, and `sweep.yaml` is no longer on disk. Only `notes.txt` was left alone.

Committed work: the reflog has it. The reset moved a ref and deleted no object, and the reflog recorded the move:

```

$ git reflog -2
67b1722 HEAD@{0}: reset: moving to HEAD~1
e574174 HEAD@{1}: commit: Blend dense scores into ranker
$ git reset --hard HEAD@{1}
HEAD is now at e574174 Blend dense scores into ranker
$ git log --oneline
e574174 Blend dense scores into ranker
67b1722 Record baseline metrics
9a98a06 Add BM25 ranker
$ git status -s
?? notes.txt

```

Staged work: an object without a name. `git add` wrote the content of `sweep.yaml` into the object database as a blob. The hard reset removed the index entry, the only thing that referred to it. `git fsck --lost-found` lists objects that nothing refers to and writes a copy of each under `.git/lost-found/`:

```
$ git fsck --lost-found
dangling blob acc57c7b481e66cf03ec96980cc21fadcadc8a5b
$ git cat-file -p acc57c7
alpha: [0.5, 0.7, 0.9]
$ ls .git/lost-found/other
acc57c7b481e66cf03ec96980cc21fadcadc8a5b
$ git cat-file -p acc57c7 > sweep.yaml
$ git status -s
?? notes.txt
?? sweep.yaml
```

The content is back, the file name is not: a blob stores content only, and the name lived in the index entry. After a reset with ten staged files you get ten anonymous blobs and identify them by reading them. (On Git 2.55 `--lost-found` does not count relog entries as references: run one step earlier, it would have listed commit `e574174` too.)

Never-staged work: nothing to find. `git hash-object --stdin` computes the ID that a piece of content would have. For the staged file the object exists. For the unstaged edit it does not:

```
# The staged file: its content was hashed by "git add", so the object exists.
$ printf 'alpha: [0.5, 0.7, 0.9]\n' | git hash-object --stdin
acc57c7b481e66cf03ec96980cc21fadcadc8a5b
$ git cat-file -t acc57c7b481e66cf03ec96980cc21fadcadc8a5b
blob
# The never-staged edit: this is the ID it would have had. No such object was ever written.
$ printf 'recall@10: 0.78\n' | git hash-object --stdin
b812359d6f71b2a9857c24b3c631b8ea58213fcd
$ git cat-file -t b812359d6f71b2a9857c24b3c631b8ea58213fcd
fatal: git cat-file: could not get object info
[exit status: 128]
```

Untracked files: untouched, with one exception. `notes.txt` survived because a reset touches only paths that are in the index or in the target commit. The exception is an untracked file whose path the target commit tracks. Here `sweep.yaml` was removed from tracking one commit ago, and the copy on disk holds two days of tuning:

```
$ git log --oneline
9a6d9e0 Stop tracking sweep config
533927f Add sweep config
5e23be0 Add BM25 ranker
$ git status -s
?? sweep.yaml
$ cat sweep.yaml
alpha: 0.9 # two days of tuning
$ git reset --keep HEAD~1
error: Untracked working tree file 'sweep.yaml' would be overwritten by merge.
fatal: Could not reset index file to revision 'HEAD~1'.
[exit status: 128]
$ git reset --hard HEAD~1
HEAD is now at 533927f Add sweep config
$ cat sweep.yaml
alpha: 0.7
```

`--keep` refused. `--hard` overwrote the file without a word. The manual admits it in one clause: `--hard` "may overwrite untracked files".

```
Observed behavior : After "git reset --hard", an edit is gone. Neither the relog nor fsck shows it.
Git state          : HEAD, index and working tree agree. No blob with the lost content exists.
Mechanism          : Git writes a blob when content is staged or committed. An edit that was only
                    saved in the editor was never stored.
Root cause         : The lost version existed only in the working tree, and --hard overwrote the file.
Why Git does this  : The working tree is the one place without history, and --hard is defined as
                    "make it match the commit".
Correct fix        : None inside Git. Editor history, a backup, or retyping.
Prevention         : Commit or stash before a hard reset, or use "git reset --keep" (section 11.6).
```

How long the first two kinds survive is set by two defaults ([git-config](#)). A relog entry for a commit that its branch no longer reaches is kept for 30 days (`gc.reflogExpireUnreachable`), and garbage collection removes unreachable objects once they are older than two weeks (`gc.pruneExpire`). Chapter 13 (Recovery) covers the relog and `git fsck` in full.

For this  command:

- **What it changes:** the branch ref, the index, and every tracked file that differs from the target commit.
- **What it can destroy:** every uncommitted change to tracked files, and untracked files that are in the target's way.
- **How to preview:** three read-only commands.

```
# Commits that will leave the branch (they stay in the relog):
$ git log --oneline HEAD~2..HEAD
e574174 Blend dense scores into ranker
```



```
67b1722 Record baseline metrics
# Uncommitted work (a hard reset destroys the first two lines and keeps the third):
$ git status -s
M metrics.txt
A sweep.yaml
?? notes.txt
# Tracked content that differs from the last commit:
$ git diff HEAD --stat
metrics.txt | 2 +-
sweep.yaml | 1 +
2 files changed, 2 insertions(+), 1 deletion(-)
```

- **How to recover:** commits through the reflog or `ORIG_HEAD`, staged content through `git fsck --lost-found`, nothing else at all.
- **When it is appropriate:** when `git status` is clean, or after you have read `git diff HEAD` and want none of it. Otherwise turn the work into objects first, with two commands:

```
$ git stash push -u -m "before reset to HEAD~2"
Saved working directory and index state On main: before reset to HEAD~2
$ git branch backup/before-reset
$ git status -s
$ git reset --hard HEAD~2
HEAD is now at 9a98a06 Add BM25 ranker
$ git log --oneline
9a98a06 Add BM25 ranker
```

```
$ git log --oneline backup/before-reset
e574174 Blend dense scores into ranker
67b1722 Record baseline metrics
9a98a06 Add BM25 ranker
$ git stash list
stash@{0}: On main: before reset to HEAD~2
$ git stash show --include-untracked
metrics.txt | 2 +-
notes.txt   | 1 +
sweep.yaml | 1 +
3 files changed, 3 insertions(+), 1 deletion(-)
```

The stash entry holds the uncommitted work and the branch holds the commits. Delete both once you are sure.

11.6 `--keep` and `--merge`: resets that check before they overwrite

In one sentence. 🟡 `git reset --keep <commit>` moves the branch like `--hard`, carries your uncommitted changes across, and refuses to run when it would have to overwrite one of them; 🔴 `git reset --merge <commit>` does the same for unstaged changes and throws staged changes away.

Precisely. Both modes update the files that differ between HEAD and `<commit>`, and they differ in what happens to your local changes (`git-reset`).

- `--keep` leaves local changes in all other files alone. The index is reset, so a staged change comes out unstaged. If a file that must be rewritten has local changes, the reset aborts and nothing moves.
- `--merge` keeps unstaged changes and discards staged ones, from the index and from the file. It exists for backing out of a merge, where whatever is staged is merge output and should go.

See it. One commit to drop (it changed `rank.py`), a staged change in `serve.yaml` (batch size 32 to 64) and an unstaged change in `metrics.txt` (0.71 to 0.74):

```
$ git log --oneline
92831a4 Blend dense scores into ranker
c6fd923 Add ranker, serving config and baseline metrics
$ git status -s
M metrics.txt
M serve.yaml
```

```
$ git reset --keep HEAD~1
$ git log --oneline
c6fd923 Add ranker, serving config and baseline metrics
$ git status -s
M metrics.txt
M serve.yaml
$ cat serve.yaml metrics.txt
batch_size: 64
recall@10: 0.74
```

Both edits survived `--keep`, and both are now unstaged.

```
$ git reset --merge HEAD~1
$ git log --oneline
c6fd923 Add ranker, serving config and baseline metrics
$ git status -s
```

```
M metrics.txt
$ cat serve.yaml metrics.txt
batch_size: 32
recall@10: 0.74
```

After `--merge` the unstaged edit to `metrics.txt` is still there, and the staged `batch_size: 64` is gone: the file says 32 again. `--hard` from the same state loses both (`labs/run ch11/reset-keep-merge`).

When the local change is in a file that the dropped commit also changed, `--keep` stops:

```
$ git log --oneline
92831a4 Blend dense scores into ranker
c6fd923 Add ranker, serving config and baseline metrics
$ git status -s
M rank.py
$ git reset --keep HEAD~1
error: Entry 'rank.py' not uptodate. Cannot merge.
fatal: Could not reset index file to revision 'HEAD~1'.
[exit status: 128]
$ git log --oneline -1
92831a4 Blend dense scores into ranker
$ git status -s
M rank.py
```

The branch did not move. The message speaks of a merge because `--keep` carries local changes across the move the way a branch switch does (the Git 1.7.1 release notes introduce the option with that comparison).

Situation	<code>--keep</code>	<code>--merge</code>	<code>--hard</code>
Staged change in a file the reset does not rewrite	kept, becomes unstaged	discarded	discarded
Unstaged change in such a file	kept	kept	discarded
Unstaged change in a file the reset must rewrite	refuses	refuses	discarded
Staged change in a file the reset must rewrite	refuses	discarded	discarded
Untracked file at a path the target commit tracks	refuses	refuses	overwritten

The rule that follows: to drop commits, type `--keep`, not `--hard`. On a clean tree the two do the same thing. On a dirty tree `--keep` carries your work across or stops and says why.

`--merge` earns its 🚫 through staged work only: preview with `git diff --cached`. Its proper uses are `git reset --merge ORIG_HEAD`, which undoes a completed merge and keeps unrelated edits (section 11.13), and `git reset --merge` without a commit, which clears a conflicted index (section 11.11).

In the state table of section 11.4, both modes read like `--hard` in every column except Working tree, where the table above applies.

11.7 `git commit --amend`: a soft reset plus a new commit

In one sentence. 🟡 `git commit --amend` replaces the last commit with a new commit that has the same parent and is built from the current index.

Precisely. Git's manual calls it "a rough equivalent" of `git reset --soft HEAD^`, some changes, and `git commit -c ORIG_HEAD` ([git-commit](#)). Chapter 6 (Commits) dissects the commit object that results. Here the equivalence is run both ways from one starting state: a line that was forgotten in the last commit.

```
$ git log --oneline
d59fe4c Add serving config
9a98a06 Add BM25 ranker
$ git status -s
M serve.yaml
$ git add serve.yaml
$ git commit --amend --no-edit
[main ab64793] Add serving config
Date: Mon Sep 7 10:04:00 2026 +0530
1 file changed, 2 insertions(+)
create mode 100644 serve.yaml
$ git log --oneline
ab64793 Add serving config
9a98a06 Add BM25 ranker
$ git reflog -2
ab64793 HEAD@{0}: commit (amend): Add serving config
d59fe4c HEAD@{1}: commit: Add serving config
$ git show -s --format="%h parent %p tree %t" HEAD HEAD@{1}
ab64793 parent 9a98a06 tree b35836e
d59fe4c parent 9a98a06 tree 7e9e572
```

```
$ git add serve.yaml
$ git reset --soft HEAD~1
$ git status -s
A serve.yaml
```

```
$ git commit -q -C ORIG_HEAD
$ git log --oneline
732ac95 Add serving config
9a98a06 Add BM25 ranker
$ git show -s --format="%h parent %p tree %t" HEAD ORIG_HEAD
732ac95 parent 9a98a06 tree b35836e
d59fe4c parent 9a98a06 tree 7e9e572
```

Both routes end with a commit whose parent is `9a98a06` and whose tree is `b35836e`. The two commit IDs differ only because the committer timestamps differ. An amend is therefore a rewrite of exactly one commit, and the rule of section 11.2 applies: private history only. Chapter 6 shows the diverged branch that results from amending a pushed commit.

One difference matters when you want the old commit back. `--amend` does not write `ORIG_HEAD`. The previous commit is one step back in the reflog:

```
$ git rev-parse --verify --short ORIG_HEAD
fatal: Needed a single revision
[exit status: 128]
$ git reset --soft HEAD@{1}
$ git log --oneline
d59fe4c Add serving config
9a98a06 Add BM25 ranker
$ git status -s
M serve.yaml
$ git reflog -3
d59fe4c HEAD@{0}: reset: moving to HEAD@{1}
ab64793 HEAD@{1}: commit (amend): Add serving config
d59fe4c HEAD@{2}: commit: Add serving config
```

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git commit</code> <code>--amend</code>	unchanged	unchanged; it becomes the tree of the new commit	file unchanged; resolves to the new commit	set to the new commit	new commit object; reflog line <code>commit (amend); ORIG_HEAD</code> not written	unchanged	unchanged

11.8 `git revert`: a new commit that applies the inverse change

In one sentence. 🟡 `git revert <commit>` adds a commit whose change is the opposite of what `<commit>` changed, and removes nothing.

Analogy. A reversing entry in a ledger. A posted entry is never erased. A second entry with the opposite sign is posted, and both stay visible. The analogy breaks in two places. A ledger reversal always nets to zero, while a revert is computed against the files as they are now, so later changes to the same lines produce a conflict. And a revert reverses one commit, not everything since.

Precisely. A revert is a three-way merge (Chapter 8, Merge) with unusual inputs: the base is the commit being reverted, "ours" is HEAD, and "theirs" is the parent of the reverted commit ([How git cherry-pick and revert use 3-way merge](#)). Chapter 10 (Cherry-pick) shows the mirror image. Three consequences follow. Any commit in the history can be reverted, not only the last. A revert can conflict. And the result is an ordinary commit on top of HEAD, so the branch only moves forward.

See it. `d51208b` disabled a cache. One more commit has been made since.

```
$ git log --oneline
73572b2 Record baseline metrics
d51208b Disable cache
a49e359 Cut timeout to 5s
6cde22d Add serving config
$ git revert --no-edit d51208b
[main 124de33] Revert "Disable cache"
Date: Mon Sep 7 10:08:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline
124de33 Revert "Disable cache"
73572b2 Record baseline metrics
d51208b Disable cache
a49e359 Cut timeout to 5s
6cde22d Add serving config
$ cat serve.yaml
timeout_s: 5
retries: 2
cache: on
```

The log grew by one commit, and `d51208b` is still in it. The cache is on again, and the timeout change of `a49e359` is untouched.

Inside .git.

```
$ git cat-file -p HEAD
tree e9938f9d1028c2cd21aaeb07fcf0d0660529e062
parent 73572b26215e527178ad8d299a39fbe669e07869
author Lab User <you@example.com> 1788755880 +0530
committer Lab User <you@example.com> 1788755880 +0530
```

```
Revert "Disable cache"
```

```
This reverts commit d51208bb375a31b82829c887ecdfe818c7fcde5f.  
$ git reflog -1  
124de33 HEAD@{0}: revert: Revert "Disable cache"
```

One new commit, with the previous HEAD commit as its only parent and a generated message. The reflog line starts with `revert: . ORIG_HEAD` is not written. While a revert is stopped at a conflict, `REVERT_HEAD` names the commit being reverted and `.git/sequencer/` holds the steps still to do.

Three refusals. Reverting the same commit again finds nothing to commit:

```
$ git revert --no-edit d51208b  
On branch main  
nothing to commit, working tree clean  
[exit status: 1]
```

On `git revert`, `-m` is not a message but the parent number for reverting a merge (section 11.9). The reason for a revert goes into the editor, which `git revert` opens when it runs in a terminal:

```
$ git revert -m "Cache must stay on" HEAD~1  
error: option 'mainline' expects a number greater than zero  
[exit status: 129]
```

And the index must match HEAD. The manual asks for a clean working tree; Git 2.55 accepts unstaged edits in files that the revert does not touch:

```
# An unstaged change in a file that the revert does not touch: allowed.  
$ printf 'recall@10: 0.74\n' > metrics.txt  
$ git revert --no-edit d51208b  
[main 029666f] Revert "Disable cache"  
Date: Mon Sep 7 10:17:00 2026 +0530  
1 file changed, 1 insertion(+), 1 deletion(-)  
$ git status -s  
M metrics.txt  
# The same change, staged: refused, because the index must match HEAD.  
$ git add metrics.txt  
$ git revert --no-edit a49e359  
error: your local changes would be overwritten by revert.  
hint: commit your changes or stash them to proceed.  
fatal: revert failed  
[exit status: 128]
```

Several commits. A range `A..B` excludes `A` and is reverted newest first, one revert commit per original:

```
$ git revert --no-edit HEAD~3..HEAD~1  
[main 0978d3a] Revert "Disable cache"  
Date: Mon Sep 7 10:21:00 2026 +0530  
1 file changed, 1 insertion(+), 1 deletion(-)  
[main 209e7c2] Revert "Cut timeout to 5s"  
Date: Mon Sep 7 10:21:00 2026 +0530  
1 file changed, 1 insertion(+), 1 deletion(-)  
$ git log --oneline  
209e7c2 Revert "Cut timeout to 5s"  
0978d3a Revert "Disable cache"  
73572b2 Record baseline metrics  
d51208b Disable cache  
a49e359 Cut timeout to 5s  
6cde22d Add serving config  
$ cat serve.yaml  
timeout_s: 30  
retries: 2  
cache: on
```

With `-n` (`--no-commit`) the inverse changes go into the index and the working tree and nothing is committed, so one commit of your own can carry them all:

```
$ git revert -n HEAD~3..HEAD~1  
$ git status -s  
M serve.yaml  
$ git commit -m "Revert timeout and cache changes from the latency experiment"  
[main 716dfe1] Revert timeout and cache changes from the latency experiment  
1 file changed, 2 insertions(+), 2 deletions(-)  
$ git log --oneline -3  
716dfe1 Revert timeout and cache changes from the latency experiment  
73572b2 Record baseline metrics  
d51208b Disable cache
```

Conflicts. `a49e359` changed the timeout from 30 to 5. A later commit changed it from 5 to 8. The revert wants to turn 5 back into 30, and the line no longer says 5:

```
$ git log --oneline
fe1b93a Tune timeout to 8s
73572b2 Record baseline metrics
d51208b Disable cache
a49e359 Cut timeout to 5s
6cde22d Add serving config
$ git revert --no-edit a49e359
Auto-merging serve.yaml
CONFLICT (content): Merge conflict in serve.yaml
error: could not revert a49e359... Cut timeout to 5s
hint: After resolving the conflicts, mark them with
hint: "git add/rm <paths>", then run
hint: "git revert --continue".
hint: You can instead skip this commit with "git revert --skip".
hint: To abort and get back to the state before "git revert",
hint: run "git revert --abort".
hint: Disable this message with "git config set advice.mergeConflict false"
[exit status: 1]
```

```
$ git status -s
UU serve.yaml
$ cat serve.yaml
<<<<<< HEAD
timeout_s: 8
=====
timeout_s: 30
>>>>>> parent of a49e359 (Cut timeout to 5s)
retries: 2
cache: off
$ git ls-files -s serve.yaml
100644 41e864ea1714917500825ad4d31512bfcf3210f4 1      serve.yaml
100644 a9bab3597b4e820f98072a889124e79c2228134e 2      serve.yaml
100644 6c8d072bcfd31ba25469b03c0024d6f4a5098848 3      serve.yaml
$ git rev-parse a49e359:serve.yaml HEAD:serve.yaml a49e359~1:serve.yaml
41e864ea1714917500825ad4d31512bfcf3210f4
a9bab3597b4e820f98072a889124e79c2228134e
6c8d072bcfd31ba25469b03c0024d6f4a5098848
$ git rev-parse --short REVERT_HEAD
a49e359
```

The three index stages prove the "Precisely" paragraph. Stage 1, the base, is the file as it is in `a49e359`, the commit being reverted. Stage 2 is HEAD's version. Stage 3 is the file in the parent of `a49e359`. Git cannot know whether 8 or 30 is right today. You decide, stage the result and continue:

```
$ printf 'timeout_s: 30\nretries: 2\ncache: off\n' > serve.yaml
$ git add serve.yaml
$ git revert --continue
[main 4761b40] Revert "Cut timeout to 5s"
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline -2
4761b40 Revert "Cut timeout to 5s"
fe1b93a Tune timeout to 8s
```

A range that stops halfway. When the second revert of a range conflicts, the first is already committed. Here `git revert --no-edit HEAD~4..HEAD~2` has stopped at its second step:

```
$ git log --oneline -2
407f639 Revert "Disable cache"
f6c769e Tune timeout to 8s
$ git status
On branch main
You are currently reverting commit a49e359.
  (fix conflicts and run "git revert --continue")
  (use "git revert --skip" to skip this patch)
  (use "git revert --abort" to cancel the revert operation)

Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
    both modified:   serve.yaml

no changes added to commit (use "git add" and/or "git commit -a")
$ ls .git/sequencer
abort-safety
head
opts
todo
$ cat .git/sequencer/todo
revert a49e359 Cut timeout to 5s
$ git rev-parse --short REVERT_HEAD
a49e359
$ git log --oneline -1 $(cat .git/sequencer/head)
f6c769e Tune timeout to 8s
```

Exit	Result
<code>git revert --continue</code>	After you resolve and <code>git add</code> : commits this step and runs the remaining ones
<code>git revert --skip</code>	Drops this step, keeps the reverts already made, runs the remaining ones
<code>git revert --abort</code>	Returns the branch to the commit recorded in <code>.git/sequencer/head</code> ; revert commits already made leave the branch
<code>git revert --quit</code>	Forgets the sequence; keeps the reverts already made and leaves the conflict in the index and in the files

```
$ git revert --abort
$ git log --oneline -2
f6c769e Tune timeout to 8s
73572b2 Record baseline metrics
$ git status -s
$ git reflog -2
f6c769e HEAD@{0}: reset: moving to f6c769e91225728dcb887fa30d34b1c1278764e5
407f639 HEAD@{1}: revert: Revert "Disable cache"
$ test -d .git/sequencer
[exit status: 1]
```

`labs/run ch11/revert-sequence` shows `--quit` and `--skip` from the same stopped state.

Picture.

before	after <code>git revert d51208b</code>
6cde22d--a49e359--d51208b--73572b2 main (HEAD)	6cde22d--a49e359--d51208b--73572b2--124de33 main (HEAD)
tree of 124de33 = tree of 73572b2 with the change of d51208b undone	

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git revert <commit></code>	files touched by the inverse change rewritten	updated to the tree of the new commit	file unchanged; resolves to the new commit	advanced by one commit	new tree and commit; reflog line <code>revert:</code>	unchanged until you push	unchanged until you push
<code>git revert -n <commit></code>	inverse change applied	inverse change staged	unchanged	unchanged	<code>REVERT_HEAD</code> and <code>MERGE_MSG</code> until you commit	unchanged	unchanged

In production. A change that raised a batch size was merged, deployed by CI, and exhausts memory on the feature workers. Three teammates have pulled it. `git revert <commit>` followed by `git push` needs no force, takes the same route as any other commit (a pull request, if `main` requires one), and reaches every clone as a normal update. The Git FAQ names this as the usual way ([gitfaq](#)), and the revert manual strongly recommends that the message explain why. Lab 8.2 runs the incident, including the teammate's side.

11.9 Reverting a merge, and the re-merge problem

In one sentence. `git revert -m 1 <merge>` undoes the content that a merge brought into the branch, and leaves the merge itself in the commit graph, which changes what every later merge of the same branch does.

Precisely. A merge commit has two parents, so "the change it made" has two meanings. `-m <parent-number>` (`--mainline`) chooses: the inverse is computed relative to that parent. On a branch that receives feature branches, the first parent is the branch's own previous tip (Chapter 8, Merge), so `-m 1` takes out what the merge brought in.

See it.

```
$ git merge feature/reranker
Merge made by the 'ort' strategy.
 rank.py | 2 ++
 rerank.py | 2 ++
 2 files changed, 4 insertions(+)
 create mode 100644 rerank.py
$ git log --oneline --graph
* 27bb9f5 Merge branch 'feature/reranker'
|\
| * c3762df Call the reranker from the ranker
| * 719cc13 Add cross-encoder reranker
* | 31a221c Record baseline metrics
|/
* fbb8230 Add BM25 ranker and serving config
$ ls
metrics.txt
rank.py
rerank.py
serve.yaml
```

```
$ git revert --no-edit HEAD
error: commit 27bb9f5a47a08961498a25b25288c07070ab2576 is a merge but no -m option was given.
```

```
fatal: revert failed
[exit status: 128]
$ git show -s --format="%h has parents: %p" HEAD
27bb9f5 has parents: 31a221c c3762df
$ git revert --no-edit -m 1 HEAD
[main 7e5a38e] Revert "Merge branch 'feature/reranker'"
Date: Mon Sep 7 10:14:00 2026 +0530
2 files changed, 4 deletions(-)
delete mode 100644 rerank.py
$ ls
metrics.txt
rank.py
serve.yaml
```

`rerank.py` is gone from `main`, and so is the import line in `rank.py`. The generated message records which parent was treated as the mainline:

```
$ git show -s --format=%B HEAD
Revert "Merge branch 'feature/reranker'"

This reverts commit 27bb9f5a47a08961498a25b25288c07070ab2576, reversing
changes made to 31a221c51824ee599e5d3059dd928398b65a63e6.

$ git log --oneline --graph -3
* 7e5a38e Revert "Merge branch 'feature/reranker'"
* 27bb9f5 Merge branch 'feature/reranker'
|\
| * c3762df Call the reranker from the ranker
```

The revert commit `7e5a38e` is an ordinary commit on top of the merge. It changed content. It did not change the graph, and no commit can: `27bb9f5` is still an ancestor of `main`, and through its second parent so are both commits of the feature branch. Git's how-to on this situation says the same: a revert undoes the data a merge brought, and none of its effect on history ([revert-a-faulty-merge](#)).

The problem. Try to merge the branch again:

```
$ git merge feature/reranker
Already up to date.
$ git merge-base main feature/reranker
c3762df49c6a0dd31105d001abb9e995132ca99e
$ git rev-parse feature/reranker
c3762df49c6a0dd31105d001abb9e995132ca99e
```

The merge base of `main` and the branch is the branch tip: for Git the branch is already merged. The dangerous variant looks like a success. The team fixes the bug with one more commit on the branch and merges:

```
$ git log --oneline main..feature/reranker
6b4e788 Cap reranker candidates at 50
$ git merge feature/reranker
Merge made by the 'ort' strategy.
serve.yaml | 1 +
1 file changed, 1 insertion(+)
$ ls
metrics.txt
rank.py
serve.yaml
$ cat serve.yaml
timeout_s: 30
rerank_top_k: 50
$ cat rank.py
def score(q, d):
    return bm25(q, d)
```

No conflict, no warning. `main` received the fix for a feature it does not contain: there is no `rerank.py` and no import.

```
Observed behavior : After a merge was reverted, merging the fixed branch brings only the new commit.
Git state          : merge-base(main, feature/reranker) is c3762df, the old tip of the branch.
Mechanism          : A merge combines what each side changed since the merge base. The first merge
                    : made the old branch commits ancestors of main, so only the commit after c3762df
                    : counts as the branch's change. On main's side the revert is a change since the
                    : base (it deletes the feature), nothing opposes it, and it stays.
Root cause         : The revert removed the content of the merge and left the merge in the graph.
Why Git does this  : Merge reads the graph, not intentions. The manual says that reverting a merge
                    : "declares that you will never want the tree changes brought in by the merge".
Correct fix        : Revert the revert and then merge, or recreate the branch as new commits.
Prevention         : Write the re-merge procedure into the message of the revert commit. Where you
                    : can, revert the one faulty commit and keep the merge.
```

Way out 1: revert the revert, then merge.

```
$ git reset --hard ORIG_HEAD
HEAD is now at 7e5a38e Revert "Merge branch 'feature/reranker'"
$ git revert --no-edit 7e5a38e
[main 4b3560c] Reapply "Merge branch 'feature/reranker'"
Date: Mon Sep 7 10:30:00 2026 +0530
2 files changed, 4 insertions(+)
```

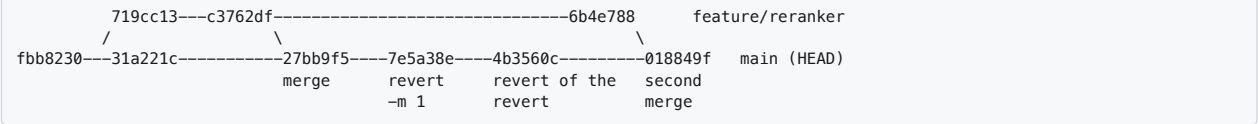
```
create mode 100644 rerank.py
$ git merge feature/reranker
Merge made by the 'ort' strategy.
  serve.yaml | 1 +
  1 file changed, 1 insertion(+)
$ ls
metrics.txt
rank.py
rerank.py
serve.yaml
```

The first command takes the half-working merge away again. `4b3560c` reverts the revert and brings the content of the first merge back; Git 2.55 names such a commit `Reapply "..."`. The merge then adds the fix:

```
$ git log --oneline --graph
* 018849f Merge branch 'feature/reranker'
|\
| * 6b4e788 Cap reranker candidates at 50
* | 4b3560c Reapply "Merge branch 'feature/reranker'"
* | 7e5a38e Revert "Merge branch 'feature/reranker'"
* | 27bb9f5 Merge branch 'feature/reranker'
|\
| * c3762df Call the reranker from the ranker
| * 719cc13 Add cross-encoder reranker
* | 31a221c Record baseline metrics
|/
* fbb8230 Add BM25 ranker and serving config
$ cat rank.py
from rerank import rerank

def score(q, d):
    return bm25(q, d)
```

Picture. The same history:



Way out 2: recreate the branch. New commits are not ancestors of `main`, so an ordinary merge brings all of them. A plain `git rebase main` does not create them. It replays only what `main` lacks:

```
$ git log --online --graph --all
* 52be493 Cap reranker candidates at 50
| * 9a2eb67 Revert "Merge branch 'feature/reranker'"
| * 27bb9f5 Merge branch 'feature/reranker'
| | \
| | /
| /|
* | c3762df Call the reranker from the ranker
* | 719cc13 Add cross-encoder reranker
| * 31a221c Record baseline metrics
| /
* fbb8230 Add BM25 ranker and serving config
$ git rebase main feature/reranker
Rebasing (1/1)
Successfully rebased and updated refs/heads/feature/reranker.
$ git log --online main..feature/reranker
2b155de Cap reranker candidates at 50
$ ls
metrics.txt
rank.py
serve.yaml
```

One commit was replayed, on top of the revert, and the feature is still missing. What works is `git rebase --no-ff` from the commit where the branch started, an option the manual documents for exactly this case ([git-rebase](#)):

```
# Where the branch started: the merge base of the two parents of the reverted merge.
$ git merge-base 27bb9f5^1 27bb9f5^2
fbb82301684fb07a59ac805d60f313294b62d3d5
$ git rebase --no-ff fbb8230 feature/reranker
Current branch feature/reranker is up to date, rebase forced.
Rebasing (1/3)
Rebasing (2/3)
Rebasing (3/3)
Successfully rebased and updated refs/heads/feature/reranker.
$ git log --oneline --graph --all
* 910d8d3 Cap reranker candidates at 50
* e9aea46 Call the reranker from the ranker
* b981ce8 Add cross-encoder reranker
| * 9a2eb67 Revert "Merge branch 'feature/reranker'"
| * 27bb9f5 Merge branch 'feature/reranker'
| |\
```



```
| | * c3762df Call the reranker from the ranker
| | * 719cc13 Add cross-encoder reranker
| | /
| | /
| * 31a221c Record baseline metrics
| /
* fbb8230 Add BM25 ranker and serving config
```

The three commits have new IDs and hang off `fbb8230` beside the old ones. `git merge feature/reranker` now brings `rerank.py`, the import and the fix (`labs/run ch11/revert-merge-rebuild`). The price is a rewritten feature branch (Chapter 9, Rebase).

Squash merges do not have the problem. If the branch went in as one ordinary commit (`git merge --squash`), there is no second parent, the branch commits never became ancestors of `main`, and the merge base stays where the branch started. After a plain revert of that commit, a later merge brings the whole branch:

```
$ git log --oneline -1 $(git merge-base main feature/reranker)
fbb8230 Add BM25 ranker and serving config
$ git merge feature/reranker
Merge made by the 'ort' strategy.
 rank.py      | 2 ++
 rerank.py    | 2 ++
 serve.yaml   | 1 +
 3 files changed, 5 insertions(+)
 create mode 100644 rerank.py
$ ls
metrics.txt
rank.py
rerank.py
serve.yaml
$ git log --oneline --graph
* 18eb394 Merge branch 'feature/reranker'
| \
| * Seedafc Cap reranker candidates at 50
| * c3762df Call the reranker from the ranker
| * 719cc13 Add cross-encoder reranker
* | f3affb6 Revert "Add cross-encoder reranker (squashed)"
* | e195948 Add cross-encoder reranker (squashed)
* | 31a221c Record baseline metrics
| /
* fbb8230 Add BM25 ranker and serving config
```

GitHub, not Git. The **Revert** button on a merged pull request "creates a new pull request that reverts the original merge commit" ([Reverting a pull request](#)), and `gh pr revert <number>` does the same from the command line ([manual](#)). This is described from the documentation, not run here. The page says nothing about a later re-merge. By the mechanics above, a pull request that was merged with a merge commit and then reverted needs one of the two ways out before its branch is merged again.

In production. Reverting a merge has a second cost: for `git bisect` the revert is one large commit that undoes many small ones, and so is the later revert of the revert. The how-to therefore advises finding and reverting the single faulty commit, or fixing forward, and reverting a whole merge only when the merge as such was the mistake. The state table of section 11.8 applies to `git revert -m 1 unchanged`.

11.10 `git clean`: the undo that has no undo

`git restore` and `git reset --hard` leave untracked files alone (section 11.5). The command that removes them is  `git clean`, the only undo whose effect no `reflog`, no `git fsck` and no `stash` can reverse, because an untracked file was never an object. Chapter 4 (The Working Tree), section 4.14, explains every option. What matters for undo is shown here in a repository with untracked files, ignored files, a nested repository and one tracked edit:

```
$ git status -s --ignored
M serve.yaml
?? debug_dump.json
?? scratch/
?? vendor/
!! .env
!! __pycache__/
!! train.log
```

```
$ git clean -n
Would remove debug_dump.json
$ git clean -n -d
Would remove debug_dump.json
Would remove scratch/
Would skip repository vendor/tokenizers
$ git clean -n -d -X
Would remove .env
Would remove __pycache__/
Would remove train.log
$ git clean -n -d -x
Would remove .env
Would remove __pycache__/
```

```
Would remove debug_dump.json
Would remove scratch/
Would remove train.log
Would skip repository vendor/tokenizers
$ git clean -n -d -e scratch/
Would remove debug_dump.json
Would skip repository vendor/tokenizers
```

Read the five dry runs as a blast radius that widens and narrows. Plain `-n` lists untracked files in the current directory. `-d` adds untracked directories. `-X` lists only ignored paths, which include `.env`. `-x` lists both kinds. `-e` protects a pattern. The nested repository `vendor/tokenizers` is never on the removal list, and the tracked edit to `serve.yaml` is not clean's business at all.

```
$ git clean -f -d
Removing debug_dump.json
Removing scratch/
Skipping repository vendor/tokenizers
$ git status -s --ignored
M serve.yaml
?? vendor/
!! .env
!! __pycache__/
!! train.log
```

```
$ printf '{"query": "q1", "scores": [0.9, 0.4]}\n' | git hash-object --stdin
d702a51f40a7856802f55fc97f9a9c06a50bcad4
$ git cat-file -t d702a51f40a7856802f55fc97f9a9c06a50bcad4
fatal: git cat-file: could not get object info
[exit status: 128]
$ git fsck
```

The ID that the deleted file's content would have names no object, and `git fsck` has nothing to report.

- **What it changes:** the working tree only. It deletes untracked paths, and ignored ones too with `-x` or `-X`.
- **What it can destroy:** anything Git does not track: notebooks, results, `.env` files, datasets, checkpoints.
- **How to preview:** the same command with `-n` in place of `-f`. Git insists on one of them: without `-f`, `-n` or `-i` it refuses to run (`clean.requireForce`).
- **How to recover:** not through Git.
- **When it is appropriate:** after you have read the dry run line by line. When in doubt, `git stash push -u` clears the same untracked files and keeps them as objects (section 11.11).

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git clean -f [-d] [-x or -X]</code>	the listed untracked or ignored paths deleted	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged

11.11 Stash: uncommitted work parked as commits

In one sentence. 🟡 `git stash push` records the index and the working tree as commits that only `refs/stash` reaches, then resets both to HEAD, so that the work can be brought back later, on this commit or on another.

Analogy. A labelled tray. You slide the half-built work off the bench onto a tray, and the bench is clear for the urgent job. The analogy breaks in two places. The tray remembers which commit the work was based on, and putting it back onto a changed bench is a merge, which can conflict. And trays never leave your workshop: a stash is not pushed, not fetched, and not shown by `git log`.

Precisely. A stash entry is a commit whose tree is the state of your tracked files. Its first parent is the commit that was HEAD and its second parent records the index ([git-stash](#)). With `-u`, a third parent holds the untracked files (Lab 8.6 reads one). `refs/stash` points at the newest entry, and older entries live in the reflog of that ref, hence the names `stash@{1}`, `stash@{2}`. `git stash apply` merges the entry's changes into the current files. `git stash pop` is `apply` followed by `drop`, and the drop happens only if the apply succeeded. Chapter 14C (Stash internals, rerere, attributes, hooks) opens the objects.

Inside .git. New commit objects, the ref `refs/stash` and its reflog `logs/refs/stash`. HEAD and the branch do not move. `ORIG_HEAD` is overwritten (section 11.4).

Picture.

```

      .----W
      /    /
-----H-----I
      refs/stash -> W      W: the tracked files as they were on disk
                          I: the index as it was
      main (HEAD -> main) stays at H; files and index are reset to H
```

See it. The starting state has one staged change, one unstaged change and one untracked file:

```
$ git status -s
M rank.py
M serve.yaml
```

```

?? plan.md
$ git stash push -m "wip: blend dense scores"
Saved working directory and index state On main: wip: blend dense scores
$ git status -s
?? plan.md
$ git stash list
stash@{0}: On main: wip: blend dense scores

```

The tracked changes are gone from the files. The untracked `plan.md` stayed: a plain stash takes tracked files only. `git stash show` summarizes an entry, and `-p` prints its full difference from the commit it was made on:

```

$ git stash show
rank.py      | 2 +-
serve.yaml  | 2 +-
2 files changed, 2 insertions(+), 2 deletions(-)
$ git stash show -p
diff --git a/rank.py b/rank.py
index 4b0bb9d..b29d4bd 100644
--- a/rank.py
+++ b/rank.py
@@ -1,2 +1,2 @@
def score(q, d):
-    return bm25(q, d)
+    return 0.7 * bm25(q, d) + 0.3 * dense(q, d)
diff --git a/serve.yaml b/serve.yaml
index 946ab07..40933f2 100644
--- a/serve.yaml
+++ b/serve.yaml
@@ -1,2 +1,2 @@
-timeout_s: 30
+timeout_s: 10
retries: 2

```

apply **versus** `pop`, and `--index`.

```

$ git stash apply -q
$ git status -s
M rank.py
M serve.yaml
?? plan.md
$ git stash list
stash@{0}: On main: wip: blend dense scores

```

After `apply` the entry is still in the list. `rank.py` was staged when it was stashed and came back unstaged. Only `--index` restores the split:

```

$ git stash pop --index
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   rank.py

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   serve.yaml

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    plan.md

Dropped refs/stash@{0} (3041fc614ff2d5678e22fcce3440c08ea4709d63)
$ git status -s
M rank.py
M serve.yaml
?? plan.md
$ git stash list

```

`pop` dropped the entry and printed the ID of the commit it let go of. That line matters later.

What goes into an entry.

Command	Goes into the entry	Left in your files and index
<code>git stash push</code>	staged and unstaged changes to tracked files	untracked files
<code>git stash push -u</code>	the same, plus untracked files (not ignored ones)	nothing: <code>git status</code> is clean
<code>git stash push --staged</code>	only what is staged	unstaged changes, untracked files
<code>git stash push --keep-index</code>	staged and unstaged changes, as without the option	the staged changes, still staged
<code>git stash push -- <path></code>	changes to <code><path></code> only	everything else

The two that are confused most often:

```
$ git status -s
M rank.py
M serve.yaml
?? plan.md
$ git stash push --staged -m "ranker change only"
Saved working directory and index state On main: ranker change only
$ git status -s
M serve.yaml
?? plan.md
$ git stash show
rank.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```

```
$ git status -s
M rank.py
M serve.yaml
?? plan.md
$ git stash push --keep-index -m "everything; index left in place"
Saved working directory and index state On main: everything; index left in place
$ git status -s
M rank.py
?? plan.md
$ git stash show
rank.py | 2 +-
serve.yaml | 2 +-
2 files changed, 2 insertions(+), 2 deletions(-)
```

`--staged` stashes the index and nothing else. `--keep-index` stashes everything and also leaves the staged part in place, so you can test exactly what you are about to commit. `labs/run ch11/stash-basics` shows `-u` and the `pathspe` form.

A pop that conflicts. Work is stashed, a hotfix changes the same line, and the stash comes back:

```
$ git status -s
M serve.yaml
$ git stash push -m "wip: try a 10s timeout"
Saved working directory and index state On main: wip: try a 10s timeout
$ printf 'timeout_s: 20\nretries: 2\ncache: on\n' > serve.yaml
$ git commit -am "Hotfix: lower timeout to 20s"
[main 00eed5e] Hotfix: lower timeout to 20s
1 file changed, 1 insertion(+), 1 deletion(-)
```

```
$ git stash pop
Auto-merging serve.yaml
CONFLICT (content): Merge conflict in serve.yaml
On branch main
Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
    both modified:   serve.yaml

no changes added to commit (use "git add" and/or "git commit -a")
The stash entry is kept in case you need it again.
[exit status: 1]
```

```
$ git status -s
UU serve.yaml
$ cat serve.yaml
<<<<<< Updated upstream
timeout_s: 20
=====
timeout_s: 10
>>>>>> Stashed changes
retries: 2
cache: on
$ git stash list
stash@{0}: On main: wip: try a 10s timeout
```

It is an ordinary conflict (Chapter 8, Merge) with its own labels: `Updated upstream` is the file as it is now, `Stashed changes` is the entry. The last line of the pop is the important one: the entry was kept. There are two ways on. Resolve the file, clear the conflict state with `git restore --staged` or `git add`, and drop the entry yourself:

```
$ printf 'timeout_s: 10\nretries: 2\ncache: on\n' > serve.yaml
$ git restore --staged serve.yaml
$ git status -s
M serve.yaml
$ git stash list
stash@{0}: On main: wip: try a 10s timeout
$ git stash drop
Dropped refs/stash@{0} (c94265dd6415b63fc30a245db496836c128b5b90)
$ git stash list
```

Or back out with `git reset --merge` and use `git stash branch`. It creates a branch at the commit the entry was made on, applies the entry there, where it cannot conflict, and drops it:

```

$ git reset --merge
$ git status -s
$ git stash branch try-10s-timeout
Switched to a new branch 'try-10s-timeout'
On branch try-10s-timeout
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   serve.yaml

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (c94265dd6415b63fc30a245db496836c128b5b90)
$ git log --oneline --decorate --all
00eed5e (main) Hotfix: lower timeout to 20s
6cde22d (HEAD -> try-10s-timeout) Add serving config
$ git stash list

```

A dropped entry. 🚫 `git stash drop` and `git stash clear` delete the only ref to work that was never committed. The objects stay for a while, and with the ID that `drop` printed the entry can be put back:

```

$ git stash show c94265d
serve.yaml | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git stash store -m 'recovered: try a 10s timeout' c94265d
$ git stash list
stash@{0}: recovered: try a 10s timeout

```

Without the ID, the stash manual's `git fsck --unreachable` recipe lists candidates (Lab 8.6). Preview any drop with `git stash show -p stash@{n}`, and drop an entry only when its content is committed elsewhere or you want none of it.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git stash push</code>	tracked files reset to HEAD; with <code>-u</code> untracked files removed	reset to HEAD (kept with <code>--keep-index</code>)	unchanged	unchanged	stash commits created; <code>refs/stash</code> and its reflog updated; <code>ORIG_HEAD</code> overwritten	unchanged	unchanged
<code>git stash apply</code> , <code>git stash pop</code>	entry's changes merged in; conflict markers possible	unchanged, or restored with <code>--index</code> ; unmerged entries on conflict	unchanged	unchanged	<code>pop</code> removes the entry after a clean apply	unchanged	unchanged
<code>git stash drop</code> , <code>git stash clear</code>	unchanged	unchanged	unchanged	unchanged	entries removed from <code>refs/stash</code> and its reflog	unchanged	unchanged

In production. An incident interrupts a refactoring: `git stash push -u -m "wip: per-tenant limits"`, fix, commit, `git stash pop`. Always give `-m`: without it entries are listed as WIP on `<branch>` plus the subject of the base commit, and five of those look alike. For work that must outlive the day, a commit on a branch is the better container: it shows in `git log`, can be pushed, and no single command drops it.

11.12 One table and one decision tree

"HEAD" is the commit that HEAD resolves to, "Branch" is the ref of the current branch, and "History" is the set of commits reachable from that branch.

Command	HEAD	Branch	Index	Working tree	History
<code>git reset --soft <commit></code>	moves to <code><commit></code>	set to <code><commit></code>	unchanged	unchanged	commits after <code><commit></code> leave the branch; the reflog keeps them
<code>git reset --mixed <commit></code> (the default)	moves to <code><commit></code>	set to <code><commit></code>	matches <code><commit></code>	unchanged	as above
<code>git reset --hard <commit></code>	moves to <code><commit></code>	set to <code><commit></code>	matches <code><commit></code>	matches <code><commit></code> ; uncommitted changes destroyed	as above
<code>git reset --keep <commit></code>	moves to <code><commit></code>	set to <code><commit></code>	matches <code><commit></code>	local changes carried across; refuses if they overlap	as above
<code>git reset --merge <commit></code>	moves to <code><commit></code>	set to <code><commit></code>	matches <code><commit></code>	unstaged changes carried across, staged ones discarded	as above
<code>git reset -- <path></code>	unchanged	unchanged	<code><path></code> from HEAD	unchanged	unchanged
<code>git restore <path></code>	unchanged	unchanged	unchanged	<code><path></code> from the index	unchanged
<code>git restore --staged <path></code>	unchanged	unchanged	<code><path></code> from HEAD	unchanged	unchanged
<code>git restore --source= <commit> <path></code>	unchanged	unchanged	unchanged	<code><path></code> from <code><commit></code>	unchanged

Command	HEAD	Branch	Index	Working tree	History
<code>git restore --staged --worktree <path></code>	unchanged	unchanged	<path> from HEAD	<path> from HEAD	unchanged
<code>git commit --amend</code>	moves to a new commit	set to the new commit	unchanged	unchanged	the last commit is replaced by a new one with the same parent
<code>git revert <commit></code>	moves to a new commit	advanced by one commit	matches the new commit	inverse change applied	one commit added, none removed

Read the Branch column first. "Set to" means the branch can lose commits: private history only. "Advanced" and "unchanged" are safe on a branch that other people have.

The decision tree.

```

What do you want to undo?
|
+-- Changes that are not committed
|   |
|   +-- staged, and the edit should stay ..... git restore --staged <path>
|   +-- an unstaged edit, to be discarded ..... git diff <path>, then git restore <path>      (no way back)
|   +-- some hunks of a file ..... git restore -p <path>                                (no way back)
|   +-- everything, possibly wanted later ..... git stash push -u -m "<why>"
|   +-- untracked files ..... git clean -n [-d], then the same with -f      (no way back)
|
+-- Commits
|   |
|   +-- Can anyone else already have them?  git fetch; git branch -r --contains <commit>
|       |
|       +-- NO: private history, rewriting is allowed
|           |
|           +-- last commit: content or message ..... git commit --amend
|           +-- last commits: keep the changes ..... git reset --soft <commit>      (staged)
|           |                                           git reset <commit>      (unstaged)
|           +-- last commits: drop the changes ..... git reset --keep <commit>     (--hard only on a clean tree)
|           +-- a merge made a moment ago ..... git reset --merge ORIG_HEAD
|           +-- a commit further back ..... git rebase -i                          (Chapter 9, Rebase)
|       |
|       +-- YES: shared history, add commits
|           |
|           +-- one commit ..... git revert <commit>
|           +-- several commits ..... git revert -n <A>..<B>, then git commit
|           +-- a merge ..... git revert -m 1 <merge>, and plan the re-merge (11.9)
|           +-- one file back to an old version ..... git restore --source=<commit> <path>, then git commit

```

11.13 Worked scenarios

Seven situations, handled with the tree of section 11.12 in one clone whose `origin` is a bare repository on disk, so "pushed" is a real state. The demo ran scenario 3 before scenario 2, which is why scenario 2's log shows the commits of scenario 3.

1. Undo the last commit, which is not pushed. `ahead 1`, and no remote-tracking branch contains the commit: private history. `--soft` takes the commit off the branch and leaves its change staged.

```

$ git status -sb
## main...origin/main [ahead 1]
$ git branch -r --contains HEAD
$ git reset --soft HEAD~1
$ git status -sb
## main...origin/main
M rank.py
$ git log --oneline -1
fc2e2d8 Record baseline metrics

```

Use `git reset HEAD~1` to get the change back unstaged, and `git reset --keep HEAD~1` to drop it.

2. A bad commit is on main and pushed. `origin/main` contains `c6f2cfd`: shared history. Revert and push.

```

$ git log --oneline
377247a Ignore local .env files
90e7514 Retry BM25 scoring with backoff
fc2e2d8 Record baseline metrics
c6f2cfd Disable retries
91788a6 Add BM25 ranker and serving config
$ git branch -r --contains c6f2cfd
origin/main
$ git revert --no-edit c6f2cfd
[main 20cf7b0] Revert "Disable retries"
Date: Mon Sep 7 10:27:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
$ git status -sb
## main...origin/main [ahead 1]
$ git push
To $LAB/ch11/scenarios/server.git
377247a..20cf7b0 main -> main

```

3. Remove a file from the last commit, which is not pushed. A `.env` file went in with `git add .` `git rm --cached` removes the index entry and keeps the file, and `--amend` replaces the commit.

```
$ git show --stat --format="%h %s" HEAD
ccec13 Retry BM25 scoring with backoff

.env      | 1 +
rank.py   | 2 +-
2 files changed, 2 insertions(+), 1 deletion(-)
$ git rm --cached .env
rm '.env'
$ git commit --amend --no-edit
[main 90e7514] Retry BM25 scoring with backoff
Date: Mon Sep 7 10:17:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
$ git show --stat --format="%h %s" HEAD
90e7514 Retry BM25 scoring with backoff

rank.py   | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git status -s
?? .env
```

The file is now untracked, so add it to `.gitignore` next (Chapter 4). The old commit `ccec13` still contains the file and stays in your reflog, which is harmless for a commit that never left your machine. If a commit with a real secret was ever pushed, amending is not the fix: rotate the secret (Chapter 21, Security).

4. Unstage. The index entry goes back to HEAD's version and the file keeps the edit.

```
$ git status -s
M rank.py
M serve.yaml
?? probe_output.txt
$ git restore --staged rank.py
$ git status -s
M rank.py
M serve.yaml
?? probe_output.txt
```

5. Discard local changes. Tracked files: read `git diff`, then `git restore`. Untracked files: `git clean -n`, then `-f`. Neither step can be taken back. When in doubt, `git stash push -u` does the same job and keeps the content.

```
$ git restore rank.py serve.yaml
$ git status -s
?? probe_output.txt
$ git clean -n
Would remove probe_output.txt
$ git clean -f
Removing probe_output.txt
$ git status -s
```

6. Undo a merge that is not pushed, with an unrelated edit in progress. `git reset --merge ORIG_HEAD` puts the branch back on the commit before the merge and keeps the edit to `metrics.txt`. If anything has overwritten `ORIG_HEAD` since the merge, take the commit from the reflog.

```
$ git merge feature/reranker
Merge made by the 'ort' strategy.
 rerank.py | 2 ++
1 file changed, 2 insertions(+)
 create mode 100644 rerank.py
$ git status -sb
## main...origin/main [ahead 2]
M metrics.txt
$ git reset --merge ORIG_HEAD
$ git status -sb
## main...origin/main
M metrics.txt
$ git log --oneline -1
20cf7b0 Revert "Disable retries"
```

7. Undo a merge that is pushed. `origin/main` contains the merge: shared history. `git revert -m 1`, push, and write the re-merge plan of section 11.9 into the ticket.

```
$ git log --oneline --graph -4
* 5777a43 Merge branch 'feature/reranker'
|\
| * ecf26a5 Add cross-encoder reranker
* | 20cf7b0 Revert "Disable retries"
* | 377247a Ignore local .env files
$ git branch -r --contains HEAD
origin/main
$ git revert --no-edit -m 1 HEAD
```

```
[main 0dd3130] Revert "Merge branch 'feature/reranker'"
Date: Mon Sep 7 10:50:00 2026 +0530
1 file changed, 2 deletions(-)
delete mode 100644 rerank.py
$ git push
To $LAB/ch11/scenarios/server.git
5777a43..0dd3130 main -> main
$ ls
metrics.txt
rank.py
serve.yaml
```

11.14 What can go wrong

Symptom	Diagnosis	Fix	Prevention
Edits are gone after <code>git reset --hard</code> or <code>git restore</code>	<code>git reflog</code> lists commits only; <code>git fsck --lost-found</code> lists blobs for content that was once staged	<code>git cat-file -p <blob></code> for staged content; nothing for the rest	<code>git status</code> first; <code>git stash push -u; --keep</code> , not <code>--hard</code>
<code>git reset --hard</code> <code>ORIG_HEAD</code> lands on the wrong commit	<code>git log -1 ORIG_HEAD</code> : a later reset, merge, rebase or stash rewrote the slot	Reset to the right reflog entry	Use <code>ORIG_HEAD</code> only as the very next command
<code>git push</code> is rejected as non-fast-forward after a reset or an amend	<code>git status -sb</code> shows behind: the commits you removed were already pushed	Save new work, <code>git reset --hard @{u}</code> , then <code>git revert</code>	<code>git branch -r --contains <commit></code> before rewriting
A branch that was merged again lacks most of its changes	<code>git log --oneline main..<branch></code> lists only the newest commits, and <code>main</code> has a <code>Revert "Merge ..." commit</code>	Revert the revert and merge again, or recreate the branch (section 11.9)	Put the re-merge procedure into the revert message
<code>git revert</code> stops with a conflict	<code>git status</code> ; <code>git ls-files -s <path></code> shows three stages	Resolve, <code>git add</code> , <code>git revert --continue</code> ; or <code>--abort</code>	Revert early, before other commits build on the bad one
<code>git restore --source=<commit></code> . deleted files	<code>git status</code> shows <code>D</code> : tracked paths that the source lacks were removed	<code>git restore .</code> returns the index versions; unstaged edits stay lost (Lab 8.4)	Name the paths you mean
<code>git stash pop</code> reports a conflict and the entry is still listed	<code>git status</code> shows <code>UU</code>	Resolve and <code>git stash drop</code> , or <code>git reset --merge</code> and <code>git stash branch <name></code>	Pop soon; keep stashes small
A pop was refused or failed, yet files from the entry are there	The entry was made with <code>-U</code> : on Git 2.55 a refused pop still writes its untracked files, and the next pop fails on them (transcript in Lab 8.6)	Compare with <code>git diff 'stash@{0}'</code> and <code>git show 'stash@{0}^3:<path>'</code> , then <code>git stash drop</code>	Clear local edits before popping
A stash entry was dropped by mistake	<code>drop</code> and <code>pop</code> print the ID	<code>git stash store -m "<message>" <id></code> ; without the ID, the <code>git fsck --unreachable</code> recipe of the stash manual	<code>git stash show -p</code> before <code>drop</code>

11.15 When not to use it, and dangerous edge cases

Do not rewrite what others have (section 11.2).

Outdated advice. Several widely viewed tutorials teach "`git reset --hard <old commit>`", then `git push -f`" as the way to undo pushed commits, including on the main branch (example). The first step destroys your uncommitted work and the second destroys other people's commits on the server. On shared history the undo is `git revert`.

No command in this chapter removes a secret. A revert leaves it in history in plain sight. A reset or an amend leaves it in your reflog and, once pushed, on the server and in every clone and fork. The remedy is to rotate the secret (Chapter 21, Security).

A revert does not win a merge. If the same change exists on two branches and you revert it on one, merging the two keeps the change: one side changed the lines since the merge base, the other shows no net change, and Git takes the change. The Git FAQ has an entry on this (gitfaq), and Chapter 8 (Merge) demonstrates it (labs/run ch08/revert-one-side). After a revert on a release branch, check `main` separately.

Do not revert a revert blindly. It is right when the branch was fixed by adding commits. If the branch was rebuilt as new commits, a revert of the revert conflicts with them wherever they differ from the old ones, as the how-to warns: merge the rebuilt branch directly (section 11.9).

`git restore .` **at the top of a repository** is the working-tree half of `git reset --hard`, without `ORIG_HEAD` and without a reflog entry. With `--source` it also deletes tracked files that the source lacks.

A stash is not a backup. It lives in one repository, no push sends it, and one command drops it. Old entries also rot: the further the branch moves on, the likelier the pop conflicts.

11.16 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git restore --staged <path>, git reset --<path></code>	●	Index entries	<code>git diff --cached --<path></code>	<code>git add <path>; git fsck --lost-found</code> for a staged version that differed from the file
<code>git restore <path>, git restore -p <path></code>	●	Files, from the index	<code>git diff -- <path></code>	None for content that was never staged
<code>git restore --source=<commit> [--staged] [--worktree] <path></code>	●	Files (and index) from a commit; deletes tracked paths the commit lacks	<code>git diff <commit> --<path></code>	None for unstaged edits
<code>git reset --soft <commit></code>	●	Branch ref	<code>git log --oneline <commit>..HEAD</code>	The same mode with <code>ORIG_HEAD</code> , or the relog
<code>git reset [--mixed] <commit></code>	●	Branch ref and index	as above, and <code>git diff --cached</code>	as above
<code>git reset --keep <commit></code>	●	Branch ref, index, files that differ between the two commits	as for <code>--soft</code>	as above
<code>git reset --merge <commit></code>	●	As <code>--keep</code> ; discards staged changes	<code>git diff --cached</code>	Commits: relog. Staged content: <code>git fsck --lost-found</code>
<code>git reset --hard <commit></code>	●	Branch ref, index, every tracked file	<code>git status -s, git diff HEAD --stat, git log --oneline <commit>..HEAD</code>	Commits: relog. Staged content: <code>git fsck --lost-found</code> . Unstaged: none
<code>git commit --amend</code>	●	Replaces the last commit	<code>git diff --cached</code>	<code>git reset --soft HEAD@{1}</code>
<code>git revert <commit>, git revert -m 1 <merge></code>	●	Adds one commit: the branch moves, index and files are updated, nothing is removed. After <code>-m 1</code> , a later merge of the same branch brings back nothing until the revert is reverted	<code>git show <commit></code>	Revert the revert; before pushing, <code>git reset --keep HEAD~1</code>
<code>git revert -n <commit></code>	●	Index and files	<code>git show <commit></code>	<code>git revert --abort</code>
<code>git clean -n</code>	●	Nothing	it is the preview	not needed
<code>git clean -f [-d] [-x or -X]</code>	●	Deletes untracked (with <code>-x</code> or <code>-X</code> , ignored) files	the same options with <code>-n</code>	None through Git
<code>git stash push [-u]</code>	●	Saves index and files, then resets them to HEAD	<code>git status -s</code>	<code>git stash pop --index</code>
<code>git stash apply, git stash pop, git stash branch <name></code>	●	Merges an entry into the files	<code>git stash show -p</code>	The entry is kept after a conflict; <code>git reset --merge</code> backs out
<code>git stash drop, git stash clear</code>	●	Removes one entry, or all	<code>git stash show -p stash@{n}</code>	<code>git stash store <id></code> while the objects exist

11.17 Version notes

Version note. Older behavior: `git checkout -- <path>` and `git checkout <commit> -- <path>` restored files, and `git reset HEAD <path>` unstaged. Current behavior: `git restore` does these jobs, and the old forms still work. Since: introduced in Git 2.23 ([release notes](#)), no longer labelled experimental from Git 2.51 ([release notes](#)). Recommended: write `restore`, read `checkout`.

Version note. Older behavior: `git stash save "<message>"`. Current behavior: `git stash push -m "<message>"`, which also takes a `paths`pec; `save` is deprecated and still runs. Since: `push` in Git 2.13, deprecation in Git 2.16 ([git-stash](#)); `--staged` in Git 2.35 ([release notes](#)). Recommended: `git stash push -m`.

Version note. Older behavior: a different default subject for a revert of a revert (not run here). Current behavior: Reapply `"<subject>"`, as in section 11.9. Since: Git 2.43, whose release notes say this default message "has been tweaked" ([release notes](#)). Recommended: reword such subjects by hand, as the revert manual asks.

Unverified. Two behaviors in this chapter were observed on Git 2.55.0 and are not stated in the manuals: `git fsck --lost-found` does not count relog entries as references (section 11.5), and a refused `git stash pop` still writes the untracked files of the entry (section 11.14). Other versions were not tested. GitHub's page on reverting a pull request does not say how the Revert button treats squash-merged or rebase-merged pull requests.

11.18 Practice

- Labs 8.1 to 8.7 in the Module 8 lab manual: the reset prediction table, a revert of a pushed commit, a reverted merge and its re-merge, the restore variants, a safe clean, a stash with a pop conflict, and six scenario cards.
- Replay any transcript with `labs/run ch11/<demo>`. The sandbox stays in place, so you can continue by hand in it.
- Four drills, each in a sandbox left by a replay:
 1. In `ch11/reset-modes/ranker-hard`, find out which of the two lost versions, 0.80 and 0.90, still exists, and explain the difference.
 2. In `ch11/revert-basics/ranker-conflict`, predict whether `git revert d51208b` conflicts, then run it.
 3. In `ch11/revert-merge/ranker`, run `git switch -c scratch 27bb9f5` and `git revert -m 2 HEAD`. Explain which file disappeared and why.
 4. In `ch11/stash-basics/ranker`, bring the remaining entry back with `git stash branch`, and explain why that form cannot conflict.

11.19 Interview questions

1. A colleague asks for "the undo command". Name the five commands of this chapter and say which of working tree, index, branch ref and history each one writes.
2. HEAD, the index and the working tree hold three different versions of one file. What does each hold after `git reset --soft HEAD~1`, after `--mixed` and after `--hard`, and what does `git status -s` print?
3. An engineer ran `git reset --hard HEAD~3` in a dirty tree. What can you recover, with which commands and for how long, and what is gone? Why is that part gone?
4. What is `ORIG_HEAD`, which commands write it, and why is the reflog the more reliable tool?
5. When do you choose `git reset --keep over --hard`? What does `--merge` do to staged changes, and why was it designed that way?
6. Describe `git revert` as a three-way merge: what are base, ours and theirs? Use the answer to explain why reverting an old commit can conflict.
7. A bad commit is on `main` and the team has pulled it. Compare "revert and push" with "reset and force-push": what does each do to the server, to the teammates' clones and to CI?
8. You reverted a merge with `-m 1` last week. Today the fixed branch was merged again and the feature is incomplete. Explain the mechanism with the merge base and give two correct procedures. What would `-m 2` have undone, and why does a squash merge not show the problem?
9. Compare `git clean -fdx` with `git reset --hard`: which files does each destroy, and for which of them does Git still hold an object?
10. What is inside a stash entry and where is it stored? What happens when `git stash pop` conflicts? Give three reasons not to use the stash for long-term storage.

11.20 Sources

Primary sources

- `git-reset` (with the tables in its "Discussion" section), `git-revert`, `git-restore`, `git-clean`, `git-stash`, `git-commit`, `git-rebase`, `git-fsck`, `git-config`, `gitrevisions` and `git(1)`. The transcripts were checked against the local Git 2.55.0 copies (`git help -m <command>`).
- [How to revert a faulty merge](#), the how-to that the revert manual points to.
- [gitfaq](#), the entries on undoing a change that is already in the main branch and on a change reverted on one of two branches.
- Release notes [2.23.0](#), [2.35.0](#), [2.43.0](#), [2.51.0](#), and 1.7.1 as shipped with Git 2.55.0 (directory `RelNotes` under the path that `git --html-path` prints).
- GitHub Docs: [Reverting a pull request](#) and [Available rules for rulesets](#); GitHub CLI manual, [gh pr revert](#).

Secondary sources

- Julia Evans, [How git cherry-pick and revert use 3-way merge](#).
- Pro Git, [Reset Demystified](#). Caveat: it contrasts `reset` with `git checkout` and does not mention `git restore`.
- Pro Git, [Undoing Things](#) (old and new spellings side by side) and [Stashing and Cleaning](#). Caveat: the stash chapter mostly uses bare `git stash` and does not cover `--staged`.
- The Phase 0 report of this course, sections 4, 12 and 13, for the Stack Overflow figures and the retention defaults.

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [How to Undo Mistakes With Git Using the Command Line](#), Tobias Günther for freeCodeCamp, 55 minutes, 24 November 2020: `git restore` including `-p`, `amend`, `revert`, `reset`, `reflog` recovery from 24:02. Caveat: `master` naming.
- [Revert and Reset Commits Like a Pro](#), ProCodrr episode 6, Hindi, 56 minutes, March 2023: `reset` versus `revert`, with `reflog` recovery of a hard-reset commit. Caveat: `master`.

Further reading

- [Oh Shit, Git!?!](#), a short scenario list. Caveat: it uses the `checkout` and `reset` spellings.

Chapter 12: Remote Operations

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch12/`.

12.1 Why this matters

Three questions a CTO can ask in the same week:

1. "The deploy job says the hotfix is not on the server. The engineer says they pushed it, and their terminal printed `Everything up-to-date`. Which one is wrong?"
2. "Someone force-pushed with the 'safe' option, `--force-with-lease`, and a colleague's commit still disappeared from the shared branch. How, and where is that commit now?"
3. "`git status` said 'up to date' on the release laptop. We tagged, and the tag does not contain yesterday's fix. Why did Git say that?"

Neither is wrong: the push named a branch that had nothing new (section 12.14). The lease was satisfied by a fetch that nobody looked at, and the commit still sits in places you can name (section 12.8). And `git status` never contacts a server (section 12.4).

All three answers come from one model. There are at least three things called `main`:

Name	Full ref	Lives in	Moved by
your branch	<code>refs/heads/main</code>	your repository	you: commit, merge, rebase, reset, the second half of pull
your remote-tracking branch	<code>refs/remotes/origin/main</code>	your repository	Git, when it talks to the remote: fetch, pull, a successful push
the server's branch	<code>refs/heads/main</code>	another repository	whoever pushes there

Git contacts another repository only when you run a command that says so: `clone`, `fetch`, `pull`, `push`, `ls-remote`, and a few `git remote` subcommands. Every other command, including `status`, `log origin/main` and `branch -vv`, reads refs stored in your own `.git`. A remote operation is fully described by which of the three kinds of ref it reads and which it moves:

Operation	Server's refs	Your remote-tracking refs	Your branches	Index and working tree
<code>git fetch</code>	read	moved	untouched	untouched
<code>git pull</code>	read	moved	current branch moved	updated
<code>git push</code>	moved	moved, for the refs pushed	untouched	untouched
<code>git clone</code>	read	created	one created	checked out

The rest of the chapter makes each cell of that table precise. The demos use one project, `support-bot` (retrieval-augmented answers for a help desk), a bare repository on disk that plays the server, and three clones: yours, Asha's and Ravi's. A bare repository on disk and GitHub differ in authentication and in the rules a platform adds on top. The Git mechanics are the same, and where the platform changes the picture the text says so.

12.2 What a remote is

In one sentence. A remote is a name in your repository's configuration that stands for another repository: its URL, and the rules for copying refs between that repository and yours.

Analogy. An entry in an address book with a filing instruction attached: "the supplier we call `origin` is at this address; file our copy of whatever they call `X` under `origin/X`". The analogy breaks in two places. The entry is not a connection: nothing is live, and the filed copies are as old as your last visit. And the supplier is not special: it is a full Git repository like yours, and the name `origin` exists only in your address book.

Precisely. A remote is a configuration section `[remote "<name>"]` holding one or more URLs (`url`, optionally `pushurl`) and one or more fetch refsspecs (`fetch`). A refspect has the form `[+<source>:<destination>]` and maps ref names on one side to ref names on the other. The default written by `git clone` and `git remote add` is `+refs/heads/*:refs/remotes/<name>/*`: every branch of the remote becomes a ref of yours under `refs/remotes/<name>/`, and the `+` allows the update even when it is not a fast-forward (section 12.12). The refs it creates are **remote-tracking branches**. The glossary defines one as "a ref that is used to follow changes from another repository", which "should not contain direct modifications or have local commits made to it" ([gitglossary](#)). The data-model manual is blunter: it is how Git stores the last-known state of a branch in a remote repository ([gitdatamodel](#)). `origin` is only the name that `git clone` picks by default.

Inside `.git`. Four places. The `[remote]` and `[branch]` sections of `.git/config`; the refs under `refs/remotes/<name>/`; their reflogs under `logs/refs/remotes/<name>/` (paths as in a files-format repository, Chapter 3); and `FETCH_HEAD`, rewritten by every fetch. There is no object store per remote. Objects from every remote land in the one object database, which is why fetching from a second remote that shares history transfers very little.

See it. A fresh clone, and the configuration it wrote:

```
$ git clone server/support-bot.git you/support-bot
Cloning into 'you/support-bot'...
done.
$ cd you/support-bot
$ cat .git/config
[core]
    repositoryformatversion = 0
    filemode = true
    bare = false
    logallrefupdates = true
    ignorecase = true
    precomposeunicode = true
[remote "origin"]
    url = $LAB/ch12/remote-anatomy/server/support-bot.git
    fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
    remote = origin
    merge = refs/heads/main
```

[remote "origin"] is the whole remote: a URL (an absolute path here, because the clone source was a local path) and one refspec. [branch "main"] records the upstream of your branch `main`, the subject of section 12.5. `ignorecase` and `precomposeunicode` describe the macOS filesystem (Chapter 4). Now compare the refs on the two sides:

```
$ git remote -v
origin $LAB/ch12/remote-anatomy/server/support-bot.git (fetch)
origin $LAB/ch12/remote-anatomy/server/support-bot.git (push)
# Refs in your clone:
$ git show-ref --abbrev
510ee94 refs/heads/main
510ee94 refs/remotes/origin/HEAD
336d5ee refs/remotes/origin/feature/reranker
510ee94 refs/remotes/origin/main
2828fba refs/tags/v0.1.0
# Refs in the server repository:
$ git -C ../server/support-bot.git show-ref --abbrev
336d5ee refs/heads/feature/reranker
510ee94 refs/heads/main
2828fba refs/tags/v0.1.0
```

The server has two branches and a tag. Your clone has the tag under the same name, both branches under `refs/remotes/origin/`, and exactly one local branch. The server's `feature/reranker` is not a branch of yours. It is `origin/feature/reranker`, a bookmark of where that branch was when you cloned. `refs/remotes/origin/HEAD` is a symbolic ref that records which branch the server treats as its default (section 12.3).

```
$ git branch -a -vv
* main                    510ee94 [origin/main] Add retrieval config
  remotes/origin/HEAD      -> origin/main
  remotes/origin/feature/reranker 336d5ee Add reranker stub
  remotes/origin/main      510ee94 Add retrieval config
$ git log --oneline --graph --decorate --all
* 336d5ee (origin/feature/reranker) Add reranker stub
* 510ee94 (HEAD -> main, tag: v0.1.0, origin/main, origin/HEAD) Add retrieval config
* 95671d3 Add retriever skeleton
* f56c1bb Add README
```

`git branch -a -vv` is the quickest complete view: local branches with their upstream in brackets, then the remote-tracking branches.

Picture.

server/support-bot.git (bare)		you/support-bot
HEAD -> refs/heads/main		refs/remotes/origin/HEAD -> origin/main
refs/heads/main 510ee94	---->	refs/remotes/origin/main 510ee94
refs/heads/feature/reranker 336d5ee	---->	refs/remotes/origin/feature/reranker 336d5ee
refs/tags/v0.1.0 2828fba	---->	refs/tags/v0.1.0 2828fba
		refs/heads/main 510ee94 (HEAD -> main)

fetch copies names left to right
push copies names right to left, only the refs you name, only if the rules allow it

A bare repository. The server in every demo is a bare repository: a repository with no working tree, whose directory is itself the Git directory.

```
$ ls -F ../server/support-bot.git
config
description
HEAD
hooks/
info/
objects/
refs/
$ git -C ../server/support-bot.git config get core.bare
true
$ git -C ../server/support-bot.git status
```

```
fatal: this operation must be run in a work tree
[exit status: 128]
```

There is no `index` and no working tree, so nothing can be edited there and commands that need a working tree refuse. That is the point: a repository that only receives pushes must not hold files that a push could leave stale (section 12.9). Notice also what the listing lacks: `logs/.core.logAllRefUpdates` defaults to true in a repository with a working tree and to false in a bare one, so a plain bare server keeps no reflog (core configuration). When a branch on such a server is overwritten, the server has no record of the old value. Section 12.8 depends on this.

Version note. Older behavior: Git works inside any bare repository it finds. Current behavior: the same (`safe.bareRepository=all`), but the manual of Git 2.55 states that `explicit` will be the default in Git 3.0. Under `explicit`, `git -C <bare-repository>`, which this chapter's demos use to look inside the server, fails; `git --git-dir=<path>` works, and fetching from or pushing to a bare repository is not affected. Since the setting is documented in the `git config` manual of 2.55. Recommended: in scripts, address bare repositories with `--git-dir`.

The URL in the demos. One line of housekeeping that every later transcript relies on:

```
$ git remote set-url origin ../../server/support-bot.git
$ git remote -v
origin ../../server/support-bot.git (fetch)
origin ../../server/support-bot.git (push)
$ git config get --all --show-names --regex "(remote|branch)\."
remote.origin.url ../../server/support-bot.git
remote.origin.fetch +refs/heads/*:refs/remotes/origin/*
branch.main.remote origin
branch.main.merge refs/heads/main
```

● `git remote set-url` rewrites the URL. `git remote -v` prints one line for fetching and one for pushing, because the two can differ (section 12.10). The demos store a relative path so that the commit IDs printed in this book do not depend on where your lab root is: a `git pull` that merges writes the URL into the merge message, and the message is part of what a commit ID is computed from. Real remotes are HTTPS or SSH URLs (section 12.13). `git config get --all --show-names --regex` needs Git 2.46 or later.

In production. When a repository moves (a renamed organization, a migration from a self-hosted server to GitHub), nothing in your objects or branches changes. The fix is one `git remote set-url origin <new-url>` per clone and per CI configuration. The opposite problem causes real incidents: two people say "origin" and mean different repositories, for example a fork and the shared repository. In an incident, read `git remote -v` before you read anything else.

12.3 `git clone`, step by step

In one sentence. ● `git clone` creates a new repository, registers the source as the remote `origin`, fetches everything the default refspec covers, and checks out one branch.

Analogy. Moving into a furnished copy of an office: you receive every document, a board that shows where the head office's bookmarks stood this morning, and one desk set up for work. It breaks because nothing ties the copy to the original afterwards except the address on file.

Precisely. Six steps, each of which you can run yourself:

1. Create the directory and an empty repository (`git init`).
2. Write the remote: its URL and the default fetch refspec (`git remote add origin <url>`).
3. Fetch: copy the objects, create a remote-tracking branch for every branch of the remote, copy the tags (`git fetch origin`).
4. Record the remote's default branch as the symbolic ref `refs/remotes/origin/HEAD`.
5. Create one local branch, named after the branch that the remote's `HEAD` points at, starting at the same commit, with that branch as its upstream.
6. Check it out: fill the index and the working tree.

See it. The same result built by hand. Steps 1 and 2:

```
$ git init by-hand
Initialized empty Git repository in $LAB/ch12/clone-by-hand/by-hand/.git/
$ cd by-hand
$ git remote add origin ../server/support-bot.git
$ git config get --all --show-names --regex "^remote\."
remote.origin.url ../server/support-bot.git
remote.origin.fetch +refs/heads/*:refs/remotes/origin/*
```

Step 3, with step 4 as a side effect:

```
$ git fetch origin
From ../server/support-bot
 * [new branch]   feature/reranker -> origin/feature/reranker
 * [new branch]   main          -> origin/main
 * [new tag]      v0.1.0         -> v0.1.0
$ git show-ref --abbrev
510ee94 refs/remotes/origin/HEAD
```

```

336d5ee refs/remotes/origin/feature/reranker
510ee94 refs/remotes/origin/main
2828fba refs/tags/v0.1.0
$ git symbolic-ref refs/remotes/origin/HEAD
refs/remotes/origin/main
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)

```

The fetch created remote-tracking branches, a tag and `origin/HEAD`, and nothing else. `git status` still says "No commits yet": there is no local branch, and `HEAD` names a `main` that has not been born. Steps 5 and 6:

```

$ git switch main
Already on 'main'
branch 'main' set up to track 'origin/main'.
$ git config get --all --show-names --regexp "^branch\,"
branch.main.remote origin
branch.main.merge refs/heads/main
$ git branch -vv
* main 510ee94 [origin/main] Add retrieval config
$ ls
app
config.yaml
README.md

```

`git switch main` found no local `main`, found exactly one remote-tracking branch of that name, and created the branch from `origin/main` with upstream configuration. This is the `--guess` behavior of `git switch`, on by default; the older spelling is `git checkout main`. "Already on 'main'" appears because `HEAD` already named the unborn branch. A real `git clone` differs from the hand-built version in three details. It takes the name of the initial branch from the remote's `HEAD`, not from your `init.defaultBranch`. It writes `clone: from <url>` into the reflogs of `HEAD` and of the new branch. And when the source is a local path it does not transfer a pack: the object files are hard-linked or copied (`--local`, the default for paths; a `file://` URL or `--no-local` uses the normal pack transfer).

State table for `git clone <url> <directory>`, in the new repository:

Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
created: files of the default branch's tip	created to match	ref: refs/heads/<default>	created at the remote's tip	config (remote and upstream), refs/remotes/origin/*, origin/HEAD, tags, reflogs	unchanged (read only)	unchanged

Options that change the result.

```

$ git clone --origin company --branch release/0.1 server/support-bot.git by-name
Cloning into 'by-name'...
done.
$ git -C by-name config get --all --show-names --regexp "^(remote|branch)\,"
remote.company.url $LAB/ch12/clone-options/server/support-bot.git
remote.company.fetch +refs/heads/*:refs/remotes/company/*
branch.release/0.1.remote company
branch.release/0.1.merge refs/heads/release/0.1
$ git -C by-name branch -a -vv
* release/0.1          510ee94 [company/release/0.1] Add retrieval config
  remotes/company/HEAD -> company/main
  remotes/company/feature/reranker f1552ff Add reranker stub
  remotes/company/main 510ee94 Add retrieval config
  remotes/company/release/0.1 510ee94 Add retrieval config

```

`--origin company` changes the name of the remote and with it the prefix of every remote-tracking branch. `--branch release/0.1` changes which local branch is created and checked out. It does not limit what is fetched, and `company/HEAD` still records the remote's own default, `main`. `--branch` also accepts a tag:

```

$ git clone --branch v0.1.0 server/support-bot.git at-tag 2>&1 | head -4
Cloning into 'at-tag'...
done.
warning: refs/tags/v0.1.0 5ff339bf8f9ce7c8035bcd803ebf0afe0fc417b is not a commit!
Note: switching to '510ee948fb6354959cf03862f0ebe47c58eb2a74'.
$ git -C at-tag status
Not currently on any branch.
nothing to commit, working tree clean

```

The result is a detached `HEAD` at the tagged commit. The warning only says that the name given was an annotated tag object and not a commit. A Docker build that clones "a version" like this has no current branch, so a plain `git pull` inside it stops and asks which branch you mean.

```

$ git clone --single-branch server/support-bot.git narrow
Cloning into 'narrow'...
done.

```

```
$ git -C narrow config get --all remote.origin.fetch
+refs/heads/main:refs/remotes/origin/main
$ git -C narrow branch -r
origin/HEAD -> origin/main
origin/main
```

`--single-branch` narrows the refspec to one branch. Every later `git fetch` in that clone follows only that branch. `--depth <n>` implies it, and CI systems produce clones like this; section 12.12 widens one. Shallow and partial clones (`--depth`, `--filter`) belong to Chapter 26, Performance.

Bare and mirror clones.

```
$ git clone --bare server/support-bot.git copy.git
Cloning into bare repository 'copy.git'...
done.
$ git -C copy.git config get --all --show-names --regexp "^(core\.bare|remote)"
core.bare true
remote.origin.url $LAB/ch12/clone-options/server/support-bot.git
$ git -C copy.git show-ref --abbrev
f1552ff refs/heads/feature/reranker
510ee94 refs/heads/main
510ee94 refs/heads/release/0.1
5ff339b refs/tags/v0.1.0
```

```
$ git clone --mirror server/support-bot.git mirror.git
Cloning into bare repository 'mirror.git'...
done.
$ git -C mirror.git config get --all --show-names --regexp "^(core\.bare|remote)"
core.bare true
remote.origin.url $LAB/ch12/clone-options/server/support-bot.git
remote.origin.tagopt --no-tags
remote.origin.fetch +refs/*:refs/*
remote.origin.mirror true
$ git -C mirror.git show-ref --abbrev
f1552ff refs/heads/feature/reranker
510ee94 refs/heads/main
510ee94 refs/heads/release/0.1
5ff339b refs/tags/v0.1.0
```

Both are bare, and both copied the branches to `refs/heads/*` instead of `refs/remotes/origin/*`. The difference is in the configuration. The `--bare` clone has a URL and no fetch refspec. The `--mirror` clone has the refspec `+refs/*:refs/*` (every ref, same name, forced) and `mirror = true`, which makes a push from it behave as `git push --mirror`. The consequence appears when the source moves:

```
# The server has moved: one new commit on main and a new branch release/1.0.
$ git ls-remote --branches ../server/support-bot.git
04db75fbd9bdfc617297a4d0d76042bffa057ba refs/heads/main
04db75fbd9bdfc617297a4d0d76042bffa057ba refs/heads/release/1.0
$ git -C copy.git fetch
From ../server/support-bot
* branch HEAD -> FETCH_HEAD
$ git -C copy.git show-ref --abbrev
510ee94 refs/heads/main
$ git -C copy.git config get --all remote.origin.fetch || echo "(no fetch refspec configured)"
(no fetch refspec configured)
```

With no refspec, `git fetch` downloads the remote's `HEAD` into `FETCH_HEAD` and moves no ref. This "backup" still has the old `main` and has never heard of `release/1.0`.

```
$ git -C mirror.git fetch
From ../server/support-bot
510ee94..04db75f main -> main
* [new branch] release/1.0 -> release/1.0
$ git -C mirror.git show-ref --abbrev
04db75f refs/heads/main
04db75f refs/heads/release/1.0
```

```
# release/1.0 has been deleted on the server.
$ git -C mirror.git remote update --prune
From ../server/support-bot
- [deleted] (none) -> release/1.0
$ git -C mirror.git show-ref --abbrev
04db75f refs/heads/main
```

The mirror follows, and with `--prune` it deletes what the source deleted. That faithfulness is the catch. A mirror reproduces a destructive force push or a branch deletion at its next update, and being bare it keeps no reflog. It is a replica of the present, not a record of the past. A backup needs history of its own: reflogs switched on in the mirror (`core.logAllRefUpdates`), or periodic bundles (section 12.13).

`origin/HEAD`.

```
$ cat .git/refs/remotes/origin/HEAD
ref: refs/remotes/origin/main
```

```
$ git rev-parse --abbrev-ref origin/HEAD
origin/main
$ git log --oneline -1 origin
510ee94 Add retrieval config
```

In a files-format repository the symbolic ref is a one-line file. It makes the bare remote name a valid revision: `origin` resolves to `refs/remotes/origin/HEAD` (the last rule of the name lookup in [gitrevisions](#)), which is why `git log origin` works. Scripts use it to find the default branch without hard-coding `main`. It is a local record, and it goes stale:

```
# The server default branch is now trunk (its HEAD was repointed).
$ git ls-remote --symref origin HEAD
ref: refs/heads/trunk HEAD
510ee948fb6354959cf03862f0ebe47c58eb2a74 HEAD
$ git fetch
From ../server/support-bot
* [new branch] trunk -> origin/trunk
$ git rev-parse --abbrev-ref origin/HEAD
origin/main
$ git remote set-head origin --auto
'origin/HEAD' has changed from 'main' and now points to 'trunk'
$ git rev-parse --abbrev-ref origin/HEAD
origin/trunk
```

`git ls-remote --symref` asks the server, whose default is now `trunk`. The fetch brought the new branch and left `origin/HEAD` alone.  `git remote set-head origin --auto` asks the server and repoints it. If the old default branch is later deleted and pruned, the symbolic ref dangles until someone repairs it:

```
$ cd ../asha/support-bot
$ git fetch --prune
From ../server/support-bot
- [deleted] (none) -> origin/main
refs/remotes/origin/HEAD has become dangling after refs/remotes/origin/main was deleted
$ git rev-parse --verify origin/HEAD
warning: ignoring dangling symref refs/remotes/origin/HEAD
fatal: Needed a single revision
[exit status: 128]
$ git remote set-head origin --auto
'origin/HEAD' has changed from 'main' and now points to 'trunk'
$ git branch -r
origin/HEAD -> origin/trunk
origin/trunk
```

A fetch creates `origin/HEAD` when it is missing:

```
$ git remote set-head origin --delete
$ git branch -r
origin/trunk
$ git fetch --dry-run
$ git branch -r
origin/HEAD -> origin/trunk
origin/trunk
```

Look at the command that created it: on Git 2.55.0 `git fetch --dry-run` wrote the ref. "Dry run" does not cover this side effect. What a fetch does when `origin/HEAD` exists and disagrees with the server is a per-remote setting, `remote.<name>.followRemoteHEAD`, whose default `create` never touches an existing ref:

```
$ git rev-parse --abbrev-ref origin/HEAD
origin/main
$ git config set remote.origin.followRemoteHEAD warn
$ git fetch
hint: Run 'git remote set-head origin trunk' to follow the change, or set
hint: 'remote.origin.followRemoteHEAD' configuration option to a different value
hint: if you do not want to see this message. Specifically running
hint: 'git config set remote.origin.followRemoteHEAD warn-if-not-branch-trunk'
hint: will disable the warning until the remote changes HEAD to something else.
hint: Disable this message with "git config set advice.fetchRemoteHEADWarn false"
'HEAD' at 'origin' is 'trunk', but we have 'main' locally.
$ git rev-parse --abbrev-ref origin/HEAD
origin/main
```

`warn` reports the mismatch and changes nothing. The hint proposes a value that silences the warning for one branch name. On Git 2.55.0 the spelling printed in the hint does not silence anything; the spelling described in the `git config` manual, `warn-if-not-<branch>`, does:

```
# The value suggested by the hint above, then the value described in the manual:
$ git config set remote.origin.followRemoteHEAD warn-if-not-branch-trunk
$ git fetch 2>&1 | tail -1
'HEAD' at 'origin' is 'trunk', but we have 'main' locally.
$ git config set remote.origin.followRemoteHEAD warn-if-not-trunk
$ git fetch
```

`always` makes every fetch follow the server silently:


```
$ git config set remote.origin.followRemoteHEAD always
$ git fetch
$ git rev-parse --abbrev-ref origin/HEAD
origin/trunk
```

Version note. Older behavior: `git fetch` never wrote `refs/remotes/<remote>/HEAD`; it came from `git clone` or `git remote set-head`. Current behavior: `git fetch` creates it when missing and honors `remote.<name>.followRemoteHEAD` (`create`, `warn`, `warn-if-not-<branch>`, `always`, `never`). Since: Git 2.48 ([release notes](#)). A global `fetch.followRemoteHEAD` was added in Git 2.56 (not run here; [fetch configuration at 2.56](#)). Recommended: `warn` in long-lived clones, so that a renamed default branch announces itself.

GitHub, not Git. Which branch is the default is a property of the server repository, its `HEAD`. On GitHub a repository administrator changes it in the repository settings ([Changing the default branch](#)) or by renaming the branch. After a rename GitHub redirects web URLs and retargets open pull requests, but it cannot reach into clones: its documentation lists four commands for every existing clone, ending with `git remote set-head origin -a` ([Renaming a branch](#)).

In production. A nightly evaluation job clones with `--depth 1`. An engineer debugging in that workspace types `git switch release/2.3` and gets `fatal: invalid reference`, although the branch exists on the server: the clone's refspec does not include it. And after a default-branch rename, every script that says `origin/HEAD` or `origin/master` keeps comparing against the old branch until `set-head` has run in that clone.

12.4 `git fetch`: which refs move

In one sentence. 🟢 `git fetch` asks a remote where its refs point, downloads the objects you lack, and moves your remote-tracking refs to match; it never moves a local branch, the index or the working tree.

Analogy. A scout returns and you redraw the wall map: the other team's flags are placed where the scout saw them. Your own flags stay where they are, and the map starts to age the moment it is drawn. The analogy breaks in one place: the scout also brings back a complete copy of everything the other team built, not only the positions.

Precisely. `git fetch [<remote>] [<refspec>...]`. The remote defaults to the upstream remote of the current branch, otherwise `origin`. Without a refspec on the command line, the configured `remote.<name>.fetch` values select the server's refs and say which local ref records each. For every selected ref Git downloads missing objects and updates the destination if the update is a fast-forward or the refspec begins with `+`. Three extras: tags that point into the fetched history are created (tag following), `FETCH_HEAD` is rewritten, and a missing `origin/HEAD` is created.

Inside `.git`. New objects; `refs/remotes/<remote>/*` and their reflogs; possibly `refs/tags/*`; `FETCH_HEAD`. Not touched: `refs/heads/*`, `HEAD`, `index`.

See it. Asha has published a commit on `main`, a new branch and a tag. You have not fetched since you cloned:

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
$ git show-ref --abbrev
510ee94 refs/heads/main
510ee94 refs/remotes/origin/HEAD
510ee94 refs/remotes/origin/main
```

"Your branch is up to date with 'origin/main'" compares two refs in your own repository, both at `510ee94`. No connection was made. The sentence is true of your last fetch and says nothing about the server. To ask the server without changing anything, use `git ls-remote`:

```
$ git ls-remote origin
ef22149d97a296904ccbf984cf520a1fc387e195      HEAD
698e2261b0e2c60654fc059c6016c8d743a05571      refs/heads/feature/reranker
ef22149d97a296904ccbf984cf520a1fc387e195      refs/heads/main
effd2621f6264b45f395c4ba776533482d6f466b      refs/tags/v0.1.0
ef22149d97a296904ccbf984cf520a1fc387e195      refs/tags/v0.1.0^{}
```

One line per ref on the server; `main` is at `ef22149`. The line ending in `^{}` is the commit that the annotated tag points to. `git ls-remote` accepts `--branches`, `--tags` and name patterns, and with `--exit-code` it fails when nothing matches, which is how a script should test whether a branch exists on a server.

```
$ git fetch
From ../server/support-bot
 510ee94..ef22149  main           -> origin/main
* [new branch]    feature/reranker -> origin/feature/reranker
* [new tag]       v0.1.0         -> v0.1.0
$ cat .git/FETCH_HEAD
ef22149d97a296904ccbf984cf520a1fc387e195      branch 'main' of ../server/support-bot
```

698e2261b0e2c60654fc059c6016c8d743a05571	not-for-merge	branch 'feature/reranker' of ../../server/support-bot
effd2621f6264b45f395c4ba776533482d6f466b	not-for-merge	tag 'v0.1.0' of ../../server/support-bot

510ee94..ef22149 main -> origin/main is a fast-forward of a remote-tracking branch, printed as a range that you can paste into `git log`. * [new branch] and * [new tag] are creations. The name after the arrow is always a ref of yours. `FETCH_HEAD` holds one line per fetched ref: object ID, a marker, a description. Exactly one line is not marked `not-for-merge`: the upstream of the branch you are on. That line is what `git pull` integrates (section 12.6).

```
$ git show-ref --abbrev
510ee94 refs/heads/main
ef22149 refs/remotes/origin/HEAD
698e226 refs/remotes/origin/feature/reranker
ef22149 refs/remotes/origin/main
effd262 refs/tags/v0.1.0
$ git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
(use "git pull" to update your local branch)

nothing to commit, working tree clean
```

`refs/heads/main` did not move. `git status`, still without any network access, now gives a different answer because one of the two refs it compares has changed.

```
$ git log --oneline --graph --decorate --all
* 698e226 (origin/feature/reranker) Add reranker stub
* ef22149 (tag: v0.1.0, origin/main, origin/HEAD) Implement retrieval
* 510ee94 (HEAD -> main) Add retrieval config
* 95671d3 Add retriever skeleton
* f56c1bb Add README
$ git log --oneline main..origin/main
ef22149 Implement retrieval
```

Between a fetch and an integration you can inspect exactly what arrived: `main..origin/main` lists the commits the remote has and your branch lacks. This is the practical argument for fetching instead of pulling: you look first and decide second.

```
$ git reflog show origin/main
ef22149 refs/remotes/origin/main@{0}: fetch: fast-forward
$ git reflog show main
510ee94 main@{0}: clone: from $LAB/ch12/fetch-anatomy/server/support-bot.git
```

Every remote-tracking ref has a reflog. Its entries are a history of what you observed on the server, and in plain Git it is the only such history, because the bare server keeps none. The reflog of the local branch shows that it has not moved since the clone.

```
$ git merge --ff-only origin/main
Updating 510ee94..ef22149
Fast-forward
 app/retriever.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git status -sb
## main...origin/main
```

Integration is a separate, local step, here a fast-forward merge. `## main...origin/main` with nothing in brackets means the two refs are equal.

Picture.

before git fetch			after git fetch		
server	...---510ee94---ef22149	main	server	unchanged	
you	...---510ee94	main	you	...---510ee94---ef22149	
	origin/main			main origin/main	
				(HEAD) tag: v0.1.0	

State table for `git fetch`:

Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
unchanged	unchanged	unchanged	unchanged	remote-tracking refs and their reflogs; followed tags; <code>FETCH_HEAD</code> ; <code>origin/HEAD</code> if missing; new objects	unchanged	unchanged

Fetching one branch, or from a URL.

```
$ git fetch origin main
From ../../server/support-bot
 * branch      main      -> FETCH_HEAD
  ef22149..3f226f1 main    -> origin/main
$ cat .git/FETCH_HEAD
3f226f11ffbe22690dad919d155901b0885028cc      branch 'main' of ../../server/support-bot
```

```
$ git status -sb
## main...origin/main [behind 1]
```

With a branch named on the command line only that branch is fetched, and `FETCH_HEAD` has a single line. `origin/main` is updated as well, because the configured refspec says where `main` of `origin` is recorded.

```
$ git fetch ../../server/support-bot.git feature/reranker
From ../../server/support-bot
 * branch          feature/reranker -> FETCH_HEAD
$ cat .git/FETCH_HEAD
698e2261b0e2c60654fc059c6016c8d743a05571      branch 'feature/reranker' of ../../server/support-bot
$ git log --oneline -1 FETCH_HEAD
698e226 Add reranker stub
```

A fetch from a URL records its result in `FETCH_HEAD` and nowhere else, although the URL is the same repository as `origin`. Git knows remotes by name, not by URL. Use this form for a single look at somebody's branch (`git log FETCH_HEAD`).

Tags. A fetch creates tags that point into the history it fetched. It does not update a tag you already have:

```
$ git ls-remote --tags origin
e4e2561179ddf88e121ea1ebbb2485416f92363c      refs/tags/experiment/rejected-1
f858aefbb840dff2a665a6adbba8103bb645238      refs/tags/v0.1.0
99b643016420ca155f7432cf4555c1991c7679a5      refs/tags/v0.1.0^{}
$ git fetch
From ../../server/support-bot
 510ee94..99b6430  main      -> origin/main
$ git show-ref --abbrev --tags
5ff339b refs/tags/v0.1.0
```

On the server `v0.1.0` has been moved by a forced tag push, and a second tag sits on a commit that is on no branch. The plain fetch brought neither: your `v0.1.0` already exists, and the other tag does not point into fetched history.

```
$ git fetch --tags
From ../../server/support-bot
 * [new tag] experiment/rejected-1 -> experiment/rejected-1
 ! [rejected] v0.1.0                -> v0.1.0 (would clobber existing tag)
[exit status: 1]
$ git show-ref --abbrev --tags
e4e2561 refs/tags/experiment/rejected-1
5ff339b refs/tags/v0.1.0
```

```
$ git fetch --tags --force
From ../../server/support-bot
 t [tag update]      v0.1.0      -> v0.1.0
$ git show-ref --abbrev --tags
e4e2561 refs/tags/experiment/rejected-1
f858aef refs/tags/v0.1.0
```

`--tags` asks for every tag: the new one arrives, the moved one is rejected ("would clobber existing tag"), and only  `git fetch --tags --force` replaces it. Until someone forces, your `v0.1.0` and the server's are different commits under one name, and a build "by tag" gives different results on different machines. That is why moving a published tag is a release incident and not a convenience (Chapter 14B).

Observed behavior :	"Your branch is up to date with 'origin/main'", yet the server has newer commits.
Git state :	refs/heads/main and refs/remotes/origin/main name the same commit; the server's refs/heads/main names a newer one.
Mechanism :	git status compares two local refs. It opens no connection.
Root cause :	origin/main is a record of your last fetch, not a view of the server.
Why Git does this :	every command except the transfer commands works offline and instantly; a status that needed the network would be slow, and useless on a plane.
Correct fix :	git fetch, then read the status again; or git ls-remote origin main to ask the server without changing anything.
Prevention :	fetch before any decision that depends on the server (tagging, releasing, cutting a hotfix branch). In scripts compare against git ls-remote, never against a remote-tracking ref of unknown age.

In production. A release engineer tags from a laptop whose status said "up to date". The last fetch was two days old, so the tag misses yesterday's hotfix. The control is not "be careful". It is a release script that begins with `git fetch` and refuses to continue unless `git rev-parse HEAD` equals the ID that `git ls-remote origin refs/heads/main` prints.

12.5 Upstream branches and `push.default`

In one sentence. The upstream of a local branch is the branch on a remote that `status` compares it with and that `pull` integrates from; it is two lines of configuration.

Analogy. A default correspondent printed on a letter template, so that `pull` and `push` need no address. It breaks because incoming and outgoing mail can have different defaults: `@{upstream}` and `@{push}` are two questions (section 12.10).

Precisely. `branch.<name>.remote` names a remote. `branch.<name>.merge` names a ref on that remote, such as `refs/heads/main`: the server's name, not your remote-tracking ref. `<branch>@{upstream}`, short `@{u}`, resolves the pair to the remote-tracking branch that records it. `git status` and `git branch -vv` use it for the ahead and behind counts, `git pull` for what to integrate, and `git push` for where to push under the default `push.default`.

Inside .git. Only `.git/config`. An upstream is not a ref and not an object.

See it. A new local branch has no upstream, and a bare 🟡 `git push` refuses to guess:

```
$ git switch -c feature/eval-harness
Switched to a new branch 'feature/eval-harness'
$ git branch -vv
* feature/eval-harness b83418c Add eval harness entry point
  main                  510ee94 [origin/main] Add retrieval config
$ git push
fatal: The current branch feature/eval-harness has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin feature/eval-harness

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.

[exit status: 128]
```

```
$ git push -u origin feature/eval-harness
To ../../server/support-bot.git
 * [new branch]      feature/eval-harness -> feature/eval-harness
branch 'feature/eval-harness' set up to track 'origin/feature/eval-harness'.
$ git config get --all --show-names --regex "^branch\\.feature/eval-harness\\."
branch.feature/eval-harness.remote origin
branch.feature/eval-harness.merge refs/heads/feature/eval-harness
$ git branch -vv
* feature/eval-harness b83418c [origin/feature/eval-harness] Add eval harness entry point
  main                  510ee94 [origin/main] Add retrieval config
```

`-u` (`--set-upstream`) pushes and then writes the two keys. Look at the value of `merge`: the name of the branch on the remote.

```
$ git rev-parse --abbrev-ref @{upstream}
origin/feature/eval-harness
$ git rev-parse --symbolic-full-name @{u} @{push}
refs/remotes/origin/feature/eval-harness
refs/remotes/origin/feature/eval-harness
$ git log --oneline @{u}..
fc5bd70 Add first eval case
$ git status -sb
## feature/eval-harness...origin/feature/eval-harness [ahead 1]
```

`@{u}` resolves to the remote-tracking ref. `@{push}` is where `git push` would send the branch; with one remote it is the same ref. `git log @{u}..` lists what you have not pushed, and `git log ..@{u}` what you have fetched and not integrated.

```
$ git switch feature/reranker
Switched to a new branch 'feature/reranker'
branch 'feature/reranker' set up to track 'origin/feature/reranker'.
$ git config get --all --show-names --regex "^branch\\.feature/reranker\\."
branch.feature/reranker.remote origin
branch.feature/reranker.merge refs/heads/feature/reranker
```

```
$ git switch -c docs/runbook main
Switched to a new branch 'docs/runbook'
$ git rev-parse --abbrev-ref @{u}
fatal: no upstream configured for branch 'docs/runbook'
[exit status: 128]
$ git branch -u origin/main
branch 'docs/runbook' set up to track 'origin/main'.
$ git status -sb
## docs/runbook...origin/main
$ git branch --unset-upstream
$ git status -sb
## docs/runbook
```

Switching to a name that exists only as a remote-tracking branch creates the local branch with its upstream (the guess from section 12.3). 🟡 `git branch -u` sets an upstream for an existing branch without pushing, and `--unset-upstream` removes it. The ways an upstream comes to exist:

Command	Upstream is set
<code>git clone</code>	for the initial branch
<code>git switch <name></code> (guess), <code>git switch -c <new></code> <code><remote>/<branch></code>	when the start point is a remote-tracking branch (<code>branch.autoSetupMerge</code> , default <code>true</code>)

Command	Upstream is set
<code>git push -u <remote> <branch></code>	after a successful push
<code>git branch -u <remote>/<branch></code>	explicitly, no transfer
<code>git push</code> with <code>push.autoSetupRemote=true</code>	on the first push of a branch that has none

```
$ git config set push.autoSetupRemote true
$ git push
To ../../server/support-bot.git
 * [new branch]      docs/runbook -> docs/runbook
branch 'docs/runbook' set up to track 'origin/docs/runbook'.
$ git branch -vv
* docs/runbook       9416de1 [origin/docs/runbook] Start the runbook
feature/eval-harness fc5bd70 [origin/feature/eval-harness: ahead 1] Add first eval case
feature/reranker     336d5ee [origin/feature/reranker] Add reranker stub
main                 510ee94 [origin/main] Add retrieval config
```

`push.default`: what a bare `git push` updates.

```
$ git switch -c latency-fix origin/main
Switched to a new branch 'latency-fix'
branch 'latency-fix' set up to track 'origin/main'.
$ git branch -vv
docs/runbook       9416de1 [origin/docs/runbook] Start the runbook
feature/eval-harness fc5bd70 [origin/feature/eval-harness: ahead 1] Add first eval case
feature/reranker    336d5ee [origin/feature/reranker] Add reranker stub
* latency-fix       70d0911 [origin/main: ahead 1] Set request timeout
main               510ee94 [origin/main] Add retrieval config
$ git push
fatal: The upstream branch of your current branch does not match
the name of your current branch.  To push to the upstream branch
on the remote, use

    git push origin HEAD:main

To push to the branch of the same name on the remote, use

    git push origin HEAD

To choose either option permanently, see push.default in 'git help config'.

To avoid automatically configuring an upstream branch when its name
won't match the local branch, see option 'simple' of branch.autoSetupMerge
in 'git help config'.

[exit status: 128]
```

`latency-fix` was created from `origin/main`, so automatic setup made `origin/main` its upstream. Under the default `simple`, Git refuses to push when the upstream's name differs from the branch's name: it cannot know whether you mean "update `main`" or "publish `latency-fix`". The other values of `push.default` each answer that question. `-c push.default=<value>` with `--dry-run` shows them without changing anything:

```
$ git -c push.default=upstream push --dry-run
To ../../server/support-bot.git
 510ee94..70d0911 latency-fix -> main
$ git -c push.default=current push --dry-run
To ../../server/support-bot.git
 * [new branch]      latency-fix -> latency-fix
$ git -c push.default=nothing push --dry-run
fatal: You didn't specify any refsspecs to push, and push.default is "nothing".
[exit status: 128]
$ git -c push.default=matching push --dry-run
To ../../server/support-bot.git
 b83418c..fc5bd70 feature/eval-harness -> feature/eval-harness
```

Value	A bare <code>git push</code> on <code>latency-fix</code> , upstream <code>origin/main</code>
<code>simple</code> (default since Git 2.0)	refuses: the names differ
<code>upstream</code>	updates <code>main</code> on the server
<code>current</code>	creates or updates <code>latency-fix</code> on the server
<code>nothing</code>	refuses: you must name a refspec
<code>matching</code> (default before Git 2.0)	pushes every local branch that has a branch of the same name on the server

Read the last transcript line again. Under `matching` the command pushed `feature/eval-harness`, a branch you were not on, and did not push the branch you were on. That is why the default changed ([push configuration](#)).

In production. An ML engineer branches `exp/lr-sweep` from `origin/main` and commits a sweep configuration. If their configuration says `push.default=upstream`, a bare `git push` sends those commits straight to `main`. With `simple` it stops. Two clean setups for topic branches:

`branch.autoSetupMerge=simple` (an upstream only when the names match; Git 2.37 or later), or `git switch -c <name> --no-track origin/main` together with `push.autoSetupRemote`.

12.6 `git pull`: fetch plus one integration step

In one sentence. 🟡 `git pull` is `git fetch` followed by one integration of the fetched upstream into the current branch: a fast-forward, a merge or a rebase; when the two branches have diverged, Git makes you say which.

Analogy. "Collect the mail and file it" as a single instruction. It breaks when filing needs a decision: if both sides have new commits there are two legitimate ways to combine them, they produce different histories, and pull will not choose for you.

Precisely. Step 1 runs `git fetch` with the same arguments, so remote-tracking refs and `FETCH_HEAD` are updated. Step 2 integrates the `FETCH_HEAD` line that is marked for merging. If your branch is an ancestor of it, the branch is fast-forwarded. If it is an ancestor of your branch, there is nothing to do. If neither is true the branches have diverged, and `pull.rebase`, `pull.ff` or a flag must select the method.

Inside .git. Everything a fetch changes, then everything a merge or a rebase changes: the branch ref, the reflogs (entries begin with `pull`), `ORIG_HEAD`, the index and the working tree; with conflicts, the state described in Chapter 8 and Chapter 9.

See it. First the case with no local commits:

```
$ git pull
From ../server/support-bot
 510ee94..ef22149 main    -> origin/main
Updating 510ee94..ef22149
Fast-forward
 app/retriever.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git reflog -1
ef22149 HEAD@{0}: pull: Fast-forward
$ git reflog show origin/main -1
ef22149 refs/remotes/origin/main@{0}: pull: fast-forward
```

The output is two outputs glued together: the `From` block is the fetch, and `Updating ... Fast-forward` is the merge. Now both sides have a new commit:

```
# Asha has pushed "Raise top_k to 8"; you have committed "Add dev requirements".
$ git status -sb
## main...origin/main [ahead 1]
$ git pull
From ../server/support-bot
 ef22149..527a708 main    -> origin/main
hint: You have divergent branches and need to specify how to reconcile them.
hint: You can do so by running one of the following commands sometime before
hint: your next pull:
hint:
hint:   git config pull.rebase false # merge
hint:   git config pull.rebase true  # rebase
hint:   git config pull.ff only      # fast-forward only
hint:
hint: You can replace "git config" with "git config --global" to set a default
hint: preference for all repositories. You can also pass --rebase, --no-rebase,
hint: or --ff-only on the command line to override the configured default per
hint: invocation.
fatal: Need to specify how to reconcile divergent branches.
[exit status: 128]
```

`[ahead 1]` was the stale view. The fetch half worked (`ef22149..527a708 main -> origin/main`), then Git stopped with exit status 128.

```
$ git status -sb
## main...origin/main [ahead 1, behind 1]
$ git log --oneline --graph --decorate --all -4
* c498de8 (HEAD -> main) Add dev requirements
| * 527a708 (origin/main, origin/HEAD) Raise top_k to 8
|/
* ef22149 Implement retrieval
* 510ee94 Add retrieval config
```

A pull that ends in this error is not a no-op. Your remote-tracking ref moved, and status now tells the truth: `[ahead 1, behind 1]`. Your branch, index and working tree are untouched.

Root cause. The fatal error is not a network failure and not damage. Git fetched successfully and then declined to pick one of two possible histories on your behalf.

The hint offers three answers. Each one as a flag:

```
$ git pull --ff-only
hint: Diverging branches can't be fast-forwarded, you need to either:
```

```
hint:
hint:   git merge --no-ff
hint:
hint: or:
hint:
hint:   git rebase
hint:
hint: Disable this message with "git config set advice.diverging false"
fatal: Not possible to fast-forward, aborting.
[exit status: 128]
```

`--ff-only` (configuration: `pull.ff=only`) means "move my branch only when there is nothing to combine". On diverged branches it refuses, with a different message from the unconfigured case.

```
$ git pull --no-rebase
Merge made by the 'ort' strategy.
 config.yaml | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline --graph --decorate -5
* 7044e06 (HEAD -> main) Merge branch 'main' of ../../server/support-bot
| \
| * 527a708 (origin/main, origin/HEAD) Raise top_k to 8
* | c498de8 Add dev requirements
|/
* ef22149 Implement retrieval
* 510ee94 Add retrieval config
$ git reflog -2
7044e06 HEAD@{0}: pull --no-rebase: Merge made by the 'ort' strategy.
c498de8 HEAD@{1}: commit: Add dev requirements
```

`--no-rebase` (`pull.rebase=false`) creates a merge commit with two parents. Its message contains the URL because pull merges `FETCH_HEAD`, not `origin/main`.

```
$ git reset --hard ORIG_HEAD
HEAD is now at c498de8 Add dev requirements
$ git status -sb
## main...origin/main [ahead 1, behind 1]
```

The merge recorded the previous tip in `ORIG_HEAD`, so `git reset --hard ORIG_HEAD` takes back a pull you did not want. It is a  command: it also discards uncommitted changes (Chapter 11).

```
$ git pull --rebase
Rebasing (1/1)
Successfully rebased and updated refs/heads/main.
$ git log --oneline --graph --decorate -4
* 45eec24 (HEAD -> main) Add dev requirements
* 527a708 (origin/main, origin/HEAD) Raise top_k to 8
* ef22149 Implement retrieval
* 510ee94 Add retrieval config
$ git reflog -4
45eec24 HEAD@{0}: pull --rebase (finish): returning to refs/heads/main
45eec24 HEAD@{1}: pull --rebase (pick): Add dev requirements
527a708 HEAD@{2}: pull --rebase (start): checkout 527a708c84be3e9847d2e65528c2d8cb7ce6b863
c498de8 HEAD@{3}: reset: moving to ORIG_HEAD
```

`--rebase` (`pull.rebase=true`) re-creates your commit on top of the upstream: `c498de8` became `45eec24`. History stays linear, and the old commit remains reachable from the reflog.

Picture.

diverged, after the fetch half:	<code>--no-rebase</code> (merge):	<code>--rebase</code> :
<pre> c498de8 main (HEAD) / ef22149---527a708 origin/main </pre>	<pre> c498de8---7044e06 main / / ef22149---527a708 origin/main </pre>	<pre> ef22149---527a708---45eec24 main / origin/main (45eec24 replaces c498de8) </pre>
<code>--ff-only</code> : refuses, nothing moves		

Which setting wins. The script in `labs/ch12/pull-matrix.sh` runs `git pull` from one diverged state under every combination of the two configuration keys, and then with each flag:

```
$ sh ../../try-pull.sh
pull.rebase=unset pull.ff=unset -> fatal: Need to specify how to reconcile divergent branches.
pull.rebase=unset pull.ff=only -> fatal: Not possible to fast-forward, aborting.
pull.rebase=unset pull.ff=false -> merge commit
pull.rebase=unset pull.ff=true -> merge commit
pull.rebase=false pull.ff=unset -> merge commit
pull.rebase=false pull.ff=only -> fatal: Not possible to fast-forward, aborting.
pull.rebase=false pull.ff=false -> merge commit
pull.rebase=false pull.ff=true -> merge commit
pull.rebase=true pull.ff=unset -> rebase
pull.rebase=true pull.ff=only -> fatal: Not possible to fast-forward, aborting.
```

```
pull.rebase=true pull.ff=false -> rebase
pull.rebase=true pull.ff=true  -> rebase
```

```
$ sh ../../try-pull.sh --rebase | cut -d'>' -f2 | sort | uniq -c
12 rebase
$ sh ../../try-pull.sh --no-rebase | cut -d'>' -f2 | sort | uniq -c
12 merge commit
$ sh ../../try-pull.sh --ff-only | cut -d'>' -f2 | sort | uniq -c
12 fatal: Not possible to fast-forward, aborting.
```

Three rules, as observed on Git 2.55.0. With nothing set, a diverged pull is fatal. `pull.ff=only` in configuration beats `pull.rebase` in configuration: the row `pull.rebase=true pull.ff=only` refuses. A flag on the command line beats all configuration: each flag gave the same result in all twelve rows.

State table for `git pull`, by outcome:

Outcome	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
fast-forward	updated	updated	unchanged	moved forward	as fetch, plus <code>ORIG_HEAD</code>	unchanged	unchanged
merge	merge result	merge result	unchanged	new merge commit	as fetch, plus <code>ORIG_HEAD</code>	unchanged	unchanged
rebase	result of replay	updated	detached during the replay, then the branch again	moved to the new commits	as fetch, plus <code>ORIG_HEAD</code>	unchanged	unchanged
fatal, or <code>--ff-only</code> refusal	unchanged	unchanged	unchanged	unchanged	as fetch	unchanged	unchanged

Choosing. Rebase keeps history linear, but every replayed commit is a new commit that nobody has tested, local merge commits are flattened unless you use `--rebase=merges`, and it treats commits that used to be on the upstream in a way that can surprise you (section 12.8). Merge preserves exactly what you tested and adds one merge commit per pull. A global `pull.ff=only` is a defensible default: pull never creates a commit unless you pass `--rebase` or `--no-rebase` at that moment.

Version note. Older behavior: an unconfigured `git pull` on diverged branches merged silently (before Git 2.27), then merged with a warning (2.27 to 2.33.0). Current behavior: it fetches, prints the hint and stops with `fatal: Need to specify how to reconcile divergent branches`. Since: Git 2.33.1 and 2.34.0 ([pull.c at 2.56](#), [release notes 2.34](#)). Recommended: decide once, in configuration, and override with a flag when the situation differs.

In production. A deploy host or a CI job should never create commits. Its update step is `git pull --ff-only`, or `git fetch` followed by an explicit checkout of the fetched commit. A host that merges on pull will one day build a commit that exists nowhere else.

12.7 `git push`: asking another repository to move its refs

In one sentence. 🟡 `git push` asks a remote to make some of its refs point at commits of yours, sends the objects it lacks, and succeeds for each ref only if both your Git and the remote accept the update.

Analogy. Posting entries to a shared ledger: "I have seen everything up to entry N; here are my entries after it." Your own clerk checks the claim before you reach the counter, then the office applies its house rules. The analogy breaks because you can instruct your clerk to skip the check (force), and the office then refuses only if someone configured it to.

Precisely. A push runs in a fixed order:

```
your clone                                server (git-receive-pack)
1 read the server's refs                  <---- advertisement: refs/heads/main = 719650d
2 client rules for each ref:
  fast-forward? tag exists? lease holds?
  no --> "! [rejected]" (nothing is sent)
3 send "<old> <new> <ref>" + packfile      ---->
                                           4 objects go into a quarantine directory
                                           5 ref still at <old>? receive.deny* settings?
                                           pre-receive and update hooks?
                                           no --> "! [remote rejected]"
                                           6 refs updated; post-receive hooks run
                                           report: ok refs/heads/main
7 move refs/remotes/origin/<branch>      <----
```

What is pushed when you name nothing was section 12.5. With arguments, each refspec is `[+]<source>:<destination>`: the source is any commit expression of yours, the destination a ref name on the remote. The client rules of step 2 are in the manual's "push rules": an existing branch may only be fast-forwarded, an existing tag may not be updated at all, and creations and deletions are allowed unless configuration or hooks forbid them ([git-push](#)).

Inside .git. In your repository, only the remote-tracking ref of each ref that was pushed, and its reflog. On the server: new objects and the updated refs.

See it: one rule, two messages. Asha has pushed. You have one local commit and a stale view:

```
$ git status -sb
## main...origin/main [ahead 1]
$ git push
To ../../server/support-bot.git
! [rejected]        main -> main (fetch first)
error: failed to push some refs to ' ../../server/support-bot.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
[exit status: 1]
$ git status -sb
## main...origin/main [ahead 1]
```

(fetch first): the server's `main` names a commit that your repository does not contain, so your Git cannot even test ancestry. Status still shows the stale `[ahead 1]`, because a rejected push fetches nothing.

```
$ git fetch
From ../../server/support-bot
 510ee94..719650d main      -> origin/main
$ git status -sb
## main...origin/main [ahead 1, behind 1]
$ git push
To ../../server/support-bot.git
! [rejected]        main -> main (non-fast-forward)
error: failed to push some refs to ' ../../server/support-bot.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
[exit status: 1]
```

After a fetch you have the server's commit, the ancestry test is possible, and it fails: (non-fast-forward). The server's tip is not an ancestor of what you push, so accepting it would drop Asha's commit from `main`. One cause, two messages, and the only difference is whether you had fetched. Both lines say `! [rejected]`, not `! [remote rejected]`. Your own Git decided:

```
# The client side of the conversation during the rejected push:
$ GIT_TRACE_PACKET=1 git push 2>&1 | sed -n 's/.*packet: *\(\push[<>]\)/\1/p' | fold -s -w 76
push< 719650da56c2910ce851d3c0f47ec270ebb88945
refs/heads/main@0report-status report-status-v2 delete-refs side-band-64k
quiet atomic ofs-delta object-format=sha1 agent=git/2.55.0-Darwin
push< 0000
push> 0000
```

The server advertised its ref (push<). Your Git answered with an empty request (push> 0000) and closed the connection. The server was never asked.

```
$ git rebase origin/main
Rebasing (1/1)
Successfully rebased and updated refs/heads/main.
$ git push --dry-run
To ../../server/support-bot.git
 719650d..8bf53be main -> main
$ git push
To ../../server/support-bot.git
 719650d..8bf53be main -> main
$ git status -sb
## main...origin/main
$ git reflog show origin/main
8bf53be refs/remotes/origin/main@{0}: update by push
719650d refs/remotes/origin/main@{1}: fetch: fast-forward
```

Integrate (a rebase here; a merge works as well), preview with `--dry-run`, push. The successful push moved `origin/main` immediately, recorded as `update by push`: your Git knows what the server has now, so no fetch is needed.

Pushes that fail before any comparison.

```
$ git branch
* hotfix/timeout
main
$ git push origin master
error: src refspec master does not match any
error: failed to push some refs to ' ../../server/support-bot.git'
[exit status: 1]
$ git push origin hotfix
error: src refspec hotfix does not match any
error: failed to push some refs to ' ../../server/support-bot.git'
[exit status: 1]
```

```
# A new repository with a remote but no commit yet:
$ git init -q ../../scratch
$ git -C ../../scratch remote add origin ../server/support-bot.git
$ git -C ../../scratch push -u origin main
error: src refspec main does not match any
error: failed to push some refs to '../server/support-bot.git'
[exit status: 1]
```

`src refspec <name> does not match any` is about your side: the source name matched no ref of yours. Either the name is wrong (`master` where your branch is `main`, or half a branch name), or the repository has no commit yet, because an unborn branch is not a ref. This message is among the thirty highest-voted Git questions on [Stack Overflow](#).

```
$ git switch --detach
HEAD is now at c19ab53 Set request timeout
$ git push
fatal: You are not currently on a branch.
To push the history leading to the current (detached HEAD)
state now, use

    git push origin HEAD:<name-of-remote-branch>

[exit status: 128]
$ git push origin HEAD:refs/heads/hotfix/timeout
To ../../server/support-bot.git
 * [new branch]      HEAD -> hotfix/timeout
$ git switch -
Switched to branch 'hotfix/timeout'
```

With a detached HEAD there is no current branch whose upstream could supply a destination, so you name both sides. The destination is written in full, `refs/heads/...`, because a detached HEAD is not a ref under `refs/heads/` from which Git could infer that you mean a branch.

```
$ git push origin hotfix/timeout
fatal: 'origin' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
[exit status: 128]
$ git remote
origin
```

A misspelled remote name is taken for a path to a repository, hence the misleading text about access rights.

Deleting, tags, several refs.

```
$ git push origin --delete feature/eval-harness
To ../../server/support-bot.git
 - [deleted]          feature/eval-harness
$ git branch -vv
feature/eval-harness edc57ed [origin/feature/eval-harness: gone] Add eval harness entry point
* main                510ee94 [origin/main] Add retrieval config
release/0.1           510ee94 [origin/release/0.1] Add retrieval config
$ git ls-remote --branches origin
510ee948fb6354959cf03862f0ebe47c58eb2a74      refs/heads/main
510ee948fb6354959cf03862f0ebe47c58eb2a74      refs/heads/release/0.1
```

🔴 `git push <remote> --delete <branch>` (or a refspec with an empty source, `:feature/eval-harness`) removes the branch on the server and your remote-tracking ref. Your local branch stays, with its upstream marked `gone` (section 12.11). For this 🔴 command: it changes one ref on the server. It can destroy the only name that the branch's commits have there, and a bare server keeps no record of the deletion. Preview with `git log --oneline origin/<branch> --not origin/main`, which lists the commits that only that branch holds. Recover by pushing the tip again from any clone that has it: `git push origin <commit>:refs/heads/<branch>`. It is appropriate for branches that are merged, or abandoned by agreement.

```
$ git tag -a v0.2.0 -m "Release 0.2.0"
$ git tag nightly
$ git push
Everything up-to-date
$ git ls-remote --tags origin
```

```
$ git push origin v0.2.0
To ../../server/support-bot.git
 * [new tag]          v0.2.0 -> v0.2.0
$ git tag -a v0.2.1 -m "Release 0.2.1"
$ git push --follow-tags
To ../../server/support-bot.git
 510ee94..7aec635    main -> main
 * [new tag]         v0.2.1 -> v0.2.1
$ git ls-remote --tags origin
e4cbd524cd2ce2e1757843ad5ee0c371484fd6f9      refs/tags/v0.2.0
510ee948fb6354959cf03862f0ebe47c58eb2a74      refs/tags/v0.2.0^{}
```

```
3b6d020723c18782f5703818be9f01f07146e2f7      refs/tags/v0.2.1
7aec635a45c70d81f8a60b04ce73a0dcfdbb15c8      refs/tags/v0.2.1^{}
```

Tags travel only when asked. `git push origin <tag>` pushes one. `--follow-tags` adds annotated tags that point into the commits being pushed: `v0.2.1` went, the lightweight `nightly` did not. `--tags` pushes all of them.

```
$ git tag -f -a v0.2.0 -m "Release 0.2.0, retagged"
Updated tag 'v0.2.0' (was e4cbd52)
$ git push origin v0.2.0
To ../../server/support-bot.git
! [rejected]          v0.2.0 -> v0.2.0 (already exists)
error: failed to push some refs to ' ../../server/support-bot.git'
hint: Updates were rejected because the tag already exists in the remote.
[exit status: 1]
```

That is the tag rule: a tag that exists on the remote is not replaced without force.

```
# main is behind the server (Asha pushed); release/0.1 is one commit ahead.
$ git config set advice.pushUpdateRejected false
$ git push --atomic origin main release/0.1
error: atomic push failed for ref refs/heads/main. status: 5
To ../../server/support-bot.git
! [rejected]          main -> main (fetch first)
! [rejected]          release/0.1 -> release/0.1 (atomic push failed)
error: failed to push some refs to ' ../../server/support-bot.git'
[exit status: 1]
$ git ls-remote --branches origin
bb34561abaa5aa4fabf570a6dd8417f74b6abb81      refs/heads/main
510ee948fb6354959cf03862f0ebe47c58eb2a74      refs/heads/release/0.1
```

```
$ git push origin main release/0.1
To ../../server/support-bot.git
510ee94..0c10e7b  release/0.1 -> release/0.1
! [rejected]          main -> main (fetch first)
error: failed to push some refs to ' ../../server/support-bot.git'
[exit status: 1]
$ git ls-remote --branches origin
bb34561abaa5aa4fabf570a6dd8417f74b6abb81      refs/heads/main
0c10e7b5f43e3b449f31c690a82fee2a5af38d81      refs/heads/release/0.1
```

Refs in one push are independent by default: `release/0.1` was updated although `main` was rejected, and the exit status is 1. With `--atomic` nothing was sent because one ref failed. A release script that pushes a branch and a tag needs `--atomic` and must check the exit status. (`advice.pushUpdateRejected=false` in the transcript only hides the hint block that the earlier rejections printed.)

The server's own rules.

```
$ git status -sb
## main...origin/main [ahead 1, behind 1]
$ git -C ../../server/support-bot.git config set receive.denyNonFastForwards true
$ git push --force
remote: error: denying non-fast-forward refs/heads/main (you should pull first)
To ../../server/support-bot.git
! [remote rejected] main -> main (non-fast-forward)
error: failed to push some refs to ' ../../server/support-bot.git'
[exit status: 1]
```

```
$ git push origin HEAD:refs/heads/tmp/scratch
To ../../server/support-bot.git
* [new branch]      HEAD -> tmp/scratch
$ git -C ../../server/support-bot.git config set receive.denyDeletes true
$ git push origin --delete tmp/scratch
remote: error: denying ref deletion for refs/heads/tmp/scratch
To ../../server/support-bot.git
! [remote rejected] tmp/scratch (deletion prohibited)
error: failed to push some refs to ' ../../server/support-bot.git'
[exit status: 1]
```

```
$ git -C ../../server/support-bot.git config unset receive.denyNonFastForwards
$ cp ../../pre-receive ../../server/support-bot.git/hooks/pre-receive
$ chmod +x ../../server/support-bot.git/hooks/pre-receive
$ cat ../../server/support-bot.git/hooks/pre-receive
#!/bin/sh
# Refuse every direct update of main. Standard input has one line per ref:
# <old-id> <new-id> <ref-name>
while read old new ref; do
    if [ "$ref" = "refs/heads/main" ]; then
        echo "policy: main only changes through a reviewed merge" >&2
        exit 1
    fi
done
exit 0
$ git push --force
remote: policy: main only changes through a reviewed merge
To ../../server/support-bot.git
```

```
! [remote rejected] main -> main (pre-receive hook declined)
error: failed to push some refs to '../server/support-bot.git'
[exit status: 1]
```

Now the lines read `! [remote rejected]`, and the `remote:` lines carry the server's own words. `--force` switches off your Git's checks, not the server's. Plain Git has `receive.denyNonFastForwards`, `receive.denyDeletes` and hooks: `pre-receive` reads one `old new ref` line per proposed update and can refuse the whole push, while the pushed objects wait in quarantine ([git-receive-pack](#)).

GitHub, not Git. On GitHub the server-side rules are rulesets, branch protection and push protection. They answer in the same shape. GitHub's documentation gives `remote: error: GH006: Protected branch update failed for refs/heads/main.` as the reply of a branch protection rule ([About protected branches](#)); Chapter 18 covers them.

State table for a successful `git push`:

Working tree	Index	HEAD	Current branch ref	Other refs and files in <code>.git</code>	Remote	GitHub
unchanged	unchanged	unchanged	unchanged	remote-tracking ref of each pushed ref and its reflog; upstream keys with <code>-u</code>	refs created, moved or deleted; new objects; hooks run	push events: pull request updates, workflow runs, rule evaluation

In production. "It said rejected" is not a diagnosis. Read the word before the bracket closes. `rejected` means your clone must change: fetch and integrate. `remote rejected` means a policy on the server spoke: read the `remote:` lines and talk to whoever owns that policy. Retrying with `--force` answers neither.

12.8 Forcing a push

A forced push tells step 2 of the pipeline to skip the fast-forward test. The server's ref is set to your commit, and whatever it named before is no longer reachable from that ref.

 **DANGEROUS:** `git push --force` (also `-f`, or a `+` before one refspec).

- **What it changes:** the ref on the server, and your remote-tracking ref.
- **What it can destroy:** every commit on the server's branch that is not in your history, including commits you have never seen. A default bare server has no reflog, so it keeps no pointer to them.
- **How to preview:** `git fetch`, then `git log --oneline HEAD..origin/<branch>` lists exactly the commits that would be dropped. `git push --dry-run --force` prints the server's current commit ID to the left of the three dots.
- **How to recover:** the old tip survives in any clone that had it (a teammate's branch, or the reflog of anybody's remote-tracking ref) and, until the server's garbage collection removes it, as an unreachable object on the server. Push it back.
- **When it is appropriate:** on a branch that only you use, such as your own pull request branch after a rebase, or in a coordinated history rewrite (Chapter 21B). Then use a lease.

See it: plain force destroys what you never saw. You rewrote a published commit on `docs/runbook`. Asha has since pushed a commit there, and you have not fetched:

```
# What your clone believes the server has, and what the server reports during a dry run:
$ git rev-parse --short origin/docs/runbook
17783a7
$ git push --dry-run --force origin docs/runbook
To ../server/support-bot.git
+ 73f4877...ca3cabb docs/runbook -> docs/runbook (forced update)
$ git cat-file -t 73f4877
fatal: Not a valid object name 73f4877
[exit status: 128]
$ git push --force origin docs/runbook
To ../server/support-bot.git
+ 73f4877...ca3cabb docs/runbook -> docs/runbook (forced update)
$ git reflog show origin/docs/runbook
ca3cabb refs/remotes/origin/docs/runbook@{0}: update by push
17783a7 refs/remotes/origin/docs/runbook@{1}: update by push
```

Your clone believes the server is at `17783a7`. The dry run prints `73f4877...ca3cabb`: the server is at `73f4877`, an object your repository does not have. That is Asha's commit. The forced push replaces it, and the reflog of your remote-tracking ref goes from `17783a7` straight to `ca3cabb`. Your clone never held the commit it destroyed.

```
# The overwritten commit still exists: on the server, unreachable, and in the clone of Asha.
$ git -C ../server/support-bot.git fsck --unreachable --no-reflogs | grep commit
unreachable commit 45de2936b350e2be1d34e97b160ee2af2d260b6d
unreachable commit 17783a7948f064198be834dfcce16644cba85192
unreachable commit 73f4877fe6ae70fc95aacb4cd751411a277c5f0e
$ git -C ../asha/support-bot log --oneline -2 docs/runbook
73f4877 Add paging section
17783a7 Start the runbook
```

```
# Asking the server for the overwritten commit by its object ID:
$ git fetch origin 73f4877
fatal: couldn't find remote ref 73f4877
[exit status: 128]
$ git -c protocol.version=0 fetch origin 73f4877fe6ae70fc95aacb4cd751411a277c5f0e
error: Server does not allow request for unadvertised object 73f4877fe6ae70fc95aacb4cd751411a277c5f0e
[exit status: 1]
$ git fetch origin 73f4877fe6ae70fc95aacb4cd751411a277c5f0e
From .././server/support-bot
 * branch                73f4877fe6ae70fc95aacb4cd751411a277c5f0e -> FETCH_HEAD
$ git log --oneline -2 FETCH_HEAD
73f4877 Add paging section
17783a7 Start the runbook
```

The commit survives in Asha's clone and, unreachable, in the server's object database; the other two unreachable commits are your own earlier versions, replaced on purpose. Under protocol version 2, the default, a plain Git server hands out any object it has when asked by full ID; an abbreviation is not accepted, and under protocol version 0 the request is refused. This is a recovery route for as long as the object exists. It is also one reason why a forced push does not remove a leaked secret from a server.

~~--force-with-lease~~ : **force only if the server is where I think it is**. The idea is the compare-and-swap of optimistic locking: "set the branch to my commit only if it still has the value I last read". The analogy breaks at "I": your Git does the reading, and a fetch that you never looked at refreshes the value.

You squash two published commits of `feature/prompt-cache` into one, after marking the old tip with a branch:

```
$ git log --oneline --decorate -3
12cdd3d (HEAD -> feature/prompt-cache, origin/feature/prompt-cache) wip: ttl
bdbe39f Add prompt cache
510ee94 (origin/main, origin/HEAD, main) Add retrieval config
$ git branch backup/prompt-cache
$ git reset --soft HEAD~2
$ git commit -q -m "Add prompt cache with TTL"
$ git log --oneline --graph --decorate --all -4
* 1a1fa49 (HEAD -> feature/prompt-cache) Add prompt cache with TTL
| * 12cdd3d (origin/feature/prompt-cache, backup/prompt-cache) wip: ttl
| * bdb39f Add prompt cache
|/
* 510ee94 (origin/main, origin/HEAD, main) Add retrieval config
$ git status -sb
## feature/prompt-cache...origin/feature/prompt-cache [ahead 1, behind 2]
```

Meanwhile Asha pushes a test commit on top of the old history. You try to publish:

```
# Asha, in her clone: git push (her test commit lands on top of "wip: ttl")
$ git push --force-with-lease
To .././server/support-bot.git
 ! [rejected]        feature/prompt-cache -> feature/prompt-cache (stale info)
error: failed to push some refs to '.././server/support-bot.git'
[exit status: 1]
$ git rev-parse --short origin/feature/prompt-cache
12cdd3d
$ git ls-remote origin feature/prompt-cache
833bc8ad64cffe287b9e4280a0c93cb6537d625b      refs/heads/feature/prompt-cache
```

The bare form means "overwrite only if the server's branch still equals my remote-tracking ref". Your `origin/feature/prompt-cache` says `12cdd3d`, the server has `833bc8a`: (stale info), decided by your Git from the advertisement. Asha's commit is safe. Now something fetches without your attention: an editor's auto-fetch, a scheduled job, or you in another terminal.

```
# What an editor or a scheduled job does behind your back:
$ git fetch
From .././server/support-bot
 12cdd3d..833bc8a feature/prompt-cache -> origin/feature/prompt-cache
$ git status -sb
## feature/prompt-cache...origin/feature/prompt-cache [ahead 1, behind 3]
```

Your remote-tracking ref now equals the server. You have not looked at Asha's commit, but the lease cannot know that. Two stronger forms still refuse:

```
$ git push --force-with-lease --force-if-includes
To .././server/support-bot.git
 ! [rejected]        feature/prompt-cache -> feature/prompt-cache (remote ref updated since checkout)
error: failed to push some refs to '.././server/support-bot.git'
hint: Updates were rejected because the tip of the remote-tracking branch has
hint: been updated since the last checkout. If you want to integrate the
hint: remote changes, use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
[exit status: 1]
$ git push --force-with-lease=feature/prompt-cache:backup/prompt-cache
To .././server/support-bot.git
 ! [rejected]        feature/prompt-cache -> feature/prompt-cache (stale info)
error: failed to push some refs to '.././server/support-bot.git'
[exit status: 1]
```

`--force-if=includes` adds a second condition: the tip of the remote-tracking ref must be reachable from some entry in the reflog of your local branch, meaning your branch contained that commit at some time. It did not: "remote ref updated since checkout". The explicit form `--force-with=lease=<ref>:<expect>` ignores the remote-tracking ref and compares the server with a commit you name, here the backup branch at the commit you rewrote. The bare form has no such defense:

```
$ git push --force-with=lease
To ../../server/support-bot.git
+ 833bc8a...1a1fa49 feature/prompt-cache -> feature/prompt-cache (forced update)
[exit status: 0]
$ git ls-remote origin feature/prompt-cache
1a1fa4964ca5cb5de17112e84ab62441aeac53fd      refs/heads/feature/prompt-cache
```

Observed behavior : git push --force-with=lease overwrote a teammate's commit that I had never looked at.
Git state : refs/remotes/origin/feature/prompt-cache had been moved to the teammate's commit by a fetch; my local branch did not contain that commit.
Mechanism : the lease compares the server's ref with my remote-tracking ref. After the fetch they were equal, so the condition "nothing changed since I looked" was true.
Root cause : a remote-tracking ref records what my clone has fetched, not what I have seen or integrated. Any fetch renews the lease.
Why Git does this : the remote-tracking ref is the only record Git has of "the state I based my work on", and the manual says so and calls the forms without an explicit value experimental.
Correct fix : restore the overwritten commit (below); nothing can be un-pushed.
Prevention : --force-with=lease together with --force-if=includes (or push.useForceIfIncludes=true), or the explicit --force-with=lease=<ref>:<commit>; and never rewrite a branch that someone else pushes to without telling them.

```
$ git reflog show origin/feature/prompt-cache
1a1fa49 refs/remotes/origin/feature/prompt-cache@{0}: update by push
833bc8a refs/remotes/origin/feature/prompt-cache@{1}: fetch: fast-forward
12cdd3d refs/remotes/origin/feature/prompt-cache@{2}: update by push
$ ls ../../server/support-bot.git/logs
ls: ../../server/support-bot.git/logs: No such file or directory
[exit status: 1]
$ git -C ../../server/support-bot.git fsck --unreachable --no-reflogs | grep commit
unreachable commit 833bc8ad64cffe287b9e4280a0c93cb6537d625b
unreachable commit 12cdd3d76a43de79f4cbd643a62ead3eb6d44dec
unreachable commit bdb639fab454ec59484b2f16d05ff952de9cd6e6
```

This time your clone does hold the overwritten commit: the fetch recorded it, and `origin/feature/prompt-cache@{1}` names it. The server has no `logs` directory at all, only unreachable objects.

The other forms. The next two transcripts come from the same demo as the plain forced push at the start of this section and were recorded before it, while `docs/runbook` on the server still held Asha's commit.

```
# Both branches were rewrote locally. Asha has pushed to docs/runbook; you have not fetched.
$ git branch -vv
docs/runbook          ca3cabb [origin/docs/runbook: ahead 1, behind 1] Start the on-call runbook
* feature/prompt-cache 886feab [origin/feature/prompt-cache: ahead 1, behind 1] Add prompt cache module
main                  510ee94 [origin/main] Add retrieval config
$ git config set advice.pushUpdateRejected false
$ git push --force-with=lease=feature/prompt-cache origin feature/prompt-cache docs/runbook
To ../../server/support-bot.git
+ 45de293...886feab feature/prompt-cache -> feature/prompt-cache (forced update)
! [rejected] docs/runbook -> docs/runbook (fetch first)
error: failed to push some refs to ' ../../server/support-bot.git'
[exit status: 1]
$ git ls-remote --branches origin
73f4877fe6ae70fc95aacb4cd751411a277c5f0e      refs/heads/docs/runbook
886feab6db3f48e44fe9bb9310d582ba8d094014      refs/heads/feature/prompt-cache
510ee948fb6354959cf03862f0ebe47c58eb2a74      refs/heads/main
```

`--force-with=lease=<ref>` leases and forces one ref. Every other ref in the same push follows the normal rules, so `docs/runbook` was rejected. The push was also not atomic: one ref was overwritten and one refused.

```
$ git push --force-with=lease=release/1.0: origin main:refs/heads/release/1.0
To ../../server/support-bot.git
* [new branch] main -> release/1.0
$ git push --force-with=lease=release/1.0: origin feature/prompt-cache:refs/heads/release/1.0
To ../../server/support-bot.git
! [rejected] feature/prompt-cache -> release/1.0 (stale info)
error: failed to push some refs to ' ../../server/support-bot.git'
[exit status: 1]
```

An empty expected value means "the ref must not exist yet": a create-only push. Without the lease, the second command would have been a legal fast-forward of `release/1.0`.

Form	The server's ref must equal	Defeated by a background fetch
<code>--force, -f, +<refspec></code>	nothing is checked	not applicable
<code>--force-with=lease</code>	your remote-tracking ref, for every ref pushed	yes

Form	The server's ref must equal	Defeated by a background fetch
<code>--force-with-lease=<ref></code>	your remote-tracking ref, for that ref	yes
<code>--force-with-lease=<ref>:<expect></code>	the commit you name; empty means "absent"	no
<code>--force-with-lease[=<ref>]</code> plus <code>--force-if-includes</code>	your remote-tracking ref, whose tip must also have been in your branch at some time	no

What the teammate sees.

```
$ cd ../../asha/support-bot
$ git status -sb
## feature/prompt-cache...origin/feature/prompt-cache
$ git fetch
From ../../server/support-bot
+ 833bc8a...1a1fa49 feature/prompt-cache -> origin/feature/prompt-cache (forced update)
$ git status -sb
## feature/prompt-cache...origin/feature/prompt-cache [ahead 3, behind 1]
$ git log --oneline --graph --decorate --all -5
* 1a1fa49 (origin/feature/prompt-cache) Add prompt cache with TTL
| * 833bc8a (HEAD -> feature/prompt-cache) Add cache test
| * 12cdd3d wip: ttl
| * bdb39f Add prompt cache
|/
* 510ee94 (origin/main, origin/HEAD, main) Add retrieval config
```

+ 833bc8a...1a1fa49 ... (forced update) in fetch output is the signal that a branch was rewritten under you. Asha is [ahead 3, behind 1]. The reflex is `git pull --rebase`:

```
$ git reflog show origin/feature/prompt-cache
1a1fa49 refs/remotes/origin/feature/prompt-cache@{0}: fetch: forced-update
833bc8a refs/remotes/origin/feature/prompt-cache@{1}: update by push
$ git merge-base --fork-point origin/feature/prompt-cache
833bc8ad64cffe287b9e4280a0c93cb6537d625b
$ git pull --rebase
Successfully rebased and updated refs/heads/feature/prompt-cache.
$ git log --oneline --decorate -3
1a1fa49 (HEAD -> feature/prompt-cache, origin/feature/prompt-cache) Add prompt cache with TTL
510ee94 (origin/main, origin/HEAD, main) Add retrieval config
95671d3 Add retriever skeleton
```

"Successfully rebased", no conflict, and her commit `Add cache test` is no longer on the branch.

Observed behavior : after a teammate's forced push, `git pull --rebase` succeeded and my commit vanished from the branch.

Git state : I had pushed that commit earlier, so the reflog of `origin/feature/prompt-cache` contains it ("update by push"). The server's branch was then replaced.

Mechanism : `pull --rebase` replays only the commits after the fork point, and the fork point is found through the reflog of the remote-tracking ref (`git merge-base --fork-point`). It was my own pushed commit, so nothing was left to replay.

Root cause : Git cannot tell "the upstream removed this commit on purpose" from "someone overwrote it by accident". The reflog says only: it was upstream once, it is not now.

Why Git does this : without the rule, everyone downstream of a deliberately rebased branch would re-apply the commits that the upstream had dropped.

Correct fix : the commit is in my reflog; cherry-pick it onto the new tip and push.

Prevention : after any "(forced update)" line, look before integrating:
`git log --oneline --graph HEAD @{u}`. Do not pull on a branch that others rewrite.

```
$ git reflog -4
1a1fa49 HEAD@{0}: pull --rebase (finish): returning to refs/heads/feature/prompt-cache
1a1fa49 HEAD@{1}: pull --rebase (start): checkout 1a1fa4964ca5cb5de17112e84ab62441aeac53fd
833bc8a HEAD@{2}: commit: Add cache test
12cdd3d HEAD@{3}: checkout: moving from main to feature/prompt-cache
$ git cherry-pick HEAD@{2}
[feature/prompt-cache 9074b47] Add cache test
Date: Mon Sep 7 10:15:00 2026 +0530
1 file changed, 2 insertions(+)
create mode 100644 tests/test_cache.py
$ git push
To ../../server/support-bot.git
1a1fa49..9074b47 feature/prompt-cache -> feature/prompt-cache
$ git log --oneline --decorate -3
9074b47 (HEAD -> feature/prompt-cache, origin/feature/prompt-cache) Add cache test
1a1fa49 Add prompt cache with TTL
510ee94 (origin/main, origin/HEAD, main) Add retrieval config
```

State table for a forced push that succeeds:

Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
unchanged	unchanged	unchanged	unchanged	remote-tracking ref moved; its reflog holds the old tip only if a	ref moved to a non-descendant; old commits unreachable; no reflog on a	pull requests built on the old commits are disturbed; rules

Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
				fetch recorded it	default bare server	may refuse the push

GitHub, not Git. "Block force pushes" is enabled by default in a new ruleset, and classic branch protection rules disable force pushes by default ([available rules](#)). GitHub offers no relog that Git can query, but its Activity view lists force pushes and offers a comparison for each entry ([Activity view](#)), and its troubleshooting page gives the plain-Git answer: ask a collaborator who still has the commit to push it to a new branch ([Troubleshooting commits](#)). Chapter 30 drills this.

In production. A routine for rewriting a branch that is already published: fetch and read `git status -sb`; mark the old tip (`git branch backup/<name>`); rewrite; push with `--force-with-lease=<name>:backup/<name>`; tell everyone who has the branch what to do next. Set `push.useForceIfIncludes=true` once, for the days you forgo the explicit form.

Outdated advice. Tutorials that teach "git reset --hard, then git push -f" as a routine undo, even on `main`, teach the one habit in this chapter that loses other people's work. Undo published commits with `git revert` (Chapter 11).

12.9 Pushing into a repository that has a working tree

A push moves refs. It checks nothing out. If the pushed ref is the branch that the receiving repository has checked out, that repository's `HEAD` would name a commit that its index and working tree do not match, and its next `git status` would present the pushed changes as local edits that undo them. Git refuses by default:

```
$ git remote add staging ../../staging-box
$ git push staging main
remote: error: refusing to update checked out branch: refs/heads/main
remote: error: By default, updating the current branch in a non-bare repository
remote: is denied, because it will make the index and work tree inconsistent
remote: with what you pushed, and will require 'git reset --hard' to match
remote: the work tree to HEAD.
remote:
remote: You can set the 'receive.denyCurrentBranch' configuration variable
remote: to 'ignore' or 'warn' in the remote repository to allow pushing into
remote: its current branch; however, this is not recommended unless you
remote: arranged to update its work tree to match what you pushed in some
remote: other way.
remote:
remote: To squelch this message and still keep the default behaviour, set
remote: 'receive.denyCurrentBranch' configuration variable to 'refuse'.
To ../../staging-box
 ! [remote rejected] main -> main (branch is currently checked out)
error: failed to push some refs to ' ../../staging-box'
[exit status: 1]
```

`receive.denyCurrentBranch` defaults to `refuse`. Any branch that is not checked out can be pushed to:

```
$ git push staging main:refs/heads/incoming/timeout
To ../../staging-box
 * [new branch]      main -> incoming/timeout
$ git -C ../../staging-box branch -vv
incoming/timeout c19ab53 Set request timeout
* main          510ee94 [origin/main] Add retrieval config
```

For the legitimate case, a test or deployment machine that you reach over SSH, the receiving side can opt in to `updateInstead`, which updates the working tree as part of the push:

```
$ git -C ../../staging-box config set receive.denyCurrentBranch updateInstead
$ git push staging main
To ../../staging-box
 510ee94..c19ab53 main -> main
$ git -C ../../staging-box log --oneline -1
c19ab53 Set request timeout
$ ls ../../staging-box/app
retriever.py
settings.py
```

```
$ printf 'hotfix typed directly on the box\n' >> ../../staging-box/README.md
$ git push staging main
To ../../staging-box
 ! [remote rejected] main -> main (Working directory has unstaged changes)
error: failed to push some refs to ' ../../staging-box'
[exit status: 1]
```

It works only while the target is clean. One uncommitted edit on the box makes every later push fail with `Working directory has unstaged changes`, which is the correct outcome: somebody hot-fixed on the machine, and the push tells you before it overwrites anything. The general

rule stands: a repository that people push to is bare.

12.10 More than one remote

In one sentence. A repository can have any number of remotes, each with its own URL, refsspecs and namespace of remote-tracking refs; nothing connects them except the objects they share.

See it: a fork, in plain Git. By convention `origin` is the repository you push to and `upstream` is the shared repository you take changes from:

```
# The Fork button, in plain Git: a server-side clone.
$ git clone --bare server/support-bot.git forks/you/support-bot.git
Cloning into bare repository 'forks/you/support-bot.git'...
done.
$ git clone forks/you/support-bot.git you/support-bot
Cloning into 'you/support-bot'...
done.
$ cd you/support-bot
$ git remote add upstream ../../server/support-bot.git
$ git fetch upstream
From ../../server/support-bot
 * [new branch]      main      -> upstream/main
$ git remote -v
origin  ../../forks/you/support-bot.git (fetch)
origin  ../../forks/you/support-bot.git (push)
upstream    ../../server/support-bot.git (fetch)
upstream    ../../server/support-bot.git (push)
```

```
$ git show-ref --abbrev
510ee94 refs/heads/main
510ee94 refs/remotes/origin/HEAD
510ee94 refs/remotes/origin/main
510ee94 refs/remotes/upstream/HEAD
510ee94 refs/remotes/upstream/main
```

Two namespaces, `origin/*` and `upstream/*`, each with its own `HEAD`, all naming the same commit, which is stored once.

GitHub, not Git. Git has no notion of a fork. The Fork button creates a GitHub object: a server-side copy that stays connected to its parent and shares Git data with it in a repository network ([Forks](#)). To Git it is one more repository with a URL. `gh repo fork --clone` names your fork `origin` and the parent `upstream`, the convention used here.

A triangular workflow fetches from one repository and pushes to another:

```
$ git config set remote.pushDefault origin
$ git config set push.default current
$ git switch -c feature/streaming upstream/main
Switched to a new branch 'feature/streaming'
branch 'feature/streaming' set up to track 'upstream/main'.
# One commit made here: "Add streaming responses".
$ git push
To ../../forks/you/support-bot.git
 * [new branch]      feature/streaming -> feature/streaming
$ git rev-parse --abbrev-ref @{upstream} @push
upstream/main
origin/feature/streaming
$ git config get --all --show-names --regexp "(remote\.pushdefault|push\.default|branch\.feature)"
remote.pushdefault origin
push.default current
branch.feature/streaming.remote upstream
branch.feature/streaming.merge refs/heads/main
```

`remote.pushDefault=origin` sends every bare `git push` to your fork, whatever the upstream of the branch is. The branch tracks `upstream/main`, so `git pull` takes changes from the shared repository. `@{upstream}` and `@{push}` are now two different refs.

```
upstream = server/support-bot.git      shared; you fetch from it
|
| git fetch upstream, git pull --rebase    (@{upstream} = upstream/main)
v
you/support-bot      feature/streaming
|
| git push                                (@{push} = origin/feature/streaming)
v
origin = forks/you/support-bot.git      yours; a pull request offers this branch to upstream
```

Which remote a bare push uses is decided by three settings in a fixed order, and the branch name by `push.default`:

```
$ git branch -vv
* feature/streaming 08b274c [upstream/main: ahead 1] Add streaming responses
  main              510ee94 [origin/main] Add retrieval config
$ git config set remote.pushDefault origin
```

```
$ git push --dry-run
To ../../forks/you/support-bot.git
* [new branch]      feature/streaming -> feature/streaming
$ git rev-parse --abbrev-ref @{push}
fatal: cannot resolve 'simple' push to a single destination
[exit status: 128]
```

With only `remote.pushDefault` set, the push goes to the fork under the branch's own name, but Git 2.55 cannot answer `@{push}` under `push.default=simple`.

```
$ git config set push.default current
$ git push
To ../../forks/you/support-bot.git
* [new branch]      feature/streaming -> feature/streaming
$ git rev-parse --abbrev-ref @{upstream} @{push}
upstream/main
origin/feature/streaming
$ git for-each-ref --format=%(refname:short): pull from %(upstream:short), push to %(push:short)' refs/heads
feature/streaming: pull from upstream/main, push to origin/feature/streaming
main: pull from origin/main, push to origin/main
```

`push.default=current` makes the destination a plain function of the branch name. The `for-each-ref` line is the one to remember in an incident: for every branch, where it pulls from and where it pushes to.

```
$ git config set branch.feature/streaming.pushRemote upstream
$ git push --dry-run
To ../../server/support-bot.git
* [new branch]      feature/streaming -> feature/streaming
$ git config unset branch.feature/streaming.pushRemote
$ git push --dry-run
Everything up-to-date
```

The order is `branch.<name>.pushRemote`, then `remote.pushDefault`, then `branch.<name>.remote`, then `origin`.

```
$ git config set status.compareBranches "@{upstream} @{push}"
$ git fetch upstream
From ../../server/support-bot
 510ee94..f11a751 main    -> upstream/main
$ git status
On branch feature/streaming
Your branch and 'upstream/main' have diverged,
and have 1 and 1 different commits each, respectively.
(use "git pull" if you want to integrate the remote branch with yours)

Your branch is up to date with 'origin/feature/streaming'.

nothing to commit, working tree clean
```

`status.compareBranches` (Git 2.54 or later) makes `git status` report both relationships.

```
$ git pull --rebase
Rebasing (1/1)
Successfully rebased and updated refs/heads/feature/streaming.
$ git status -sb
## feature/streaming...upstream/main [ahead 1]
$ git push
To ../../forks/you/support-bot.git
! [rejected]      feature/streaming -> feature/streaming (non-fast-forward)
error: failed to push some refs to ' ../../forks/you/support-bot.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
[exit status: 1]
$ git push --force-with-lease
To ../../forks/you/support-bot.git
+ 5d5414c...4ce24f2 feature/streaming -> feature/streaming (forced update)
```

After a rebase onto the moved upstream, the branch in your fork still holds the old commit, so the push is not a fast-forward. This is the legitimate forced push: a branch that only you push to, with a lease.

```
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
$ git merge --ff-only upstream/main
Updating 510ee94..f11a751
Fast-forward
 config.yaml | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git push
To ../../forks/you/support-bot.git
 510ee94..f11a751 main -> main
$ git log --oneline --graph --decorate --all -4
* 4ce24f2 (origin/feature/streaming, feature/streaming) Add streaming responses
```

```
* f11a751 (HEAD -> main, upstream/main, upstream/HEAD, origin/main, origin/HEAD) Raise top_k to 8
* 510ee94 Add retrieval config
* 95671d3 Add retriever skeleton
```

Bringing the fork's `main` up to date is a fast-forward from `upstream/main` and a push to `origin`. GitHub documents the same steps for the command line, next to its "Sync fork" button and `gh repo sync` ([Syncing a fork](#)).

Fetch URL, push URL and other URL tools.

```
$ git remote add upstream ../../server/support-bot.git
$ git remote set-url --push upstream DISABLED
$ git remote -v
origin    ../../server/support-bot.git (fetch)
origin    ../../server/support-bot.git (push)
upstream  ../../server/support-bot.git (fetch)
upstream  DISABLED (push)
$ git config get --all --show-names --regexp "^remote\.upstream\."
remote.upstream.url ../../server/support-bot.git
remote.upstream.fetch +refs/heads/*:refs/remotes/upstream/*
remote.upstream.pushurl DISABLED
$ git push upstream main
fatal: 'DISABLED' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
[exit status: 128]
```

`git remote set-url --push` gives a remote a separate push URL. Here it is deliberately not a repository, a cheap guard that makes pushing to `upstream` impossible from this clone. The manual warns that a real fetch URL and push URL of one remote must lead to the same repository; for two different places, use two remotes ([git-remote](#)).

```
$ git remote set-url --add --push origin ../../server/support-bot.git
$ git remote set-url --add --push origin ../../backup/support-bot.git
$ git remote -v
origin    ../../server/support-bot.git (fetch)
origin    ../../server/support-bot.git (push)
origin    ../../backup/support-bot.git (push)
upstream  ../../server/support-bot.git (fetch)
upstream  DISABLED (push)
$ git push origin main
To ../../server/support-bot.git
 510ee94..c19ab53  main -> main
To ../../backup/support-bot.git
 510ee94..c19ab53  main -> main
```

Several push URLs make one push update several repositories, one after the other and not atomically.

```
$ git config set url. ../../server/.insteadOf https://git.example.com/acme/
$ git remote add company https://git.example.com/acme/support-bot.git
$ git config get remote.company.url
https://git.example.com/acme/support-bot.git
$ git remote get-url company
../../server/support-bot.git
$ git ls-remote company main
c19ab53ef3dba868f5f8b8c60d19a9687069f366      refs/heads/main
```

`url.<base>.insteadOf` rewrites URLs when they are used. The configuration keeps the URL that was written; `git remote get-url` shows what will be contacted. Teams use it in the global configuration to move a whole host from HTTPS to SSH, or to an internal mirror, without editing any clone.

```
$ git remote add backup ../../backup/support-bot.git
$ git config set remotes.offsite "company backup"
$ git fetch offsite
Fetching company
From ../../server/support-bot
 * [new branch]      main    -> company/main
Fetching backup
From ../../backup/support-bot
 * [new branch]      main    -> backup/main
$ git push --dry-run offsite main
Pushing to company
Everything up-to-date
Pushing to backup
Everything up-to-date
```

`remotes.<group>` names a list of remotes. `git fetch <group>` has long accepted one; `git push <group>` does since Git 2.55.

```
$ git branch -u company/main
branch 'main' set up to track 'company/main'.
$ git remote rename company canonical
$ git branch -r
backup/HEAD -> backup/main
```

```

backup/main
canonical/HEAD -> canonical/main
canonical/main
origin/HEAD -> origin/main
origin/main
$ git config get --all --show-names --regexp ""^(remote\.canonical|branch\.main)\.
branch.main.remote canonical
branch.main.merge refs/heads/main
remote.canonical.url https://git.example.com/acme/support-bot.git
remote.canonical.fetch +refs/heads/*:refs/remotes/canonical/*

```

```

$ git remote remove canonical
$ git branch -r
backup/HEAD -> backup/main
backup/main
origin/HEAD -> origin/main
origin/main
$ git branch -vv
* main c19ab53 Set request timeout
$ git remote remove canonical
error: No such remote: 'canonical'
[exit status: 2]

```

🟡 `git remote rename` moves the remote-tracking refs and rewrites the configuration, including the upstream of `main`. 🟡 `git remote remove` deletes the remote-tracking refs with their reflogs and removes upstream settings that pointed at the remote: `main` is left with none.

In production. You contribute a fix to an open-source evaluation library: fork, clone, add `upstream`, branch from `upstream/main`, push to `origin`, open the pull request. Inside a company the same triangle appears when a team keeps a patched copy of a vendor repository. In both cases write down which remote is which before the first push.

12.11 Pruning and branches whose upstream is gone

In one sentence. A fetch creates and moves remote-tracking refs but by default never deletes one, so branches deleted on the server live on in your clone until you prune.

See it. This transcript runs in Asha's clone. Since she last fetched, `feature/reranker` was merged and deleted on the server, and the team replaced the branch `release` by `release/1.0`:

```

$ git branch -r
origin/HEAD -> origin/main
origin/feature/eval-harness
origin/feature/reranker
origin/main
origin/release
$ git ls-remote --branches origin
edc57ed67d3ef2451ee0ad599a1769c54adc92c5      refs/heads/feature/eval-harness
52d3c1ada36809341bc3fac2af09a619cf85ec00      refs/heads/main
52d3c1ada36809341bc3fac2af09a619cf85ec00      refs/heads/release/1.0

```

```

$ git remote show origin
* remote origin
Fetch URL: ../../server/support-bot.git
Push URL: ../../server/support-bot.git
HEAD branch: main
Remote branches:
feature/eval-harness      tracked
main                      tracked
refs/remotes/origin/feature/reranker  stale (use 'git remote prune' to remove)
refs/remotes/origin/release    stale (use 'git remote prune' to remove)
release/1.0                new (next fetch will store in remotes/origin)
Local branches configured for 'git pull':
feature/eval-harness merges with remote feature/eval-harness
feature/reranker      merges with remote feature/reranker
main                  merges with remote main
Local refs configured for 'git push':
feature/eval-harness pushes to feature/eval-harness (up to date)
main                pushes to main (local out of date)

```

`git branch -r` reads the stale local view and `git ls-remote` the server. `git remote show origin` contacts the server and classifies every branch as tracked, stale or new; it also lists what each local branch pulls from and pushes to.

```

$ git fetch
error: some local refs could not be updated; try running
'git remote prune origin' to remove any old, conflicting branches
From ../../server/support-bot
510ee94..52d3c1a  main      -> origin/main
! [new branch]    release/1.0 -> origin/release/1.0 (unable to update local ref)
[exit status: 1]

```

A stale ref can block a fetch. `origin/release` still exists, so `origin/release/1.0` cannot be created: one ref name cannot be a directory-like prefix of another, in any ref storage format.

```
$ git remote prune --dry-run origin
Pruning origin
URL: ../../server/support-bot.git
* [would prune] origin/feature/reranker
* [would prune] origin/release
$ git fetch --prune
From ../../server/support-bot
- [deleted]      (none)      -> origin/feature/reranker
- [deleted]      (none)      -> origin/release
* [new branch]   release/1.0 -> origin/release/1.0
$ git branch -r
origin/HEAD -> origin/main
origin/feature/eval-harness
origin/main
origin/release/1.0
```

`git remote prune --dry-run` previews.  `git fetch --prune` deletes remote-tracking refs that no longer exist on the server and then fetches.

```
$ git branch -vv
feature/eval-harness edc57ed [origin/feature/eval-harness] Add eval harness entry point
feature/reranker      5176652 [origin/feature/reranker: gone] Add reranker stub
* main                510ee94 [origin/main: behind 2] Add retrieval config
$ git for-each-ref --format="%{refname:short} %{upstream:track}" refs/heads
feature/eval-harness
feature/reranker [gone]
main [behind 2]
```

Pruning never touches local branches. Their upstream is now reported as `gone`, and `%{upstream:track}` gives the same fact to scripts. A branch in that state hides a trap:

```
$ git switch feature/reranker
Switched to branch 'feature/reranker'
Your branch is based on 'origin/feature/reranker', but the upstream is gone.
(use "git branch --unset-upstream" to fixup)
$ git pull
Your configuration specifies to merge with the ref 'refs/heads/feature/reranker'
from the remote, but no such ref was fetched.
[exit status: 1]
$ git push
To ../../server/support-bot.git
* [new branch]   feature/reranker -> feature/reranker
$ git ls-remote --branches origin
edc57ed67d3ef2451ee0ad599a1769c54adc92c5      refs/heads/feature/eval-harness
5176652525c1216c70b16c80ecb9625c8f350595      refs/heads/feature/reranker
52d3c1ada36809341bc3fac2af09a619cf85ec00      refs/heads/main
52d3c1ada36809341bc3fac2af09a619cf85ec00      refs/heads/release/1.0
```

`git pull` fails with a clear message. `git push` succeeds and re-creates the branch on the server: a branch that the team merged and deleted is back, with old commits, and somebody will build on it.

```
$ git push origin --delete feature/reranker
To ../../server/support-bot.git
- [deleted]      feature/reranker
$ git switch main
Switched to branch 'main'
Your branch is behind 'origin/main' by 2 commits, and can be fast-forwarded.
(use "git pull" to update your local branch)
$ git pull --ff-only
Updating 510ee94..52d3c1a
Fast-forward
 app/reranker.py | 2 ++
 1 file changed, 2 insertions(+)
 create mode 100644 app/reranker.py
$ git branch -d feature/reranker
Deleted branch feature/reranker (was 5176652).
```

Delete it on the server again, bring `main` up to date, and delete the local branch with `-d`, which refuses if the commits are not merged.

```
$ git config set fetch.prune true
# You have deleted feature/eval-harness on the server in the meantime.
$ git fetch
From ../../server/support-bot
- [deleted]      (none)      -> origin/feature/eval-harness
$ git branch -vv
feature/eval-harness edc57ed [origin/feature/eval-harness: gone] Add eval harness entry point
* main            52d3c1a [origin/main] Merge branch feature/reranker
```

With `fetch.prune=true` every fetch prunes. Set globally, it keeps clones free of the stale `origin/*` refs that deleted feature branches leave behind. Section 12.16 shows the price.

State table for `git remote prune <remote>; git fetch --prune` adds the effects of a fetch:

Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
unchanged	unchanged	unchanged	unchanged	stale remote-tracking refs and their reflogs deleted; upstream settings of local branches kept	unchanged	unchanged

12.12 Refspecs in depth

In one sentence. A refspec, `[+]<source>:<destination>`, is the only thing that decides which refs a fetch or a push touches: the source is on the sending side, the destination on the receiving side.

Analogy. A mail-forwarding rule: "whatever is addressed to `refs/heads/X` over there, file it here as `refs/remotes/origin/X`". It breaks because the rule runs only at the moment of a fetch or a push; nothing is forwarded in between.

Refspec	Where	Meaning
<code>+refs/heads/*:refs/remotes/origin/*</code>	fetch, configured	every branch to a remote-tracking ref, forced
<code>+refs/heads/main:refs/remotes/origin/main</code>	fetch, configured	one branch only
<code>^refs/heads/dependabot/*</code>	fetch or push	negative: exclude what matches (Git 2.29 or later)
<code>+refs/pull/*/head:refs/remotes/origin/pr/*</code>	fetch, configured	map another namespace
<code>pull/7/head:pr-7</code>	fetch, command line	one ref into a local branch
<code>main</code>	push	<code>refs/heads/main:refs/heads/main</code>
<code>HEAD~1:main</code>	push	any commit of yours to a branch of theirs
<code>HEAD:refs/heads/review/x</code>	push	create a branch under another name
<code>:review/x</code>	push	delete
<code>+main</code>	push	force this ref only

See it. A clone made with `--single-branch`, as a CI job makes it:

```
$ git clone --single-branch server/support-bot.git you/support-bot
Cloning into 'you/support-bot'...
done.
$ cd you/support-bot
$ git config get --all remote.origin.fetch
+refs/heads/main:refs/remotes/origin/main
$ git branch -r
  origin/HEAD -> origin/main
  origin/main
$ git switch release/0.1
fatal: invalid reference: release/0.1
[exit status: 128]
```

```
$ git remote set-branches --add origin release/0.1
$ git config get --all remote.origin.fetch
+refs/heads/main:refs/remotes/origin/main
+refs/heads/release/0.1:refs/remotes/origin/release/0.1
$ git fetch
From ../server/support-bot
 * [new branch]      release/0.1 -> origin/release/0.1
$ git remote set-branches origin "*"
$ git config get --all remote.origin.fetch
+refs/heads/*:refs/remotes/origin/*
$ git fetch
From ../server/support-bot
 * [new branch]      dependabot/pip/requests-2.33 -> origin/dependabot/pip/requests-2.33
```

`git remote set-branches --add` appends a refspec for one more branch, and `set-branches origin "*"` restores the wildcard. Each `git fetch` then brings exactly what the configured refspecs cover.

```
# Asha has rewritten the tip of release/0.1 on the server. Your refspec, without its ++:
$ git config set remote.origin.fetch "refs/heads/*:refs/remotes/origin/*"
$ git fetch
From ../server/support-bot
 ! [rejected] release/0.1 -> origin/release/0.1 (non-fast-forward)
[exit status: 1]
$ git config set remote.origin.fetch "+refs/heads/*:refs/remotes/origin/*"
$ git fetch
From ../server/support-bot
 + d5379f1...6c2ddc6 release/0.1 -> origin/release/0.1 (forced update)
```

The meaning of `+`. Asha rewrote the tip of `release/0.1`. Without the plus sign, your fetch refuses the non-fast-forward update and your picture of the server stays wrong. With it, the remote-tracking ref is forced. The default refspec carries the `+` on purpose: a remote-tracking ref has to record the server as it is, rewritten or not. (The `git fetch` manual of 2.55 says updates outside `refs/heads/` and `refs/tags/` are accepted without `+`. This transcript shows that Git 2.55.0 rejects a non-fast-forward update of `refs/remotes/origin/release/0.1` without it.)

```
$ git config set --append remote.origin.fetch "^refs/heads/dependabot/*"
$ git branch -r -d origin/dependabot/pip/requests-2.33
Deleted remote-tracking branch origin/dependabot/pip/requests-2.33 (was 510ee94).
$ git fetch
$ git branch -r
origin/HEAD -> origin/main
origin/main
origin/release/0.1
```

A negative refspec excludes matching refs from later fetches. It does not delete a ref you already have, hence the `git branch -r -d`.

```
$ git ls-remote origin "refs/pull/*"
a3b0283892821a8f9fc95bda5760237390817915      refs/pull/7/head
$ git fetch origin pull/7/head:pr-7
From ../server/support-bot
 * [new ref]          refs/pull/7/head -> pr-7
$ git config set --append remote.origin.fetch "+refs/pull/*/head:refs/remotes/origin/pr/*"
$ git fetch
From ../server/support-bot
 * [new ref]          refs/pull/7/head -> origin/pr/7
$ git config get --all remote.origin.fetch
+refs/heads/*:refs/remotes/origin/*
^refs/heads/dependabot/*
+refs/pull/*/head:refs/remotes/origin/pr/*
```

Refs outside `refs/heads/` are invisible to the default refspec. Name one on the command line to fetch it into a local branch, or map the namespace permanently.

GitHub, not Git. `refs/pull/<number>/head` is how GitHub exposes the head commit of every pull request. Its documentation gives `git fetch origin pull/ID/head:BRANCH_NAME` and states that the namespace is read-only: a push to it is answered with `! [remote rejected] ... (deny updating a hidden ref)` ([Checking out pull requests locally](#)). `gh pr checkout <number>` wraps the fetch.

```
# Two commits made on main: "Start the runbook", then "Add on-call notes".
$ git push origin HEAD~1:main
To ../server/support-bot.git
 510ee94..73bb3c8 HEAD~1 -> main
$ git push origin HEAD:refs/heads/review/oncall-notes
To ../server/support-bot.git
 * [new branch]      HEAD -> review/oncall-notes
$ git push origin :review/oncall-notes
To ../server/support-bot.git
- [deleted]          review/oncall-notes
$ git status -sb
## main...origin/main [ahead 1]
```

Three push refspecs: publish everything except your newest commit, create a branch under a different name, and delete it. Status then shows the one commit that stayed local.

In production. A fetch refspec is configuration that silently shapes what a clone can see. Lab 7.7 ends with a mirror-style refspec, `+refs/heads/*:refs/heads/*`, pasted into an ordinary clone: the next fetch overwrites local branches. When a clone "cannot see" a branch or "keeps losing" one, print `git config get --all remote.origin.fetch` first.

12.13 Transports in brief

The URL selects how the two Gits reach each other. What they say to each other is the same.

```
# One scheme per URL, in the order given:
$ git url-parse -c scheme https://github.com/acme/support-bot.git git@github.com:acme/support-bot.git
ssh://git@github.com/acme/support-bot.git file:///srv/git/support-bot.git
https
ssh
ssh
file
$ git url-parse -c host git@github.com:acme/support-bot.git
github.com
$ git url-parse -c path git@github.com:acme/support-bot.git
/acme/support-bot.git
$ git url-parse -c scheme ../server/support-bot.git
fatal: '../server/support-bot.git' is not a URL; if you meant a local repository, use a 'file:/' URL with an absolute path
[exit status: 128]
```

`git url-parse` (Git 2.55 or later) shows how Git reads a URL. The form `git@github.com:acme/support-bot.git` is SSH. A plain path is not a URL at all: it names a local repository.

Transport	URL form	Authentication	Notes
local	/path/repo.git, ../repo.git	filesystem permissions	<code>git clone</code> hard-links or copies object files

Transport	URL form	Authentication	Notes
file	file:///path/repo.git	filesystem permissions	always the pack protocol
SSH	ssh://git@host/path, git@host:path	SSH keys	runs Git's server programs on the host
HTTPS	https://host/path	credentials supplied by a credential helper	the same conversation carried in HTTP requests
git	git://host/path	none	anyone who can reach the port can read
bundle	a file	none	offline; fetch and clone only

```
# Which program does Git start for the other side of the conversation?
$ GIT_TRACE=1 git fetch 2>&1 | sed -n 's/.*trace: run_command: //p' | head -1
unset GIT_PREFIX; GIT_PROTOCOL=version=2 'git-upload-pack '\'.../server/support-bot.git\'\'
$ GIT_TRACE=1 git push 2>&1 | sed -n 's/.*trace: run_command: //p' | head -1
unset GIT_PREFIX; 'git-receive-pack '\'.../server/support-bot.git\'\'
```

The other side of a fetch is `git-upload-pack`; the other side of a push is `git-receive-pack`. Over SSH the same two programs are started on the server.

```
$ GIT_TRACE_PACKET=1 git fetch 2>&1 | sed -n 's/.*packet: *(fetch[<>])\/1\/p'
fetch< version 2
fetch< agent=git/2.55.0-Darwin
fetch< ls-refs=unborn
fetch< fetch=shallow wait-for-done
fetch< server-option
fetch< object-format=sha1
fetch< 0000
fetch> command=ls-refs
fetch> agent=git/2.55.0-Darwin
fetch> object-format=sha1
fetch> 0001
fetch> peel
fetch> symrefs
fetch> unborn
fetch> ref-prefix refs/heads/
fetch> ref-prefix refs/heads/main
fetch> ref-prefix refs/tags/
fetch> ref-prefix HEAD
fetch> 0000
fetch< 719650da56c2910ce851d3c0f47ec270ebb88945 HEAD symref-target:refs/heads/main
fetch< 719650da56c2910ce851d3c0f47ec270ebb88945 refs/heads/main
fetch< 0000
fetch> command=fetch
fetch> agent=git/2.55.0-Darwin
fetch> object-format=sha1
fetch> 0001
fetch> thin-pack
fetch> no-progress
fetch> include-tag
fetch> ofs-delta
fetch> want 719650da56c2910ce851d3c0f47ec270ebb88945
fetch> have 510ee948fb6354959cf03862f0ebe47c58eb2a74
fetch> have 95671d3baba80df6b53e79e6583c1a6e01fd451
fetch> have f56c1bb279a37dc704979733d4314bd9c1e2893d
fetch> 0000
fetch< acknowledgments
fetch< ACK 510ee948fb6354959cf03862f0ebe47c58eb2a74
fetch< ready
fetch< 0001
fetch< packfile
```

`GIT_TRACE_PACKET=1` prints the conversation; `fetch<` is received, `fetch>` is sent. The server announces protocol version 2 and its capabilities. The client asks for refs with `ls-refs`, limited by `ref-prefix` lines derived from your refsspecs, so a server with very many refs lists only those you can use. Then `want` names what you need and `have` what you already hold, and the server answers with a packfile of the difference.

```
# One local commit, already rebased onto the fetched origin/main:
$ GIT_TRACE_PACKET=1 git push 2>&1 | sed -n 's/.*packet: *(push[<>])\/1\/p' | cut -c1-110
push< 719650da56c2910ce851d3c0f47ec270ebb88945 refs/heads/main\0report-status report-status-v2 delete-refs sid
push< 0000
push> 719650da56c2910ce851d3c0f47ec270ebb88945 b109fc2446651be551de03e7834e6db8804e1cd1 refs/heads/main\0 repo
push> 0000
push< unpack ok
push< ok refs/heads/main
push< 0000
```

A push is shorter: the advertisement, one `<old> <new> <ref>` command, the pack, and the report `unpack ok`, `ok refs/heads/main`. These traces are the first tool when a transfer hangs or is refused without explanation (Chapter 29).

```
$ git bundle create ../../support-bot.bundle HEAD --branches
$ git bundle verify ../../support-bot.bundle
../../support-bot.bundle is okay
The bundle contains these 3 refs:
510ee948fb6354959cf03862f0ebe47c58eb2a74 HEAD
e5530217ca8545817a0931e13090569b6fe0c4 refs/heads/feature/eval-harness
```



```
510ee948fb6354959cf03862f0ebe47c58eb2a74 refs/heads/main
The bundle records a complete history.
The bundle uses this hash algorithm: sha1
```

```
$ cd ../../
$ git clone support-bot.bundle airgap/support-bot
Cloning into 'airgap/support-bot'...
$ git -C airgap/support-bot branch -a
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/feature/eval-harness
  remotes/origin/main
$ git -C airgap/support-bot remote -v
origin $LAB/ch12/transport/support-bot.bundle (fetch)
origin $LAB/ch12/transport/support-bot.bundle (push)
```

A bundle is a file that plays the part of a remote. It serves networks with no route between the two sides, such as an air-gapped training cluster: carry the file across, then clone or fetch from it. You cannot push to it.

GitHub, not Git. GitHub accepts HTTPS and SSH URLs ([About remote repositories](#)). It stopped accepting account passwords for Git over HTTPS on 13 August 2021 ([GitHub Blog](#)) and removed the unauthenticated `git://` protocol on 15 March 2022 ([GitHub Blog](#)). Chapter 16 covers authentication.

12.14 Two diagnoses from first principles

"The commit exists locally but not on the server."

```
$ git log --oneline -1
c19ab53 Set request timeout
$ git push origin main
Everything up-to-date
```

Everything up-to-date is a true statement about the refspec that was pushed. Three questions locate the commit:

```
$ git branch -a --contains HEAD
* hotfix/timeout
$ git status -sb
## hotfix/timeout
$ git log --oneline --branches --not --remotes
c19ab53 Set request timeout
```

Which branches contain it: one local branch and no remote-tracking branch. Does that branch have an upstream: `## hotfix/timeout` with nothing after it means no. Which commits has no remote-tracking ref ever seen: `git log --branches --not --remotes` lists it.

```
$ git rev-parse HEAD
c19ab53ef3dba868f5f8b8c60d19a9687069f366
$ git ls-remote origin
510ee948fb6354959cf03862f0ebe47c58eb2a74 HEAD
510ee948fb6354959cf03862f0ebe47c58eb2a74 refs/heads/main
```

And the server, asked directly, has no ref at that ID.

```
Observed behavior : git push printed "Everything up-to-date"; the deploy job cannot find the commit.
Git state          : HEAD is on hotfix/timeout, which has no upstream. The commit is on that branch only.
                    main equals origin/main.
Mechanism          : "git push origin main" is a refspec: send my main to their main. There was nothing
                    to send.
Root cause         : the command named a branch that does not hold the commit. A push transfers the
                    refs you name, not "my work".
Why Git does this  : an explicit refspec is obeyed literally; Git has no idea which commit you care about.
Correct fix        : git push -u origin hotfix/timeout, then merge it by the team's process.
Prevention         : read the last lines of push output (<source> -> <destination>); verify with
                    git branch -a --contains <commit>; let deploy scripts verify with git ls-remote.
```

```
$ git push -u origin hotfix/timeout
To ../../server/support-bot.git
 * [new branch]      hotfix/timeout -> hotfix/timeout
branch 'hotfix/timeout' set up to track 'origin/hotfix/timeout'.
$ git branch -a --contains HEAD
* hotfix/timeout
  remotes/origin/hotfix/timeout
$ git log --oneline --branches --not --remotes
```

The same three questions sort out the other causes of this symptom: a rejected push whose exit status a script ignored, a push to the wrong remote (your fork instead of the shared repository), a commit made on a detached HEAD, or a reader who looked at a stale remote-tracking ref instead of the server.

"My remote-tracking refs cannot be trusted."

```
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
$ git branch origin/main HEAD~1
$ git rev-parse --short origin/main
warning: refname 'origin/main' is ambiguous.
95671d3
$ git branch -a
hotfix/timeout
* main
origin/main
remotes/origin/HEAD -> remotes/origin/main
remotes/origin/hotfix/timeout
remotes/origin/main
$ git branch -D origin/main
Deleted branch origin/main (was 95671d3).
```

A local branch named `origin/main`, usually created by a mistyped command. Git warns that the name is ambiguous and then resolves it to the local branch, because `refs/heads/` comes before `refs/remotes/` in the lookup order. Every comparison "against `origin/main`" now uses the wrong commit. In `git branch -a` the impostor is the entry without the `remotes/` prefix.

```
$ git remote set-head origin --delete
$ git for-each-ref --format='delete %(refname)' refs/remotes/origin | git update-ref --stdin
$ git branch -r
$ git status -sb
## main...origin/main [gone]
$ git fetch
From ../../server/support-bot
* [new branch]      main      -> origin/main
* [new branch]      hotfix/timeout -> origin/hotfix/timeout
$ git status -sb
## main...origin/main
```

When the remote-tracking refs themselves are in doubt (a tool wrote to them, a URL was switched to a different repository), remember what they are: a cache. Delete them all (🟡) and fetch. Local branches and their upstream settings are not touched; status says `[gone]` in between and is correct again afterwards. The price is the reflogs of the deleted refs, your only record of earlier server states.

12.15 What can go wrong

Symptom	Diagnosis	Fix	Prevention
<code>! [rejected] ... (fetch first) or (non-fast-forward)</code>	someone pushed before you; <code>git fetch</code> , then <code>git status -sb</code> shows behind	<code>integrate (git rebase @{u} or git merge @{u})</code> , push again	fetch before you start; short-lived branches
<code>! [remote rejected] ... (pre-receive hook declined)</code> , or a platform rule	the <code>remote:</code> lines state the policy	follow the policy, usually a pull request; do not force	know which branches are protected
<code>fatal: Need to specify how to reconcile divergent branches.</code>	<code>git status -sb</code> shows ahead and behind	<code>git pull --rebase</code> or <code>--no-rebase</code> ; or look first with <code>git log --oneline --graph HEAD @{u}</code>	set <code>pull.rebase</code> or <code>pull.ff</code> on purpose
<code>fatal: Not possible to fast-forward, aborting.</code>	<code>pull.ff=only</code> is set, perhaps in another scope: <code>git config list --show-origin --show-scope</code>	pass <code>--rebase</code> or <code>--no-rebase</code> for this pull	know your global configuration
Everything up-to-date, but the commit is not on the server	<code>git branch -a --contains <commit></code> ; <code>git ls-remote origin</code>	push the branch that holds the commit	<code>read <source> -> <destination></code> in push output
<code>error: src refspec <name> does not match any</code>	<code>git branch</code> : no such branch, or no commit yet	use the real branch name; commit first	do not copy <code>master</code> or <code>main</code> from a tutorial without checking
<code>fatal: The current branch <name> has no upstream branch.</code>	<code>git branch -vv</code> shows no brackets	<code>git push -u origin <name></code>	<code>push.autoSetupRemote=true</code>
"Up to date" locally while teammates see newer commits	the last fetch is old: <code>git reflog show --date=iso origin/main -1</code> prints when that ref last moved	<code>git fetch</code>	fetch before decisions; scripts use <code>git ls-remote</code>
A commit vanished after <code>git pull --rebase</code>	the fetch printed (forced update); <code>git reflog</code> still lists the commit	<code>git cherry-pick <commit></code> , push	after a forced update, look before integrating
<code>error: some local refs could not be updated</code>	a stale remote-tracking ref blocks a new name	<code>git fetch --prune</code>	<code>fetch.prune=true</code>
A deleted branch is back on the server	pushed from a clone whose branch showed <code>gone</code>	delete it again; delete the local branch	prune, then remove <code>gone</code> branches

Symptom	Diagnosis	Fix	Prevention
warning: refname 'origin/main' is ambiguous.	<code>git branch -a</code> lists a local <code>origin/main</code>	<code>git branch -D origin/main</code> after checking what it holds	never create local branches named <code><remote>/...</code>
A tag is a different commit on two machines	<code>git rev-parse <tag></code> against <code>git ls-remote --tags origin <tag></code>	after confirming the server is right, <code>git fetch --tags --force</code>	never move a published tag
<code>git switch <branch></code> : invalid reference, though the server has the branch	<code>git config get --all remote.origin.fetch</code> shows a narrow refspec	<code>git remote set-branches --add origin <branch></code> , fetch	know how the clone was made

12.16 When not to use it, and dangerous edge cases

Pruning is forgetting. A remote-tracking ref may be the last name that some commits have in your clone:

```
# Asha has deleted spike/hybrid-search on the server. Your clone has not fetched since.
$ git branch -a --contains origin/spike/hybrid-search
remotes/origin/spike/hybrid-search
$ git log --oneline -2 origin/spike/hybrid-search
509f067 Return the query as a stub result
249e18e Try hybrid search
$ git fetch --prune
From ../.. /server/support-bot
- [deleted]          (none)      -> origin/spike/hybrid-search
$ git reflog show origin/spike/hybrid-search 2>&1 | head -1
fatal: ambiguous argument 'origin/spike/hybrid-search': unknown revision or path not in the working tree.
$ git fsck --unreachable | grep commit
unreachable commit 509f0670ffe15a66ea88e923674de467cf3a58ff
unreachable commit 249e18e16ceb304d7eea2d0b42a167965316b737
```

Asha deleted the spike branch on the server. Before the prune your clone still named its two commits. Afterwards the ref is gone, its reflog is gone with it, and the commits are unreachable. They stay in the object database until garbage collection removes them (Chapter 13), and until then `git fsck` can find them:

```
$ git fsck --lost-found | grep commit
dangling commit 509f0670ffe15a66ea88e923674de467cf3a58ff
$ git branch rescue/hybrid-search 509f067
$ git log --oneline -2 rescue/hybrid-search
509f067 Return the query as a stub result
249e18e Try hybrid search
```

With `fetch.prune=true` this happens on every fetch, without a preview. That is acceptable for merged feature branches and costly on the day someone deletes the wrong branch and every clone forgets it at its next fetch. If your clones are the team's only safety net, prune deliberately.

Other cases where the technique is the wrong one:

- **`git pull` in automation, and on branches that other people rewrite.** A job that pulls can create merge commits nobody reviewed. A pull with rebase after someone's forced push can drop commits (section 12.8). Fetch, compare, then act.
- **A forced push to a branch that anyone else pushes to.** The lease forms reduce the risk; they do not make the rewrite harmless for people who already have the old commits.
- **`git push --mirror` and `git push --prune`.** Both delete refs on the server that your repository lacks. From an ordinary clone, `--mirror` would also publish your remote-tracking refs. They are migration tools for a `--mirror` clone,  everywhere else.
- **A refspec that writes into `refs/heads/*` or `refs/tags/*` combined with pruning.** The `git fetch` manual warns that `--prune` with a tag refspec, which is what `--prune-tags` adds, deletes local tags that the remote lacks, including tags you made yourself.
- **A `--bare` clone as a backup.** It does not follow its source (section 12.3). A `--mirror` clone follows, and reproduces deletions too.
- **A token inside the URL.** `https://<token>@host/...` in `remote.origin.url` stores the secret in plain text in `.git/config`, and `git remote -v` prints it wherever it runs, CI logs included. Use a credential helper (Chapter 16).
- **A forced push as a way to remove a leaked secret.** The commit stays in clones, in the server's object database, and can be fetched by ID where the server allows it (section 12.8). Rotate the secret first (Chapter 21B).

12.17 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git remote -v</code> , <code>git remote show</code> , <code>git ls-remote</code> , <code>git branch -vv</code>	 SAFE	nothing (remote show and <code>ls-remote</code> contact the server)	not needed	not needed
<code>git clone</code>	 SAFE	creates a new directory	not needed	delete the directory
<code>git fetch</code> , <code>git remote update</code>	 SAFE	remote-tracking refs, followed tags, <code>FETCH_HEAD</code> , objects	<code>git ls-remote</code> ; <code>git fetch --dry-run</code>	<code>origin/<branch>@{1}</code> in the reflog

Command	Label	What it changes	Preview	Recovery
<code>git fetch --prune</code> , <code>git remote prune</code>	⚠ CAUTION	deletes stale remote-tracking refs and their reflogs	<code>git remote prune --dry-run <remote></code>	<code>git fsck --lost-found</code> , then <code>git branch <name> <id></code> , before garbage collection
<code>git fetch --tags --force</code>	⚠ CAUTION	replaces local tags	<code>git ls-remote --tags</code>	note <code>git rev-parse <tag></code> first, re-create from that ID
<code>git remote add</code>	✅ SAFE	adds configuration	not needed	<code>git remote remove</code>
<code>git remote set-url</code> , <code>set-head</code> , <code>set-branches</code> , <code>rename</code> ; <code>git branch -u</code> ; <code>git config set push.default</code>	⚠ CAUTION	where later fetches, pulls and pushes go	<code>git remote -v</code> ; <code>git config list --local</code>	set the previous value
<code>git remote remove</code>	⚠ CAUTION	the remote, its remote-tracking refs and reflogs, upstream settings	<code>git branch -r</code> , <code>git branch -vv</code>	add and fetch again; the reflogs are lost
<code>git pull --ff-only</code>	⚠ CAUTION	fetch, then moves the branch forward; working tree	<code>git fetch</code> ; <code>git log ..@{u}</code>	<code>git reset --hard ORIG_HEAD</code> (itself 🚫)
<code>git pull --no-rebase</code> , <code>git pull --rebase</code>	⚠ CAUTION	fetch, then a merge commit or re-created local commits	<code>git fetch</code> ; <code>git log --oneline --graph HEAD @{u}</code>	<code>git reset --hard ORIG_HEAD</code> ; the reflog
<code>git push</code> , <code>git push -u</code> , <code>git push --follow-tags</code>	⚠ CAUTION	refs on the server (fast-forward or new), your remote-tracking refs	<code>git push --dry-run</code> ; <code>git log @{u}..</code>	published commits are not taken back; <code>git revert</code> and push
<code>git push <remote> --delete <ref></code> , <code>git push <remote> :<ref></code>	🚫 DANGEROUS	deletes a ref on the server	<code>git ls-remote <remote></code>	from any clone that has the commit: <code>git push <remote> <id>:refs/heads/<name></code>
<code>git push --force</code> , <code>git push <remote> +<ref></code>	🚫 DANGEROUS	sets the server's ref with no check	<code>git fetch</code> ; <code>git log HEAD..@{u}</code> lists what would be lost	section 12.8: a clone that has the old tip, or the server's unreachable object
<code>git push --force-with-lease=<ref>[:<expect>]</code> , with or without <code>--force-if-includes</code>	🚫 DANGEROUS	the same, only if the server's ref has the expected value	the same	the same
<code>git push --mirror</code> , <code>git push --prune</code>	🚫 DANGEROUS	creates, forces and deletes server refs to match yours	<code>--dry-run</code>	from other clones, ref by ref

12.18 Version notes

Two version notes are placed where they are needed: `origin/HEAD` and `git fetch` in section 12.3, and the fatal error of `git pull` in section 12.6.

Version note. Older behavior: `push.default=matching`, so a bare `git push` pushed every branch with a same-named branch on the server. Current behavior: `simple`. Since: Git 2.0 ([release notes](#)); `push.autoSetupRemote` since 2.37 ([push configuration at 2.37](#)). Recommended: keep `simple`; set `push.autoSetupRemote=true`.

Version note. Older behavior: `git push --force` was the only way to overwrite. Current behavior: `--force-with-lease` (since Git 1.8.5) and `--force-if-includes` (since 2.30); the manual still calls every `force` form without an explicit expected value experimental ([git-push at 2.56](#), [release notes 2.30](#)). Recommended: `--force-with-lease` `--force-if-includes`, or the explicit form.

Version note. Older behavior: a fetch that included tags, such as `git fetch --tags`, replaced a tag you already had when the server's tag differed. Current behavior: it rejects the update unless forced. Since: Git 2.20 ([release notes](#)). Recommended: treat every "would clobber existing tag" as a question to the person who moved the tag.

Version note. Older behavior: `git ls-remote --heads` and `git show-ref --heads`. Current behavior: `--branches`; `--heads` remains as a deprecated synonym. Since: Git 2.46 ([release notes](#)), the release that also introduced `git config get` and `git config set`. Recommended: write `--branches`, read `--heads`.

Version note. Older behavior: the original wire protocol, in which the server lists every ref first. Current behavior: protocol version 2. Since: default in Git 2.26 ([release notes](#)). Recommended: nothing to configure; know that `protocol.version` exists when you read an old trace.

Dated additions used in this chapter: `remote.pushDefault`, `branch.<name>.pushRemote` and `--follow-tags` (Git 1.8.3), the update of `origin/<branch>` by `git fetch origin <branch>` (1.8.4), `fetch.prune` (1.8.5), `git push --atomic` (2.4), `@{push}` (2.5), negative refsspecs (2.29), `status.compareBranches` (2.54), and `git push` to a remote group and `git url-parse` (2.55), each according to the release notes of that version. Also since 2.55, terminal control sequences in the `remote:` lines that a server sends are masked, except colors (`sideband.allowControlCharacters`). The legacy remote definitions in `$GIT_DIR/remotes` and `$GIT_DIR/branches` are scheduled for removal in Git 3.0 ([BreakingChanges](#)).

Unverified. Whether a hosting platform serves unreachable commits to `git fetch <remote> <full-id>` depends on its server configuration; it was verified here only for a plain Git 2.55.0 server. The release in which `git branch -vv` began to print `gone` was not looked up.

12.19 Practice

- Labs 7.1 to 7.7 in the Module 7 lab manual. Do 7.1 first; it builds the server and both clones by hand. Answers to the lab questions are in the solutions.
- Replay any transcript with `labs/run ch12/<demo>`, for example `labs/run ch12/force-push`. The sandbox stays in place, so you can continue by hand.
- Five drills in those sandboxes. Predict, then run.
 1. `ch12/fetch-anatomy`: what will `.git/FETCH_HEAD` contain after `git fetch origin feature/reranker`, and which refs will move?
 2. `ch12/pull-matrix`: add `branch.main.rebase=false` to the row `pull.rebase=true`. Which key wins?
 3. `ch12/push-destination`: with `push.default=upstream`, what does `git push --dry-run` do on `feature/streaming`?
 4. `ch12/force-push`: name every repository that still holds `833bc8a` at the end, and how it is reachable there.
 5. `ch12/refspec-surgery`: write the configuration that fetches only `main` and `release/*`.

12.20 Interview questions

1. What exactly is `origin/main`? Where is it stored, and which commands move it?
2. `git status` says "Your branch is up to date with 'origin/main'". What did Git compare, and what do you run before tagging a release?
3. Take `git clone` apart into separate commands. What does `--mirror` add to `--bare`, and why does the difference matter for a backup?
4. A pull stopped with "Need to specify how to reconcile divergent branches". What changed in the repository and what did not? Give the three answers and their effect on history.
5. Two engineers in the same situation got `rejected (fetch first)` and `rejected (non-fast-forward)`. Explain the difference and who made each decision. How is `remote rejected` different?
6. Explain `--force-with-lease`, one way it fails to protect a teammate's commit, and the two stronger forms.
7. After a colleague's forced push, a developer ran `git pull --rebase`. It succeeded and their commit was gone. Explain the mechanism and recover the commit.
8. A forced push overwrote a commit on a self-hosted bare repository. List every place where the commit may still exist and how you would bring it back.
9. What do `branch.<name>.remote`, `branch.<name>.merge`, `remote.pushDefault` and `push.default` each decide? Configure a clone that pulls from `upstream` and pushes to `origin`.
10. A branch that was deleted a month ago is back on the server, and nobody intended it. How did it happen, and what do you change?
11. Why does Git refuse a push into a checked-out branch, and what would you configure on a staging machine?
12. The deploy job cannot find a commit, and the developer's push printed "Everything up-to-date". Give your first four commands and what each one proves.

12.21 Sources

Primary sources

- [git-clone](#), [git-fetch](#), [git-pull](#), [git-push](#), [git-remote](#), [git-ls-remote](#), [git-branch](#), [git-merge-base](#) (fork point), [git-receive-pack](#), [git-bundle](#), [git-url-parse](#), [gitrevisions](#), [gitglossary](#), [gitdatamodel](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that every flag in this chapter was checked against.
- [gitprotocol-v2](#) and [gitprotocol-pack](#): the fetch and push conversations of section 12.13, including "wants can be anything".
- Configuration reference at 2.56: [push](#), [pull](#), [fetch](#), [core](#).
- Release notes: [1.8.3](#), [1.8.4](#), [1.8.5](#), [2.0.0](#), [2.4.0](#), [2.5.0](#), [2.20.0](#), [2.26.0](#), [2.29.0](#), [2.34.0](#), [2.48.0](#), [2.54.0](#), [2.55.0](#). Homebrew installs the same files under `share/doc/git-doc/ReINotes/`.

Secondary sources

- Pro Git: [Working with Remotes](#), [Remote Branches](#), [The Refspec](#), [Transfer Protocols](#). Caveats: `master` and `git checkout` throughout; the first still says that `git pull` only warns when `pull.rebase` is unset, which was true from 2.27 to 2.33.0; the last does not mention protocol version 2.
- GitHub Docs: [About remote repositories](#), [Forks](#), [Syncing a fork](#), [Checking out pull requests locally](#), [Renaming a branch](#), [Troubleshooting commits](#).

- Julia Evans, [Confusing git terminology](#), cited by the Phase 0 report for what "Your branch is up to date with 'origin/main'" does and does not say.
- The Phase 0 report of this course, sections 1, 4, 12 and 13, for the dated defaults, the misconception table and the force-push safety facts.

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [Introduction to Git - Remotes](#), David Mahler, 31 minutes, 27 March 2018: clone, fetch versus pull and tracking branches drawn on the commit graph. Caveat: `master` throughout.
- [Git and GitHub - Full Course](#), ThePrimeagen for Boot.dev, 12 November 2024: the chapter on remotes and GitHub starts at 1:54:18, followed by forks. Caveat: digressive style.
- [Complete Git and GitHub Tutorial](#), Kunal Kushwaha, 1 August 2021: fork, upstream remote, keeping a fork in sync. Caveats from the report: recorded days before password removal, uses a plain force push, mixes `master` and `main`, and could not be transcript-audited.

Further reading

- [gitprotocol-http](#), for what "smart HTTP" does with the conversation of section 12.13.
- [git-maintenance](#), task `prefetch`: Git's own scheduled fetch writes to `refs/prefetch/` so that it does not move remote-tracking branches, and therefore does not renew a lease.
- Chapter 13 (Recovery) for `reflogs` and `git fsck`, Chapter 26 (Performance) for shallow and partial clones and bundle URLs, and Chapter 30 (Incident response) for the force-push drill.

Chapter 13: Recovery

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch13/`.

13.1 Why this matters

It is 18:40 on a release day. An engineer on your team writes in the incident channel: "I ran a reset and three days of work on the reranker are gone." Your CTO asks you two questions, and neither of them is "which command do we type". The first is "can we get it back?". The second is "how sure are you, and how long do we have?".

Both questions have exact answers. They depend on three facts about the lost work:

- 1. **Was it ever written into the object database?** A commit was. A file that was staged with `git add` was. An edit that was only saved in the editor was not.
- 2. **Does anything still name it?** A branch, a tag, a remote-tracking ref, a stash entry, a reflog entry, or another repository.
- 3. **Has the clock run out?** Reflog entries expire, and objects that nothing names are deleted after a grace period, but only when maintenance runs.

This chapter teaches you to answer those three questions from evidence, for fifteen kinds of accident: nine that lose commits (section 13.8), two that lose uncommitted work (13.9), two that involve the server (13.10), and two that damage the repository itself (13.11). It also teaches the opposite skill, which a senior engineer needs as much: to say "this cannot be recovered, and here is exactly why".

Three earlier chapters supply the foundations, and this one does not repeat them. Chapter 3: Git Internals defines reachability, shows the reflog files under `.git/logs`, and introduces `git fsck`. Chapter 11: Reset, Revert, Restore proves what `git reset --hard` destroys and explains the stash. Chapter 12: Remote Operations explains force pushes and remote-tracking refs. Here those pieces become procedures.

The transcripts in sections 13.8 to 13.11 come from the replays of the twelve disaster labs of Module 12. You meet each incident twice: here with the explanation, and in the lab manual with your own hands.

13.2 What Git keeps: four layers of protection

In one sentence. Git never deletes an object that something still names, it keeps a private log of every name that recently changed, and it waits before deleting what nothing names; work is lost only when it was never an object or when all of these protections are gone.

Analogy. A bank vault with a ledger. The safe-deposit boxes are objects. The index cards at the front desk are refs: each card names one box. When a card is rewritten, the clerk notes the old box number in a ledger, the reflog. Boxes that no card and no ledger line mentions are emptied in a periodic clear-out, but only boxes that have been untouched for two weeks. The analogy breaks in two places. A box in Git also holds the numbers of older boxes (a commit names its parents and its tree), so one card protects a whole chain. And the ledger is private to one branch office: no other clone, and no server, sees your reflog.

Precisely. An object is kept as long as it is reachable from a starting point. The manual of `git gc` lists the starting points: branches and tags, the index, remote-tracking branches, reflogs, "and anything else in the refs/" namespace" (`git help gc`, section NOTES). The same section adds one exclusion: a note attached to an object does not keep it alive. An object that no starting point reaches is unreachable. It still exists until a collection deletes it, and a collection deletes only objects older than a cut-off, two weeks by default (`gc.pruneExpire`).

That gives four layers, and a recovery is always a move from a lower layer back to the first one:

Layer	What protects the object	Lifetime by default	Tool that finds it
1. Refs	A branch, tag, remote-tracking ref, <code>refs/stash</code> , <code>HEAD</code> , or the index names it	Until the ref is moved or deleted	<code>git log --all</code> , <code>git for-each-ref</code>
2. Reflogs	A reflog entry names it	90 days, or 30 days for entries off the current history (section 13.4)	<code>git reflog</code> , <code>git log -g</code>
3. Grace period	Nothing names it; the object file is younger than the prune cut-off	Two weeks from the time the object was written, then until the next collection runs	<code>git fsck</code>
4. Other repositories	Another clone, the server, a bundle, a backup	Independent of this repository	<code>git fetch</code> , <code>git bundle</code> , a teammate

Before the layers comes a precondition, the question from section 13.1: was the work ever an object?

Form of the work	Object written?	Named by
Committed	Yes: commit, trees, blobs	A branch or <code>HEAD</code> , and the reflog
Stashed	Yes: two or three commits (Chapter 11, section 11.11)	<code>refs/stash</code> and its reflog
Staged with <code>git add</code> , not committed	Yes: a blob	Only the index entry

Form of the work	Object written?	Named by
Saved in the working tree, never staged	No	Nothing in Git
Untracked or ignored file	No	Nothing in Git

Inside `.git`. Layer 1 is `refs/`, `packed-refs`, `HEAD` and `index`. Layer 2 is `logs/`. Layer 3 is the modification time of files under `objects/`: of loose objects, and of the `.mtimes` file that accompanies a cruft pack (section 13.13). Layer 4 is not in this `.git` at all.

See it. A commit that was removed from its branch with `git reset --hard` is on layer 2. A collection with no grace period at all, `git gc --prune=now`, does not touch it, because the reflog names it:

```
$ git reflog -2
da62b60 HEAD@{0}: reset: moving to HEAD~1
dbe6ec8 HEAD@{1}: commit: Add cache TTL
$ git prune -n
$ git gc --prune=now
$ git cat-file -t dbe6ec8
commit
$ git count-objects -v | grep -e "^count" -e in-pack
count: 0
in-pack: 12
```

`git prune -n` lists what an immediate prune would delete: nothing. `git gc --prune=now` packed all twelve objects, the lost commit `dbe6ec8` included. Section 13.13 takes the same commit down the remaining layers.

Picture.

layer 1	<code>refs/heads/main</code> -----> <code>da62b60</code> --> parents, trees, blobs <code>index</code> , <code>HEAD</code> , tags, <code>refs/stash</code>	kept
layer 2	reflog entry <code>HEAD@{1}</code> -----> <code>dbe6ec8</code> --> tree, blob (<code>.git/logs/HEAD</code>)	kept while the entry lives
layer 3	nothing -----> a staged blob, a pruned fetch	kept until a collection finds it older than the cut-off
layer 4	another repository -----> its own copy of the objects	not affected by this one

In production. The first minute of a recovery is classification, not typing. "Committed on a branch that was deleted" is layer 2 or 3 and almost always recoverable. "Edited in the editor and then `git restore`" was never an object, so you move on to editor history and backups at once.

13.3 The reflog

In one sentence. A reflog is a local, append-only list of the values a ref has had, with the time, the person and the command behind each change, and every one of those old values is a name you can use.

Analogy. The "undo history" of an editor, kept per document. The analogy breaks in two places. A reflog has no undo command: it gives you old positions, and you decide what to do with them. And it records movements of refs only. Edits to files, staging and unstaging leave no entry.

Precisely. There is one reflog per ref that has logging enabled, plus one for `HEAD`. With the default `core.logAllRefUpdates=true` of a repository that has a working tree, Git logs `HEAD`, branches, remote-tracking branches and notes refs; tags get no reflog unless the setting is `always` (Chapter 3, section 3.11). Each entry stores the value before the change, the value after it, the committer identity and time, and a message written by the command. `git reflog` is `git reflog show HEAD`, and the manual defines `git reflog show` as an alias for `git log -g --abbrev-commit --pretty=oneline` (`git help reflog`). ● SAFE: all of these read.

The two kinds of reflog answer different questions:

Reflog	Records	Use it to answer
<code>HEAD</code>	Every movement of <code>HEAD</code> : commits, switches, resets, each step of a rebase, merges, cherry-picks	"What did I do, in order?" It is the only log that sees detached <code>HEAD</code>
A branch, for example <code>main</code>	Only changes of that branch's value	"Where was this branch before the operation?" A whole rebase is one entry

Inside `.git`. `logs/HEAD` and `logs/refs/heads/<branch>`, one line per change. A command that moves the current branch appends to both files.

See it. A small history: a branch, a commit, an amend, a fast-forward merge, and a reset that takes the merge back.

```
$ git switch -c feature/rerank
Switched to a new branch 'feature/rerank'
$ printf 'def rerank(q, docs):\n    return sorted(docs)\n' > rerank.py
$ git add rerank.py
$ git commit -q -m "Add reranker"
$ git commit -q --amend -m "Add cross-encoder reranker"
$ git switch main
```



```
Switched to branch 'main'
$ git merge -q feature/rerank
$ git reset -q --hard HEAD~1
```

```
$ git reflog
da62b60 HEAD@{0}: reset: moving to HEAD~1
f1f247e HEAD@{1}: merge feature/rerank: Fast-forward
da62b60 HEAD@{2}: checkout: moving from feature/rerank to main
f1f247e HEAD@{3}: commit (amend): Add cross-encoder reranker
14a18d6 HEAD@{4}: commit: Add reranker
da62b60 HEAD@{5}: checkout: moving from main to feature/rerank
da62b60 HEAD@{6}: commit: Raise top_k to 10
536f5df HEAD@{7}: commit: Add embedding client
538ea2f HEAD@{8}: commit (initial): Add retriever config
```

Read it from the bottom. Each line is "the value HEAD had after this event". `HEAD@{0}` is always the current value. The amend at `HEAD@{3}` replaced `14a18d6` with `f1f247e`; the old commit is still listed one line below. The two `checkout` lines record branch switches, which change HEAD and no branch.

```
$ git reflog show main
da62b60 main@{0}: reset: moving to HEAD~1
f1f247e main@{1}: merge feature/rerank: Fast-forward
da62b60 main@{2}: commit: Raise top_k to 10
536f5df main@{3}: commit: Add embedding client
538ea2f main@{4}: commit (initial): Add retriever config
$ git reflog show feature/rerank
f1f247e feature/rerank@{0}: commit (amend): Add cross-encoder reranker
14a18d6 feature/rerank@{1}: commit: Add reranker
da62b60 feature/rerank@{2}: branch: Created from HEAD
$ git reflog list
HEAD
refs/heads/feature/rerank
refs/heads/main
```

The reflog of `main` has five lines where HEAD has nine: the switches and the work on the other branch are absent. `git reflog list` (Git 2.45 and later) names every ref that has a log.

A reflog entry is addressed with a selector, and a selector is a revision that every command accepts:

```
# Position: the n-th previous value of HEAD, of a branch, of the current branch.
$ git rev-parse --short 'HEAD@{1}'
f1f247e
$ git rev-parse --short 'main@{2}'
da62b60
$ git rev-parse --short '@{1}'
f1f247e
# The n-th branch checked out before the current one.
$ git rev-parse --abbrev-ref '@{-1}'
feature/rerank
# A selector is a revision like any other.
$ git diff --stat 'main@{1}' main
rerank.py | 2 --
1 file changed, 2 deletions(-)
```

Selector	Meaning
<code>HEAD@{2}</code>	The value HEAD had two changes ago
<code>main@{1}</code>	The value <code>main</code> had before its latest change
<code>@{1}</code>	The same, for the current branch (not for HEAD: on <code>main</code> , <code>@{1}</code> is <code>main@{1}</code>)
<code>main@{yesterday}</code> , <code>main@{2026-09-07 10:10:30}</code> , <code>HEAD@{5.minutes.ago}</code>	The value the ref had at that time, according to this repository's log
<code>@{-1}</code>	The branch that was checked out before the current one. This is a name, not a position, and <code>git switch -</code> uses it

Quote selectors in the shell: braces are special to some shells.

Time-based selectors read the timestamps in the log:

```
$ git reflog --date=iso -4
da62b60 HEAD@{2026-09-07 10:13:00 +0530}: reset: moving to HEAD~1
f1f247e HEAD@{2026-09-07 10:12:00 +0530}: merge feature/rerank: Fast-forward
da62b60 HEAD@{2026-09-07 10:11:00 +0530}: checkout: moving from feature/rerank to main
f1f247e HEAD@{2026-09-07 10:10:00 +0530}: commit (amend): Add cross-encoder reranker
$ git rev-parse --short 'main@{2026-09-07 10:10:30}'
da62b60
$ git rev-parse --short 'HEAD@{5.minutes.ago}'
da62b60
$ git rev-parse --short 'main@{2.days.ago}'
warning: log for 'main' only goes back to Mon, 7 Sep 2026 10:03:00 +0530
538ea2f
```

At 10:10:30 `main` was still at `da62b60`, because the merge happened at 10:12. The last command asks for a time before the log begins. Git answers with the oldest value it knows and says so in a warning. A time selector is therefore a statement about your local log, never about the project: in a clone made today, `main@{1.week.ago}` cannot know where `main` was a week ago.

`git log -g` walks a relog instead of the ancestry, and it has placeholders for the relog fields:

```
$ git log -g --format='%h %gd %gs' -4
da62b60 HEAD@{0} reset: moving to HEAD~1
f1f247e HEAD@{1} merge feature/rerank: Fast-forward
da62b60 HEAD@{2} checkout: moving from feature/rerank to main
f1f247e HEAD@{3} commit (amend): Add cross-encoder reranker
$ git log -g --date=relative --format='%h %gd | %gn | %s' -3 main
da62b60 main@{15 minutes ago} | Lab User | Raise top_k to 10
f1f247e main@{16 minutes ago} | Lab User | Add cross-encoder reranker
da62b60 main@{23 minutes ago} | Lab User | Raise top_k to 10
$ git log -g --format='%h %gd %gs' --grep-reflog='^commit'
f1f247e HEAD@{3} commit (amend): Add cross-encoder reranker
14a18d6 HEAD@{4} commit: Add reranker
da62b60 HEAD@{6} commit: Raise top_k to 10
536f5df HEAD@{7} commit: Add embedding client
538ea2f HEAD@{8} commit (initial): Add retriever config
```

`%gd` is the selector, `%gs` the relog message, `%gn` the name of the person who made the change; with `--date`, the selector shows the time instead of the position. `--grep-reflog` filters by relog message. The third command lists only the entries that created commits, which is how you search a long log for the event you need instead of counting positions.

On disk, one event on the current branch is two lines, one in each file:

```
$ tail -2 .git/logs/HEAD
da62b6073a1d8c0afa10417bc244ed245ccdfff86 f1f247eb281770cea385642df710bd095c262b17 Lab User <you@example.com> 1788756120 +0530
merge feature/rerank: Fast-forward
f1f247eb281770cea385642df710bd095c262b17 da62b6073a1d8c0afa10417bc244ed245ccdfff86 Lab User <you@example.com> 1788756180 +0530
reset: moving to HEAD~1
$ tail -1 .git/logs/refs/heads/main
f1f247eb281770cea385642df710bd095c262b17 da62b6073a1d8c0afa10417bc244ed245ccdfff86 Lab User <you@example.com> 1788756180 +0530
reset: moving to HEAD~1
$ cat .git/refs/heads/main
da62b6073a1d8c0afa10417bc244ed245ccdfff86
```

The first ID of a line is the old value and the second the new one. That is more than `git reflog` prints, and it matters: the old value of the newest line tells you where a ref was even when no older line exists (section 13.4 shows a server log that starts this way).

Picture.

```
one command: git reset --hard HEAD~1      (on main)

.git/refs/heads/main      f1f247e -> da62b60      the ref itself: only the new value
.git/logs/refs/heads/main f1f247e da62b60 ... reset: moving to HEAD~1      main@{0}, and main@{1} = f1f247e
.git/logs/HEAD            f1f247e da62b60 ... reset: moving to HEAD~1      HEAD@{0}, and HEAD@{1} = f1f247e
```

In production. Read the branch relog, not the HEAD relog, when the question is "where was this branch": it has one line per operation, where HEAD has one line per step. In an incident note, record the full ID, not the selector. `main@{1}` means something else after the next command, and nothing at all on a colleague's machine.

13.4 Retention: how long the safety net lasts, and its exceptions

The defaults. Three numbers govern recoverability in a repository with default configuration ([gc configuration](#), and `git help gc` on your machine):

Setting	Default	What it removes
<code>gc.reflogExpire</code>	90 days	Reflog entries older than this
<code>gc.reflogExpireUnreachable</code>	30 days	Reflog entries older than this that are "not reachable from the current tip"
<code>gc.pruneExpire</code>	2 weeks	Unreachable objects whose file is older than this

The lab configuration overrides the first two with `never` (Chapter 1, section 1.7), because the lab clock is fixed and the expiry code reads the real clock. Everything in this section that concerns the 90 and 30 days is therefore stated from the manual, and the transcripts use only cut-offs that do not depend on a clock.

Two facts about these numbers are routinely misunderstood.

First, nothing happens when the time is up. Expiry is work done by a command: `git gc`, `git reflog expire`, or the maintenance that Git starts after some commands (section 13.13). An entry that is 100 days old is still there until one of them runs.

Second, the 30-day rule is the one that matters for recovery, because it is the one that applies to the entries you recover from. The manual explains what such entries are: they "are generally created as a result of using `git commit --amend` or `git rebase` and are the commits prior

to the amend or rebase occurring". The exact test is in the source ([reflog.c](#)): an entry is subject to the shorter period if its old value or its new value is not reachable from the ref's current tip. For the HEAD reflog the code tests against the tips of all refs. You can ask Git which entries those are, with a cut-off of `now` and `--dry-run`:

```
$ git reflog show main
dfcbd4b main@{0}: reset: moving to HEAD~1
5c75388 main@{1}: commit: Temporary debug flag
dfcbd4b main@{2}: commit (amend): Raise top_k from 5 to 10
da62b60 main@{3}: commit: Raise top_k to 10
536f5df main@{4}: commit: Add embedding client
538ea2f main@{5}: commit (initial): Add retriever config
# Which entries does the 30-day rule apply to? Ask with a cut-off of "now" and --dry-run:
$ git reflog expire --dry-run --verbose --expire-unreachable=now refs/heads/main
keep commit (initial): Add retriever config
keep commit: Add embedding client
prune commit: Raise top_k to 10
prune commit (amend): Raise top_k from 5 to 10
prune commit: Temporary debug flag
prune reset: moving to HEAD~1
# It was a dry run. All six entries are still there:
$ git reflog show main | grep -c main@
6
```

The branch was amended, extended by one commit, and reset by one commit. Four of six entries fall under the shorter rule: the commit before the amend, the commit that was reset away, and also the two entries whose *other* end is one of those commits. The amend entry is marked because its old value is the replaced commit, and the reset entry because its old value is the discarded commit. Despite the word `prune`, the dry run removed nothing, as the count shows.

When the expiry is carried out, the reflog of the branch loses exactly the lines a recovery would need:

```
$ git reflog expire --expire-unreachable=now refs/heads/main
$ git reflog show main
536f5df main@{0}: commit: Add embedding client
538ea2f main@{1}: commit (initial): Add retriever config
$ git log --oneline -1 main
dfcbd4b Raise top_k from 5 to 10
$ git reflog -3
dfcbd4b HEAD@{0}: reset: moving to HEAD~1
5c75388 HEAD@{1}: commit: Temporary debug flag
dfcbd4b HEAD@{2}: commit (amend): Raise top_k from 5 to 10
```

Observed behavior	: "git reflog show main" starts at "Add embedding client", but main points at dfcbd4b
Git state	: refs/heads/main = dfcbd4b; the newest line left in logs/refs/heads/main records 536f5df
Mechanism	: reflog expiry removed every entry with an end that the current tip does not reach, including the entries that moved the branch to its current value
Root cause	: a reflog is a list of past changes that is trimmed by a retention policy; it is not a second copy of the ref
Why Git does this	: such changes "are not part of the current project", so "most users will want to expire them sooner" (git help gc)
Correct fix	: none needed; the ref is right and the log is shorter
Prevention	: anchor anything you may want later with a branch or tag before the 30 days are over

The HEAD reflog was not named in that command and still has every entry. In a real repository both logs are trimmed in the same run.

The grace period is not added on top. A common summary is "30 days in the reflog, then two more weeks". The manual does not promise that. `gc.pruneExpire` compares the cut-off with the modification time of the object: "Any object with modification time newer than the `--prune` date is kept, along with everything reachable from it", and most operations that would write an object that already exists refresh that time (git help gc, section NOTES). The two weeks are measured against that modification time, not from the moment the object became unreachable. A commit that was written 40 days ago, has not been touched since, and loses its last reflog entry today may already be older than the cut-off. The grace period protects what never had a reflog entry and was written recently: a blob you staged yesterday, a branch you fetched last week and pruned today. The lab cannot age a file, so this is the manual's statement, not a demonstration.

Picture. The life of a commit that was replaced by `git commit --amend`, in a repository with default settings:

day 0	amend: the old commit leaves the branch. Layer 2: two reflogs name it.
day 0 to 30	"git reset --hard <id>" or "git branch <name> <id>" brings it back.
after day 30	the next maintenance run removes those reflog entries. Layer 3, or gone: the object file is older than two weeks, so the same run may delete it.
later	a collection deletes whatever is unreachable and older than the cut-off.

The exceptions

Four situations give you less than the table promises, and one gives you more.

A deleted branch loses its reflog at once. The manual of `git branch` says so for `-d` and `-D`: "If the branch currently has a reflog then the reflog will also be deleted." The commits stay reachable from the HEAD reflog if you ever had the branch checked out, and section 13.8 uses that. The same holds for a remote-tracking branch that `git fetch --prune` 🟡 removes.

A commit that was never checked out has no HEAD reflog entry. A teammate's branch that you only fetched exists in your clone as a remote-tracking ref. When that ref is pruned, nothing names the commits, and they are on layer 3 from that moment.

A bare repository keeps no reflogs by default. `core.logAllRefUpdates` is "true by default in a repository that has a working directory associated with it, and false by default in a bare repository" ([core configuration](#)). A server that receives a force push therefore has no record of the value that was overwritten:

```
$ git -C searchsvc push -q origin main
$ git -C server.git reflog list
$ git -C server.git config get core.logAllRefUpdates
[exit status: 1]
$ git -C searchsvc config get core.logAllRefUpdates
true
```

Whoever administers a bare repository can change that (`git config set` and `git config get` need Git 2.46 or later). From then on a push is an entry, and the overwritten value of a forced push can be read back:

```
# Whoever runs a bare repository can switch reflogs on:
$ git -C server.git config set core.logAllRefUpdates true
$ git -C searchsvc commit -q --amend -m "Raise top_k to ten"
$ git -C searchsvc push -q --force origin main
$ git -C server.git reflog show main
803db47 main@{0}: push
$ git -C server.git log --oneline -1 'main@{1}'
dfcbd4b Raise top_k from 5 to 10
```

The log has one line, and `main@{1}` still resolves: Git takes the old value recorded in that line. It is a setting for servers you run yourself; a hosting service has its own instruments (section 13.15).

Tags have no reflog. Moving a tag with `git tag -f` prints the old ID once (Updated tag 'v1' (was ...)), and that line is the only record.

Stash entries do not expire, until they are dropped. The stash list is the reflog of `refs/stash`, and Git's expiry code exempts that ref when nothing is configured for it: the comment in the source reads "If unconfigured, make stash never expire" ([reflog.c, lines 117 to 126](#); no manual page states it). An entry removed with `git stash drop` , `pop` or `clear`  is a different matter: it has left the reflog, and the stash manual says that such entries "cannot be recovered through the normal safety mechanisms". Section 13.9 recovers them from layer 3.

The exemption applies to the default periods only. A cut-off given on the command line applies to every ref, `refs/stash` included, and the result is one of the stranger states Git can be in:

```
$ printf 'top_k: 20\n' > retriever.yaml
$ git stash push -q -m "try top_k 20"
$ printf 'top_k: 50\n' > retriever.yaml
$ git stash push -q -m "try top_k 50"
$ git stash list
stash@{0}: On main: try top_k 50
stash@{1}: On main: try top_k 20
$ git reflog show refs/stash
71ed40a refs/stash@{0}: On main: try top_k 50
89fc6b8 refs/stash@{1}: On main: try top_k 20
```

```
$ git reflog expire --expire=now --all
$ git stash list
$ git log --oneline -1 refs/stash
71ed40a On main: try top_k 50
$ git fsck
dangling commit 89fc6b87236ad6fbf9108422f65174e9dfc095dd
dangling commit da62b6073a1d8c0afa10417bc244ed245ccdfff86
dangling commit 5c7538866f5b5c9cfc3186aa957123a55108d5a0
```

Observed behavior	: "git stash list" prints nothing after "git reflog expire --expire=now --all"
Git state	: <code>refs/stash</code> still points at 71ed40a, the newest entry; its log is empty; the older entry 89fc6b8 is a dangling commit
Mechanism	: "git stash list" prints the reflog of <code>refs/stash</code> . The command emptied the log and left the ref alone
Root cause	: only the newest stash entry is held by a ref. Every older entry is held by a reflog line and by nothing else
Why Git does this	: the stash was built as one ref plus its reflog, so that "stash@{n}" is ordinary reflog syntax
Correct fix	: "git stash pop" still restores the newest entry (below). The older ones are found with <code>git fsck</code> while their objects exist
Prevention	: do not expire reflogs with <code>--all</code> in a repository that holds stashes you need; commit long-lived work to a branch instead of stashing it

```
$ git stash pop
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   retriever.yaml

no changes added to commit (use "git add" and/or "git commit -a")
```

```
Dropped refs/stash@{0} (71ed40a90f00616a336fa87d8470ab2abfc42ce6)
$ git stash list
$ git fsck --unreachable | grep commit | cut -d' ' -f3 | xargs git log --merges --no-walk --format='%h %s'
71ed40a 0n main: try top_k 50
89fc6b8 0n main: try top_k 20
```

Removing entries by hand

`git reflog expire` 🟡 removes entries by age and reachability; `--expire=<time>` and `--expire-unreachable=<time>` override the two settings for one run, `--all` processes every ref, and `-n` previews. `git reflog delete <ref>@{<n>}` 🟡 removes single entries, and `git reflog drop <ref>` 🟡 (Git 2.50 and later) removes the whole log of a ref. The manual calls `expire` and `delete` "typically not used directly by end users". None of them changes a ref or deletes an object. What they remove is the name that kept an object on layer 2:

```
# delete removes one entry, drop removes the whole log of a ref. Neither touches the ref.
$ git reflog delete 'HEAD@{1}'
$ git reflog -3
dfc6d4b HEAD@{0}: reset: moving to HEAD~1
dfc6d4b HEAD@{1}: commit (amend): Raise top_k from 5 to 10
da62b60 HEAD@{2}: commit: Raise top_k to 10
$ git reflog drop refs/heads/main
$ git reflog exists refs/heads/main
[exit status: 1]
$ git log --oneline -1 main
dfc6d4b Raise top_k from 5 to 10
```

`git stash drop` is the everyday user of this machinery: it deletes one entry from the reflog of `refs/stash` and updates the ref to match ([builtin/stash.c](#), function `do_drop_stash`). Section 13.13 returns to `git reflog expire` as the first half of the point of no return.

13.5 ORIG_HEAD: one slot, written by four commands

In one sentence. `ORIG_HEAD` is a file that holds the commit HEAD pointed at before the last "drastic" command, so that the command can be undone with `git reset --hard ORIG_HEAD`.

Analogy. The "last number redial" button of a phone. It holds exactly one number, the newest call replaces it, and it tells you nothing about which call that was. The reflog is the full call list.

Precisely. The manual: `ORIG_HEAD` "is created by commands that move your HEAD in a drastic way (`git am`, `git merge`, `git rebase`, `git reset`), to record the position of the HEAD before their operation" (`git help revisions`). `git pull` writes it too, because it runs a merge or a rebase. So does `git stash push`, through the reset it performs (Chapter 11, section 11.4). Commands that move HEAD in an ordinary way do not write it.

Inside .git. One file, `.git/ORIG_HEAD`, containing one ID. It has no reflog, and it is not a starting point for garbage collection (section 13.13 proves that).

See it. A fresh repository has no `ORIG_HEAD`. Four commands that move HEAD leave it that way:

```
$ git log --oneline --graph --all
* 584d822 Add README
| * 68e6fff Rerank the top 20
| * 155d4ba Add reranker
|/
* da62b60 Raise top_k to 10
* 536f5df Add embedding client
* 538ea2f Add retriever config
# A fresh repository has no ORIG_HEAD. These four commands move HEAD and do not create it:
$ git commit -q --amend -m "Add a README"
$ git switch -q feature/rerank && git switch -q main
$ git cherry-pick feature/rerank-1 > /dev/null
$ git revert --no-edit HEAD > /dev/null
$ git rev-parse --verify --short ORIG_HEAD
fatal: Needed a single revision
[exit status: 128]
$ git reflog -4
78070c2 HEAD@{0}: revert: Revert "Add reranker"
1ab399b HEAD@{1}: cherry-pick: Add reranker
f912289 HEAD@{2}: checkout: moving from feature/rerank to main
68e6fff HEAD@{3}: checkout: moving from main to feature/rerank
```

An amend, two switches, a cherry-pick and a revert: five movements of HEAD, each with a reflog entry, and no `ORIG_HEAD`. A reset, a merge and a rebase each write it:

```
$ git reset -q --hard HEAD~2
$ git log --oneline -1 ORIG_HEAD
78070c2 Revert "Add reranker"
```

```
$ git log --oneline -1
f912289 Add a README
$ git merge -q feature/rerank
```

```
$ git log --oneline -1 ORIG_HEAD
f912289 Add a README
```

```
$ git reset -q --hard ORIG_HEAD
$ git switch -q feature/rerank
$ git log --oneline -1
68e6fff Rerank the top 20
$ git rebase -q main
$ git log --oneline -1 ORIG_HEAD
68e6fff Rerank the top 20
```

Each time the value is the tip from immediately before the command. The trap is the other list. After a command that does not write the slot, the slot still holds the value from an earlier one:

```
# A later command that does not write ORIG_HEAD leaves the old value in place:
$ git switch -q main
$ git cherry-pick feature/rerank > /dev/null
$ git log --oneline -1 ORIG_HEAD
68e6fff Rerank the top 20
$ git branch --contains ORIG_HEAD
$ cat .git/ORIG_HEAD
68e6fff0a6256f993946e89883a38d029db3ccea
$ git reflog exists ORIG_HEAD
[exit status: 1]
```

You are on `main`, you have cherry-picked a commit, and `ORIG_HEAD` names a commit of the feature branch from before its rebase, which no branch contains any more. `git reset --hard ORIG_HEAD` at this moment would move `main` onto abandoned history.

Writes <code>ORIG_HEAD</code>	Does not write it
<code>git reset</code> (every mode), <code>git merge</code> (also a fast-forward), <code>git rebase</code> , <code>git am</code> , <code>git pull</code> , <code>git stash push</code>	<code>git commit</code> , <code>git commit --amend</code> , <code>git switch</code> , <code>git checkout</code> , <code>git cherry-pick</code> , <code>git revert</code> , <code>git fetch</code> , <code>git push</code>

The left column is from the manual and from the transcripts of this book. The right column is observed: the transcript above for `commit`, `amend`, `switch`, `cherry-pick` and `revert`, and the lab fixtures for the rest (the repository of Lab 12.7 was built with `git checkout` and commits, the clone of Lab 12.2 with commits, a `fetch` and a `push`, and neither has a `.git/ORIG_HEAD`).

In production. Use `ORIG_HEAD` only as the very next command after the operation you want to undo, and only if that operation is in the left column. In every other case read the reflog of the branch. The rebase manual gives the same advice for long rebases: `ORIG_HEAD` "is not guaranteed to still point to that commit at the end of the rebase if other commands that change `ORIG_HEAD` (like `git reset`) are used during the rebase", while the previous tip stays available as `@{1}` (`git help rebase`).

13.6 `git fsck` as a search tool

In one sentence. `git fsck` walks the object database from every starting point and reports what it cannot reach, which makes it the tool for finding work that no ref and no reflog names.

Precisely. Chapter 3, section 3.8 introduced the command as an integrity check. For recovery, four options matter (`git help fsck`). 🟢 **SAFE**, except that `--lost-found` writes files into `.git/lost-found`.

Option	Effect
<code>--dangling</code> (the default)	Report unreachable objects that no other object refers to: the tips of lost history, and blobs that lost their index entry
<code>--unreachable</code>	Report every unreachable object, including the ancestors, trees and blobs behind a dangling commit
<code>--no-reflogs</code>	Do not count reflog entries as starting points. The manual: "meant only to search for commits that used to be in a ref, but now aren't, but are still in that corresponding reflog"
<code>--lost-found</code>	In addition, write each dangling commit's ID into <code>.git/lost-found/commit/</code> and each dangling blob's <i>content</i> into <code>.git/lost-found/other/</code>

The difference between "dangling" and "unreachable" is the difference between a tip and a chain. If three commits are lost, all three are unreachable, and only the newest is dangling, because the newest still refers to the other two. For recovery you want dangling commits: anchor the tip and the chain comes with it.

See it. A branch with three commits is deleted. A file was staged earlier and then thrown away by a reset.

```
$ git branch -D exp/hybrid
Deleted branch exp/hybrid (was 2ae0c97).
$ git fsck
dangling blob 8fb6e9c8eb419f470869b10b7fcc64cf510486a0
```

By default `git fsck` reports only the blob. The three commits are still reachable from the HEAD reflog, because the branch was checked out when they were made. That default is what you want when you ask "what is at risk of being collected?". When you ask "what is no longer on any branch?", take the reflogs out:

```
$ git fsck --no-reflogs
dangling blob 8fb6e9c8eb419f470869b10b7fcc64cf510486a0
dangling commit 2ae0c97ba880a6defccc70c29261e46cb2d37bdf
$ git fsck --no-reflogs --unreachable
unreachable blob 8fb6e9c8eb419f470869b10b7fcc64cf510486a0
unreachable blob d101568e3177507ffec6dd9a56c478cce1751b2f
unreachable tree 918936da64f9772a5456d5940fb80ffa0d220346
unreachable tree 17b30d538538f6762ec168c66bcaae783356477
unreachable commit 53c5b13f7e8f3f23e8a4363f15be846e93ff5862
unreachable blob 257d9cc0829b942c09bd1d17b1ec8490ec8bd2b1
unreachable commit 2ae0c97ba880a6defccc70c29261e46cb2d37bdf
unreachable tree b08f744630034d54db33226cb5fff7e5a816fb87
unreachable blob 34ddd608dd3b612063f7806ca172b6ab36381925
unreachable commit b9975128858772d81064c041f60f4944ec3f428f
```

One dangling commit, `2ae0c97`, stands for three unreachable commits, three trees and three more blobs. In a repository that has been used for months, the list of dangling commits is long: every amend, every rebase and every dropped stash leaves one. Turn the list into something you can read, newest first, with author and subject:

```
$ git fsck --no-reflogs | awk '/dangling commit/ {print $3}' | xargs git log --no-walk --date=iso --format='%h %cd %an %s'
2ae0c97 2026-09-07 10:09:00 +0530 Lab User Tune alpha to 0.6
$ git log --oneline 2ae0c97 --not --all
2ae0c97 Tune alpha to 0.6
53c5b13 Add hybrid weights
b997512 Add dense scorer
```

The second command shows the whole lost line: commits reachable from the found tip and from no ref. `--lost-found` saves the same findings as files, which is convenient for blobs, since a blob has no name and no date and can only be recognised by its content:

```
$ git fsck --no-reflogs --lost-found
dangling blob 8fb6e9c8eb419f470869b10b7fcc64cf510486a0
dangling commit 2ae0c97ba880a6defccc70c29261e46cb2d37bdf
$ find .git/lost-found -type f | sort
.git/lost-found/commit/2ae0c97ba880a6defccc70c29261e46cb2d37bdf
.git/lost-found/other/8fb6e9c8eb419f470869b10b7fcc64cf510486a0
$ cat .git/lost-found/other/*
alpha: [0.5, 0.6, 0.7]
```

A found commit is still unreachable. Reading it does not protect it. Give it a ref:

```
$ git branch rescue/hybrid 2ae0c97
$ git fsck --no-reflogs
dangling blob 8fb6e9c8eb419f470869b10b7fcc64cf510486a0
$ git log --oneline main..rescue/hybrid
2ae0c97 Tune alpha to 0.6
53c5b13 Add hybrid weights
b997512 Add dense scorer
```

`git branch <name> <id>`  writes one ref and changes nothing else. From this moment the three commits are on layer 1, and no expiry or collection can touch them.

Picture.

b997512 <--	53c5b13 <--	2ae0c97	8fb6e9c
unreachable	unreachable	unreachable and	a blob: unreachable and dangling
		DANGLING (nothing	(it was staged, then the index entry was reset)
		refers to it)	

In production. `git fsck` reads every object, so on a large repository it is slow; `--connectivity-only` skips reading blobs and is enough for a search. Since Git 2.53 it also ignores relog entries dated after the moment it starts (Chapter 3, section 3.8), so a machine whose clock was once wrong can report "dangling" for commits the relog still lists. Dangling is information, not damage: only lines that begin with `missing`, `broken` or `error` describe a repository that needs repair (section 13.11).

13.7 The recovery method

Every recovery in the rest of this chapter follows the same eight steps. They are the root-cause framework of Chapter 1, section 1.10, specialised for loss.

1. **Stop.** Do not run another command that moves refs or deletes files, and do not run `git gc`, `git prune`, `git clean` or `git stash clear`. Waiting loses nothing; a second "fix" can.
2. **Record the symptom and the last commands.** Copy the terminal scrollbar if you have it. Git prints the ID you need in more places than people remember: Deleted branch x (was 2f65488), Dropped refs/stash@{0} (...), HEAD is now at ..., the warning on leaving detached HEAD.
3. **Classify.** Was the work committed, stashed, staged, or only saved? That decides the layer (section 13.2) and whether to continue with Git at all.
4. **Find the ID.** From the scrollbar, the branch relog, the HEAD relog, `0RIG_HEAD`, `git fsck`, or another repository, in that order.

5. **Anchor it.** `git branch rescue/<what> <id>`. An anchored commit cannot expire.
6. **Inspect before you move anything.** `git log`, `git show --stat`, `git diff`, `git range-diff`, `git cherry` against the anchor.
7. **Integrate with the lowest-risk move.** Prefer adding (a branch, a cherry-pick, a merge) over rewinding, and `git reset --keep` over `git reset --hard` (Chapter 11, section 11.6).
8. **Verify, then clean up.** Prove that the result holds what was lost, and only then delete the rescue branch.

Picture. Where to look, as a decision flow:

```

Was the lost work ever committed, stashed, or staged with "git add"?
|
+-- no ----> Git has no object. Editor history, IDE local history, backups, Time Machine.    (13.12)
|
+-- staged only ----> git fsck --lost-found ; look in .git/lost-found/other                (13.9)
|
+-- stashed, then dropped or cleared ----> git fsck --unreachable | ... git log --merges    (13.9)
|
+-- committed
    |
    +-- Do you have the ID (scrollback, CI log, pull request, chat)? -- yes --> git branch rescue <id>
    |
    +-- Did a branch move (reset, rebase, merge, amend, cherry-pick)?
        |
        --> git reflog show <branch> ; take the entry below the bad operation                (13.8)
    |
    +-- Is the branch itself gone, or was the work done in detached HEAD?
        |
        --> git reflog (HEAD) ; or git fsck --no-reflogs for the dangling tips            (13.8)
    |
    +-- Did the server lose it (force push, deleted remote branch)?
        |
        --> git reflog show origin/<branch> ; a teammate's clone ; the host's tools        (13.10, 13.15)
    |
    +-- No reflog entry anywhere (pruned fetch, expired log, other machine)?
        |
        --> git fsck --lost-found ; then another clone, a bundle, a backup                (13.6, 13.14)

```

What the commands of a recovery change, so that you know how little they risk:

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git branch rescue/x <id></code> 🟢	unchanged	unchanged	unchanged	unchanged	New ref <code>refs/heads/rescue/x</code> with a new reflog	unchanged	unchanged
<code>git fsck --lost-found</code> 🟢	unchanged	unchanged	unchanged	unchanged	Files written under <code>.git/lost-found/</code>	unchanged	unchanged
<code>git cherry-pick <id></code> 🟡	Updated to the new commit	Updated	Moves with the branch	Advances by one new commit	Reflog entries; <code>CHERRY_PICK_HEAD</code> during a conflict	unchanged	unchanged
<code>git reset --keep <id></code> 🟡	Updated to <code><id></code> ; refuses if a file with local changes differs between the two commits	Updated	Moves with the branch	Set to <code><id></code>	<code>ORIG_HEAD</code> written; reflog entries	unchanged	unchanged
<code>git reset --hard <id></code> 🔴	Overwritten to match <code><id></code> ; uncommitted changes to tracked files destroyed	Overwritten	Moves with the branch	Set to <code><id></code>	<code>ORIG_HEAD</code> written; reflog entries	unchanged	unchanged
<code>git stash store <id></code> 🟢	unchanged	unchanged	unchanged	unchanged	<code>refs/stash</code> set to <code><id></code> , one reflog line added	unchanged	unchanged

13.8 Recovering commits

Nine accidents that involve committed work. In all of them the work is on layer 2 or 3, and all nine end with the same act: a ref is pointed at a commit that never stopped existing.

An accidental reset without `--hard`

Symptom. You typed `git reset HEAD~2` where you meant something smaller. The log is shorter, and `git status` shows modified files.

```

# The same slip without --hard: the branch moves, the files do not.
$ git reset HEAD~2
Unstaged changes after reset:
M   config.yaml
M   retriever.py
$ git log --oneline -1
798cb66 Add recall@k evaluation
$ git status -s
M config.yaml
M retriever.py
$ cat config.yaml
top_k: 10
$ git reset 'HEAD@{1}'
$ git status -s

```



```
$ git log --oneline -1
3106f68 Normalise queries before search
```

Why nothing was lost. A mixed reset moves the branch and the index and leaves the working tree alone (Chapter 11, section 11.4). The content of the two "missing" commits is still in your files, shown as modifications. `HEAD@{1}` is the commit the branch pointed at before, and a second mixed reset to it puts the branch and the index back. The working tree already matched that commit, so the status is clean. A soft reset is undone the same way.

An accidental hard reset

Symptom. One wrong digit: `HEAD~3` instead of `HEAD~1`. Three commits and their changes are gone from the branch and from the working tree.

```
# The plan: drop the last commit. The typo: 3 instead of 1.
$ git reset --hard HEAD~3
HEAD is now at 798cb66 Add recall@k evaluation
$ git log --oneline
798cb66 Add recall@k evaluation
8d2e200 Add retrieval config
7525edc Add BM25 retriever
$ cat config.yaml
top_k: 5
```

Evidence. Three places name the old tip, and they agree:

```
$ git reflog -3
798cb66 HEAD@{0}: reset: moving to HEAD~3
495f3a9 HEAD@{1}: commit: WIP: debug prints
3106f68 HEAD@{2}: commit: Normalise queries before search
$ git log --oneline -1 ORIG_HEAD
495f3a9 WIP: debug prints
$ git reflog show main -2
798cb66 main@{0}: reset: moving to HEAD~3
495f3a9 main@{1}: commit: WIP: debug prints
```

Recovery. Anchor, look, move:

```
$ git branch rescue/before-reset ORIG_HEAD
$ git log --oneline rescue/before-reset
495f3a9 WIP: debug prints
3106f68 Normalise queries before search
13ed88d Raise top_k to 10
798cb66 Add recall@k evaluation
8d2e200 Add retrieval config
7525edc Add BM25 retriever
$ git reset --hard rescue/before-reset
HEAD is now at 495f3a9 WIP: debug prints
$ git log --oneline -3
495f3a9 WIP: debug prints
3106f68 Normalise queries before search
13ed88d Raise top_k to 10
$ cat config.yaml
top_k: 10
```

`ORIG_HEAD` was safe to use here, because `git reset` wrote it and nothing has run since. The lost commits were on layer 2 for the whole time. What a hard reset destroys for good is uncommitted work, and that is section 13.9.

The usual complication. If you committed new work on the shortened branch before you noticed, a reset back to the old tip trades one loss for another. Anchor both tips, reset to the one you want as the base, and `git cherry-pick` the commits from the other (Lab 12.1).

A deleted branch

Symptom. `git branch -D`  on a branch that was not merged.

```
$ git branch -d feature/cross-encoder
error: the branch 'feature/cross-encoder' is not fully merged
hint: If you are sure you want to delete it, run 'git branch -D feature/cross-encoder'
hint: Disable this message with "git config set advice.forceDeleteBranch false"
[exit status: 1]
$ git branch -D feature/cross-encoder
Deleted branch feature/cross-encoder (was 2f65488).
$ git branch
* main
$ git log --oneline --graph --all
* 6f16e54 Document the pipeline stages
| * 60184f9 Deduplicate mined negatives
| * 0cea4ee Mine hard negatives from click logs
|/
* 0875e65 Add pipeline config
* f9b487c Add retrieval pipeline
```

Git refused the safe form, `-d`, because the branch was "not fully merged": that refusal is the first safety net. `-D` skips the check. It also prints the one fact you need, (was 2f65488).

Recovery. A branch is a ref, so recreating the ref recreates the branch:

```
$ git branch feature/cross-encoder 2f65488
$ git log --oneline main..feature/cross-encoder
2f65488 Cache reranker scores
405080d Rerank the top 50 candidates
7e43259 Add cross-encoder reranker
$ git reflog show feature/cross-encoder
2f65488 feature/cross-encoder@{0}: branch: Created from 2f65488
```

The three commits are back. The reflog of the branch is not: it has one line, `branch: Created from 2f65488`. The record of how the branch moved over its life was deleted with it and cannot be restored (section 13.12).

Without the printed ID. The HEAD reflog has an entry for the moment you last left the branch:

```
$ git reflog show feature/cross-encoder
fatal: ambiguous argument 'feature/cross-encoder': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
[exit status: 128]
$ git reflog -6
6f16e54 HEAD@{0}: commit: Document the pipeline stages
0875e65 HEAD@{1}: checkout: moving from feature/cross-encoder to main
2f65488 HEAD@{2}: commit: Cache reranker scores
405080d HEAD@{3}: commit: Rerank the top 50 candidates
7e43259 HEAD@{4}: commit: Add cross-encoder reranker
0875e65 HEAD@{5}: checkout: moving from main to feature/cross-encoder
```

`checkout: moving from feature/cross-encoder to main` records where HEAD went. The entry below it, `HEAD@{2}`, is the last thing HEAD pointed at while it was on the branch: the tip, unless the branch was moved afterwards without being checked out.

When no reflog knows the commits. A teammate's branch that you fetched and never checked out is held by a remote-tracking ref only. If she deletes the branch on the server and you fetch with `--prune`, that ref and its reflog go:

```
$ git branch -r
origin/HEAD -> origin/main
origin/exp/hard-negatives
origin/main
$ git log --oneline -2 origin/exp/hard-negatives
60184f9 Deduplicate mined negatives
0cea4ee Mine hard negatives from click logs
$ git fetch --prune
From ../server
- [deleted]          (none)    -> origin/exp/hard-negatives
$ git branch -r
origin/HEAD -> origin/main
origin/main
$ git reflog show origin/exp/hard-negatives
fatal: ambiguous argument 'origin/exp/hard-negatives': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
[exit status: 128]
$ git reflog | grep -c negatives
0
```

Git printed `(none)` where a local deletion prints the old ID. The commits are on layer 3 now, and `git fsck` finds the tip without any option, because no reflog entry competes:

```
$ git fsck
dangling commit 60184f9abb50a1f151181c08df375e5274ead3c6
$ git fsck --lost-found
dangling commit 60184f9abb50a1f151181c08df375e5274ead3c6
$ git log --format='%h %an, %ad%n' --date=short $(cat .git/lost-found/commit/*) --not --all
60184f9 Asha Rao, 2026-09-07
    Deduplicate mined negatives
0cea4ee Asha Rao, 2026-09-07
    Mine hard negatives from click logs
```

```
$ git branch exp/hard-negatives $(cat .git/lost-found/commit/*)
$ git log --oneline main..exp/hard-negatives
60184f9 Deduplicate mined negatives
0cea4ee Mine hard negatives from click logs
$ git push -u origin exp/hard-negatives
To ../server.git
 * [new branch]      exp/hard-negatives -> exp/hard-negatives
branch 'exp/hard-negatives' set up to track 'origin/exp/hard-negatives'.
```

`--not --all` limited the listing to commits that no ref reaches, which is the lost work and nothing else. Pushing the branch puts it on layer 4 as well. In a repository with default settings this rescue has a deadline: the grace period, counted from the day the objects were written into your

repository, and then the next collection.

A deleted commit

Symptom. A commit that you remember is not in the branch, and a setting it introduced is not in the file. The branch has newer commits, so the loss is not recent.

```
$ cd embedder
$ git log --oneline main..feature/batching
a7061c1 Document batching in the README
0309a41 Log batch sizes
285e3af Retry on rate limit
696a15f Batch embedding requests
$ cat client.yaml
model: embed-small-v2
$ git log --oneline --all -- client.yaml
f7736c7 Add client config
```

`git log --all -- client.yaml` knows one commit for that file. The timeout commit is on no branch.

Evidence. Commits leave a branch through three commands: `git reset`, `git rebase` (a drop line, or a line deleted from the todo list), and `git commit --amend`. The reflog of the branch says which one ran:

```
$ git reflog show feature/batching
a7061c1 feature/batching@{0}: commit: Document batching in the README
0309a41 feature/batching@{1}: rebase (finish): refs/heads/feature/batching onto f7736c7d7ea7d8c345ef1888de925509d5117e39
76a49ed feature/batching@{2}: commit: Log batch sizes
f35b47a feature/batching@{3}: commit: Retry on rate limit
2ebd700 feature/batching@{4}: commit: Add request timeout
696a15f feature/batching@{5}: commit: Batch embedding requests
f7736c7 feature/batching@{6}: branch: Created from HEAD
```

One `rebase (finish)` line, with the old tip `76a49ed` directly below it and the lost commit `2ebd700` two lines further down. To prove what the rebase changed, anchor the old tip and compare by content. `git cherry` marks with `+` the commits whose change has no equivalent on the other side (Chapter 10, section 10.10):

```
# Anchor the tip from before the rebase, then ask what the rebase left out:
$ git branch rescue/before-rebase 'feature/batching@{2}'
$ git log --oneline main..rescue/before-rebase
76a49ed Log batch sizes
f35b47a Retry on rate limit
2ebd700 Add request timeout
696a15f Batch embedding requests
$ git cherry -v feature/batching rescue/before-rebase
+ 2ebd700e9eb436ad1c04d80c284f18f294fc5011 Add request timeout
- f35b47a4e60533a59aba361e16d42aa94602ee05 Retry on rate limit
- 76a49edc7fb8cf1ba56ecd367f05eb1dae9ec89 Log batch sizes
```

Two commits were rebuilt with new IDs and the same change (`-`). One was left out (`+`).

Recovery. Do not move the branch back: a commit was added after the rebase, and a reset would lose it. Copy the one missing commit forward:

```
$ git show --stat --format='%h %s' 2ebd700
2ebd700 Add request timeout

client.yaml | 1 +
1 file changed, 1 insertion(+)
$ git cherry-pick 2ebd700
[feature/batching ff46c25] Add request timeout
Date: Mon Sep 7 10:07:00 2026 +0530
1 file changed, 1 insertion(+)
$ cat client.yaml
model: embed-small-v2
timeout_s: 10
```

The amend variant. `git commit --amend` replaces the tip. If you meant to add a commit, the previous commit has disappeared into the new one. The reflog has the commit from before the amend, and a soft reset to it leaves exactly the new change staged:

```
# A typo fix that was meant to be a commit of its own:
$ printf 'Embedding client. Requests are sent in batches of 32.\n' > README.md
$ git commit -a --amend -m "Fix typo in README"
[feature/batching 50df155] Fix typo in README
Date: Mon Sep 7 10:07:00 2026 +0530
2 files changed, 2 insertions(+), 1 deletion(-)
$ git log --oneline -3
50df155 Fix typo in README
a7061c1 Document batching in the README
0309a41 Log batch sizes
$ git show --stat --format=%s HEAD
Fix typo in README
```

```
README.md | 2 +-
client.yaml | 1 +
2 files changed, 2 insertions(+), 1 deletion(-)
```

```
$ git reflog -2
50df155 HEAD@{0}: commit (amend): Fix typo in README
ff46c25 HEAD@{1}: cherry-pick: Add request timeout
$ git reset --soft 'HEAD@{1}'
$ git status -s
M README.md
$ git diff --cached
diff --git a/README.md b/README.md
index 202d75c..5c95234 100644
--- a/README.md
+++ b/README.md
@@ -1,1 @@
-Embedding client. Requests are sent in batchs of 32.
+Embedding client. Requests are sent in batches of 32.
$ git commit -m "Fix typo in README"
[feature/batching fdbbfc7] Fix typo in README
1 file changed, 1 insertion(+), 1 deletion(-)
```

A soft reset moves the branch only. The index still holds the tree of the amended commit, so the difference between the two commits, the typo fix, is what is staged.

A wrong rebase

Symptom. A hotfix branch that was cut from `release/1.4` contains commits that belong to `main`. A pull request against the release branch would list six commits instead of three.

```
$ cd evalharness
$ git status -sb
## hotfix/judge-timeout
$ git log --oneline release/1.4..HEAD
37bb577 Log judge latency
6ef447a Retry judge on timeout
262dc41 Add judge timeout
7213db9 Pin judge model for 1.4
5783ca4 Add faithfulness metric
9288f20 Switch judge to JSON mode
$ git log --oneline --graph --all
* 37bb577 Log judge latency
* 6ef447a Retry judge on timeout
* 262dc41 Add judge timeout
* 7213db9 Pin judge model for 1.4
* 5783ca4 Add faithfulness metric
* 9288f20 Switch judge to JSON mode
| * dc1ff44 Pin judge model for 1.4
|/
* 1d7a7cc Add judge prompt
* cd80329 Add eval runner
```

`git rebase main`, typed by habit, replayed the branch onto `main`: the two hotfix commits and the release-only commit below them, all with new IDs. Chapter 9, section 9.16 covers the simple case, where `git reset --hard ORIG_HEAD` or the branch reflog undoes a rebase that was noticed at once. Here a commit was added after the rebase.

Evidence and anchor.

```
$ git reflog show hotfix/judge-timeout
37bb577 hotfix/judge-timeout@{0}: commit: Log judge latency
6ef447a hotfix/judge-timeout@{1}: rebase (finish): refs/heads/hotfix/judge-timeout onto 5783ca45370007569bfb3217980f447300484db9
eb1f140 hotfix/judge-timeout@{2}: commit: Retry judge on timeout
c5f4fb7 hotfix/judge-timeout@{3}: commit: Add judge timeout
dc1ff44 hotfix/judge-timeout@{4}: branch: Created from HEAD
```

```
$ git branch rescue/pre-rebase 'hotfix/judge-timeout@{2}'
$ git log --oneline release/1.4..rescue/pre-rebase
eb1f140 Retry judge on timeout
c5f4fb7 Add judge timeout
$ git branch backup/rebased-hotfix
```

Two refs now hold the two versions of the branch: `rescue/pre-rebase` is the tip from before the rebase, and `backup/rebased-hotfix` is the present state. Nothing can be lost by what follows.

Recovery. Move only the commit that was made after the rebase onto the old tip. `git rebase --onto <new base> <upstream>` replays the commits after `<upstream>` (Chapter 9, section 9.5):

```
$ git rebase --onto rescue/pre-rebase HEAD~1
Rebasing (1/1)
Successfully rebased and updated refs/heads/hotfix/judge-timeout.
$ git log --oneline release/1.4..HEAD
```

```
13305ef Log judge latency
eb1f140 Retry judge on timeout
c5f4fb7 Add judge timeout
```

The branch is on `release/1.4` again, with three commits. A wrong rebase is repaired with a correct rebase.

A bad merge

Three situations, three different tools.

The merge is	Tool	Reference
In progress, with conflicts you do not want to resolve	<code>git merge --abort</code>	Chapter 8
Committed, not pushed	<code>git reset --keep ORIG_HEAD</code> , or the branch reflog	Below
Pushed to a shared branch	<code>git revert -m 1 <merge></code> , and know the re-merge problem	Chapter 11, section 11.9

Symptom. The wrong branch was merged into `main`, and the merge was a fast-forward:

```
# The branch you meant to merge is feature/citations.
$ git merge exp/few-shot
Updating 0529d31..deec1c6
Fast-forward
 examples.txt | 4 +++
 system.txt   | 1 -
 2 files changed, 4 insertions(+), 1 deletion(-)
 create mode 100644 examples.txt
$ git log --oneline
deec1c6 WIP: drop the refusal rule
df44d03 WIP: longer examples
efee0cd Try few-shot examples
0529d31 Add prompt loader
5aab090 Add system prompt
$ cat system.txt
You answer questions about our product documentation.
```

Evidence.

```
$ git reflog -2
deec1c6 HEAD@{0}: merge exp/few-shot: Fast-forward
0529d31 HEAD@{1}: checkout: moving from exp/few-shot to main
$ git log --oneline -1 ORIG_HEAD
0529d31 Add prompt loader
$ git log --oneline ORIG_HEAD..HEAD
deec1c6 WIP: drop the refusal rule
df44d03 WIP: longer examples
efee0cd Try few-shot examples
```

`ORIG_HEAD..HEAD` lists what the merge brought in: three commits, and no merge commit among them.

Recovery.

```
$ git reset --keep ORIG_HEAD
$ git log --oneline
0529d31 Add prompt loader
5aab090 Add system prompt
$ git merge feature/citations
Updating 0529d31..3aae83f
Fast-forward
 system.txt | 1 +
 1 file changed, 1 insertion(+)
```

`--keep` instead of `--hard`: it moves the branch and the files in the same way and refuses if that would overwrite an uncommitted change. The experiment branch still exists; only `main` moved back.

The recipe that many tutorials give for "undo the last merge" is `git reset --hard HEAD~1`. For a fast-forward it is wrong:

```
$ cd ../promptstore-incident
$ git log --oneline
deec1c6 WIP: drop the refusal rule
df44d03 WIP: longer examples
efee0cd Try few-shot examples
0529d31 Add prompt loader
5aab090 Add system prompt
# The recipe from a tutorial: "undo the merge" by dropping one commit.
$ git reset --hard HEAD~1
HEAD is now at df44d03 WIP: longer examples
$ git log --oneline
df44d03 WIP: longer examples
efee0cd Try few-shot examples
0529d31 Add prompt loader
5aab090 Add system prompt
# Still two experiment commits. The other recipe:
```

```
$ git reset --hard ORIG_HEAD
HEAD is now at deec1c6 WIP: drop the refusal rule
$ git log --oneline
deec1c6 WIP: drop the refusal rule
df44d03 WIP: longer examples
efee0cd Try few-shot examples
0529d31 Add prompt loader
5aab090 Add system prompt
```

```
Observed behavior : "git reset --hard HEAD~1" after a merge leaves two of the three merged commits on main;
                  "git reset --hard ORIG_HEAD" then brings the third one back
Git state         : the merge was a fast-forward: main moved from 0529d31 to deec1c6 along three commits,
                  and no merge commit was created
Mechanism         : HEAD~1 is the first parent of the tip. After a true merge that is the previous tip of
                  main. After a fast-forward it is the previous commit of the branch that was merged.
                  The first reset wrote ORIG_HEAD = deec1c6, so the second one returned to the merged state
Root cause        : the recipe assumes a merge commit, and "ORIG_HEAD" was used two commands late
Why Git does this : a fast-forward only moves a ref (Chapter 8); there is nothing to mark where it started
                  except the reflog and, for one command, ORIG_HEAD
Correct fix       : read "git reflog show main" and reset to the entry below the merge line
Prevention        : undo a merge with ORIG_HEAD immediately, or by reflog entry; never by counting parents
```

```
$ git reflog show main
deec1c6 main@{0}: reset: moving to ORIG_HEAD
df44d03 main@{1}: reset: moving to HEAD~1
deec1c6 main@{2}: merge exp/few-shot: Fast-forward
0529d31 main@{3}: commit: Add prompt loader
5aab090 main@{4}: commit (initial): Add system prompt
```

The line `merge exp/few-shot: Fast-forward` is the accident. The line below it, `main@{3}`, is where `main` was before.

Commits on the wrong branch

Symptom. You committed twice on `main` while you believed you were on the feature branch. Nothing is pushed.

```
$ cd ingest
$ git status -sb
## main...origin/main [ahead 2]
$ git log --oneline --graph --all
* c8fec4d Add a table fixture for tests
* 0eece93 Keep table rows together when chunking
| * fb8cb6c Parse PDF tables
|/
* 4711dcc Add chunker
* 5a80a30 Add document loader
$ git log --oneline origin/main..main
c8fec4d Add a table fixture for tests
0eece93 Keep table rows together when chunking
```

`origin/main..main` is the precise name of the misplaced work: the commits on `main` that the server does not have.

Recovery. Nothing is lost here; the commits have the wrong name on them. Copy first, remove second:

```
$ git switch feature/pdf-tables
Switched to branch 'feature/pdf-tables'
$ git cherry-pick origin/main..main
[feature/pdf-tables c7f48b2] Keep table rows together when chunking
Date: Mon Sep 7 10:11:00 2026 +0530
1 file changed, 3 insertions(+)
[feature/pdf-tables ed2177f] Add a table fixture for tests
Date: Mon Sep 7 10:12:00 2026 +0530
1 file changed, 1 insertion(+)
create mode 100644 fixtures/table.json
$ git log --oneline -3
ed2177f Add a table fixture for tests
c7f48b2 Keep table rows together when chunking
fb8cb6c Parse PDF tables
```

```
$ git switch main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 2 commits.
(use "git push" to publish your local commits)
$ git reset --keep origin/main
$ git status -sb
## main...origin/main
```

If the target branch does not exist yet, the copy is not needed: `git branch feature/new` creates a ref at the current commit, and `git reset --keep origin/main` then moves `main` back. The order matters in both forms. If you reset `main` first, the two commits are held by the reflog only, and you recover them as `origin/main..main@{1}` (Lab 12.6 does it in that order on purpose). If the commits were already pushed to a shared `main`, do not rewind it: cherry-pick them to the feature branch and revert them on `main` (Chapter 11, section 11.8).

Lost work in detached HEAD

Symptom. Work that you did "a few days ago" is on no branch. You remember checking out a tag.

A commit made in detached HEAD belongs to no branch, so only the HEAD reflog records it (Chapter 7, section 7.7). Git warns when you leave such commits behind, and Chapter 7's Lab 4.2 rescues them from that warning. This is the case where the warning scrolled away long ago:

```
$ git reflog
f4a81dc HEAD@{0}: commit: Document the endpoints
20459b3 HEAD@{1}: checkout: moving from 852224fe93ed50676b587ace22cee739d967505f to main
852224f HEAD@{2}: commit: Experiment: disable response cache
f6f6bbb HEAD@{3}: checkout: moving from main to HEAD~1
20459b3 HEAD@{4}: commit: Export latency metrics
f6f6bbb HEAD@{5}: checkout: moving from 266d3b2a96a6a113c06fe5974b09562b1d3f440b to main
266d3b2 HEAD@{6}: commit: Hotfix: reject empty prompts
ba8c3d9 HEAD@{7}: commit: Hotfix: cap max_tokens at 4096
be53ce8 HEAD@{8}: checkout: moving from main to v1.2.0
f6f6bbb HEAD@{9}: commit: Add streaming endpoint
be53ce8 HEAD@{10}: commit: Add request limits
3838324 HEAD@{11}: commit (initial): Add inference endpoint
```

The marks of a detached session are a `checkout: moving from main to <tag or commit>` line, commits above it, and a `checkout: moving from <forty hexadecimal digits> to main` line that ends it. There are two sessions in this log. Reading reflogs of real length for such patterns is slow. `git fsck --no-reflogs` goes straight to the result: the tips that no branch and no tag reaches.

```
$ git fsck --no-reflogs
dangling commit 852224fe93ed50676b587ace22cee739d967505f
dangling commit 266d3b2a96a6a113c06fe5974b09562b1d3f440b
$ git log --oneline --graph 266d3b2 852224f --not --all
* 852224f Experiment: disable response cache
* 266d3b2 Hotfix: reject empty prompts
* ba8c3d9 Hotfix: cap max_tokens at 4096
```

Two dangling commits, and the graph shows what hangs on each: a line of two hotfix commits and a single experiment.

Recovery.

```
$ git branch hotfix/1.2.1 266d3b2
$ git branch exp/no-response-cache 852224f
$ git log --oneline --graph --all
* f4a81dc Document the endpoints
* 20459b3 Export latency metrics
| * 852224f Experiment: disable response cache
|/
* f6f6bbb Add streaming endpoint
| * 266d3b2 Hotfix: reject empty prompts
| * ba8c3d9 Hotfix: cap max_tokens at 4096
|/
* be53ce8 Add request limits
* 3838324 Add inference endpoint
```

Each line has a name now. What happens to the hotfixes next is an ordinary decision, made without time pressure.

A wrong cherry-pick

Symptom. The commit that landed on the release branch is a feature, not the fix:

```
# The fix to backport is "Fix off-by-one in rate limit window". The newest commit on main is not it.
$ git cherry-pick -x main
[release/2.1 1bccf8e] Add per-tenant quotas
Date: Mon Sep 7 10:08:00 2026 +0530
2 files changed, 2 insertions(+)
create mode 100644 quotas.yaml
$ git log --oneline -3
1bccf8e Add per-tenant quotas
e30eec3 Pin dependencies for 2.1
3171b7b Add request router
$ git show --stat --format=%B HEAD
Add per-tenant quotas

(cherry picked from commit cad5d75479789f69b0c584495258e2ad3f7ce87f)

quotas.yaml | 1 +
router.py   | 1 +
2 files changed, 2 insertions(+)
```

`-x` recorded the source, `cad5d75`, which makes the mistake visible in review. The pick itself was clean, and that is the dangerous case: a wrong pick that conflicts gets attention, and a clean one ships.

Recovery. While the pick is still in progress (it stopped on a conflict), `git cherry-pick --abort` returns to the state before it (Chapter 10, section 10.6). After it has finished and before a push, step the branch back by one reflog entry and pick the right commit:

```
$ git reset --keep 'HEAD@{1}'
$ git log --oneline -2
e30eec3 Pin dependencies for 2.1
3171b7b Add request router
$ git cherry-pick -x main~1
[release/2.1 5564cff] Fix off-by-one in rate limit window
Date: Mon Sep 7 10:06:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
```

After a push to a shared release branch, `git revert` the wrong commit instead.

The reflex that fails here is `git reset --hard ORIG_HEAD`:

```
$ cd ../apigw-incident
$ git log --oneline -3
48b4def Add per-tenant quotas
e30eec3 Pin dependencies for 2.1
3171b7b Add request router
# The wrong pick is already here. A recipe remembered from the merge chapter:
$ git reset --hard ORIG_HEAD
HEAD is now at f028350 Fix off-by-one in rate limit window
$ git log --oneline
f028350 Fix off-by-one in rate limit window
3171b7b Add request router
239cf05 Add rate limiter
$ git status -sb
## release/2.1
$ ls
limiter.py
router.py
```

```
Observed behavior : after "git reset --hard ORIG_HEAD" the release branch shows the history of main,
                    and the release-only commit "Pin dependencies for 2.1" is gone
Git state          : ORIG_HEAD = f028350, written days earlier by a fast-forward merge on main
Mechanism          : git cherry-pick does not write ORIG_HEAD (section 13.5), so the slot held a value from
                    another operation on another branch; the reset moved release/2.1 to that commit
Root cause        : ORIG_HEAD was used as "undo the last command"; it means "before the last reset,
                    merge, rebase or am"
Why Git does this : ORIG_HEAD is one file with no notion of which branch or which command it belongs to
Correct fix       : git reflog show release/2.1, then reset --keep to the entry below the cherry-pick
Prevention       : undo by reflog entry; check "git log -1 ORIG_HEAD" before every use of ORIG_HEAD
```

```
$ git reflog show release/2.1
f028350 release/2.1@{0}: reset: moving to ORIG_HEAD
48b4def release/2.1@{1}: cherry-pick: Add per-tenant quotas
e30eec3 release/2.1@{2}: commit: Pin dependencies for 2.1
3171b7b release/2.1@{3}: branch: Created from main
$ git reflog -4
f028350 HEAD@{0}: reset: moving to ORIG_HEAD
48b4def HEAD@{1}: cherry-pick: Add per-tenant quotas
e30eec3 HEAD@{2}: commit: Pin dependencies for 2.1
3171b7b HEAD@{3}: checkout: moving from main to release/2.1
```

The result looked plausible, which is what makes this failure expensive: the tip was the very fix that should have been picked. Only the missing file `requirements.txt` gave it away. In the branch reflog, `release/2.1@{2}` is the state before the wrong pick.

13.9 Recovering uncommitted work

Staged, never committed, then overwritten

Symptom. `git reset --hard` (or `git restore --staged --worktree`, or a checkout with `-f`) ran over work that was not committed.

```
$ cd finetune
$ git status -s
A  sweep.yaml
MM train.yaml
?? notes.md
$ git diff --cached --stat
sweep.yaml | 2 ++
train.yaml | 2 +-
2 files changed, 3 insertions(+), 1 deletion(-)
$ git diff --stat
train.yaml | 1 +
1 file changed, 1 insertion(+)
```

Three kinds of work are at stake: `sweep.yaml` is new and staged, `train.yaml` has a staged change and a later edit that is not staged, and `notes.md` is untracked.

```
$ git reset --hard
HEAD is now at cf76919 Add training config
$ git status -s
```



```

?? notes.md
$ ls
notes.md
train.py
train.yaml
$ cat train.yaml
lr: 3e-5
epochs: 3

```

`notes.md` survived, because a reset does not touch untracked files. `sweep.yaml` is deleted and `train.yaml` is back at the committed version.

Where the work is. Chapter 5 showed that `git add` writes a blob at the moment you run it. The reset removed the index entries, not the blobs. Nothing names them, so they are dangling, on layer 3:

```

$ git fsck --lost-found
dangling blob e199091045de591b8baa02bcd6a807b1788fb1cd
dangling blob e4e919703dbad4b896c710848d17d14d447f39ee
dangling blob 0697c1f85fe6c2da00446682c1a538103d96ea5c
$ ls .git/lost-found/other
0697c1f85fe6c2da00446682c1a538103d96ea5c
e199091045de591b8baa02bcd6a807b1788fb1cd
e4e919703dbad4b896c710848d17d14d447f39ee

```

Three blobs for two files, because `sweep.yaml` was staged twice and each `git add` wrote a blob. A blob has no file name and no date: the name lived in the index entry that is gone. Identify the blobs by content:

```

$ grep -c "" .git/lost-found/other/*
.git/lost-found/other/0697c1f85fe6c2da00446682c1a538103d96ea5c:1
.git/lost-found/other/e199091045de591b8baa02bcd6a807b1788fb1cd:2
.git/lost-found/other/e4e919703dbad4b896c710848d17d14d447f39ee:2
$ grep -l warmup_ratio .git/lost-found/other/*
.git/lost-found/other/e199091045de591b8baa02bcd6a807b1788fb1cd
$ grep -l epochs .git/lost-found/other/*
.git/lost-found/other/e4e919703dbad4b896c710848d17d14d447f39ee

```

```

$ cp .git/lost-found/other/e199091045de591b8baa02bcd6a807b1788fb1cd sweep.yaml
$ git cat-file -p e4e9197 > train.yaml
$ cat sweep.yaml train.yaml
lr: [1e-5, 3e-5]
warmup_ratio: [0.0, 0.1]
lr: 3e-5
epochs: 5
$ git add sweep.yaml train.yaml
$ git status -s
A sweep.yaml
M train.yaml
?? notes.md

```

Both staged versions are back. The third blob, `0697c1f`, is the earlier staging of `sweep.yaml`: every `git add` is a checkpoint that outlives the index entry.

What is not there. The line `early_stopping: true` was typed after the last `git add`. Its content was never hashed, so no object holds it:

```

# The line that was typed after the last "git add". This is the ID its file would have had:
$ printf 'lr: 3e-5\nepochs: 5\nearly_stopping: true\n' | git hash-object --stdin
fd0a951c99adb02cad59bdf1d4debd1d4de26084
$ git cat-file -t fd0a951
fatal: Not a valid object name fd0a951
[exit status: 128]

```

`git hash-object` computes the ID that the full file would have had. No object with that ID exists. This is the boundary of Git's safety net, drawn in two commands.

A dropped or cleared stash

Symptom. `git stash drop`, a `git stash pop` followed by a reset, or `git stash clear`.

```

$ cd ../finetune-incident
$ git stash list
stash@{0}: On main: lora: rank 16 trial
stash@{1}: WIP on main: cf76919 Add training config
$ git stash clear
$ git stash list

```

`git stash drop` prints the ID of what it dropped. `git stash clear` prints nothing. A stash entry is a commit with two or three parents (Chapter 11, section 11.11), and dropping it removed its reflog line, so the commit is unreachable. The stash manual gives a recipe that lists unreachable commits which look like stash entries (`git help stash`, section EXAMPLES):

```

# The recipe from the git-stash manual:
$ git fsck --unreachable | grep commit | cut -d' ' -f3 | xargs git log --merges --no-walk --grep=WIP --format='%h %s'

```

```
a0cbbaf WIP on main: cf76919 Add training config
# Without --grep=WIP:
$ git fsck --unreachable | grep commit | cut -d' ' -f3 | xargs git log --merges --no-walk --format='%h %s'
0b0c40d On main: lora: rank 16 trial
a0cbbaf WIP on main: cf76919 Add training config
```

The recipe ends in `--grep=WIP`, and it found one of two entries. The default message of a stash is `WIP on <branch>: ...`; an entry made with `git stash push -m` is named `On <branch>: <your message>` and does not match. Leave the `--grep` out, as the second command does, and rely on `--merges`: a stash entry is a merge commit, and few other unreachable commits are.

Recovery. `git stash apply <id>` applies a found entry directly. `git stash store` puts it back into the list:

```
$ git stash store -m 'recovered: grad clip' a0cbbaf
$ git stash store -m 'recovered: lora rank 16 trial' 0b0c40d
$ git stash list
stash@{0}: recovered: lora rank 16 trial
stash@{1}: recovered: grad clip
$ git stash show -p 'stash@{0}'
diff --git a/train.yaml b/train.yaml
index 58da445..db55f58 100644
--- a/train.yaml
+++ b/train.yaml
@@ -1,2 +1,3 @@
 lr: 3e-5
 epochs: 3
 +lora_rank: 16
```

The deadline is the one of layer 3, and it is shorter than for a rewritten commit: a dropped stash never has a relog entry to fall back on.

13.10 Recovering from the remote side

A force push, seen from your clone

Symptom. A teammate pushed with `--force` . Chapter 12, section 12.8 explains the mechanics on the pushing side. This is what the others see, and why your clone is the recovery tool.

```
$ git fetch
From ../server
+ d8a3934...882b940 main      -> origin/main (forced update)
$ git status -sb
## main...origin/main [ahead 2, behind 1]
$ git log --oneline --graph main origin/main
* 882b940 Rename settings keys
| * d8a3934 Add cache metrics
| * 2a2bf94 Add answer cache
|/
* 6955492 Add settings
* 1203ec5 Add question answering endpoint
```

The `+` and `(forced update)` in the fetch output mean that `origin/main` did not move forward: the server's branch was replaced by a commit that does not descend from the old one. Status calls it divergence, `ahead 2, behind 1`.

Evidence. Your remote-tracking ref has a relog, and it recorded the value the server had before:

```
$ git relog show origin/main
882b940 refs/remotes/origin/main@{0}: fetch: forced-update
d8a3934 refs/remotes/origin/main@{1}: update by push
2a2bf94 refs/remotes/origin/main@{2}: pull: fast-forward
6955492 refs/remotes/origin/main@{3}: update by push
$ git log --format='%h %an: %s' origin/main..'origin/main@{1}'
d8a3934 Lab User: Add cache metrics
2a2bf94 Asha Rao: Add answer cache
$ git log --format='%h %an: %s' 'origin/main@{1}'..origin/main
882b940 Ravi Menon: Rename settings keys
```

`origin/main@{1}` is the server's old tip. The two `git log` ranges are the damage report: two commits by two authors were removed from the server, and one commit was put in their place.

The server itself cannot tell you that. It is a bare repository without relogs. A teammate who has not fetched since is another witness:

```
# The server is a bare repository. It kept no record of the value that was overwritten:
$ ls ../server.git
config
description
HEAD
hooks
info
objects
refs
$ git -C ../server.git relog list
```

```
$ git -C ../server.git config get core.logAllRefUpdates
[exit status: 1]
# A teammate who has not fetched still has the value from her last contact with the server:
$ git -C ../asha log --oneline -1 origin/main
2a2bf94 Add answer cache
```

Recovery. There are two honest repairs, and the choice is a judgment about the commit that was forced in.

Repair	Command	Effect	Choose it when
Keep both	<code>git merge origin/main, then git push</code>	The lost commits return with their original IDs, the forced commit stays, and the push is a fast-forward	The forced commit is legitimate work
Restore the old value	<code>git push --force-with-lease=main:<forced id> origin <old id>:main</code>	The server's branch is set back; the forced commit leaves the branch	The forced commit must not be on the branch (a rewrite that drops history)

```
$ git merge -m 'Merge origin/main: restore commits removed by a force push' origin/main
Merge made by the 'ort' strategy.
 settings.yaml | 4 ++--
 1 file changed, 2 insertions(+), 2 deletions(-)
$ git push
To ../server.git
 882b940..d967a92 main -> main
```

The merge needed no force: the server's tip `882b940` is an ancestor of the merge commit. Every clone that already has the old commits, and every CI record that quotes their IDs, stays valid. Rebasing the lost commits onto the forced one would give them new IDs and create the duplicates of Chapter 9, section 9.15.

The second repair is a force push in its own right. Give it an explicit expected value, the forced commit, so that it fails if the server has moved again (Chapter 12, section 12.8).

A reflex to avoid. Faced with `ahead 2, behind 1`, people run `git reset --hard origin/main "to get in sync"`. That removes the lost commits from the one local branch that still held them. They remain in `main@{1}` and `origin/main@{1}`, and Lab 12.11 recovers them from there.

Remote-tracking refs that cannot be trusted

Symptom. `git status` says the branch is in sync with `origin/main`, and `git push` is rejected.

```
$ cd docsearch
$ git status -sb
## main...origin/main
$ git push
To ../server.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to '../server.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
[exit status: 1]
```

A remote-tracking ref is a local record of the last contact with the server (Chapter 12, section 12.4). Status compared two local refs. Push talked to the server. When they disagree, the server is right, and `git ls-remote` asks it without changing anything:

```
$ git ls-remote origin
e2753886b206a1781cfc9cb9520d36220feeae8e      HEAD
c6ec9d6c4248646f6256680df1f68a2772c1f442     refs/heads/feature/hybrid-search
e2753886b206a1781cfc9cb9520d36220feeae8e      refs/heads/main
$ git for-each-ref --format='%(objectname) %(refname)' refs/remotes/origin
warning: ignoring broken ref refs/remotes/origin/feature/hybrid-search
5eda6ed9f98f81e3870aadd032aaf47706d771bf refs/remotes/origin/main
```

Two discrepancies. `origin/main` holds an ID that the server does not have at all, and `origin/feature/hybrid-search` is reported as a broken ref. The ordinary repair, a fetch, fails on the second:

```
$ git fetch
fatal: bad object refs/remotes/origin/feature/hybrid-search
error: ../server.git did not send all necessary objects
[exit status: 1]
$ git fsck
error: refs/remotes/origin/feature/hybrid-search: badRefContent:
error: refs/remotes/origin/feature/hybrid-search: invalid sha1 pointer 00000000000000000000000000000000
dangling commit e2753886b206a1781cfc9cb9520d36220feeae8e
[exit status: 10]
```

`badRefContent` names the file that is damaged. The dangling commit is a side effect worth reading: the fetch downloaded the server's new commit and then failed before it could update any ref.

```
$ wc -c < .git/refs/remotes/origin/feature/hybrid-search
0
$ git reflog show origin/main
5eda6ed refs/remotes/origin/main@{0}:
6edd8a2 refs/remotes/origin/main@{1}: update by push
$ git log -g --date=iso --format='%gd | %gn | %gs' origin/main
origin/main@{2026-09-07 10:19:00 +0530} | Lab User |
origin/main@{2026-09-07 10:07:00 +0530} | Lab User | update by push
```

The ref file is empty: a broken ref of the kind the Git FAQ attributes to file-syncing tools (section 13.11). The reflog of `origin/main` explains the first discrepancy: its newest entry has no message. A fetch writes `fetch: ...` and a push writes `update by push`. An entry without a message was written by `git update-ref` from a script or by hand.

Recovery. Remote-tracking refs are derived data. Remove what is broken, and let a fetch write the truth:

```
$ git update-ref -d refs/remotes/origin/feature/hybrid-search
error: cannot lock ref 'refs/remotes/origin/feature/hybrid-search': unable to resolve reference
'refs/remotes/origin/feature/hybrid-search': reference broken
[exit status: 1]
$ rm .git/refs/remotes/origin/feature/hybrid-search
$ git fetch
From ../server
+ 5eda6ed...e275388 main -> origin/main (forced update)
* [new branch] feature/hybrid-search -> origin/feature/hybrid-search
$ git status -sb
## main...origin/main [ahead 2, behind 1]
```

`git update-ref -d` cannot delete a ref it cannot read, so the empty file has to be removed by hand 🚫. That is a rare legitimate reason to touch a file under `.git/refs`. The fetch then corrected `origin/main` with a forced update and recreated the other ref. Status now tells the truth: two commits to push, one to integrate. Chapter 12, section 12.14 has the wholesale version: delete every ref under `refs/remotes/origin`, then fetch.

13.11 A damaged repository

Losing a name is one kind of problem. Losing an object that is still needed is another: a reachable commit, tree or blob cannot be read. The causes are outside Git: failing hardware, and tools that copy the repository file by file. The Git FAQ is direct about the second: do not "use a cloud syncing service to sync any portion of a Git repository, since this can cause corruption, such as missing objects, changed or added files, broken refs" (`git help gitfaq`).

A missing or corrupt object

Symptom. Commands that need the object fail; commands that do not need it keep working, which is why damage can go unnoticed for weeks.

```
$ git status -sb
## main...origin/main
$ git log --oneline -3
98a37af Raise top_k to 10
688c9f2 Add embedding client
87fda00 Add retriever config
$ git log --oneline --stat -2
error: inflate: data stream error (incorrect header check)
error: unable to unpack 3a87d97273a9e9f1038d516277e2f8e853c4fccf header
fatal: loose object 3a87d97273a9e9f1038d516277e2f8e853c4fccf (stored in .git/objects/3a/87d97273a9e9f1038d516277e2f8e853c4fccf)
is corrupt
[exit status: 128]
```

`git status` and `git log --oneline` read commits and the index. `git log --stat` needs the tree of the second commit, whose file was overwritten with garbage, and stops.

```
$ git fsck
error: inflate: data stream error (incorrect header check)
error: unable to unpack header of .git/objects/3a/87d97273a9e9f1038d516277e2f8e853c4fccf
error: 3a87d97273a9e9f1038d516277e2f8e853c4fccf: object corrupt or missing:
.git/objects/3a/87d97273a9e9f1038d516277e2f8e853c4fccf
missing tree 3a87d97273a9e9f1038d516277e2f8e853c4fccf
[exit status: 3]
```

missing tree is the diagnosis: this is the line that distinguishes damage from the harmless `dangling` reports of section 13.6. A file that is present but unreadable counts as missing.

Why a fetch does not repair it. A fetch negotiates by commits: your repository says which commits it has, and the server sends what is not reachable from them. Your refs say you have everything.

```
$ git fetch origin
[exit status: 0]
```

```
$ git fsck 2>&1 | tail -1
missing tree 3a87d97273a9e9f1038d516277e2f8e853c4fccf
```

Recovery from another copy. An object's ID is determined by its content (Chapter 3, section 3.3), so the same object from any other repository is the object you lost. Ask a healthy clone for a pack that contains it:

```
# Ask a clone that has the object for a pack with that one object, and unpack it here:
$ echo 3a87d97273a9e9f1038d516277e2f8e853c4fccf | git -C ../asha pack-objects --stdout -q | git unpack-objects -q
$ git fsck 2>&1 | tail -1
missing tree 3a87d97273a9e9f1038d516277e2f8e853c4fccf
# Nothing changed: a file with that name exists, so the object was not written.
# Move the damaged file out of the object database (keep it as evidence), then repeat.
$ mv .git/objects/3a/87d97273a9e9f1038d516277e2f8e853c4fccf ../damaged-tree.saved
$ echo 3a87d97273a9e9f1038d516277e2f8e853c4fccf | git -C ../asha pack-objects --stdout -q | git unpack-objects -q
$ git fsck
[exit status: 0]
$ git log --oneline --stat -2
98a37af Raise top_k to 10
  retriever.yaml | 2 +-
  1 file changed, 1 insertion(+), 1 deletion(-)
688c9f2 Add embedding client
  embed.py | 2 ++
  1 file changed, 2 insertions(+)
```

`git pack-objects` reads IDs on standard input and writes a pack, and `git unpack-objects` stores the objects of a pack as loose files. The first attempt changed nothing, because `git unpack-objects` does not write "objects that already exist in the repository" (`git help unpack-objects`), and a file with that name existed. The damaged file was moved out, not deleted: the official how-to on this subject asks you to keep it, since a corrupt object can only be analysed next to its intact twin.

When many objects are affected, or you cannot reach a teammate's disk, fetch everything again. `--refetch` "fetches all objects as a fresh clone would" (`git help fetch`); the [release notes of Git 2.36](#), which added it, describe it as useful "when you cannot trust what you have in the local object store":

```
# The blunt repair: lose two objects, then download everything again from the server.
$ rm -f .git/objects/3a/87d97273a9e9f1038d516277e2f8e853c4fccf .git/objects/$(git rev-parse HEAD:embed.py | sed 's/./&\/\/')
$ git fsck
broken link from commit 688c9f2046a2fb925cc202a4387cf9b7631c5f84
                  to tree 3a87d97273a9e9f1038d516277e2f8e853c4fccf
missing blob 5501ef2431a0a61c1dde82cf50840bbfc7072980
missing tree 3a87d97273a9e9f1038d516277e2f8e853c4fccf
[exit status: 2]
$ git fetch --refetch origin
$ git fsck
[exit status: 0]
```

`--refetch` can only bring back what the server has. Commits that exist only in your clone and lost an object are repaired from your own working tree if the file is still there (`git hash-object -w <file>`, Chapter 3, section 3.8), and otherwise they are lost from that point of history on.

A corrupt index

Symptom. Every command that looks at the working tree fails with the same two lines; history commands still work.

```
$ printf 'def parse(q):\n    return PHRASE.findall(q.lower().strip())\n' > query.py
$ git add query.py
$ printf 'def cached(q):\n    return CACHE.get(q.strip())\n' > cache.py
$ git status -s
M cache.py
M query.py
# Simulate a damaged index file: overwrite its first four bytes.
$ printf 'XXXX' | dd of=.git/index bs=1 conv=notrunc 2>/dev/null
$ git status
error: bad signature 0x58585858
fatal: index file corrupt
[exit status: 128]
$ git log --oneline -1
5eda6ed Add query cache
[exit status: 0]
```

Recovery. The index is derived data too, except for one thing: the record of what you had staged. Move the file away and let `git reset` build a new index from HEAD:

```
$ mv .git/index .git/index.corrupt
$ git status -s
D cache.py
D index.py
D query.py
?? cache.py
?? index.py
?? query.py
$ git reset
Unstaged changes after reset:
```

```
M    cache.py
M    query.py
$ git status -s
M    cache.py
M    query.py
```

Without an index, every tracked file looks deleted and untracked at once. `git reset` (mixed, to HEAD) writes a fresh index and does not touch the working tree. Both edits are still in the files. What is gone is the distinction between them: `query.py` had been staged, and now both show as unstaged. If the staged content differed from the file on disk, it is a dangling blob, found as in section 13.9.

13.12 What cannot be recovered, and exactly why

Saying "it is gone" with a reason is part of the job. Every row below is a consequence of sections 13.2 to 13.4, not a separate rule.

What was lost	Why Git cannot bring it back	Evidence	What would have saved it
Edits to a tracked file that were never staged, overwritten by <code>git reset --hard</code> , <code>git restore</code> or <code>git checkout -- <path></code>	No object was ever written for that content	Section 13.9; Chapter 11, section 11.5	<code>git add</code> or a commit; editor history; a backup
Untracked or ignored files removed by <code>git clean</code> , or overwritten by a checkout	Git never stored them	Chapter 11, section 11.10	<code>git clean -n</code> first; <code>git stash -u</code>
The reflog of a deleted branch	Deleted with the branch; the commits survive through other names, the branch's own history of movements does not	Section 13.8	Renaming the branch instead of deleting it: the reflog moves with the name
The file name and staging time of a dangling blob	Names live in trees and index entries; the entry is gone	Section 13.9	A commit
Which changes were staged, after a corrupt index	That distinction existed only in the index file	Section 13.11	A commit
A commit whose reflog entries expired and whose objects were pruned	Nothing names it and the object is deleted	Section 13.13	A branch, a tag, a push, a bundle
A stash entry after its commits were pruned	The same	Section 13.9	A branch instead of a stash
Anything in a clone you deleted and that was never pushed	The only object database that held it is gone	Layer 4 never existed	<code>git push</code> , even to a personal branch
The value a server branch had before a force push, when no clone fetched it and the host kept no record	Bare repositories keep no reflog by default	Section 13.4	Protection against force pushes; section 13.15 for GitHub

Two losses that people expect in this table are not in it: a commit removed by a rebase, an amend or a reset is recoverable for weeks, and a deleted branch for as long as you can find its tip.

13.13 The point of no return

In one sentence. A lost commit becomes unrecoverable in two steps, first when the last reflog entry that names it is removed and then when a collection deletes the object, and two commands can force both steps at once.

Precisely. The destructive pair is the documented procedure for shrinking a repository after a history rewrite (`git help filter-branch`, "Checklist for shrinking a repository", which calls it "a very destructive approach"), and for the same reason it is the documented way to destroy the safety net:

```
git reflog expire --expire=now --all    # step 1: remove every reflog entry
git gc --prune=now                      # step 2: delete every unreachable object
```

For each  command of this section:

Command	What it changes	What it can destroy	Preview	Recovery	Appropriate when
<code>git reflog expire --expire=now --all</code> 	Empties every reflog, the stash list included	Every layer 2 protection, and the list of all stash entries except the newest	<code>git reflog expire --dry-run --verbose ...</code>	The objects still exist: <code>git fsck</code> , until a prune	A secret must be purged after a history rewrite (Chapter 21B)
<code>git gc --prune=now</code> 	Repacks; deletes all unreachable objects	Everything on layer 3	<code>git prune -n</code> ; <code>git fsck --unreachable</code>	None in this repository; only layer 4	The same purge, with no other process using the repository
<code>git prune</code> 	Deletes unreachable loose objects, with no grace period unless <code>--expire</code> is given	The same, for loose objects	<code>git prune -n</code>	None	Rarely; the manual says to run <code>git gc</code> instead
<code>git gc</code> 	Repacks; expires reflogs by the configured periods; deletes unreachable objects older than two weeks	Old layer 3 objects, and reflog entries past their period	<code>git count-objects -v</code> ; the previews above	None for what was deleted	Routine housekeeping

The manual attaches a second warning to `--prune=now`: it "increases the risk of corruption if another process is writing to the repository concurrently" (`git help gc`). An object that a running command has written and not yet attached to a ref looks unreachable.

Inside `.git`. Step 1 truncates files under `logs/`. Step 2 rewrites `objects/pack/` and removes loose object files.

See it. The commit `dbe6ec8` of section 13.2 survived `git gc --prune=now` because a reflog entry named it. Now the entry is removed. `git prune -n`  previews what a prune would delete:

```
$ cd ../searchsvc
$ git reflog expire --expire=now --all
$ git fsck
dangling commit dbe6ec85eb878f765ce8aad54c6e44b160a5efbb
$ git prune -n
76020a0a06a8e075a0095bdd4e66f5adfc4bad1b blob
dbe6ec85eb878f765ce8aad54c6e44b160a5efbb commit
e0c634deb63e58e853366c3dc3cff67a35e6aaa2 tree
```

Three objects: the commit, its tree and the one blob that no other commit shares. The commit is on layer 3 and still fully readable. A collection that respects a grace period does not delete such objects. It moves them into a cruft pack, a pack for unreachable objects with a `.mtimes` file that records each object's age:

```
# A collection that prunes nothing ("never"; the default cut-off is two weeks):
$ git gc --prune=never
$ git count-objects -v | grep -e "^count" -e in-pack -e "^packs"
count: 0
in-pack: 12
packs: 2
$ ls .git/objects/pack | cut -d. -f2 | sort | uniq -c
 2 idx
 1 mtimes
 2 pack
 2 rev
$ git cat-file -t dbe6ec8
commit
$ git fsck
dangling commit dbe6ec85eb878f765ce8aad54c6e44b160a5efbb
```

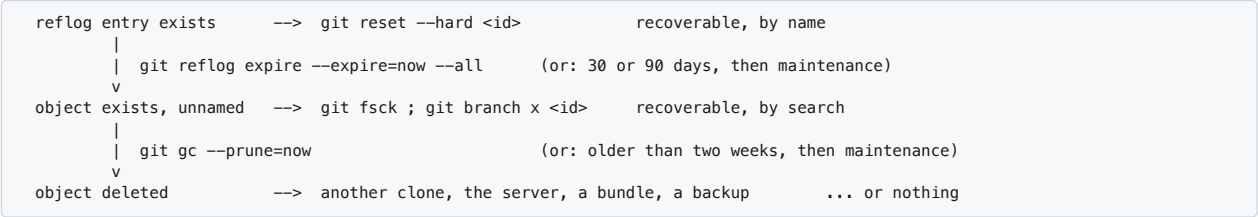
The transcript uses `--prune=never` so that it does not depend on a clock. A plain `git gc` behaves the same way for objects younger than two weeks. Without the grace period, the objects are deleted:

```
$ git gc --prune=now
$ git count-objects -v | grep -e "^count" -e in-pack -e "^packs"
count: 0
in-pack: 9
packs: 1
$ ls .git/objects/pack | cut -d. -f2 | sort | uniq -c
 1 idx
 1 pack
 1 rev
$ git cat-file -t dbe6ec8
fatal: Not a valid object name dbe6ec8
[exit status: 128]
$ git fsck
```

Nine objects are left, in one pack. `git fsck` reports nothing: the repository is healthy, and the commit does not exist. Lab 12.12 walks the same ladder and then tries every tool of this chapter against the result, including `ORIG_HEAD`, which still holds the ID of the deleted commit and resolves to nothing.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in <code>.git</code>	Remote	GitHub
<code>git reflog expire --expire=now --all</code>	unchanged	unchanged	unchanged	unchanged	Every file under <code>logs/</code> emptied; <code>refs/stash</code> keeps its value	unchanged	unchanged
<code>git gc --prune=now</code>	unchanged	unchanged	unchanged	unchanged	Objects repacked, unreachable objects deleted, refs packed into <code>packed-refs</code>	unchanged	unchanged

Picture.



Automatic maintenance. You rarely run `git gc` yourself. Some commands start maintenance when they finish (`maintenance.auto`, default true). Since Git 2.54 that automatic run uses the `geometric` strategy and not the `gc` task: it "expires data in the reflog", puts unreachable

objects into a cruft pack, and "Objects that are already part of a cruft pack will be expired" (`git help maintenance`; [maintenance configuration](#)). The Phase 0 report of this course traced such a run on Git 2.55.0 and found the same expiry values at work: `git reflog expire --all` and a repack with a cruft expiration of two weeks. The retention periods of section 13.4 therefore hold under either strategy. Chapter 26: Performance covers maintenance itself.

Version note. Older behavior: commands ran `git gc --auto`, and unreachable objects were kept as loose files. Current behavior: automatic maintenance uses the geometric strategy, and unreachable objects are kept in cruft packs. Since: cruft packs are the default from Git 2.41, the geometric strategy from Git 2.54. Recommended: nothing to configure; the manual of `git gc` still describes the older trigger, so read "automatic gc" in older texts as "automatic maintenance".

13.14 Prevention: backup refs and bundles

A backup ref before every risky operation. A branch costs one small file. Created before a rebase, a history filter, a large merge or a reset, it turns recovery into one command that needs no reflog, no search and no deadline:

```
$ git log --oneline main..HEAD
72b1150 fixup! Rerank the top 20
68e6fff Rerank the top 20
155d4ba Add reranker
$ git branch backup/rerank-before-squash
$ git rebase -q --autosquash main
$ git log --oneline main..HEAD
120120d Rerank the top 20
155d4ba Add reranker
```

```
$ git diff --stat backup/rerank-before-squash HEAD
$ git range-diff main backup/rerank-before-squash HEAD
1: 155d4ba = 1: 155d4ba Add reranker
2: 68e6fff < -: ----- Rerank the top 20
3: 72b1150 < -: ----- fixup! Rerank the top 20
-: ----- > 2: 120120d Rerank the top 20
# Had the result been wrong, one command would undo it, with no reflog involved:
$ git reset --hard backup/rerank-before-squash
HEAD is now at 72b1150 fixup! Rerank the top 20
$ git log --oneline main..HEAD
72b1150 fixup! Rerank the top 20
68e6fff Rerank the top 20
155d4ba Add reranker
```

The empty `git diff --stat` proves that the squashed branch has the same content as the original, and `git range-diff` shows how the commits map (Chapter 9, section 9.14). You compare against a name, not against a reflog position that moves. Delete the backup when the result is pushed and verified. For work in progress, a push to a personal branch on the server adds layer 4.

In one sentence, the bundle. `git bundle` writes refs and the objects they reach into one file that Git can clone and fetch from, which makes it a backup you can copy anywhere.

Precisely. A bundle is a pack file with a header that lists refs (`git help bundle`). `git bundle create <file> --all`  includes every ref; `git bundle verify` checks the file and says whether it is self-contained.

```
$ git bundle create ../searchsvc-2026-09-07.bundle --all
$ git bundle verify ../searchsvc-2026-09-07.bundle
../searchsvc-2026-09-07.bundle is okay
The bundle contains these 5 refs:
72b1150bad27ce9dc19833da51fca7015d8655c refs/heads/backup/rerank-before-squash
72b1150bad27ce9dc19833da51fca7015d8655c refs/heads/feature/rerank
da62b6073a1d8c0afa10417bc244ed245ccdfff86 refs/heads/main
da62b6073a1d8c0afa10417bc244ed245ccdfff86 refs/tags/v0.3.0
72b1150bad27ce9dc19833da51fca7015d8655c HEAD
The bundle records a complete history.
The bundle uses this hash algorithm: sha1
```

```
$ git bundle list-heads ../searchsvc-2026-09-07.bundle
72b1150bad27ce9dc19833da51fca7015d8655c refs/heads/backup/rerank-before-squash
72b1150bad27ce9dc19833da51fca7015d8655c refs/heads/feature/rerank
da62b6073a1d8c0afa10417bc244ed245ccdfff86 refs/heads/main
da62b6073a1d8c0afa10417bc244ed245ccdfff86 refs/tags/v0.3.0
72b1150bad27ce9dc19833da51fca7015d8655c HEAD
$ git clone -q ../searchsvc-2026-09-07.bundle ../restored
$ git -C ../restored log --oneline --graph --all
* 72b1150 fixup! Rerank the top 20
* 68e6fff Rerank the top 20
* 155d4ba Add reranker
* da62b60 Raise top_k to 10
* 536f5df Add embedding client
* 538ea2f Add retriever config
```


A bundle holds what its refs reach, and Git recomputes every object ID when it reads one. It does not hold reflogs, the index, hooks, the configuration, or untracked files, and of the stash at most the newest entry, which `refs/stash` names. For stashes, Git 2.51 added `git stash export` and `git stash import`, which turn the stash list into a chain of commits that can be pushed and fetched (`git help stash`). A copy of the whole directory, made while no Git command runs, holds everything, and verifies nothing. Use both kinds for a repository you cannot afford to lose, and run `git fsck` on the copy.

In production. A team that keeps experiment branches for months should not rely on the reflog of one laptop. Push the branches, and before a history rewrite store a bundle of the old state with the incident record: it preserves every old commit ID that experiment logs and model cards refer to.

13.15 The server side: what GitHub adds

GitHub, not Git. Everything in this section is a feature of the GitHub platform. It is described from GitHub's documentation, not from a run, and Chapter 30: Incident Response works through the incidents.

Git gives a server no reflog by default, and you cannot run `git reflog` or `git fsck` on GitHub's copy of your repository. GitHub provides its own instruments instead.

Instrument	What it gives you	Limits and flags
Activity view of a repository	Pushes, force pushes, merges, branch creations and deletions, with the user and a comparison	Described from the documentation
"Restore branch" on a closed pull request	Recreates the deleted head branch	Only for branches that had a pull request
Events API	<code>PushEvent</code> records whose <code>before</code> and <code>head</code> fields are the IDs on both sides of a push	The last 300 events and 30 days; delivery can lag by 30 seconds to 6 hours
Git references API	Creates a ref at a commit ID you supply	Works only while GitHub still has the commit
Audit log	Organization events for 180 days	Git events such as <code>git.push</code> are in the enterprise log only, for seven days, through the REST API, streaming or export; a search by token hash does not return them, so that question needs the export (Chapter 21B, section 21B.8)

The recipe these pieces suggest is to read the ID from before the force push in the Activity view or a `PushEvent`, and to create a branch at it:

```
gh api repos/OWNER/REPO/git/refs -f ref=refs/heads/recovered -f sha=<commit ID from before the force push>
```

Unverified. GitHub documents each instrument, not this combined procedure, and it publishes no retention period for commits that no ref reaches. One group of researchers observed that such commits appear to be kept indefinitely ([Truffle Security](#)); that is their observation, not a statement by GitHub.

GitHub's own advice for a deleted or force-pushed branch is the plain Git route of section 13.10: ask a collaborator who still has the commit to push it to a new branch ([troubleshooting commits](#)). The same persistence means that a force push does not remove a leaked secret from GitHub; rotation is the remedy (Chapter 21B). Prevention on this layer is a ruleset that blocks force pushes and deletions on shared branches (Chapter 18).

13.16 What can go wrong

Symptom	Diagnosis	Fix	Prevention
<code>git reset --hard ORIG_HEAD</code> lands on an unrelated commit	<code>git log -1 ORIG_HEAD</code> : the slot was written by an earlier reset, merge or rebase	Reset by entry of the branch reflog	Use <code>ORIG_HEAD</code> only as the next command; otherwise the reflog
<code>HEAD@{1}</code> is not the commit you expected	Any later movement, even a switch, shifts the positions	Search with <code>git log -g --grep=reflog=...</code> ; use the full ID	Anchor with a branch as soon as you find the commit
<code>git reflog show <branch></code> fails with "unknown revision"	The branch was deleted and its log with it	HEAD reflog, the printed (was ...) ID, or <code>git fsck --no-reflogs</code>	Delete with <code>-d</code> ; read the refusal
The reflog is empty in a fresh clone or on the CI runner	Reflogs are local and start with the clone	The clone where the work was done; the server; a teammate	Push work in progress
<code>git stash list</code> is empty after housekeeping	Reflogs were expired with an explicit cut-off	<code>git stash pop</code> for the newest; the fsck recipe for the rest	Branches for long-lived work
A recovered commit conflicts when cherry-picked	It depends on commits you did not pick	Abort; pick the range, or merge the rescue branch (Lab 12.7)	Inspect with <code>git log <tip> --not --all</code> first
<code>fatal: bad object refs/remotes/...</code> on fetch	An empty or damaged ref file	Remove the file, fetch	Keep repositories out of file-syncing folders

Symptom	Diagnosis	Fix	Prevention
error: bad signature and index file corrupt	Damaged <code>.git/index</code>	Move it away, <code>git reset</code>	The same
missing blob, missing tree or broken link in <code>git fsck</code>	A reachable object is gone or unreadable	Another copy of the object (section 13.11)	Backups, pushes, periodic <code>git fsck</code>

13.17 When not to use it, and dangerous edge cases

- **Do not recover a secret.** If the "lost" commit contains a credential, restoring it puts the credential back on a branch. Rotate first (Chapter 21B).
- **Do not rewind shared history to recover.** On a branch that others have pulled, add commits (revert, merge, cherry-pick). Moving the branch backwards turns one incident into several.
- **Do not "clean up" during an incident.** `git gc`, `git prune`, `git stash clear` and `git reflog expire` remove the evidence you are about to need. So does deleting and re-cloning the repository, the classic non-fix: the clone has no reflog and none of your unpushed objects.
- **Copy the repository before a doubtful repair.** `cp -R` of the whole directory, made while no Git command runs, preserves reflogs and unreachable objects. Work on the copy.
- **A reflog date is the committer date of the moment.** Time selectors such as `main@{yesterday}` trust the clock of the machine at the time of each entry, and a script that sets `GIT_COMMITTER_DATE` writes that date into the reflog.
- **Each working tree has its own HEAD reflog.** With linked working trees (Chapter 25), detached work done in one of them is recorded in that tree's HEAD log, and removing the working tree removes the log. Branch reflogs are shared.
- `git fsck --lost-found` **accumulates.** It never removes old files from `.git/lost-found`. Clear the directory before a new search.
- **Recovered is not verified.** A branch that points at the right commit proves nothing about the working tree you now have. Run the tests.

13.18 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git reflog</code> , <code>git log -g</code> , <code>git reflog list</code> , <code>git fsck</code> with <code>--no-reflogs</code> , <code>--unreachable</code> , <code>--dangling</code>	● SAFE	Nothing	Not needed	Not needed
<code>git fsck --lost-found</code>	● SAFE	Writes files under <code>.git/lost-found/</code>	<code>git fsck</code>	Delete the directory
<code>git branch <name> <id></code> , <code>git tag <name> <id></code>	● SAFE	Adds one ref	Not needed	<code>git branch -d</code> , <code>git tag -d</code>
<code>git bundle create</code>	● SAFE	Writes one file outside <code>.git</code>	<code>git bundle verify</code> afterwards	Delete the file
<code>git stash store <id></code> , <code>git stash apply <id></code>	● SAFE / ● CAUTION	<code>store</code> adds a reflog line; <code>apply</code> changes the working tree and can conflict	<code>git stash show -p <id></code>	<code>git stash drop</code> ; <code>git restore</code>
<code>git cherry-pick <id></code> , <code>git merge <rescue branch></code>	● CAUTION	New commits on the current branch	<code>git show --stat <id></code> , <code>git cherry -v</code>	<code>--abort</code> ; the branch reflog
<code>git reset --keep <id></code> , <code>git rebase --onto ...</code>	● CAUTION	Move the branch	<code>git log <id></code> , a backup branch	The branch reflog, the backup branch
<code>git reset --hard <id></code>	● DANGEROUS	Moves the branch and overwrites index and working tree; destroys unstaged work	<code>git status</code> , <code>git stash -u first</code>	Commits: reflog. Staged work: <code>git fsck --lost-found</code> . Unstaged work: none
<code>git fetch --prune</code>	● CAUTION	Deletes stale remote-tracking refs and their reflogs	<code>git remote prune --dry-run <remote></code>	<code>git fsck --lost-found</code> , within the grace period
<code>git branch -D</code> , <code>git stash drop</code> , <code>git stash clear</code>	● DANGEROUS	Delete a ref or stash entries together with their reflog lines	<code>git branch -d</code> ; <code>git stash show -p</code>	Sections 13.8 and 13.9, while the objects exist
<code>git reflog expire</code> , <code>git reflog delete</code> , <code>git reflog drop</code>	● DANGEROUS	Remove reflog entries	<code>--dry-run --verbose</code>	<code>git fsck</code> , until a prune
<code>git gc --prune=now</code> , <code>git prune</code>	● DANGEROUS	Delete unreachable objects	<code>git prune -n</code>	None in this repository
<code>rm</code> of a file under <code>.git/refs</code> or of <code>.git/index</code>	● DANGEROUS	Removes a ref or the index outside Git's locking	Move the file aside instead of deleting it	Move it back

13.19 Version notes

Version note. Older behavior: `git fsck` treated every reflog entry as a starting point. Current behavior: it skips entries dated after the moment it starts. Since: Git 2.53. Recommended: treat "dangling" as information and compare with `git reflog` (Chapter 3, section 3.8).

Version note. Older behavior: `git reflog` had the subcommands `show`, `expire`, `delete` and `exists`. Current behavior: `list` names the refs that have a log, and `drop` removes a whole log. Since: Git 2.45 and Git 2.50. Recommended: `git reflog list` as the first command in a repository you do not know.

Version note. Older behavior: stash entries could not be transferred between repositories. Current behavior: `git stash export` and `git stash import`. Since: Git 2.51 (`git help gitfaq`). Recommended: still prefer a branch for work that must survive.

Outdated advice. "Undo the last merge with `git reset --hard HEAD~1`" works only when a merge commit was created. "Run `git gc --prune=now --aggressive` to fix a slow repository" deletes the safety net as a side effect. "Delete the folder and clone again" discards every unpushed commit, stash and reflog entry.

The retention defaults of section 13.4 are the same in Git 2.55 and 2.56.

13.20 Practice

- Labs 12.1 to 12.12 in the Module 12 lab manual. Do each lab twice: first with the guide, then after running its setup script again, from the symptom alone.
- Replay any transcript with `labs/run ch13/<demo>`, for example `labs/run ch13/gc-ladder`. The sandbox stays in place for your own experiments.
- Three drills in sandboxes left by replays:
 1. In `ch13/reflog-retention/searchsvc`, explain every line of `git fsck --unreachable`.
 2. In `ch13/fsck-find/searchsvc`, delete `rescue/hybrid` again and recover it with the HEAD reflog only.
 3. In `ch13/corruption/searchsvc`, remove the object of `HEAD^{tree}` and repair the repository in two different ways.

13.21 Interview questions

1. Someone lost work. Which three questions do you ask before you type anything, and what does each answer rule in or out?
2. What is the difference between the HEAD reflog and a branch reflog? Give an incident for which only the first helps and one for which the second is the better tool.
3. State the default retention periods, say which event starts each period, and explain why "30 days plus two weeks" is not a guarantee.
4. Name the situations in which a reflog does not exist or is deleted at once. How do you recover in each?
5. `git reset --hard ORIG_HEAD` restored the wrong state. Explain the mechanism and the correct procedure.
6. What is the difference between a dangling and an unreachable object, and why does `git fsck` hide some lost commits unless you pass `--no-reflogs`?
7. A file was staged and then wiped by `git reset --hard`. What exactly can you get back, how, and what is gone? Why?
8. A teammate force-pushed over two commits on `main`. What evidence exists in your clone, on the server, and in other clones, and which repair do you choose?
9. `git fsck` reports `missing tree`. Why does `git fetch` not repair it, and what does?
10. Describe precisely what `git reflog expire --expire=now --all` followed by `git gc --prune=now` does, when it is appropriate, and what can still bring a commit back afterwards.
11. On GitHub, a branch was deleted after its pull request was closed, and another was force-pushed. Which GitHub instruments apply to each, and what are their limits?

13.22 Sources

Primary sources

- [git-reflog](#), [git-fsck](#), [git-gc](#) (with its NOTES section), [git-prune](#), [git-bundle](#), [git-stash](#) (the recovery recipe in EXAMPLES), [git-branch](#), [git-fetch](#), [gitrevisions](#), [git-maintenance](#), [git-pack-objects](#), [git-unpack-objects](#), [gitfaq](#).
- Configuration reference at the 2.56.0 tag: [gc](#), [core](#), [maintenance](#).
- Git source at the 2.55.0 tag: [reflog.c](#) (the expiry rules and the stash exemption), [builtin/stash.c](#).
- [How to recover a corrupted blob object](#), a how-to in Git's documentation, written in 2007; its method is unchanged.
- [Release notes of Git 2.36](#) for `git fetch --refetch`.
- GitHub Docs: [Activity view](#), [deleting and restoring branches in a pull request](#), [events API](#), [create a reference](#), [troubleshooting commits](#).

Secondary sources

- Pro Git, [Maintenance and Data Recovery](#). Caveats: it gives "around 7,000" loose objects as the automatic threshold where the configuration reference says 6700, and it predates cruft packs.
- The Phase 0 report of this course, sections 1, 12 and 13, for the retention defaults, the traced maintenance run and the GitHub instruments.

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [How to Rescue Your Commits with Git Reflog](#), System Crafters, 14 minutes, November 2024: a deleted branch, commits in detached HEAD, a rebase mistake. Caveat: `master` and `checkout`.
- [Intern DELETED a Git Branch!](#), Chai aur Code, Hindi, 9 minutes, 4 March 2026: a deleted branch recovered through the reflog. A single scenario.
- [The BIGGEST Git Mistake Every Intern Makes](#), Chai aur Code, Hindi, 14 minutes, 4 March 2026: commits on the wrong branch, moved with cherry-pick and reset. A single scenario.

Further reading

- Chapter 30: Incident Response, for the same accidents at team scale, and the disaster-recovery playbook that grows out of the method of section 13.7.

Chapter 14A: History investigation

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch14a/`.

14A.1 Why this matters

Four questions a CTO can ask on a Tuesday morning:

1. "The nightly evaluation scored 0.800 last Wednesday and 0.400 today, on the same model checkpoint. Is it the model, the data or our own harness? Which commit, who wrote it, why, and is it in the release that went out on Friday?"
2. "The warning that our log parser alerts on has stopped appearing. Who removed it, and what was the reasoning?"
3. "git blame says one person wrote most of the file last Friday. That cannot be right. Where is the real history?"
4. "A reviewer says the pull request deletes a setting that the pull request never touched. What were they looking at?"

None of these is answered by scrolling through `git log`. Each is a query, and Git answers three kinds:

- **Queries over the graph.** Which commits are in this set? Ranges, `--first-parent`, `--ancestry-path`, `git bisect`.
- **Queries over pairs of snapshots.** What differs between these two trees? `git diff`, and every option that prints a patch.
- **Queries over content.** Where did this line come from, when did this string appear or vanish? `git blame`, the pickaxe (`-S`, `-G`), `git log -L`, `git grep`.

One fact carries the whole chapter. **History stores snapshots and parent links, nothing else** (Chapter 2). Diffs, renames, "who wrote this line" and "which commit broke it" are not recorded anywhere. Git computes them when you ask, with heuristics that have thresholds and blind spots. An investigator has to know which answers are facts read from objects and which are inferences.

The project is `scorekit`, a small library that scores model answers against reference answers. Its history was prepared for this chapter by `labs/ch14a/fixtures/scorekit.sh`: nine days, three authors, three renames, one formatter commit, one deleted file, a merge, a squash merge, an unmerged branch, and two regressions. The symptom first:

```
$ python3 -B -m scorekit.runner data/smoke.jsonl
rows=10 exact_match=0.400 token_f1=0.444 rouge_l=0.489
[exit status: 1]
$ git describe
v0.2.0-5-ge376e5b
```

The runner exits with status 1 because `exact_match` is below the pass mark. `git describe` places the current commit five commits after the tag `v0.2.0`. This is the whole history:

```
$ git log --graph --oneline --decorate --all
* e376e5b (HEAD -> main) Mention the nightly run in the README
* 2652768 Add a separate pass mark for the nightly set
| * a10f9a1 (feat/report) Write a report when --report <path> is given
| * f37a7d8 Mention the nightly run in the README
| * a8e8550 Add per-row report writer
|/
* ca7e2b7 Fail fast on an empty reference
* d9d075d Remove the experimental BLEU scorer
* ec4fad7 Simplify token overlap in token_f1
* c0c9a30 (tag: v0.2.0) Add nightly evaluation set
* 7823232 Add ROUGE-L metric
| * 2f2883f (feat/rouge-l) Report ROUGE-L in the runner
| * 80ee55f Add ROUGE-L F-measure
| * 1cbe38a Add longest-common-subsequence helper
|/
* 9c8df98 Reformat sources: four-space indent, double quotes
* d926d3c Speed up normalize with a precompiled pattern
* be9ad1b Fix crash when --limit is not given
* 074d492 Send warnings to stderr
* 8dc82cb Add --limit option to the runner
* 72134f3 Document the runner's exit status
* 8657273 Merge branch 'feat/text-utils'
| \
| * c0d33a5 Add a tokens helper and use it in the metrics
| * dd70d9e Move normalize into scorekit/text.py
* | 389337a Raise the pass mark to 0.75
* | 4da82b5 Document the metrics
* | b4b066b Skip rows with an empty reference
|/
* 8e2cac1 Strip punctuation before comparing
* 682bc71 (tag: v0.1.0) Add config module with the pass mark
* 18bb23e Rename the scorer module to metrics
* d38a5aa Move sources into the scorekit package
* d369eac Add experimental BLEU scorer
* 633e3e6 Add evaluation runner and smoke set
* bb5eb57 Add token-level F1 scorer
```

```
* b250238 Add exact-match scorer
* 5015359 Add project skeleton
```

Somewhere between `v0.1.0`, which the team knows scored correctly, and `main` there are twenty commits. The rest of the chapter builds the tools to interrogate them, and section 14A.19 puts the tools in order.

The question	The tool	Section
What differs between two snapshots?	<code>git diff A B, A...B</code>	14A.2 to 14A.6
Which commits are in this set?	revisions and ranges, <code>git log</code>	14A.7 to 14A.9
What happened to this file, across renames?	<code>git log --follow, --diff-filter</code>	14A.10, 14A.13
When did this string appear or disappear?	<code>git log -S, -G</code>	14A.11
How did these lines evolve?	<code>git log -L</code>	14A.12
Which commit last changed each line?	<code>git blame</code>	14A.16 to 14A.18
Which commit changed the behavior?	<code>git bisect</code>	14A.20 to 14A.22
Where is this text now, or at that commit?	<code>git grep</code>	14A.23

14A.2 `git diff` compares two snapshots: `A B`, `A..B`, `A...B`

In one sentence.  `git diff` compares exactly two snapshots, and every form of the command differs only in which two it picks.

Analogy. A diff is the comparison of two photographs. It is not the film between them: whatever happened in between, and in which order, is invisible. The analogy breaks at one form. `A...B` chooses its first photograph by consulting the commit graph, so the same two branch names can give a different diff next week, after one of them has moved or been merged.

Precisely. Chapter 5, section 5.4, covered the three everyday forms: `git diff` (index against working tree), `git diff --cached` (HEAD against index) and `git diff HEAD` (HEAD against working tree). The general forms take commits, or anything that resolves to a tree:

Form	Left side	Right side	Reads the graph?
<code>git diff A B</code>	tree of A	tree of B	no
<code>git diff A..B</code>	tree of A	tree of B	no: a synonym of <code>A B</code>
<code>git diff A...B</code>	tree of the merge base of A and B	tree of B	yes, to find the merge base
<code>git diff --merge-base A B</code>	the same as <code>A...B</code>		yes
<code>git diff A^!</code>	tree of A's parent	tree of A	one step (section 14A.8)
<code>git diff <rev>:<path> <rev>:<path></code>	one blob or tree	another blob or tree	no

The manual is blunt about the dots: "diff is about comparing two endpoints, not ranges, and the range notations (`<commit>...<commit>` and `<commit>...<commit>`) do not mean a range as defined in the 'SPECIFYING RANGES' section in gitrevisions(7)" ([git-diff](#)). For `git log` the same dots select sets of commits (section 14A.8). The two meanings are among the most common misreadings of Git (Phase 0 report, section 12).

Inside `.git`. Nothing is written. Git reads two tree objects and walks them in parallel. Where both sides name the same subtree ID it does not descend, so the cost follows the size of the change, not the size of the repository.

See it. `main` and `feat/report` have diverged. The branch has three commits of its own, `main` has two, and the merge base is `ca7e2b7`:

```
$ git log --graph --oneline --decorate -6 main feat/report
* e376e5b (HEAD -> main) Mention the nightly run in the README
* 2652768 Add a separate pass mark for the nightly set
| * a10f9a1 (feat/report) Write a report when --report <path> is given
| * f37a7d8 Mention the nightly run in the README
| * a8e8550 Add per-row report writer
|/
* ca7e2b7 Fail fast on an empty reference
$ git merge-base main feat/report
ca7e2b7fcc44f8d5a494107b68440c3d4ec23b55
```

Two endpoints, with and without the two dots:

```
$ git diff --stat main feat/report
scorekit/config.py | 1 -
scorekit/report.py | 7 ++++++
scorekit/runner.py | 3 +++
3 files changed, 10 insertions(+), 1 deletion(-)
$ git diff --stat main..feat/report
scorekit/config.py | 1 -
scorekit/report.py | 7 ++++++
scorekit/runner.py | 3 +++
3 files changed, 10 insertions(+), 1 deletion(-)
```

Three dots, and the same comparison spelled out with `git merge-base`:

```
$ git diff --stat main...feat/report
 README.md      | 2 ++
 scorekit/report.py | 7 ++++++
 scorekit/runner.py | 3 +++
 3 files changed, 12 insertions(+)
$ git diff --stat $(git merge-base main feat/report) feat/report
 README.md      | 2 ++
 scorekit/report.py | 7 ++++++
 scorekit/runner.py | 3 +++
 3 files changed, 12 insertions(+)
```

The two results disagree about two files. Each disagreement is worth reading.

```
# Two endpoints: the line that main added after the fork shows up as a deletion.
$ git diff main feat/report -- scorekit/config.py
diff --git a/scorekit/config.py b/scorekit/config.py
index 12c95c2..5efef40 100644
--- a/scorekit/config.py
+++ b/scorekit/config.py
@@ -1,3 +1,2 @@
# Thresholds used by the runner.
PASS_MARK = 0.75
-NIGHTLY_PASS_MARK = 0.6
# Three dots: the branch never touched the file.
$ git diff main...feat/report -- scorekit/config.py
```

`feat/report` never touched `scorekit/config.py`. `main` added a line after the fork. Going from the tree of `main` to the tree of the branch, that line is absent, so the two-endpoint diff prints it as a deletion. This is question 4 of section 14A.1: the reviewer compared tips and read `main`'s own progress, reversed, as part of the feature.

```
# The README paragraph was cherry-picked to main. Both tips have it, the merge base does not.
$ git diff --stat main feat/report -- README.md
$ git diff main...feat/report -- README.md
diff --git a/README.md b/README.md
index bfae817..08f9933 100644
--- a/README.md
+++ b/README.md
@@ -5,3 +5,5 @@ Scores model answers against reference answers.
     python3 -m scorekit.runner data/smoke.jsonl

The runner prints one score line and exits with status 1 when exact_match is below the pass mark.
+
+The nightly job runs the same command on data/nightly.jsonl.
```

The README paragraph is the opposite case. It was written on the branch and then cherry-picked to `main` (Chapter 10). Both tips contain it, so the tips do not differ. The merge base does not contain it, so the three-dot diff shows it as a change of the branch, although `main` already has it.

Picture.

```
          a8e8550---f37a7d8---a10f9a1  feat/report
          /
...---ca7e2b7---2652768---e376e5b      main  (HEAD -> main)
      (merge base)

git diff main feat/report    compares e376e5b with a10f9a1 (two tips)
git diff main...feat/report  compares ca7e2b7 with a10f9a1 (merge base and right tip)
git diff feat/report...main  compares ca7e2b7 with e376e5b (merge base and the other tip)
```

In production. Review a branch with three dots: "what does this branch introduce", independent of what the target did meanwhile. Use two endpoints for "how do these two trees differ right now", for example a release tag against the deployed commit. Lab 10.4 has you predict both.

GitHub, not Git. The "Files changed" tab of a pull request shows a three-dot diff, from the merge base to the head of the pull request branch ([GitHub Docs: three-dot and two-dot comparisons](#)). That is why a pull request whose branch is behind its base does not show the base's newer commits as deletions, and why a change that reached the base by another route, as the README paragraph did, still appears in it until the branch is merged with or rebased onto the base.

```
Observed behavior : "git diff main feature" shows a setting being deleted that the feature never touched
Git state         : main gained commits after the branch forked; the feature's tip does not contain them
Mechanism         : A B (and A..B) compare the trees of the two tips, in the direction A to B
Root cause        : what main added after the fork is absent from the feature's tree, so it prints as "--"
Why Git does this : diff takes two endpoints; it does not look at ancestry unless asked (three dots)
Correct fix       : git diff main...feature, or git diff --merge-base main feature
Prevention        : three dots for review; in scripts the explicit --merge-base spelling
```

One more form is useful in forensics: the operands may be trees or blobs, addressed as `<rev>:<path>`. No checkout is needed to compare a directory, or one file, between two releases:

```
$ git diff --stat v0.1.0:scorekit main:scorekit
__init__.py | 2 +-
bleu.py      | 15 -----
config.py    | 3 ++-
metrics.py   | 24 ++++++-----
rouge.py     | 23 ++++++-----
runner.py    | 31 ++++++-----
text.py      | 12 ++++++
7 files changed, 71 insertions(+), 39 deletions(-)
$ git diff v0.1.0:scorekit/config.py main:scorekit/config.py
diff --git a/scorekit/config.py b/scorekit/config.py
index c22ffa6..12c95c2 100644
--- a/scorekit/config.py
+++ b/scorekit/config.py
@@ -1,2 +1,3 @@
# Thresholds used by the runner.
-PASS_MARK = 0.7
+PASS_MARK = 0.75
+NIGHTLY_PASS_MARK = 0.6
```

14A.3 Summaries and filters: `--stat`, `--name-only`, `--name-status`, `--diff-filter`

A patch between two releases can run to thousands of lines. Start with a summary and descend from there. All of these options work with `git diff`, `git show` and `git log` alike, because the three share one diff machinery.

```
$ git diff --stat v0.1.0 v0.2.0
README.md      | 2 ++
data/nightly.jsonl | 14 ++++++
docs/metrics.md | 8 ++++++
scorekit/__init__.py | 2 +-
scorekit/bleu.py | 18 ++++++-----
scorekit/config.py | 2 +-
scorekit/metrics.py | 22 ++++++-----
scorekit/rouge.py | 23 ++++++-----
scorekit/runner.py | 32 ++++++-----
scorekit/text.py | 12 ++++++
10 files changed, 104 insertions(+), 31 deletions(-)
$ git diff --shortstat v0.1.0 v0.2.0
10 files changed, 104 insertions(+), 31 deletions(-)
```

`--stat` draws one line per file with a scaled histogram of added and removed lines. `--shortstat` keeps only the total. `--numstat` prints the same numbers unscaled and tab-separated, the form for scripts: added, removed, path.

`--name-only` lists paths. `--name-status` adds one letter per path:

```
$ git diff --name-only v0.1.0 v0.2.0
README.md
data/nightly.jsonl
docs/metrics.md
scorekit/__init__.py
scorekit/bleu.py
scorekit/config.py
scorekit/metrics.py
scorekit/rouge.py
scorekit/runner.py
scorekit/text.py
$ git diff --name-status v0.1.0 main
M README.md
A data/nightly.jsonl
A docs/metrics.md
M scorekit/__init__.py
D scorekit/bleu.py
M scorekit/config.py
M scorekit/metrics.py
A scorekit/rouge.py
M scorekit/runner.py
A scorekit/text.py
```

Letter	Meaning	Letter	Meaning
A	added	R<nn>	renamed, with a similarity index (section 14A.4)
D	deleted	C<nn>	copied, with a similarity index
M	content or mode modified	T	type changed (file, symlink, submodule)
U	unmerged		

`--diff-filter` selects by those letters. An uppercase letter includes, a lowercase letter excludes:

```
$ git diff --name-status --diff-filter=A v0.1.0 main
A data/nightly.jsonl
A docs/metrics.md
A scorekit/rouge.py
```



```
A    scorekit/text.py
$ git diff --name-status --diff-filter=D v0.1.0 main
D    scorekit/bleu.py
# Lowercase letters exclude: everything except additions and deletions.
$ git diff --name-status --diff-filter=ad v0.1.0 main
M    README.md
M    scorekit/__init__.py
M    scorekit/config.py
M    scorekit/metrics.py
M    scorekit/runner.py
```

`--diff-filter=D` with `git log` is the standard way to find when a file was deleted (section 14A.13). A pathspec after `--` narrows any of these, including the exclusion form:

```
$ git diff --stat v0.1.0 main -- '*.py' ':(exclude)scorekit/runner.py'
scorekit/__init__.py | 2 +-
scorekit/bleu.py     | 15 -----
scorekit/config.py   | 3 ++
scorekit/metrics.py  | 24 ++++++-----
scorekit/rouge.py    | 23 ++++++-----
scorekit/text.py     | 12 ++++++
6 files changed, 48 insertions(+), 31 deletions(-)
```

14A.4 Renames and copies are computed: `-M` and `-C`

In one sentence. A commit does not say "this file was renamed"; `git diff` notices that a path disappeared, another appeared, and their contents are similar enough.

Precisely. Rename detection is a pass over the list of changed paths (`gitdiffcore`). A deleted path and an added path with the same blob ID are an exact rename. For the remaining pairs Git computes a similarity index, and a pair at or above the threshold becomes a rename. The threshold is 50% by default; `-M<n>` changes it (`-M90%`), and `--no-renames` switches the pass off. For the porcelain commands the pass is on by default (`diff.renames`, default `true`). `-C` also looks for copies, with the sources limited to files that the same commit modified; `--find-copies-harder`, or `-C` given twice, considers every file of the older tree and costs accordingly.

See it. Between commit `d369eac` and `v0.1.0` the sources moved into a package, one module was renamed a second time, and two files were edited on the way:

```
$ git diff --name-status d369eac v0.1.0
M    README.md
A    scorekit/__init__.py
R090 bleu.py scorekit/bleu.py
A    scorekit/config.py
R100 scorer.py scorekit/metrics.py
R082 run_eval.py scorekit/runner.py
```

`R100` is an exact rename: two commits moved `scorer.py` twice and never changed it, and the diff of the endpoints sees one rename. `R090` and `R082` are renames with edits. Without the pass, the same comparison is a list of unrelated deletions and additions:

```
$ git diff --name-status --no-renames d369eac v0.1.0
M    README.md
D    bleu.py
D    run_eval.py
A    scorekit/__init__.py
A    scorekit/bleu.py
A    scorekit/config.py
A    scorekit/metrics.py
A    scorekit/runner.py
D    scorer.py
```

The threshold decides. At 90% the runner, whose similarity is 82%, is no longer a rename:

```
$ git diff --name-status -M90% d369eac v0.1.0
M    README.md
D    run_eval.py
A    scorekit/__init__.py
R090 bleu.py scorekit/bleu.py
A    scorekit/config.py
R100 scorer.py scorekit/metrics.py
A    scorekit/runner.py
$ git diff --name-status -M d369eac v0.1.0 -- run_eval.py scorekit/runner.py
R082 run_eval.py scorekit/runner.py
```

In a patch, a detected rename is printed as extended header lines, and only the changed lines follow:

```
$ git diff d369eac v0.1.0 -- run_eval.py scorekit/runner.py
diff --git a/run_eval.py b/scorekit/runner.py
similarity index 82%
rename from run_eval.py
rename to scorekit/runner.py
```

```

index 9f41591..d846906 100644
--- a/run_eval.py
+++ b/scorekit/runner.py
@@ -1,9 +1,8 @@
import json
import sys

-from scorer import exact_match, token_f1
-
-PASS_MARK = 0.7
+from scorekit.config import PASS_MARK
+from scorekit.metrics import exact_match, token_f1

def main(path):

```

Copies need the harder search when the source file was not modified by the same commit. Here Asha seeded the nightly data set from the smoke set:

```

$ git show --stat --format='%h %s' c0c9a30
c0c9a30 Add nightly evaluation set

data/nightly.jsonl | 14 ++++++
1 file changed, 14 insertions(+)
$ git show --stat --format='%h %s' -C c0c9a30
c0c9a30 Add nightly evaluation set

data/nightly.jsonl | 14 ++++++
1 file changed, 14 insertions(+)
$ git show --stat --format='%h %s' --find-copies-harder c0c9a30
c0c9a30 Add nightly evaluation set

data/{smoke.jsonl => nightly.jsonl} | 4 +++
1 file changed, 4 insertions(+)
$ git show --name-status --format='%h %s' --find-copies-harder c0c9a30
c0c9a30 Add nightly evaluation set

C071    data/smoke.jsonl    data/nightly.jsonl

```

`-C` alone finds nothing, because `data/smoke.jsonl` itself did not change in that commit. `--find-copies-harder` finds the copy with a similarity of 71% and reduces fourteen added lines to the four that are new.

The limit. Similarity is a heuristic over content. Rename a file and rewrite half of it in the same commit, and the link is gone:

```

# docs/metrics.md was renamed to docs/scoring.md and half of its text rewritten, in one step.
$ git status --short
D docs/metrics.md
A docs/scoring.md
$ git diff --cached --name-status -M40%
R043 docs/metrics.md docs/scoring.md

```

`git status` prints a deletion and an addition, because it uses the default threshold and this pair is 43% similar. A lower threshold still finds it, but nobody reading the log later will think to pass `-M40%`. Everything that relies on rename detection inherits this limit: `git log --follow` (section 14A.10), `git blame` (section 14A.18) and the rename handling of `git merge` (Chapter 8).

In production. Rename in one commit, edit in the next. A commit that only moves files shows `R100` everywhere, reviews in seconds, and keeps every history tool working. Lab 11.1 breaks this rule on purpose and repairs it. Two more limits matter on large changes: the inexact comparison is quadratic in the number of unpaired files and is skipped above `diff.renameLimit` candidates (1000 by default according to the local manual), and a directory move with hundreds of edited files can therefore degrade to deletions and additions.

14A.5 Reading through noise: `-w` and `--word-diff`

Two kinds of change bury the lines you are looking for: whitespace, and small edits inside long lines.

Commit `9c8df98` ran a formatter over the sources. Its patch touches 40 lines. With `-w` (`--ignore-all-space`), lines that differ only in whitespace are treated as equal:

```

$ git show --stat --format='%h %an: %s' 9c8df98
9c8df98 Asha Rao: Reformat sources: four-space indent, double quotes

scorekit/__init__.py | 2 +-
scorekit/bleu.py      | 16 ++++++-----
scorekit/metrics.py   | 18 ++++++-----
scorekit/runner.py    | 36 ++++++-----
scorekit/text.py      | 8 +++++-
5 files changed, 40 insertions(+), 40 deletions(-)
$ git show --stat --format='%h %an: %s' -w 9c8df98
9c8df98 Asha Rao: Reformat sources: four-space indent, double quotes

scorekit/__init__.py | 2 +-
scorekit/runner.py   | 14 ++++++-----

```

```
scorekit/text.py | 6 +++---
3 files changed, 11 insertions(+), 11 deletions(-)
```

Two files drop out entirely: their only change was the indentation. Eleven lines remain, and `-w` shows what they are:

```
$ git show --format='%h %s' -w 9c8df98 -- scorekit/text.py
9c8df98 Reformat sources: four-space indent, double quotes

diff --git a/scorekit/text.py b/scorekit/text.py
index 3075545..d404d86 100644
--- a/scorekit/text.py
+++ b/scorekit/text.py
@@ -1,11 +1,11 @@
 import re

-_PUNCTUATION = re.compile(r'^\w\s')
+_PUNCTUATION = re.compile(r"^\w\s")

def normalize(text):
- text = _PUNCTUATION.sub(' ', text)
- return ' '.join(text.split())
+ text = _PUNCTUATION.sub("", text)
+ return " ".join(text.split())

def tokens(text):
```

The formatter also replaced single quotes with double quotes. That is not whitespace, so `-w` cannot hide it. Remember this split: it returns in section 14A.17, where `git blame -w` sees through the indentation and needs a different option for the quotes.

Option	Ignores
<code>--ignore-space-at-eol</code>	whitespace changes at the end of a line
<code>-b</code> , <code>--ignore-space-change</code>	changes in the amount of whitespace
<code>-w</code> , <code>--ignore-all-space</code>	all whitespace, including whitespace where the other side has none
<code>--ignore-blank-lines</code>	changes whose lines are all blank
<code>-I<regex></code>	changes whose lines all match the expression

For prose, configuration strings and long lines, a line-based diff says "this line changed" and leaves the comparison to your eyes:

```
$ git diff -- docs/metrics.md
diff --git a/docs/metrics.md b/docs/metrics.md
index 7b59436..9bd401e 100644
--- a/docs/metrics.md
+++ b/docs/metrics.md
@@ -1,7 +1,7 @@
 # Metrics

- exact_match: 1 when prediction and reference are equal after normalization, else 0.
-- token_f1: harmonic mean of token precision and recall.
+- token_f1: harmonic mean of token-level precision and recall.
- rouge_l: F-measure of the longest common subsequence of tokens.

-Normalization lowercases the text, strips punctuation and collapses whitespace.
+Normalization strips punctuation and collapses whitespace.
```

`--word-diff` compares words and marks them in place, removed text as `[-...-]` and added text as `{+...+}`:

```
$ git diff --word-diff -- docs/metrics.md
diff --git a/docs/metrics.md b/docs/metrics.md
index 7b59436..9bd401e 100644
--- a/docs/metrics.md
+++ b/docs/metrics.md
@@ -1,7 +1,7 @@
 # Metrics

- exact_match: 1 when prediction and reference are equal after normalization, else 0.
- token_f1: harmonic mean of [-token-]{+token-level+} precision and recall.
- rouge_l: F-measure of the longest common subsequence of tokens.

Normalization[-lowercases the text,-] strips punctuation and collapses whitespace.
```

Words are runs of non-whitespace by default; `--word-diff-regex=<regex>` defines them differently, for example to split on punctuation. `--color-words` is the same comparison shown with colors only, for the terminal. The output of `--word-diff` is for reading, not for `git apply`.

14A.6 Diff algorithms, and `--check`

In one sentence. For two given files there are many correct diffs, and the algorithm decides which one you read.

Precisely. A diff is a set of deletions and insertions that turns one file into the other. Git has four algorithms ([git-diff](#)): `myers`, the default, "the basic greedy diff algorithm"; `minimal`, which spends extra time to produce the smallest diff; `patience`; and `histogram`, which "extends the patience algorithm to support low-occurrence common elements". The last two anchor on lines that occur rarely, which tends to keep moved blocks together. Choose with `--diff-algorithm=<name>` (or `--minimal`, `--patience`, `--histogram`) and make the choice permanent with `diff.algorithm`.

See it. One file, two similar functions, and an edit that swaps them:

```
# The two functions of scorekit/overlap.py were swapped. Nothing else changed.
$ git diff --diff-algorithm=myers
diff --git a/scorekit/overlap.py b/scorekit/overlap.py
index 1394725..49c341c 100644
--- a/scorekit/overlap.py
+++ b/scorekit/overlap.py
@@ -1,12 +1,12 @@
-def token_precision(pred, ref):
+def token_recall(pred, ref):
+    overlap = len(set(pred) & set(ref))
-    if not pred:
+    if not ref:
         return 0.0
-    return overlap / len(pred)
+    return overlap / len(ref)

-def token_recall(pred, ref):
+def token_precision(pred, ref):
+    overlap = len(set(pred) & set(ref))
-    if not ref:
+    if not pred:
         return 0.0
-    return overlap / len(ref)
+    return overlap / len(pred)
```

Myers found the shortest edit: six lines out, six lines in. It is correct, and it tells a false story, that someone edited both functions line by line. Histogram tells what happened:

```
$ git diff --diff-algorithm=histogram
diff --git a/scorekit/overlap.py b/scorekit/overlap.py
index 1394725..49c341c 100644
--- a/scorekit/overlap.py
+++ b/scorekit/overlap.py
@@ -1,12 +1,12 @@
-def token_precision(pred, ref):
-    overlap = len(set(pred) & set(ref))
-    if not pred:
-        return 0.0
-    return overlap / len(pred)
-
+def token_recall(pred, ref):
+    overlap = len(set(pred) & set(ref))
+    if not ref:
+        return 0.0
+    return overlap / len(ref)
+
+def token_precision(pred, ref):
+    overlap = len(set(pred) & set(ref))
+    if not pred:
+        return 0.0
+    return overlap / len(pred)
```

The line counts differ as well:

```
$ git diff --shortstat --diff-algorithm=myers
1 file changed, 6 insertions(+), 6 deletions(-)
$ git diff --shortstat --diff-algorithm=histogram
1 file changed, 7 insertions(+), 7 deletions(-)
$ git diff --shortstat --patience
1 file changed, 7 insertions(+), 7 deletions(-)
```

In production. Three consequences. First, "lines changed" is a property of the algorithm, not of the commit, so two tools can report different numbers for the same pull request. Second, a patch is still valid under any algorithm: all of them transform the same old file into the same new file. Third, the algorithm changes what `git blame` and `git log -L` attribute (both run diffs internally; `git blame` accepts `--diff-algorithm`). Many teams set `diff.algorithm=histogram`; the `ort` merge strategy already uses histogram internally ([merge strategies](#)).

`--check` is not about reading diffs at all. It inspects the lines a diff adds and reports whitespace errors and leftover conflict markers, with an exit status a script can test:

```
# A line with two trailing spaces in config.py, a forgotten conflict marker in docs/metrics.md.
$ git diff --check
docs/metrics.md:8: leftover conflict marker
```

```
scorekit/config.py:4: trailing whitespace.
+SMOKE_LIMIT = 10
[exit status: 2]
```

By default, trailing whitespace and a space before a tab in the indentation count as errors; `core.whitespace` adjusts the rules. As with every `git diff` form, which changes are inspected depends on the operands:

```
$ git add -A
$ git diff --check
[exit status: 0]
$ git diff --cached --check
docs/metrics.md:8: leftover conflict marker
scorekit/config.py:4: trailing whitespace.
+SMOKE_LIMIT = 10
[exit status: 2]
```

After `git add -A` the unstaged diff is empty, so the first command finds nothing. `git diff --cached --check` inspects what the next commit would record. That is the form for a pre-commit hook (Chapter 14C), and with two commits as operands it is the form for continuous integration.

14A.7 Naming one commit: revision selection

Every investigation command takes revisions as arguments. Chapter 3, section 3.6, showed how `git rev-parse` turns a name into an object ID, and Chapter 7 introduced `@{-1}` and `@{upstream}`. This section is the working set for forensics, from [gitrevisions](#).

Form	Names	Read from
<code><rev>~<n></code>	the n-th ancestor, following first parents only	the graph
<code><rev>^, <rev>^<n></code>	the first parent, the n-th parent	the graph
<code><ref>@{<n>}</code>	the value that ref had n updates ago	your reflog
<code><ref>@{<date>}</code>	the value that ref had at that time in this repository	your reflog
<code>@{u}, <branch>@{upstream}</code>	the remote-tracking branch the branch is set to follow	configuration
<code>:/<text></code>	the youngest commit, reachable from any ref, whose message matches the regular expression	all refs
<code><rev>^/(<text>)</code>	the youngest such commit reachable from <code><rev></code>	the graph
<code><rev>^{ }, ^{commit}, ^{tree}</code>	the object after peeling tags; the commit; its tree	objects
<code><rev>:<path></code>	the blob or tree at that path in that revision	objects
<code>:<path>, :<n>:<path></code>	the index entry for the path, at stage 0 or stage n	the index

Three of these deserve a transcript. First, `^<n>` chooses among the parents of one commit, and `~<n>` walks n generations back along first parents; the difference shows on merges. `8657273` is the merge of `feat/text-utils`:

```
# 8657273 is the merge of feat/text-utils. ^1 and ^2 choose a parent; ~1 always follows the first parent.
$ git rev-parse 8657273^1 8657273~1 8657273^2 8657273^2~1
389337aa37f6f099b27867f23fdf426b5c0307bd
389337aa37f6f099b27867f23fdf426b5c0307bd
c0d33a57b14ce8f345d5cf625ec8500fd30d85a5
dd70d9e494fe1312ef1f62b858ada29744f9edbe
$ git show -s --format='%h %s' 8657273^1 8657273^2 8657273^2~1
389337a Raise the pass mark to 0.75
c0d33a5 Add a tokens helper and use it in the metrics
dd70d9e Move normalize into scorekit/text.py
```

`^1` and `~1` are the same commit, the first parent, which is the branch that was merged into. `^2` is the tip of the branch that was merged in, and `^2~1` walks one step back on that side.

Second, the reflog forms read your local reflog and nothing else:

```
$ git show -s --format='%h %s' 'HEAD@{1}' 'main@{2}'
e376e5b Mention the nightly run in the README
2652768 Add a separate pass mark for the nightly set
$ git show -s --format='%h %cd %s' --date=format:'%a %H:%M' 'main@{yesterday}'
d9d075d Mon 12:31 Remove the experimental BLEU scorer
```

`main@{yesterday}` does not mean "the commit made yesterday". It means "what `main` pointed at in this clone 24 hours ago", here a commit from Monday 12:31, because the lab clock stands at Tuesday shortly before 13:00. On a teammate's machine, or in a fresh clone whose reflog starts today, the same expression gives a different answer or a warning. Use reflog forms to investigate your own repository (Chapter 13); use dates with `git log --since` to investigate the project.

Third, searching by message:

```
$ git show -s --format='%h %s' ':/Reformat sources'
9c8df98 Reformat sources: four-space indent, double quotes
$ git show -s --format='%h %s' 'v0.2.0^{/pass mark}'
389337a Raise the pass mark to 0.75
```

```
$ git show -s --format='%h %s' ':/nightly run'
e376e5b Mention the nightly run in the README
```

`:/nightly run` matches two commits in this repository, the original on `feat/report` and its copy on `main`. The rule "youngest, from any ref" picked the copy. `<rev>^{<text>}` restricts the search to the ancestors of one revision and is the form to use in scripts.

An annotated tag is an object of its own, so the tag name and the commit it marks have different IDs. `^{}` peels:

```
$ git cat-file -t v0.1.0
tag
$ git rev-parse v0.1.0 v0.1.0^{ } v0.1.0^{commit} v0.1.0^{tree}
2b135670afee8cdfbb883de84523a5c97a25d914
682bc717015f0ab43b873d1df1ac9da0ba01c393
682bc717015f0ab43b873d1df1ac9da0ba01c393
2d7515ae10f361f36e63b9496cd27f732f656c01
$ git rev-parse v0.1.0^0
682bc717015f0ab43b873d1df1ac9da0ba01c393
```

Commands that need a commit peel for you. You need the suffix when you compare IDs, as in "is the tag still on the commit we deployed". Paths select inside a revision:

```
$ git rev-parse main:scorekit/config.py v0.1.0:scorekit/config.py :scorekit/config.py
12c95c24c2ec9a14f3b6770dc3cf21994a1fd054
c22ffa6cffb75db7b502a54f0e0d158689414374
12c95c24c2ec9a14f3b6770dc3cf21994a1fd054
$ git show v0.1.0:scorekit/config.py
# Thresholds used by the runner.
PASS_MARK = 0.7
$ git cat-file -t main:scorekit
tree
```

`@{u}` resolves through the upstream configuration (Chapter 12); `@{u}..` is then "my commits that the remote-tracking branch does not have":

```
$ git rev-parse --abbrev-ref @{u}
origin/main
$ git show -s --format='%h %s' @{u}
e376e5b Mention the nightly run in the README
$ git log --oneline @{u}..
fd9fa62 Start the 0.3 cycle
```

14A.8 Naming sets of commits: ranges

In one sentence. A range is a set of commits defined by reachability: everything you can reach from the included tips, minus everything you can reach from the excluded ones.

Analogy. Colour every commit reachable from B green, then colour every commit reachable from A red, red winning. `A..B` is what stays green. The picture breaks if you think of "between": a range is not an interval on a line, and with merges in the graph the green set can contain commits made long before A.

Precisely. For history-walking commands, one revision means "this commit and all its ancestors", and several revisions mean the union. A leading `^` excludes a commit and its ancestors. The dotted forms are shorthands ([gitrevisions](#)):

Form	Selects for <code>git log</code> and <code>git rev-list</code>	What <code>git diff</code> does with the same text
<code>B</code>	B and every ancestor of B	compares the working tree with B
<code>^A B, B --not A, A..B</code>	reachable from B and not from A	compares the trees of A and B
<code>A...B</code>	reachable from A or B, not from both	compares the merge base with B
<code>A^!</code>	A alone: A minus all its parents	compares A's parent with A
<code>A^@</code>	all parents of A, and their ancestors	(used as <code>git diff A A^@</code> for a merge)
<code>A^-</code>	<code>A^1..A</code> : A and, if A is a merge, everything the merge brought in	compares A's first parent with A
<code>--all, --branches, --tags, --remotes</code>	every ref of that kind as a starting tip	not applicable

An omitted side means HEAD: `origin/main..` is `origin/main..HEAD`.

Inside `.git`. Nothing changes. Git walks parent pointers from the tips, marks what is reachable from the excluded side, and stops descending where both colours meet. On a large repository the commit-graph file makes this walk fast (Chapter 26).

See it. Three spellings of one set, and its mirror image:

```
$ git log --oneline main..feat/report
a10f9a1 Write a report when --report <path> is given
f37a7d8 Mention the nightly run in the README
a8e8550 Add per-row report writer
$ git log --oneline ^main feat/report
```

```
a10f9a1 Write a report when --report <path> is given
f37a7d8 Mention the nightly run in the README
a8e8550 Add per-row report writer
$ git log --oneline feat/report --not main
a10f9a1 Write a report when --report <path> is given
f37a7d8 Mention the nightly run in the README
a8e8550 Add per-row report writer
```

```
$ git log --oneline feat/report..main
e376e5b Mention the nightly run in the README
2652768 Add a separate pass mark for the nightly set
```

The symmetric difference is the union of the two. `--left-right` says which side each commit is on, and `--boundary` adds the commit where the walk stopped, marked `o`:

```
$ git log --oneline main...feat/report
e376e5b Mention the nightly run in the README
2652768 Add a separate pass mark for the nightly set
a10f9a1 Write a report when --report <path> is given
f37a7d8 Mention the nightly run in the README
a8e8550 Add per-row report writer
$ git log --oneline --left-right main...feat/report
< e376e5b Mention the nightly run in the README
< 2652768 Add a separate pass mark for the nightly set
> a10f9a1 Write a report when --report <path> is given
> f37a7d8 Mention the nightly run in the README
> a8e8550 Add per-row report writer
$ git log --oneline --graph --boundary main...feat/report
* e376e5b Mention the nightly run in the README
* 2652768 Add a separate pass mark for the nightly set
| * a10f9a1 Write a report when --report <path> is given
| * f37a7d8 Mention the nightly run in the README
| * a8e8550 Add per-row report writer
|/
o ca7e2b7 Fail fast on an empty reference
```

The parent shorthands, on the merge commit:

```
$ git rev-parse 8657273^@
389337aa37f6f099b27867f23fdf426b5c0307bd
c0d33a57b14ce8f345d5cf625ec8500fd30d85a5
$ git log --oneline 8657273^!
8657273 Merge branch 'feat/text-utils'
$ git log --oneline 8657273^~
8657273 Merge branch 'feat/text-utils'
c0d33a5 Add a tokens helper and use it in the metrics
dd70d9e Move normalize into scorekit/text.py
$ git log --oneline 8657273^1..8657273
8657273 Merge branch 'feat/text-utils'
c0d33a5 Add a tokens helper and use it in the metrics
dd70d9e Move normalize into scorekit/text.py
```

`8657273^~` is the answer to "what did this merge bring in": the merge itself and the two commits of the side branch. For an ordinary commit, `A^!` gives `git diff` the pair that `git show` uses:

```
$ git diff --stat ec4fad7^!
scorekit/metrics.py | 4 +---
1 file changed, 1 insertion(+), 3 deletions(-)
$ git diff --stat ec4fad7~1 ec4fad7
scorekit/metrics.py | 4 +---
1 file changed, 1 insertion(+), 3 deletions(-)
```

Picture. The same graph, three questions:

```

      a8e8550---f37a7d8---a10f9a1   feat/report
      /
...---ca7e2b7---2652768---e376e5b   main

main..feat/report   = { a8e8550, f37a7d8, a10f9a1 }   what the branch has that main lacks
feat/report..main   = { 2652768, e376e5b }           what main has that the branch lacks
main...feat/report   = { 2652768, e376e5b, a8e8550, f37a7d8, a10f9a1 } both of the above
git diff main..feat/report   compares the trees of e376e5b and a10f9a1
git diff main...feat/report   compares the trees of ca7e2b7 and a10f9a1
```

In production. `git log --oneline origin/main..HEAD` before a push is "what am I about to publish". `git log --oneline HEAD..origin/main` after a fetch is "what will a merge bring me". An empty `A..B` means B is an ancestor of A, which is the test `git merge-base --is-ancestor` performs:

```
$ git log --oneline main..v0.2.0
$ git merge-base --is-ancestor v0.2.0 main && echo "v0.2.0 is an ancestor of main"
v0.2.0 is an ancestor of main
```

And one query lists all unmerged work in a repository:

```
# Everything that is on some branch and not yet in main, in one query:
$ git log --oneline --branches --not main
a10f9a1 Write a report when --report <path> is given
f37a7d8 Mention the nightly run in the README
a8e8550 Add per-row report writer
2f2883f Report ROUGE-L in the runner
80ee55f Add ROUGE-L F-measure
1cbe38a Add longest-common-subsequence helper
$ git rev-list --count v0.1.0..main
20
```

feat/rouge-l appears here although its content is on main: it was squash-merged, and a squash creates a new commit that has no parent link to the branch (section 14A.18). The graph knows ancestry, not content.

14A.9 git log is a graph query

In one sentence.  `git log` walks the graph from the tips you name, filters the commits it meets, and prints each survivor in the format you choose.

Analogy. A database query: `FROM` is the set of commits (section 14A.8), `WHERE` is the filter, `ORDER BY` is the ordering, `SELECT` is the output format. The analogy breaks in two places. There is no index on authors or messages, so every filter is a scan of the walked commits. And the default order is by commit date, newest first; only `--topo-order`, which `--graph` implies, guarantees that no parent is printed before all of its children.

Precisely. Think in four stages, and place every option in one of them.

Stage	Question	Options
1. Walk	Which commits are visited?	revisions and ranges, <code>--all</code> , <code>--branches</code> , <code>--reflog</code> , <code>--first-parent</code> , <code>--ancestry-path</code>
2. Filter	Which visited commits are shown?	<code>--author</code> , <code>--committer</code> , <code>--grep</code> , <code>--since</code> , <code>--until</code> , <code>--merges</code> , <code>--no-merges</code> , <code>-S</code> , <code>-G</code> , <code>-- <path></code> , <code>--diff-filter</code> , <code>-<n></code>
3. Order	In which sequence?	<code>--date-order</code> , <code>--topo-order</code> , <code>--author-date-order</code> , <code>--reverse</code>
4. Print	What is printed per commit?	<code>--oneline</code> , <code>--format</code> , <code>--graph</code> , <code>--decorate</code> , <code>--stat</code> , <code>-p</code> , <code>--name-status</code>

`--graph --oneline --decorate --all`, the command from section 14A.1, is one option from each of stages 1, 3 and 4: walk from every ref, order topologically and draw the edges, print one line with ref names.

See it: the walk. Part of the history has a side branch:

```
$ git log --graph --oneline 8e2cac1~1..8657273
* 8657273 Merge branch 'feat/text-utils'
|\
| * c0d33a5 Add a tokens helper and use it in the metrics
| * dd70d9e Move normalize into scorekit/text.py
* | 389337a Raise the pass mark to 0.75
* | 4da82b5 Document the metrics
* | b4b066b Skip rows with an empty reference
|/
* 8e2cac1 Strip punctuation before comparing
```

`--first-parent` follows only the first parent of every merge. On a branch that receives work through merges it shows the sequence of things that landed, and hides the commits inside each landed branch:

```
$ git log --oneline --first-parent 8e2cac1~1..8657273
8657273 Merge branch 'feat/text-utils'
389337a Raise the pass mark to 0.75
4da82b5 Document the metrics
b4b066b Skip rows with an empty reference
8e2cac1 Strip punctuation before comparing
```

`--merges` and `--no-merges` filter by parent count. `git rev-list --count` returns the size of a set without printing it:

```
$ git log --oneline --merges
8657273 Merge branch 'feat/text-utils'
$ git rev-list --count v0.1.0..main
20
$ git rev-list --count --first-parent v0.1.0..main
18
$ git rev-list --count --no-merges v0.1.0..main
19
```

Twenty commits since `v0.1.0`; eighteen on the first-parent line, because two arrived through the merge. Keep the number twenty: section 14A.20 turns it into the cost of a bisection. For an overview of a repository you do not know, `--simplify-by-decoration` keeps only the

commits that a branch or tag points at:

```
$ git log --graph --oneline --decorate --simplify-by-decoration --all
* e376e5b (HEAD -> main) Mention the nightly run in the README
| * a10f9a1 (feat/report) Write a report when --report <path> is given
|/
* c0c9a30 (tag: v0.2.0) Add nightly evaluation set
| * 2f2883f (feat/rouge-l) Report ROUGE-L in the runner
|/
* 682bc71 (tag: v0.1.0) Add config module with the pass mark
* 5015359 Add project skeleton
```

See it: the filters. By person:

```
$ git log --oneline --author=Ravi
e376e5b Mention the nightly run in the README
d9d075d Remove the experimental BLEU scorer
d926d3c Speed up normalize with a precompiled pattern
074d492 Send warnings to stderr
8e2cac1 Strip punctuation before comparing
d369eac Add experimental BLEU scorer
$ git log --format='%h %an (committed by %cn) %s' --author=Ravi --committer='Lab User'
e376e5b Ravi Menon (committed by Lab User) Mention the nightly run in the README
```

`--author` and `--committer` are regular expressions matched against "Name ". The second command finds a commit written by one person and committed by another: the cherry-pick. By message:

```
$ git log --oneline --grep=crash
be9ad1b Fix crash when --limit is not given
$ git log --oneline -i --grep=rouge
d9d075d Remove the experimental BLEU scorer
7823232 Add ROUGE-L metric
$ git log --oneline -i --grep=rouge --all
d9d075d Remove the experimental BLEU scorer
7823232 Add ROUGE-L metric
2f2883f Report ROUGE-L in the runner
80ee55f Add ROUGE-L F-measure
$ git log --oneline --grep=limit --grep=crash
be9ad1b Fix crash when --limit is not given
8dc82cb Add --limit option to the runner
$ git log --oneline --grep=limit --grep=crash --all-match
be9ad1b Fix crash when --limit is not given
```

Without `--all` the walk starts at HEAD, so commits that exist only on other branches are not searched. Several `--grep` options are alternatives unless `--all-match` is given. And `--grep` searches messages, not content: the BLEU removal is found only because its message mentions ROUGE-L.

By date:

```
$ git log --oneline --since='2026-09-10 00:00' --until='2026-09-11 00:00'
d926d3c Speed up normalize with a precompiled pattern
be9ad1b Fix crash when --limit is not given
074d492 Send warnings to stderr
8dc82cb Add --limit option to the runner
72134f3 Document the runner's exit status
$ git log --oneline --since='2 days ago'
e376e5b Mention the nightly run in the README
2652768 Add a separate pass mark for the nightly set
ca7e2b7 Fail fast on an empty reference
d9d075d Remove the experimental BLEU scorer
ec4fad7 Simplify token overlap in token_fl
```

Dates have two traps. The first: `--since` and `--until` test the committer date, the time the commit object was created, not the author date (Chapter 6, section 6.5):

```
# --since and --until test the committer date. The newest commit was authored at 10:46 and committed at 12:41:
$ git log --format='%h authored %ad, committed %cd' --date=format:'%a %H:%M' --since='2026-09-15 12:30'
e376e5b authored Tue 10:46, committed Tue 12:41
```

The second trap produces wrong answers silently:

```
# A date without a time takes the current time of day. The lab clock reads 12:51.
$ git log --oneline --since=2026-09-10 --until=2026-09-11
9c8df98 Reformat sources: four-space indent, double quotes
d926d3c Speed up normalize with a precompiled pattern
```

```
Observed behavior : --since=2026-09-10 --until=2026-09-11 omits four commits made on the morning of the 10th
                    and includes one made on the morning of the 11th
Git state          : nothing unusual; the commits exist and carry those dates
Mechanism          : Git's date parser took the time of day, which the argument left out, from the clock
Root cause         : "2026-09-10" was read as "10 September at the present time of day", here 12:51
```

```
Why Git does this : the same parser serves "yesterday" and "2 days ago", which are meant relative to now;
                   the manual notes only that "today" means the last midnight
Correct fix       : write the time: --since='2026-09-10 00:00' --until='2026-09-11 00:00'
Prevention        : in scripts and incident notes always give a full timestamp, and state the time zone
```

This behavior was observed on Git 2.55.0, in the transcript above; the manual page of `git log` does not describe it. A third property: `--since` stops the walk at the first commit that is too old. In a history whose commit dates are not in order, for example after an import or with a wrong clock, an old-looking commit can hide newer ones behind it; `--since-as-filter` visits the whole range instead.

By path, and with content:

```
$ git log --oneline -- scorekit/config.py
2652768 Add a separate pass mark for the nightly set
389337a Raise the pass mark to 0.75
682bc71 Add config module with the pass mark
$ git log --oneline v0.2.0..main -- docs scorekit/metrics.py
d9d075d Remove the experimental BLEU scorer
ec4fad7 Simplify token overlap in token_f1
```

`git log -- <path>` shows the commits whose version of the path differs from their parent's. With merges in the history, the default mode also prunes side branches that did not contribute to the final content, so a change that was made and later undone on a branch may not appear; `--full-history` disables that pruning. `--stat` and `-p` add the summary or the patch to each commit and, combined with a path, show only that path's part:

```
$ git log --stat --format='%h %s' -2 -- scorekit/config.py
2652768 Add a separate pass mark for the nightly set

scorekit/config.py | 1 +
1 file changed, 1 insertion(+)
389337a Raise the pass mark to 0.75

scorekit/config.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git log -p --format='%h %s' -1 -- scorekit/config.py
2652768 Add a separate pass mark for the nightly set

diff --git a/scorekit/config.py b/scorekit/config.py
index 5efef40..12c95c2 100644
--- a/scorekit/config.py
+++ b/scorekit/config.py
@@ -1,2 +1,3 @@
# Thresholds used by the runner.
PASS_MARK = 0.75
+NIGHTLY_PASS_MARK = 0.6
```

In production. State the stage when you explain a query. "`git log --author=asha -5`" is "walk from HEAD, keep Asha's commits, print the first five that pass". The limit counts after the filter and before `--reverse`, so `--reverse -1` prints the newest commit, not the oldest; `git rev-list --max-parents=0 HEAD` names the root commits.

14A.10 The history of one file: `--follow` and its limits

`git log -- <path>` matches a path, not a file. When the file had another name, the history stops at the commit that gave it the current name:

```
$ git log --oneline -- scorekit/metrics.py
ec4fad7 Simplify token overlap in token_f1
9c8df98 Reformat sources: four-space indent, double quotes
c0d33a5 Add a tokens helper and use it in the metrics
dd70d9e Move normalize into scorekit/text.py
8e2cac1 Strip punctuation before comparing
18bb23e Rename the scorer module to metrics
```

`--follow` runs rename detection at each step and continues under the old name:

```
$ git log --oneline --follow -- scorekit/metrics.py
ec4fad7 Simplify token overlap in token_f1
9c8df98 Reformat sources: four-space indent, double quotes
c0d33a5 Add a tokens helper and use it in the metrics
dd70d9e Move normalize into scorekit/text.py
8e2cac1 Strip punctuation before comparing
18bb23e Rename the scorer module to metrics
d38a5aa Move sources into the scorekit package
bb5eb57 Add token-level F1 scorer
b250238 Add exact-match scorer
```

Three more commits appear: the move into the package and the two commits from the time the file was `scorer.py`. Combined with `--diff-filter` and `--name-status`, `--follow` prints the chain of names:

```
$ git log --follow --diff-filter=AR --name-status --format='%h %s' -- scorekit/metrics.py
18bb23e Rename the scorer module to metrics
```

```
R100    scorekit/scorer.py      scorekit/metrics.py
d38a5aa Move sources into the scorekit package

R100    scorer.py              scorekit/scorer.py
b250238 Add exact-match scorer

A       scorer.py
```

The limits, all consequences of section 14A.4:

- **One path only.** `--follow` "works only for a single file" ([git-log](#)):

```
$ git log --oneline --follow -- scorekit/metrics.py scorekit/text.py
fatal: --follow requires exactly one pathspec
[exit status: 128]
```

- **The heuristic can miss.** A rename combined with a rewrite below the similarity threshold ends the trail (Lab 11.1).
- **It is not `git log` with full history semantics.** The manual says of `log.follow` that it "does not work well on non-linear history". The release notes of Git 2.56 record an improvement for histories "in which the path being tracked gets renamed differently in multiple history lines" ([RelNotes 2.56.0](#)); that version was not run here.

If you know the old names, plain path limiting takes several paths and needs no heuristic:

```
# Without --follow, the old names can be given as extra paths, if you know them:
$ git log --oneline -- scorekit/metrics.py scorekit/scorer.py scorer.py
ec4fad7 Simplify token overlap in token_f1
9c8df98 Reformat sources: four-space indent, double quotes
c0d33a5 Add a tokens helper and use it in the metrics
dd70d9e Move normalize into scorekit/text.py
8e2cac1 Strip punctuation before comparing
18bb23e Rename the scorer module to metrics
d38a5aa Move sources into the scorekit package
bb5eb57 Add token-level F1 scorer
b250238 Add exact-match scorer
```

14A.11 The pickaxe: `-S` versus `-G`

In one sentence. `-S<string>` finds commits that changed how many times a string occurs in a file; `-G<regex>` finds commits whose patch has an added or removed line matching a pattern.

Analogy. `-S` is an inventory count: it compares the number of items before and after and reports a difference. `-G` is a goods-movement ledger: it reports every delivery and removal, including the day one crate left and an identical crate arrived. The analogy breaks at files: both options look at one file at a time, so a string that moves from one file to another is reported by `-S` as well.

Precisely. Both are filters of stage 2 and are documented in [gitdiffcore](#). `-S` "detects filepairs whose preimage and postimage have different number of occurrences of the specified block of text. By definition, it will not detect in-file moves." `-G` "detects filepairs whose textual diff has an added or a deleted line that matches the given regular expression." The argument of `-S` is a literal string unless `--pickaxe-regex` is given. The argument of `-G` is always a regular expression.

See it. The history of the pass mark, asked both ways:

```
$ git log --oneline -S'PASS_MARK'
2652768 Add a separate pass mark for the nightly set
682bc71 Add config module with the pass mark
633e3e6 Add evaluation runner and smoke set
```

```
$ git log --oneline -G'PASS_MARK'
2652768 Add a separate pass mark for the nightly set
9c8df98 Reformat sources: four-space indent, double quotes
389337a Raise the pass mark to 0.75
682bc71 Add config module with the pass mark
633e3e6 Add evaluation runner and smoke set
```

`-G` reports two commits that `-S` does not. The first is the one a person asking "who changed the pass mark" wants most:

```
$ git show --format='%h %s' 389337a
389337a Raise the pass mark to 0.75

diff --git a/scorekit/config.py b/scorekit/config.py
index c22ffa6..5efef40 100644
--- a/scorekit/config.py
+++ b/scorekit/config.py
@@ -1,2 +1,2 @@
# Thresholds used by the runner.
-PASS_MARK = 0.7
+PASS_MARK = 0.75
$ git grep -c PASS_MARK 389337a~1 389337a
```

```
389337a~1:scorekit/config.py:1
389337a~1:scorekit/runner.py:2
389337a:scorekit/config.py:1
389337a:scorekit/runner.py:2
```

The value changed and the name stayed. One occurrence before, one after: for `-S` nothing happened. The line was removed and added: for `-G` it matches twice. The second extra commit is the formatter, which re-indented a line that mentions `PASS_MARK`. That is the noise `-G` brings.

	<code>-S<string></code>	<code>-G<regex></code>
Argument	literal (regex with <code>--pickaxe-regex</code>)	regular expression
Reports a commit when	the number of occurrences in a file changes	an added or removed line matches
A line containing the string is edited, string stays	not reported	reported
The string moves inside one file	not reported	reported
Typical question	"when was this introduced, when was it removed?"	"which commits touched lines that mention this?"
Cost	cheap	higher: every candidate is diffed

The regression of this chapter is a removal, so `-S` is the right tool. The call `.lower()` no longer exists in the sources:

```
$ git log --format='%h %ad %<(10)%an %s' --date=short -S'.lower()'
d926d3c 2026-09-10 Ravi Menon Speed up normalize with a precompiled pattern
dd70d9e 2026-09-09 Asha Rao Move normalize into scorekit/text.py
b250238 2026-09-07 Lab User Add exact-match scorer
$ git log --format='%h %ad %<(10)%an %s' --date=short -G'\.lower\(\)'
d926d3c 2026-09-10 Ravi Menon Speed up normalize with a precompiled pattern
dd70d9e 2026-09-09 Asha Rao Move normalize into scorekit/text.py
8e2cac1 2026-09-09 Ravi Menon Strip punctuation before comparing
b250238 2026-09-07 Lab User Add exact-match scorer
```

Three commits changed the count: the one that introduced the call, the one that moved the function to another file (one file lost an occurrence, another gained one), and `d926d3c`, where it disappeared for good. `-G` adds a fourth, which edited the line and kept the call. Add `-p` to read the evidence in the same command:

```
$ git log -p --format='%h %s' -S'.lower()' -1
d926d3c Speed up normalize with a precompiled pattern

diff --git a/scorekit/text.py b/scorekit/text.py
index 48081af..3075545 100644
--- a/scorekit/text.py
+++ b/scorekit/text.py
@@ -1,10 +1,10 @@
-import string
+import re

-_PUNCTUATION = str.maketrans('', '', string.punctuation)
+_PUNCTUATION = re.compile(r'[^\w\s]')

def normalize(text):
- text = text.lower().translate(_PUNCTUATION)
+ text = _PUNCTUATION.sub('', text)
return ' '.join(text.split())
```

By default the patch shows only the files that matched; `--pickaxe-all` shows the whole commit.

Picture.

```
occurrences of ".lower()" per commit along main
b250238 0 -> 1 reported by -S and -G (added)
8e2cac1 1 -> 1 reported by -G only (line edited, call kept)
dd70d9e 1 -> 1 reported by both (metrics.py 1 -> 0, text.py 0 -> 1: counted per file)
d926d3c 1 -> 0 reported by -S and -G (removed)
```

In production. Search for the shortest string that is specific. A whole statement copied from an old file stops matching as soon as anyone edits its tail, and `-S` then reports that edit as the removal (Lab 11.2). The pickaxe searches only the commits of stage 1:

```
$ git log --oneline -S'write_report'
$ git log --oneline --all -S'write_report'
a10f9a1 Write a report when --report <path> is given
a8e8550 Add per-row report writer
```

Use `--all` when the code may live on another branch, and `--reflog` when it may have been rebased away (section 14A.14).

14A.12 Line history: `git log -L`

`git log -L` follows a range of lines backwards and prints, for each commit that touched those lines, the part of its patch that concerns them. The range is `<start>,<end>:<file>`, or `:<funcname>:<file>` for a whole function:

```
$ git log --oneline -s -L :token_f1:scorekit/metrics.py
ec4fad7 Simplify token overlap in token_f1
9c8df98 Reformat sources: four-space indent, double quotes
c0d33a5 Add a tokens helper and use it in the metrics
bb5eb57 Add token-level F1 scorer
```

`-s` suppresses the patches and leaves the list. Compare it with the nine commits of `git log --follow` on the same file in section 14A.10: four of them touched this function. The oldest one predates both renames, which `-L` followed on its own. One line works the same way:

```
$ git log --format='%h %an: %s' -L 2,2:scorekit/config.py
389337a Lab User: Raise the pass mark to 0.75

diff --git a/scorekit/config.py b/scorekit/config.py
index c22ffa6..5efef40 100644
--- a/scorekit/config.py
+++ b/scorekit/config.py
@@ -2,1 +2,1 @@
-PASS_MARK = 0.7
+PASS_MARK = 0.75
682bc71 Lab User: Add config module with the pass mark

diff --git a/scorekit/config.py b/scorekit/config.py
new file mode 100644
index 0000000..c22ffa6
--- /dev/null
+++ b/scorekit/config.py
@@ -0,0 +2,1 @@
+PASS_MARK = 0.7
```

And this is the function at the centre of the regression, limited to its two most recent changes:

```
$ git log --format='%h %an: %s' -L :normalize:scorekit/text.py -2
9c8df98 Asha Rao: Reformat sources: four-space indent, double quotes

diff --git a/scorekit/text.py b/scorekit/text.py
index 3075545..d404d86 100644
--- a/scorekit/text.py
+++ b/scorekit/text.py
@@ -6,5 +6,5 @@
 def normalize(text):
- text = _PUNCTUATION.sub('', text)
- return ' '.join(text.split())
+ text = _PUNCTUATION.sub("", text)
+ return " ".join(text.split())

d926d3c Ravi Menon: Speed up normalize with a precompiled pattern

diff --git a/scorekit/text.py b/scorekit/text.py
index 48081af..3075545 100644
--- a/scorekit/text.py
+++ b/scorekit/text.py
@@ -6,5 +6,5 @@
 def normalize(text):
- text = text.lower().translate(_PUNCTUATION)
+ text = _PUNCTUATION.sub('', text)
 return ' '.join(text.split())
```

Read from the bottom: Ravi's commit replaced the line that lowercased the text; the formatter then changed quotes and indentation. `-L` shows deleted lines, which `git blame` cannot (section 14A.16).

How the range is found matters. A number is a line number in the starting revision, HEAD unless you name another, not in your working tree. `:<funcname>` is a regular expression matched against the lines that Git considers function headers, the same lines it prints after `@@` in hunk headers; the range runs to the next such line. For languages where the default guess is poor, a `diff` attribute selects a language-specific pattern (Chapter 14C). The errors are explicit:

```
$ git log --oneline -L 40,45:scorekit/config.py
fatal: file scorekit/config.py has only 3 lines
[exit status: 128]
$ git log --oneline -L :no_such_function:scorekit/text.py
fatal: -L parameter 'no_such_function' starting at line 1: no match
[exit status: 128]
$ git log --oneline -L 2,2:scorekit/config.py -- scorekit/config.py
fatal: -L<range>:<file> cannot be used with pathspec
[exit status: 128]
```

14A.13 Finding a deleted file, and a file on another branch

A file that is not in the working tree cannot be opened, blamed or grepped. Its history is still there. Ask for the commits that deleted something:

```
$ git log --diff-filter=D --name-status --format="%h %ad %an: %s" --date=short
d9d075d 2026-09-14 Ravi Menon: Remove the experimental BLEU scorer

D      scorekit/bleu.py
```

If you remember part of the name, a pathspec with a wildcard finds every commit that touched a matching path, under any directory it lived in:

```
$ git log --oneline -- scorekit/bleu.py
d9d075d Remove the experimental BLEU scorer
9c8df98 Reformat sources: four-space indent, double quotes
c0d33a5 Add a tokens helper and use it in the metrics
dd70d9e Move normalize into scorekit/text.py
18bb23e Rename the scorer module to metrics
d38a5aa Move sources into the scorekit package
$ git log --oneline -- '*bleu*'
d9d075d Remove the experimental BLEU scorer
9c8df98 Reformat sources: four-space indent, double quotes
c0d33a5 Add a tokens helper and use it in the metrics
dd70d9e Move normalize into scorekit/text.py
18bb23e Rename the scorer module to metrics
d38a5aa Move sources into the scorekit package
d369eac Add experimental BLEU scorer
```

The first command shows the history under the last name, with the deletion on top. The second also reaches the commit that created the file as `bleu.py` at the top level. The `--` is required: without it Git must decide whether the word is a revision or a path, and for a file that no longer exists it gives up with "ambiguous argument". The deleting commit says why:

```
$ git show --stat d9d075d
commit d9d075d63dcc94ce04c8e404e06b5cc3173180c2
Author: Ravi Menon <ravi@example.com>
Date:   Mon Sep 14 12:31:00 2026 +0530

    Remove the experimental BLEU scorer

    Nothing imports it, and for one reference its unigram precision duplicates token_f1.
    ROUGE-L covers the use case it was added for.

 docs/metrics.md | 1 -
 scorekit/bleu.py | 15 -----
 2 files changed, 16 deletions(-)
```

The content lives in the parent of the deleting commit, not in the deleting commit itself:

```
$ git show d9d075d:scorekit/bleu.py
fatal: path 'scorekit/bleu.py' does not exist in 'd9d075d'
[exit status: 128]
```

`git show <commit>~1:<path>` prints it, and  `git restore --source=<commit>~1 -- <path>` writes it into the working tree. The label is the one Chapter 11 gives the command: it overwrites an existing file of that name without asking. Here none exists:

```
$ git restore --source=d9d075d~1 -- scorekit/bleu.py
$ git status --short
?? scorekit/bleu.py
```

The file comes back untracked. Git has not undone the deletion; you have a copy of old content to add or to discard. Lab 11.3 walks through the whole sequence.

A file can also be "missing" because it was never on this branch:

```
$ git log --oneline -- scorekit/report.py
$ git log --oneline --all -- scorekit/report.py
a8e8550 Add per-row report writer
$ git branch --contains $(git log --all --format=%h -1 -- scorekit/report.py)
feat/report
```

`--all -- <path>` searches the history of every ref for the path. `git branch --contains <commit>`, with `-a` to include remote-tracking branches, then says where that commit lives.

14A.14 Set questions: `--left-right`, `--cherry-pick`, `--ancestry-path`, `--reflog`

Which side has what. Chapter 7 counted divergence with `--left-right --count`, and Chapter 10 explained patch IDs and `--cherry-mark`. For reading, `--cherry-pick` drops the commits that have an equivalent on the other side:

```
$ git log --oneline --left-right main...feat/report
< e376e5b Mention the nightly run in the README
< 2652768 Add a separate pass mark for the nightly set
```

```
> a10f9a1 Write a report when --report <path> is given
> f37a7d8 Mention the nightly run in the README
> a8e8550 Add per-row report writer
$ git log --oneline --left-right --cherry-pick main...feat/report
< 2652768 Add a separate pass mark for the nightly set
> a10f9a1 Write a report when --report <path> is given
> a8e8550 Add per-row report writer
```

The README commit exists on both sides as two commits with the same patch. With `--cherry-pick` both disappear, and what remains is the work that differs.

Through which commits did a change arrive. `dd70d9e` was made on a side branch. `dd70d9e..v0.2.0` is "everything the release has that this commit does not have", which includes commits that have nothing to do with it:

```
# dd70d9e moved normalize() into its own module, on a side branch. Everything v0.2.0 has that dd70d9e lacks:
$ git log --oneline dd70d9e..v0.2.0
c0c9a30 Add nightly evaluation set
7823232 Add ROUGE-L metric
9c8df98 Reformat sources: four-space indent, double quotes
d926d3c Speed up normalize with a precompiled pattern
be9ad1b Fix crash when --limit is not given
074d492 Send warnings to stderr
8dc82cb Add --limit option to the runner
72134f3 Document the runner's exit status
8657273 Merge branch 'feat/text-utils'
389337a Raise the pass mark to 0.75
4da82b5 Document the metrics
b4b066b Skip rows with an empty reference
c0d33a5 Add a tokens helper and use it in the metrics
```

`--ancestry-path` keeps only the commits of the range that are descendants of the excluded commit, the chain along which the change travelled:

```
# Only the commits that are descendants of dd70d9e as well: the path the change travelled.
$ git log --oneline --ancestry-path dd70d9e..v0.2.0
c0c9a30 Add nightly evaluation set
7823232 Add ROUGE-L metric
9c8df98 Reformat sources: four-space indent, double quotes
d926d3c Speed up normalize with a precompiled pattern
be9ad1b Fix crash when --limit is not given
074d492 Send warnings to stderr
8dc82cb Add --limit option to the runner
72134f3 Document the runner's exit status
8657273 Merge branch 'feat/text-utils'
c0d33a5 Add a tokens helper and use it in the metrics
```

Three commits of `main` that were made in parallel are gone. The oldest merge on that path is the merge that first carried the change onto another line:

```
$ git log --oneline --merges --ancestry-path dd70d9e..main | tail -n 1
8657273 Merge branch 'feat/text-utils'
```

In a repository where every change lands through a merge commit, this is "which pull request brought this commit in". With several levels of merges, the oldest merge on the path may be a merge into another topic branch; then intersect with `--first-parent` of the target branch.

What only the reflogs know. After an amend or a rebase, the old commits are reachable from no branch. `--all` does not find them; `--reflog` adds every commit mentioned in any reflog as a starting tip:

```
$ git log --oneline --all --grep='nightly run'
f37a7d8 Mention the nightly run in the README
$ git log --oneline --reflog --grep='nightly run'
e376e5b Mention the nightly run in the README
f37a7d8 Mention the nightly run in the README
$ git log --oneline -g -2
87f090b HEAD@{0}: commit (amend): Describe the nightly job in the README
e376e5b HEAD@{1}: cherry-pick: Mention the nightly run in the README
```

The commit that the amend replaced, `e376e5b`, is found only by the second search. `--reflog` is a stage 1 option: it widens the walk, and any filter, including the pickaxe, then applies to the wider set. `-g` (`--walk-reflogs`) is different: it lists reflog entries in order, which Chapter 13 uses for recovery.

14A.15 Output: pretty formats and `git shortlog`

Chapter 6, section 6.12, introduced `--format`. For investigations, a fixed one-line format with a short date and the author answers most questions at a glance:

```
$ git log -4 --format='%h %ad %<(10,trunc)%an %s' --date=short
e376e5b 2026-09-15 Ravi Menon Mention the nightly run in the README
```

```
2652768 2026-09-15 Lab User Add a separate pass mark for the nightly set
ca7e2b7 2026-09-14 Asha Rao Fail fast on an empty reference
d9d075d 2026-09-14 Ravi Menon Remove the experimental BLEU scorer
$ git log -3 --format='%h %ar%x09s'
e376e5b 2 hours ago Mention the nightly run in the README
2652768 32 minutes ago Add a separate pass mark for the nightly set
ca7e2b7 22 hours ago Fail fast on an empty reference
```

Placeholder	Prints	Placeholder	Prints
%h, %H	abbreviated, full commit ID	%s, %b	subject, body
%an, %ae	author name, email	%cn, %ce	committer name, email
%ad, %cd	author date, committer date (shaped by --date=)	%ar, %cr	the same, relative
%d	ref names, as --decorate	%p	abbreviated parent IDs
%<(N), %<(N,trunc)	pad, or pad and truncate, the next placeholder to N columns	%(trailers:...)	trailers of the message

Relative dates such as "2 hours ago" are computed from the clock at the moment you run the command; paste absolute dates into a report. Trailers (Chapter 6, section 6.9) carry attribution that the author field cannot, such as the people behind a squash merge:

```
$ git log -1 --format='%h %s%n author: %an%n with: %(trailers:key=Co-authored-by,valueonly,separator=%x2C )' 7823232
7823232 Add ROUGE-L metric
author: Lab User
with: Asha Rao <asha@example.com>, Ravi Menon <ravi@example.com>
```

`git shortlog` groups a set of commits by person. With `-s` it counts, with `-n` it sorts by count:

```
$ git shortlog -sn HEAD
14 Lab User
8 Asha Rao
6 Ravi Menon
$ git shortlog -sn --no-merges --group=author --group=trailer:co-authored-by HEAD
13 Lab User
9 Asha Rao
7 Ravi Menon
```

The second form adds a second grouping key, the `Co-authored-by` trailer, so Asha and Ravi are credited for the squashed commit as well. Without `-s`, `shortlog` is a draft of release notes:

```
$ git shortlog --no-merges v0.1.0..v0.2.0
Asha Rao (4):
  Move normalize into scorekit/text.py
  Add a tokens helper and use it in the metrics
  Reformat sources: four-space indent, double quotes
  Add nightly evaluation set

Lab User (7):
  Skip rows with an empty reference
  Document the metrics
  Raise the pass mark to 0.75
  Document the runner's exit status
  Add --limit option to the runner
  Fix crash when --limit is not given
  Add ROUGE-L metric

Ravi Menon (3):
  Strip punctuation before comparing
  Send warnings to stderr
  Speed up normalize with a precompiled pattern
```

Commit counts measure commits. They say nothing about the size or the value of the work, and a squash-merge workflow moves the counts to whoever presses the merge button.

Root cause. `git shortlog` with no revision prints nothing when its standard input is not a terminal, as in a script, a hook or a CI job: in that situation it reads a log from standard input instead of walking from HEAD ([git-shortlog](#)). Always pass the revision.

```
# Without a revision, and with standard input that is not a terminal, shortlog reads a log from standard input:
$ git shortlog -sn </dev/null
$ git log v0.2.0..main | git shortlog -sn
2 Asha Rao
2 Ravi Menon
1 Lab User
```

14A.16 `git blame`: what it tells you and what it does not

In one sentence.  `git blame` annotates every line of a file, as it is in one revision, with the commit that last changed that line.

Analogy. A "last edited by" column beside a document. The column is reliable about the last edit and silent about everything else: the edit before it, the paragraph that used to stand here, and whether the last editor wrote the sentence or only fixed its spacing. The analogy also breaks on "line": Git decides which old line corresponds to which new line by running a diff, so the attribution depends on how the diff aligned them.

Precisely. Blame starts from the file in the given revision (HEAD by default; the working tree file if it has uncommitted changes, which are attributed to "Not Committed Yet") and compares it with the same file in the parent commit. Lines that the diff shows as unchanged are passed to the parent; lines that it shows as added stay with the commit. The parent is then treated the same way, until every line has an owner. "The origin of lines is automatically followed across whole-file renames", and "the report does not tell you anything about lines which have been deleted or replaced" ([git-blame](#)).

Inside .git. Nothing is written. The work is one diff per commit that touched the file, so blame on an old, busy file in a large repository takes noticeable time.

See it.

```
$ git blame scorekit/text.py
d926d3c5 (Ravi Menon 2026-09-10 13:36:00 +0530 1) import re
dd70d9e4 (Asha Rao 2026-09-09 11:01:00 +0530 2)
9c8df982 (Asha Rao 2026-09-11 10:16:00 +0530 3) _PUNCTUATION = re.compile(r"^\w\s]")
dd70d9e4 (Asha Rao 2026-09-09 11:01:00 +0530 4)
dd70d9e4 (Asha Rao 2026-09-09 11:01:00 +0530 5)
dd70d9e4 (Asha Rao 2026-09-09 11:01:00 +0530 6) def normalize(text):
9c8df982 (Asha Rao 2026-09-11 10:16:00 +0530 7)     text = _PUNCTUATION.sub("", text)
9c8df982 (Asha Rao 2026-09-11 10:16:00 +0530 8)     return " ".join(text.split())
c0d33a57 (Asha Rao 2026-09-09 13:21:00 +0530 9)
c0d33a57 (Asha Rao 2026-09-09 13:21:00 +0530 10)
c0d33a57 (Asha Rao 2026-09-09 13:21:00 +0530 11) def tokens(text):
9c8df982 (Asha Rao 2026-09-11 10:16:00 +0530 12)     return normalize(text).split()
```

Each row: the abbreviated commit ID, the author and author date of that commit, the line number in this revision, and the line. `-s` drops author and date, `-e` shows the email, `--date=short` shortens the date. `-L` limits the output, with the same range syntax as `git log -L`:

```
$ git blame -s -L 6,8 scorekit/text.py
dd70d9e4 6) def normalize(text):
9c8df982 7)     text = _PUNCTUATION.sub("", text)
9c8df982 8)     return " ".join(text.split())
$ git blame -s -L :normalize scorekit/text.py
dd70d9e4 6) def normalize(text):
9c8df982 7)     text = _PUNCTUATION.sub("", text)
9c8df982 8)     return " ".join(text.split())
c0d33a57 9)
c0d33a57 10)
$ git blame --date=short -L '/^def load/,+3' scorekit/runner.py
b4b066b4 (Lab User 2026-09-09 9) def load(path):
9c8df982 (Asha Rao 2026-09-11 10)     rows = []
9c8df982 (Asha Rao 2026-09-11 11)     for line in open(path):
```

A revision before the path asks about an older state of the file. The extra column appears whenever a line comes from a commit in which the file had another name:

```
$ git blame -s v0.1.0 -- scorekit/metrics.py
bb5eb57c scorer.py 1) from collections import Counter
bb5eb57c scorer.py 2)
bb5eb57c scorer.py 3)
b2502383 scorer.py 4) def normalize(text):
b2502383 scorer.py 5)     return ' '.join(text.lower().split())
b2502383 scorer.py 6)
b2502383 scorer.py 7)
b2502383 scorer.py 8) def exact_match(prediction, reference):
b2502383 scorer.py 9)     return normalize(prediction) == normalize(reference)
bb5eb57c scorer.py 10)
bb5eb57c scorer.py 11)
bb5eb57c scorer.py 12) def token_f1(prediction, reference):
bb5eb57c scorer.py 13)     pred = normalize(prediction).split()
bb5eb57c scorer.py 14)     ref = normalize(reference).split()
bb5eb57c scorer.py 15)     overlap = sum((Counter(pred) & Counter(ref)).values())
bb5eb57c scorer.py 16)     if overlap == 0:
bb5eb57c scorer.py 17)         return 0.0
bb5eb57c scorer.py 18)     precision = overlap / len(pred)
bb5eb57c scorer.py 19)     recall = overlap / len(ref)
bb5eb57c scorer.py 20)     return 2 * precision * recall / (precision + recall)
```

With a range, blame stops at the excluded commit. Lines older than that are attributed to the boundary and marked with a caret:

```
$ git blame -s v0.1.0..main -- scorekit/config.py
^682bc71 1) # Thresholds used by the runner.
389337aa 2) PASS_MARK = 0.75
26527689 3) NIGHTLY_PASS_MARK = 0.6
```

For scripts, `--porcelain` and `--line-porcelain` print full IDs and one header per attribution, including the commit before the blamed one (previous), the key to stepping further back:

```
$ git blame --line-porcelain -L 7,7 scorekit/text.py
9c8df98229e009115b3044edb1181d11075ecbbb 7 7 1
author Asha Rao
author-mail <asha@example.com>
author-time 1789101960
author-tz +0530
committer Asha Rao
committer-mail <asha@example.com>
committer-time 1789101960
committer-tz +0530
summary Reformat sources: four-space indent, double quotes
previous d926d3c5a77b586297eb25335c33fed4aee6f053 scorekit/text.py
filename scorekit/text.py
      text = _PUNCTUATION.sub("", text)
```

What blame does not tell you.

You want to know	Blame gives	Use instead
Who wrote this logic	who last changed the line, possibly a formatter or a rename of a variable	<code>--ignore-rev</code> (14A.17), <code>git log -L</code> (14A.12)
Who removed the line that used to be here	nothing: there is no row for a line that no longer exists	<code>git log -S</code> (14A.11), <code>git log -L</code>
Why it was changed	an ID	<code>git show <id></code> : the message and the whole commit
Who to ask about code that arrived in a squash merge	the person who ran the merge	the trailers of the commit, or the pull request (14A.18)
Whether the line is the bug	nothing: the defect may be a line that is missing, or one in another file	<code>git bisect</code> (14A.20)

The regression of this chapter shows the second row. Before commit `d926d3c`, line 7 lowercased the text. Today a different line 7 exists, and blame reports on that one:

```
# The line that lowercased the text is gone. Blame annotates lines that exist; it has no row for it.
$ git blame -s d926d3c~1 -L 7,7 -- scorekit/text.py
dd70d9e4 7)      text = text.lower().translate(_PUNCTUATION)
$ git blame -s -L 7,7 scorekit/text.py
9c8df982 7)      text = _PUNCTUATION.sub("", text)
```

Picture. One line of `scorekit/text.py`, and where each way of asking stops (sections 14A.17 and 14A.18 supply the options):

```
line 7 on main:      text = _PUNCTUATION.sub("", text)

9c8df98  formatter: quotes changed          <- plain blame stops here
d926d3c  Ravi: line replaced, .lower() gone  <- blame --ignore-rev 9c8df98 stops here
dd70d9e  Asha: function moved into text.py  <- blame d926d3c~1 stops here (the older line 7)
8e2cac1  Ravi: older line written, metrics.py <- blame -C dd70d9e reaches this
```

In production. Treat a blame row as a pointer into history, never as a verdict about a person. The next command after `git blame` is `git show <id>`. Blame an old revision when the current one is uninformative: `git blame <id>~1 -- <path>` steps behind any commit, and the `previous` field of the porcelain format supplies the argument.

14A.17 Blame through a reformatting commit: `-w`, `--ignore-rev`, `--ignore-revs-file`

One formatter run makes blame nearly useless for a whole file. This is the loader function of the runner; Asha's commit of 11 September is the formatter run:

```
$ git blame --date=short -L '/^def load/,+7' scorekit/runner.py
b4b066b4 (Lab User 2026-09-09 9) def load(path):
9c8df982 (Asha Rao 2026-09-11 10)     rows = []
9c8df982 (Asha Rao 2026-09-11 11)     for line in open(path):
9c8df982 (Asha Rao 2026-09-11 12)         row = json.loads(line)
9c8df982 (Asha Rao 2026-09-11 13)         if not row["reference"].strip():
ca7e2b7f (Asha Rao 2026-09-14 14)             raise ValueError("empty reference in row %d" % row["id"])
9c8df982 (Asha Rao 2026-09-11 15)         rows.append(row)
```

`-w` ignores whitespace when blame compares a commit with its parent. Lines that the formatter only re-indented fall through to their real origin:

```
$ git blame --date=short -w -L '/^def load/,+7' scorekit/runner.py
b4b066b4 (Lab User 2026-09-09 9) def load(path):
b4b066b4 (Lab User 2026-09-09 10)     rows = []
b4b066b4 (Lab User 2026-09-09 11)     for line in open(path):
b4b066b4 (Lab User 2026-09-09 12)         row = json.loads(line)
9c8df982 (Asha Rao 2026-09-11 13)         if not row["reference"].strip():
```

```
ca7e2b7f (Asha Rao 2026-09-14 14)         raise ValueError("empty reference in row %d" % row["id"])
b4b066b4 (Lab User 2026-09-09 15)         rows.append(row)
```

Line 13 stays with the formatter, because there it also changed the quotes (section 14A.5). `--ignore-rev <rev>` goes further: it removes one commit from consideration, "as if the change never happened", and blames its lines "on the previous commit that changed that line or nearby lines":

```
$ git blame --date=short --ignore-rev 9c8df98 -L '/^def load/,+7' scorekit/runner.py
b4b066b4 (Lab User 2026-09-09 9) def load(path):
b4b066b4 (Lab User 2026-09-09 10)     rows = []
b4b066b4 (Lab User 2026-09-09 11)     for line in open(path):
b4b066b4 (Lab User 2026-09-09 12)         row = json.loads(line)
b4b066b4 (Lab User 2026-09-09 13)         if not row["reference"].strip():
ca7e2b7f (Asha Rao 2026-09-14 14)             raise ValueError("empty reference in row %d" % row["id"])
b4b066b4 (Lab User 2026-09-09 15)         rows.append(row)
```

Typing the option every time does not scale. A file lists the commits to ignore, one full object ID per line, with `#` comments; `--ignore-revs-file` reads it:

```
$ cat .git-blame-ignore-revs
# Formatter run: four-space indent, double quotes
9c8df98229e009115b3044edb1181d11075ecbbb
$ git blame -s --ignore-revs-file .git-blame-ignore-revs scorekit/text.py
d926d3c5 1) import re
dd70d9e4 2)
d926d3c5 3) _PUNCTUATION = re.compile(r"^[^\w\s]")
dd70d9e4 4)
dd70d9e4 5)
dd70d9e4 6) def normalize(text):
d926d3c5 7)     text = _PUNCTUATION.sub("", text)
dd70d9e4 8)     return " ".join(text.split())
c0d33a57 9)
c0d33a57 10)
c0d33a57 11) def tokens(text):
c0d33a57 12)     return normalize(text).split()
```

 `git config set blame.ignoreRevsFile` makes it the default for this repository, and `blame.markIgnoredLines` flags every line whose attribution passed through an ignored commit with a `?`:

```
$ git config set blame.ignoreRevsFile .git-blame-ignore-revs
$ git config set blame.markIgnoredLines true
$ git blame -s scorekit/text.py
d926d3c5 1) import re
dd70d9e4 2)
?d926d3c 3) _PUNCTUATION = re.compile(r"^[^\w\s]")
dd70d9e4 4)
dd70d9e4 5)
dd70d9e4 6) def normalize(text):
?d926d3c 7)     text = _PUNCTUATION.sub("", text)
?dd70d9e 8)     return " ".join(text.split())
c0d33a57 9)
c0d33a57 10)
c0d33a57 11) def tokens(text):
?c0d33a5 12)     return normalize(text).split()
```

The mark is worth having. The attribution of an ignored line is a guess: Git maps it to the line of the parent that looks like its origin, and for a line that the ignored commit really introduced there is no origin (`blame.markUnblamableLines` marks those with `*`).

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git blame</code> with any of these options	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged
<code>git config set blame.ignoreRevsFile <path></code>	unchanged	unchanged	unchanged	unchanged	one key in <code>.git/config</code>	unchanged	unchanged
committing <code>.git-blame-ignore-revs</code>	a tracked file	as for any commit	as for any commit	moves	new objects	after a push: the file	the blame view starts to use it

GitHub, not Git. The file name is a convention, not a Git default: Git reads no ignore file unless the option or the configuration key names one. GitHub's blame view does look for a file with exactly the name `.git-blame-ignore-revs` in the root directory of the repository and hides the listed revisions ([GitHub Docs: ignore commits in the blame view](#)). Using that name serves both. The configuration key is local: every clone has to set it, which is one line in a setup script.

Two failures are common enough to show. The file must contain full IDs:

```
$ cat ../short-ids.txt
9c8df98
$ git blame -s --ignore-revs-file ../short-ids.txt -L 6,8 scorekit/text.py
```

```
fatal: invalid object name: 9c8df98
[exit status: 128]
```

And once the key is set, the file must exist in every commit you visit. Check out a commit that predates the file, and blame fails:

```
# The list is committed on main. An older commit does not have the file:
$ git switch --detach --quiet v0.2.0
$ git blame -s -L 6,8 scorekit/text.py
fatal: could not open object name list: .git-blame-ignore-revs
[exit status: 128]
$ git blame -s --ignore-revs-file "" -L 6,8 scorekit/text.py
fatal: could not open object name list: .git-blame-ignore-revs
[exit status: 128]
```

An empty `--ignore-revs-file ""` clears the list of revisions; it does not stop Git from opening the configured file. The local manual documents a prefix for path-valued settings: with `:(optional)`, "the configuration variable is treated as if it does not exist, if the named path does not exist" ([git-config](#)):

```
$ git config set blame.ignoreRevsFile ':(optional).git-blame-ignore-revs'
$ git blame -s -L 6,8 scorekit/text.py
dd70d9e4 6) def normalize(text):
9c8df982 7)     text = _PUNCTUATION.sub("", text)
9c8df982 8)     return " ".join(text.split())
$ git switch --quiet main
$ git blame -s -L 6,8 scorekit/text.py
dd70d9e4 6) def normalize(text):
?d926d3c 7)     text = _PUNCTUATION.sub("", text)
?dd70d9e 8)     return " ".join(text.split())
```

At `v0.2.0` blame runs and nothing is ignored; back on `main` the list applies again.

In production. List only commits that change no behavior: formatter runs, mass renames of a symbol, license header updates. Listing a commit that also changed logic hides that logic from blame. Rebasing or squashing the listed commit changes its ID and silently empties the entry, so add the ID after the commit has reached the shared branch.

14A.18 Blame and moved code: renames, `-M`, `-C`, and squash merges

A rename of the whole file does not disturb blame. `scorekit/metrics.py` was `scorer.py` when most of its lines were written, and blame says so in a column of its own:

```
$ git blame -s -w scorekit/metrics.py
c0d33a57 scorekit/metrics.py 1) from scorekit.text import normalize, tokens
b2502383 scorer.py          2)
b2502383 scorer.py          3)
b2502383 scorer.py          4) def exact_match(prediction, reference):
b2502383 scorer.py          5)     return normalize(prediction) == normalize(reference)
bb5eb57c scorer.py          6)
bb5eb57c scorer.py          7)
bb5eb57c scorer.py          8) def token_f1(prediction, reference):
c0d33a57 scorekit/metrics.py 9)     pred = tokens(prediction)
c0d33a57 scorekit/metrics.py 10)    ref = tokens(reference)
ec4fad75 scorekit/metrics.py 11)    overlap = len(set(pred) & set(ref))
bb5eb57c scorer.py          12)    if overlap == 0:
bb5eb57c scorer.py          13)        return 0.0
bb5eb57c scorer.py          14)    precision = overlap / len(pred)
bb5eb57c scorer.py          15)    recall = overlap / len(ref)
bb5eb57c scorer.py          16)    return 2 * precision * recall / (precision + recall)
```

Code that moves between files is different. Commit `dd70d9e` cut `normalize()` out of `metrics.py` and pasted it into a new file. To plain blame, a new file is new lines:

```
# dd70d9e moved normalize() from scorekit/metrics.py into the new file scorekit/text.py.
$ git blame -s dd70d9e -- scorekit/text.py
dd70d9e4 1) import string
dd70d9e4 2)
dd70d9e4 3) _PUNCTUATION = str.maketrans('', '', string.punctuation)
dd70d9e4 4)
dd70d9e4 5)
dd70d9e4 6) def normalize(text):
dd70d9e4 7)     text = text.lower().translate(_PUNCTUATION)
dd70d9e4 8)     return ' '.join(text.split())
$ git blame -s -C dd70d9e -- scorekit/text.py
dd70d9e4 scorekit/text.py 1) import string
bb5eb57c scorer.py        2)
8e2cac1a scorekit/metrics.py 3) _PUNCTUATION = str.maketrans('', '', string.punctuation)
8e2cac1a scorekit/metrics.py 4)
bb5eb57c scorer.py        5)
b2502383 scorer.py        6) def normalize(text):
8e2cac1a scorekit/metrics.py 7)     text = text.lower().translate(_PUNCTUATION)
8e2cac1a scorekit/metrics.py 8)     return ' '.join(text.split())
```

With `-C`, blame also searches the other files that the commit modified for the origin of the lines, and attributes them to the commits that wrote them there. The options form a ladder of cost:

Option	Additionally looks for the origin of a line in	Minimum match
<code>-M</code>	other places of the same file: lines moved or copied within it	20 alphanumeric characters
<code>-C</code>	files modified in the same commit	40 alphanumeric characters
<code>-C -C</code>	also every file of the parent, for commits that create the blamed file	40
<code>-C -C -C</code>	every file of the parent, in every commit	40

The minimum is why moved code is sometimes not recognized. After later edits, only short fragments of the moved block survive in today's file, and the default threshold rejects them; a lower one, given as `-C<num>`, accepts one:

```
# Today only two short lines of that move are left unchanged. -C wants 40 alphanumeric characters.
$ git blame -s -C --ignore-rev 9c8df98 -L 6,8 scorekit/text.py
dd70d9e4 6) def normalize(text):
d926d3c5 7)     text = _PUNCTUATION.sub("", text)
dd70d9e4 8)     return " ".join(text.split())
$ git blame -s -C5 --ignore-rev 9c8df98 -L 6,8 scorekit/text.py
b2502383 scorer.py      6) def normalize(text):
d926d3c5 scorekit/text.py 7)     text = _PUNCTUATION.sub("", text)
dd70d9e4 scorekit/text.py 8)     return " ".join(text.split())
```

A copy from a file that the commit did not modify needs the second `-C`:

```
$ git blame -s -L 9,12 data/nightly.jsonl
c0c9a304 9) {"id": 9, "prediction": "seven", "reference": "7"}
c0c9a304 10) {"id": 10, "prediction": "oxygen", "reference": "oxygen"}
c0c9a304 11) {"id": 11, "prediction": "Canberra", "reference": "canberra"}
c0c9a304 12) {"id": 12, "prediction": "the speed of light", "reference": "speed of light"}
$ git blame -s -C -L 9,12 data/nightly.jsonl
c0c9a304 9) {"id": 9, "prediction": "seven", "reference": "7"}
c0c9a304 10) {"id": 10, "prediction": "oxygen", "reference": "oxygen"}
c0c9a304 11) {"id": 11, "prediction": "Canberra", "reference": "canberra"}
c0c9a304 12) {"id": 12, "prediction": "the speed of light", "reference": "speed of light"}
$ git blame -s -C -C -L 9,12 data/nightly.jsonl
633e3e60 data/smoke.jsonl 9) {"id": 9, "prediction": "seven", "reference": "7"}
633e3e60 data/smoke.jsonl 10) {"id": 10, "prediction": "oxygen", "reference": "oxygen"}
c0c9a304 data/nightly.jsonl 11) {"id": 11, "prediction": "Canberra", "reference": "canberra"}
c0c9a304 data/nightly.jsonl 12) {"id": 12, "prediction": "the speed of light", "reference": "speed of light"}
```

`-M` handles a block that changes position inside a file. A new commit puts `tokens()` above `normalize()`:

```
# A new commit moves tokens() above normalize() and changes nothing else.
$ git blame -s scorekit/text.py
d926d3c5 1) import re
dd70d9e4 2)
9c8df982 3) _PUNCTUATION = re.compile(r"^[^\w\s]")
dd70d9e4 4)
dd70d9e4 5)
fad07941 6) def tokens(text):
fad07941 7)     return normalize(text).split()
fad07941 8)
fad07941 9)
dd70d9e4 10) def normalize(text):
9c8df982 11)     text = _PUNCTUATION.sub("", text)
9c8df982 12)     return " ".join(text.split())
$ git blame -s -M scorekit/text.py
d926d3c5 1) import re
dd70d9e4 2)
9c8df982 3) _PUNCTUATION = re.compile(r"^[^\w\s]")
dd70d9e4 4)
dd70d9e4 5)
c0d33a57 6) def tokens(text):
9c8df982 7)     return normalize(text).split()
fad07941 8)
fad07941 9)
dd70d9e4 10) def normalize(text):
9c8df982 11)     text = _PUNCTUATION.sub("", text)
9c8df982 12)     return " ".join(text.split())
```

Without `-M`, the lines that moved up are blamed on the commit that moved them. With it, lines 6 and 7 go back to their earlier commits; the two blank lines are too short to qualify.

Squash merges. A squash merge records the content of a branch as one new commit (Chapter 8). Blame on the target branch then knows one commit and one author:

```
$ git blame -L 15,18 scorekit/rouge.py
78232321 (Lab User 2026-09-11 15:01:00 +0530 15) def rouge_l(prediction, reference):
78232321 (Lab User 2026-09-11 15:01:00 +0530 16)     pred = tokens(prediction)
78232321 (Lab User 2026-09-11 15:01:00 +0530 17)     ref = tokens(reference)
78232321 (Lab User 2026-09-11 15:01:00 +0530 18)     lcs = _lcs(pred, ref)
```

```
$ git blame -L 15,18 feat/rouge-l -- scorekit/rouge.py
80ee55fe (Asha Rao 2026-09-11 12:11:00 +0530 15) def rouge_l(prediction, reference):
80ee55fe (Asha Rao 2026-09-11 12:11:00 +0530 16)     pred = tokens(prediction)
80ee55fe (Asha Rao 2026-09-11 12:11:00 +0530 17)     ref = tokens(reference)
80ee55fe (Asha Rao 2026-09-11 12:11:00 +0530 18)     lcs = _lcs(pred, ref)
```

On `main`, the person who squashed "wrote" the function. The branch, while it still exists, has the real history. Nothing links the two in the graph:

```
$ git log -1 --format='%h %an: %s%n%(trailers:key=Co-authored-by)' 7823232
7823232 Lab User: Add ROUGE-L metric
Co-authored-by: Asha Rao <asha@example.com>
Co-authored-by: Ravi Menon <ravi@example.com>

$ git branch --no-merged main
feat/report
feat/rouge-l
```

`git branch --no-merged main` lists `feat/rouge-l` as unmerged for the same reason. After the branch is deleted, the `Co-authored-by` trailers and the pull request are the only record of who wrote which part. That is a real cost of squash-merge workflows, to be weighed against the linear history they give (Chapter 17).

The opposite question, "which merge put this line on `main`", is `--first-parent`: blame then stops at merges instead of descending into the merged branch.

```
$ git blame -s --first-parent -L 10,12 scorekit/text.py
86572733 10) def normalize(text):
9c8df982 11)     text = _PUNCTUATION.sub("", text)
9c8df982 12)     return " ".join(text.split())
```

14A.19 A method for finding the origin of a bug

The tools are now in place. This is the order to use them in, as an instance of the root-cause framework of Chapter 1.

Step	Do	With
1. Pin the symptom	Reduce it to one command with a yes-or-no outcome	the smallest input that shows the problem
2. Fix two endpoints	One commit known good, one known bad	tags, deploy records, <code>git describe</code>
3. Locate	Find the code that produces the behavior	<code>git grep</code>
4. Read the lines' history	Last change per line; through formatters and moves	<code>git blame -w -C --ignore-rev</code> , <code>git log -L</code>
5. Look for what is gone	A removed call, check or string	<code>git log -S, -G</code>
6. Search by behavior	When steps 3 to 5 give no suspect or several	<code>git bisect</code>
7. Confirm	The suspect shows the symptom, its parent does not	a test at both commits
8. Measure the spread	Which releases and branches contain the commit	<code>git tag --contains</code> , <code>git branch -a --contains</code>
9. Choose the fix	Revert or fix forward; add the test from step 1	Chapter 11

Steps 3 to 5 are fast and need an idea of where the fault is. Step 6 is slower and needs no idea at all, only the test. Do not skip step 7: every tool before it produces a suspect, not a proof.

Worked on regression 1. The symptom is the score line of section 14A.1. The documentation says what `exact_match` should do, and `git grep` finds the code:

```
$ python3 -B -m scorekit.runner data/smoke.jsonl
rows=10 exact_match=0.400 token_f1=0.444 rouge_l=0.489
[exit status: 1]
$ git grep -n "exact_match" -- docs
docs/metrics.md:3:- exact_match: 1 when prediction and reference are equal after normalization, else 0.
$ git grep -n -W "def exact_match"
scorekit/metrics.py:4:def exact_match(prediction, reference):
scorekit/metrics.py:5-     return normalize(prediction) == normalize(reference)
```

"Equal after normalization" points at `normalize()`:

```
$ git grep -n "def normalize"
scorekit/text.py:6:def normalize(text):
$ git blame --date=short -L :normalize scorekit/text.py
dd70d9e4 (Asha Rao 2026-09-09 6) def normalize(text):
9c8df982 (Asha Rao 2026-09-11 7)     text = _PUNCTUATION.sub("", text)
9c8df982 (Asha Rao 2026-09-11 8)     return " ".join(text.split())
c0d33a57 (Asha Rao 2026-09-09 9)
c0d33a57 (Asha Rao 2026-09-09 10)
```

Both body lines are owned by the formatter commit. Look through it:

```
$ git blame --date=short --ignore-rev 9c8df98 -L :normalize scorekit/text.py
dd70d9e4 (Asha Rao 2026-09-09 6) def normalize(text):
d926d3c5 (Ravi Menon 2026-09-10 7)     text = _PUNCTUATION.sub("", text)
dd70d9e4 (Asha Rao 2026-09-09 8)     return " ".join(text.split())
c0d33a57 (Asha Rao 2026-09-09 9)
c0d33a57 (Asha Rao 2026-09-09 10)
```

One line was last changed for a real reason, by `d926d3c`, the day before the formatter ran. Read that commit:

```
$ git show d926d3c
commit d926d3c5a77b586297eb25335c33fed4aee6f053
Author: Ravi Menon <ravi@example.com>
Date: Thu Sep 10 13:36:00 2026 +0530

    Speed up normalize with a precompiled pattern

    str.translate built a table lookup per call. One compiled pattern is faster on the nightly set.

diff --git a/scorekit/text.py b/scorekit/text.py
index 48081af..3075545 100644
--- a/scorekit/text.py
+++ b/scorekit/text.py
@@ -1,10 +1,10 @@
-import string
+import re

-_PUNCTUATION = str.maketrans('', '', string.punctuation)
+_PUNCTUATION = re.compile(r'[\w\s]')

def normalize(text):
- text = text.lower().translate(_PUNCTUATION)
+ text = _PUNCTUATION.sub('', text)
+ return ' '.join(text.split())
```

The message promises speed and no change of behavior. The patch replaces `text.lower().translate(...)` with a pattern substitution, and `.lower()` is gone. `docs/metrics.md` still says "Normalization lowercases the text". The pickaxe confirms that this commit is where the call left the code base (step 5):

```
$ git log --format='%h %ad %<(10)%an %s' --date=short -S'.lower()'
d926d3c 2026-09-10 Ravi Menon Speed up normalize with a precompiled pattern
dd70d9e 2026-09-09 Asha Rao Move normalize into scorekit/text.py
b250238 2026-09-07 Lab User Add exact-match scorer
```

Step 7, on both sides of the suspect, in two temporary worktrees (🟢 `git worktree add`, Chapter 25) so that the main working tree is not touched:

```
# One call that separates right from wrong, at the suspect and at its parent, without touching the working tree:
$ git worktree add --detach --quiet ../before d926d3c~1
$ git worktree add --detach --quiet ../after d926d3c
$ (cd ../before && python3 -B -c 'from scorekit.metrics import exact_match; print(exact_match("Paris", "paris"))')
True
$ (cd ../after && python3 -B -c 'from scorekit.metrics import exact_match; print(exact_match("Paris", "paris"))')
False
$ git worktree remove ../before && git worktree remove ../after
```

Step 8:

```
$ git tag --contains d926d3c
v0.2.0
$ git branch --all --contains d926d3c
  feat/report
  feat/rouge-l
* main
$ git describe --contains d926d3c
v0.2.0~3
$ git log --oneline d926d3c..main -- scorekit/text.py
9c8df98 Reformat sources: four-space indent, double quotes
```

The answer for the CTO: the harness regressed, not the model. Commit `d926d3c` by Ravi on Thursday 10 September dropped lowercasing from normalization while optimizing it. Release `v0.2.0` contains the commit, and so does every branch cut since. Nothing after it has touched the function except the formatter. The fix is one line, plus a test with a capitalized answer.

This case was kind: the faulty line still existed and blame pointed near it. When the fault is in a file nobody suspects, the method arrives at step 6.

14A.20 `git bisect`: binary search over commits

In one sentence. 🟡 `git bisect` finds the commit that changed a property by checking out a commit in the middle of the suspects, asking you for a verdict, and halving the suspects until one is left.

Analogy. Looking up a word in a printed dictionary: open in the middle, see whether your word comes before or after, repeat with that half. The analogy breaks in two places. History is a graph, not a list, so "the middle" has to be computed. And a dictionary is sorted by construction, while "good before, bad after" is an assumption about your project that `bisect` cannot verify.

Precisely. You give one commit that has the property ("bad" or "new") and at least one ancestor that lacks it ("good" or "old"). The candidates are the commits reachable from the bad one and not from any good one, the range of section 14A.8. Git assumes there is a single first bad commit: "all its descendants are 'bad' and all the other commits are 'good'". It tests the candidate X that maximizes $\min(\text{ancestors of X among the candidates}, N \text{ minus that number})$, the commit that splits the candidates most evenly ([Fighting regressions with git bisect](#)). A "bad" verdict keeps X and its ancestors; a "good" verdict removes them.

The arithmetic. Every verdict halves the candidates, so k verdicts can separate 2^k candidates, and N candidates need about $\log_2(N)$ verdicts.

Candidates	Verdicts needed, about	A linear search, worst case
20	4 or 5	20
1,000	10	1,000
1,000,000	20	1,000,000

Doubling the history adds one test. This is why "it worked in the last release" is enough of a starting point, however long ago that was.

See it. `v0.1.0` scored the smoke set correctly, `main` does not, and there are twenty candidates:

```
$ git rev-list --count v0.1.0..main
20
$ git bisect start
status: waiting for both 'good' and 'bad' commits
$ git bisect bad main
status: waiting for 'good' commit(s), 'bad' commit known
$ git bisect good v0.1.0
Bisecting: 9 revisions left to test after this (roughly 3 steps)
[074d49249bcd54bc33537fe26519dd38e3c3ba85] Send warnings to stderr
```

Git checked out `074d492`. That commit and its ancestors account for ten of the twenty candidates, so either verdict leaves ten suspects, one of which already has a verdict: "9 revisions left to test after this". The step count is an estimate computed from the number of candidates ([bisect.c](#)). HEAD is now detached:

```
$ git status
HEAD detached at 074d492
You are currently bisecting, started from branch 'main'.
(use "git bisect reset" to get back to the original branch)

nothing to commit, working tree clean
```

Run the test. At this commit it gives neither answer:

```
$ python3 -B -m scorekit.runner data/smoke.jsonl 2>&1 | tail -n 1
KeyError: '--limit'
$ git bisect skip
Bisecting: 9 revisions left to test after this (roughly 3 steps)
[d9d075d63dcc94cc04c8e404e06b5cc3173180c2] Remove the experimental BLEU scorer
```

The runner crashes for a reason that has nothing to do with scoring. Calling this "bad" would be a false statement about the property under investigation, and section 14A.21 shows what a false statement costs. `git bisect skip` says "no verdict", and Git picks another commit. From here the test answers:

```
$ python3 -B -m scorekit.runner data/smoke.jsonl 2>&1 | tail -n 1
rows=10 exact_match=0.400 token_f1=0.444 rouge_l=0.489
$ git bisect bad
Bisecting: 8 revisions left to test after this (roughly 3 steps)
[72134f33df03602df3b2fa85f5b7dfb0aa08c190] Document the runner's exit status
```

```
$ python3 -B -m scorekit.runner data/smoke.jsonl 2>&1 | tail -n 1
rows=10 exact_match=0.800 token_f1=0.889
$ git bisect good
Bisecting: 3 revisions left to test after this (roughly 2 steps)
[9c8df98229e009115b3044edb1181d11075ecbbb] Reformat sources: four-space indent, double quotes
```

```
$ python3 -B -m scorekit.runner data/smoke.jsonl 2>&1 | tail -n 1
rows=10 exact_match=0.400 token_f1=0.489
$ git bisect bad
Bisecting: 2 revisions left to test after this (roughly 1 step)
[be9ad1b4894348317c1e7ccb8276b8ad4760fb89] Fix crash when --limit is not given
$ python3 -B -m scorekit.runner data/smoke.jsonl 2>&1 | tail -n 1
rows=10 exact_match=0.800 token_f1=0.889
```



```
$ git bisect good
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[d926d3c5a77b586297eb25335c33fed4aee6f053] Speed up normalize with a precompiled pattern
```

```
$ python3 -B -m scorekit.runner data/smoke.jsonl 2>&1 | tail -n 1
rows=10 exact_match=0.400 token_f1=0.489
$ git bisect bad
d926d3c5a77b586297eb25335c33fed4aee6f053 is the first 'bad' commit
commit d926d3c5a77b586297eb25335c33fed4aee6f053
Author: Ravi Menon <ravi@example.com>
Date: Thu Sep 10 13:36:00 2026 +0530

    Speed up normalize with a precompiled pattern

    str.translate built a table lookup per call. One compiled pattern is faster on the nightly set.

scorekit/text.py | 6 +++---
1 file changed, 3 insertions(+), 3 deletions(-)
```

Five verdicts and one skip, and the same commit that blame and the pickaxe named. Bisect got there without knowing which file to look at.

Inside .git. A bisection is state, kept until you end it:

```
$ git for-each-ref --format="%(objectname:short) %(refname)" refs/bisect
d926d3c refs/bisect/bad
682bc71 refs/bisect/good-682bc717015f0ab43b873d1df1ac9da0ba01c393
72134f3 refs/bisect/good-72134f33df03602df3b2fa85f5b7dfb0aa08c190
be9ad1b refs/bisect/good-be9ad1b4894348317c1e7ccb8276b8ad4760fb89
074d492 refs/bisect/skip-074d49249bcd54bc33537fe26519dd38e3c3ba85
$ ls .git | grep BISECT
BISECT_ANCESTORS_OK
BISECT_EXPECTED_REV
BISECT_LOG
BISECT_NAMES
BISECT_START
BISECT_TERMS
$ cat .git/BISECT_START .git/BISECT_TERMS
main
bad
good
$ git rev-parse --abbrev-ref HEAD
HEAD
```

Every verdict is a ref under `refs/bisect/`. `BISECT_START` holds where to return to, `BISECT_TERMS` the two words in use, `BISECT_LOG` the session so far. `git bisect log` prints that file, as commands that would reproduce the session:

```
$ git bisect log
git bisect start
# status: waiting for both 'good' and 'bad' commits
# bad: [e376e5b709142415a87d49ca10af71977aaeabdc] Mention the nightly run in the README
git bisect bad e376e5b709142415a87d49ca10af71977aaeabdc
# status: waiting for 'good' commit(s), 'bad' commit known
# good: [682bc717015f0ab43b873d1df1ac9da0ba01c393] Add config module with the pass mark
git bisect good 682bc717015f0ab43b873d1df1ac9da0ba01c393
# skip: [074d49249bcd54bc33537fe26519dd38e3c3ba85] Send warnings to stderr
git bisect skip 074d49249bcd54bc33537fe26519dd38e3c3ba85
# bad: [d9d075d63dcc94ce04c8e404e06b5cc3173180c2] Remove the experimental BLEU scorer
git bisect bad d9d075d63dcc94ce04c8e404e06b5cc3173180c2
# good: [72134f33df03602df3b2fa85f5b7dfb0aa08c190] Document the runner's exit status
git bisect good 72134f33df03602df3b2fa85f5b7dfb0aa08c190
# bad: [9c8df98229e009115b3044edb1181d11075ecbbb] Reformat sources: four-space indent, double quotes
git bisect bad 9c8df98229e009115b3044edb1181d11075ecbbb
# good: [be9ad1b4894348317c1e7ccb8276b8ad4760fb89] Fix crash when --limit is not given
git bisect good be9ad1b4894348317c1e7ccb8276b8ad4760fb89
# bad: [d926d3c5a77b586297eb25335c33fed4aee6f053] Speed up normalize with a precompiled pattern
git bisect bad d926d3c5a77b586297eb25335c33fed4aee6f053
# first 'bad' commit: [d926d3c5a77b586297eb25335c33fed4aee6f053] Speed up normalize with a precompiled pattern
```

`git bisect reset` returns to the starting branch and deletes all of it:

```
$ git bisect reset
Previous HEAD position was d926d3c Speed up normalize with a precompiled pattern
Switched to branch 'main'
$ git status --short --branch
## main
$ git for-each-ref refs/bisect | wc -l
0
```

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git bisect start</code> [<code><bad></code> <code><good></code> ...]	once both ends are known: files of the chosen commit	matches that commit	detached at that commit	unchanged	<code>BISECT_START</code> , <code>BISECT_LOG</code> , <code>BISECT_TERMS</code> and others; <code>refs/bisect/bad</code> , <code>refs/bisect/good-<code><id></code></code> ; one HEAD reflog entry per checkout	unchanged	unchanged

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git bisect good, bad, skip</code>	files of the next commit	matches it	detached at the next commit	unchanged	one more ref under <code>refs/bisect/</code> ; <code>BISECT_LOG</code> grows	unchanged	unchanged
<code>git bisect reset</code>	files of the starting branch	matches it	attached to the starting branch again	unchanged	every <code>BISECT_*</code> file and <code>refs/bisect/*</code> ref removed	unchanged	unchanged
<code>git bisect start --no-checkout</code>	unchanged	unchanged	unchanged	unchanged	as above, and the ref <code>BISECT_HEAD</code> names the commit to test	unchanged	unchanged

Picture. The candidates in the order of the first-parent line, with the verdicts of this session (the two commits of the side branch are omitted):

v0.1.0	8e2cac1 ... 8657273	72134f3	8dc82cb	074d492	be9ad1b	d926d3c	9c8df98 ... d9d075d ... main
good		good(3)		skip(1)	good(5)	BAD(6)	bad(4) bad(2) bad
						^ first bad commit	

In production. Bisect needs commits that can be tested one by one. A history of small commits that each build (Chapter 6, section 6.10) gives an answer you can read in a minute; a history of large squashes gives an answer that is a 2,000-line diff. Start a bisection from a clean working tree: Git refuses to check out the next commit over uncommitted changes to files it has to replace.

```
# An uncommitted edit in scorekit/config.py, a file that differs between the commits to visit:
$ git bisect start main v0.1.0
Bisecting: 9 revisions left to test after this (roughly 3 steps)
error: Your local changes to the following files would be overwritten by checkout:
    scorekit/config.py
Please commit your changes or stash them before you switch branches.
Aborting
[exit status: 1]
$ git bisect reset
```

14A.21 `git bisect run` and the exit-code protocol

Answering by hand is error-prone and does not scale past a handful of steps. `git bisect run <cmd>` runs a command at every step and takes the verdict from its exit status ([git-bisect](#)):

Exit status	Meaning
0	good (old)
1 to 127, except 125	bad (new)
125	this commit cannot be tested: skip it
anything else, such as 128 and above	abort the bisection

The script for regression 1 has three outcomes, because the question has three answers:

```
$ cat ../check-em.sh
#!/bin/sh
# Exit 0 if this commit scores the smoke set correctly, 1 if it does not,
# 125 if the runner cannot produce a score line at all (untestable: skip).
out=$(python3 -B -m scorekit.runner data/smoke.jsonl 2>/dev/null)
case "$out" in
    *exact_match=0.800*) exit 0 ;;
    *exact_match=*)      exit 1 ;;
    *)                  exit 125 ;;
esac
$ ../check-em.sh; echo "exit status on main: $?"
exit status on main: 1
```

It lives one directory above the repository. The manual advises keeping the test outside the repository "to prevent interactions between the bisect, make and test processes and the scripts"; a tracked script would be replaced by each commit's own version of it, or be absent in older commits. The whole search is now two commands:

```
$ git bisect start main v0.1.0
Bisecting: 9 revisions left to test after this (roughly 3 steps)
[074d49249bcd54bc33537fe26519dd38e3c3ba85] Send warnings to stderr
$ git bisect run ../check-em.sh
running '../check-em.sh'
Bisecting: 9 revisions left to test after this (roughly 3 steps)
[d9d075d63dcc94ce04c8e404e06b5cc3173180c2] Remove the experimental BLEU scorer
running '../check-em.sh'
Bisecting: 8 revisions left to test after this (roughly 3 steps)
[72134f33df03602df3b2fa85f5b7dfb0aa08c190] Document the runner's exit status
running '../check-em.sh'
Bisecting: 3 revisions left to test after this (roughly 2 steps)
```

```
[9c8df98229e009115b3044edb1181d11075ecbbb] Reformat sources: four-space indent, double quotes
running './check-em.sh'
Bisecting: 2 revisions left to test after this (roughly 1 step)
[be9ad1b4894348317c1e7ccb8276b8ad4760fb89] Fix crash when --limit is not given
running './check-em.sh'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[d926d3c5a77b586297eb25335c33fed4aee6f053] Speed up normalize with a precompiled pattern
running './check-em.sh'
d926d3c5a77b586297eb25335c33fed4aee6f053 is the first 'bad' commit
commit d926d3c5a77b586297eb25335c33fed4aee6f053
Author: Ravi Menon <ravi@example.com>
Date: Thu Sep 10 13:36:00 2026 +0530

    Speed up normalize with a precompiled pattern

    str.translate built a table lookup per call. One compiled pattern is faster on the nightly set.

scorekit/text.py | 6 +++---
1 file changed, 3 insertions(+), 3 deletions(-)
bisect found first 'bad' commit
```

The log shows that the script's 125 was recorded as a skip:

```
$ git bisect log
# bad: [e376e5b709142415a87d49ca10af71977aaeabdc] Mention the nightly run in the README
# good: [682bc717015f0ab43b873d1df1ac9da0ba01c393] Add config module with the pass mark
git bisect start 'main' 'v0.1.0'
# skip: [074d49249bcd54bc33537fe26519dd38e3c3ba85] Send warnings to stderr
git bisect skip 074d49249bcd54bc33537fe26519dd38e3c3ba85
# bad: [d9d075d63dcc94ce04c8e404e06b5cc3173180c2] Remove the experimental BLEU scorer
git bisect bad d9d075d63dcc94ce04c8e404e06b5cc3173180c2
# good: [72134f33df03602df3b2fa85f5b7dfb0aa08c190] Document the runner's exit status
git bisect good 72134f33df03602df3b2fa85f5b7dfb0aa08c190
# bad: [9c8df98229e009115b3044edb1181d11075ecbbb] Reformat sources: four-space indent, double quotes
git bisect bad 9c8df98229e009115b3044edb1181d11075ecbbb
# good: [be9ad1b4894348317c1e7ccb8276b8ad4760fb89] Fix crash when --limit is not given
git bisect good be9ad1b4894348317c1e7ccb8276b8ad4760fb89
# bad: [d926d3c5a77b586297eb25335c33fed4aee6f053] Speed up normalize with a precompiled pattern
git bisect bad d926d3c5a77b586297eb25335c33fed4aee6f053
# first 'bad' commit: [d926d3c5a77b586297eb25335c33fed4aee6f053] Speed up normalize with a precompiled pattern
$ git bisect reset
Previous HEAD position was d926d3c Speed up normalize with a precompiled pattern
Switched to branch 'main'
```

Why 125 matters. Suppose the test were the runner's own exit status: 0 when the score passes, 1 when it does not. A crash also exits with 1. The first commit tested would be marked bad, Git would search among its ancestors, where the score is fine, and it would report the commit that introduced the crash as the first bad commit. The answer would be precise, reproducible and wrong. Lab 11.5 runs exactly this. A test for bisect must distinguish "has the property", "does not have it" and "cannot tell".

Exit codes outside the protocol stop the run instead of producing a verdict:

```
$ git bisect start main v0.1.0
Bisecting: 9 revisions left to test after this (roughly 3 steps)
[074d49249bcd54bc33537fe26519dd38e3c3ba85] Send warnings to stderr
$ git bisect run sh -c 'exit 200'
running 'sh' '-c' 'exit 200'
error: bisect run failed: exit code 200 from 'sh' '-c' 'exit 200' is < 0 or >= 128
[exit status: 56]
$ git bisect reset
Previous HEAD position was 074d492 Send warnings to stderr
Switched to branch 'main'
```

A mistyped path is caught in a way worth knowing. Shells return 127 for "command not found", which the protocol would read as "bad". Git 2.55 does not take 126 or 127 at face value on the first step: it runs the command on the known good commit as well, and stops when it gets the same code there.

```
$ git bisect start main v0.1.0
Bisecting: 9 revisions left to test after this (roughly 3 steps)
[074d49249bcd54bc33537fe26519dd38e3c3ba85] Send warnings to stderr
$ git bisect run ./check-em.sh
running './check-em.sh'
'./check-em.sh': ./check-em.sh: No such file or directory
[682bc717015f0ab43b873d1df1ac9da0ba01c393] Add config module with the pass mark
running './check-em.sh'
'./check-em.sh': ./check-em.sh: No such file or directory
error: bogus exit code 127 for 'good' revision
[074d49249bcd54bc33537fe26519dd38e3c3ba85] Send warnings to stderr
[exit status: 1]
$ git bisect reset
Previous HEAD position was 074d492 Send warnings to stderr
Switched to branch 'main'
```

In production. For AI/ML work the test is often slow or noisy: an evaluation run, a benchmark, a training smoke test. Three rules follow. Make the test as small as the symptom allows; one input that separates good from bad beats the full suite. Make it deterministic: fix seeds, pin the

data, and compare against a threshold with a margin; a flaky test gives bisect false verdicts, and it will converge on an innocent commit without complaint. And rebuild whatever is derived from the sources at every step: installed packages, compiled extensions, caches. A commit is only its tracked files; the environment around them is your responsibility.

14A.22 Custom terms, replay, `--first-parent`, and the pitfalls

Terms. "Good" and "bad" are awkward when you are looking for an improvement. `old` and `new` are built-in alternatives, and `--term-old` and `--term-new` let you choose words. Here the question is which commit fixed the crash:

```
# Since which commit does the runner work again without --limit? 8dc82cb is known to crash.
$ git bisect start --term-old broken --term-new fixed
status: waiting for both 'broken' and 'fixed' commits
$ git bisect fixed main
status: waiting for 'broken' commit(s), 'fixed' commit known
$ git bisect broken 8dc82cb
Bisecting: 5 revisions left to test after this (roughly 3 steps)
[78232321c82cb23d6fb732e4c8ee614c6f005919] Add ROUGE-L metric
$ git bisect terms
Your current terms are 'broken' for the old state
and 'fixed' for the new state.
```

The subcommands are now named after your words. For `git bisect run` the mapping stays positional: exit status 0 means the old state, whatever it is called.

```
# For bisect run, exit status 0 means the OLD state. grep exits 0 when it finds the crash.
$ git bisect run sh -c 'python3 -B -m scorekit.runner data/smoke.jsonl 2>&1 | grep -q KeyError'
running 'sh' '-c' 'python3 -B -m scorekit.runner data/smoke.jsonl 2>&1 | grep -q KeyError'
Bisecting: 2 revisions left to test after this (roughly 1 step)
[be9ad1b4894348317c1e7ccb8276b8ad4760fb89] Fix crash when --limit is not given
running 'sh' '-c' 'python3 -B -m scorekit.runner data/smoke.jsonl 2>&1 | grep -q KeyError'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[074d49249bcd54bc33537fe26519dd38e3c3ba85] Send warnings to stderr
running 'sh' '-c' 'python3 -B -m scorekit.runner data/smoke.jsonl 2>&1 | grep -q KeyError'
be9ad1b4894348317c1e7ccb8276b8ad4760fb89 is the first 'fixed' commit
commit be9ad1b4894348317c1e7ccb8276b8ad4760fb89
Author: Lab User <you@example.com>
Date: Thu Sep 10 12:31:00 2026 +0530

    Fix crash when --limit is not given

    Without the option the runner stopped with KeyError: '--limit'.

scorekit/runner.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
bisect found first 'fixed' commit
```

Within one session the vocabulary is fixed:

```
$ git bisect start --term-old broken --term-new fixed
status: waiting for both 'broken' and 'fixed' commits
$ git bisect bad main 2>&1 | head -n 2
error: Invalid command: you're currently in a fixed/broken bisect
fatal: unknown command: 'bad'
$ git bisect reset
Already on 'main'
```

Log and replay. `git bisect log` output is replayable. Save it, and a session can be repeated, handed to a colleague, or repaired:

```
$ git bisect replay ../bisect.log
status: waiting for both 'broken' and 'fixed' commits
be9ad1b4894348317c1e7ccb8276b8ad4760fb89 is the first 'fixed' commit
commit be9ad1b4894348317c1e7ccb8276b8ad4760fb89
Author: Lab User <you@example.com>
Date: Thu Sep 10 12:31:00 2026 +0530

    Fix crash when --limit is not given

    Without the option the runner stopped with KeyError: '--limit'.

scorekit/runner.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git bisect reset
Already on 'main'
```

If you gave a wrong verdict, save the log, delete the wrong line and everything after it, run `git bisect reset`, and replay the edited file. You continue from the last correct verdict instead of starting over (Lab 11.4).

More controls.

Form	Effect
<code>git bisect start <bad> <good> --<path>...</code>	considers only commits that touch the paths; fewer steps when you know the area
<code>git bisect start --first-parent ...</code>	follows only first parents; a regression that arrived through a merge is attributed to the merge commit
<code>git bisect start --no-checkout ...</code>	does not touch the working tree; sets <code>BISECT_HEAD</code> for tests that read objects directly
<code>git bisect skip <rev> or <range></code>	marks commits untestable in advance
<code>git bisect visualize (view)</code>	shows the remaining candidates: in <code>gitk</code> if a graphical session is detected, otherwise with <code>git log</code> , and it accepts log options such as <code>--oneline</code>
<code>git bisect reset <commit></code>	ends the session and checks out that commit instead of the starting branch

`--first-parent` changes the candidate set, and with it the path of the search:

```
$ git rev-list --count v0.1.0..main
20
$ git rev-list --count --first-parent v0.1.0..main
18
$ git bisect start --first-parent main v0.1.0
Bisecting: 8 revisions left to test after this (roughly 3 steps)
[be9ad1b4894348317c1e7ccb8276b8ad4760fb89] Fix crash when --limit is not given
$ git bisect reset
Previous HEAD position was be9ad1b Fix crash when --limit is not given
Switched to branch 'main'
```

Use it when the commits inside merged branches are not required to work on their own.

Pitfalls.

- **The property is not monotonic.** A bug that was introduced, fixed and introduced again has more than one first bad commit. Bisect finds one transition and cannot tell you there are others. If the result looks implausible, test the reported commit and its parent yourself.
- **Good and bad are swapped, or unrelated.** Git checks that every good commit is an ancestor of the bad one:

```
$ git bisect start v0.1.0 main
Some 'good' revs are not ancestors of the 'bad' rev.
git bisect cannot work properly in this case.
Maybe you mistook 'good' and 'bad' revs?
[exit status: 1]
$ git bisect reset
```

- **Skipped commits next to the culprit.** If the first bad commit is adjacent to skipped ones, Git cannot decide between them and ends with a list of candidates instead of one commit. The manual states it: "if you skip a commit adjacent to the one you are looking for, Git will be unable to tell exactly which of those commits was the first bad one."
- **The culprit is a merge.** Both parents are good and the merge is bad: the fault is in the combination, a merge that was textually clean and semantically wrong (Chapter 8).
- **The cause is not in the repository.** A dependency that floated to a new version, a changed data file outside Git, a different machine. Bisect then reports whatever commit the noise happened to select, or every commit is bad. Check that the good commit is still good today, in today's environment, before you start.
- **A shallow clone.** Bisect can only visit commits that exist locally. In a clone made with `--depth`, as CI checkouts often are (Chapter 20A), the good commit may not be present at all.
- **Forgetting to reset.** Until `git bisect reset`, HEAD is detached. Commits made in that state belong to no branch (Chapter 7, section 7.7).

Version note. Older behavior: a finished bisection leaves you on the last tested commit until you run `git bisect reset`. Current behavior on Git 2.55: unchanged, as shown above. Since Git 2.56 teaches `git bisect` an option `--reset-when-found[=<where>]`, which runs the reset automatically, to the original state or to the culprit ([RelNotes 2.56.0](#)); not run here, and its exact usage was not checked against the 2.56 manual. Recommended: on 2.55, end every `git bisect` run in a script with an explicit `git bisect reset`.

14A.23 `git grep`

 `git grep` searches the content that Git tracks. Compared with a recursive `grep` it skips `.git`, ignored build output and untracked files by default, and it can search any revision without checking it out.

```
$ git grep -n PASS_MARK
scorekit/config.py:2:PASS_MARK = 0.75
scorekit/config.py:3:NIGHTLY_PASS_MARK = 0.6
scorekit/runner.py:4:from scorekit.config import PASS_MARK
scorekit/runner.py:28:    return 0 if em >= PASS_MARK else 1
$ git grep -c tokens
docs/metrics.md:1
```

```
scorekit/metrics.py:3
scorekit/rouge.py:3
scorekit/text.py:1
$ git grep -l normalize -- scorekit
scorekit/metrics.py
scorekit/text.py
```

`-n` adds line numbers, `-c` counts matches per file, `-l` lists file names only. `-p` shows the function a match belongs to, and `-W` prints that whole function:

```
$ git grep -n -p "overlap == 0"
scorekit/metrics.py:8: def token_f1(prediction, reference):
scorekit/metrics.py:12:     if overlap == 0:
$ git grep -W "set(pred)"
scorekit/metrics.py: def token_f1(prediction, reference):
scorekit/metrics.py:     pred = tokens(prediction)
scorekit/metrics.py:     ref = tokens(reference)
scorekit/metrics.py:     overlap = len(set(pred) & set(ref))
scorekit/metrics.py:     if overlap == 0:
scorekit/metrics.py:         return 0.0
scorekit/metrics.py:     precision = overlap / len(pred)
scorekit/metrics.py:     recall = overlap / len(ref)
scorekit/metrics.py:     return 2 * precision * recall / (precision + recall)
```

A revision before the pathspec searches that tree. The exit status is 1 when nothing matches, which makes `git grep -q` usable as a test:

```
$ git grep -n "lower()" v0.1.0
v0.1.0:scorekit/metrics.py:5: return ' '.join(text.lower().split())
$ git grep -n "lower()" main
[exit status: 1]
$ git grep -n -i bleu v0.2.0 -- docs
v0.2.0:docs/metrics.md:6:- bleu1: unigram precision with a brevity penalty (experimental).
```

Several revisions give a quick history of one line, with each hit prefixed by its revision:

```
$ git grep -n 'PASS_MARK = ' v0.1.0 v0.2.0 main -- scorekit/config.py
v0.1.0:scorekit/config.py:2: PASS_MARK = 0.7
v0.2.0:scorekit/config.py:2: PASS_MARK = 0.75
main:scorekit/config.py:2: PASS_MARK = 0.75
main:scorekit/config.py:3: NIGHTLY_PASS_MARK = 0.6
```

What is searched	Option
tracked files, as they are in the working tree	default
tracked files, as they are in the index	<code>--cached</code>
a commit or tree	<code><rev> before --</code>
tracked and untracked files	<code>--untracked</code>
files in a directory that is not a repository	<code>--no-index</code>

Patterns are basic regular expressions unless you pass `-E` (extended), `-P` (Perl-compatible) or `-F` (fixed string); `-i` ignores case and `-w` matches whole words. `-e <p1> --and -e <p2>` requires both patterns on one line, and `--not` negates one.

The division of labour with the pickaxe: `git grep` searches states, "where is this text at commit X"; `git log -S` and `-G` search transitions, "which commit changed it". To learn when a string came and went, ask the pickaxe; looping `git grep` over every revision reads every tree.

14A.24 `git whatchanged` is deprecated

Older tutorials show `git whatchanged` for "the log with the files each commit touched". On Git 2.55 it refuses to run:

```
$ git whatchanged -2
'git whatchanged' is nominated for removal.

hint: You can replace 'git whatchanged <opts>' with:
hint:   git log <opts> --raw --no-merges
hint: Or make an alias:
hint:   git config set --global alias.whatchanged 'log --raw --no-merges'

If you still use this command, here's what you can do:

- read https://git-scm.com/docs/BreakingChanges.html
- check if anyone has discussed this on the mailing
  list and if they came up with something that can
  help you: https://lore.kernel.org/git/?q=git%20whatchanged
- send an email to <git@vger.kernel.org> to let us
  know that you still use this command and were unable
  to determine a suitable replacement
```

```
fatal: refusing to run without --i-still-use-this
[exit status: 128]
```

The replacement it names prints the same thing:

```
$ git log --raw --no-merges --oneline -2
e376e5b Mention the nightly run in the README
:100644 100644 bfae817 08f9933 M      README.md
2652768 Add a separate pass mark for the nightly set
:100644 100644 5efef40 12c95c2 M      scorekit/config.py
$ git whatchanged --i-still-use-this --oneline -2
e376e5b Mention the nightly run in the README
:100644 100644 bfae817 08f9933 M      README.md
2652768 Add a separate pass mark for the nightly set
:100644 100644 5efef40 12c95c2 M      scorekit/config.py
```

Each raw line is: old mode, new mode, old blob ID, new blob ID, status letter, path. In practice `git log --name-status` or `git log --stat` reads better.

Outdated advice. "Use `git whatchanged` to see what each commit changed." The command is "merely `git log` with different defaults", carries a deprecation warning in its manual page since Git 2.51, and is scheduled for removal in Git 3.0 ([git-whatchanged](#), [BreakingChanges](#)). Write `git log --raw --no-merges`, and do not add `--i-still-use-this` to scripts.

Going the other way, Git 2.52 added `git last-modified`, which reports the commit that last touched each path. Its manual page marks it experimental, so its interface may change ([git-last-modified](#)):

```
# A newer plumbing command answers "which commit last touched each path". It is marked experimental.
$ git last-modified -r -- scorekit
26527689cd0ae7dc79b9627c6bea60ade2590465      scorekit/config.py
ca7e2b7fcc44f8d5a494107b68440c3d4ec23b55      scorekit/runner.py
ec4fad75a7e73526c6694c2559094bbe05e00980      scorekit/metrics.py
78232321c82cb23d6fb732e4c8ee614c6f005919      scorekit/rouge.py
9c8df98229e009115b3044edb1181d11075ecbbb      scorekit/text.py
9c8df98229e009115b3044edb1181d11075ecbbb      scorekit/__init__.py
```

14A.25 What can go wrong

Symptom	Diagnosis	Fix	Prevention
A review diff shows deletions of things the branch never touched	Two endpoints were compared and the target moved on: <code>git merge-base A B</code> is not A	<code>git diff A...B</code>	Three dots, or <code>--merge-base</code> , for review (14A.2)
<code>git log -- <path></code> ends at a rename	Path limiting matches names	<code>--follow</code> , or list the old names	Rename and edit in separate commits (14A.4)
<code>--follow</code> ends early although the file is older	Rename plus rewrite below 50% similarity: <code>git show --stat -M30% <commit></code>	Read on under the old path	As above
<code>-S</code> names a commit that only edited the line	The search string was longer than the stable part	Shorter string; check with <code>git show</code> and <code>-G</code>	Read the patch before naming a commit (14A.11)
<code>-S</code> or <code>--grep</code> finds nothing, the code exists on a branch	The walk started at HEAD	<code>--all</code> , or <code>--reflog</code>	State the walk in the query
<code>--since=<date></code> returns a strange subset	Date without a time; or committer dates out of order	Full timestamp; <code>--since-as-filter</code>	14A.9
Blame shows one author and one date for most lines	A formatter or squash commit: <code>git show --stat <id></code>	<code>-w</code> , <code>--ignore-rev</code> , or blame the branch	<code>.git-blame-ignore-revs</code> plus <code>blame.ignoreRevsFile</code>
fatal: could not open object name list on every blame	<code>blame.ignoreRevsFile</code> names a file this commit lacks	<code>:(optional) prefix</code> , or <code>git config unset blame.ignoreRevsFile</code>	Set the key with the prefix from the start
fatal: invalid object name from blame	A short ID in the ignore file	Replace with <code>git rev-parse <id></code> output	Append IDs with <code>git rev-parse</code>
<code>git log -L</code> traces the wrong line	Line number taken from a modified working file	A <code>/regex/</code> or <code>:funcname range</code>	Content anchors, not numbers (Lab 11.7)
Bisect names a commit that cannot be the cause	A wrong verdict, a flaky test, or a crash counted as bad: read <code>git bisect log</code>	Edit the log, <code>reset</code> , <code>replay</code> ; exit 125 for untestable commits	A three-way test script (14A.21)
Bisect ends with "only skipped commits left"	The culprit is next to untestable commits	Test those by hand, with the build fix applied temporarily	Keep every commit buildable

14A.26 When not to use it, and dangerous edge cases

When another tool is the right one.

- Do not bisect what you can read. If the pickaxe or blame names one commit, and a test at that commit and its parent confirms it, you are done.
- Do not bisect before you have checked that the known-good commit is still good in today's environment.
- Do not use blame to decide who is responsible, or `git shortlog` counts to measure people.
- Do not use `git diff A B` to judge what a branch introduces, and do not use `A...B` to check whether two trees are equal.

Edge cases.

- **Everything in this chapter that mentions renames is a heuristic.** Two investigators with different thresholds, or different Git versions, can see different rename pairs for the same commits. Quote the options you used.
- `:/<text>` is unstable. It returns the youngest match across all refs, so a new commit or a fetched branch can change what a saved command resolves to. In scripts use `<rev>^{/}<text>`, or better, an ID.
- **Shallow and partial clones.** In a shallow clone, log, blame and bisect stop at the boundary. In a blobless partial clone, blame and `-S` fetch old blobs on demand and can be slow (Chapter 26).
- **Author and dates are claims.** Every name and date that these tools print was supplied by whoever created the commit (Chapter 6). For questions of accountability, look at signatures (Chapter 14B) and at server-side records.
- **Bisect checks out old code and you run it.** `git bisect run` executes whatever the old commits contain: build scripts, test harnesses, hooks of the build system. On an untrusted repository that is arbitrary code execution; do it in a sandbox.

14A.27 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git diff</code> , <code>git log</code> , <code>git show</code> , <code>git blame</code> , <code>git grep</code> , <code>git shortlog</code> , <code>git describe</code> , <code>git rev-parse</code> , <code>git rev-list</code> , <code>git last-modified</code>	● SAFE	Nothing	not needed	not needed
<code>git config set blame.ignoreRevsFile <path></code> , <code>diff.algorithm</code> , <code>blame.markIgnoredLines</code>	● CAUTION	One key in <code>.git/config</code> , a file that keeps no history; it changes what later commands print	<code>git config get <key></code>	<code>git config unset <key></code>
<code>git bisect start</code> , <code>good</code> , <code>bad</code> , <code>skip</code> , <code>run</code> , <code>replay</code>	● CAUTION	Detaches HEAD and checks out other commits; writes <code>refs/bisect/*</code> and <code>BISECT_*</code>	<code>git status</code> must be clean; <code>git rev-list --count <good>..<bad></code>	<code>git bisect reset</code>
<code>git bisect reset</code>	● CAUTION	Checks out the starting branch; deletes the bisect state	<code>git bisect log > file</code> to keep the session	<code>git bisect replay file</code>
<code>git bisect start --no-checkout</code>	● SAFE	Bisect state and <code>BISECT_HEAD</code> only	not needed	<code>git bisect reset</code>
<code>git worktree add --detach <dir> <commit></code>	● SAFE	A new directory and administrative files under <code>.git/worktrees</code>	not needed	<code>git worktree remove <dir></code>
<code>git restore --source=<commit> -- <path></code>	● DANGEROUS	Overwrites the working tree file	<code>git status --short -- <path></code> ; <code>git diff <commit> -- <path></code>	None for uncommitted edits to that file

For the ● row: it sets working tree files to their content in the named commit and destroys uncommitted edits in them; preview with the two commands in the table; Git cannot recover content that was never staged or committed; it is appropriate for bringing back a deleted file, as in section 14A.13, where no file of that name exists.

14A.28 Version notes

Version note. Older behavior: `git whatchanged` ran without complaint. Current behavior: on Git 2.55 it stops with "refusing to run without --i-still-use-this". Since: the deprecation warning has been in the manual since Git 2.51; removal is planned for Git 3.0 ([BreakingChanges](#)). Recommended: `git log --raw --no-merges`.

- The `--reset-when-found[=<where>]` option of `git bisect` and the improved `git log --follow` on non-linear history are Git 2.56 changes ([RelNotes 2.56.0](#)); not run here. The same release notes mention that blame output reserves a column for the `^`, `?` and `*` marks only when they are shown, so the blame transcripts of this chapter may be aligned differently on 2.56.
- `git blame --diff-algorithm` exists since Git 2.53 ([RelNotes 2.53.0](#)).
- `git last-modified` exists since Git 2.52 and is experimental ([RelNotes 2.52.0](#)). Apple's Git 2.50.1 at `/usr/bin/git` does not have it (Phase 0 report, section 1).
- `git config set`, `get` and `unset`, used in this chapter, need Git 2.46 or later.

Unverified. Three behaviors shown in this chapter rest on runs of Git 2.55.0, and the releases that introduced them were not looked up: `--since=<date>` taking the current time of day for a date without a time (not in the manual); `git bisect run` re-testing the good commit after an exit status of 126 or 127 (not in the manual); and the `:(optional)` prefix for path-valued configuration (in the 2.55 manual).

14A.29 Practice

- Lab 10.4 in `m10-range-notation.md`: predict two-dot and three-dot output for `git log` and for `git diff`.
- Labs 11.1 to 11.7 in the Module 11 lab manual: a line across a rename; a removed string with `-S` and `-G`; a deleted file; a manual bisect; `git bisect run`; blame through a formatter commit; line history with `-L`.
- Replay any transcript with `labs/run ch14a/<demo>`, for example `labs/run ch14a/bisect-manual`. The sandbox stays in place for your own queries.
- Three drills in the sandbox of `ch14a/history-tour`. Find the commit that introduced the word "experimental" into `docs/metrics.md`; every commit by Asha that touched `scorekit/text.py`; and the number of commits between `v0.1.0` and `v0.2.0` that are not on the first-parent line.

14A.30 Interview questions

1. `git diff main..feature` and `git log main..feature` use the same two dots. What does each do, and which diff form matches the log form?
2. A pull request shows a change that the base branch already contains. Explain how that can happen and how to make it disappear without closing the pull request.
3. Git stores no renames. Name three commands whose output depends on rename detection, and describe a commit that defeats it.
4. Explain `-S` and `-G` to a colleague, with one commit that the first misses and the second reports, and one case where `-G` is noise.
5. What exactly does a row of `git blame` assert? Give three situations in which the named author did not write the logic on that line.
6. Your team introduces a formatter. What do you commit, what does each developer configure, and what breaks if someone checks out a commit from before the formatter?
7. A regression appeared somewhere in 4,000 commits. How many tests does a bisection need, what does it assume about the history, and how do you check that assumption when the result looks wrong?
8. Design the test script for `git bisect run` for a nightly evaluation whose score has run-to-run noise and whose build is broken in some commits. Which exit codes do you use, and why?
9. Bisect reports a merge commit as the first bad commit, and both parents are good. What does that tell you, and what do you look at next?
10. `git log --since=2026-09-10` omitted commits from the morning of that day. Explain the cause and state how you would write the query in an incident report.
11. After a squash merge, how do you find out who wrote a given function and why? What would you change in the team's process to make that question cheaper?
12. You found the commit that introduced a bug. Which commands tell you which releases and which open branches contain it?

14A.31 Sources

Primary sources

- [git-diff](#), [gitrevisions](#), [git-log](#), [gitdiffcore](#), [git-blame](#), [git-bisect](#), [git-grep](#), [git-shortlog](#), [git-describe](#), [git-config](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that every option in this chapter was checked against.
- Christian Couder, [Fighting regressions with git bisect](#), distributed with Git: the bisection and skip algorithms. The step estimate is in [bisect.c](#) at v2.55.0.
- Release notes: [2.51.0](#), [2.52.0](#), [2.53.0](#), [2.56.0](#); [BreakingChanges](#); [git-whatchanged](#); [git-last-modified](#); [merge strategies](#).
- GitHub Docs, described and not run here: [three-dot and two-dot comparisons](#); [ignore commits in the blame view](#).

Secondary sources

- The Phase 0 report of this course, sections 1, 4 and 12: the deprecation table, and the two meanings of the dots among commonly misunderstood topics.
- Julia Evans, [Confusing git terminology](#), cited by the report for several of those misunderstandings.

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- Tekin Süleyman, "[RubyConf 2018 - Branch in Time](#)", 27 minutes, December 2018. Why history quality matters, told through `git blame`, `git log -S`, interactive rebase and `--force-with-lease`. Nothing in it depends on a Git version.
- ThePrimeagen for Boot.dev, "[Git and GitHub - Full Course](#)", November 2024; the bisect chapter starts at 3:54:53. Current; digressive commentary style.
- glich.stream, "[Chad level git: advanced concepts \(2025\)](#)", 1 hour 14 minutes, January 2025. Includes bisect, hands-on.

Further reading

- Pro Git, part "Git Tools": [Revision Selection](#), [Searching](#) and [Debugging with Git](#).
- Chapter 5 for the three everyday diffs; Chapter 7 for counting divergence; Chapter 8 for `--remerge-diff`; Chapter 9 for `git range-diff`; Chapter 10 for patch IDs; Chapter 13 for the reflog as a recovery tool.

Chapter 14B: Configuration, Aliases, Tags and Signing

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch14b/`. The signing demos generate throwaway SSH keys inside the sandbox; they are marked where they appear, and their keys, signatures and object IDs differ on every run.

14B.1 Why this matters

Four questions a CTO can ask in one release week.

- 1. "Two engineers ran `git pull` on the same diverged branch. One got a merge commit, one got a rebase, the third got an error. Same repository, same Git. Why?"
- 2. "Half of the commits a contractor pushed to the company repository carry her private email address. She says she set the work address months ago. Who is right?"
- 3. "The container labelled `v1.2.0` in production does not contain the fix that the tag `v1.2.0` contains on my laptop. Which one is `v1.2.0`?"
- 4. "A commit on `main` says it was written by our staff engineer. She was on a flight. Can Git tell us who made it?"

The first two are configuration: Git assembles its behavior from several files, the command line and the environment (sections 14B.2 to 14B.7). The third is a tag that was moved after it had been published (sections 14B.8 to 14B.14). The fourth is identity: author and committer are text that the committer supplies, and only a signature ties an object to a key (sections 14B.15 to 14B.18).

One idea runs through all three parts: Git records what it is told. Configuration decides what Git does on *your* machine and is never transferred. A tag is a name that every clone holds separately. A signature is the only statement in an object that somebody else can check.

14B.2 Scopes: where a setting comes from, and which value wins

In one sentence. A Git setting is a key with one or more values, read from several files plus the command line, and when the same key appears more than once the value read last wins.

Analogy. Company policy, team convention, and what your manager tells you this morning: the most specific instruction overrides the general one. The analogy breaks for multi-valued keys, where nothing is overridden: every value from every file counts.

Precisely. The manual names five scopes (`git-config`, "SCOPES"), read in this order:

Scope	Source	Written with	Typical content
system	<code>\$(prefix)/etc/gitconfig</code>	<code>--system</code>	defaults an administrator sets for every user of the machine
global	<code>\$XDG_CONFIG_HOME/git/config</code> , then <code>~/.gitconfig</code>	<code>--global</code>	your identity, your editor, your aliases
local	<code>.git/config</code> of the repository	<code>--local</code> (the default for writing)	remotes, upstreams, repository-specific overrides
worktree	<code>.git/config.worktree</code>	<code>--worktree</code>	settings for one working tree, only if <code>extensions.worktreeConfig</code> is set
command	<code>git -c <key>=<value></code> , and the <code>GIT_CONFIG_COUNT</code> family of environment variables	not stored	one invocation

"Global" means "for this user". `$(prefix)` is the directory Git was installed under, so the Homebrew Git and Apple's Git on your Mac have different system files. Outside the lab, `git var GIT_CONFIG_SYSTEM` prints the path.

Inside `.git`. Only `config` and, optionally, `config.worktree` live there. Configuration is not an object and not a ref: it has no history, and no transfer command carries it. A fresh clone has the local configuration that `git clone` wrote and nothing of the configuration of the repository it came from.

See it. In the lab the global file is the sandbox's `home/.gitconfig` and the system scope is switched off (`GIT_CONFIG_NOSYSTEM=1`), so your real configuration is never read. In this chapter's demos that directory is also `HOME`, so `~` means `$LAB/ch14b/<demo>/home`. A new repository, and everything Git knows:

```
$ git config list --show-scope --show-origin
global file:$LAB/ch14b/scopes/home/.gitconfig user.name=Lab User
global file:$LAB/ch14b/scopes/home/.gitconfig user.email=you@example.com
global file:$LAB/ch14b/scopes/home/.gitconfig init.defaultbranch=main
global file:$LAB/ch14b/scopes/home/.gitconfig gc.reflogexpire=never
global file:$LAB/ch14b/scopes/home/.gitconfig gc.reflogexpireunreachable=never
local file:.git/config core.repositoryformatversion=0
local file:.git/config core.filemode=true
```

local	file:./git/config	core.bare=false
local	file:./git/config	core.logallrefupdates=true
local	file:./git/config	core.ignorecase=true
local	file:./git/config	core.precomposeunicode=true

Three columns: scope, origin, key=value. Five lines come from the global file; the six `core.*` lines were written by `git init`. Keys print in lower case because section and variable names are case-insensitive. This command is the first thing to run whenever Git behaves differently on two machines.

Now set one key in two scopes (`git config set` 🟡 CAUTION: it rewrites a file that keeps no history):

```
$ git config set --global pull.rebase true
$ git config set pull.rebase false
$ git config get pull.rebase
false
$ git config get --all --show-scope --show-origin pull.rebase
global file:$LAB/ch14b/scopes/home/.gitconfig true
local file:./git/config false
```

`get` prints the winning value. `get --all` prints every value in reading order, and the last line is the winner. The command `scope` beats both files:

```
$ git -c pull.rebase=merges config get --show-scope --show-origin pull.rebase
command command line: merges
$ GIT_CONFIG_COUNT=1 GIT_CONFIG_KEY_0=pull.rebase GIT_CONFIG_VALUE_0=interactive git config get --show-scope --show-origin
pull.rebase
command command line: interactive
$ GIT_CONFIG_COUNT=1 GIT_CONFIG_KEY_0=pull.rebase GIT_CONFIG_VALUE_0=interactive git -c pull.rebase=merges config get --all --
show-scope pull.rebase
global true
local false
command interactive
command merges
```

The `GIT_CONFIG_COUNT` variables are the same scope for scripts that cannot pass `-c` everywhere; `-c` is read last and wins.

The system scope, shown with a stand-in file inside the sandbox (the machine's real system file is not touched):

```
# A stand-in for the system file, inside the sandbox. The real one is never read.
$ printf '[pull]\n\tff = only\n\trebase = interactive\n' > ~/etc-gitconfig
$ GIT_CONFIG_NOSYSTEM=0 GIT_CONFIG_SYSTEM=~/.etc-gitconfig git var GIT_CONFIG_SYSTEM
$LAB/ch14b/scopes/home/etc-gitconfig
$ GIT_CONFIG_NOSYSTEM=0 GIT_CONFIG_SYSTEM=~/.etc-gitconfig git config get --all --show-scope --show-origin --regexp "^pull\."
system file:$LAB/ch14b/scopes/home/etc-gitconfig only
system file:$LAB/ch14b/scopes/home/etc-gitconfig interactive
global file:$LAB/ch14b/scopes/home/.gitconfig true
local file:./git/config false
$ GIT_CONFIG_NOSYSTEM=0 GIT_CONFIG_SYSTEM=~/.etc-gitconfig git config get --show-scope pull.rebase
local false
$ git var GIT_CONFIG_SYSTEM
[exit status: 1]
$ git var GIT_CONFIG_GLOBAL
$LAB/ch14b/scopes/home/.gitconfig
```

System says `interactive`, global says `true`, local says `false`, and `false` wins. The last two commands show the lab's normal state: no system file, one global file.

Without `GIT_CONFIG_GLOBAL`, the global scope has two files, read in a fixed order:

```
# A second global file in the XDG location. Without GIT_CONFIG_GLOBAL, Git reads it and then ~/.gitconfig.
$ mkdir -p ~/.config/git
$ printf '[pull]\n\trebase = merges\n[core]\n\tabbrev = 12\n' > ~/.config/git/config
$ env -u GIT_CONFIG_GLOBAL git config get --all --show-scope --show-origin pull.rebase
global file:$LAB/ch14b/scopes/home/.config/git/config merges
global file:$LAB/ch14b/scopes/home/.gitconfig true
local file:./git/config false
$ env -u GIT_CONFIG_GLOBAL git config get --show-scope --show-origin core.abbrev
global file:$LAB/ch14b/scopes/home/.config/git/config 12
$ git config get --all --show-scope --show-origin pull.rebase
global file:$LAB/ch14b/scopes/home/.gitconfig true
local file:./git/config false
```

`~/.gitconfig` is read after the XDG file and wins where both set a key. `git config set --global` writes to `~/.gitconfig` unless only the XDG file exists. If you keep your configuration in the XDG location, a stray `~/.gitconfig` created by some tool silently overrides it.

The worktree scope has a trap:

```
$ git config set --worktree core.sparseCheckout true
$ git config list --local --show-scope --show-origin
local file:./git/config core.repositoryformatversion=0
local file:./git/config core.filemode=true
local file:./git/config core.bare=false
```

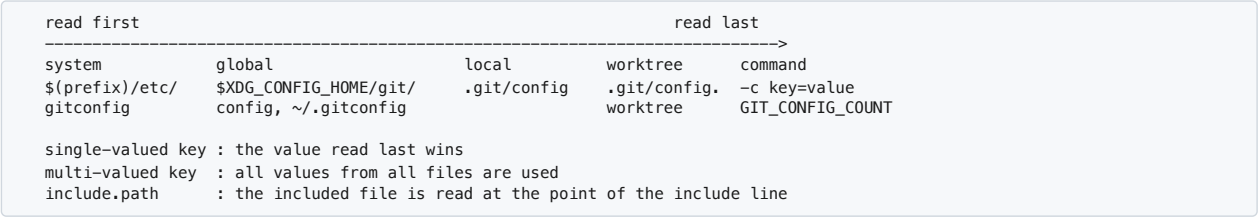
```

local file:.git/config core.logallrefupdates=true
local file:.git/config core.ignorecase=true
local file:.git/config core.precomposeunicode=true
local file:.git/config core.sparsecheckout=true
local file:.git/config pull.rebase=false
$ git config set extensions.worktreeConfig true
$ git config set --worktree core.sparseCheckout true
$ cat .git/config.worktree
[core]
    sparseCheckout = true
$ git config get --all --show-scope --show-origin core.sparseCheckout
local file:.git/config true
worktree file:.git/config.worktree true

```

Without `extensions.worktreeConfig`, `--worktree` silently writes to `.git/config`. With it, each working tree gets its own file (Chapter 25).

Picture.



The environment sits beside this stack, not inside it. Several things that look like configuration are decided by environment variables first:

```

$ cd ~/work/inference-gateway
$ git config set core.editor "code --wait"
$ git var GIT_EDITOR
true
$ (unset GIT_EDITOR; git var GIT_EDITOR)
code --wait
$ (unset GIT_EDITOR; EDITOR=nano git var GIT_EDITOR)
code --wait
$ (unset GIT_EDITOR; git config unset core.editor; EDITOR=nano git var GIT_EDITOR)
nano
$ git var GIT_AUTHOR_IDENT
Lab User <you@example.com> 1788757980 +0530
$ git -c user.name="Set With -c" var GIT_AUTHOR_IDENT
Lab User <you@example.com> 1788758040 +0530
$ (unset GIT_AUTHOR_NAME; git -c user.name="Set With -c" var GIT_AUTHOR_IDENT)
Set With -c <you@example.com> 1788758100 +0530

```

`GIT_EDITOR` (set by the lab so that no editor opens) beats `core.editor`, which beats `EDITOR`. `GIT_AUTHOR_NAME` beats `user.name` even when `user.name` is given with `-c`. The orders are documented per variable ([git-var](#), [git](#)): `GIT_EDITOR`, `core.editor`, `VISUAL`, `EDITOR`; `GIT_PAGER`, `core.pager`, `PAGER`; `GIT_SSH_COMMAND`, `core.sshCommand`. `git config list` cannot show these overrides. `git var` shows the result. (The demo uses `EDITOR` because Git skips `VISUAL` on a "dumb" terminal, which is what a script or a CI job has: [editor.c](#).)

Outside a repository there is no local scope:

```

$ cd ~
$ git config list --show-scope
global user.name=Lab User
global user.email=you@example.com
global init.defaultbranch=main
global gc.reflogexpire=never
global gc.reflogexpireunreachable=never
global pull.rebase=true
$ git config set pull.ff only
fatal: not in a git directory
[exit status: 128]
$ git config list --local
fatal: --local can only be used inside a git repository
[exit status: 128]

```

Protected configuration. The system, global and command scopes are "protected": a few security-sensitive keys, `safe.directory` among them, are honored only there. The local file belongs to whoever created the repository directory, which is not always you: a repository unpacked from an archive brings its `.git/config` along (Chapter 21B).

In production. The CTO's first question. Engineer A has `pull.rebase=true` in `~/gitconfig`. Engineer B has nothing, so a diverged `git pull` stops (Chapter 12, section 12.6). Engineer C's clone has `pull.rebase=false` in `.git/config`, left by an old setup script. One command per machine settles it: `git config get --all --show-scope --show-origin pull.rebase`.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
git config set <key> <value>	unchanged	unchanged	unchanged	unchanged	.git/config rewritten	unchanged	unchanged

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git config set --global ...</code>	unchanged	unchanged	unchanged	unchanged	unchanged; ~/.gitconfig (or the XDG file) rewritten	unchanged	unchanged

14B.3 Reading and writing configuration

The subcommands. Since Git 2.46 `git config` has subcommands, and the older option forms are documented as deprecated modes that still work (`git-config`, "DEPRECATED MODES"). Write the new form; read the old one, because scripts and most tutorials still use it.

Task	Current form (Git 2.46 or later)	Legacy form
Read one value	<code>git config get <key></code>	<code>git config <key></code> , <code>git config --get <key></code>
Read all values	<code>git config get --all <key></code>	<code>git config --get-all <key></code>
Search by pattern	<code>git config get --all --show-names --regexp <regex></code>	<code>git config --get-regexp <regex></code>
Write	<code>git config set <key> <value></code>	<code>git config <key> <value></code>
Add a value	<code>git config set --append <key> <value></code>	<code>git config --add <key> <value></code>
Remove	<code>git config unset <key></code>	<code>git config --unset <key></code>
List	<code>git config list</code>	<code>git config -l</code> , <code>--list</code>
Open in an editor	<code>git config edit</code>	<code>git config -e</code> , <code>--edit</code>

```
$ git config set merge.conflictStyle zdiff3
$ git config get merge.conflictStyle
zdiff3
$ git config get merge.conflictstyle
zdiff3
$ git config get merge.conflictStyl
[exit status: 1]
$ git config get --default merge merge.conflictStyl
merge
$ git config unset merge.conflictStyle
$ git config unset merge.conflictstyle
[exit status: 5]
```

The exit status is part of the interface: 1 from `get` for a key that is not set, 5 from `unset` for a key that does not exist. The subcommands do not exist before Git 2.46, so a script that must run on older installations (Ubuntu 24.04 ships 2.43.0, according to the Phase 0 report) either checks the version or uses the legacy spelling:

```
# The same four operations with the flags that older scripts and tutorials use.
$ git config merge.conflictStyle zdiff3
$ git config merge.conflictstyle
zdiff3
$ git config --get merge.conflictStyle
zdiff3
$ git config --list --local | tail -1
merge.conflictstyle=zdiff3
$ git config --unset merge.conflictStyle
```

The file format. Sections, optional subsections in double quotes, and `name = value` lines. On the command line a key is `section.name` or `section.subsection.name`: everything between the first and the last dot is the subsection, which is case-sensitive and may contain dots and slashes.

```
$ git config set --comment "decided 2026-09-07, see docs/git-setup.md" rebase.autoSquash true
$ git config set branch.release/1.2.description "Maintenance line for 1.2"
$ git config set credential.https://git.corp.example.username lab.user
$ tail -6 .git/config
[rebase]
  autoSquash = true # decided 2026-09-07, see docs/git-setup.md
[branch "release/1.2"]
  description = Maintenance line for 1.2
[credential "https://git.corp.example"]
  username = lab.user
$ git config list --local --name-only | tail -3
rebase.autosquash
branch.release/1.2.description
credential.https://git.corp.example.username
$ git config get --url=https://git.corp.example/platform/inference-gateway.git credential.username
lab.user
$ git config get --url=https://code.other.example/evalkit.git credential.username
[exit status: 1]
```

`--comment` (Git 2.45) writes the reason next to the value. The last two commands show URL matching: `get --url` returns the value from the subsection whose URL matches best.

Types. Git stores text. A type is applied when a value is read:

```
$ git config set core.bigFileThreshold 2g
$ git config get core.bigFileThreshold
2g
$ git config get --type=int core.bigFileThreshold
2147483648
$ git config set --global core.excludesFile "~/.config/git/ignore"
$ git config get core.excludesFile
~/.config/git/ignore
$ git config get --type=path core.excludesFile
$LAB/ch14b/config-commands/home/.config/git/ignore
$ printf '[rerere]\n\tenabled\n[fetch]\n\tprunes = yes\n' >> .git/config
$ git config get --type=bool rerere.enabled
true
$ git config get --type=bool fetch.prune
true
$ git config get fetch.prune
yes
$ git config set --type=bool fetch.prune sometimes
fatal: bad boolean config value 'sometimes' for 'fetch.prune'
[exit status: 128]
$ git config get --type=bool core.bigFileThreshold
fatal: bad boolean config value '2g' for 'core.bigfilethreshold'
[exit status: 128]
```

2g is an integer with a scale suffix. A path keeps its `~` in the file and is expanded on reading. A bare name with no `=` is boolean true, as are yes, on and 1. `--type on set` validates before writing; without it Git stores anything and complains only when a command needs the value.

Multi-valued keys. Some keys are lists: `remote.<name>.fetch`, `safe.directory`, `credential.helper`, `versionsort.suffix`, `include.path`.

```
$ git config set --append remote.origin.fetch "+refs/heads/*:refs/remotes/origin/*"
$ git config set --append remote.origin.fetch "+refs/pull/*:head:refs/remotes/origin/pr/*"
$ git config get remote.origin.fetch
+refs/pull/*:head:refs/remotes/origin/pr/*
$ git config get --all remote.origin.fetch
+refs/heads/*:refs/remotes/origin/*
+refs/pull/*:head:refs/remotes/origin/pr/*
$ git config set remote.origin.fetch "+refs/tags/*:refs/tags/*"
warning: remote.origin.fetch has multiple values
error: cannot overwrite multiple values with a single value
    Use --value=<pattern>, --append or --all to change remote.origin.fetch.
[exit status: 5]
$ git config unset remote.origin.fetch
warning: remote.origin.fetch has multiple values
[exit status: 5]
$ git config unset --value="refs/pull" remote.origin.fetch
$ git config get --all remote.origin.fetch
+refs/heads/*:refs/remotes/origin/*
```

Plain `get` prints only the last value of a list. `set` and `unset` refuse a key with several values (status 5) until you say which: `--value=<regex>` selects, `--all` takes everything, `--append` adds.

Finding keys.

```
$ git config get --all --show-names --regexp "^rebase\."
rebase.autosquash true
$ git help --config | grep "^rebase\."
rebase.abbreviateCommands
rebase.autoSquash
rebase.autoStash
rebase.backend
rebase.forkPoint
rebase.instructionFormat
rebase.maxLabelLength
rebase.missingCommitsCheck
rebase.rebaseMerges
rebase.rescheduleFailedExec
rebase.stat
rebase.updateRefs
```

`git help --config` lists every variable this Git knows. Git does not check a key against that list:

```
$ git config set --global pull.rebsae true
$ git config get pull.rebsae
true
$ git config get pull.rebase
[exit status: 1]
$ git config set --global pull_rebase true
error: key does not contain a section: pull_rebase
[exit status: 2]
$ git config unset --global pull.rebsae
```

`pull.rebsae` is stored, reported, and ignored forever; only a key without a section is rejected. Other tools rely on this freedom to keep their settings in Git's files. The cost is that a typing mistake is silent. Lab 5.4 builds a check from `git help --config`.

Which file does `edit` open?

```
# Which file would each scope open? An "editor" that only prints its argument tells you.
$ GIT_EDITOR="echo would edit:" git config edit
would edit: $LAB/ch14b/config-commands/home/work/inference-gateway/.git/config
$ GIT_EDITOR="echo would edit:" git config edit --global
would edit: $LAB/ch14b/config-commands/home/.gitconfig
$ GIT_EDITOR="echo would edit:" git config edit --worktree
would edit: $LAB/ch14b/config-commands/home/work/inference-gateway/.git/config
```

A broken file stops everything.

```
$ printf '[alias\n\tst = status -sb\n' >> .git/config
$ git status
fatal: bad config line 19 in file .git/config
[exit status: 128]
$ git config list --local
fatal: bad config line 19 in file .git/config
[exit status: 128]
$ git config list --global
fatal: bad config line 19 in file .git/config
[exit status: 128]
$ git config edit
fatal: bad config line 19 in file .git/config
[exit status: 128]
# Git cannot repair it. A text tool on the named line can. Here: delete the two appended lines.
$ sed -i.bak -e '/^\[alias$/, $d' .git/config && rm .git/config.bak
$ git status -sb
## main
```

Observed behavior	: every Git command in one repository fails with "fatal: bad config line 19 in file .git/config", including git config edit and git config list --global.
Git state	: objects, refs and index are intact. One line of .git/config is not valid syntax.
Mechanism	: every command reads the whole configuration before it does anything, and a parse error in a file it must read is fatal.
Root cause	: a hand edit (here a section header without its closing bracket).
Why Git does this	: running with half a configuration could mean the wrong identity, the wrong remote or a skipped safety setting; refusing is the conservative choice.
Correct fix	: open the named file at the named line with a text editor, not with Git.
Prevention	: change settings with git config set; after a hand edit run git config list.

14B.4 Include files and conditional includes: two identities

In one sentence. `include.path` reads another file at the point where the line stands, and `includeIf.<condition>.path` does so only when a condition about the current repository is true.

Analogy. A footnote that says "insert appendix D here, but only in the edition for the Bengaluru office". The analogy breaks because order matters: text inserted early is overridden by a later line of the including file.

Precisely. The conditions in Git 2.55 are `gitdir:<pattern>` and `gitdir/i:<pattern>` (the location of the repository's `.git` directory matches a glob), `onbranch:<pattern>` (the branch that is checked out), and `hasconfig:remote.*.url:<pattern>` (some remote of the repository has a matching URL). In a `gitdir` pattern a leading `~/` is your home directory, a trailing `/` means "everything below", and a pattern that starts with neither `~/`, `./` nor `/` gets `**/` put in front of it (git-config, "Conditional includes").

See it. You work for a company and maintain an open-source project. Work repositories live under `~/work`. The global file holds the personal address, and a second file holds the work address:

```
$ printf '[user]\n\temail = lab.user@corp.example\n' > ~/.gitconfig-work
$ git config set --global 'includeIf.gitdir:~/work/.path' '~/.gitconfig-work'
$ cat ~/.gitconfig
[user]
    name = Lab User
    email = you@example.com

[init]
    defaultBranch = main

[gc]
    reflogExpire = never
    reflogExpireUnreachable = never

[includeIf "gitdir:~/work/"]
    path = ~/.gitconfig-work
```

```
$ cd ~/work/inference-gateway
$ git config get --all --show-scope --show-origin user.email
global   file:$LAB/ch14b/includes/home/.gitconfig          you@example.com
global   file:$LAB/ch14b/includes/home/.gitconfig-work     lab.user@corp.example
$ git var GIT_AUTHOR_IDENT
Lab User <lab.user@corp.example> 1788756300 +0530
$ cd ~/oss/evalkit
$ git config get --all --show-scope --show-origin user.email
global   file:$LAB/ch14b/includes/home/.gitconfig          you@example.com
```



```
$ git var GIT_AUTHOR_IDENT
Lab User <you@example.com> 1788756480 +0530
```

Inside `~/work/inference-gateway` there are two global values, and the second, from the included file, wins. In `~/oss/evalkit` the file is not read. The commits prove it (in this section the identity comes from configuration; the lab library normally pins it through environment variables, which would hide the effect):

```
$ cd ~/work/inference-gateway
$ printf 'timeout_seconds: 30\n' >> config/limits.yaml
$ git commit -q -am "Add request timeout"
$ git log -2 --format='%h %an <%ae> %s'
ad95595 Lab User <lab.user@corp.example> Add request timeout
eb112a5 Lab User <you@example.com> Add rate limits
```

Picture.

<pre>~/gitconfig +-----+ [user] name = Lab User email = you@example.com [includeIf "gitdir:~/work/"] path = ~/gitconfig-work +-----+</pre>	<pre>~/gitconfig-work +-----+ [user] email = lab.user@corp.example +-----+</pre>
read here, and only if the repository's <code>.git</code> directory is somewhere under <code>~/work/</code>	

reading order in `~/work/inference-gateway`: you@example.com, then lab.user@corp.example (wins)
reading order in `~/oss/evalkit`: you@example.com (wins)

Five ways the rule silently does not apply. Each one produces commits with the wrong address and no message.

The trailing slash. `gitdir:~/work` matches only a repository whose `.git` directory is `~/work`:

```
$ git config unset --global 'includeIf.gitdir:~/work/.path'
$ git config set --global 'includeIf.gitdir:~/work.path' '~/gitconfig-work'
$ git config get user.email
you@example.com
$ git config unset --global 'includeIf.gitdir:~/work.path'
$ git config set --global 'includeIf.gitdir:~/work/.path' '~/gitconfig-work'
$ git config get user.email
lab.user@corp.example
```

Order. The include is read where it stands. If `[user]` comes after it in the including file, the general value is read last and wins:

```
# The same two sections in the other order: the conditional include first, [user] after it.
$ cp ~/.gitconfig ~/.gitconfig.good
$ printf '[includeIf "gitdir:~/work/"]\n\tpath = ~/.gitconfig-work\n[user]\n\tname = Lab User\n\temail = you@example.com\n[init]\n\tdefaultBranch = main\n' > ~/.gitconfig
$ git config get --all --show-origin user.email
file:$LAB/ch14b/includes/home/.gitconfig-work lab.user@corp.example
file:$LAB/ch14b/includes/home/.gitconfig you@example.com
$ git config get user.email
you@example.com
$ mv ~/.gitconfig.good ~/.gitconfig
$ git config get user.email
lab.user@corp.example
```

Keep conditional includes at the end of the file.

Not in a repository. `gitdir` needs a `.git` directory. In `~/work` itself there is none:

```
$ cd ~/work
$ git config get user.email
you@example.com
$ git init -q rag-indexer
$ git -C rag-indexer config get user.email
lab.user@corp.example
```

A missing file. An include that names a file that does not exist is skipped without an error:

```
$ git config set --global 'includeIf.gitdir:~/clients/.path' '~/gitconfig-clients'
$ git init -q ~/clients/acme-chatbot
$ ls ~/.gitconfig-clients
ls: $LAB/ch14b/includes/home/.gitconfig-clients: No such file or directory
[exit status: 1]
$ git -C ~/clients/acme-chatbot config get --show-origin user.email
file:$LAB/ch14b/includes/home/.gitconfig you@example.com
```

Asking the wrong question. A scope option switches include processing off, so the usual diagnostic lies unless you add `--includes`:

```
# Asking one scope by name switches include processing off unless you ask for it.
$ git config get --global user.email
```

```

you@example.com
$ git config get --global --includes user.email
lab.user@corp.example
$ git config list --global --show-origin | grep user
file:$LAB/ch14b/includes/home/.gitconfig      user.name=Lab User
file:$LAB/ch14b/includes/home/.gitconfig      user.email=you@example.com
$ git config list --global --includes --show-origin | grep user
file:$LAB/ch14b/includes/home/.gitconfig      user.name=Lab User
file:$LAB/ch14b/includes/home/.gitconfig      user.email=you@example.com
file:$LAB/ch14b/includes/home/.gitconfig-work user.email=lab.user@corp.example

```

By remote, not by directory. `hasconfig:remote.*.url:` chooses by where the repository is pushed, wherever it lies on disk:

```

$ printf '[user]\n\temail = lab.user@corp.example\n' > ~/.gitconfig-corp-remote
$ git config set --global 'includeIf.hasconfig:remote.*.url:git@git.corp.example:platform/**.path' '~/.gitconfig-corp-remote'
$ git init -q ~/tmp/prompt-library
$ cd ~/tmp/prompt-library
$ git config get --show-origin user.email
file:$LAB/ch14b/includes/home/.gitconfig      you@example.com
$ git remote add origin git@git.corp.example:platform/prompt-library.git
$ git config get --show-origin user.email
file:$LAB/ch14b/includes/home/.gitconfig-corp-remote      lab.user@corp.example

```

The identity changed the moment the remote was added. Files included this way may not themselves define remote URLs.

A team file, opted into. Git never reads configuration from the files it tracks; if it did, cloning a repository would let its author set your pager, your SSH command and your hooks. A team that wants shared settings commits a file and asks each member to include it once:

```

$ cd ~/work/inference-gateway
$ printf '[merge]\n\tconflictStyle = zdiff3\n[rebase]\n\tupdateRefs = true\n' > .gitconfig-team
$ git config get merge.conflictStyle
[exit status: 1]
$ git config set include.path ../.gitconfig-team
$ git config get --show-scope --show-origin merge.conflictStyle
local   file:./.gitconfig-team      zdiff3
$ git config list --local --includes --show-origin | tail -3
file:./.git/config      include.path=../.gitconfig-team
file:./.gitconfig-team  merge.conflictstyle=zdiff3
file:./.gitconfig-team  rebase.updateRefs=true

```

A relative `include.path` is resolved against the file that contains it, here `./.git/config`. Review changes to such a file like code: it can define an alias that runs a shell command.

In production. The CTO's second question. The contractor did set her work address, in `~/.gitconfig-work`, with `[includeIf "gitdir:~/work"]`. The pattern has no trailing slash, so it never matched. Diagnose with `git config get --all --show-origin user.email` inside the repository; fix the pattern; set `user.useConfigOnly=true` and remove the global default address, so that a repository outside both directories refuses to commit (Chapter 6, section 6.13). Published commits keep the address they have.

14B.5 The settings to decide deliberately

Most settings never need your attention. The twenty below, in nineteen rows because `user.name` and `user.email` share one, change what everyday commands do, so "I never set it" is also a decision, made by the default. The mechanics are in the chapter named; the facts are from [git-config](#) unless a source is given.

Setting	What it does	Why it matters	Downside	Optional?	Personal or team
<code>user.name</code> , <code>user.email</code>	Identity copied into every commit and annotated tag	Attribution; GitHub links commits to accounts by this email (Chapter 6)	Unchecked text; the wrong scope gives the wrong identity for years	Required	Personal, one per context (section 14B.4)
<code>init.defaultBranch</code>	Name of the first branch in <code>git init</code>	Unconfigured Git 2.55 creates <code>master</code> ; Git 3.0 will create <code>main</code> (Phase 0 report)	None found; new repositories only	Recommended	Personal, following the team's name
<code>pull.rebase</code>	How <code>git pull</code> integrates: <code>false</code> merge, <code>true</code> rebase, merges, <code>interactive</code>	With nothing set, a diverged pull is fatal (Chapter 12, section 12.6)	<code>true</code> gives unpublished commits new IDs and flattens local merges	Decide one way	Personal; a team convention keeps history uniform
<code>pull.ff</code>	<code>only</code> : pull refuses anything but a fast-forward	You integrate deliberately	In configuration <code>only</code> wins over <code>pull.rebase</code> (Chapter 12), so a diverged pull needs a flag	Optional	Personal
<code>push.default</code>	What a bare <code>git push</code> updates; <code>simple</code> since Git 2.0	<code>matching</code> and <code>upstream</code> can	<code>simple</code> refuses when the upstream has another	Leave the default	Personal

Setting	What it does	Why it matters	Downside	Optional?	Personal or team
		update a branch you did not mean (Chapter 12, section 12.5)	name		
<code>push.autoSetupRemote</code>	The first push of a branch sets its upstream (Git 2.37)	No "has no upstream branch" stop	A mistyped branch name is published at once	Optional	Personal
<code>fetch.prune</code>	Fetch deletes remote-tracking branches whose branch is gone	<code>git branch -r</code> stays true (Chapter 12, section 12.11)	That ref may have been your last name for those commits	Optional	Personal
<code>merge.conflictStyle</code>	<code>merge</code> (default), <code>diff3</code> , <code>zdiff3</code> (Git 2.35): adds the common ancestor to a conflict	You see what both sides changed <i>from</i> (Chapter 8)	Longer conflict regions; a third marker	Recommended	Personal
<code>rebase.autoSquash</code>	Places <code>fixup!</code> , <code>squash!</code> and <code>amend!</code> commits in an interactive rebase	Makes fixup commits routine (Chapter 9)	A title that starts with those words by accident is moved	Optional	Personal
<code>rebase.autoStash</code>	Stashes before a rebase, applies after	Rebase with a dirty working tree	The final apply can conflict (manual)	Optional	Personal
<code>rebase.updateRefs</code>	Branches pointing into the rebased range move with it (Git 2.38)	Stacked branches stay stacked (Chapter 9)	Moves branches you did not name	Optional	Personal
<code>rerere.enabled</code>	Records conflict resolutions and replays them	Repeated rebases of a long branch (Chapter 14C)	A wrong resolution is replayed too	Optional	Personal
<code>diff.algorithm</code>	<code>myers</code> (default), <code>minimal</code> , <code>patience</code> , <code>histogram</code>	Readability when code moves (below)	Your diff differs from a colleague's	Optional	Personal
<code>diff.colorMoved</code>	Colors moved lines differently	Review of refactorings	Terminal only	Optional	Personal
<code>core.editor</code>	Editor for messages and todo lists, unless <code>GIT_EDITOR</code> is set	A graphical editor needs its <code>wait</code> option, such as <code>code --wait</code> (GitHub Docs)	None, once tested	Recommended	Personal
<code>core.autocrlf</code>	Per-user line-ending conversion	A convenience on Windows; not deprecated (Phase 0 report)	Normalizes for you only; the repository-wide tool is <code>.gitattributes</code> (Chapter 14C)	Optional; unset on macOS	Team decides through attributes
<code>core.excludesFile</code>	Personal ignore file; default <code>\$XDG_CONFIG_HOME/git/ignore</code>	Editor and OS files stay out of project <code>.gitignore</code> files (Chapter 4)	A colleague without it commits what you ignore	Optional	Personal
<code>commit.gpgSign</code>	Signs every commit	Evidence that a key holder made the commit (section 14B.16)	Every commit fails when the key is unavailable	Optional, unless a rule requires signatures	Personal setting, team policy
<code>help.autocorrect</code>	A mistyped command: show the guess (default), <code>prompt</code> , <code>never</code> , <code>immediate</code> , or a delay	Saves retyping	<code>immediate</code> runs a guess; since Git 2.49 the value <code>1</code> means <code>immediate</code> (release notes)	Optional	Personal

Three of these are shown here because no other chapter shows them.

`init.defaultBranch`. `git var GIT_DEFAULT_BRANCH` answers "what would `git init` call the first branch", with and without the lab's global file:

```
$ git var GIT_DEFAULT_BRANCH
main
$ git config get --show-origin init.defaultBranch
file:$LAB/ch14b/settings/home/.gitconfig      main
# The same question with no global file at all, which is how unconfigured Git 2.55 answers:
$ GIT_CONFIG_GLOBAL=/dev/null git var GIT_DEFAULT_BRANCH
master
```

`help.autocorrect`.

```
$ git stauts -sb
git: 'stauts' is not a git command. See 'git --help'.

The most similar command is
    status
[exit status: 1]
```

```
$ git -c help.autocorrect=never stauts -sb
git: 'stauts' is not a git command. See 'git --help'.
[exit status: 1]
$ git -c help.autocorrect=immediate stauts -sb
WARNING: You called a Git command named 'stauts', which does not exist.
Continuing under the assumption that you meant 'status'.
## main
M gateway/retry.py
[exit status: 0]
```

`immediate` ran the guess. With a typo that resolves to a destructive command, it would have run that.

`diff.algorithm`. A function was moved to the top of a file and edited. The default algorithm pairs lines of the two functions with each other:

```
$ git diff --stat
gateway/retry.py | 16 ++++++-----
1 file changed, 8 insertions(+), 8 deletions(-)
$ git diff | tail -n +5
@@ -1,16 +1,16 @@
-def call_small(prompt):
-    for attempt in range(3):
+def call_large(prompt):
+    for attempt in range(5):
+        try:
-            return small.complete(prompt)
+            return large.complete(prompt)
-        except Timeout:
+        except Timeout:
-            continue
-        raise Unavailable("small")
+        backoff(attempt)
+        raise Unavailable("large")

-def call_large(prompt):
+def call_small(prompt):
+    for attempt in range(3):
+        try:
-            return large.complete(prompt)
+            return small.complete(prompt)
-        except Timeout:
+        except Timeout:
-            continue
-        raise Unavailable("large")
+        raise Unavailable("small")
```

```
$ git config set diff.algorithm histogram
$ git diff --stat
gateway/retry.py | 18 ++++++-----
1 file changed, 9 insertions(+), 9 deletions(-)
$ git diff | tail -n +5
@@ -1,3 +1,12 @@
+def call_large(prompt):
+    for attempt in range(5):
+        try:
+            return large.complete(prompt)
+        except Timeout:
+            backoff(attempt)
+        raise Unavailable("large")
+
+def call_small(prompt):
+    for attempt in range(3):
+        try:
@@ -5,12 +14,3 @@ def call_small(prompt):
+        except Timeout:
+            continue
+        raise Unavailable("small")
-
-
-def call_large(prompt):
-    for attempt in range(3):
-        try:
-            return large.complete(prompt)
-        except Timeout:
-            continue
-        raise Unavailable("large")
```

Both diffs are correct. `histogram` reports one block added and one removed, which is what happened. The setting changes what you see, never what is stored (Chapter 2).

A deliberate file. Lab 5.4 builds a global file one decision at a time. One defensible result, with the reason beside each line:

```
$ cat ~/.gitconfig
[user]
    name = Lab User
    email = you@example.com
    useConfigOnly = true # refuse to guess an identity

[init]
```

```

    defaultBranch = main
[gc]
    reblogExpire = never
    reblogExpireUnreachable = never
[pull]
    ff = only # never integrate by accident
[push]
    autoSetupRemote = true # first push sets the upstream
[fetch]
    prune = true # forget branches deleted on the server
[merge]
    conflictStyle = zdiff3 # show the common ancestor in conflicts
[rebase]
    autoSquash = true # fixup! commits find their place
    updateRefs = true # stacked branches move together
[rerere]
    enabled = true # remember conflict resolutions
[diff]
    algorithm = histogram # moved code reads as a move
    colorMoved = default
[help]
    autocorrect = prompt # show the suggestion, run nothing
[core]
    excludesFile = ~/.config/git/ignore

```

The `[gc]` section belongs to the lab (Chapter 1, section 1.7); do not copy it.

Credential helpers, in one paragraph. `credential.helper` names a program that stores and returns the username and token for HTTPS remotes; `credential.<url>.username` fixes a username per host ([gitcredentials](#)). `git help -a | grep credential-` lists the installed helpers. Which one to use, and why a token never belongs in a remote URL, is Chapter 16.

In production. A platform team runs a script of `git config set --global` lines on every laptop. Six months later nobody remembers why `pull.rebase` is `true`. Two things prevent that: a comment on each line, and a split between the few settings a team must agree on (the default branch name, line-ending attributes, whether signatures are required) and everything else, which stays personal.

14B.6 Aliases

In one sentence. An alias is a configuration value under `alias.<name>` that Git substitutes for `<name>` when it is the first word of a command line, appending any further arguments.

Analogy. A speed-dial key. The analogy breaks at the arguments: whatever you type after the alias is added to the end of the expansion.

Precisely. If the value starts with `!`, it is handed to the shell and run from the top-level directory of the repository. Otherwise it is a Git command line, split on spaces with shell-like quoting. An alias cannot replace an existing command. The first word of the expansion may be an option to `git` itself, such as `-c <key>=<value>` ([git-config](#), `alias.*`).

See it. Five aliases that earn their place:

```

$ git config set --global alias.st 'status -sb'
$ git config set --global alias.lg 'log --graph --decorate --oneline --all'
$ git config set --global alias.last 'log -1 --stat HEAD'
$ git config set --global alias.unstage 'restore --staged --'
$ git config set --global alias.amend 'commit --amend --no-edit'
$ git config get --all --show-names --regex '^alias\.'
alias.st status -sb
alias.lg log --graph --decorate --oneline --all
alias.last log -1 --stat HEAD
alias.unstage restore --staged --
alias.amend commit --amend --no-edit

```

```

$ git st
## main
$ git lg
* 1adfaa7 (HEAD -> main) Raise the rate limit
| * f29324a (feature/fallback-route) Add fallback route
|/
* eb112a5 Add rate limits
* 516c4c3 Add model router
* 37da450 Add README
$ git last
commit 1adfaa7ec5b01b116b6d926ce544561f6b29583d
Author: Lab User <you@example.com>
Date: Mon Sep 7 10:09:00 2026 +0530

    Raise the rate limit

config/limits.yaml | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)

```

- `st` is `status -sb`: the short format plus the branch line (Chapter 4).
- `lg` draws every ref as a graph, one line per commit: the fourth command of Chapter 1's diagnosis ritual, in two letters.

- `last` shows the newest commit with the files it touched.

```
$ printf 'burst: 20\n' >> config/limits.yaml
$ printf 'scratch\n' > notes.txt
$ git add .
$ git st
## main
M config/limits.yaml
A notes.txt
$ git unstage notes.txt
$ git st
## main
M config/limits.yaml
?? notes.txt
```

`unstage` is `restore --staged --`. The trailing `--` makes everything after it a path. Older aliases with this name say `reset HEAD --`; the effect on the index is the same (Chapter 5).

```
$ git log -1 --format="%h %s"
1adfaa7 Raise the rate limit
$ git amend
[main e1bef49] Raise the rate limit
Date: Mon Sep 7 10:09:00 2026 +0530
1 file changed, 2 insertions(+), 1 deletion(-)
$ git log -1 --format="%h %s"
e1bef49 Raise the rate limit
$ git show --stat --format="%h %s" HEAD
e1bef49 Raise the rate limit

config/limits.yaml | 3 ++
1 file changed, 2 insertions(+), 1 deletion(-)
$ git reflog -2
e1bef49 HEAD@{0}: commit (amend): Raise the rate limit
1adfaa7 HEAD@{1}: commit: Raise the rate limit
```

`amend` 🟡 CAUTION is `commit --amend --no-edit`: it replaces the tip commit with one that includes whatever is staged. The old commit is one `reflog` entry away, and everything Chapter 6 says about amending published commits applies.

What Git runs. `GIT_TRACE=1` prints the expansion (the commands here cut the time stamp off each trace line):

```
$ GIT_TRACE=1 git st 2>&1 | grep -E -o '(alias expansion|built-in): .*'
alias expansion: st => status -sb
built-in: git status -sb
$ GIT_TRACE=1 git lg -1 2>&1 | grep -E -o '(alias expansion|built-in): .*'
alias expansion: lg => log --graph --decorate --oneline --all
built-in: git log --graph --decorate --oneline --all -1
```

Arguments are appended. That is convenient for `lg -2`. It is a trap for `last`:

```
# Arguments after the alias name are appended to the expansion.
$ git lg -2
* e1bef49 (HEAD -> main) Raise the rate limit
| * f29324a (feature/fallback-route) Add fallback route
|/
$ git last --format="%h %s" feature/fallback-route
e1bef49 Raise the rate limit

config/limits.yaml | 3 ++
1 file changed, 2 insertions(+), 1 deletion(-)
$ GIT_TRACE=1 git last --format="%h %s" feature/fallback-route 2>&1 | grep -E -o 'built-in: .*'
built-in: git log -1 --stat HEAD '--format=%h %s' feature/fallback-route
$ git config set --global alias.last 'log -1 --stat'
$ git last --format="%h %s" feature/fallback-route
f29324a Add fallback route

gateway/router.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```

The alias contained `HEAD`, so the command became `git log -1 --stat HEAD ... feature/fallback-route`: the newest commit reachable from either, which is the tip of `main`. Without the hard-coded `HEAD` the alias shows the last commit of whatever you name.

Shell aliases. A value that starts with `!` is a shell command:

```
$ git config set --global alias.root '!pwd'
$ cd gateway
$ git root
$LAB/ch14b/aliases/home/work/inference-gateway
$ git config set --global alias.where '!echo "prefix=$GIT_PREFIX top=$(pwd)"'
$ git where
prefix=gateway/ top=$LAB/ch14b/aliases/home/work/inference-gateway
$ cd ..
```

It ran in the top-level directory although you were in `gateway/`; `GIT_PREFIX` holds the directory you came from. The second trap is again the arguments:

```
$ git config set --global alias.tracked '!git ls-tree -r --name-only $1 | head -2'
$ git tracked HEAD
head: HEAD: No such file or directory
[exit status: 1]
$ GIT_TRACE=1 git tracked HEAD 2>&1 | grep -o 'start_command: .*'
start_command: /bin/sh -c 'git ls-tree -r --name-only $1 | head -2 "$@"' 'git ls-tree -r --name-only $1 | head -2' HEAD
$ git config set --global alias.tracked '!f() { git ls-tree -r --name-only "${1:-HEAD}" | head -2; }; f'
$ git tracked HEAD
README.md
config/limits.yaml
$ git tracked feature/fallback-route
README.md
config/limits.yaml
```

Git runs `sh -c '<alias> "$@"'`: the arguments are available as `$1` and are *also* appended, here to `head`. The manual's remedy is a function that consumes them: `!f() { ...; }; f`.

What an alias cannot do.

```
$ git config set --global alias.status 'status -sb'
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    notes.txt

nothing added to commit but untracked files present (use "git add" to track)
$ GIT_TRACE=1 git status 2>&1 | grep -E -o '(alias expansion|built-in): .*'
built-in: git status
$ git config unset --global alias.status
```

`alias.status` is ignored: built-in commands win. Never use aliases in scripts, hooks or CI: they exist in your configuration only.

```
$ sed -n "/^\[alias\]/,/$p" ~/.gitconfig
[alias]
    st = status -sb
    lg = log --graph --decorate --oneline --all
    last = log -1 --stat
    unstage = restore --staged --
    amend = commit --amend --no-edit
    root = !pwd
    where = !echo \ "prefix=$GIT_PREFIX top=$(pwd)\ "
    tracked = "!f() { git ls-tree -r --name-only \"${1:-HEAD}\" | head -2; }; f"
```

The Git 2.55 manual also documents a subsection form, `[alias "<name>"]` with a `command` key, for names that contain characters outside ASCII letters, digits and dashes; it works on 2.55.0.

Unverified. Which release introduced the `[alias "<name>"] command = ...` form was not checked; older installations may not understand it.

In production. `pushf = push --force-with-lease --force-if-includes` is the alias most worth having: the safe form is long enough that people type `--force` instead (Chapter 12, section 12.8). The alias most worth avoiding is a `! alias` pasted from a web page: a shell command that runs with your permissions.

14B.7 Environment variables for diagnosis

Configuration says what Git should do. These variables make Git say what it is doing.

Variable	Shows	Use it when
<code>GIT_TRACE=1</code>	alias expansion, built-in and external commands that are started	"what did that command actually run?"
<code>GIT_TRACE_SETUP=1</code>	the <code>.git</code> directory, the working tree, the current directory and prefix Git settled on	Git picks up the wrong repository, or none
<code>GIT_TRACE_PACKET=1</code>	the protocol conversation with a remote	fetch and push puzzles (Chapter 12, section 12.13)
<code>GIT_TRACE_CURL=1</code> , <code>GIT_CURL_VERBOSE=1</code>	the HTTP exchange: requests, response codes, headers	HTTPS authentication and proxy failures (Chapter 16)
<code>GIT_SSH_COMMAND='ssh -v ...'</code>	replaces the <code>ssh</code> program and its options for this command	SSH picks the wrong key, or you need <code>ssh -v</code> output
<code>GIT_TRACE2_PERF=1</code>	timings of regions inside a command	"why is <code>git status</code> slow?" (Chapter 26)

A value of 1, 2 or true writes to standard error; an absolute path appends to that file (`git`, "Environment Variables").

```
$ GIT_TRACE=1 git st 2>&1 | grep -E -o '(trace: (exec|alias expansion|built-in)|##).*'
trace: exec: git-st
trace: alias expansion: st => status -sb
trace: exec: git status -sb
trace: built-in: git status -sb
## main
```

```
$ cd gateway
$ GIT_TRACE_SETUP=1 git st 2>&1 | grep -o 'setup: .*' | grep -v chdir
setup: git_dir: .git
setup: git_common_dir: .git
setup: worktree: $LAB/ch14b/diagnose-env/home/work/inference-gateway
setup: cwd: $LAB/ch14b/diagnose-env/home/work/inference-gateway
setup: prefix: gateway/
$ cd ..
```

When Git "does not see" a repository, or sees a parent directory's repository instead of yours, these lines say which.

`GIT_SSH_COMMAND`, shown without a network: a stand-in script named `ssh` prints its arguments and exits.

```
# ~/lab-bin/ssh is a stand-in that prints its arguments and exits. No connection is made.
$ git remote add origin git@git.corp.example:platform/inference-gateway.git
$ GIT_SSH_COMMAND=~/.lab-bin/ssh -i ~/.ssh/id_ed25519_work -o IdentitiesOnly=yes' git fetch origin
ssh was asked to run: -i $LAB/ch14b/diagnose-env/home/.ssh/id_ed25519_work -o IdentitiesOnly=yes -o SendEnv=GIT_PROTOCOL
git@git.corp.example git-upload-pack 'platform/inference-gateway.git'
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
[exit status: 128]
```

Git appended its own arguments (the host and the remote command `git-upload-pack '<path>'`) to yours. With the real `ssh`, `GIT_SSH_COMMAND='ssh -v' git fetch` shows which keys are offered. `core.sshCommand` does the same permanently, and the variable overrides it:

```
$ git config set core.sshCommand '~/lab-bin/ssh -i ~/.ssh/id_ed25519_work'
$ git ls-remote origin
ssh was asked to run: -i $LAB/ch14b/diagnose-env/home/.ssh/id_ed25519_work -o SendEnv=GIT_PROTOCOL git@git.corp.example git-
upload-pack 'platform/inference-gateway.git'
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
[exit status: 128]
$ GIT_SSH_COMMAND=~/.lab-bin/ssh -v' git ls-remote origin
ssh was asked to run: -v -o SendEnv=GIT_PROTOCOL git@git.corp.example git-upload-pack 'platform/inference-gateway.git'
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
[exit status: 128]
```

The HTTP variables need a server, so they are not run here:

```
GIT_TRACE_CURL=1 git ls-remote https://github.com/<owner>/<repo>.git
GIT_CURL_VERBOSE=1 git ls-remote https://github.com/<owner>/<repo>.git
```

Version note. Older behavior: tutorials name `GIT_CURL_VERBOSE` as the HTTP debugging switch. Current behavior: the Git 2.55 manual documents `GIT_TRACE_CURL` and `GIT_TRACE_CURL_NO_DATA`; `GIT_CURL_VERBOSE` is still read by the HTTP code, which treats it as a curl trace without payload data ([http.c at 2.55.0](http://c.2.55.0)). Since: not established. Recommended: `GIT_TRACE_CURL=1`, and keep `GIT_TRACE_REDACT` at its default so that `Authorization` headers stay out of the output you paste into a ticket.

In production. The first reply to "Git does something strange on the build machine" is two commands run there: `git config list --show-scope --show-origin`, and the failing command with `GIT_TRACE=1`.

14B.8 Tags: three kinds, two mechanisms

In one sentence. A tag is a ref under `refs/tags/` that is not expected to move; a lightweight tag points straight at a commit, and an annotated tag points at a tag object that names the commit and records who tagged it, when and why.

Analogy. A lightweight tag is a sticky note on a page of the ledger. An annotated tag is a certificate that cites the page number and is filed in the ledger with a page number of its own. The analogy breaks because nobody certifies anything unless the tag is also signed.

Precisely. Chapter 7 (section 7.11) introduced the lightweight tag as a ref. The annotated tag adds the fourth object type of Chapter 3. A tag object has four header lines and a message ([git-tag](#), [gitformat-signature](#)):

Field	Content
object	the ID of the object being tagged
type	its type: almost always <code>commit</code> , but <code>tree</code> , <code>blob</code> and <code>tag</code> are legal
tag	the tag's name, repeated inside the object
tagger	name, email, time and offset of the person who created the tag
message	free text; in a signed tag the signature block is appended to it

A *signed* tag is an annotated tag whose text ends in a signature (section 14B.16). So there are three kinds and two mechanisms: a bare ref, or a ref plus an object.

See it. A lightweight tag creates no object:

```
$ git log --oneline
eb112a5 Add rate limits
516c4c3 Add model router
37da450 Add README
$ git tag staging-2026-09-07
$ cat .git/refs/tags/staging-2026-09-07
eb112a5c44b0b2092c4ef2cae6e75b16da000f73
$ git cat-file -t staging-2026-09-07
commit
$ git count-objects | cut -d, -f1
11 objects
```

An annotated tag (`git tag -a`  **SAFE**) creates one:

```
$ git tag -a v1.0.0 -m "inference-gateway 1.0.0" -m "First release with rate limits."
$ git count-objects | cut -d, -f1
12 objects
$ cat .git/refs/tags/v1.0.0
b69624883fa40e51cbbf4c4baf49365ba1caa02e
$ git cat-file -t v1.0.0
tag
$ git cat-file -p v1.0.0
object eb112a5c44b0b2092c4ef2cae6e75b16da000f73
type commit
tag v1.0.0
tagger Lab User <you@example.com> 1788756060 +0530

inference-gateway 1.0.0

First release with rate limits.
```

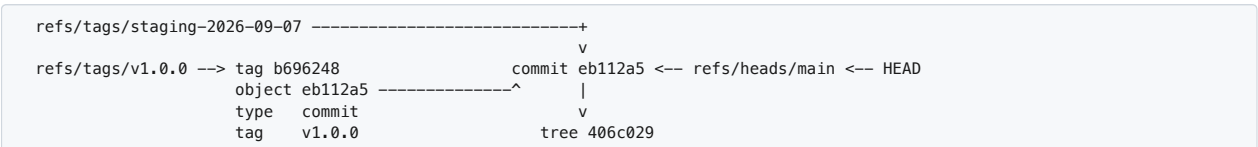
Twelve objects where there were eleven. The ref holds `b696248`, the ID of the tag object; the tag object's `object` line holds the commit `eb112a5`. Getting from one to the other is called *peeling*:

```
$ git rev-parse v1.0.0 'v1.0.0^{tag}' 'v1.0.0^{commit}' 'v1.0.0^{*}' 'v1.0.0^{tree}'
b69624883fa40e51cbbf4c4baf49365ba1caa02e
b69624883fa40e51cbbf4c4baf49365ba1caa02e
eb112a5c44b0b2092c4ef2cae6e75b16da000f73
eb112a5c44b0b2092c4ef2cae6e75b16da000f73
406c0291b052f665e4581982583c94937f169d80
$ git show-ref --tags --dereference
eb112a5c44b0b2092c4ef2cae6e75b16da000f73 refs/tags/staging-2026-09-07
b69624883fa40e51cbbf4c4baf49365ba1caa02e refs/tags/v1.0.0
eb112a5c44b0b2092c4ef2cae6e75b16da000f73 refs/tags/v1.0.0^{*}
$ git for-each-ref refs/tags --format='%(%{refname:short}) | (%{objecttype}) (%{objectname:short}) | peeled: (%{objecttype}) %
(%{objectname:short})'
staging-2026-09-07 | commit eb112a5 | peeled:
v1.0.0 | tag b696248 | peeled: commit eb112a5
```

`v1.0.0^{commit}` follows tag objects until it reaches a commit; `^{*}` until it reaches anything that is not a tag. In `git show-ref --dereference` and `git ls-remote`, the line ending in `^{*}` is the peeled value of the line above. Commands that want a commit (`git log v1.0.0`, `git switch --detach v1.0.0`) peel for you.

Inside .git. `git tag -a` writes one object and one ref. No relog is written for tags unless `core.logAllRefUpdates` is `always` (Chapter 7). HEAD, the index and the working tree are not involved.

Picture.



```
tagger Lab User ... +0530
"inference-gateway 1.0.0"
```

More than the latest commit. A tag can name an older commit, `-m` alone implies `-a`, and the tagger date is the moment of tagging, not of the commit:

```
$ git tag -a v0.9.0 -m "Internal preview" HEAD~1
$ git tag -m "Rate limits verified on staging" v1.0.0-verified
$ git tag -n1
staging-2026-09-07 Add rate limits
v0.9.0          Internal preview
v1.0.0          inference-gateway 1.0.0
v1.0.0-verified Rate limits verified on staging
$ git for-each-ref refs/tags --format='%(refname:short) %(objecttype) %(creatordate:iso)'
staging-2026-09-07 commit 2026-09-07 10:05:00 +0530
v0.9.0 tag 2026-09-07 10:21:00 +0530
v1.0.0 tag 2026-09-07 10:11:00 +0530
v1.0.0-verified tag 2026-09-07 10:22:00 +0530
```

A tag can also name a blob, a tree or another tag. Pro Git reports that the Git project's maintainer publishes a public key this way, as a tagged blob ([Maintaining a Project](#)). A nested tag is almost always a mistake, and Git 2.55 says so:

```
# A tag can name any object. Here: one blob, the limits file as released, and then a tag of a tag.
$ git tag -a limits-schema-v1 -m "Limits file format, version 1" HEAD:config/limits.yaml
$ git cat-file -p limits-schema-v1 | head -3
object cb775a8efb9ad28e94389151972d6b06a5b263ee
type blob
tag limits-schema-v1
$ git tag -a v1.0.0-audited -m "Audit ticket SEC-88 closed" v1.0.0
hint: You have created a nested tag. The object referred to by your new tag is
hint: already a tag. If you meant to tag the object that it points to, use:
hint:
hint:   git tag -f v1.0.0-audited v1.0.0^{tag}
hint: Disable this message with "git config set advice.nestedTag false"
$ git cat-file -p v1.0.0-audited | head -3
object b69624883fa40e51cbbf4c4baf49365ba1caa02e
type tag
tag v1.0.0-audited
$ git rev-parse v1.0.0-audited 'v1.0.0-audited^{tag}' 'v1.0.0-audited^{*}'
6b410c8476aa6fcc8612bdcee3310b2654ae803d
6b410c8476aa6fcc8612bdcee3310b2654ae803d
eb112a5c44b0b2092c4ef2cae6e75b16da000f73
```

Tag names obey the ref-name rules of [git-check-ref-format](#): no spaces, no `~`, `^` or `:`. The `+` of a build-metadata suffix is allowed:

```
$ git tag "v1.0 final"
fatal: 'v1.0 final' is not a valid tag name.
[exit status: 128]
$ git tag v1.0.0
fatal: tag 'v1.0.0' already exists
[exit status: 128]
$ git check-ref-format refs/tags/v1.0.0+build.7
[exit status: 0]
$ git check-ref-format "refs/tags/v1.0.0~1"
[exit status: 1]
```

Which kind when. The manual's rule: annotated tags for releases, lightweight tags for private or temporary labels; `git describe` ignores lightweight tags by default for that reason. An annotated tag also answers "who released this, and when", which the last commit's author line cannot.

In production. An ML team tags the commit that produced each published model with an annotated tag `model/reranker/2026-09-07` and writes the evaluation run ID into the message. Two years later `git show` on that tag still says which code, who released it and which evaluation justified it.

14B.9 Listing and sorting tags

`git tag` lists in byte order, which is wrong for version numbers:

```
$ git log --oneline --decorate
a247d58 (HEAD -> main) Document streaming
c4f3ec7 (tag: v2.0.0-rc.2, tag: v2.0.0, tag: nightly) Fix stream flush
004673b (tag: v2.0.0-rc.1) Add streaming responses
e9fdbea (tag: v1.10.0) Add token check
ad88f54 (tag: v1.9.0) Add health endpoint
eb112a5 (tag: v1.2.0) Add rate limits
516c4c3 Add model router
37da450 Add README
$ git tag
nightly
v1.10.0
v1.2.0
```

```
v1.9.0
v2.0.0
v2.0.0-rc.1
v2.0.0-rc.2
```

```
$ git tag --sort=version:refname
nightly
v1.2.0
v1.9.0
v1.10.0
v2.0.0
v2.0.0-rc.1
v2.0.0-rc.2
$ git -c versionsort.suffix=-rc tag --sort=version:refname
nightly
v1.2.0
v1.9.0
v1.10.0
v2.0.0-rc.1
v2.0.0-rc.2
v2.0.0
```

`--sort=version:refname` compares the numeric parts as numbers, so `v1.10.0` follows `v1.9.0`. It still lists a pre-release *after* its release, because `-rc.1` is a longer string. `versionsort.suffix` names the suffixes that sort *before* the bare version. Both can be made the default:

```
$ git config set tag.sort version:refname
$ git config set versionsort.suffix -rc
$ git tag --list 'v2.*'
v2.0.0-rc.1
v2.0.0-rc.2
v2.0.0
$ git tag --list 'v[0-9]*' --sort=-version:refname | head -1
v2.0.0
```

The last command is the usual scripted answer to "what is the newest release". Filters select by history, not by name:

```
$ git tag --contains HEAD~2
nightly
v2.0.0-rc.1
v2.0.0-rc.2
v2.0.0
$ git tag --no-contains HEAD~2
v1.2.0
v1.9.0
v1.10.0
$ git tag --points-at HEAD~1
nightly
v2.0.0-rc.2
v2.0.0
$ git tag --merged v1.9.0
v1.2.0
v1.9.0
```

`--contains <commit>` answers "which releases include this fix"; `--merged <commit>` answers "which releases are in the history of this commit". `--format` takes the fields of [git-for-each-ref](#).

14B.10 Tags and remotes

Chapter 12 covers the transfer mechanics; this is the part specific to the tag namespace.

Push sends tags only when asked.

```
$ git tag -a v1.0.0 -m "inference-gateway 1.0.0"
$ printf 'burst: 20\n' >> config/limits.yaml
$ git commit -q -am "Allow short bursts"
$ git push
To ../../server/inference-gateway.git
 d20ef7a..fa279e8 main -> main
$ git ls-remote --tags origin
```

```
$ git push origin v1.0.0
To ../../server/inference-gateway.git
 * [new tag]          v1.0.0 -> v1.0.0
$ git ls-remote --tags origin
3caaa3907d5f3c3071cdfefbad5c416881fa9be07      refs/tags/v1.0.0
d20ef7a61e04035281b8b38ed0d78ee612252220      refs/tags/v1.0.0^{}
```

`git push origin v1.0.0` 🟡 CAUTION publishes one tag. `git push --follow-tags` sends, with the branch, every *annotated* tag that points into the commits being pushed and is missing on the server; `push.followTags=true` makes that the default. `git push --tags` sends every tag you have, including the private ones:

```
$ git tag -a v1.0.1 -m "inference-gateway 1.0.1"
$ git tag canary-ok
$ printf 'retry_after_seconds: 2\n' >> config/limits.yaml
$ git commit -q -am "Tell clients when to retry"
$ git tag wip-retry-header
$ git push --follow-tags
To ../../server/inference-gateway.git
 fa279e8..46cb046  main -> main
 * [new tag]         v1.0.1 -> v1.0.1
$ git ls-remote --tags origin
3caaa3907d5f3c3071cdfebad5c416881fa9be07  refs/tags/v1.0.0
d20ef7a61e04035281b8b38ed0d78ee612252220  refs/tags/v1.0.0^{*}
65ad079e6bee92a0439fecf06a757ee24052b697  refs/tags/v1.0.1
fa279e82e36171f73f18c59df2c68bef182e5601  refs/tags/v1.0.1^{*}
```

v1.0.1 went. The lightweight tags `canary-ok` and `wip-retry-header` stayed, which is the point of `--follow-tags`.

Fetch follows tags. A fetch brings every tag that points into the history it downloads, of either kind, and never updates a tag you already have (Chapter 12, section 12.4):

```
$ cd ../../asha/inference-gateway
$ git fetch
From ../../server/inference-gateway
 d20ef7a..46cb046  main -> origin/main
 * [new tag]         v1.0.0 -> v1.0.0
 * [new tag]         v1.0.1 -> v1.0.1
$ git tag
v1.0.0
v1.0.1
```

Deleting. There are three places a tag lives, and each needs its own command:

```
$ cd ../../you/inference-gateway
$ git tag -d canary-ok
Deleted tag 'canary-ok' (was fa279e8)
$ git push origin --delete v1.0.1
To ../../server/inference-gateway.git
 - [deleted]         v1.0.1
$ git ls-remote --tags origin
3caaa3907d5f3c3071cdfebad5c416881fa9be07  refs/tags/v1.0.0
d20ef7a61e04035281b8b38ed0d78ee612252220  refs/tags/v1.0.0^{*}
$ git tag
v1.0.0
v1.0.1
wip-retry-header
```

`git tag -d` 🟡 CAUTION removes your ref. `git push origin --delete <tag>` 🔴 DANGEROUS removes the server's (the same as pushing `:refs/tags/<tag>`); your local `v1.0.1` is still there, and so is everyone else's:

```
$ cd ../../asha/inference-gateway
$ git tag bisect-good-2026-09-07 HEAD~1
$ git fetch --prune
$ git tag
bisect-good-2026-09-07
v1.0.0
v1.0.1
$ git fetch --prune --prune-tags
From ../../server/inference-gateway
 - [deleted]         (none) -> bisect-good-2026-09-07
 - [deleted]         (none) -> v1.0.1
$ git tag
v1.0.0
```

`git fetch --prune` does not touch tags. `git fetch --prune --prune-tags` 🔴 DANGEROUS makes your tags equal to the server's: it deleted Asha's private `bisect-good-2026-09-07` along with `v1.0.1`. And a deleted tag that someone still holds comes back with their next `git push --tags`.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git tag <name></code> , <code>-a</code> , <code>-s</code>	unchanged	unchanged	unchanged	unchanged	new <code>refs/tags/<name></code> ; with <code>-a</code> or <code>-s</code> also a tag object	unchanged	unchanged
<code>git tag -f <name></code>	unchanged	unchanged	unchanged	unchanged	ref rewritten; the old tag object becomes unreachable; no reflog	unchanged	unchanged
<code>git tag -d <name></code>	unchanged	unchanged	unchanged	unchanged	ref deleted	unchanged	unchanged
<code>git push origin <tag></code>	unchanged	unchanged	unchanged	unchanged	unchanged	new <code>refs/tags/<tag></code> and its objects	the tag appears; a ruleset may refuse it (Chapter 18)

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git push --force origin <tag>, --delete <tag></code>	unchanged	unchanged	unchanged	unchanged	unchanged	tag ref replaced or deleted	a release built on the tag is affected (Chapter 15)
<code>git fetch --tags --force</code>	unchanged	unchanged	unchanged	unchanged	local tags overwritten by the server's	unchanged	unchanged

14B.11 Why a published tag must not move

In one sentence. Every clone keeps its own copy of each tag and never updates it by itself, so moving a tag on the server does not move it anywhere else: it creates two things with one name.

See it. `v1.2.0` is released; you and Asha both have it. An hour later a bug is fixed on `main`, and someone decides that `v1.2.0` should include the fix:

```
# A bug is found an hour after the release. The fix lands on main, and the tag is "corrected".
$ printf 'requests_per_minute: 60\nmax_tokens: 4096\n' > config/limits.yaml
$ git commit -q -am "Fix max_tokens limit"
$ git push -q
$ git tag -f -a v1.2.0 -m "inference-gateway 1.2.0"
Updated tag 'v1.2.0' (was 5b231b5)
$ git push origin v1.2.0
To ../../server/inference-gateway.git
 ! [rejected]        v1.2.0 -> v1.2.0 (already exists)
error: failed to push some refs to ' ../../server/inference-gateway.git'
hint: Updates were rejected because the tag already exists in the remote.
[exit status: 1]
$ git push --force origin v1.2.0
To ../../server/inference-gateway.git
+ 5b231b5...5a0dc77 v1.2.0 -> v1.2.0 (forced update)
```

Git refused twice: `git tag` needed `-f` ⚠ CAUTION, and the push was rejected ("already exists") until `--force` 🚫 DANGEROUS. Asha pulls:

```
$ cd ../../asha/inference-gateway
$ git pull
From ../../server/inference-gateway
 d20ef7a..2518733 main    -> origin/main
Updating d20ef7a..2518733
Fast-forward
 config/limits.yaml | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline --decorate
2518733 (HEAD -> main, origin/main, origin/HEAD) Fix max_tokens limit
d20ef7a (tag: v1.2.0) Add rate limits
73c05d1 Add model router
dfbc830 Add README
$ git show v1.2.0:config/limits.yaml
requests_per_minute: 60
max_tokens: 2048
```

Her `main` has the fix. Her `v1.2.0` is where it always was. A machine that clones now gets the other one:

```
$ cd ../../
$ git clone -q server/inference-gateway.git ci/inference-gateway
$ cd ci/inference-gateway
$ git log --oneline --decorate
2518733 (HEAD -> main, tag: v1.2.0, origin/main, origin/HEAD) Fix max_tokens limit
d20ef7a Add rate limits
73c05d1 Add model router
dfbc830 Add README
$ git show v1.2.0:config/limits.yaml
requests_per_minute: 60
max_tokens: 4096
```

One clone's `v1.2.0` has `max_tokens: 2048`, the other's `4096`. Both builds are labelled `1.2.0`.

Picture.

server	Asha's clone	CI clone (cloned after the move)
<code>v1.2.0 -> 2518733 (moved)</code>	<code>v1.2.0 -> d20ef7a (kept)</code>	<code>v1.2.0 -> 2518733</code>
<code>d20ef7a Add rate limits</code>	<code><- what was released as 1.2.0</code>	
<code>2518733 Fix max_tokens limit</code>	<code><- what the server now calls 1.2.0</code>	

Detect it. Compare the server's peeled tag with yours; `git fetch --tags` also reports the clash:

```
$ cd ../../asha/inference-gateway
# What the server says the tag is, and what this clone says it is:
```

```
$ git ls-remote origin 'refs/tags/v1.2.0^{*}'
2518733b711886bd82418641eb2434765079e288      refs/tags/v1.2.0^{*}
$ git rev-parse 'v1.2.0^{commit}'
d20ef7a61e04035281b8b38ed0d78ee612252220
$ git fetch --tags
From ../../server/inference-gateway
! [rejected] v1.2.0      -> v1.2.0  (would clobber existing tag)
[exit status: 1]
```

```
Observed behavior : two builds labelled v1.2.0 contain different code.
Git state          : refs/tags/v1.2.0 names d20ef7a in clones that fetched before the move and
                    2518733 on the server and in clones made after it.
Mechanism          : fetch creates missing tags and refuses to overwrite existing ones ("would
                    clobber existing tag"); a plain fetch does not even report the difference.
Root cause         : a published tag was replaced with git tag -f and git push --force.
Why Git does this  : a tag name is a promise about content. The manual: Git "does not (and it
                    should not) change tags behind users back".
Correct fix        : put the tag back where it was published; release the new content under a new name.
Prevention         : server-side rule that existing release tags cannot be updated or deleted;
                    deploy by commit ID or image digest, not by tag name alone.
```

Repair it. `v1.2.0` goes back, and the fix ships as `v1.2.1`. Asha's clone still has the original tag object, so she can restore it:

```
# The sane repair: put v1.2.0 back where it was published, and release the fix as v1.2.1.
# Asha still has the original tag object, so she can restore it.
$ git push --force origin v1.2.0
To ../../server/inference-gateway.git
+ 5a0dc77...5b231b5 v1.2.0 -> v1.2.0 (forced update)
$ cd ../../you/inference-gateway
$ git fetch --tags --force
From ../../server/inference-gateway
t [tag update]      v1.2.0      -> v1.2.0
$ git tag -a v1.2.1 -m "inference-gateway 1.2.1: fix max_tokens limit"
$ git push origin v1.2.1
To ../../server/inference-gateway.git
* [new tag]         v1.2.1 -> v1.2.1
$ git log --oneline --decorate
2518733 (HEAD -> main, tag: v1.2.1, origin/main, origin/HEAD) Fix max_tokens limit
d20ef7a (tag: v1.2.0) Add rate limits
73c05d1 Add model router
dfbc830 Add README
```

Clones that picked up the moved tag in between must be corrected by hand with `git fetch --tags --force`. If nobody kept the original ref, the tag object is usually still in someone's object database as a "dangling tag" (Lab 13.2).

Prevent it. Two settings of plain Git sound as if they would help:

```
# Two server settings that sound as if they protect tags. Set them and try.
$ cd ../../you/inference-gateway
$ git -C ../../server/inference-gateway.git config set receive.denyNonFastForwards true
$ git -C ../../server/inference-gateway.git config set receive.denyDeletes true
$ git tag -a v1.3.0-rc.1 -m "Release candidate"
$ git push -q origin v1.3.0-rc.1
$ git tag -f -a v1.3.0-rc.1 -m "Release candidate, moved" HEAD~1
Updated tag 'v1.3.0-rc.1' (was d15e77e)
$ git push --force origin v1.3.0-rc.1
To ../../server/inference-gateway.git
+ d15e77e...cc3836f v1.3.0-rc.1 -> v1.3.0-rc.1 (forced update)
[exit status: 0]
$ git push origin --delete v1.3.0-rc.1
To ../../server/inference-gateway.git
- [deleted]         v1.3.0-rc.1
[exit status: 0]
```

Both were accepted. In Git 2.55.0 the code behind `receive.denyNonFastForwards` and `receive.denyDeletes` applies them only to refs under `refs/heads/` ([receive-pack.c](#)); the manual's wording does not say so. On a plain Git server, tags are protected by a hook:

```
# A server-side update hook: an existing tag under refs/tags/v* can be neither moved nor deleted.
$ cat ../../update-hook
#!/bin/sh
# update hook: called once per ref with <ref> <old-id> <new-id>.
# A release tag may be created. It may not be moved or deleted.
ref=$1 old=$2
case "$ref" in
refs/tags/v*)
if ! printf '%s' "$old" | grep -q '^0*$'; then
echo "policy: $ref is published and immutable; release a new version" >&2
exit 1
fi ;;
esac
exit 0
$ cp ../../update-hook ../../server/inference-gateway.git/hooks/update
$ chmod +x ../../server/inference-gateway.git/hooks/update
$ git push -q origin v1.3.0-rc.1
$ git tag -f -a v1.3.0-rc.1 -m "Release candidate, moved again" HEAD
Updated tag 'v1.3.0-rc.1' (was cc3836f)
```

```
$ git push --force origin v1.3.0-rc.1
remote: policy: refs/tags/v1.3.0-rc.1 is published and immutable; release a new version
remote: error: hook declined to update refs/tags/v1.3.0-rc.1
To ../../server/inference-gateway.git
! [remote rejected] v1.3.0-rc.1 -> v1.3.0-rc.1 (hook declined)
error: failed to push some refs to ' ../../server/inference-gateway.git'
[exit status: 1]
$ git push origin --delete v1.3.0-rc.1
remote: policy: refs/tags/v1.3.0-rc.1 is published and immutable; release a new version
remote: error: hook declined to update refs/tags/v1.3.0-rc.1
To ../../server/inference-gateway.git
! [remote rejected] v1.3.0-rc.1 (hook declined)
error: failed to push some refs to ' ../../server/inference-gateway.git'
[exit status: 1]
```

GitHub, not Git. On GitHub the equivalent is a tag ruleset that restricts updates and deletions (Chapter 18); tag protection rules were retired in August 2024 in favor of rulesets ([sunset notice](#)). A *release* is a GitHub object layered on a Git tag, and an immutable release additionally locks the tag and its assets (Chapter 15).

In production. The CTO's third question is also a supply-chain question. The Phase 0 report records that third-party GitHub Actions referenced by tag were repointed in real attacks, which is why this course pins actions by full commit ID (Chapter 21A). A tag is a mutable name held by someone else: record the commit ID next to every tag you depend on.

14B.12 `git describe`: a version from history

In one sentence. `git describe` names a commit by the nearest annotated tag in its history, the number of commits since that tag, and the commit's abbreviated ID.

Precisely. If a tag points at the commit, the output is the tag name. Otherwise Git walks back through history, finds tagged ancestors, picks the one with the fewest commits between it and the commit, and prints `<tag>-<count>-g<abbreviated-id>`; the `g` stands for "git" ([git-describe](#)). Lightweight tags are ignored unless you pass `--tags`.

```
$ git log --oneline --decorate
c053f0d (HEAD -> main) Add token check
f37f723 Add health endpoint
eb112a5 (tag: v1.0.0, tag: staging-ok) Add rate limits
516c4c3 Add model router
37da450 Add README
$ git describe
v1.0.0-2-gc053f0d
$ git rev-parse --short HEAD
c053f0d
$ git rev-list --count v1.0.0..HEAD
2
$ git log --oneline "$(git describe)" -1
c053f0d Add token check
```

`v1.0.0-2-gc053f0d`: two commits after `v1.0.0`, at commit `c053f0d`. The whole string is also a valid revision, so a version printed by a running service can be pasted into `git show`.

```
$ printf 'def check(token):\n    return token == "letmein"\n' > gateway/auth.py
$ git describe --dirty
v1.0.0-2-gc053f0d-dirty
$ git describe --dirty=+local
v1.0.0-2-gc053f0d+local
$ git restore gateway/auth.py
$ git describe --dirty
v1.0.0-2-gc053f0d
```

`--dirty` appends a mark when tracked files differ from HEAD, so a build from a modified working tree cannot pass as a release.

```
$ git tag -a v1.1.0-rc.1 -m "Release candidate" HEAD~1
$ git tag deployed-staging
$ git describe
v1.1.0-rc.1-1-gc053f0d
$ git describe --exclude '*-rc.*'
v1.0.0-2-gc053f0d
$ git describe --tags
deployed-staging
$ git describe --tags --match 'v[0-9]*'
v1.1.0-rc.1-1-gc053f0d
```

`--match` and `--exclude` take globs and decide which tags may be used. Other options: `--abbrev=0` prints only the nearest tag; `--long` always prints the full form; `--exact-match` fails unless the commit itself is tagged; `--first-parent` ignores tags that arrived through merged branches; `--contains` asks which tag comes *after* this commit.

Three ways it fails.

```
$ git describe
fatal: No names found, cannot describe anything.
[exit status: 128]
$ git describe --always
eb112a5
$ git tag staging-ok
$ git describe
fatal: No annotated tags can describe 'eb112a5c44b0b2092c4ef2cae6e75b16da000f73'.
However, there were unannotated tags: try --tags.
[exit status: 128]
$ git describe --tags
staging-ok
```

No tags at all; only lightweight tags; and the third, which is the common one in CI:

```
$ cd ../../
$ git clone -q --depth 1 "file://$PWD/server/inference-gateway.git" ci/inference-gateway
$ cd ci/inference-gateway
$ git rev-parse --is-shallow-repository
true
$ git log --oneline
72212ba Add token check
$ git tag
$ git describe
fatal: No names found, cannot describe anything.
[exit status: 128]
$ git describe --always
72212ba
```

A depth-1 clone has one commit and no tags. Fetching the tag is not enough, because the commits *between* the tag and the tip are still missing and Git cannot count them:

```
$ git fetch -q --depth 1 origin tag v1.0.0
$ git tag
v1.0.0
$ git describe
fatal: No tags can describe '72212bae7bd9bd85bd7300a9a92d9f8d50f89807'.
Try --always, or create some tags.
[exit status: 128]
```

```
$ git fetch --unshallow
$ git rev-parse --is-shallow-repository
false
$ git describe
v1.0.0-2-g72212ba
```

```
Observed behavior : the build in CI gets a fallback version or fails; locally the version is v1.0.0-2-g72212ba.
Git state          : the CI checkout is shallow (one commit) and has no refs/tags/.
Mechanism          : git describe needs a tagged ancestor and the commits between it and HEAD.
Root cause         : the checkout step fetched one commit; tools that derive versions from tags
                    (git describe, setuptools-scm and similar) fall back or fail.
Why Git does this  : a shallow clone is a deliberate trade of history for speed.
Correct fix        : fetch full history and tags in jobs that compute a version.
Prevention         : make the release job fail when git describe --exact-match fails.
```

GitHub, not Git. `actions/checkout` fetches one commit and no tags by default; `fetch-depth: 0` fetches everything ([checkout README](#)). Chapter 20A returns to this.

14B.13 Semantic Versioning 2.0.0

Git attaches no meaning to a tag name. [Semantic Versioning 2.0.0](#) is the convention most projects put on top:

- A version is `MAJOR.MINOR.PATCH`. Incompatible API changes raise MAJOR; backward-compatible additions raise MINOR; backward-compatible fixes raise PATCH.
- A released version is immutable: its contents "MUST NOT be modified". Any change is a new version. This is the same rule as section 14B.11, stated for packages.
- `0.y.z` is initial development, where anything may change.
- A pre-release is marked by a hyphen and dot-separated identifiers (`1.3.0-rc.1`) and has *lower* precedence than the release (`1.3.0-rc.1 < 1.3.0`). Numeric identifiers compare as numbers, so `rc.2 < rc.11`.
- Build metadata follows a plus sign (`1.3.0+build.7`) and is ignored when versions are compared.
- The specification's FAQ says that `v1.2.3` is not itself a semantic version; the `v` prefix is a common way to mark a tag as a version. The version is `1.2.3`.

Three consequences for Git. The pre-release rule is why section 14B.9 needed `versionsort.suffix`. The output of `git describe (v1.0.0-2-gc053f0d)` is not a valid semantic version of a later build, because by the pre-release rule it would sort *before* `1.0.0`; tools that publish

packages translate it into their ecosystem's format. And SemVer describes a public API: for a service that nobody links against, a date-based scheme carries the same information with less ceremony.

14B.14 Release branches, on the Git side

When more than one version is in use, a tag is not enough: the old version needs fixes that must not bring new features with them. A release branch is an ordinary branch that starts at the release tag.

```
$ git cherry-pick -x main~1
[release/1.2 c40960e] Fix max_tokens limit
Date: Mon Sep 7 10:08:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
$ git log -1 --format=%B
Fix max_tokens limit

(cherry picked from commit 362b359d29ea9c07389c0782fd46c24269b6a3f)

$ git tag -a v1.2.1 -m "inference-gateway 1.2.1: fix max_tokens limit"
```

The fix was made on `main` first and copied to `release/1.2` with `git cherry-pick -x`, which records the source commit in the message (Chapter 10). Then the patch release is tagged on the release branch:

```
$ git log --graph --oneline --decorate --all
* c40960e (HEAD -> release/1.2, tag: v1.2.1) Fix max_tokens limit
| * 8ed5c59 (main) Add batch endpoint
| * 362b359 Fix max_tokens limit
| * 27d1fb6 Add streaming responses
|/
* eb112a5 (tag: v1.2.0) Add rate limits
* 516c4c3 Add model router
* 37da450 Add README
$ git describe release/1.2
v1.2.1
$ git describe main
v1.2.0-3-g8ed5c59
```

`main` describes itself as `v1.2.0-3-g8ed5c59` although 1.2.1 has been released: `v1.2.1` is not an ancestor of `main`, and `git describe` only looks backward along history. Questions about content need the tools that understand copies:

```
$ git tag --contains main~1
$ git tag --contains release/1.2
v1.2.1
$ git tag --merged main
v1.2.0
$ git cherry -v release/1.2 main
+ 27d1fb6a67cdb41e87d5771c3e10310823ee0dcb Add streaming responses
- 362b359d29ea9c07389c0782fd46c24269b6a3f Fix max_tokens limit
+ 8ed5c59df2f47f80f396b5ba11769efbe1d6491e Add batch endpoint
```

No tag *contains* the original fix commit `362b359`, because the release has the copy `c40960e`. `git cherry` compares patches, not IDs, and its `-` line says that the fix on `main` has an equivalent on the release branch. When 1.3.0 is cut from `main`, the same comparison separates what is new from what was backported:

```
$ git switch -q main
$ git tag -a v1.3.0 -m "inference-gateway 1.3.0"
$ git tag --sort=version:refname
v1.2.0
v1.2.1
v1.3.0
$ git describe main
v1.3.0
$ git log --oneline v1.2.1..v1.3.0
8ed5c59 Add batch endpoint
362b359 Fix max_tokens limit
27d1fb6 Add streaming responses
$ git log --oneline --cherry-pick --right-only v1.2.1..v1.3.0
8ed5c59 Add batch endpoint
27d1fb6 Add streaming responses
```

Two conventions exist for the direction of a fix (Phase 0 report): the Git project fixes on the oldest maintained branch and merges upward ([gitworkflows](#)); trunk-based practice fixes on the main line and cherry-picks to the release branch, as here. Chapter 27 discusses the choice.

In production. A model-serving team runs `1.2.x` for a regulated customer and `1.3.x` for everyone else. "Is the token-limit fix in what the regulated customer runs?" is answered by `git cherry -v release/1.2 main` and by the trailer that `-x` wrote, not by looking for the commit ID on the release branch, where it will never be.

14B.15 Signatures: what is signed, and by which program

In one sentence. A Git signature is a detached cryptographic signature over the bytes of one commit or one annotated tag, produced by an external program and stored inside that object.

Analogy. A seal pressed onto one page of the ledger. Because each page cites earlier pages by numbers computed from their content, the seal vouches for the whole history behind it. The analogy breaks where it matters most: the seal says which *ring* was used, not whose hand held it.

Precisely. Git computes the object without a signature, hands those bytes to a signing program, and embeds the result: in a commit as the `gpgsig` header introduced in Chapter 6 (section 6.11), in a tag as a block appended to the message ([gitformat-signature](#)). Since the signature is part of the object, it is covered by the object ID; signing an existing commit means writing a new commit. `gpg.format` chooses the program:

<code>gpg.format</code>	Program (overridable with <code>gpg.<format>.program</code>)	Key chosen by	Who decides that the key is trusted
<code>openpgp</code> (default)	<code>gpg</code>	<code>user.signingKey</code> , else the committer identity	your GnuPG keyring and its trust levels (<code>gpg.minTrustLevel</code>)
<code>x509</code> (Git 2.19)	<code>gpgsm</code>	<code>user.signingKey</code> , else the committer identity	the certificate chain
<code>ssh</code> (Git 2.34)	<code>ssh-keygen</code>	<code>user.signingKey</code> (a key file, or a literal <code>key:... with an agent</code>), else <code>gpg.ssh.defaultKeyCommand</code>	the file named by <code>gpg.ssh.allowedSignersFile</code>

Git itself does no cryptography. To show what it asks of the program without starting `gpg`, `gpgsm` or an agent, this demo puts a stand-in in their place that prints its arguments and fails:

```
$ git -c gpg.program=~/.lab-bin/show-args commit -S --allow-empty -m "Signed with OpenPGP"
error: gpg failed to sign the data:
signing program called with: --status-fd=2 -bsau Lab User <you@example.com>

fatal: failed to write commit object
[exit status: 128]
```

"gpg failed to sign the data" is the generic message for "the signing program returned an error": no key for that identity, an agent that cannot ask for the passphrase, a wrong `gpg.program`. The key was selected by the committer identity because `user.signingKey` is not set.

```
$ git -c gpg.format=ssh -c gpg.ssh.program=~/.lab-bin/show-args commit -S --allow-empty -m "Signed with SSH"
fatal: either user.signingkey or gpg.ssh.defaultKeyCommand needs to be configured
[exit status: 128]
$ git -c gpg.format=ssh -c gpg.ssh.program=~/.lab-bin/show-args -c user.signingKey=~/.keys/signing_key.pub commit -S --allow-empty -m "Signed with SSH"
error: signing program called with: -Y sign -n git -f $LAB/ch14b/signing-backends/home/keys/signing_key.pub <file with the payload>

fatal: failed to write commit object
[exit status: 128]
```

The SSH backend has no default key. With one, it runs `ssh-keygen -Y sign` with the namespace `git`. The rest of this chapter uses SSH signing: you already have SSH keys, and the trust file is plain text. OpenPGP and X.509 signing are not run here, because both would start an agent process.

14B.16 SSH signing, end to end

This demo creates a key pair inside the sandbox, without passphrase or agent. It is volatile: your key, signatures and IDs will differ. A real signing key has a passphrase and lives in an agent or a hardware token (Chapter 16).

```
$ mkdir ~/keys
$ ssh-keygen -q -t ed25519 -N '' -C 'you@example.com signing key 2026' -f ~/keys/signing_key
$ ls ~/keys
signing_key
signing_key.pub
$ cat ~/keys/signing_key.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIPeZ8qdHW0UuKtL/yAKkDk4n+e6oo2s02JTJ8lmdYFDE you@example.com signing key 2026
$ ssh-keygen -l -f ~/keys/signing_key.pub
256 SHA256:wIC1YB2G8ErM9XCgJQ+pItmnhwF/fv4lGfAmWeriHG5 you@example.com signing key 2026 (ED25519)
```

```
$ git config set --global gpg.format ssh
$ git config set --global user.signingKey ~/keys/signing_key.pub
$ git config get --all --show-names --regexp "^(gpg|user|.signingkey)"
user.signingkey $LAB/ch14b/ssh-signing/home/keys/signing_key.pub
gpg.format ssh
```

`user.signingKey` names the *public* key file. `ssh-keygen` finds the private key beside it, or in an agent. A signed commit, read as an object:

```
$ printf 'burst: 20\n' >> config/limits.yaml
$ git commit -q -S -am "Allow short bursts"
$ git cat-file -p HEAD
tree c84890b7f7cbb711c04b9a7fb78410beb5897d9b
parent eb112a5c44b0b2092c4ef2cae6e75b16da000f73
author Lab User <you@example.com> 1788756300 +0530
```

```

committer Lab User <you@example.com> 1788756300 +0530
gpgsig -----BEGIN SSH SIGNATURE-----
U1N1U01HAAAAAQAADMAAAALc3NoLWVkJU1MTkAAAg95nyp0dY5S4q0v/IAq00Tif57q
ijaw7YlMnyWZ1gUMQAAADZ2l0AAAAAaAZzaGE1MTIAAABTAAAC3NzaC1lZDI1NTE5
AAAAQJMccrn4Dmj2cHUPAHhz39NlMtXXlS2u+/l0ewunSjVhvREL3WkYdp/G0PrihonpVt
+ZHT0fNnmCm7pVB7KLH04=
-----END SSH SIGNATURE-----

Allow short bursts

```

-S  SAFE signs this one commit. The `gpgsig` header carries the signature; continuation lines start with a space. Now verify it:

```

$ git verify-commit HEAD
error: gpg.ssh.allowedSignersFile needs to be configured and exist for ssh signature verification
[exit status: 1]
$ git log -1 --show-signature --format="%h %s"
error: gpg.ssh.allowedSignersFile needs to be configured and exist for ssh signature verification
No signature
f215ce4 Allow short bursts
$ git log -1 --format="%h %G? %s"
error: gpg.ssh.allowedSignersFile needs to be configured and exist for ssh signature verification
f215ce4 N Allow short bursts

```

Observed behavior : `git log --show-signature` prints "No signature", and `%G?` prints N, for a commit that visibly has a `gpgsig` header.

Git state : the commit is signed. `gpg.ssh.allowedSignersFile` is not set.

Mechanism : SSH verification means "find this key in my list of allowed signers". Without a list Git does not start `ssh-keygen` at all and reports no result.

Root cause : signing needs one key; verifying needs a statement of whom you trust.

Why Git does this : an SSH key has no owner written into it and no web of trust. The manual: trust is "fully" when the key is in the file and "undefined" otherwise.

Correct fix : create the `allowed-signers` file and point `gpg.ssh.allowedSignersFile` at it.

Prevention : distribute the file with the signing setup; a team can keep it in the repository.

An `allowed-signers` line is a principal (an email address by convention), optional options, and a public key ([ssh-keygen](#), "ALLOWED SIGNERS"). `namespaces="git"` restricts the key to Git signatures:

```

$ printf 'you@example.com namespaces="git" %s\n' "$(cut -d' ' -f1,2 ~/keys/signing_key.pub)" > ~/allowed_signers
$ cat ~/allowed_signers
you@example.com namespaces="git" ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIPeZ8qdHW0UuKtL/yAKkDk4n+e6oo2s02JTJ8lmdYFDE
$ git config set --global gpg.ssh.allowedSignersFile ~/allowed_signers
$ git verify-commit HEAD
Good "git" signature for you@example.com with ED25519 key SHA256:wIC1YB2G8ErM9XCgJQ+pItmnhwF/fv4lGfAmWerIHGs
[exit status: 0]
$ git log -2 --show-signature --format="%h %an: %s"
Good "git" signature for you@example.com with ED25519 key SHA256:wIC1YB2G8ErM9XCgJQ+pItmnhwF/fv4lGfAmWerIHGs
f215ce4 Lab User: Allow short bursts
eb112a5 Lab User: Add rate limits

```

The same object now verifies. Nothing in the repository changed; your statement of trust did. For scripts, the format placeholders:

```

$ git log -2 --format="%h %G? signer=%GS trust=%GT %s"
f215ce4 G signer=you@example.com trust=fully Allow short bursts
eb112a5 N signer= trust=undefined Add rate limits
$ git log -1 --format="%GK"
SHA256:wIC1YB2G8ErM9XCgJQ+pItmnhwF/fv4lGfAmWerIHGs

```

%G?	Meaning in the manual (git-log)	Seen with SSH in this chapter
G	good signature	key is in the <code>allowed-signers</code> file and valid at the signature's time
B	bad signature	object altered after signing; key listed in the revocation file
U	good signature, unknown validity	key not in the file, or outside its validity window
N	no signature	unsigned; also printed, with an error, when no <code>allowed-signers</code> file is configured
X, Y, R, E	expired signature, expired key, revoked key, cannot be checked	not produced by the SSH demos here

Signing by default, and tags.

```

$ git config set --global commit.gpgSign true
$ git config set --global tag.gpgSign true
$ printf 'retry_after_seconds: 2\n' >> config/limits.yaml
$ git commit -q -am "Tell clients when to retry"
$ git commit -q --no-gpg-sign --allow-empty -m "Trigger the nightly evaluation"
$ git log -4 --format="%h %G? %GS %s"
e32a07f N Trigger the nightly evaluation
9d47104 G you@example.com Tell clients when to retry
f215ce4 G you@example.com Allow short bursts
eb112a5 N Add rate limits

```

`commit.gpgSign=true` signs every commit; `--no-gpg-sign` opts one out. `tag.gpgSign=true` turns every annotated tag into a signed one, so `-m` is enough:

```

$ git tag -m "inference-gateway 1.0.0" v1.0.0 HEAD~1
$ git cat-file -p v1.0.0
object 9d471046051e8d578b339ee37eb466c71e16b6ec
type commit
tag v1.0.0
tagger Lab User <you@example.com> 1788757380 +0530

inference-gateway 1.0.0
-----BEGIN SSH SIGNATURE-----
U1NIU0LHAAAAQAADMAAAALc3NoLWVkJmU1MTkAAAAg95nyp0dY5S4q0v/IAQ00Tif57q
ijaw7YlMnyWZ1gUMQAAAAADZ2l0AAAAAaZaGE1MTIAAABTAAAC3NzaC1lZDI1NTE5
AAAAQENhU0YSEgUB1RmZBsnzJdHgXZdddaBvXKoPIFK23QLUTlL6Gr+o9kyWszNlomBF
apz4b70k5FFB+R/+c0swo=
-----END SSH SIGNATURE-----
$ git verify-tag v1.0.0
Good "git" signature for you@example.com with ED25519 key SHA256:wIC1YB2G8ErM9XCgJQ+pItmnhwF/fv4lGfAmWerIHGs
[exit status: 0]
$ git tag -v v1.0.0
Good "git" signature for you@example.com with ED25519 key SHA256:wIC1YB2G8ErM9XCgJQ+pItmnhwF/fv4lGfAmWerIHGs
object 9d471046051e8d578b339ee37eb466c71e16b6ec
type commit
tag v1.0.0
tagger Lab User <you@example.com> 1788757380 +0530

inference-gateway 1.0.0

```

In a tag the signature follows the message. `git verify-tag`, `git tag -v` and `git verify-commit` exit non-zero on anything but a trusted good signature, so they work in scripts. What Git starts:

```

# Git does no cryptography itself. The programs it starts to sign and to verify:
$ GIT_TRACE=1 git commit -q --allow-empty -m 'Re-run the nightly evaluation' 2>&1 | grep -o 'run_command: ssh-keygen -Y [a-z-]*'
-n git
run_command: ssh-keygen -Y sign -n git
$ GIT_TRACE=1 git verify-commit HEAD 2>&1 | grep -o 'run_command: ssh-keygen -Y [a-z-]*'
run_command: ssh-keygen -Y find-principals
run_command: ssh-keygen -Y verify
$ git verify-commit --raw HEAD
Good "git" signature for you@example.com with ED25519 key SHA256:wIC1YB2G8ErM9XCgJQ+pItmnhwF/fv4lGfAmWerIHGs

```

One program to sign; two steps to verify: find which principal the key belongs to, then verify the signature for that principal.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git commit -S</code>	unchanged	as <code>git commit</code>	as <code>git commit</code>	moves to the new, signed commit	new commit object with a <code>gpgsig</code> header	unchanged	after a push: a verification badge, if the public key is registered as a signing key (Chapter 21B)
<code>git tag -s <name></code>	unchanged	unchanged	unchanged	unchanged	new tag object with an appended signature; new ref	unchanged	unchanged until pushed
<code>git verify-commit</code> , <code>git verify-tag</code> , <code>git log --show-signature</code>	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged

14B.17 What a signature covers, and what verification proves

Everything in the object is covered. Copy a signed commit, change one word of its message, keep the signature:

```

$ git log -1 --format='%h %G? %GS %s'
b389204 G you@example.com Allow short bursts
# Write a second commit object: same headers, same signature, one word of the message changed.
$ git cat-file commit HEAD | sed 's/short bursts/unlimited bursts/' | git hash-object -t commit -w --stdin > ../forged-id
$ git cat-file -p "$(cat ../forged-id)" | tail -1
Allow unlimited bursts
$ git verify-commit "$(cat ../forged-id)"
Could not verify signature.
Signature verification failed: incorrect signature
[exit status: 1]
$ git log -1 --format='%G? %s' "$(cat ../forged-id)"
B Allow unlimited bursts

```

The forged object has another ID, and its signature is B, bad. Tree, parents, author, committer, dates and message are all signed bytes; through the tree and parent IDs, a signature vouches for every file and every ancestor.

Nothing outside the object is covered.

```

# The signature is inside the object. Which refs point at the object is not part of it.
$ git branch hotfix/anything HEAD
$ git log -1 --format='%G? %D' hotfix/anything

```

```
G HEAD -> main, tag: v1.0.0, hotfix/anything
$ git update-ref refs/tags/v9.9.9 "$(git rev-parse v1.0.0)"
$ git verify-tag v9.9.9
Good "git" signature for you@example.com with ED25519 key SHA256:IBY0fL0BtdFj0fbHvPhR5Di74E0PfNUAA+UYVYfgHgw
[exit status: 0]
$ git cat-file -p v9.9.9 | sed -n 1,3p
object b3892048e77c43a09f64eedfeccc408eeb532e3a
type commit
tag v1.0.0
$ git tag -d v9.9.9
Deleted tag 'v9.9.9' (was 849e258)
```

A signed commit stays validly signed on any branch, in any repository. Anyone who can push can serve your signed tag object under the name v9.9.9; the object still says tag v1.0.0 inside, which is why a careful verifier compares that name with the one it asked for.

Rewrites drop signatures.

```
$ git commit -q --allow-empty -m "Trigger the nightly evaluation"
$ git log -2 --format='%h %G? %s'
65bce98 G Trigger the nightly evaluation
b389204 G Allow short bursts
$ git commit -q --amend --allow-empty --no-gpg-sign -m "Trigger the nightly evaluation run"
$ git log -2 --format='%h %G? %s'
2bc6aa4 N Trigger the nightly evaluation run
b389204 G Allow short bursts
$ git rebase -q --no-gpg-sign --force-rebase HEAD~2
$ git log -2 --format='%h %G? %s'
93c3833 N Trigger the nightly evaluation run
14656b3 N Allow short bursts
$ git rebase -q --gpg-sign --force-rebase HEAD~2
$ git log -2 --format='%h %G? %s'
7129bbb G Trigger the nightly evaluation run
acc22ee G Allow short bursts
```

Amend, rebase and cherry-pick write new commits (Chapters 6, 9 and 10), signed only if the person running the command signs, and then with *their* key.

GitHub, not Git. The same logic runs on the platform. According to GitHub's documentation, merge and squash commits made in the web interface are signed by GitHub, and commits produced by "rebase and merge" are not signed at all ([about commit signature verification](#)); Chapter 17 covers the merge methods.

A key you do not list proves nothing to you.

```
$ git -c user.signingKey=~/.keys/stranger.pub commit -q --allow-empty -m "Bump the model default"
$ git verify-commit HEAD
Good "git" signature with ED25519 key SHA256:dvbZrGKgxrd5uBx6kt8ux+C+H3qGogE5fD3p/a8CHZA
No principal matched.
[exit status: 1]
$ git log -1 --format='%h %G? signer=%GS trust=%GT %s'
81b4414 U signer= trust=undefined Bump the model default
```

The mathematics is fine ("Good signature") and the answer is still a failure (U, exit status 1): nobody you trust is attached to that key.

Key lifetimes rest on a date the signer chooses. An allowed-signers entry can carry `valid-after` and `valid-before`; Git passes the *committer date* of the object as the time to check against:

```
# The lab clock says 7 September 2026. Give the key a validity window that ended on 31 August.
$ printf 'you@example.com namespaces="git",valid-before="20260831" %s\n' "$(cut -d' ' -f1,2 ~/.keys/you_2026.pub)" >
~/allowed_signers
$ git verify-commit HEAD~1
Good "git" signature with ED25519 key SHA256:IBY0fL0BtdFj0fbHvPhR5Di74E0PfNUAA+UYVYfgHgw
$LAB/ch14b/signature-scope/home/allowed_signers:1: key has expired: verify time 2026-09-07T10:30:00 > valid-before 2026-08-31T00:00:00

No principal matched.
[exit status: 1]
# The verify time is the committer date, and the committer date is whatever the committer says.
$ GIT_COMMITTER_DATE='2026-08-30T12:00:00+05:30' git commit -q --allow-empty -m 'Rotate the staging credentials'
$ git verify-commit HEAD
Good "git" signature for you@example.com with ED25519 key SHA256:IBY0fL0BtdFj0fbHvPhR5Di74E0PfNUAA+UYVYfgHgw
[exit status: 0]
$ git log -1 --format='%h %G? committed %cs %s'
8c7d0a6 G committed 2026-08-30 Rotate the staging credentials
```

A commit dated after the key's end is rejected. A commit made today with `GIT_COMMITTER_DATE` set inside the window is accepted. Validity windows let you rotate keys without invalidating old signatures, which is what the manual offers them for. They do not protect against a stolen key: the thief chooses the date. For that there is the revocation file:

```
$ cut -d' ' -f1,2 ~/.keys/you_2026.pub > ~/revoked_keys
$ git config set --global gpg.ssh.revocationFile ~/revoked_keys
```

```
$ git verify-commit HEAD
Could not verify signature.
[exit status: 1]
$ git log -1 --format='%h %G? %s'
8c7d0a6 B Rotate the staging credentials
```

A revoked key fails for every date, including all the honest signatures made before the theft.

Picture.

signed bytes (the commit without gpgsig)

+-----+
| tree -> every file, by hash |
| parent(s) -> all earlier history, by hash |
| author name <email> date |
| committer name <email> date |
| message |
+-----+
| ssh-keygen -Y sign -n git
v
gpgsig header (commit) or block after the message (tag)

not signed

+-----+
| refs: which branch or tag name |
| which repository, which fork |
| who pushed it, and when |
| reviews, CI results, approvals |
| whether the author line is true |
+-----+

A verified signature proves	It does not prove
The object has not changed since it was signed	That the change is correct, reviewed or harmless
Someone with access to the private key signed it	Which person that was, if the key was shared, stolen or left unlocked
The key is one that <i>you</i> (or your platform) have decided to trust for that principal	That the author and committer lines name the signer (section 14B.18)
	When it was signed: the dates are the signer's claim
	That it belongs on this branch or in this release

The Phase 0 report's example is the xz-utils backdoor of 2024: introduced by a contributor who had earned commit rights over two years, activated by a script that existed only in the release tarball (timeline). Provenance is not safety.

14B.18 Author spoofing, locally

In one sentence. Author and committer are text supplied by whoever creates the commit, so anyone can create a commit that names you, and a server running plain Git will accept it.

See it. Asha has one genuine commit in the shared repository. You add two more "by Asha", first with an option and then with two configuration values:

```
$ printf 'requests_per_minute: 6000\n' > config/limits.yaml
$ git commit -q -am "Raise the rate limit" --author="Asha Rao <asha@example.com>"
$ git log -1 --format='%h author=%an <ae> committer=%cn <ce> %s'
fee79f6 author=Asha Rao <asha@example.com> committer=Lab User <you@example.com> Raise the rate limit
```

```
$ printf 'requests_per_minute: 60000\n' > config/limits.yaml
$ git -c user.name="Asha Rao" -c user.email=asha@example.com commit -q -am "Raise the rate limit again"
$ git log -3 --format='%h author=%an <ae> committer=%cn <ce> %s'
6e10a38 author=Asha Rao <asha@example.com> committer=Asha Rao <asha@example.com> Raise the rate limit again
fee79f6 author=Asha Rao <asha@example.com> committer=Lab User <you@example.com> Raise the rate limit
39f0449 author=Asha Rao <asha@example.com> committer=Asha Rao <asha@example.com> Add health endpoint
```

`--author` changed one field and left you as committer, which is how a patch from someone else is honestly recorded (Chapter 6). The second commit claims both fields. Compared with her real commit, there is no field that tells them apart:

```
# Her real commit and your forgery, header by header (tree, parent and dates differ, as for any two commits):
$ git cat-file -p HEAD~2 | sed -n '/^author/,/^committer/p'
author Asha Rao <asha@example.com> 1788756000 +0530
committer Asha Rao <asha@example.com> 1788756000 +0530
$ git cat-file -p HEAD | sed -n '/^author/,/^committer/p'
author Asha Rao <asha@example.com> 1788756480 +0530
committer Asha Rao <asha@example.com> 1788756480 +0530
```

```
$ git push
To ../../server/inference-gateway.git
 39f0449..6e10a38 main -> main
$ git -C ../../server/inference-gateway.git log -3 --format='%h %an <ae> %s'
6e10a38 Asha Rao <asha@example.com> Raise the rate limit again
fee79f6 Asha Rao <asha@example.com> Raise the rate limit
39f0449 Asha Rao <asha@example.com> Add health endpoint
$ git shortlog -sne HEAD
 3 Asha Rao <asha@example.com>
 3 Lab User <you@example.com>
```

The push was accepted, and `git shortlog` credits Asha with three commits. Git has no rule that the pusher must be the committer; who pushed is known only to the server's logs.

With signatures. The same forgery in a repository where both of you sign and both keys are in the allowed-signers file (a volatile demo):

```
# You cannot sign as Asha: you do not have her private key. You can sign with your own.
$ printf 'requests_per_minute: 60000\n' > config/limits.yaml
$ git -c user.name="Asha Rao" -c user.email=asha@example.com commit -q -S -am "Raise the rate limit again"
$ git verify-commit HEAD
Good "git" signature for you@example.com with ED25519 key SHA256:XpH8NGLAagHBiC1XlKwmz7zMcGZ2uC4HE68dHGQP3zg
[exit status: 0]
$ git log -3 --format='%h %G? author=%ae signer=%GS %s'
4687f7f G author=asha@example.com signer=you@example.com Raise the rate limit again
bc6a046 N author=asha@example.com signer= Raise the rate limit
4306257 G author=asha@example.com signer=asha@example.com Add health endpoint
```

Three commits claim Asha as author. One is signed by her key. One is unsigned. One is signed, validly, by *your* key: `git verify-commit` exits 0, because Git verifies a signature against the list of trusted keys and does not compare the signer with the author line. That comparison is a policy, and you have to run it:

```
# Git verified a signature. It did not compare the signer with the author. A policy check does:
$ git log -3 --format='%h %G? %ae %GS' | awk '{ ok = ($2 == "G" && $3 == $4) ? "ok" : "FAIL"; print ok, $0 }'
FAIL 4687f7f G asha@example.com you@example.com
FAIL bc6a046 N asha@example.com
ok 4306257 G asha@example.com asha@example.com
```

`git merge --verify-signatures` is not that policy either. It checks one commit:

```
$ git switch -q -c integration main~2
$ git merge --verify-signatures --ff-only main~1
fatal: Commit bc6a046 does not have a GPG signature.
[exit status: 128]
# The option looks at one commit only: the tip of what is being merged.
$ git merge --verify-signatures --ff-only main
Commit 4687f7f has a good GPG signature by you@example.com
Updating 4306257..4687f7f
Fast-forward
 config/limits.yaml | 3 +--
 1 file changed, 1 insertion(+), 2 deletions(-)
[exit status: 0]
$ git log -3 --format='%h %G? author=%ae signer=%GS'
4687f7f G author=asha@example.com signer=you@example.com
bc6a046 N author=asha@example.com signer=
4306257 G author=asha@example.com signer=asha@example.com
```

The merge of an unsigned tip was refused. The merge of a signed tip was accepted, and it brought the unsigned commit underneath along: the manual says the option verifies "the tip commit of the side branch".

GitHub, not Git. GitHub runs its own verification against the keys registered on accounts and labels commits "Verified", "Partially verified" (in vigilant mode) or "Unverified"; enforcement is the "require signed commits" rule (Phase 0 report; [about commit signature verification](#)). Chapter 21B treats the platform side: how a signature is tied to an account, what vigilant mode adds for commits that name you as author, and a published demonstration of the spoofing shown here ([Gruntwork](#)).

In production. The CTO's fourth question. If the commit is unsigned, Git cannot say who made it; the honest answer is "anyone with push access". If it is signed, Git can say which key signed it, and the investigation moves to the server's push log and the key's owner. The report separates three questions: who is displayed (email matching), who signed (a key), and what is enforced (rules).

14B.19 What can go wrong

Symptom	Diagnosis	Fix	Prevention
The same command behaves differently on two machines	<code>git config get --all --show-scope --show-origin <key></code> on both; <code>git var</code> for editor, pager, identity	Set or unset the key in the scope that should own it	Keep local overrides rare and commented
A setting "does not take effect"	The key is misspelled, or a later scope or an environment variable overrides it	Correct the key; remove the override	Check keys against <code>git help --config</code> (Lab 5.4)
<code>fatal: bad config line N in file ...</code> on every command	File and line are in the message	Repair the line with a text editor	<code>git config set</code> , not hand edits
Work commits carry the personal address	<code>git config get --all --show-origin user.email</code> shows one value: the <code>includeIf</code> pattern lacks its slash, precedes <code>[user]</code> , or names a missing file	Fix the pattern; <code>git commit --amend --reset-author</code> for unpublished commits	<code>user.useConfigOnly=true</code> , no global default address

Symptom	Diagnosis	Fix	Prevention
An alias misbehaves with an argument	<code>GIT_TRACE=1</code> shows the expansion	No hard-coded revisions; wrap shell aliases in a function	Test each alias with and without arguments
<code>git describe</code> fails in CI	<code>git rev-parse --is-shallow-repository</code> ; <code>git tag</code> is empty	Fetch tags and full history in the versioning job	Fail the release job when <code>git describe --exact-match</code> fails
Two machines disagree about a tag	<code>git ls-remote origin 'refs/tags/<tag>^{'</code> against <code>git rev-parse '<tag>^{commit}'</code>	Restore the published tag; release a new version	A server rule that makes release tags immutable
A deleted tag keeps coming back	Some clone still has it and pushes with <code>--tags</code>	Delete it there	Push single tags or <code>--follow-tags</code>
<code>error: gpg failed to sign the data</code>	<code>git config get gpg.format</code> ; <code>user.signingKey</code> ; run the signing program by hand	Configure the format and key you mean	Test signing after every key or machine change
A signed commit shows <code>No signature</code>	<code>gpg.ssh.allowedSignersFile</code> is unset or the file is missing	Configure the file	Ship the file with the signing setup
Old commits turn from <code>G</code> to <code>U</code> after a key rotation	The old key was removed from the allowed-signers file	List both keys (Lab 24.1)	Rotation adds, never replaces
A signed history contains a commit its named author never made	Signer and author differ: <code>git log --format='%h %G? %ae %GS'</code>	Treat as an incident (Chapter 21B)	Enforce signatures <i>and</i> compare signer with author

14B.20 When not to use it, and dangerous edge cases

- **Configuration is not policy.** Nothing in a developer's configuration is enforced. Rules belong on the server and in CI.
- **No secrets in configuration.** A token in `remote.<name>.url` or in `http.extraHeader` is plain text, and `git config list` prints it into every bug report (Chapter 16).
- **! aliases, `core.sshCommand`, `core.pager`, `core.editor`, `core.hooksPath` and include paths execute or load things.** Whoever can write your configuration can run code as you. Never paste configuration you have not read.
- **`-c` values are visible in the process list of a shared machine.** `--config-env=<name>=<envvar>` reads the value from the environment instead (`git`).
- **Do not re-tag what has been published**, not even "before anyone noticed". A mirror, a cache or a build noticed.
- `git push --tags` **publishes everything**, including `wip-*` and `bisect` markers.
- **A lightweight tag is invisible to `git describe` and to `--follow-tags`.** Release tags are annotated.
- **A signature is evidence about a key.** A passphrase-less key on a laptop, a key copied into CI, or a shared team key reduce "signed by Asha" to "signed by something that had Asha's file".
- `commit.gpgSign=true` **without a working key blocks every commit**, including the one you need during an incident. Know `--no-gpg-sign`, and whether your server accepts the result.
- **Signatures do not survive rewrites.** A rebase, a squash or a history filter re-signs with the operator's key or with none (the Phase 0 report notes that `git-filter-repo` removes signatures).
- `git merge --verify-signatures` **checks only the tip.**

14B.21 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git config get, list</code> ; <code>git var</code> ; <code>git describe</code> ; <code>git verify-commit</code> ; <code>git verify-tag</code> ; <code>git tag -v</code>	🟢 SAFE	nothing	not needed	not needed
<code>git config set, unset, edit</code>	🟡 CAUTION	one configuration file, which has no history	<code>git config get --show-origin <key></code>	set the old value again; keep the global file under version control
<code>git tag <name></code> , <code>-a</code> , <code>-s</code> ; <code>git commit -S</code>	🟢 SAFE	adds a ref or an object	<code>git log -1 <commit></code>	<code>git tag -d <name></code> while unpublished
<code>git tag -f</code> , <code>git tag -d</code>	🟡 CAUTION	moves or deletes a local tag; no reflog	<code>git rev-parse <tag></code> , and write the ID down	<code>git tag <name> <old-id></code> ; <code>git fsck</code> lists a dangling tag object
<code>git push origin <tag></code> , <code>--follow-tags</code> , <code>-tags</code>	🟡 CAUTION	creates tags on the server	<code>git push --dry-run</code>	none that is clean: others may have fetched
<code>git fetch --tags --force</code>	🟡 CAUTION	overwrites local tags with the server's	<code>git fetch --tags --dry-run</code>	the old tag objects stay, dangling, until pruned
<code>git push --force origin <tag></code>	🔴 DANGEROUS	replaces a published tag	<code>git ls-remote --tags origin</code>	force the old tag object back from a clone that has it
<code>git push origin --delete <tag></code>	🔴 DANGEROUS	deletes a published tag	<code>git ls-remote --tags origin</code>	push it again from a clone that kept it

Command	Label	What it changes	Preview	Recovery
<code>git fetch --prune --prune-tags</code>	 DANGEROUS	deletes every local tag the remote lacks	add <code>--dry-run</code>	annotated tags: <code>git fsck</code> ; lightweight: only the IDs in the command's output

What the three  commands can destroy, and when they are appropriate:

- `git push --force origin <tag>` destroys the meaning of a release name for everyone who fetches later. Appropriate for putting a tag back at its *published* position (section 14B.11); a name that is meant to move should be a branch.
- `git push origin --delete <tag>` removes the only published name of a release commit, and the tag under a GitHub release. Appropriate for a tag pushed by mistake minutes ago, announced.
- `git fetch --prune --prune-tags` destroys private lightweight tags, which have no reflog and no object to find them by. Appropriate for a mirror or a CI cache that must equal the server.

14B.22 Version notes

Version note. Older behavior: `git config --get`, `--add`, `--unset`, `-l`. Current behavior: `git config get`, `set`, `unset`, `list`, `edit`; the old modes are deprecated and still work. Since: Git 2.46 ([release notes](#)); `git config list` was declared the official spelling of `-l` in 2.54 ([release notes](#)). Recommended: the subcommands, with a version check in scripts that run elsewhere.

Version note. Older behavior: `includeIf` conditions are `gitdir`, `gitdir/i`, `onbranch` and `hasconfig:remote.*.url`; `gitdir` exists since Git 2.13 (`git-config`). Current behavior: Git 2.56 adds a condition on the location of the working tree (not run here; [release notes](#)). Recommended: `gitdir:` with a trailing slash, which every Git since 2.13 understands.

Version note. Older behavior: signing meant GPG. Current behavior: `gpg.format` is `openpgp`, `x509` or `ssh`. Since: X.509 in Git 2.19, SSH in Git 2.34 ([gpg configuration](#)). Recommended: SSH signing where it is accepted; the option names keep "gpg" for all three.

Version note. Older behavior: `git init` creates `master`. Current behavior: still `master` on unconfigured Git 2.55, with a hint that Git 3.0 will use `main`; `init.defaultBranch` exists since Git 2.28 (Phase 0 report). Recommended: set it.

Version note. Older behavior: `help.autocorrect=1` waited a tenth of a second before running the guess. Current behavior: `1` means run immediately. Since: Git 2.49. Recommended: `prompt`, or the default.

Unverified. The minimum OpenSSH version for SSH signing was not established (Phase 0 report). The Git manual states only that `valid-after` and `valid-before` need OpenSSH 8.8; this machine runs 10.2.

14B.23 Practice

- **Module 5** (lab manual): 5.1 scopes and origins; 5.2 two identities with `includeIf`; 5.3 aliases; 5.4 your deliberate configuration, a worksheet.
- **Module 13** (lab manual): 13.1 three kinds of tag; 13.2 a moved tag between two clones; 13.3 describe and versions.
- **Module 24, local part** (lab manual): 24.1 SSH signing locally; 24.2 a spoofed-author commit.
- Replay any transcript with `labs/run ch14b/<demo>`. A drill: in the sandbox left by `ch14b/includes`, make the identity depend on the branch with `onbranch:`.

14B.24 Interview questions

1. A setting is in the system, global and local files with three different values, and an environment variable also exists for it. Which wins, and which single command shows you?
2. Your `includeIf` rule for `~/work/` exists, yet a work repository commits with your personal address. Give four causes and the command that distinguishes them.
3. Why does Git not read configuration from a file committed in the repository? What do teams do instead?
4. What exactly runs when you type `git x foo` and `alias.x` is `!cmd $1`? How do you write it correctly?
5. Describe the objects and refs created by `git tag v1`, `git tag -a v1` and `git tag -s v1`. What does `v1^{}` resolve to in each case?
6. A release tag was force-moved yesterday. Who has which version of it today, how do you find out, and what is the lowest-risk repair?
7. `receive.denyNonFastForwards` is set on our Git server. Are our tags safe? How would you prove your answer?
8. `git describe` prints `v1.0.0-2-gc053f0d` on a laptop and fails in CI. Explain every part of the string and the failure.

9. What bytes does a commit signature cover? Name three things about a commit that it does not establish.
10. All commits on `main` show a good signature. Does that mean each was written by the person in its author field? What check is missing?
11. We rotate signing keys yearly. What happens to last year's signatures, and why do validity dates not help when a key is stolen?
12. Which Git settings would you standardize across a team, and which would you leave to each engineer?

14B.25 Sources

Primary sources

- [git-config](#), [git](#), [git-var](#), [gitcredentials](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text the transcripts were checked against.
- [git-tag](#) (including "On Re-tagging"), [git-describe](#), [git-for-each-ref](#), [git-check-ref-format](#), [git-push](#), [git-fetch](#), [gitworkflows](#), [githooks](#).
- [gitformat-signature](#), [git-verify-commit](#), [git-verify-tag](#), [git-log](#) (pretty formats), [git-merge](#), the [gpg configuration reference](#), and [ssh-keygen\(1\)](#), "ALLOWED SIGNERS".
- Git source at 2.55.0, read for behavior the manual does not state: [receive-pack.c](#) (the `receive.deny*` settings apply to `refs/heads/` only), [http.c](#) (`GIT_CURL_VERBOSE`), [editor.c](#) (`VISUAL` and dumb terminals).
- Release notes [2.34](#), [2.45](#), [2.46](#), [2.49](#), [2.54](#), [2.56](#).
- [Semantic Versioning 2.0.0](#).
- GitHub Docs: [about commit signature verification](#), [telling Git about your signing key](#), [about releases](#), [associating text editors with Git](#).

Secondary sources

- Pro Git: [Git Configuration](#), [Git Aliases](#), [Tagging](#), [Signing Your Work](#), [Environment Variables](#). Caveats: legacy `git config` syntax, GPG only, `master`.
- Julia Evans, [Popular git config options](#) (2024): a crowd-sourced list that overlaps the table in section 14B.5.
- Taylor Blau, [Highlights from Git 2.34](#): the introduction of SSH signing.
- Russ Cox, [the xz attack timeline](#); Gruntwork, [How to spoof any user on GitHub](#).

Videos (optional; assessments in the Phase 0 report rest on captions and chapter lists, not on full viewing)

- [So You Think You Know Git - FOSDEM 2024](#), Scott Chacon, 47 minutes: `includeIf` at 08:23, SSH signing and the "Verified" badge from 19:00. Caveats: a survey, not a deep dive; about one minute of product pitch.
- [Simplify signing Git commits and tags with SSH keys](#), Andy Feller, Git Merge 2022 workshop, 56 minutes. Caveat: assessed from automatic captions and the description.
- [Tag, You're Leaked: Surviving the tj-actions Supply Chain Attack](#), BSides PDX 2025, 24 minutes: what a moved tag costs at scale.

Further reading

- The "DISCUSSION" section of `git-tag(1)`: three short essays on re-tagging, tag following and backdating.

Chapter 14C: Stash Internals, Rerere, Attributes, Hooks

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch14c/`. One demo (`rerere-lock-race`) measures a timing race, so its numbers differ from run to run; it is marked where it appears.

14C.1 Why this matters

Four questions a CTO can ask, one for each part of this chapter:

1. "An engineer popped a stash and says the careful staging is gone. Was anything lost?"
2. "The team switched on rerere because one conflict came back at every rebase. Last week a wrong value reached `main` without anyone typing it. How?"
3. "We committed a `.gitattributes` that strips notebook outputs. A new hire's first commit contains 40 MB of outputs. Why did the rule not apply to her?"
4. "We have a pre-commit hook that blocks secrets. A secret is on `main`. Whose fault is that?"

The four topics share one design decision. A repository holds content that is versioned and travels with `clone`, `fetch` and `push` (commits and the files in them, such as `.gitattributes`), and machinery that is local and never travels (`.git/config`, `.git/hooks`, `.git/rr-cache`, `refs/stash`). Everything in this chapter sits on the local side of that line, or straddles it.

The short answers:

1. Nothing was lost. A stash entry is a commit with two or three parents, and the second parent holds the index as it was. A plain `git stash pop` ignores it (section 14C.2).
2. Rerere replays a recorded resolution whenever the same conflict text appears, a wrong one as faithfully as a right one, and with `rerere.autoUpdate` it also stages the result (section 14C.3).
3. `.gitattributes` names a filter. The filter is defined in configuration, and configuration is not cloned. Her clone had the name without the definition, and Git treats that as "no filter" without a warning (sections 14C.4 and 14C.8).
4. Nobody's, as far as Git is concerned. A client-side hook is a file in one clone that one option skips. Policy is enforced where every push arrives: on the server and in CI (sections 14C.11 to 14C.13).

Chapter 11 (Reset, Revert, Restore) teaches the stash commands, Chapter 8 (Merge) conflicts, and Chapter 4 (Working Tree) introduces line endings.

14C.2 Stash internals: a stash entry is a small commit graph

In one sentence. A stash entry is a commit `W` that records your tracked files, with the commit that was HEAD as its first parent, a commit `I` that records the index as its second parent and, when you stash untracked files, a parentless commit `U` as its third; `refs/stash` points at the newest `W`, and the stash list is the reflog of that ref.

Analogy. A labelled envelope with two or three sheets: a photocopy of the bench (the files), a photocopy of the outbox (the index), and the loose notes that were lying around (untracked files). The analogy breaks in one place: there is no drawer for the envelopes. They are ordinary commits in the object database, held by one ref and its log, which is why a dropped stash can be found again.

Precisely. The stash manual gives the shape: `W` is "a commit whose tree records the state of the working directory", its first parent is the commit at HEAD, and "the tree of the second parent records the state of the index" ([git-stash](#)). The manual does not name the third parent; `git stash show --include-untracked` and `--only-untracked` read it. Consequences of the shape:

- `git stash show -p` is the diff from `H` to `W`: the staged and the unstaged part appear as one change.
- `git stash apply` merges `W` into your files with `H` as the base (Chapter 11). Only `--index` also reads `I`.
- Revision syntax works on an entry: `stash@{1}^2` is the index commit of the second entry, `stash@{0}:config.yaml` one file of the newest. `stash@{n}` is reflog syntax; there is one ref.

Inside `.git`. New objects for `W`, `I` and (with `-u` or `-a`) `U`; the ref `refs/stash`; the log `logs/refs/stash`, one line per entry.

See it. A repository with one staged change (`score.py`), one unstaged change (`config.yaml`) and one untracked file (`notes.md`) was stashed with `git stash push -u`. The graph of the entry, with the parents of every commit in brackets:

```
# Each line: commit, [its parents], subject.
$ git log --graph --format='%h [%p] %s' 'stash@{0}'
*-- 962e38f [8f8672d 7ee160e 8fb87d5] 0n main: wip: strip whitespace
|\ \
| | * 8fb87d5 [] untracked files on main: 8f8672d Add scorer and config
| * 7ee160e [8f8672d] index on main: 8f8672d Add scorer and config
|/
* 8f8672d [] Add scorer and config
```

962e38f is W, a merge commit with three parents. 8f8672d is the commit you were on. 7ee160e is I, a child of that commit. 8fb87d5 is U: its bracket is empty, so it has no parent. The raw commit shows the same:

```
$ git show -s --format=raw 'stash@{0}'
commit 962e38fce2cce7e6ec1b3897921414fceb38409b
tree 36051caa44c7daac943f640b3d43f0a50a135455
parent 8f8672db13ec116a0e66b597dde4c3f41fdc2dd2
parent 7ee160e569bbb323ff60b5e84b8f54b9ef16013
parent 8fb87d55587295bedf701e94a8d5a9d1de921a08
author Lab User <you@example.com> 1788755880 +0530
committer Lab User <you@example.com> 1788755880 +0530
```

On main: wip: strip whitespace

Now the four trees. Compare the blob IDs column by column:

```
# H, the commit that was HEAD:
$ git ls-tree -r 'stash@{0}'^1
100644 blob 464dad17678e625818cf5f96c2e878a8b8e21647    config.yaml
100644 blob 2c9e3f43f09999102c43be95c5d1f3ef1d6f6bc0    score.py
# I, the index as it was (second parent):
$ git ls-tree -r 'stash@{0}'^2
100644 blob 464dad17678e625818cf5f96c2e878a8b8e21647    config.yaml
100644 blob 6ba285dc88264166ccae82826a80ecacdd417104    score.py
# W, the tracked files as they were on disk (the stash commit itself):
$ git ls-tree -r 'stash@{0}'
100644 blob b571eab8daac6f68438fc7580829f656c4427932    config.yaml
100644 blob 6ba285dc88264166ccae82826a80ecacdd417104    score.py
# U, the untracked files (third parent, only with -u or -a):
$ git ls-tree -r 'stash@{0}'^3
100644 blob 8b2489b1aa1ff7ba005bedb45f156e8dd99f77f9    notes.md
```

score.py changes between H and I (2c9e3f4 becomes 6ba285d): the staged change. config.yaml changes between I and W (464dad1 becomes b571eab): the unstaged one. U holds only the untracked file. No tree mixes the three kinds of work, so they can be taken apart again.

```
# What "git stash show -p" prints is the difference between H and W:
$ git stash show -p
diff --git a/config.yaml b/config.yaml
index 464dad1..b571eab 100644
--- a/config.yaml
+++ b/config.yaml
@@ -1,1 @@
-threshold: 0.5
+threshold: 0.7
diff --git a/score.py b/score.py
index 2c9e3f4..6ba285d 100644
--- a/score.py
+++ b/score.py
@@ -1,2 +1,2 @@
 def score(pred, gold):
-    return pred == gold
+    return pred.strip() == gold.strip()
# The staged part is H against I, the unstaged part is I against W:
$ git diff --stat 'stash@{0}'^1 'stash@{0}'^2
score.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git diff --stat 'stash@{0}'^2 'stash@{0}'
config.yaml | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git stash show --only-untracked
notes.md | 1 +
1 file changed, 1 insertion(+)
```

git stash show -p prints both tracked changes together. The two git diff commands separate them, because each compares the right pair of commits.

Picture.

```

      8fb87d5 U   untracked files (no parent)
           \
           -----962e38f W   <- refs/stash   tracked files as on disk
          /
         /
8f8672d H -----7ee160e I   the index as it was
(main, HEAD)
```

The ref and its log. A second entry is pushed, and then the two files that make up the stash list are printed:

```
$ printf 'threshold: 0.9\n' > config.yaml
$ git stash push -m "wip: threshold 0.9"
Saved working directory and index state On main: wip: threshold 0.9
$ git stash list
stash@{0}: On main: wip: threshold 0.9
stash@{1}: On main: wip: strip whitespace
$ cat .git/refs/stash
```

```
e5fe33887289800d1015fbecba39e952ff467eb9
$ cat .git/logs/refs/stash
0000000000000000000000000000000000 962e38fce2cce7e6ec1b3897921414fceb38409b Lab User <you@example.com> 1788755880 +0530
On main: wip: strip whitespace
962e38fce2cce7e6ec1b3897921414fceb38409b e5fe33887289800d1015fbecba39e952ff467eb9 Lab User <you@example.com> 1788756720 +0530
On main: wip: threshold 0.9
$ git rev-parse 'stash@{0}' 'stash@{1}'
e5fe33887289800d1015fbecba39e952ff467eb9
962e38fce2cce7e6ec1b3897921414fceb38409b
```

The ref holds the ID of the newest entry. The log has one line per entry, oldest first, in the reflog format of Chapter 3. `git stash list` prints the message column of this file, which is not necessarily the subject of the commit, as the next transcript shows.

Building an entry without touching anything. Two subcommands split `push` into its halves. Scripts use them, and so does Git: the autostash of `git rebase --autostash` is a stash commit that is never put on the list (Chapter 9, section 9.8).

```
$ printf 'threshold: 0.8\n' > config.yaml
# "create" builds the commits and prints the ID. No ref is written and the files stay as they are.
$ git stash create "wip: threshold 0.8"
4bd62aa12a2c6896b596b2b7e718393ed816ccb1
$ git stash list
stash@{0}: On main: wip: threshold 0.9
stash@{1}: On main: wip: strip whitespace
$ git status -s
M config.yaml
# "store" puts an existing stash commit on the list:
$ git stash store -m 'stored by hand' 4bd62aa
$ git stash list
stash@{0}: stored by hand
stash@{1}: On main: wip: threshold 0.9
stash@{2}: On main: wip: strip whitespace
```

`git stash create` wrote the commits and printed the ID of `W`; the list did not change and the file is still modified. `git stash store` appended a reflog line with your message. The commit's own subject is still `On main: wip: threshold 0.8`.

What drop does.

```
$ git stash drop 'stash@{1}'
Dropped stash@{1} (e5fe33887289800d1015fbecba39e952ff467eb9)
$ git stash list
stash@{0}: stored by hand
stash@{1}: On main: wip: strip whitespace
$ cat .git/logs/refs/stash
0000000000000000000000000000000000 962e38fce2cce7e6ec1b3897921414fceb38409b Lab User <you@example.com> 1788755880 +0530
On main: wip: strip whitespace
962e38fce2cce7e6ec1b3897921414fceb38409b 4bd62aa12a2c6896b596b2b7e718393ed816ccb1 Lab User <you@example.com> 1788757260 +0530
stored by hand
# The dropped commit still exists as an unreachable object:
$ git cat-file -t e5fe338
commit
```

Dropping the middle entry deleted its line from the log and rewrote the next line to follow the entry before it. No object was deleted: `e5fe338` is still a commit, unreachable now, until unreachable objects are pruned (two weeks by default; Chapter 13: Recovery). Lab 8.6 recovers such an entry.

Stashes that travel. Nothing fetches or pushes `refs/stash`. Since Git 2.51, `git stash export` writes all entries into one chain of ordinary commits that can be pushed like a branch, and `git stash import` turns such a chain back into entries:

```
$ git stash export --to-ref refs/stashes/laptop
$ git log --graph --format='%h %s' refs/stashes/laptop
* 14ab5f1 git stash: On main: wip: threshold 0.8
|\
| * 4bd62aa On main: wip: threshold 0.8
| \
| | * b01d0a0 index on main: 8f8672d Add scorer and config
| | /
| |
* | 1f87106 git stash: On main: wip: strip whitespace
|\ \
| | \
| | \
| | \
| *-. \ 962e38f On main: wip: strip whitespace
| \ \ \
| | \_/_/
| |/_/_/
| |/_/_/
| | * 8fb87d5 untracked files on main: 8f8672d Add scorer and config
| | * 7ee160e index on main: 8f8672d Add scorer and config
| | /
| |
| * 8f8672d Add scorer and config
* 73c9bab
```

Each `git stash: ...` commit has two parents: the previous commit of the chain and one stash commit. After a push of that ref and a fetch elsewhere, `git stash import <commit>` rebuilds the list; the demo `ch14c/stash-anatomy` runs it. Imported entries show their commit subjects, because messages such as `stored by hand` exist only in the reflog of the repository that wrote them.

State table.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
 <code>git stash push [-u]</code>	tracked files reset to HEAD; with <code>-u</code> untracked files removed	reset to HEAD	unchanged	unchanged	objects for <code>W, I, (U)</code> ; <code>refs/stash</code> moved; one line added to <code>logs/refs/stash</code> ; <code>ORIG_HEAD</code> rewritten	unchanged	unchanged
 <code>git stash create</code> , then <code>store <commit></code>	unchanged	unchanged	unchanged	unchanged	objects; then <code>refs/stash</code> moved and one log line added	unchanged	unchanged
 <code>git stash drop</code>	unchanged	unchanged	unchanged	unchanged	one log line removed; <code>refs/stash</code> moved or deleted; objects stay, unreachable	unchanged	unchanged

In production. When the staging mattered, bring work back with `git stash pop --index`; when you forgot, restore the index from the second parent of the dropped commit, whose ID `pop` printed (Lab 14.6). When you need one file from an entry, read it with `git show 'stash@{0}:path'` or `git restore --source='stash@{0}' -- path` and leave the entry alone.

14C.3 Rerere: resolve a conflict once

In one sentence. With `rerere.enabled` set, Git records the text of every conflict together with what you made of it, and when a conflict with the same text appears again, in any merge, rebase, cherry-pick, revert or `git am -3`, it writes your earlier resolution into the file instead of the conflict markers.

Analogy. A notebook next to the merge desk, each page with a conflict on the left and your answer on the right. When the same left side turns up, Git copies the right side into the file. The analogy breaks where the danger is: the notebook matches text, not intent, and never asks before it copies.

Precisely. "Rerere" stands for "reuse recorded resolution" ([git-rerere](#)). `git merge`, `git rebase`, `git commit` and the other commands that can stop on a conflict call `git rerere` for you; you rarely type it. It works in two moments:

1. **When a conflict appears.** Git normalizes the conflict in each file (labels stripped from the markers, the two sides sorted, the base part of `diff3` output removed), hashes the conflict hunks, and uses the hash as the name of a directory in `.git/rr-cache`. Without a recorded resolution there, it stores the normalized conflict as `preimage` and prints `Recorded preimage`. With one, it merges the old conflict, the old resolution and the new conflict and, when that merge is clean, writes the result into your file and prints `Resolved ... using previous resolution`.
2. **When you conclude.** At the commit (or `git rebase --continue`), Git stores the resolved file as `postimage` and prints `Recorded resolution`.

The normalization is why one record serves both directions of a merge, and a merge as well as a rebase: the name depends on the two competing texts, not on branch names, on who is "ours", or on the path ([rerere technical notes](#)). Rerere leaves the index alone: a replayed file stays unmerged until you `git add` it, unless `rerere.autoUpdate` is true.

Inside .git. `rr-cache/<conflict-id>/preimage` and `postimage` (and at times a scratch file `thisimage`); `MERGE_RR`, which maps the conflicted paths of the operation in progress to conflict IDs; and, briefly, `MERGE_RR.lock`. All of it is local: `rr-cache` is not pushed, fetched or cloned. `git rerere gc` prunes unresolved records after 15 days and resolved ones after 60 (`gc.rerereUnresolved`, `gc.rerereResolved`).

See it: a repeated merge. `feature/rerank` is a long-lived branch. It and `main` changed the same two lines of `retrieval.yaml`. You want to test the branch against the newest `main` today and merge it for real next week, without a test merge in its history: the workflow the rerere manual describes.

```
$ git log --oneline --graph --all
* 96196ef Raise top_k to 20
| * 62e40dc Add reranker module
| * c9ca15a Enable reranking over the top 50 candidates
|/
* d9ec3c5 Add retrieval config
$ git config set rerere.enabled true
$ git switch -q feature/rerank
$ git merge main
Auto-merging retrieval.yaml
CONFLICT (content): Merge conflict in retrieval.yaml
Recorded preimage for 'retrieval.yaml'
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
```

The new line is `Recorded preimage for 'retrieval.yaml'`. What was recorded, next to what is in your file:

```
$ ls .git/rr-cache
ec4b2990356a9f0ab0af396e6ce888496e91b4c7
$ cat .git/rr-cache/*/preimage
model: bge-small
```

```

<<<<<<
top_k: 20
rerank: false
=====
top_k: 50
rerank: true
>>>>>>
$ cat retrieval.yaml
model: bge-small
<<<<<< HEAD
top_k: 50
rerank: true
=====
top_k: 20
rerank: false
>>>>>> main
$ git rerere status
retrieval.yaml

```

The preimage has bare markers and the two sides in sorted order (`top_k: 20` first), although your file shows `HEAD` first. The directory name is the conflict ID. Resolve as usual; `git rerere diff` compares the recorded conflict with your file:

```

$ printf 'model: bge-small\ntop_k: 50\nrerank: true\n' > retrieval.yaml
$ git rerere diff
--- a/retrieval.yaml
+++ b/retrieval.yaml
@@ -1,8 +1,3 @@
 model: bge-small
-<<<<<<
-top_k: 20
-rerank: false
-=====
 top_k: 50
 rerank: true
->>>>>>
$ git commit -am "Test merge of main"
Recorded resolution for 'retrieval.yaml'.
[feature/rerank 047a898] Test merge of main
$ ls .git/rr-cache/*
postimage
preimage
$ cat .git/rr-cache/*/postimage
model: bge-small
top_k: 50
rerank: true

```

The commit printed `Recorded resolution`, and the directory now has both halves. The test merge has done its job and is removed:

```

# The test merge served its purpose. Remove it; the recorded resolution stays.
$ git reset --hard HEAD^
HEAD is now at 62e40dc Add reranker module
$ ls .git/rr-cache/*
postimage
preimage

```

A week later the branch is merged into `main`, in the opposite direction from the test:

```

$ git switch -q main
$ git merge feature/rerank
Auto-merging retrieval.yaml
CONFLICT (content): Merge conflict in retrieval.yaml
Resolved 'retrieval.yaml' using previous resolution.
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]

```

Read this output with care. The merge still reports `CONFLICT`, still says `Automatic merge failed`, and still exits with status 1. Only the line `Resolved 'retrieval.yaml' using previous resolution.` tells you that the file no longer contains markers:

```

$ git status -s
A  rerank.py
UU retrieval.yaml
$ cat retrieval.yaml
model: bge-small
top_k: 50
rerank: true
$ git rerere remaining
$ git ls-files -u
100644 fa370c130d79ca732a1fe0c9a0f8d556609b92b9 1      retrieval.yaml
100644 7ac2dd090cea0e852d25ee65af7e485b7a90e698 2      retrieval.yaml
100644 820d682527a387ff7f594053d7bb1a01c230cf715 3      retrieval.yaml

```

The file holds your resolution. `git status` says `UU` and the index still has three stages, because `rerere` wrote the file and nothing else. `git rerere remaining` prints nothing: no path is left to resolve by hand. The stop is your review point. Read the file, run the tests, then stage and conclude:

```
$ git add retrieval.yaml
$ git commit --no-edit
[main cad2174] Merge branch 'feature/rerank'
$ git log --oneline --graph
*   cad2174 Merge branch 'feature/rerank'
| \
|  * 62e40dc Add reranker module
|  * c9ca15a Enable reranking over the top 50 candidates
* | 96196ef Raise top_k to 20
|/
* d9ec3c5 Add retrieval config
```

See it: a rebase. The same record, made in a merge, serves a rebase of the branch onto `main`. In a rebase the sides are swapped (Chapter 9, section 9.11), and the normalization makes that irrelevant:

```
$ git switch -q feature/rerank
$ git rebase main
Rebasing (1/2)
Auto-merging retrieval.yaml
CONFLICT (content): Merge conflict in retrieval.yaml
error: could not apply c9ca15a... Enable reranking over the top 50 candidates
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
hint: Disable this message with "git config set advice.mergeConflict false"
Resolved 'retrieval.yaml' using previous resolution.
Could not apply c9ca15a... # Enable reranking over the top 50 candidates
[exit status: 1]
```

```
$ git status -s
UU retrieval.yaml
$ cat retrieval.yaml
model: bge-small
top_k: 50
rerank: true
$ git add retrieval.yaml
$ git rebase --continue
[detached HEAD 74fea46] Enable reranking over the top 50 candidates
1 file changed, 2 insertions(+), 2 deletions(-)
Rebasing (2/2)
Successfully rebased and updated refs/heads/feature/rerank.
$ git log --oneline --graph --all
* 4dd76a7 Add reranker module
* 74fea46 Enable reranking over the top 50 candidates
* 96196ef Raise top_k to 20
* d9ec3c5 Add retrieval config
```

With `rerere.autoUpdate` set, the replayed file is also staged. The rebase still stops, and only `git rebase --continue` is left to do:

```
$ git config set rerere.autoUpdate true
$ git -c advice.mergeConflict=false rebase main
Rebasing (1/2)
Auto-merging retrieval.yaml
CONFLICT (content): Merge conflict in retrieval.yaml
error: could not apply c9ca15a... Enable reranking over the top 50 candidates
Staged 'retrieval.yaml' using previous resolution.
Could not apply c9ca15a... # Enable reranking over the top 50 candidates
[exit status: 1]
$ git status -s
M retrieval.yaml
$ git rebase --continue
[detached HEAD f0e118a] Enable reranking over the top 50 candidates
1 file changed, 2 insertions(+), 2 deletions(-)
Rebasing (2/2)
Successfully rebased and updated refs/heads/feature/rerank.
```

Picture.

conflict appears	you resolve and commit	the same conflict again
file: <<<<<<<< ... >>>>>>	file: your resolution	file: <<<<<<<< ... >>>>>>
normalize, hash		normalize, hash: same <id>
v	v	v
rr-cache/<id>/preimage	rr-cache/<id>/postimage	3-way merge of preimage, postimage, new conflict
		v
		file: your resolution (index still unmerged)

State table.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
A conflict that rerere has not seen	conflict markers, as usual	stages 1 to 3	unchanged	unchanged	rr-cache/<id>/preimage, MERGE_RR	unchanged	unchanged
Concluding commit or <code>--continue</code>	unchanged	resolved	new commit	moved	rr-cache/<id>/postimage	unchanged	unchanged
A conflict that rerere knows	your recorded resolution	stages 1 to 3 (stage 0 with <code>rerere.autoUpdate</code>)	unchanged	unchanged	MERGE_RR	unchanged	unchanged
🟡 <code>git rerere forget <path></code>	unchanged	unchanged	unchanged	unchanged	postimage deleted, preimage replaced by the current conflict	unchanged	unchanged

The risk: a wrong resolution is replayed as faithfully as a right one

Suppose the test merge had been resolved in a hurry: `top_k` kept at 20 from `main`, `rerank: true` from the branch. Each line is valid, and together they starve the reranker of candidates. Git cannot know that, and the real merge replays the record:

```
$ cat .git/rr-cache/*/postimage
model: bge-small
top_k: 20
rerank: true
$ git merge feature/rerank
Auto-merging retrieval.yaml
CONFLICT (content): Merge conflict in retrieval.yaml
Resolved 'retrieval.yaml' using previous resolution.
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ cat retrieval.yaml
model: bge-small
top_k: 20
rerank: true
```

Apart from the file content, the output is identical to the correct case. With `rerere.autoUpdate` and a rebase, the wrong value is one git rebase `--continue` away from being a commit (Lab 14.5 repairs that case). The correction has three steps: make rerere forget, bring the conflict back, resolve again.

```
$ git rerere forget retrieval.yaml
Updated preimage for 'retrieval.yaml'
Forgot resolution for 'retrieval.yaml'
$ ls .git/rr-cache/*
preimage
thisimage
$ git restore --merge retrieval.yaml
$ cat retrieval.yaml
model: bge-small
<<<<<< ours
top_k: 20
rerank: false
=====
top_k: 50
rerank: true
>>>>>> theirs
```

`git rerere forget <path>` deletes the recorded resolution for the conflict in that path, and 🟡 `git restore --merge <path>` (older: `git checkout --merge <path>`) overwrites the file with the conflict rebuilt from the index. The new resolution is recorded at the commit:

```
$ printf 'model: bge-small\ntop_k: 50\nrerank: true\n' > retrieval.yaml
$ git add retrieval.yaml
$ git commit --no-edit
Recorded resolution for 'retrieval.yaml'.
[main f22692f] Merge branch 'feature/rerank'
$ cat .git/rr-cache/*/postimage
model: bge-small
top_k: 50
rerank: true
```

If the wrong merge is already committed and not pushed, undo it first (`git reset --keep HEAD^`, Chapter 11), merge again, and then forget. If it is pushed, fix forward with a new commit and still forget, or the next rebase brings the mistake back.

The lock race between rerere and automatic maintenance

While this chapter was written, replays of the rebase transcripts failed now and then with a message that has nothing to do with conflicts. Two correct features collide:

```
Observed behavior : "git rebase --continue" with rerere enabled sometimes ends at the next conflicting commit with
"fatal: Unable to create '.../.git/MERGE_RR.lock': File exists." (exit status 128).
```

Git state	: the rebase is stopped at that commit with conflict markers; rerere has neither recorded the conflict nor replayed a resolution for it.
Mechanism	: each commit of a rebase starts "git maintenance run --auto" in the background. Since Git 2.54 its default strategy includes the task rerere-gc, which runs whenever rr-cache has an entry (maintenance.rerere-gc.auto defaults to 1) and takes MERGE_RR.lock while it works.
Root cause	: the background task and the next step of the rebase want the same lock at the same moment.
Why Git does this	: the lock protects MERGE_RR from two writers; pruning rr-cache became a maintenance task.
Correct fix	: nothing is damaged. Run "git rerere", resolve, "git add", "git rebase --continue".
Prevention	: "git config set maintenance.rerere-gc.auto 0" where you rebase with rerere, and an occasional "git rerere gc" by hand.

The failure and the recovery on demand: an `exec` line of the rebase creates the lock file after each replayed commit, as a stand-in for the background task.

```
# Stand-in for the background task: after each replayed commit, hold the lock that rerere needs.
$ git -c advice.mergeConflict=false rebase -x 'touch .git/MERGE_RR.lock' main
Rebasing (1/6)
Auto-merging retrieval.yaml
CONFLICT (content): Merge conflict in retrieval.yaml
error: could not apply 4c042d7... Enable reranking over the top 50 candidates
Recorded preimage for 'retrieval.yaml'
Could not apply 4c042d7... # Enable reranking over the top 50 candidates
[exit status: 1]
$ printf 'model: bge-small\ntop_k: 50\nrerank: true\n' > retrieval.yaml && git add retrieval.yaml
$ git -c advice.mergeConflict=false rebase --continue
Recorded resolution for 'retrieval.yaml'.
[detached HEAD 4f7e530] Enable reranking over the top 50 candidates
1 file changed, 2 insertions(+), 2 deletions(-)
Rebasing (2/6)
Executing: touch .git/MERGE_RR.lock
Rebasing (3/6)
Auto-merging scoring.py
CONFLICT (content): Merge conflict in scoring.py
error: could not apply 27f7ed2... Blend dense scores into the ranking
fatal: Unable to create '$LAB/ch14c/rerere-lock/ranker/.git/MERGE_RR.lock': File exists.

Another git process seems to be running in this repository, or the lock file may be stale
[exit status: 128]
```

```
$ git status -s
UU scoring.py
# The real background task removes its lock when it ends. Remove the stand-in by hand:
$ rm .git/MERGE_RR.lock
# rerere never saw this conflict. Run it yourself:
$ git rerere status
$ git rerere
Recorded preimage for 'scoring.py'
$ git rerere status
scoring.py
```

After the failure `git rerere status` prints nothing: rerere never saw this conflict, so a resolution made now would not be recorded. One manual `git rerere` repairs that. How often does the real race occur? The demo `rerere-lock-race` runs the same rebase twenty times per configuration:

```
# Twenty rebases with two conflicting commits each, per configuration (labs/ch14c/rerere-lock-race.sh):
default: MERGE_RR.lock failures in 20 rebases: 1
no-rerere-gc: MERGE_RR.lock failures in 20 rebases: 0
```

This snippet is volatile: the first number depends on timing and varied between 1 and 10 in the runs made for this chapter; the second was always 0. Every rerere repository in this chapter and in Lab 14.5 therefore sets `maintenance.rerere-gc.auto=0` (`labs/ch14c/fixtures.bash`).

Unverified. The race was observed on Git 2.55.0 on macOS only. No manual or release note read for this course describes it, and Git 2.56 was not tested. The mechanism is inferred from the [maintenance configuration](#) and the measurement above.

In production. Turn rerere on for people who rebase long branches or rebuild integration branches. Leave `rerere.autoupdate` off unless tests run before every `--continue`: the stop with `UU` is the only review a replay gets. Know the limits: records are per clone, they expire, and a conflict whose text has changed is a new conflict. To teach rerere from merges that were resolved before it was switched on, the Git distribution ships [contrib/rerere-train.sh](#), which replays old merge commits; it is not installed as a command.

14C.4 `.gitattributes`: per-path settings that travel, drivers that do not

In one sentence. A `.gitattributes` file attaches named attributes to paths, and Git consults them whenever it converts, compares, merges or archives a file; the file is versioned and reaches every clone, while the programs that some attributes name are defined in configuration and reach nobody.

Analogy. Labels on folders in a shared filing cabinet: "text", "do not merge", "show through viewer X". Everyone sees the labels. Whether viewer X exists is a matter of each reader's own desk, and here the analogy breaks: a missing viewer produces no complaint.

Precisely. Each line is a pattern followed by attributes ([gitattributes](#)). For one path, an attribute is in one of four states:

Written as	State	git check-attr prints
text	set	set
-text	unset	unset
eol=lf	set to a value	the value
not mentioned, or !text	unspecified	unspecified

Patterns follow the rules of `.gitignore` with two exceptions: negative patterns are forbidden, and a pattern that matches a directory does not apply to the files inside it (write `docs/**` to reach them). Within one file a later line overrides an earlier one, per attribute. Between files the order of precedence is:

1. `.git/info/attributes` (local to this clone, not versioned);
2. the `.gitattributes` in the directory of the path, then those of its parent directories up to the top of the working tree, the nearer the stronger;
3. the file named by `core.attributesFile` (default `$XDG_CONFIG_HOME/git/attributes` or `~/.config/git/attributes`);
4. the system-wide file.

A macro attribute sets several others at once; the one built-in macro is `binary`, defined as `-diff -merge -text`.

Inside `.git`. Nothing, unless you use `.git/info/attributes`. A `.gitattributes` file is an ordinary tracked file. The drivers it may name live in `.git/config` or in a global configuration file.

See it. A training repository with two attribute files:

```
$ cat .gitattributes
# pattern      attributes
*              text=auto
*.sh           text eol=lf
*.csv          text eol=crlf
*.bin          binary
CHANGELOG.md   merge=union
docs           export-ignore
.gitattributes export-ignore
$ cat data/.gitattributes
# Vendor exports: keep every byte as delivered.
*.csv -text
```

`git check-attr` answers "which attributes does Git see for this path", the first diagnostic for every problem in the next four sections:

```
$ git check-attr -a -- src/train.py scripts/run.sh data/encoder.bin
src/train.py: text: auto
scripts/run.sh: text: set
scripts/run.sh: eol: lf
data/encoder.bin: binary: set
data/encoder.bin: diff: unset
data/encoder.bin: merge: unset
data/encoder.bin: text: unset
# Ask for named attributes to see the unspecified state as well:
$ git check-attr text eol merge -- src/train.py
src/train.py: text: auto
src/train.py: eol: unspecified
src/train.py: merge: unspecified
```

`data/encoder.bin` matched `*` and `*.bin`: the later line won for `text`, and the macro `binary` expanded into three unset attributes.

```
# The same pattern, one file at the top and one below data/:
$ git check-attr text eol -- labels.csv data/labels.csv
labels.csv: text: set
labels.csv: eol: crlf
data/labels.csv: text: unset
data/labels.csv: eol: crlf
# .git/info/attributes is not versioned and beats every .gitattributes file:
$ printf 'data/labels.csv text eol=lf\n' > .git/info/attributes
$ git check-attr text eol -- data/labels.csv
data/labels.csv: text: set
data/labels.csv: eol: lf
$ rm .git/info/attributes
```

For `data/labels.csv` the file in `data/` unset `text`, and `eol` still comes from the top-level file: precedence is decided attribute by attribute. Then `.git/info/attributes` overrode both.

`export-ignore` concerns none of the conversions: it keeps a path out of `git archive`, for what belongs in the repository and not in a release tarball:

```
$ git ls-files
.gitattributes
data/.gitattributes
```

```

data/encoder.bin
data/labels.csv
docs/design.md
labels.csv
scripts/run.sh
src/train.py
$ git archive --format=tar HEAD | tar -tf -
data/
data/encoder.bin
data/labels.csv
labels.csv
scripts/
scripts/run.sh
src/
src/train.py

```

`docs` and both attribute files are tracked and absent from the archive. Its sibling `export-subst` expands placeholders such as `$Format:%H$` while archiving, so that a tarball can carry the ID of its commit.

Which attributes need configuration. `text`, `eol`, `export-ignore`, `-diff`, `-merge`, the built-in diff patterns and the built-in merge drivers work from the attribute alone. `diff=<name>`, `merge=<name>` and `filter=<name>` with names of your own need a definition in configuration; the table in section 14C.8 says what happens without one. The manual also describes `whitespace`, `conflict-marker-size`, `working-tree-encoding`, `ident` and `delta`.

In production. Commit a `.gitattributes` at the top of the repository early in the life of the project. When a file "behaves strangely" (always modified, shown as binary, never conflicting), run `git check-attr -a -- <path>` first.

GitHub, not Git. GitHub reads `.gitattributes` for purposes of its own, such as the `linguist-*` attributes for language statistics and collapsed diffs. Those names mean nothing to Git.

14C.5 Line endings: `text`, `eol`, and what `core.autocrlf` cannot do

In one sentence. A path with the `text` attribute is stored with LF line endings in the repository whatever its endings are on disk, `eol` chooses the endings that a checkout writes, and `* text=auto` lets Git decide per file which files are text.

Precisely. Chapter 4 (section 4.13) showed the symptom: an editor rewrites the line endings and every line of the file counts as changed. The states of `text` decide whether Git converts ([gitattributes](#)):

text is	On git add	On checkout
set	CRLF becomes LF, always	LF becomes CRLF if <code>eol=crlf</code> (or the platform default says so)
unset (<code>-text</code>)	bytes are stored as they are	bytes are written as they are
<code>auto</code>	as "set" if Git judges the content to be text and the file is not already stored with CRLF	the same condition
unspecified	<code>core.autocrlf</code> decides	<code>core.autocrlf</code> and <code>core.eol</code> decide

`eol=lf` and `eol=crlf` fix the working-tree endings for a path and imply `text`. Without `eol`, a text file is checked out with CRLF on Windows and LF elsewhere, unless `core.eol` or `core.autocrlf` say otherwise.

`core.autocrlf` is the older, per-user mechanism: `true` converts in both directions, `input` only on the way in. It is not deprecated, and it is not a team policy, for two reasons the transcript shows:

```

$ git ls-files --eol
i/crlf w/crlf attr/      loader.py
i/lf   w/lf   attr/      run.sh
i/crlf w/crlf attr/      sample.csv

```

A teammate on Windows committed two files with CRLF (`i/` is the index, `w/` the working tree, `attr/` the attributes in force).

```

# A personal setting converts what YOU add from now on:
$ printf 'def clean(row):\r\n    return row.strip()\r\n' > clean.py
$ git -c core.autocrlf=input add clean.py
warning: in the working copy of 'clean.py', CRLF will be replaced by LF the next time Git touches it
$ git ls-files --eol clean.py loader.py
i/lf   w/crlf attr/      clean.py
i/crlf w/crlf attr/      loader.py
# It does not touch a file that is already stored with CRLF, and a teammate without the setting
# still commits CRLF:
$ printf 'def split(row):\r\n    return row.split()\r\n' > split.py
$ git add split.py
$ git ls-files --eol split.py
i/crlf w/crlf attr/      split.py

```

Your setting normalized the file you added. It did not touch `loader.py`, which is already stored with CRLF, and it had no power over the next commit made without it. A configuration value covers what one person adds on one machine. An attribute is part of the project.

```
$ printf '* text=auto\n*.sh text eol=lf\n*.csv text eol=crlf\n' > .gitattributes
$ git status -s
M sample.csv
?? .gitattributes
$ git add --renormalize .
$ git status -s
M loader.py
M sample.csv
M split.py
?? .gitattributes
$ git diff --cached --stat
loader.py | 4 +---
sample.csv | 4 +---
split.py | 4 +---
3 files changed, 6 insertions(+), 6 deletions(-)
```

Writing the attributes changed what `git status` reports: `sample.csv` now has `text` set, so its stored CRLF no longer matches what Git would store. `loader.py` and `split.py` fall under `text=auto` and are left alone because they are already stored with CRLF, so that introducing `text=auto` does not by itself rewrite a project. `git add --renormalize .` is the deliberate rewrite: it applies the current "clean" conversion to every tracked file and stages the result ([git-add](#)).

```
$ git add .gitattributes
$ git commit -q -m "Normalize line endings with .gitattributes"
$ git ls-files --eol
i/lf w/lf attr/text=auto .gitattributes
i/lf w/crlf attr/text=auto clean.py
i/lf w/crlf attr/text=auto loader.py
i/lf w/lf attr/text eol=lf run.sh
i/lf w/crlf attr/text eol=crlf sample.csv
i/lf w/crlf attr/text=auto split.py
```

Every file is `i/lf` now. Files on disk keep their endings until Git writes them again:

```
# The index now holds LF. Files on disk keep their old endings until they are written again:
$ rm loader.py sample.csv split.py
$ git restore .
$ git ls-files --eol
i/lf w/lf attr/text=auto .gitattributes
i/lf w/crlf attr/text=auto clean.py
i/lf w/lf attr/text=auto loader.py
i/lf w/lf attr/text eol=lf run.sh
i/lf w/crlf attr/text eol=crlf sample.csv
i/lf w/lf attr/text=auto split.py
```

The rewritten `loader.py` and `split.py` have LF on this Mac, `sample.csv` has CRLF on every platform because of `eol=crlf`, and `clean.py`, which was not rewritten, keeps its CRLF.

State table.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
🟡 <code>git add --renormalize .</code>	unchanged	every tracked file re-added through the current conversion and filters; new blobs where the stored form changes	unchanged	unchanged	new blobs	unchanged	unchanged

In production. The renormalizing commit touches every line of every converted file. Make it a commit of its own, announce it, merge open branches first where you can, and list its ID in a file that `git blame --ignore-revs-file <file>` (or `blame.ignoreRevsFile`) reads. Unmerged work will conflict on those files; `git merge -X renormalize` (or `merge.renormalize=true`) normalizes all three versions before merging and removes the conflicts that come from the conversion alone. Mark files that must keep their bytes (vendor exports, fixtures with deliberate CRLF, everything binary) with `-text` or `binary`.

Outdated advice. "Set `core.autocrlf` and you are done." The setting is a convenience for one user. The Git FAQ recommends marking text and binary files in `.gitattributes` ([gitfaq](#)), and the attributes manual gives the procedure used above: `* text=auto`, then one `git add --renormalize .`

14C.6 Diff drivers: `textconv`, hunk headers and `binary`

In one sentence. The `diff` attribute tells Git how to show changes to a path: not at all (`-diff`), with a named built-in pattern set for hunk headers (`diff=python`), or through a driver that you define in configuration, of which `textconv` is the most useful kind.

Precisely. A `textconv` program receives the name of a temporary file and prints a text rendering of it; Git then diffs the renderings (`gitattributes`). The conversion is one-way and meant for people: `git diff`, `git show` and `git log -p` use it, and `git format-patch` never does, because a patch made from converted text could not be applied. `diff.<name>.cachetextconv` caches renderings, and `diff.<name>.command` replaces the diff program altogether. `-diff` (or the `binary` macro) marks a file as binary: Git prints `Binary files differ`.

See it. A notebook was run again after one cell was edited. The raw diff mixes the edit with a new execution count and new output:

```
$ git diff --stat
eda.ipynb | 6 +++---
1 file changed, 3 insertions(+), 3 deletions(-)
$ git diff eda.ipynb
diff --git a/eda.ipynb b/eda.ipynb
index 41de20e..e0025c4 100644
--- a/eda.ipynb
+++ b/eda.ipynb
@@ -9,19 +9,19 @@
     },
     {
       "cell_type": "code",
-      "execution_count": 7,
+      "execution_count": 9,
       "metadata": {},
       "outputs": [
         {
           "name": "stdout",
           "output_type": "stream",
           "text": [
-            "accuracy 0.81\n"
+            "accuracy 0.84\n"
           ]
         }
       ],
       "source": [
-        "print('accuracy', evaluate(model))"
+        "print('accuracy', evaluate(model, split='test'))"
       ]
     }
   ],
 }
```

Three changed lines, of which one matters. Name a driver in `.gitattributes`:

```
$ printf '*.ipynb diff=notebook\n' > .gitattributes
$ git check-attr diff -- eda.ipynb
eda.ipynb: diff: notebook
# The attribute names a driver that no configuration defines yet, so nothing changes:
$ git diff --stat eda.ipynb
eda.ipynb | 6 +++---
1 file changed, 3 insertions(+), 3 deletions(-)
```

`git check-attr` reports the attribute, and the diff is unchanged: an attribute that names an undefined driver has no effect and causes no message. Define the driver:

```
$ cat tools/nbsource.py
#!/usr/bin/env python3
"""textconv for notebooks: print only the source of each cell."""
import json
import sys

nb = json.load(open(sys.argv[1], encoding="utf-8"))
for n, cell in enumerate(nb["cells"], 1):
    print(f"# cell {n} ({cell['cell_type']})")
    print("".join(cell["source"]))
$ git config set diff.notebook.textconv 'python3 tools/nbsource.py'
$ git diff eda.ipynb
diff --git a/eda.ipynb b/eda.ipynb
index 41de20e..e0025c4 100644
--- a/eda.ipynb
+++ b/eda.ipynb
@@ -1,4 +1,4 @@
# cell 1 (markdown)
# Error analysis
# cell 2 (code)
-print('accuracy', evaluate(model))
+print('accuracy', evaluate(model, split='test'))
```

The header still names `eda.ipynb` and the real blob IDs. The body is the difference between the two renderings: one line of code.

```
# textconv is for reading. Commands that produce patches for machines ignore it:
$ git diff --no-textconv --stat eda.ipynb
eda.ipynb | 6 +++---
1 file changed, 3 insertions(+), 3 deletions(-)
$ git add . && git commit -q -m "Evaluate on the test split"
$ git log -1 -p --format=%s -- eda.ipynb
Evaluate on the test split
```

```
diff --git a/eda.ipynb b/eda.ipynb
index 41de20e..e0025c4 100644
--- a/eda.ipynb
+++ b/eda.ipynb
@@ -1,4 +1,4 @@
# cell 1 (markdown)
# Error analysis
# cell 2 (code)
-print('accuracy', evaluate(model))
+print('accuracy', evaluate(model, split='test'))
$ git format-patch -1 --stdout -- eda.ipynb | grep -c "execution_count"
2
```

`--no-textconv` switches the conversion off, `git log -p` uses it, and the patch that `format-patch` writes contains the raw JSON (hence the two `execution_count` lines).

Hunk headers. The text after `@@ ... @@` is meant to name the enclosing function. Git finds it with a pattern, and the default pattern does not know your language. A set of built-in drivers does (the manual lists `python`, `java`, `golang`, `rust`, `markdown`, `bash` and about twenty more), and they need only the attribute:

```
$ git diff metrics.py
diff --git a/metrics.py b/metrics.py
index 75ceae5..7bfb222 100644
--- a/metrics.py
+++ b/metrics.py
@@ -5,7 +5,7 @@ class Accuracy:
    if gold is None:
        return
    self.pairs.append((pred, gold))
-    self.hits += int(pred == gold)
+    self.hits += int(pred.strip() == gold.strip())

    def result(self):
        return self.hits / self.total
$ printf '*.py diff=python\n' >> .gitattributes
$ git diff metrics.py
diff --git a/metrics.py b/metrics.py
index 75ceae5..7bfb222 100644
--- a/metrics.py
+++ b/metrics.py
@@ -5,7 +5,7 @@ def update(self, pred, gold):
    if gold is None:
        return
    self.pairs.append((pred, gold))
-    self.hits += int(pred == gold)
+    self.hits += int(pred.strip() == gold.strip())

    def result(self):
        return self.hits / self.total
```

The first header says `class Accuracy:`, the second `def update(self, pred, gold):`.

In production. The same technique serves other formats that are text to a machine and noise to a reader: lock files, exported JSON, SVG. Two cautions. A `textconv` program runs with your permissions on every diff of a matching file. And a rendering hides what it leaves out: the diff above no longer shows that the output changed.

14C.7 Merge drivers: `union`, `-merge` and your own

In one sentence. The `merge` attribute chooses the file-level merge for a path when both sides changed it: the normal three-way text merge, a refusal to merge by content (`-merge`, also spelled `merge=binary`), the built-in `union` driver that keeps the lines of both sides, or a command of your own.

Precisely. A custom driver is a configuration section `merge.<name>` with a `driver` command line. Git calls it with temporary files for the three versions: `%0` (base), `%A` (current branch) and `%B` (other branch). The driver leaves the result in `%A` and exits with status 0 for a clean merge or 1 to 128 for a conflict ([gitattributes](#)). The attribute applies to merges, cherry-picks, reverts and rebases alike: all use the same file-level merge (Chapter 8).

See it. A service repository. Both branches appended a line to `CHANGELOG.md` and a dependency to `requirements.lock`:

```
$ git merge feat/cache
Auto-merging CHANGELOG.md
CONFLICT (content): Merge conflict in CHANGELOG.md
Auto-merging requirements.lock
CONFLICT (content): Merge conflict in requirements.lock
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git merge --abort
```

Two conflicts of a kind that returns at every merge. Add two attributes, one naming a built-in driver and one naming a driver of your own that does not exist yet:

```
$ printf 'CHANGELOG.md      merge=union\nrequirements.lock merge=keep-ours\n' > .gitattributes
$ git add .gitattributes && git commit -q -m "Add merge attributes"
$ git merge feat/cache
Auto-merging CHANGELOG.md
Auto-merging requirements.lock
CONFLICT (content): Merge conflict in requirements.lock
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git status -s
M  CHANGELOG.md
UU  requirements.lock
$ cat CHANGELOG.md
# Changelog

- Add retrieval endpoint
- Add rate limiting
- Cache embeddings on disk
$ git merge --abort
```

`union` merged the changelog: both new lines are there. The lock file still conflicts, because `merge=keep-ours` names an undefined driver and Git fell back to the text merge without a word. Define it:

```
# A merge driver is a command that leaves its result in %A and reports success with exit status 0.
# "true" changes nothing, so the version of the current branch stays:
$ git config set merge.keep-ours.name 'keep our version of generated files'
$ git config set merge.keep-ours.driver true
$ git merge feat/cache
Auto-merging CHANGELOG.md
Auto-merging requirements.lock
Merge made by the 'ort' strategy.
  CHANGELOG.md | 1 +
  1 file changed, 1 insertion(+)
[exit status: 0]
$ cat requirements.lock
torch==2.4.0
limits==3.13.0
$ git show --stat --format=%s HEAD
Merge branch 'feat/cache'

CHANGELOG.md | 1 +
1 file changed, 1 insertion(+)
```

The merge is clean, and the lock file is the version of `main`: the dependency that `feat/cache` added is not in it. A lock file is generated, so the honest sequence is "keep one side, then regenerate from the merged manifest and commit".

Where silently keeping one side would be wrong, unset the attribute:

```
$ printf 'CHANGELOG.md      merge=union\nrequirements.lock -merge\n' > .gitattributes
$ git commit -q -am "Lock file: never merge by content"
$ git merge feat/cache
Auto-merging CHANGELOG.md
warning: Cannot merge binary files: requirements.lock (HEAD vs. feat/cache)
Auto-merging requirements.lock
CONFLICT (content): Merge conflict in requirements.lock
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git status -s
M  CHANGELOG.md
UU  requirements.lock
$ cat requirements.lock
torch==2.4.0
limits==3.13.0
$ git ls-files -u
100644 59fbd9ace1e2d2383413a0183f124e7e501d03ec 1      requirements.lock
100644 09a1d7444516306c5387d7b61274d47d30e30803 2      requirements.lock
100644 b678f049b99091f662d137755b1bc3b3121cf6af 3      requirements.lock
```

The path conflicts, the working file is the version of the current branch without markers, and the three stages are in the index for you to choose from (Chapter 8, section 8.11 on binary conflicts).

In production. `union` suits append-only files where order and duplicates matter little, such as changelogs. The manual warns that it "tends to leave the added lines in the resulting file in random order"; it also keeps both versions of a line that both sides edited. A keep-ours driver is safe only with a CI check that the lock file matches the manifest. And a custom driver exists only in the clones that defined it: a teammate without the definition gets ordinary conflicts.

GitHub, not Git. Merges for a pull request run on GitHub's servers, where your `merge.<name>.driver` configuration does not exist. Whether GitHub honors the built-in `union` driver is not stated in the pages read for this course, so treat merge attributes as a local aid.

14C.8 Clean and smudge filters

In one sentence. A filter driver is a pair of commands: `clean` rewrites a file's content on its way from the working tree into the repository (`git add`), and `smudge` rewrites it on its way from the repository into the working tree (checkout); the blob holds the cleaned form.

Analogy. A customs desk between your files and the object database: goods are repacked on the way in and unpacked on the way out. Where it breaks: every traveller brings a desk of their own, or none.

Precisely. `filter=<name>` in `.gitattributes` names the driver; `filter.<name>.clean` and `filter.<name>.smudge` in configuration define it. Each command reads the content on standard input and writes the converted content to standard output; `%f` in the command line is replaced by the path. Either half may be missing. The manual distinguishes two purposes and Git treats them differently:

- **Convenience filters** make content nicer (strip notebook outputs, normalize formatting), and the project stays usable without them. Every filter is of this kind by default: "a missing filter driver definition in the config, or a filter driver that exits with a non-zero status, is not an error but makes the filter a no-op passthru."
- **Required filters** turn stored content that is unusable by itself (a pointer, ciphertext) into the real thing. With `filter.<name>.required = true` a failing filter becomes an error. Git LFS (Chapter 22) is the best-known example; its `process` variant handles all files of a command in one long-running process.

On the way in the filter runs before the line-ending conversion, on the way out after it. A clean filter should be idempotent: cleaning cleaned content changes nothing.

Inside `.git`. The filter definition in `config`. Blobs hold cleaned content, and `git status` compares the cleaned form of your file with the index.

See it: a convenience filter. The clean filter below removes outputs and execution counts from notebooks. The notebook was committed before the filter existed:

```
# The notebook was committed with its outputs:
$ grep -c output_type eda.ipynb
1
$ git cat-file -p HEAD:eda.ipynb | grep -c output_type
1
```

```
$ cat tools/nbstrip.py
#!/usr/bin/env python3
"""clean filter for notebooks: drop outputs and execution counts (stdin to stdout)."""
import json
import sys

nb = json.load(sys.stdin)
for cell in nb["cells"]:
    if cell["cell_type"] == "code":
        cell["outputs"] = []
        cell["execution_count"] = None
json.dump(nb, sys.stdout, indent=1, sort_keys=True)
sys.stdout.write("\n")
$ printf '*.ipynb filter=nbstrip\n' > .gitattributes
$ git config set filter.nbstrip.clean 'python3 tools/nbstrip.py'
$ git config set filter.nbstrip.smudge cat
```

`smudge` is `cat`: nothing is added on the way out. Once the filter is defined, the committed notebook counts as modified, because its cleaned form differs from the stored blob. `--renormalize` stages the cleaned form:

```
$ git status -s
M eda.ipynb
?? .gitattributes
$ git add --renormalize .
$ git status -s
M eda.ipynb
?? .gitattributes
$ git add .gitattributes && git commit -q -m "Strip notebook outputs on the way into the repository"
$ git cat-file -p HEAD:eda.ipynb | grep -c output_type
0
$ grep -c output_type eda.ipynb
1
```

The commit has no outputs and your file still has them. From now on, running the notebook again does not produce a change:

```
# Run the notebook again (simulated with sed): a new execution count, the same source.
$ sed 's/"execution_count": 7/"execution_count": 8/' eda.ipynb > eda.tmp && mv eda.tmp eda.ipynb
$ grep execution_count eda.ipynb
    "execution_count": 8,
$ git status -s
$ git diff --stat
```

The file on disk differs from the blob, and `git status` and `git diff` are silent. That is the purpose, and a trap when you debug: with a filter, "clean working tree" means "cleans to what is committed".

See it: the clone. A teammate clones and commits:

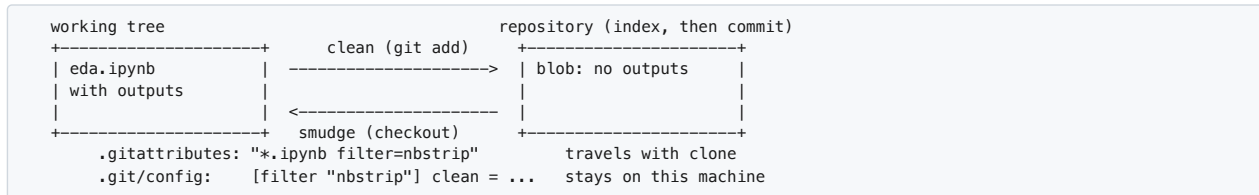
```

$ cd ..
$ git clone -q server.git analysis-asha
$ cd analysis-asha
$ git check-attr filter -- eda.ipynb
eda.ipynb: filter: nbstrip
$ git config get filter.nbstrip.clean
[exit status: 1]
# Asha runs the notebook (simulated by copying an executed copy over it) and commits.
# Nothing strips her outputs, and nothing warns her:
$ cp ../executed.ipynb eda.ipynb
$ git commit -q -am "Rerun error analysis"
$ git cat-file -p HEAD:eda.ipynb | grep -c output_type
1

```

Her clone knows that `eda.ipynb` has `filter=nbstrip`. It has no `filter.nbstrip.clean`, so the filter is a no-op and her commit puts outputs into history, without a warning: the third question of section 14C.1.

Picture.



The rule for all three kinds of driver.

	Named in (versioned)	Defined in (local)	In a clone without the definition
Diff driver	<code>diff=<name></code>	<code>diff.<name>.textconv</code> , <code>.command</code> , <code>.xfuncname</code>	default diff, no message
Merge driver	<code>merge=<name></code>	<code>merge.<name>.driver</code>	default text merge, no message
Filter, convenience	<code>filter=<name></code>	<code>filter.<name>.clean</code> , <code>.smudge</code>	content passes through unchanged, no message
Filter, required	<code>filter=<name></code>	the same, plus <code>filter.<name>.required = true</code>	still passes through: <code>required</code> is part of the missing definition

The last row surprises people. `required` protects against a filter that is defined and fails, not against a clone that never defined it. Lab 14.4 builds a required pointer filter, a small model of a large-file extension, and reproduces what happens when a teammate without the definition commits real content over a pointer.

Git does not let the repository define the commands because that would let anyone whose repository you clone run programs on your machine. The line between "versioned" and "local" is a security boundary (section 14C.12).

In production. Three arrangements work. (1) A setup step that every clone runs once executes the `git config set filter...` commands. (2) The definition lives in a tracked configuration file that each engineer includes once with `git config set include.path ../.gitconfig`; later changes to that file take effect on pull, which is code execution by pull request, so review it like CI configuration. (3) A tool installs its filter into the global configuration, as `git lfs install` does. In all three, what cannot be skipped is a CI job that fails when the stored form is wrong.

14C.9 Hooks: programs that Git runs at fixed points

In one sentence. A hook is an executable file with a fixed name in the hooks directory (or, since Git 2.54, a command named in configuration) that Git runs at a defined point of a command, passing information as arguments and standard input, and for some hooks treating a non-zero exit status as "stop".

Analogy. Tripwires in your own workshop: you place them, they do nothing in a colleague's workshop, and you can step over them. The analogy fits client-side hooks and is wrong for server-side ones, which sit at the door that every delivery must pass.

Precisely. The hooks manual lists 28 hook names ([githooks](#)). Git looks for them in `$GIT_DIR/hooks`, or in the directory named by `core.hooksPath`, and ignores files without the executable bit. A hook runs at the top of the working tree (in `$GIT_DIR` for a bare repository and for the receiving side of a push), with `GIT_DIR` and related variables exported. `git init` copies inactive samples from the template directory:

```

$ ls .git/hooks
applypatch-msg.sample
commit-msg.sample
fsmonitor-watchman.sample
post-update.sample
pre-applypatch.sample
pre-commit.sample
pre-merge-commit.sample
pre-push.sample
pre-rebase.sample

```

```

pre-receive.sample
prepare-commit-msg.sample
push-to-checkout.sample
sendemail-validate.sample
update.sample

```

The complete list. "Stops" means that a non-zero exit status makes the command fail.

Hook	Run by	Arguments	Standard input	Stops?	Skipped by
pre-commit	git commit, before the message is obtained	none	none	yes	--no-verify
prepare-commit-msg	git commit, after the default message is prepared	message file, source, commit ID	none	yes	nothing
commit-msg	git commit, git merge	message file	none	yes	--no-verify
post-commit	git commit	none	none	no	nothing
pre-merge-commit	git merge, after a clean automatic merge	none	none	yes	--no-verify
post-merge	git merge and so git pull, after success	squash flag	none	no	nothing
pre-rebase	git rebase	upstream, branch (absent for the current one)	none	yes	--no-verify
post-rewrite	git commit --amend, git rebase	amend or rebase	per commit: old ID, new ID	no	nothing
post-checkout	git switch, git checkout, git restore from a commit, git clone, git worktree add	old HEAD, new HEAD, flag (1 branch, 0 file)	none	no; its status becomes the command's	nothing
pre-push	git push, before anything is sent	remote name, URL	per ref: local ref, local ID, remote ref, remote ID	yes	--no-verify
applypatch-msg, pre-applypatch, post-applypatch	git am	message file; none; none	none	the first two	--no-verify (first two)
reference-transaction	every ref update, on either side	preparing, prepared, committed or aborted	per ref: old value, new value, ref	in the first two states	nothing
post-index-change, pre-auto-gc, sendemail-validate, fsmonitor-watchman, four p4-* hooks	index writes, git gc --auto, git send-email, index refresh, git p4 submit	see the manual	none	varies	see the manual
Server side (run by git receive-pack)					
pre-receive	once per push, before any ref is updated	none	per ref: old ID, new ID, ref	yes, the whole push	nothing on the client
update	once per pushed ref	ref, old ID, new ID	none	yes, that ref	nothing on the client
proc-receive	for refs matching receive.procReceiveRefs	none	a packet protocol	per ref	nothing on the client
post-receive	once, after the updates	none	as pre-receive	no	nothing on the client
post-update	once, after the updates	the updated refs	none	no	nothing on the client
push-to-checkout	a push into the checked-out branch under receive.denyCurrentBranch=updateInstead	the new commit	none	yes	nothing on the client

An all-zero object ID in these inputs means "does not exist": the ref is being created (old side) or deleted (new side).

See it. One tracing script is installed under ten client-side names and four server-side names:

```

$ cat .git/hooks/pre-commit
#!/bin/sh
# Installed under many hook names: print the name, the arguments and the standard input.
name=$(basename "$0")
echo "[${name}] args: $*"
case "${name}" in

```

```
$ printf 'def route(request):\n    return upstream(request)\n' > router.py && git add router.py
$ git commit -m "Add router"
[pre-commit] args:
[prepare-commit-msg] args: .git/COMMIT_EDITMSG message
[commit-msg] args: .git/COMMIT_EDITMSG
[post-commit] args:
[main (root-commit) 3f5eb96] Add router
1 file changed, 2 insertions(+)
create mode 100644 router.py
```

```
$ git commit --amend -m "Add request router"
[pre-commit] args:
[prepare-commit-msg] args: .git/COMMIT_EDITMSG message
[commit-msg] args: .git/COMMIT_EDITMSG
[post-commit] args:
[post-rewrite] args: amend
[post-rewrite] stdin: 3f5eb9642223832c879d35738acfddec6b6e70042d 9533ef0c056b35d8b7869ef013c63c330b112994
[main 9533ef0] Add request router
Date: Mon Sep 7 10:12:00 2026 +0530
1 file changed, 2 insertions(+)
create mode 100644 router.py
```

```
$ git switch -c feature/timeouts
Switched to a new branch 'feature/timeouts'
[post-checkout] args: 9533ef0c056b35d8b7869ef013c63c330b112994 9533ef0c056b35d8b7869ef013c63c330b112994 1
$ git switch main
Switched to branch 'main'
[post-checkout] args: a9d0f65fdda8978f82efaaee132e7ecd93270647 9533ef0c056b35d8b7869ef013c63c330b112994 1
# Restoring a file from a commit is a "file checkout": the last argument is 0.
$ git restore --source=feature/timeouts limits.yml
[post-checkout] args: 9533ef0c056b35d8b7869ef013c63c330b112994 9533ef0c056b35d8b7869ef013c63c330b112994 0
```

```
$ git merge feature/timeout
[pre-merge-commit] args:
[prepare-commit-msg] args: .git/MERGE_MSG merge
[commit-msg] args: .git/MERGE_MSG
Merge made by the 'ort' strategy.
 limits.yaml | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 limits.yaml
[post-merge] args: 0
```

```
$ git rebase main
[pre-rebase] args: main
[post-checkout] args: 76794b68b300e425c07a0027a44a530b5df0129b 12328180f791c3157820266ae1969bc1189457a4 1
Rebasing (1/1)
[prepare-commit-msg] args: .git/COMMIT_EDITMSG message
[post-commit] args:
[post-rewrite] args: rebase
[post-rewrite] stdin: 76794b68b300e425c07a0027a44a530b5df0129b 927de69e6948cd33c8e970e5ddb2af33f79a99a9
Successfully rebased and updated refs/heads/feature/retries.
```

```
$ git push origin main
[pre-push] args: origin ../server.git
[pre-push] stdin: refs/heads/main 12328180f791c3157820266ae1969bc1189457a4 refs/heads/main
0000000000000000000000000000000000000000000000000000000000000000
remote: [pre-receive] args:
remote: [pre-receive] stdin: 0000000000000000000000000000000000000000000000000000000000000000 12328180f791c3157820266ae1969bc1189457a4 refs/heads/main
remote: [update] args: refs/heads/main 0000000000000000000000000000000000000000000000000000000000000000 12328180f791c3157820266ae1969bc1189457a4
remote: [post-receive] args:
remote: [post-receive] stdin: 0000000000000000000000000000000000000000000000000000000000000000 12328180f791c3157820266ae1969bc1189457a4 refs/heads/main
```

```
remote: [post-update] args: refs/heads/main
To ../server.git
* [new branch]      main -> main
```

`pre-push` ran in your clone. The lines that start with `remote:` were printed by hooks in the other repository and sent back: `pre-receive` with all refs on standard input, `update` once per ref, then `post-receive` and `post-update`.

```
$ git commit --no-verify --allow-empty -m "Skip the checks"
[prepare-commit-msg] args: .git/COMMIT_EDITMSG message
[post-commit] args:
[main cc3cbfb] Skip the checks
```

`--no-verify` skipped `pre-commit` and `commit-msg`. The two hooks that `--no-verify` does not cover still ran: `prepare-commit-msg` and `post-commit`. Of these, `prepare-commit-msg` can still stop a commit by exiting non-zero, with or without `--no-verify` (run on Git 2.55.0 to confirm); `post-commit` cannot.

In production. Use client-side hooks for fast feedback to the person at the keyboard, and keep them fast, or people learn the bypass.

14C.10 Worked hooks

The scripts are in `labs/ch14c/files/`.

pre-commit: inspect what is staged. A commit is made from the index, so the hook reads the index, not the files on disk: `git diff --cached` for content and `git cat-file -s :<path>` for sizes.

```
$ cat .git/hooks/pre-commit
#!/bin/sh
# pre-commit: look at what is STAGED (the index), not at the files on disk.
# Refuse a commit that adds a private key or a file larger than 500 kB.
fail=0

if git diff --cached -U0 | grep -q '^+.*-----BEGIN [A-Z ]*PRIVATE KEY-----'; then
    echo "pre-commit: a private key is staged. Unstage it and rotate the key." >&2
    fail=1
fi

limit=512000
for path in $(git diff --cached --name-only --diff-filter=AM); do
    size=$(git cat-file -s ":$path")
    if [ "$size" -gt "$limit" ]; then
        echo "pre-commit: $path is $size bytes (limit $limit). Use Git LFS or a data store." >&2
        fail=1
    fi
done

exit $fail
$ chmod +x .git/hooks/pre-commit
```

```
# A deploy key and a 600 kB checkpoint are staged together with a real change:
$ printf -- '-----BEGIN OPENSSSH PRIVATE KEY-----\nnot-a-real-key\n-----END OPENSSSH PRIVATE KEY-----\n' > deploy_key
$ head -c 600000 /dev/zero > checkpoint.pt
$ printf 'import torch\nimport wandb\n' > train.py
$ git add .
$ git commit -m "Log runs to the tracker"
pre-commit: a private key is staged. Unstage it and rotate the key.
pre-commit: checkpoint.pt is 600000 bytes (limit 512000). Use Git LFS or a data store.
[exit status: 1]
$ git log --oneline
8d50bdb Add training script
```

No commit was made. Deleting the two files from disk does not satisfy the hook, because they are still staged:

```
# The hook reads the index. Deleting the files on disk is not enough:
$ rm deploy_key checkpoint.pt
$ git commit -m "Log runs to the tracker"
pre-commit: a private key is staged. Unstage it and rotate the key.
pre-commit: checkpoint.pt is 600000 bytes (limit 512000). Use Git LFS or a data store.
[exit status: 1]
$ git restore --staged deploy_key checkpoint.pt
$ git commit -m "Log runs to the tracker"
[main 6053c0f] Log runs to the tracker
1 file changed, 1 insertion(+)
[exit status: 0]
```

Know the limits of this hook: file names without spaces only (a robust version reads `--name-only -z`), one secret pattern, and one option removes it:

```
$ printf -- '-----BEGIN OPENSSSH PRIVATE KEY-----\nnot-a-real-key\n-----END OPENSSSH PRIVATE KEY-----\n' > deploy_key
$ git add deploy_key
$ git commit --no-verify -m "Add deploy key"
```

```
[main 86a145d] Add deploy key
1 file changed, 3 insertions(+)
create mode 100644 deploy_key
[exit status: 0]
$ git show --stat --format=%s HEAD
Add deploy key

deploy_key | 3 +++
1 file changed, 3 insertions(+)
```

commit-msg: enforce a message convention. The hook receives the name of the file that holds the proposed message and may edit or refuse it. Lab 14.2 installs this one:

```
$ printf 'def exact(pred, gold):\n    return pred.strip() == gold.strip()\n' > metrics.py
$ git commit -am "updated scorer"
commit-msg: the subject must look like 'fix(scorer): handle empty labels'
commit-msg: got: updated scorer
[exit status: 1]
$ git status -s
M metrics.py
$ git log --oneline -1
1b79baa docs: describe the metrics module
$ git commit -am "fix(metrics): ignore surrounding whitespace"
[main 6757929] fix(metrics): ignore surrounding whitespace
1 file changed, 1 insertion(+), 1 deletion(-)
[exit status: 0]
```

After the refusal, `git status -s` shows the change as unstaged: `git commit -a` stages into a temporary index and discards it when the commit fails. The lab also shows the classic defect of such hooks: they refuse the subjects that Git writes itself (`Merge branch ...`, `fixup!`, `...`), and a merge then stops half-way.

pre-push: a last look before commits leave. Standard input has one line per ref. The hook below refuses to delete `main`, to rewrite it, and to push commits whose subject starts with `WIP`, `fixup!` or `squash!` to it. Lab 14.3 prints the script and installs it; here is its effect:

```
$ git switch -q main
$ git merge -q --ff-only feature/limits
$ git push origin main
pre-push: unfinished commits must not reach main:
6378124 WIP: per-tenant limits, quota still undecided
error: failed to push some refs to '$LAB/ch14c/lab-14-3-pre-push-guard/server.git'
[exit status: 1]
$ git status -sb
## main...origin/main [ahead 2]
```

post-checkout and post-merge: tell the developer what changed. Neither can stop anything. They are the place for "your dependencies are out of date":

```
$ cat .git/hooks/post-checkout
#!/bin/sh
# post-checkout <old-HEAD> <new-HEAD> <flag>: flag 1 is a branch checkout, 0 a file checkout.
# Tell the developer when the dependency lock file differs between the two commits.
[ "$3" = 1 ] || exit 0
if ! git diff --quiet "$1" "$2" -- requirements.lock; then
    echo "post-checkout: requirements.lock changed. Run: pip install -r requirements.lock"
fi
$ cat .git/hooks/post-merge
#!/bin/sh
# post-merge <squash-flag>: runs after a merge that succeeded, which includes "git pull".
# ORIG_HEAD is the commit the branch was on before the merge.
if ! git diff --quiet ORIG_HEAD HEAD -- requirements.lock; then
    echo "post-merge: requirements.lock changed. Run: pip install -r requirements.lock"
fi
```

```
$ git switch feature/tracing
Switched to branch 'feature/tracing'
post-checkout: requirements.lock changed. Run: pip install -r requirements.lock
$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
post-checkout: requirements.lock changed. Run: pip install -r requirements.lock
# A file checkout calls the hook with flag 0, and the hook stays silent:
$ git restore --source=feature/tracing app.py
```

```
$ git pull
From $LAB/ch14c/hook-deps/server
8848493..977e90f main    -> origin/main
Updating 8848493..977e90f
Fast-forward
 requirements.lock | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
post-merge: requirements.lock changed. Run: pip install -r requirements.lock
```

The pull was a fast-forward, and `post-merge` ran all the same. The exit status of `post-checkout` is passed on without undoing the switch:

```
# post-checkout cannot undo the switch, but its exit status becomes the exit status of the command:
$ printf '#!/bin/sh\nexit 3\n' > .git/hooks/post-checkout
$ git switch feature/tracing
Switched to branch 'feature/tracing'
[exit status: 1]
$ git status -sb
## feature/tracing
```

The hook exited with 3, `git switch` exited with 1, and the branch was switched: a failing `post-checkout` hook breaks scripts that test the status of `git switch`.

`pre-rebase`: **refuse to rewrite what is published**. The hook receives the upstream and, unless the current branch is rebased, the branch:

```
$ cat .git/hooks/pre-rebase
#!/bin/sh
# pre-rebase <upstream> [<branch>]: <branch> is absent when the current branch is rebased.
# Refuse to rebase commits that a remote-tracking branch already contains.
upstream=$1
branch=${2:-HEAD}
all=$(git rev-list --count "$upstream..$branch")
unpublished=$(git rev-list --count "$upstream..$branch" --not --remotes)
if [ "$all" != "$unpublished" ]; then
    echo "pre-rebase: $((all - unpublished)) of $all commits are already on a remote." >&2
    echo "pre-rebase: rebasing them rewrites published history (Chapter 9)." >&2
    exit 1
fi
$ git log --oneline --graph --all
* eb2bc26 Add README
| * 6c7937a Add version endpoint
|/
| * 8cd8387 Add health endpoint
|/
* 0443504 Add service skeleton
```

```
$ git switch -q feature/local
$ git rebase main
Rebasing (1/1)
Successfully rebased and updated refs/heads/feature/local.
[exit status: 0]
```

```
$ git switch -q feature/shared
$ git rebase main
pre-rebase: 1 of 1 commits are already on a remote.
pre-rebase: rebasing them rewrites published history (Chapter 9).
error: The pre-rebase hook refused to rebase.
[exit status: 1]
$ git status -sb
## feature/shared...origin/feature/shared
```

The local branch was rebased, the pushed one was not. And the usual door:

```
$ git rebase --no-verify main
Rebasing (1/1)
Successfully rebased and updated refs/heads/feature/shared.
[exit status: 0]
$ git status -sb
## feature/shared...origin/feature/shared [ahead 2, behind 1]
```

The branch is now 2 ahead of and 1 behind its upstream (Chapter 9, section 9.15).

14C.11 `--no-verify`, `core.hooksPath`, and sharing hooks

Why hooks are not cloned. `git clone` transfers objects and refs. `.git/hooks` and `.git/config` are not transferred, by design: otherwise cloning a repository would mean agreeing to run its author's programs.

```
$ cat .git/hooks/pre-commit
#!/bin/sh
# Refuse a commit whose staged changes add the marker NOCOMMIT.
if git diff --cached -U0 | grep -q '^+.*NOCOMMIT'; then
    echo "check-marker: a staged line contains NOCOMMIT" >&2
    exit 1
fi
$ printf 'def infer(batch):\n    print(batch) # NOCOMMIT\n    return model(batch)\n' > infer.py
$ git commit -am "Debug inference"
check-marker: a staged line contains NOCOMMIT
[exit status: 1]
# A teammate clones the same repository:
$ git clone -q ../server.git ../asha
$ ls ../asha/.git/hooks | grep -v sample
$ printf 'def infer(batch):\n    print(batch) # NOCOMMIT\n    return model(batch)\n' > ../asha/infer.py
$ git -C ../asha commit -q -am "Debug inference"
[exit status: 0]
```

```
$ git -C ../asha log --oneline -1
069b8ab Debug inference
```

Your clone refuses the debug line. Asha's clone has no hook, and her commit is made.

Sharing, first way: a tracked directory and `core.hooksPath`. Keep the hooks in the repository and tell Git to look there. The directory travels. The setting is configuration, and each clone must make it once:

```
# Put the hook into a tracked directory and point core.hooksPath at it:
$ mkdir .githubhooks && mv .git/hooks/pre-commit .githubhooks/pre-commit
$ git add .githubhooks && git commit -q -m "Share hooks through .githubhooks" && git push -q
$ git config set core.hooksPath .githubhooks
$ git rev-parse --git-path hooks
.githubhooks
# The directory arrives with a pull. The setting does not: every clone must make it once.
$ git -C ../asha pull -q
$ ls ../asha/.githubhooks
pre-commit
$ git -C ../asha config get core.hooksPath
[exit status: 1]
$ git -C ../asha config set core.hooksPath .githubhooks
$ printf 'def infer(batch):\n    print(batch) # NOCOMMIT\n    return model(batch)\n' > ../asha/infer.py
$ git -C ../asha commit -q -am "Debug inference"
check-marker: a staged line contains NOCOMMIT
[exit status: 1]
```

A relative `core.hooksPath` is relative to the directory in which the hook runs, the top of the working tree. From then on every pull can change what runs on the next commit: review changes under `.githubhooks/` as carefully as changes to CI.

Sharing, second way: hooks defined in configuration. Since Git 2.54 a hook can be declared as `hook.<name>.command` plus one or more `hook.<name>.event` values, in any configuration file ([git-hook](#)). One script can serve several events and several repositories, and several hooks can share an event:

```
# Since Git 2.54 a hook can be a configuration entry: a name, a command, and one or more events.
$ git config unset core.hooksPath
$ git config set hook.marker.command .githubhooks/pre-commit
$ git config set --append hook.marker.event pre-commit
$ git config set --global hook.whoami.command 'echo "committing as $(git config get user.email)"'
$ git config set --global hook.whoami.event pre-commit
$ printf '#!/bin/sh\nnecho "hook file in .git/hooks ran"\n' > .git/hooks/pre-commit && chmod +x .git/hooks/pre-commit
$ git hook list --show-scope pre-commit
global whoami
local marker
hook from hookdir
$ git commit --allow-empty -m "Empty commit to watch the hooks"
committing as you@example.com
hook file in .git/hooks ran
[main 8696828] Empty commit to watch the hooks
```

`git hook list --show-scope <event>` is the first diagnostic for "why did something run": it shows, in order of execution, the configured hooks with the scope of the file that defined them, and last the file in the hooks directory. Here the global hook ran, then `marker` (silently, since nothing was staged), then the hook file. `hook.<name>.enabled=false` switches one hook off; `hook.<name>.parallel` and `hook.jobs` allow parallel runs since Git 2.55, except for hooks such as `pre-commit` and `commit-msg` that touch shared state.

```
# core.hooksPath=/dev/null silences the hooks directory, not the configured hooks:
$ git -c core.hooksPath=/dev/null commit --allow-empty -m "Second empty commit"
committing as you@example.com
[main 2ac6f6a] Second empty commit
# --no-verify skips every pre-commit hook, wherever it is defined:
$ git commit --no-verify --allow-empty -m "Third empty commit"
[main 500a741] Third empty commit
# One named hook off, in this repository only:
$ git config set hook.whoami.enabled false
$ git hook list --show-scope pre-commit
global disabled whoami
local marker
hook from hookdir
```

The manual describes `core.hooksPath=/dev/null` as a way to "disable all hooks entirely"; on Git 2.55.0 it silenced the hooks directory and the configured hook still ran. `--no-verify` skipped the `pre-commit` hooks of both kinds.

`git hook run` executes the hooks of an event, so CI and wrapper tools can run what a developer's commit would run:

```
# Run the hooks of an event by hand, for example from a CI job:
$ git hook run pre-commit
hook file in .git/hooks ran
[exit status: 0]
$ git hook list commit-msg
warning: no hooks found for event 'commit-msg'
[exit status: 1]
```


Other ways to distribute. `init.templateDir` (or `git init --template`) names a directory whose contents are copied into every new `.git`, hooks included, on one person's machine. A setup script that copies files into `.git/hooks` works everywhere and has to be run. Hook managers automate that step (section 14C.13).

`--no-verify` **in full.** With `git commit` it skips `pre-commit` and `commit-msg`; with `git merge`, `pre-merge-commit` and `commit-msg`; with `git push`, `pre-push`; with `git rebase`, `pre-rebase`; with `git am`, `applypatch-msg` and `pre-applypatch`. No option skips a server-side hook.

14C.12 Hook security

In one sentence. A hook, and several configuration values, are commands that run with your permissions, so the question "is it safe to use this repository" is the question "where did its `.git` directory come from".

Precisely. The security section of the `git` manual draws the line: because configuration and hooks are not copied by `git clone`, it is generally safe to clone a repository with untrusted content and inspect it; it is not safe to run Git commands in a `.git` directory, or the working tree around it, that itself came from an untrusted source such as an archive or a shared folder (git).

See it. A repository arrives as a tar file. Its author has put a command into `core.fsmonitor` and a `post-checkout` hook into `.git/hooks`. Both only print a line here:

```
# You download research-code.tar, unpack it and look around:
$ tar -xf research-code.tar
$ cd research-code
$ git status -s
>> a command from the archive ran (core.fsmonitor)
>> a command from the archive ran (core.fsmonitor)
$ git switch -c look-around
Switched to a new branch 'look-around'
>> a command from the archive ran (post-checkout hook)
```

`git status`, the command people run to look around, executed a command from the archive, and the switch ran the hook.

```
$ git config list --local --show-origins | grep fsmonitor
file:.git/config      core.fsmonitor=echo " >> a command from the archive ran (core.fsmonitor)" >&2; false
$ ls .git/hooks | grep -v sample
post-checkout
```

The manual's remedy is a clone, with `--no-local` so that objects are transferred through the normal protocol and not copied as files:

```
# A clone copies objects and refs. It copies neither .git/config nor .git/hooks:
$ cd ..
$ git clone -q --no-local research-code safe-copy
$ cd safe-copy
$ git status -s
$ git switch -c inspect
Switched to a new branch 'inspect'
$ git config get core.fsmonitor
[exit status: 1]
$ ls .git/hooks | grep -v sample
```

The clone has the history, without the setting and without the hook.

The rest of the threat model, in the order in which you are likely to meet it:

- **Hooks you chose to run.** A tracked hooks directory, an included configuration file and a hook manager turn "pull" into "update the programs that run on my machine".
- **Hooks in global configuration.** Since Git 2.54 a hook defined in `~/.gitconfig` runs in every repository of that user, which suits a personal secret scanner and equally an attacker who can write to that file.
- **Repositories owned by someone else.** Git refuses them unless `safe.directory` lists them.
- **Bugs that write into `.git` during a clone.** CVE-2024-32002 got a hook written into a submodule's `.git` and run while the clone was in progress. Keep Git current, and do not clone untrusted repositories with `--recurse-submodules` (Chapters 21 and 23).
- **A bare repository inside a working tree.** Chapter 14D shows the case and `safe.bareRepository`.

14C.13 Why hooks cannot enforce policy, and what does

A client-side hook fails open in four ways: it is not installed by cloning, one option skips it, it does not run for commits made in the web interface, through an API or by a tool with its own Git implementation (a coding agent's, for example), and several Git commands that create commits do not run the commit hooks at all (section 14C.9).

Enforcement needs a place that every change must pass and that its author does not control. There are two.

The server. On a Git server that you run, `pre-receive` and `update` are that place. Lab 14.3 moves the rule of its `pre-push` hook into a `pre-receive` hook in the bare repository, and the bypass stops working:

```
$ git commit -q --allow-empty -m "WIP: another placeholder"
$ git push --no-verify origin main
remote: policy: unfinished commits must not reach main:
remote: f1edb7c WIP: another placeholder
To $LAB/ch14c/lab-14-3-pre-push-guard/server.git
! [remote rejected] main -> main (pre-receive hook declined)
error: failed to push some refs to '$LAB/ch14c/lab-14-3-pre-push-guard/server.git'
[exit status: 1]
$ git reset -q --hard origin/main
$ cd ../asha
$ git pull -q
$ git commit -q --allow-empty -m "fixup! Add per-tenant limits"
$ git push origin main
remote: policy: unfinished commits must not reach main:
remote: f76de82 fixup! Add per-tenant limits
To $LAB/ch14c/lab-14-3-pre-push-guard/server.git
! [remote rejected] main -> main (pre-receive hook declined)
error: failed to push some refs to '$LAB/ch14c/lab-14-3-pre-push-guard/server.git'
[exit status: 1]
$ git reset -q --hard origin/main
```

`--no-verify` made no difference, and Asha's clone, which has no hooks, was refused as well. `[remote rejected]` with `pre-receive` hook declined is how every client reports it.

GitHub, not Git. On github.com you cannot install server-side hooks. The equivalents are rulesets and branch protection (required pull requests and status checks, blocked force pushes and deletions), push rulesets (file paths, file sizes) and push protection, which rejects a push that contains a recognized secret ([about rulesets](#), [about push protection](#)). GitHub Enterprise Server additionally supports pre-receive hook scripts ([documentation](#)). Chapter 18 covers rulesets; this paragraph is described from the documentation.

CI as a required check. Whatever a hook checks locally, a workflow can check for every pull request, and a ruleset can require it before a merge. The hook then saves a round trip, and the check is the control. Run the same script in both places.

The pre-commit framework. `pre-commit` ([pre-commit.com](#)) is a hook manager that is common in Python repositories. It is not installed on the lab machine: this paragraph is described from its documentation (version 4.6.2 when this was written), and nothing in it was run. A file `.pre-commit-config.yaml` in the repository lists hook repositories, each pinned with `rev` and each providing hooks by `id`:

```
repos:
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: <a tag or a commit ID of that repository>
  hooks:
  - id: ruff-check
  - id: ruff-format
```

`pre-commit install` sets up the Git hook script in `.git/hooks/pre-commit`; `--hook-type` does the same for other events such as `commit-msg` and `pre-push`. The framework builds an environment for each hook and caches it under `~/.cache/pre-commit`. `pre-commit run --all-files` is the documented CI usage, and `pre-commit autoupdate` moves each `rev` forward. A branch name as `rev` is not supported: pin a tag or a commit ID. Typical hooks for an ML repository, from the Phase 0 report: `ruff-check` and `ruff-format`, `check-added-large-files` (default threshold 500 kB), `detect-private-key`, `nbstripout`, and one secret scanner ([pre-commit-hooks](#), [ruff-pre-commit](#)).

The configuration is versioned and the tools are pinned, so every engineer runs the same checks. That makes the framework better than hand-copied scripts and does not make it a control: `pre-commit install` is still a step that each clone has to take, `git commit --no-verify` and the framework's `SKIP` variable skip it, and each `repo` entry is third-party code that runs on your machine and, in CI, next to your credentials. What enforces is the CI run, required by a ruleset.

14C.14 What can go wrong

Symptom	Diagnosis	Fix	Prevention
After <code>git stash pop</code> , staged changes are unstaged	<code>pop</code> ran without <code>--index</code> ; it printed the ID of the dropped commit	<code>git restore --staged --source='<id>^2' -- .</code> (Lab 14.6)	<code>git stash pop --index</code>
A merge or rebase produced a value nobody typed	The output said <code>Resolved ...</code> or <code>Staged ...</code> using previous resolution	Redo, <code>git rerere forget <path></code> , <code>git restore --merge <path></code> , <code>resolve</code> (Lab 14.5)	<code>rerere.autoUpdate</code> off; read replayed files
Unable to create <code>'.../MERGE_RR.lock'</code> in a rebase	Background <code>git rerere gc</code> held the lock; <code>git rerere status</code> is empty	<code>git rerere, resolve, continue</code>	<code>maintenance.rerere-gc.auto=0</code>
A file is "modified" right after clone or checkout	<code>git check-attr -a -- <path></code> , <code>git ls-files --eol <path></code> : stored form and attributes disagree	<code>git add --renormalize .</code> , commit	A renormalizing commit when attributes or filters change
A teammate's diff, merge or filter behaves differently	<code>git config get filter.<name>.clean</code> (or the diff, merge key) fails in their clone	Define the driver there	A setup step per clone and a CI check

Symptom	Diagnosis	Fix	Prevention
A hook does not run, or one runs that you did not install	Executable bit; <code>git rev-parse --git-path hooks</code> ; <code>git hook list --show-scope <event></code>	<code>chmod +x</code> ; fix or remove the entry	Install hooks with a script; know your global configuration
A merge stops with <code>Not committing merge</code>	A <code>commit-msg</code> hook refused Merge branch ...	Fix the hook, <code>git commit --no-edit</code> (Lab 14.2)	Let hooks pass subjects that Git writes
A secret or WIP commit is on <code>main</code> despite a hook	No hook in that clone, <code>--no-verify</code> , or a command that runs no hook	Rotate the secret (Chapter 21); enforce on the server	Rulesets, push protection, required checks

14C.15 When not to use it, and dangerous edge cases

The stash is not storage. An entry is reachable only through one line of one relog, and `drop` and `clear` delete such lines.

Do not enable `rerere.autoUpdate` to save a keystroke. It removes the only pause in which a replay can be reviewed. And records are local: two people can replay different resolutions of one conflict.

Do not use `union` on files with structure. JSON, nested YAML, lock files and code survive a union merge as text and break as data.

Do not put secrets behind a clean filter. The first commit from a clone without the definition stores the secret. Use a secret manager (Chapter 21).

A filter changes what "unchanged" means. `git status`, `git diff` and `git stash` compare cleaned content. A slow filter slows all of them, and one that is not idempotent makes files flicker between modified and clean.

Hooks that modify the commit. A `pre-commit` hook that reformats files changes the working tree, not the index, unless it runs `git add`, and then it commits content you did not review. Prefer hooks that refuse and say what to run.

● `git stash drop` and `git stash clear` delete one line, or all lines, of the log of `refs/stash`. They can destroy stashed changes once the unreachable commits are pruned. Preview with `git stash show -p`. Recover with `git stash store <id>` or the `git fsck --unreachable` recipe of Lab 8.6. `clear` is almost never appropriate.

● `git restore --merge <path>` overwrites one working file with the conflicted version rebuilt from the index, destroying your edits to that file. Preview with `git diff <path>`. There is no recovery. It is appropriate when a resolution, yours or `rerere`'s, is wrong and you want to start over.

14C.16 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git stash show</code> , <code>create</code> , <code>git log 'stash@{n}'</code>	●	nothing, or objects only	not needed	not needed
<code>git stash store</code> , <code>import</code> , <code>export --to-ref</code>	●	<code>refs/stash</code> and its log, or the named ref	<code>git stash list</code>	<code>git stash drop</code> ; <code>git update-ref -d <ref></code>
<code>git stash drop</code> , <code>git stash clear</code>	●	removes relog lines of <code>refs/stash</code>	<code>git stash show -p 'stash@{n}'</code>	<code>git stash store <id></code> while the objects exist
<code>git rerere status</code> , <code>diff</code> , <code>remaining</code> ; <code>git check-attr</code> ; <code>git ls-files --eol</code>	●	nothing	not needed	not needed
<code>git rerere forget <path></code> , <code>clear</code> , <code>gc</code>	●	deletes recorded resolutions	<code>git rerere diff</code> ; <code>ls .git/rr-cache</code>	resolve again
<code>git add --renormalize .</code>	●	stages every file whose stored form changes	<code>git status</code> before committing	<code>git restore --staged .</code>
<code>git restore --merge <path></code>	●	overwrites the file with conflict markers	<code>git diff <path></code>	none for your edits to that file
<code>git config set core.hooksPath, hook. <name>.*, filter.*, diff.*, merge.*</code>	●	which programs Git runs on your machine	<code>git hook list --show-scope <event></code> ; <code>git config list --show-origin</code>	unset the keys
<code>--no-verify</code> ; <code>git hook run <event></code>	●	skips local checks; runs them	<code>git hook list <event></code>	the server may still refuse

14C.17 Version notes

Version note. Older behavior: a hook was a file in the hooks directory, one per event. Current behavior: hooks can also be declared in configuration, listed with `git hook list`, and run in parallel. Since: `git hook` in Git 2.36 (dated in the Phase 0 report from the manual)

page, not from a release note), configuration-defined hooks in Git 2.54, parallel execution in Git 2.55 ([release notes 2.54, 2.55](#)). Recommended: hook files for repository-specific checks, configuration for personal ones.

Version note. Older behavior: `git gc --auto` ran after some commands. Current behavior: `git maintenance run --auto` with the geometric strategy, which includes a `rerere-gc` task. Since: Git 2.54. Recommended: see the lock race in section 14C.3.

Version note. Older behavior: stashes could not leave a repository. Current behavior: `git stash export` and `git stash import`. Since: Git 2.51 ([release notes](#)). Recommended: export to a ref and push it.

Unverified. Four observations in this chapter were made on Git 2.55.0 and are not stated in the manuals: `git revert` and the picks of a rebase do not run `pre-commit` and `commit-msg`; `core.hooksPath=/dev/null` does not stop configuration-defined hooks; a `post-checkout` hook that exits with 3 makes `git switch` exit with 1; and the `MERGE_RR.lock` race. A fifth is from one exploration run without a transcript: with `hook.<event>.enabled=false` the hook file in the hooks directory still ran, although the manual says that no hooks fire. Other versions were not tested.

14C.18 Practice

- **Module 14** (lab manual): 14.2 a `commit-msg` hook; 14.3 a pre-push guard and its server-side twin; 14.4 a clean and smudge filter that stores pointers; 14.5 `rerere`, resolve once, and the repair of a wrong resolution; 14.6 stash anatomy. Lab 14.1 on worktrees belongs to Chapter 25.
- Replay any transcript with `labs/run ch14c/<demo>`; the sandbox stays in place.
- Three drills, each in the sandbox a replay leaves. `ch14c/attr-merge`: write a merge driver for `requirements.lock` that keeps the lines of both sides, sorted and without duplicates, and decide whether that is correct for a lock file. `ch14c/hooks-sharing`: make the `marker` hook run on `pre-push` as well, without creating a file. `ch14c/stash-anatomy`: find the dropped entry with `git fsck` and read one of its files without restoring it.

14C.19 Interview questions

1. Draw the commits of a stash entry made with `git stash push -u`. Which tree holds the staged change, which the unstaged one, and what does `git stash show -p` compare?
2. Where is the stash list stored, and what exactly does `git stash drop 'stash@{1}'` write?
3. An engineer lost their staging after `git stash pop`. Why does the information still exist, and which single command restores it?
4. What does `rerere` record, and when? Why does a resolution recorded in a merge also serve a rebase in the other direction?
5. A wrong value reached `main` through a replayed resolution. Give the diagnosis and the complete fix. Why is a new commit alone not enough?
6. A rebase with `rerere` dies with `Unable to create MERGE_RR.lock`. Root cause, recovery, prevention?
7. Your team adds `* text=auto` to a five-year-old repository. What does the next `git status` show, what does `git add --renormalize .` do, and how do you protect open branches and `git blame`?
8. Explain "the attribute travels, the driver does not" for a diff driver, a merge driver and a required filter.
9. Which hooks run for `git commit`, in which order, and which does `--no-verify skip`? Which commit-creating commands run none of the checking hooks?
10. Why are hooks not cloned? Describe three ways to share them and the security cost of each.
11. The CTO asks for a guarantee that no commit with a private key reaches `main`. Which layers do you propose, and what does each one fail to catch?

14C.20 Sources

Primary sources

- [git-stash](#), [git-rerere](#), [gitattributes](#), [git-check-attr](#), [git-archive](#), [git-add](#), [git-restore](#), [githooks](#), [git-hook](#), [git-receive-pack](#), [git-config](#), [git-maintenance](#), [gitfaq](#) and the security section of [git](#). Every command and option in this chapter was checked against the local Git 2.55.0 copies (`git help -m <command>`).
- [Rerere technical notes](#) for conflict normalization and conflict IDs; [contrib/rerere-train.sh](#).
- [Maintenance configuration](#) and [hook configuration](#).
- Release notes [2.51.0](#), [2.54.0](#) and [2.55.0](#).
- GitHub Docs: [About rulesets](#), [About push protection](#), [About pre-receive hooks](#) (GitHub Enterprise Server).
- [pre-commit](#), the documentation of the framework, with [pre-commit-hooks](#) and [ruff-pre-commit](#).

Secondary sources

- Pro Git, [Git Hooks](#), [Git Attributes](#) and [Rerere](#). Caveat: the second edition predates `git restore`, `git hook` and configuration-defined hooks, and uses `master`.
- The Phase 0 report of this course, sections 1, 4, 14 and 15, for release dates, the hook count, the fail-open argument and the pre-commit facts.
- [nbsstripout](#), the tool whose idea the notebook filter of section 14C.8 imitates in twelve lines.

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [So You Think You Know Git Part 2](#), Scott Chacon, DevWorld 2024, 23 minutes, 20 March 2024: hooks, attributes, smudge and clean filters. Caveat from the report: no depth on hook security.
- [So You Think You Know Git](#), Scott Chacon, FOSDEM 2024, 47 minutes, 8 February 2024: rerere among many other settings. Caveat: a survey, with about one minute of product pitch.

Further reading

- Chapter 8 for conflicts, Chapter 9 for rebase, Chapter 11 for the stash commands, Chapter 13 for unreachable objects, Chapter 14D for `safe.bareRepository`.

Chapter 14D: The Frontier

Baseline. Git 2.55.0 on macOS; facts about releases and about GitHub as of 1 October 2026. Transcripts are real output from `labs/ch14d/`. Git 2.56 features are described from its documentation and marked "not run here". Dates that rest on secondary sources are marked as such.

14D.1 Why this matters

Three questions a CTO can ask in the next twelve months:

- 1. "I read that Git 3.0 changes the hash and the default branch. What breaks for us, and when?"
- 2. "Should new repositories use SHA-256 now?"
- 3. "A blog post says `git history` replaces interactive rebase. Do we change our guidelines?"

Short answers. The planned changes of Git 3.0 are defaults for newly created repositories and the removal of a few long-deprecated features; existing repositories keep their formats, and no official date exists (sections 14D.2 and 14D.3). SHA-256 works locally and cannot be pushed to `github.com` today, so a repository that must live there stays SHA-1 (section 14D.5). `git history` is experimental, refuses histories with merges and runs no hooks, so it is a convenience for local cleanup and not a guideline (section 14D.6).

The skill this chapter trains is not knowing those answers. They will be out of date. It is finding them in primary sources within a few minutes: the `BreakingChanges` document, the release notes, the manual page of your installed version, and a sandbox. Chapter 3 (Git Internals) explains the SHA-256 and reftable formats and Chapter 9 (Rebase) introduces `git replay` and `git history`; this chapter builds on both.

14D.2 The road to Git 3.0

What is official. The Git project keeps a document named `BreakingChanges`, shipped with every installation ([BreakingChanges](#)). It says that breaking releases happen rarely (1.6.0 in August 2008, 2.0 in May 2014), lists what Git 3.0 will change, and states: "There is no planned release date for this breaking version yet." That sentence is in the copy installed with Git 2.55.0 and, according to the Phase 0 report, unchanged at 2.56.0. The same document promises that the last version before 3.0 will be a long-term-support release with important bug fixes for at least four release cycles and security fixes for six, and that every breaking change is guarded by a build switch, `WITH_BREAKING_CHANGES`, so that the future behavior can be tested before it ships.

What the maintainer announced. Git 2.56.0 was released on 28 September 2026. On the same day the maintainer wrote that the next version will be numbered 2.98, "scheduled near the end of this year", to be followed by 2.99 "to solidify the codebase in preparation for Git 3.0" ([mailing-list mirror](#)). The jump in the number is the signal.

What only secondary sources say. Calendar months and the relation between 2.99 and 3.0 are not in any official document.

Statement	Source	Status
Git 3.0 is planned; no date	<code>BreakingChanges</code>	official
Next release is 2.98, near the end of 2026; then 2.99	maintainer's message of 28 September 2026	primary, read through a mirror
2.98 in December 2026	LWN , GitLab	secondary
2.99 in April 2027 (LWN) or spring 2027 (GitLab), as the long-term-support release	the same two	secondary
2.99 and 3.0 ship together and differ only in the breaking-changes switch	GitLab; LWN similar	secondary

Picture.

2.55.0 29 Jun 2026 (this course)	2.56.0 28 Sep 2026 (current)	2.98 "near the end of this year"	2.99 after 2.98 LTS (secondary)	3.0 no official date with 2.99 (secondary)
----- released ----- ----- announced ----- ----- reported only -----				

Unverified. Every date in the last three rows of the table. Treat them as a plan reported from conference talks, and re-check `BreakingChanges` and the release announcement before you put a date into a migration plan.

14D.3 The planned defaults and removals

All of the following is from `BreakingChanges` as installed with Git 2.55.0.

Planned change	Today (2.55)	In 3.0	Condition or note	Opt in or out with
Hash function of new repositories	sha1	sha256	when libraries, applications and forges are ready; SHA-1 is not being deprecated	init.defaultObjectFormat
Ref storage of new repositories	files	reftable	JGit, libgit2 and Gitoxide need to support it	init.defaultRefFormat
Initial branch name	master, with a hint	main		init.defaultBranch
Bare repositories found by walking up directories	used (all)	refused (explicit)	section 14D.4	safe.bareRepository
Rust in the build	optional	mandatory	may be deferred if distributions are hit hard	a build option, not a setting

Removals: grafts (use `git replace`), `git pack-redundant`, the directories `.git/branches/` and `.git/remotes/` as sources of remotes, `git name-rev --stdin` (use `--annotate-stdin`), `git whatchanged`, and the values `core.commentString=auto` and `core.preferSymlinkRefs=true`. One section of the document records a decision not to remove something: `git checkout` stays next to `git switch` and `git restore`.

Two of the removals already announce themselves on Git 2.55:

```
$ git -C today whatchanged -1 2>&1 | sed -n "1,5p;\$p"
'git whatchanged' is nominated for removal.

hint: You can replace 'git whatchanged <opts>' with:
hint:   git log <opts> --raw --no-merges
hint: Or make an alias:
fatal: refusing to run without --i-still-use-this
$ git -C today whatchanged -1 > /dev/null 2>&1
[exit status: 128]
$ git -C today log -1 --raw --no-merges --format="%h %s"
c63b350 First commit

:000000 100644 0000000 587be6b A      a.txt
```

Three points keep the scale of this in proportion. The changed defaults apply when a repository is created; nothing converts an existing repository. A proposal to accept only lowercase hexadecimal object IDs was still waiting for review on 28 September 2026 and is not a decision ([mailing-list mirror](#)). And hosting decides more than Git does: see the callout in section 14D.5.

14D.4 Opting in today, and opting out later

In one sentence. Four configuration keys select, for the repositories you create, the behavior that Git 3.0 will make the default, and the same keys keep today's behavior afterwards.

See it. What this Git was built with:

```
$ git version
git version 2.55.0
$ git version --build-options | grep -e rust -e default
rust: disabled
default-ref-format: files
default-hash: sha1
```

An unconfigured `git init` on 2.55 (the lab configuration normally sets `init.defaultBranch`, so the demo removes it first):

```
# The lab configuration sets init.defaultBranch. Remove it to see what Git 2.55 does on its own:
$ git config unset --global init.defaultBranch
$ git init today 2>&1 | grep -v "^hint: *$"
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:   git config --global init.defaultBranch <name>
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:   git branch -m <name>
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in $LAB/ch14d/git3-optin/today/.git/
$ git -C today symbolic-ref HEAD
refs/heads/master
$ git -C today rev-parse --show-object-format --show-ref-format
sha1
files
```

The hint names Git 3.0. Now the three `init.*` keys:

```
$ git config set --global init.defaultBranch main
$ git config set --global init.defaultObjectFormat sha256
$ git config set --global init.defaultRefFormat reftable
$ git init tomorrow
```

```

Initialized empty Git repository in $LAB/ch14d/git3-optin/tomorrow/.git/
$ git -C tomorrow symbolic-ref HEAD
refs/heads/main
$ git -C tomorrow rev-parse --show-object-format --show-ref-format
sha256
reftable
$ cat tomorrow/.git/config
[extensions]
    objectformat = sha256
    refstorage = reftable
[core]
    repositoryformatversion = 1
    filemode = true
    bare = false
    logallrefupdates = true
    ignorecase = true
    precomposeunicode = true

```

A repository with either new format has `repositoryformatversion = 1` and an `[extensions]` section, which makes older Git versions refuse it instead of misreading it (Chapter 3, sections 3.13 and 3.14).

```

# The settings act when a repository is created. An existing repository keeps what it has,
# and a clone takes its object format from the source and its ref format from your settings:
$ git -C today rev-parse --show-object-format --show-ref-format
sha1
files
$ git clone -q today today-clone
$ git -C today-clone rev-parse --show-object-format --show-ref-format
sha1
reftable

```

The existing repository did not change. Its clone took the object format from the source, as it must, and the ref format from your setting, because ref storage is a local matter that the protocol never sees.

```

# The same keys keep the old behavior after 3.0, for the tools that need it:
$ git config set --global init.defaultObjectFormat sha1
$ git config set --global init.defaultRefFormat files
$ git init -q legacy-style
$ git -C legacy-style rev-parse --show-object-format --show-ref-format
sha1
files
$ git config list --global | grep init
init.defaultbranch=main
init.defaultobjectformat=sha1
init.defaultrefformat=files

```

`safe.bareRepository`. The fourth key is about which repositories Git agrees to use. With `explicit`, a bare repository is used only when you name it:

```

$ cd server.git
$ git log --oneline
0169310 Add service
$ git config set --global safe.bareRepository explicit
$ git log --oneline
fatal: cannot use bare repository '$LAB/ch14d/safe-bare/server.git' (safe.bareRepository is 'explicit')
[exit status: 128]
# Naming the repository is what "explicit" asks for:
$ git --git-dir=. log --oneline
0169310 Add service
$ cd ..

```

Fetching from it, pushing to it and cloning it still work, because those commands name the repository, and a `.git` directory inside a working tree is not "bare" in this sense. The reason for the change is the case that Chapter 14C, section 14C.12 left open. A clone never brings a `.git/config`. It can bring a tracked directory that is shaped like a bare repository, with a `HEAD`, a `config`, `objects/` and `refs/`:

```

# You clone a project and step into one of its directories:
$ git clone -q server.git victim
$ cd victim/vendor/cache.git
$ ls
config
HEAD
objects
refs
# With the default (safe.bareRepository=all) Git takes this directory for a repository and reads
# the configuration file that arrived with the clone:
$ git rev-parse --git-dir --is-bare-repository
.
true
$ git config get core.fsmonitor
echo this-would-run

```

Inside that directory Git found a repository and read a `config` file that arrived as ordinary tracked content. This demo stops at reading a value; `BreakingChanges` describes the attack in which such a file names a command (through `core.fsmonitor`, for example) that runs when a shell

prompt calls `git status`. With `explicit`:

```
$ git config set --global safe.bareRepository explicit
$ git rev-parse --git-dir
fatal: cannot use bare repository '$LAB/ch14d/safe-bare/victim/vendor/cache.git' (safe.bareRepository is 'explicit')
[exit status: 128]
$ git config get core.fsmonitor
[exit status: 1]
$ cd ../../
$ git status -sb
## main...origin/main
```

State table.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
 <code>git config set --global init.default*</code>	unchanged	unchanged	unchanged	unchanged	none in existing repositories; the global configuration file changes	unchanged	unchange
 <code>git init</code> with those keys	new, empty	none yet	<code>ref: refs/heads/<init.defaultBranch></code> (a stub with reftable)	unborn	<code>extensions.objectformat</code> , <code>extensions.refstorage</code> , <code>reftable/</code>	unchanged	change
 <code>git config set --global safe.bareRepository explicit</code>	unchanged	unchanged	unchanged	unchanged	none; Git commands run inside bare repositories now need <code>--git-dir</code> or <code>GIT_DIR</code>	unchanged	change

In production. Set `init.defaultBranch=main` everywhere now; it is what 3.0 will do. Set `safe.bareRepository=explicit` on developer machines now, and check the scripts that `cd` into bare repositories (backup jobs, mirror scripts, self-hosted Git servers) before 3.0 does it for you. Leave the object and ref formats at their defaults in shared configuration until your hosting and your tools are ready.

14D.5 SHA-256 and reftable repositories, locally

Chapter 3 explains both formats and shows that SHA-1 and SHA-256 repositories cannot exchange objects. Two practical facts complete the picture, and Lab 42.1 exercises both.

History can cross the format boundary only as a stream. `git fast-export` writes history as text and `git fast-import` rebuilds it in the format of the receiving repository. Every object gets a new ID:

```
# Object formats cannot be mixed, so history moves as a stream and every object gets a new name:
$ cd ..
$ git -C future fetch -q ../inference main
fatal: mismatched algorithms: client sha256; server sha1
[exit status: 128]
$ git init -q --object-format=sha256 inference-sha256
$ git -C inference fast-export --all | git -C inference-sha256 fast-import --quiet
$ git -C inference log --oneline --all
c284765 Pad batches to a fixed length
53a79b6 Raise batch size to 16
3e283dd Add inference service
$ git -C inference-sha256 log --oneline --all
4fea7f1 Pad batches to a fixed length
587a0ea Raise batch size to 16
6daacca Add inference service
$ git -C inference-sha256 repo info object.format references.format
object.format=sha256
references.format=files
```

The subjects match and no ID does. Signatures do not survive, and commit IDs quoted in messages, issue trackers and CI configuration point at nothing in the new repository. A conversion is a migration project with a cut-over date, not a setting.

The ref format can change in place and back. `git refs migrate --ref-format=reftable` converts an existing repository, with the limits the manual lists (not with linked worktrees, and no concurrent writers during the migration). Code that reads `.git/refs/heads/<branch>` breaks at that moment, and code that expects forty hexadecimal digits breaks on SHA-256:

```
# The release script has not changed. Two repositories, two different failures:
$ sh ../scripts/release-id.sh
cat: .git/refs/heads/main: Not a directory
[exit status: 1]
$ cd ../inference-sha256
$ sh ../scripts/release-id.sh
release-id: not a commit ID: 587a0ea1903116442c9bc9d9014567a4bfdd436c0268f95ddbf247362f5639
[exit status: 1]
```

The repair is the habit this course has used since Chapter 3: ask Git (`git rev-parse`, `git for-each-ref`, `git refs list`) and never read the directory.

GitHub, not Git. GitHub had no publicly available support for SHA-256 repositories on 1 October 2026. A private preview is reported by GitLab's blog, a community comment and a conference speaker, and no GitHub blog post, changelog entry or documentation page announces it, so the Phase 0 report marks it unverified ([community discussion](#)). Whether a host stores refs with reftable on its servers does not concern you: the ref format is local.

14D.6 `git history`: single-purpose rewrites

In one sentence. 🟡 `git history` rewrites one commit (its message, its content, or its division into two) and replays every descendant on top, updating all local branches that contain the commit, without touching a working tree and without a todo list.

Precisely. The command arrived in Git 2.54 with `reword` and `split`; 2.55 added `fixup`; `drop` is new in Git 2.56 (not run here) ([git-history at 2.56](#)). The first line of its manual says EXPERIMENTAL. Compared with `git rebase -i` the manual names three differences: most subcommands need neither index nor working tree, so they work in a bare repository; no hooks are run; and by default every branch that descends from the commit is updated, where a rebase moves one branch unless you add `--update-refs`. Two limits are by design for now: no merges in the affected history, and no operation that would conflict.

Inside .git. New commit objects for the target and all its descendants; every affected branch ref moves, each with a reflog entry `reword: updating <what you typed>`; HEAD's reflog gets one entry.

See it. The repository has a typo in its oldest commit and three branches above it:

```
$ git log --oneline --graph --all
* f0d1c5a Add development requirements
| * 67d2d0a Add judge prompt
|/
* 39f5b0c Add README
* 156aedd Add F1 and its test
* ec69d27 Add exact match metirc
$ git history reword HEAD~3
$ git log --oneline --graph --all
* 544b01e Add development requirements
| * 131e7a7 Add judge prompt
|/
* 545a235 Add README
* ccbad89 Add F1 and its test
* ff71a66 Add exact-match metric
```

The editor opened with the old message, and the message was replaced. Every commit ID changed, and `main`, `release/0.1` and `topic/judge` all point into the new history:

```
$ git reflog -1
544b01e HEAD@{0}: reword: updating HEAD~3
$ git reflog show -1 release/0.1
545a235 release/0.1@{0}: reword: updating HEAD~3
$ git reflog show -1 topic/judge
131e7a7 topic/judge@{0}: reword: updating HEAD~3
$ git status -sb
## main
```

`--dry-run` shows what would move. It writes the new objects and prints the ref updates in the input format of `git update-ref --stdin`:

```
# A dry run writes the new objects and prints the ref updates instead of making them:
$ git history reword --dry-run HEAD~2
update refs/heads/release/0.1 3d69bd220ec3abddad18c7e8b3346dfce1e86875 545a2357ebf779e3ff3fe25722f31fa7cdec7c18
update refs/heads/topic/judge 817d179159be202eaac73a802d2d6ee5809a7d0d 131e7a7a4f0c7f2fec731a761cea3d6826886f6
update refs/heads/main a61938caa6eebb699fad4e1eb3ac960d840a2e26 544b01e4b86859029e518ca1f02c4779bedc01eb
$ git log --oneline -3
544b01e Add development requirements
545a235 Add README
ccbad89 Add F1 and its test
```

`fixup` folds the staged change into an older commit, which is the autosquash workflow of Chapter 9 in one step:

```
# A staged change is folded into an older commit, and its descendants are replayed:
$ printf 'from metrics import f1\n\nndef test_f1():\n    assert f1(1, 0, 0) == 1.0\n    assert f1(0, 1, 1) == 0.0\n' >
test_metrics.py
$ git add test_metrics.py
$ git history fixup HEAD~2
[exit status: 0]
$ git status -s
$ git log --oneline --stat --format='%h %s' -3
6794ec8 Add development requirements

requirements-dev.txt | 1 +
1 file changed, 1 insertion(+)
f5e9c9c Add README
```

```

README.md | 3 +++
1 file changed, 3 insertions(+)
49ba6c8 Add F1 and its test

metrics.py | 4 ++++
test_metrics.py | 6 ++++++
2 files changed, 10 insertions(+)

```

split asks, hunk by hunk, what to move into a new commit that becomes the parent of the original. The demo answers y and n and supplies two messages:

```

# Answers typed at the two prompts: y (move this hunk into the new, earlier commit), then n.
$ printf 'y\n\n' | git history split HEAD~2
diff --git a/metrics.py b/metrics.py
index d6bc9be..e1c56b4 100644
--- a/metrics.py
+++ b/metrics.py
@@ -1,2 +1,6 @@
def exact(pred, gold):
    return pred == gold
+
+
+def f1(tp, fp, fn):
+    return 2 * tp / (2 * tp + fp + fn)
(1/1) Stage this hunk [y,n,q,a,d,?]?
diff --git a/test_metrics.py b/test_metrics.py
new file mode 100644
index 0000000..d3c2e3e
--- /dev/null
+++ b/test_metrics.py
@@ -0,0 +1,6 @@
+from metrics import f1
+
+
+def test_f1():
+    assert f1(1, 0, 0) == 1.0
+    assert f1(0, 1, 1) == 0.0
(1/1) Stage addition [y,n,q,a,d,?]?

```

```

$ git log --stat --format='%h %s' -4
f6c5fad Add development requirements

requirements-dev.txt | 1 +
1 file changed, 1 insertion(+)
06c85c9 Add README

README.md | 3 +++
1 file changed, 3 insertions(+)
298f4d9 Test the F1 metric

test_metrics.py | 6 ++++++
1 file changed, 6 insertions(+)
24381a5 Add F1 metric

metrics.py | 4 ++++
1 file changed, 4 insertions(+)

```

The limits, each with its real message:

```

# A commit-msg hook that refuses every message does not stop it: git history runs no hooks.
$ printf '#!/bin/sh\nnecho "commit-msg: refused" >&2\nexit 1\n' > .git/hooks/commit-msg && chmod +x .git/hooks/commit-msg
$ git commit --allow-empty -m "probe"
commit-msg: refused
[exit status: 1]
$ git history reword HEAD
$ git log --oneline -1
82c3f29 Add dev requirements

```

```

# A fixup that would conflict is refused, and nothing changes:
$ printf 'def exact(pred, gold):\n    return pred.strip() == gold.strip()\n\n\ndef f1(tp, fp, fn):\n    return 2 * tp / (2 * tp + fp + fn)\n' > metrics.py && git add metrics.py
$ git history fixup HEAD~4
error: fixup would produce conflicts; aborting
[exit status: 255]
$ git status -s
M metrics.py
# History with a merge commit above the target is refused as well:
$ git log --oneline --graph -4
* b8a371c Merge topic/judge
|\
| * 5ac3007 Add judge prompt
* | 82c3f29 Add dev requirements
|/
* 06c85c9 Add README
$ git history reword HEAD~2

```

```
error: replaying merge commits is not supported yet!
[exit status: 255]
```

State table.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
 <code>git history</code> <code>reword</code> <code><commit></code>	unchanged	unchanged	follows its branch	moved to the rewritten tip	every other local branch that contains the commit is moved too; reflog entries; new objects	unchanged; branches that were pushed now diverge	unchanged
 <code>git history</code> <code>fixup <commit></code>	unchanged	the staged change is consumed	as above	as above	as above	as above	unchanged
 <code>git history</code> <code>... --dry-run</code>	unchanged	unchanged	unchanged	unchanged	new objects only	unchanged	unchanged

In production. The default of moving every descendant branch is right for a stack of local branches and wrong the moment one of them is published: Lab 42.2 rewords a pushed commit and then puts two branches back from their reflogs. Because no hooks run, a message convention enforced by `commit-msg` is not checked. The lab also shows that the committer date of the rewritten commit changes although the manual says that all other details stay as they were. Use it for local cleanup before the first push, read `--dry-run` first, and keep `git rebase -i` for everything with merges or conflicts.

14D.7 `git replay`: rebase and cherry-pick without a working tree

Chapter 9, section 9.20 showed `git replay --onto` and its silent failure on a conflict. The command exists for servers: a hosting service that rebases or cherry-picks for you has no working tree to do it in. In the bare repository of the demo, `fix/timeout` branches off an old commit of `main`:

```
# Put the two commits of fix/timeout on top of main, and only say which ref would move:
$ git replay --ref-action=print --advance=main main..fix/timeout
update refs/heads/main 1653f1a9ffb2929d943f9eea08a045ec877c6e21 36761eb01933290987d1bcfb23c957a4a0f35f8d
[exit status: 0]
$ git log --oneline -1 main
36761eb Raise the rate limit to 120
```

`--ref-action=print` creates the commits and prints the update, which a server can inspect or feed to `git update-ref --stdin`. Without it (the default since Git 2.53) the ref moves in one transaction:

```
$ git replay --advance=main main..fix/timeout
[exit status: 0]
$ git log --oneline --graph --all
* 863b8b5 Document the timeout
* dc10709 Add a 10 second timeout to routing
* 36761eb Raise the rate limit to 120
| * 9e11dcc Document the timeout
| * f21ac0f Add a 10 second timeout to routing
|/
* c55bd6f Add gateway skeleton
$ git log -2 --format='%h author: %an, committer: %cn | %s' main
863b8b5 author: Lab User, committer: Lab User | Document the timeout
dc10709 author: Lab User, committer: Lab User | Add a 10 second timeout to routing
```

`--advance=main` is a cherry-pick of the range onto `main`; the original author is kept. Since Git 2.54 the same machinery reverts:

```
# Since Git 2.54 the same machinery reverts:
$ git replay --revert=main main~1..main
[exit status: 0]
$ git log -1 --format=%B main
Revert "Document the timeout"

This reverts commit 863b8b52c9ef9bbf4d884372524d9fd9a4cb9021.

# A bare repository keeps no reflog unless it is configured to, so this prints nothing:
$ git reflog show main
```

The last command printed nothing: a bare repository keeps no reflog unless `core.logAllRefUpdates` says so, so a replay there has no local undo. `git replay --linearize` was added in Git 2.56 (not run here) ([git-replay at 2.56](#)). The command is experimental, and its value for you is mostly indirect: it is the kind of operation behind a "rebase and merge" button.

14D.8 `git last-modified` and `git repo`

Two experimental commands from Git 2.52 answer questions that used to need a loop or a peek into `.git`.

```
$ git log --oneline --graph
* 5bcf6ca Merge branch 'fix/timeout'
| \
| * 9e11dcc Document the timeout
| * f21ac0f Add a 10 second timeout to routing
* | 36761eb Raise the rate limit to 120
| /
* c55bd6f Add gateway skeleton
# One line per entry of the top-level tree: the commit that last changed it.
$ git last-modified
5bcf6ca2deb65d80283c0a190f491cf04040bd48      src
9e11dccdbbb9baf1ebd438d9185161322cc66779      docs
$ git last-modified -r
9e11dccdbbb9baf1ebd438d9185161322cc66779      docs/README.md
f21ac0fee598130815a3870c12ac15b75494d550      src/router.py
36761eb01933290987d1bcfb23c957a4a0f35f8d      src/limits.yaml
```

`git last-modified` prints, for each path, the commit that last changed it: the "last commit" column of a file browser. For the directory `src` the answer is the merge commit, because the two sides changed different files in it and only the merge produced the present tree. With `-r` it descends to files.

```
$ git repo info --keys
layout.bare
layout.shallow
object.format
references.format
$ git repo info --all
layout.bare=false
layout.shallow=false
object.format=sha1
references.format=files
$ git -C ../server.git repo info layout.bare references.format
layout.bare=true
references.format=files
```

`git repo info` returns named facts about a repository in a stable `key=value` form, which is what a script should use in place of reading `.git/config`. Git 2.56 adds `path.*` keys (not run here). `git repo structure` counts refs and objects:

```
$ git repo structure --format=lines | grep -e count -e inflated
references.branches.count=2
references.tags.count=0
references.remotes.count=2
references.others.count=0
objects.commits.count=5
objects.trees.count=11
objects.blobs.count=6
objects.tags.count=0
objects.commits.inflated_size=1132
objects.trees.inflated_size=683
objects.blobs.inflated_size=230
objects.tags.inflated_size=0
```

14D.9 Rust in Git, stacked workflows, and Git-compatible tools

Rust. BreakingChanges gives the milestones: Rust code entered Git as optional in 2.49, was auto-detected by one of the two build systems in 2.52, is enabled by default in both from 2.55, and becomes mandatory in 3.0 unless the impact on distributions leads to a deferral. The long-term-support promise of section 14D.2 exists for platforms without a Rust toolchain. For you as a user nothing changes except where the binary comes from. The first transcript of section 14D.4 printed `rust: disabled`: Homebrew builds its Git 2.55.0 with `NO_RUST=1` (formula, read on 2 October 2026). So "enabled by default" is a statement about the build system, and each distributor decides.

Stacked workflows. A stack is a chain of small dependent branches, each reviewed on its own. In Git the tools are `git rebase --update-refs` (Chapter 9, Lab 9.7), `git range-diff` for comparing versions, and now `git history`, whose default of moving all descendant branches fits a stack.

GitHub, not Git. Stacked pull requests entered public preview on GitHub on 30 July 2026, with a `gh stack` extension for the command line (changelog). Described from the changelog; not exercised here.

Tools worth knowing by name. None is part of Git, and none is needed for this course.

Tool	What it is	Relation to Git
Jujutsu (jj)	A version-control system with no index, automatic rebasing of descendants and conflicts that can be committed	stores its data in Git repositories; its README calls it experimental
Sapling	Meta's client with first-class stacks	Git-compatible client
git-branchless	Adds undo, a smart log and restacking	a set of commands on top of Git; self-described alpha

Tool	What it is	Relation to Git
Graphite, GitButler	Commercial products for stacked changes and branch management	on top of Git and GitHub
libgit2 , JGit , Gitoxide	Implementations of Git as libraries, in C, Java and Rust	what many IDEs, servers and tools use in place of the <code>git</code> program

The last row is the one that matters for the 3.0 defaults: `BreakingChanges` names exactly these three libraries as the ones that must support reftable before it becomes the default. A tool built on a library reads your repository with that library's abilities, not with those of the `git` you installed.

14D.10 Reading release notes, `BreakingChanges` and the GitHub Changelog

Release notes. Every release has one file, on [GitHub](#) and on your disk:

```
$ ls "$(git --html-path)/RelNotes" | grep -c adoc
541
$ ls "$(git --html-path)/RelNotes" | grep "^2\\.5[3-5]"
2.53.0.adoc
2.54.0.adoc
2.54.1.adoc
2.55.0.adoc
```

A file for 2.54.1 exists, and the Phase 0 report found no tag and no tarball for that version: a notes file is not proof of a release. The tag list is. Each file has the same three parts:

```
$ grep -n -B1 "^---" "$(git --html-path)/RelNotes/2.55.0.adoc" | grep -v -e "----" -e "^---$"
4-UI, Workflows & Features
82-Performance, Internal Implementation, Development Support etc.
236-Fixes since v2.54
```

Read the first part completely: it is short and it is where behavior changes are. Search the rest for the commands you depend on:

```
# What did 2.55 say about the commands of this chapter?
$ grep -n -A1 -e "git history" -e "Rust support" -e "Hook scripts" "$(git --html-path)/RelNotes/2.55.0.adoc"
7: * Hook scripts defined via the configuration system can now be
8-   configured to run in parallel.
--
28: * "git history" learned "fixup" command.
29-
--
88: * Rust support is enabled by default (but still allows opting out) in
89-   some future version of Git.
```

The style is terse, one bullet per topic, often without the option names. The procedure that works: find the bullet, open the manual page of that command at the new version, and try it in a sandbox. Note the third hit. The wording about Rust in the installed notes is vaguer than the milestone in `BreakingChanges`, and the Phase 0 report quotes a different sentence for the same release, which shows why this course cites the installed copy when it can.

BreakingChanges.

```
$ grep -n "^==" "$(git --html-path)/BreakingChanges.adoc"
62:== Procedure
80:== Git 3.0
89:=== Changes
243:=== Removals
342:== Superseded features that will not be deprecated
$ grep -n "planned release date" "$(git --html-path)/BreakingChanges.adoc"
83:is no planned release date for this breaking version yet.
# One line per planned change of a default:
$ sed -n '/^=== Changes/,/^=== Removals/p' "$(git --html-path)/BreakingChanges.adoc" | grep '^\\*' | cut -c1-78
* The default hash function for new repositories will be changed from "sha1"
* The default storage format for references in newly created repositories will
* In new repositories, the default branch name will be `main`. We have been
* Git will require Rust as a mandatory part of the build process. While Git
* The default value of `safe.bareRepository` will change from `all` to
```

The headings are the whole structure: the procedure, the changes, the removals, and what will not be deprecated. Each item links to the mailing-list thread where it was decided.

The other channels. The maintainer's "What's cooking" messages list every topic in flight and its state, and [Git Rev News](#) summarizes the list monthly. Vendor posts (the highlight posts of GitHub and GitLab for each release) are readable and selective: use them to learn that something exists and the primary text to learn what it does.

GitHub, not Git. The [GitHub Changelog](#) is the primary source for platform behavior, dated entry by entry. A feature there has a state (preview or generally available) and often a plan gate. Nothing in it describes the `git` program on your machine, and a Git release changes

nothing on github.com until GitHub deploys it.

A routine that takes fifteen minutes per release: read the first section of the notes; search for your team's commands; look at the diff of BreakingChanges; decide whether any default you rely on is named; write down the minimum version for anything new you want to adopt.

14D.11 The patch-based workflow of the Git project

In one sentence. Git itself is developed by sending commits as e-mail: `git format-patch` turns commits into message files, reviewers answer on the list, and the maintainer applies accepted series with `git am`; no pull request is involved.

Precisely. A contribution starts on a topic branch. Each commit carries a `Signed-off-by` trailer (`git commit -s`), by which the author certifies the right to submit it. `git format-patch` writes one file per commit plus an optional cover letter, and `git send-email` posts them to `git@vger.kernel.org`; GitGitGadget is a bridge that turns a pull request on GitHub into such a series ([SubmittingPatches](#), [MyFirstContribution](#), [GitGitGadget](#)). After review the author sends a complete new version (`v2`, `v3`) with a range-diff against the previous one. The maintainer queues topics on the branch `seen`, merges them to `next` for testing and then to `master`; `maint` receives fixes for the last release ([maintain-git](#)).

See it. Asha has a branch with two signed-off commits and no push access to the maintainer's repository:

```
$ cd contributor
$ git log --format='%h %an | %s' origin/main..casefold
4309e9b Asha Rao | README: document the lowercasing
0cddac1 Asha Rao | tok: lowercase the input before splitting
$ git format-patch --cover-letter --base=origin/main -o ../outbox/v1 origin/main
../outbox/v1/0000-cover-letter.patch
../outbox/v1/0001-tok-lowercase-the-input-before-splitting.patch
../outbox/v1/0002-README-document-the-lowercasing.patch
```

```
$ cat ../outbox/v1/0001-tok-lowercase-the-input-before-splitting.patch
From 0cddac17b0e047370ed3799c71bf9c20fd534983 Mon Sep 17 00:00:00 2001
From: Asha Rao <asha@example.com>
Date: Mon, 7 Sep 2026 10:08:00 +0530
Subject: [PATCH 1/2] tok: lowercase the input before splitting
```

```
Signed-off-by: Asha Rao <asha@example.com>
---
 tok.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

diff --git a/tok.py b/tok.py
index 2800645..d89f8b7 100644
--- a/tok.py
+++ b/tok.py
@@ -1,2 +1,2 @@
 def tokenize(text):
-    return text.split()
+    return text.lower().split()
--
2.55.0
```

A patch file is an e-mail. The `From <id>` line and the fixed date after it mark the format. `From:`, `Date:` and `Subject:` become author, author date and subject. The message body follows, then `---`, then a diffstat that is ignored on application, then the diff. `[PATCH 1/2]` is stripped by `git am`.

```
$ sed -n '/^Subject/, $p' ../outbox/v1/0000-cover-letter.patch
Subject: [PATCH 0/2] *** SUBJECT HERE ***

*** BLURB HERE ***

Asha Rao (2):
  tok: lowercase the input before splitting
  README: document the lowercasing

README.md | 2 +-
tok.py    | 2 +-
2 files changed, 2 insertions(+), 2 deletions(-)

base-commit: 9ba24bca99d4152c3fd1aff140540eb6cb16a88f
--
2.55.0
```

The cover letter is a template to fill in. `base-commit` tells reviewers and tools which commit the series applies to. The maintainer applies the two patches:

```
# The maintainer, in a repository that has never seen the branch:
$ cd ../upstream
$ git switch -q -c review/casefold
$ git am ../outbox/v1/0001-*.patch ../outbox/v1/0002-*.patch
```

```
Applying: tok: lowercase the input before splitting
Applying: README: document the lowercasing
$ git log --format='%h author: %an, committer: %cn | %s' -2
632b3e7 author: Asha Rao, committer: Ravi Menon | README: document the lowercasing
463b82b author: Asha Rao, committer: Ravi Menon | tok: lowercase the input before splitting
$ git log -1 --format=%B
README: document the lowercasing

Signed-off-by: Asha Rao <asha@example.com>
```

The author is Asha, the committer is the person who ran `git am`: the two identities of a commit (Chapter 6) exist for this workflow. The commits are new objects with the same content:

```
# The applied commits have new IDs and the same content as the originals:
$ git rev-parse 'HEAD^{tree}'
95b752c13b6b8cd0443d6e6cc40521f652ffafe6
$ git -C ../contributor rev-parse 'casefold^{tree}'
95b752c13b6b8cd0443d6e6cc40521f652ffafe6
$ git show HEAD~1 | git patch-id --stable | cut -d" " -f1
20e1452a5c438e93046c3c9fcf204de5c866748a
$ git -C ../contributor show casefold~1 | git patch-id --stable | cut -d" " -f1
20e1452a5c438e93046c3c9fcf204de5c866748a
```

Equal trees, equal patch IDs, different commit IDs (Chapter 10, section 10.10 on patch IDs). Review asks for a change, and the contributor rewrites the branch and sends version 2:

```
# Review asked for casefold() instead of lower(). The contributor keeps v1 and rewrites the branch:
$ cd ../contributor
$ git branch casefold-v1
$ git range-diff origin/main casefold-v1 casefold
1: 0cddac1 < -: ----- tok: lowercase the input before splitting
-: ----- > 1: 3e0656d tok: lowercase the input before splitting
2: 4309e9b = 2: 5b37458 README: document the lowercasing
```

```
$ git format-patch -v2 --cover-letter --range-diff=casefold-v1 --base=origin/main -o ../outbox/v2 origin/main
../outbox/v2/v2-0000-cover-letter.patch
../outbox/v2/v2-0001-tok-lowercase-the-input-before-splitting.patch
../outbox/v2/v2-0002-README-document-the-lowercasing.patch
$ sed -n '/^Range-diff/,/^-- /p' ../outbox/v2/v2-0000-cover-letter.patch
Range-diff against v1:
1: 0cddac1 ! 1: 3e0656d tok: lowercase the input before splitting
@@ tok.py
@@
def tokenize(text):
- return text.split()
++ return text.lower().split()
++ return text.casefold().split()
2: 4309e9b = 2: 5b37458 README: document the lowercasing

base-commit: 9ba24bca99d4152c3fd1aff140540eb6cb16a88f
--
```

`git range-diff` pairs the commits of the two versions. On its own it judged the rewritten first patch a total rewrite and printed it as removed (<) and added (>): its `--creation-factor` defaults to 60, and in a two-line patch one changed line is a large share. `format-patch --range-diff` defaults to 999, because it compares iterations of one topic, so the cover letter shows the same pair with `!` and a diff of the two diffs. The second patch is `=`: unchanged. This is how a reviewer of version 2 sees what changed since version 1 without reading everything again. Chapter 9, section 9.14 uses the same command to review a rebase.

In production. You will meet this workflow in three places besides the Git project: the Linux kernel and other list-based projects, vendors who send fixes as patch files, and environments where two repositories cannot reach each other. `git am` also underlies `git rebase --apply`. Lab 42.3 does the round trip and repairs a patch that no longer applies. On GitHub the same ideas have other names: the pull request is the cover letter, a force-pushed branch is a new version, and "compare" is a weaker range-diff.

14D.12 What can go wrong

Symptom	Diagnosis	Fix	Prevention
fatal: mismatched algorithms: client sha256; server sha1	The two repositories use different object formats (<code>git rev-parse --show-object-format</code> in each)	Recreate one side in the other format through <code>fast-export</code> and <code>fast-import</code> ; plan it as a migration	Decide the format at <code>git init</code> ; SHA-1 for everything that goes to GitHub
New repositories are SHA-256 or refutable and a tool cannot open them	<code>git config list --show-origin</code> shows <code>init.defaultObjectFormat</code> or <code>init.defaultRefFormat</code>	Unset the key; <code>git refs migrate --ref-format=files</code> for the ref format	Opt in per repository with <code>git init --object-format</code> , <code>--ref-format</code> , not globally
A script fails with Not a directory or No such file under .git/refs	The repository uses refutable, or the ref is packed	Use <code>git rev-parse</code> , <code>git for-each-ref</code>	Never read <code>.git</code> directly

Symptom	Diagnosis	Fix	Prevention
fatal: cannot use bare repository ... (safe.bareRepository is 'explicit')	A command ran inside a bare repository without naming it	<code>git --git-dir=<path> ...</code> or <code>GIT_DIR</code>	Fix server and backup scripts before 3.0
'git whatchanged' is nominated for removal	The command is on the removal list	<code>git log --raw --no-merges</code>	Read the Removals list once per release
After <code>git history</code> , branches are "ahead N, behind M"	A commit that was already pushed was rewritten; all descendant branches moved	Put each branch back from its reflog (Lab 42.2)	<code>git branch -r --contains <commit> first; --dry-run</code>
error: replaying merge commits is not supported yet!	<code>git history</code> met a merge above the target	<code>git rebase -i --rebase-merges</code>	Clean up before merging
<code>git replay</code> exits 1 without output	The replay would conflict	Rebase in a working tree	Not applicable
error: patch failed ... patch does not apply in git am	The base moved; <code>git am --show-current-patch=diff</code>	<code>git am --abort</code> , then <code>git am -3</code> and resolve (Lab 42.3)	<code>--base</code> in <code>format-patch</code> ; apply to the stated base
<code>git range-diff</code> shows a changed patch as < and >	The pair fell under the creation factor	<code>--creation-factor=<higher></code>	Not applicable

14D.13 When not to use it, and dangerous edge cases

Do not adopt experimental commands in shared automation. `git history`, `git replay`, `git last-modified` and `git repo` carry the word EXPERIMENTAL in their manuals: output and options may change between releases. Use them at the keyboard, and pin the Git version where a script depends on one.

Do not set the 3.0 formats globally on a machine that talks to GitHub. A repository created with SHA-256 cannot be pushed there today, and the error appears at the first push, after the work is done.

Do not treat a secondary date as a deadline. The only dated facts about 3.0 are that 2.56 is out and that 2.98 is next.

`git history` **and published commits.** It moves every local branch that contains the commit, in one command and without asking. The reflog of each branch has the old tip; there is no single undo.

`git am` **in a dirty tree, and** `git apply`. `git am` refuses to start when the index has changes. `git apply` without `--index` or `--3way` patches working files only and creates no commit; it is a tool for trying a patch, not for integrating one.

`safe.bareRepository=explicit` **and tools.** Editors and prompts that run Git inside bare repositories stop working until they pass `--git-dir`.

14D.14 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git version --build-options</code> , <code>git repo info</code> , <code>git repo structure</code> , <code>git last-modified</code>	●	nothing	not needed	not needed
<code>git config set --global init.default*, safe.bareRepository</code>	●	behavior of later commands	<code>git config list --show-origin</code>	unset the keys
<code>git init --object-format=sha256 --ref-format=reftable</code>	●	creates a repository	not needed	delete the directory
<code>git refs migrate --ref-format=<format></code>	●	rewrites ref storage of the repository	<code>--dry-run</code>	migrate back
<code>git fast-export --all git fast-import</code>	●	fills the receiving repository; the source is untouched	not needed	delete the new repository
<code>git history reword</code> , <code>fixup</code> , <code>split</code>	●	new commits; every descendant local branch moves	<code>--dry-run</code>	each branch's reflog
<code>git replay --onto</code> , <code>--advance</code> , <code>--revert</code>	●	new commits; refs move in one transaction	<code>--ref-action=print</code>	reflog, where one exists; none by default in a bare repository
<code>git format-patch</code> , <code>git range-diff</code> , <code>git apply --check</code> , <code>--stat</code>	●	writes patch files, or nothing	not needed	delete the files
<code>git am</code>	●	new commits on the current branch	<code>git apply --check</code>	<code>git am --abort</code> while it is stopped; afterwards the reflog of the branch
<code>git apply</code>	●	working files (and the index with <code>--index</code>)	<code>--check</code> , <code>--stat</code>	<code>git apply -R</code>

14D.15 Version notes

Version note. Older behavior: SHA-1 object IDs, refs in files, `master`, bare repositories used wherever found. Current behavior: the same on Git 2.55 and 2.56. Since: the changes are planned for Git 3.0, which has no official date; 2.98 and 2.99 come first. Recommended: `init.defaultBranch=main` and `safe.bareRepository=explicit` now; the formats when your hosting supports them.

Version note. Older behavior: interactive rebase for every rewrite. Current behavior: `git history reword` and `split` (Git 2.54), `fixup` (2.55), `drop` (2.56, not run here), all experimental. Recommended: local cleanup only.

Version note. Older behavior: `git replay` printed ref updates. Current behavior: it updates refs in a transaction, and `--ref-action=print` restores the old output. Since: Git 2.53; the command exists since 2.44, `--revert` since 2.54, `--linearize` since 2.56 (not run here).

Version note. Older behavior: scripts read `.git/config` and counted files. Current behavior: `git repo info`, `git repo structure` and `git last-modified`, all experimental. Since: Git 2.52. Recommended: prefer them to reading `.git`, and check the manual of your version.

Unverified. The calendar months for 2.98, 2.99 and 3.0; a GitHub preview of SHA-256 repositories; and one observation made on Git 2.55.0 that the manual does not state: `git history reword` gives the rewritten commit a new committer date (Lab 42.2).

14D.16 Practice

- **Module 42** (lab manual): 42.1 SHA-256 and reftable repositories, and a script that breaks on both; 42.2 `git history reword`, and a pushed commit rewritten by mistake; 42.3 a format-patch and am round trip, and a patch that no longer applies.
- Replay any transcript with `labs/run ch14d/<demo>`.
- Two drills. Read the "UI, Workflows & Features" section of the newest release notes on your disk and write down, for three bullets, the command you would run to see the change. In the sandbox left by `ch14d/patch-series`, apply version 2 in `upstream` on a fresh branch and use `git range-diff` there to compare it with the applied version 1.

14D.17 Interview questions

1. What will Git 3.0 change, for which repositories, and what is the official statement about its date? Which parts of the schedule come from secondary sources?
2. Name the four configuration keys that choose the 3.0 defaults ahead of time. Which of them would you set on a developer laptop today, and which not?
3. What does `safe.bareRepository=explicit` refuse, what does it still allow, and which attack is it aimed at?
4. A team wants SHA-256 for a new service that is hosted on GitHub. What do you tell them, and how would an existing repository be converted later?
5. Compare `git history reword` with `git rebase -i` and `reword`: working tree, hooks, which branches move, merges, conflicts.
6. Why does a server need `git replay`, and what does `--ref-action=print` give it?
7. How do you find out whether a behavior change in a new Git release affects your team? Which documents do you read, in which order?
8. Explain a patch file line by line. Which parts become the commit, and why do the author and the committer differ after `git am`?
9. What does `git range-diff` compare, and what do `=`, `!`, `<` and `>` mean?
10. Your Git reports `rust: disabled`. What does that tell you, and what does the BreakingChanges document promise to platforms without Rust?

14D.18 Sources

Primary sources

- [BreakingChanges](#), read in the copy installed with Git 2.55.0, and the [release notes directory](#).
- `git-init`, `git-config` (`init.*`, `safe.bareRepository`), `git-refs`, `hash-function-transition`, `git-fast-export`, `git-fast-import`.
- `git-history`, `git-replay`, `git-last-modified`, `git-repo`; for 2.56-only parts, `git-history at 2.56` and `git-replay at 2.56`.
- `git-format-patch`, `git-am`, `git-apply`, `git-range-diff`, `git-patch-id`, `git-send-email`, `SubmittingPatches`, `MyFirstContribution`, `maintain-git`.
- The maintainer's message of 28 September 2026, through a [mailing-list mirror](#), as cited in the Phase 0 report.
- The [GitHub Changelog](#), with the entry on [stacked pull requests](#); the [Homebrew formula for Git](#).

Secondary sources

- [LWN on the road to 3.0](#) and [GitLab on Git 2.56](#): the only sources for calendar months. The Phase 0 report records a conflict between LWN's paraphrase and the primary message.
- The Phase 0 report of this course, section 1, with its table of flags.
- [Git Rev News](#); the repositories and documentation of [Jujutsu](#), [Sapling](#) and [git-branchless](#).

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [Reftable Backend: What it is, where it's headed and why should you care](#), Patrick Steinhardt, 20 minutes, 3 September 2025. Caveat: assumes knowledge of internals.
- [SHA-256 at a Hyperscaler](#), Emily Shaffer, Git Merge 2026, 32 minutes, uploaded 1 October 2026. Caveat from the report: almost no views when checked, so there is no secondary quality signal.
- [You Don't Know Git](#), Edward Thomson, NDC London 2025, 1 hour 2 minutes: rerere, the reflog, worktrees and rebase variants.

Further reading

- Chapter 3 for the formats, Chapter 9 for rebase, `--update-refs` and `git range-diff`, Chapter 10 for patch IDs, Chapter 14C for hooks and their security.